

EML-2 release: the corpus, the judgements, and what was thrown away

What this is

535 generated theorems, each with a complete Lean proof. Every one carries the reasoning behind each judgement it received, including the passes that disagreed, and the result of every mechanical check. The rows that did not make it are summarised here and listed in full in `eml2_rejected.jsonl`, because a corpus that shows only its survivors cannot be checked.

Canonical artifact. The Lean statements and certificates are the source of truth; this document is a review rendering.

Contents

1	What was thrown away	2
2	How a row got here	3
3	How much a verdict is worth	4
4	The corpus	5

1 What was thrown away

1228 certified rows were considered; 535 were admitted. Missing the bar happens three ways, and they are not the same finding: a row both passes reject is a defect the pipeline caught, a row they split on is a fact about the judge, and a row both keep but neither calls strong is simply below the bar this release sets.

Outcome	Rows
admitted — strong from both passes	535
rejected by both passes	276
passes disagreed on keep/reject	249
kept by both passes but not strong in both	102
Lean proved it without a hypothesis it states	41
prose does not describe the elaborated goal	13
held back by hand: a binder that can be substituted away	6
goal round-trip never returned a verdict	4
the exam environment's header opens 'Nat', under which the identifier phi binds to Nat.phi and the statement will not elaborate. Certified, but not playable by the evaluation this release is built for, so it is withdrawn rather than shipped as a row every model fails for the same non-reason.	2

Among the 276 rows both passes rejected, the named failure modes were:

Failure	Rows	What it means
recall	125	solvable by recalling the parent
parallel	61	the parents are proved separately and joined at the end
decoration	45	the change is cosmetic; the parent's proof skeleton still closes it
universal_supplier	12	one parent proves the child on its own, so the other contributes nothing
weakening	7	the child is weaker than its parent
constant_supplier	5	a parent is used only as a constant, not as an argument
repeated_device	4	the same device as a sibling already kept from these parents
not_equivalent	4	
tractability_bound	3	the bound makes the statement true for reasons unrelated to the mathematics
conjunction	2	two unrelated claims stapled together
unlabelled	2	
parallel_crossover	2	the parents are proved separately and joined at the end
arithmetic_on_answers	2	
exists_unique_wrapper	1	an existing result rewrapped as an existence-and-uniqueness claim
wrong_tier	1	

2 How a row got here

Each gate, and what kind of thing settles it. Lean decides the questions that have an answer; a model is asked only the questions that do not.

Check	By	Coverage	What it establishes
statement_type_check	Lean	535/535	The statement elaborates under Mathlib with <code>autoImplicit false</code> , so no free variable is silently invented.
proof_accepted	Lean	535/535	<code>take env</code> lean accepts the proof and it contains no <code>sorry</code> .
axiom_audit	Lean	535/535	<code>#print axioms</code> lists what the proof depends on; anything outside Lean's three standard axioms fails the row.
vacuity	Lean	535/535	The hypotheses are jointly satisfiable, so the theorem is not true merely by having no instances.
dead_hypotheses	Lean	535/535	Each hypothesis whose name the proof never writes is removed and the file recompiled; a proof that still closes proves the hypothesis was dead. Repeated to a fixpoint, because removing one can make another dead.
corpus_dedup	hash	535/535	The statement, alpha-normalised, is absent from every earlier campaign and from this one.
redundancy	Lean	518/518 applicable	Whether one parent already carries the child: for a crossover, that one parent proves it alone; for a mutation, that the parent gives it directly. Finding such a proof is decisive; failing to find one is not, and the inline probe runs without a prover so it reaches only what the tactic ladder can close.
hypothesis_preservation	parser	16/17 applicable	Silent mutations only. The parent's and child's binders are compared after alpha-normalisation. It cannot tell a hypothesis that was re-encoded from one that was dropped, so a finding is read, not obeyed.
comparator	Lean	535/535	An independent runner rebuilds the statement from the row alone, compares it against the submitted proof with <code>lean4export</code> , checks the axiom allowlist, and replays the proof through the Lean kernel. This is what separates <code>proof_checked</code> from <code>kernel_replayed</code> .
goal_roundtrip	model	535/535	Lean's elaborated goal goes to an informalizer that sees no prose, and a separate judge compares its output with the stated problem. Neither model sees the other's input.

3 How much a verdict is worth

The judge was run twice over all 1233 rows — same inputs, same order, no shared state — so that its disagreement with itself could be measured rather than assumed.

Measure	Agreement
keep or reject	984/1233 (79%)
exact quality grade	883/1233 (71%)
failure label, when both rejected	215/275 (78%)

pass 1 ↓ / pass 2 →	strong	acceptable	weak
strong	639	54	93
acceptable	50	22	36
weak	85	32	222

Admission requires the first cell: strong twice. That is why the release is 535 rows and not 1228.

4 The corpus

4.1 001. common_topology_open_iff

The problem

Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X . Show that there is a largest topology contained in every $\mathcal{T}_i$, and that a set $A \subseteq X$ is open in it exactly when every point of A has a neighborhood open in every $\mathcal{T}_i$ and contained in A .

Lean statement

```
theorem common_topology_open_iff (X I : Type*) (T : I → TopologicalSpace X) (A : Set X) :
  ∃ τ : TopologicalSpace X,
    (∀ U : Set X, @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U) ∧
    (∀ σ : TopologicalSpace X,
      (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
        ∀ U, @IsOpen X σ U → @IsOpen X τ U) ∧
    (@IsOpen X τ A ↔
      ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
        (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A)
```

Parent 1. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent 1 in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

What it contributes. Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

Parent 2. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent 2 in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

What it contributes. Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: Parent 2 constructs the topology of sets open in every member of the family and proves its maximality, while Parent 1 supplies the local-neighborhood characterization inside that constructed topology. The local iff is not an independent conjunction or parallel use of the parent results.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the parents: it constructs the topology of sets open in every member topology, then uses the local-neighborhood criterion to characterize when A is open in that constructed topology. Neither the common-topology construction nor the local openness argument alone yields the stated iff.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child constructs the topology of sets open in every parent topology, using Parent 2’s common-lower-topology construction, and then transfers Parent 1’s local-neighborhood criterion into that constructed topology. The local witness condition cannot be established independently of the topology supplied by the family bridge.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2339 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Clause (2) fails to specify the relevant topologies for its ‘open’ predicates, so it does not clearly assert that every topology contained in all T_i is contained in τ ; as written, it is tautological or ill-formed rather than the required maximality statement.

Goal round-trip: what Lean elaborated

```
theorem common_topology_open_iff.extracted_1_1.{u_2, u_1} (X : Type u_1) (I : Type u_2) (T : I → TopologicalSpace X)
(A : Set X) :
  ∃ τ,
    (∀ (U : Set X), IsOpen U ↔ ∀ (i : I), IsOpen U) ∧
    (∀ (σ : TopologicalSpace X), (∀ (i : I) (U : Set X), IsOpen U → IsOpen U) → ∀ (U : Set X), IsOpen U → IsOpen U) ∧
    (IsOpen A ↔ ∀ x ∈ A, ∃ U, x ∈ U ∧ (∀ (i : I), IsOpen U) ∧ U ⊆ A)
```

Read back by a model that saw only the Lean. For types X and I , a family of topological spaces T indexed by I on X , and a set $A \subseteq X$, there exists a topology τ on X such that: (1) for every subset U of X , U is open if and only if U is open for every index $i \in I$; (2) for every topology σ on X , if for every $i \in I$ and every subset U of X , U being open implies U being open, then for every subset U of X , U being open implies U being open; and (3) A is open if and only if every point $x \in A$ has a subset U of X such that $x \in U$, U is open for every $i \in I$, and $U \subseteq A$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem common_topology_open_iff (X I : Type*) (T : I → TopologicalSpace X) (A : Set X) :
  ∃ τ : TopologicalSpace X,
    (∀ U : Set X, @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U) ∧
    (∀ σ : TopologicalSpace X,
      (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
      ∀ U, @IsOpen X σ U → @IsOpen X τ U) ∧
    (@IsOpen X τ A ↔
      ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
        (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A) := by
  let τ : TopologicalSpace X :=
  { IsOpen := fun U => ∀ i, @IsOpen X (T i) U
    isOpen_univ := by
      intro i
      exact (T i).isOpen_univ
    isOpen_inter := by
      intro s t hs ht i
      exact (T i).isOpen_inter s t (hs i) (ht i)
    isOpen_sUnion := by
      intro S hS i
      exact (T i).isOpen_sUnion S (by
```

```

      intro s hs
      exact hS s hs i) }
have h_common : ∀ U : Set X,
  @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U := by
  intro U
  rfl
have h_max : ∀ σ : TopologicalSpace X,
  (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
  ∀ U, @IsOpen X σ U → @IsOpen X τ U := by
  intro σ hσ U hU
  exact (h_common U).2 (fun i => hσ i U hU)
have h_local :
  (∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
    (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A) →
  @IsOpen X τ A := by
  intro hA
  letI : TopologicalSpace X := τ
  change IsOpen A
  refine isOpen_iff_forall_mem_open.mpr ?_
  intro x hx
  rcases hA x hx with ⟨U, hxU, hU, hUA⟩
  refine ⟨U, hUA, ?_, hxU⟩
  exact (h_common U).2 hU
have h_iff :
  @IsOpen X τ A ↔
  ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
    (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A := by
  constructor
  · intro hA x hx
    refine ⟨A, hx, ?_, Set.Subset.rfl⟩
    exact (h_common A).1 hA
  · exact h_local
exact ⟨τ, h_common, h_max, h_iff⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_13_4b2__xe__62f50f2f

Fusion mechanism. goal_form_transplant Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 62f50f2fa78706b1

4.2 002. isOpenMap_fst_restricted_bounded_union

The problem

Let X and Y be topological spaces, let $n \in \mathbb{N}$, and let $(A_i)_{i=1}^n$ be a finite family of subsets of $X \times Y$ such that every point of each A_i has an open neighborhood contained in A_i . Prove that the first-coordinate projection, restricted to $\bigcup_{i=1}^n A_i$, is an open map.

Lean statement

```
theorem isOpenMap_fst_restricted_bounded_union (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ i x, x ∈ A i → ∃ U : Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A i) : IsOpenMap (fun p : (U i : Fin n, A i) => p.1.1)
```

Parent 1. A bounded indexed family of locally open sets has an open union.

Parent 1 in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  IsOpen (U i : Fin n, A i)
```

What it contributes. The local-openness checkpoint produces an open restricted domain; the product-coordinate open-map checkpoint is then applied to open subsets of that domain to establish one restricted-map theorem.

Parent 2. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent 2 in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. The local-openness checkpoint produces an open restricted domain; the product-coordinate open-map checkpoint is then applied to open subsets of that domain to establish one restricted-map theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

Parent 1 supplies openness of the union, which is used to make the subtype inclusion an open map; Parent 2's projection result then composes with that inclusion to prove the restricted first-coordinate map is open. This is a genuine sequential pipeline, not a conjunction or arithmetic combination.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1 supplies openness of the union, which is transformed into openness of the subtype inclusion; Parent 2 supplies openness of the first projection, and the two are genuinely composed to obtain the restricted map.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: Parent 1 proves the union is open, which makes the subtype inclusion an open map; Parent 2 then supplies openness of the first projection, and composition yields openness of the restricted first-coordinate map.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1957 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpenMap_fst_restricted_bounded_union.extracted_1_1.{u_2, u_1} (X : Type u_1) (Y : Type u_2)
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ (i : Fin n), ∀ x ∈ A i, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) : IsOpenMap fun p => (↑p).1
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let n be a natural number, and let A assign to each $i \in \text{Fin } n$ a subset $A(i)$ of $X \times Y$. Assume that for every $i \in \text{Fin } n$ and every $x \in A(i)$, there exists an open set U in $X \times Y$ such that $x \in U$ and $U \subseteq A(i)$. Then the map from the subtype of $\bigcup i : \text{Fin } n, A(i)$ to X that sends each point p to the first coordinate of its underlying point in $X \times Y$ is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem isOpenMap_fst_restricted_bounded_union (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ i x, x ∈ A i → ∃ U : Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A i) : IsOpenMap (fun p : (U i : Fin n, A i) => p.1.1) := by
  have hUnion : IsOpen (U i : Fin n, A i) := by
    refine isOpen_iff_forall_mem_open.mpr ?_
    intro x hx
    have hx_index : ∃ i : Fin n, x ∈ A i := by
      simpa only [Set.mem_iUnion] using hx
    rcases hx_index with ⟨i, hxi⟩
    rcases h i x hxi with ⟨U, hxU, hUopen, hUsub⟩
    have hUunion : U ⊆ U i : Fin n, A i := by
      intro y hy
      exact Set.mem_iUnion.mpr ⟨i, hUsub hy⟩
    exact ⟨U, hUunion, hUopen, hxU⟩
  have hSubtype : IsOpenMap ((↑) : (U i : Fin n, A i) → X × Y) :=
    hUnion.isOpenMap_subtype_val
  have hRestricted : IsOpenMap (Prod.fst ∘ ((↑) : (U i : Fin n, A i) → X × Y)) :=
    isOpenMap_fst.comp hSubtype
  simpa only [Function.comp_apply] using hRestricted
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_16_4__xe__8a753487

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 82965c7c7c358aa6

4.3 003. first_projection_image_isOpen

The problem

Let X and Y be topological spaces, and let A be a subset of $X \times Y$. Suppose that for each $(x, y) \in A$ there is an open set U containing (x, y) such that $U \subset A$. Show that the set of first coordinates of points in A is open in X .

Lean statement

```
theorem first_projection_image_isOpen {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U :  
  ↪ Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A) : IsOpen (Prod.fst '' A)
```

Parent 1. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent 1 in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)  
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :  
  IsOpen A := by
```

What it contributes. The locally open subset becomes an open domain object; openness of Prod.fst transports that object to an open projected image.

Parent 2. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent 2 in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]  
  (π₁ : X × Y → X)  
  (π₂ : X × Y → Y)  
  (h₁ : π₁ = Prod.fst)  
  (h₂ : π₂ = Prod.snd) :  
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. The locally open subset becomes an open domain object; openness of Prod.fst transports that object to an open projected image.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: the local-neighborhood condition first yields $\text{IsOpen } A$, and the first-projection open-map result then transforms that into $\text{IsOpen } (\text{Prod.fst } '' A)$. Neither parent alone supplies the complete argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 turns the local-neighborhood hypothesis into $\text{IsOpen } A$, and Parent 2's projection open-map result then consumes that fact to establish $\text{IsOpen } (\text{Prod.fst } '' A)$. Neither parent alone supplies the complete argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 yields $\text{IsOpen } A$, which is then consumed as the openness hypothesis by Parent 2's open-map result for Prod.fst to prove the image is open. Neither parent alone establishes the child's conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1943 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem first_projection_image_isOpen.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
IsOpen (Prod.fst '' A)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, and let A be a subset of $X \times Y$. Suppose that for every point x in A , there exists a subset U of $X \times Y$ such that x belongs to U , U is open, and U is contained in A . Then the image of A under the first projection map from $X \times Y$ to X is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem first_projection_image_isOpen {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U :
  Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A) : IsOpen (Prod.fst '' A) := by
  have hA : IsOpen A := by
    refine isOpen_iff_forall_mem_open.mpr ?_
    intro x hx
    rcases hlocal x hx with ⟨U, hxU, hU, hUA⟩
    exact ⟨U, hUA, hU, hxU⟩
  have hfst : IsOpenMap (Prod.fst : X × Y → X) := isOpenMap_fst
  have himage : IsOpen ((Prod.fst : X × Y → X) '' A) := hfst A hA
  exact himage
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_16_4__xe__b107534f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 37c8df224f940d99

4.4 004. fiberwise_fixed_step_gap

The problem

Let Y be a nonempty topological space and let $F : \mathbb{R} \times Y \rightarrow \mathbb{R}$ be continuous and constant on the fibers of the first projection. If the descended response has derivative tending to 0 at infinity, prove that it admits a continuous descent $f : \mathbb{R} \rightarrow \mathbb{R}$ and that $|F(x+1, y) - F(x, y)| \leq 1$ for all sufficiently large x , uniformly in y .

Lean statement

```
theorem fiberwise_fixed_step_gap {Y : Type*} [TopologicalSpace Y] [Nonempty Y]
  (F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F)
  (hfiber : ∀ p q : ℝ × Y, p.1 = q.1 → F p = F q)
  (hquot : Topology.IsQuotientMap (Prod.fst : ℝ × Y → ℝ))
  (hslice_limit : Filter.Tendsto
    (fun x : ℝ => deriv (fun t : ℝ => F (t, y₀)) x)
    Filter.atTop (nhds 0))
  (hslice_deriv : ∀ x : ℝ,
    HasDerivAt (fun t : ℝ => F (t, y₀))
      (deriv (fun t : ℝ => F (t, y₀)) x) x)
  (hanalytic : ∀ g : ℝ → ℝ, Continuous g →
    (∀ x : ℝ, HasDerivAt g (deriv g x) x) →
    Filter.Tendsto (fun x : ℝ => deriv g x) Filter.atTop (nhds 0) →
    ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |g (x + 1) - g x| ≤ 1) :
  ∃ f : ℝ → ℝ, Continuous f ∧
    (∀ x : ℝ, ∀ y : Y, F (x, y) = f x) ∧
    (∃ C : ℝ, ∀ x : ℝ, C ≤ x → ∀ y : Y,
      |F (x + 1, y) - F (x, y)| ≤ 1)
```

Parent 1. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent 1 in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

What it contributes. Use quotientness of `Prod.fst` to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

Parent 2. Suppose a response curve is differentiable for every positive input and its derivative tends to zero as the input grows without bound. Show that for every tolerance $\varepsilon > 0$, the change in response between inputs one unit apart is eventually smaller than ε .

Parent 2 in Lean

```
theorem Rudin_exercise_5_5_eventual
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (nhds 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x : ℝ, N ≤ x → |f (x + 1) - f x| < ε
```

What it contributes. Use quotientness of `Prod.fst` to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The proof genuinely fuses the parents: quotientness of the first projection transfers continuity from the fiber-constant F to the descended function f , after which the analytic derivative-limit hypothesis is applied to f and transported back uniformly along fibers. Neither parent's conclusion alone supplies both the descended continuous response and the fiberwise fixed-step estimate.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines both parents: quotientness of the first projection is used to construct and prove continuity of the descended function f , and the analytic hypothesis is then applied to that newly constructed f before its estimate is transferred uniformly across fibers.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quotient-map hypothesis is used to descend the fiber-constant function to a continuous f , and the analytic hypothesis is then applied specifically to that constructed f ; its estimate is transferred back uniformly along fibers. The parents' roles interact through this witness exchange rather than being combined in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2822 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fiberwise_fixed_step_gap.extracted_1.1.{u_1} {Y : Type u_1} [inst : TopologicalSpace Y] [inst_1 : Nonempty Y]
(F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F) (hfiber : ∀ (p q : ℝ × Y), p.1 = q.1 → F p = F q)
(hquot : Topology.IsQuotientMap Prod.fst)
(hslice_limit : Filter.Tendsto (fun x => deriv (fun t => F (t, y₀)) x) Filter.atTop (nhds 0))
(hslice_deriv : ∀ (x : ℝ), HasDerivAt (fun t => F (t, y₀)) (deriv (fun t => F (t, y₀)) x) x) :
∃ f,
  Continuous f ∧
  (∀ (x : ℝ) (y : Y), F (x, y) = f x) ∧ ∃ C, ∀ (x : ℝ), C ≤ x → ∀ (y : Y), |F (x + 1, y) - F (x, y)| ≤ 1
```

Read back by a model that saw only the Lean. Let Y be a topological space that is nonempty. Let $F : \mathbb{R} \times Y \rightarrow \mathbb{R}$ and $y_0 \in Y$. Assume that F is continuous; whenever $p, q \in \mathbb{R} \times Y$ have the same first coordinate, $F(p) = F(q)$; the projection $\text{Prod.fst} : \mathbb{R} \times Y \rightarrow \mathbb{R}$ is a quotient map; the function $x \mapsto \text{deriv } (t \mapsto F(t, y_0)) x$ tends to 0 as x tends to $+\infty$; and for every $x \in \mathbb{R}$, the function $t \mapsto F(t, y_0)$ has derivative $\text{deriv } (t \mapsto F(t, y_0)) x$ at x . Then there exists a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$ such that $F(x, y) = f(x)$ for every $x \in \mathbb{R}$ and $y \in Y$, and there exists a real constant C such that for every $x \in \mathbb{R}$ with $C \leq x$ and every $y \in Y$, $|F(x+1, y) - F(x, y)| \leq 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem fiberwise_fixed_step_gap {Y : Type*} [TopologicalSpace Y] [Nonempty Y]
(F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F)
(hfiber : ∀ p q : ℝ × Y, p.1 = q.1 → F p = F q)
(hquot : Topology.IsQuotientMap (Prod.fst : ℝ × Y → ℝ))
```

```

(hslice_limit : Filter.Tendsto
  (fun x : ℝ => deriv (fun t : ℝ => F (t, y₀)) x)
  Filter.atTop (nhds 0))
(hslice_deriv : ∀ x : ℝ,
  HasDerivAt (fun t : ℝ => F (t, y₀))
    (deriv (fun t : ℝ => F (t, y₀)) x) x)
(hanalytic : ∀ g : ℝ → ℝ, Continuous g →
  (∀ x : ℝ, HasDerivAt g (deriv g x) x) →
  Filter.Tendsto (fun x : ℝ => deriv g x) Filter.atTop (nhds 0) →
  ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |g (x + 1) - g x| ≤ 1) :
∃ f : ℝ → ℝ, Continuous f ∧
  (∀ x : ℝ, ∀ y : ℝ, F (x, y) = f x) ∧
  (∃ C : ℝ, ∀ x : ℝ, C ≤ x → ∀ y : ℝ,
    |F (x + 1, y) - F (x, y)| ≤ 1) := by
let f : ℝ → ℝ := fun x => F (x, y₀)
have hfactor : F = f ∘ (Prod.fst : ℝ × ℝ → ℝ) := by
  funext p
  exact hfiber p (p.1, y₀) rfl
have hcomp : Continuous (f ∘ (Prod.fst : ℝ × ℝ → ℝ)) := by
  rw [← hfactor]
  exact hF
have hfcont : Continuous f := by
  exact hquot.continuous_iff.mpr hcomp
have hderiv : ∀ x : ℝ, HasDerivAt f (deriv f x) x := by
  intro x
  simpa [f] using hslice_deriv x
have hlim : Filter.Tendsto (fun x : ℝ => deriv f x) Filter.atTop (nhds 0) := by
  simpa [f] using hslice_limit
have hgap : ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |f (x + 1) - f x| ≤ 1 :=
  hanalytic f hfcont hderiv hlim
refine ⟨f, hfcont, ?_, ?_⟩
· intro x y
  exact hfiber (x, y) (x, y₀) rfl
· obtain ⟨C, hC⟩ := hgap
  refine ⟨C, ?_⟩
  intro x hx y
  rw [hfiber (x + 1, y) (x + 1, y₀) rfl,
    hfiber (x, y) (x, y₀) rfl]
  simpa [f] using hC x hx

```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__a3e56ed4__x__Rudin_exercise_5_5__ms.xe__85f96354

Fusion mechanism. witness_exchange Use quotientness of Prod.fst to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 85f96354a354a2bc

4.5 005. boolean_ring_product_order_closed

The problem

Let R be a Boolean ring with the order topology, and let s be a closed subset of a topological space X . If $f, g : X \rightarrow R$ are continuous in the sense that $x \mapsto f(x)g(x)$ is continuous, show that $\{x \in s \mid f(x)g(x) \leq g(x)f(x)\}$ is closed.

Lean statement

```
theorem boolean_ring_product_order_closed {X R : Type*} [TopologicalSpace X] [TopologicalSpace R]
  [Ring R] [LinearOrder R] [OrderTopology R] {s : Set X} {f g : X → R}
  (hR : ∀ a : R, a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous (fun x => f x * g x)) :
  IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x}
```

Parent 1. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent 1 in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Parent 2. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent 2 in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a*b = b*a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The Boolean-ring parent supplies commutativity, which is used to rewrite the swapped product and derive its continuity from the given continuity of f, g ; this is then consumed by the closed-order argument. The interaction is genuine, though the resulting inequality is trivial once commutativity is established.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The Boolean-ring hypothesis is used to derive commutativity, which transfers continuity of f, g to continuity of f, g and enables the closed-order-set argument. The topological closure construction from the first parent is therefore genuinely dependent on the algebraic conclusion of the second.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: idempotence yields commutativity of products, which converts continuity of f, g into continuity of f, g ; those two continuities are then consumed by the closed-restricted-inequality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2822 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem boolean_ring_product_order_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {R : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace R] [inst_2 : Ring R] [inst_3 : LinearOrder R]
  [inst_4 : OrderTopology R] {s : Set X} {f g : X → R} (hR : ∀ (a : R), a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous fun x => f x * g x) : IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x}
```

Read back by a model that saw only the Lean. *Let X and R be topological spaces, with R also equipped with a ring structure, a linear order, and the order topology. Suppose every element a of R is idempotent, meaning $a^2 = a$. If s is a closed subset of X and the function $x \mapsto f(x) \cdot g(x)$ is continuous, then the set of all $x \in X$ such that $x \in s$ and $f(x) \cdot g(x) \leq g(x) \cdot f(x)$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem boolean_ring_product_order_closed {X R : Type*} [TopologicalSpace X] [TopologicalSpace R]
  [Ring R] [LinearOrder R] [OrderTopology R] {s : Set X} {f g : X → R}
  (hR : ∀ a : R, a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous (fun x => f x * g x)) :
  IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x} := by
  have hneg : ∀ a : R, -a = a := by
    intro a
    have h1 : (-a) ^ 2 = -a := hR (-a)
    calc
      -a = (-a) ^ 2 := h1.symm
      _ = a ^ 2 := by noncomm_ring
      _ = a := hR a
  have htwo : ∀ a : R, a + a = 0 := by
    intro a
    have h := congrArg (fun z : R => z + a) (hneg a)
    have hz : (0 : R) = a + a := by simpa using h
    exact hz.symm
  have hcomm : ∀ a b : R, a * b = b * a := by
    intro a b
    have hmix : a * b + b * a = 0 := by
      calc
        a * b + b * a = (a + b) ^ 2 - a ^ 2 - b ^ 2 := by noncomm_ring
        _ = (a + b) - a - b := by rw [hR (a + b), hR a, hR b]
        _ = 0 := by noncomm_ring
    calc
      a * b = a * b + 0 := by simp
      _ = a * b + (a * b + b * a) := by rw [hmix, add_zero]
      _ = (a * b + a * b) + b * a := by abel
      _ = b * a := by rw [htwo (a * b)]; simp
  have hgf : Continuous (fun x => g x * f x) := by
    have heq : (fun x => g x * f x) = (fun x => f x * g x) := by
      funext x
      exact hcomm (f x) (g x) |>.symm
    rw [heq]
    exact hfg
  have hineq : IsClosed {x | f x * g x ≤ g x * f x} := by
    exact isClosed_le hfg hgf
  have hmaster : IsClosed (s ∩ {x | f x * g x ≤ g x * f x}) := by
    exact hs.inter hineq
  have hset : {x | x ∈ s ∧ f x * g x ≤ g x * f x} =
    s ∩ {x | f x * g x ≤ g x * f x} := by
```

```
ext x
rfl
rw [hset]
exact hmaster
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__6cceb54a__x__Dummit_Foote_exercise_7_1_15__xe__0501cfb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0501cfb772c3eb68

4.6 006. finite_closed_connected_union_perfect_decomposition

The problem

Let $A_0, \dots, A_{n-1}$ be closed connected subsets of a separable metric space, with $n > 0$, and suppose each A_k for $k > 0$ meets the union of its predecessors. If their union contains two distinct points, show that it is the union of a countable set and a nonempty perfect set.

Lean statement

```
theorem finite_closed_connected_union_perfect_decomposition {X : Type*} [MetricSpace X] [SecondCountableTopology X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsClosed (A k)) →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  (∃ x y : X, x ∈ ⋃ k ∈ Finset.range n, A k ∧ y ∈ ⋃ k ∈ Finset.range n, A k ∧ x ≠ y) →
  ∃ V D : Set X, V.Countable ∧ Perfect D ∧ D.Nonempty ∧
    (⋃ k ∈ Finset.range n, A k) = V ∪ D
```

Parent 1. Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Parent 1 in Lean

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

Parent 2. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 2 in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines Parent 1's connectedness result into a noncountability argument using the two-point hypothesis, then uses Parent 2's closed-set decomposition to force the perfect component to be nonempty. Neither parent alone supplies the resulting nonempty perfect-kernel conclusion.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses the connected-union result together with two distinct points to prove the union is uncountable, then consumes that fact to show the perfect component supplied by the decomposition is nonempty. This is substantive interaction, not a conjunction or final parallel combination.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses Parent 1's connected-union result to prove the union is uncountable, then consumes Parent 2's Cantor-Bendixson decomposition to force its perfect component to be nonempty. This is substantive interaction, not a conjunction or final parallel combination.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2246 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_closed_connected_union_perfect_decomposition.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X]
[inst_1 : SecondCountableTopology X] (A : ℕ → Set X) (n : ℕ) :
0 < n →
(∀ k < n, IsClosed (A k)) →
(∀ k < n, IsConnected (A k)) →
(∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
(∃ x y, x ∈ ⋃ k ∈ Finset.range n, A k ∧ y ∈ ⋃ k ∈ Finset.range n, A k ∧ x ≠ y) →
∃ V D, V.Countable ∧ Perfect D ∧ D.Nonempty ∧ ⋃ k ∈ Finset.range n, A k = V ∪ D
```

Read back by a model that saw only the Lean. *Let X be a metric space with a second-countable topology, let A be a sequence of subsets of X , and let n be a natural number. If $n > 0$; every A_k with $k < n$ is closed; every A_k with $k < n$ is connected; for every natural number k with $0 < k < n$, the set A_k intersects the union of A_j over all $j < k$; and the union of A_k over $k < n$ contains two distinct points, then there exist subsets V and D of X such that V is countable, D is perfect, D is nonempty, and the union of A_k over $k < n$ is equal to $V \cup D$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
(A : ℕ → Set X) (n : ℕ) :
0 < n →
(∀ k < n, IsConnected (A k)) →
(∀ k, 0 < k → k < n →
(A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
IsConnected (⋃ k ∈ Finset.range n, A k) := by
induction n with
| zero =>
  intro hn
  omega
```

```

| succ n ih =>
  intro hn hconn hattach
  by_cases hzero : n = 0
  · subst n
    simpa [Finset.mem_range] using hconn 0 (by omega)
  · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
    have hprev : IsConnected (U k ∈ Finset.range n, A k) := by
      apply ih
      · exact hnpos
      · intro k hk
        exact hconn k (by omega)
      · intro k hkpos hklit
        exact hattach k hkpos (by omega)
    have hlast : IsConnected (A n) := hconn n (by omega)
    have hinter : ((U k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
      simpa [Set.inter_comm] using hattach n (by omega) (by omega)
    have hunion : IsConnected ((U k ∈ Finset.range n, A k) ∪ A n) :=
      IsConnected.union hinter hprev hlast
    have heq : (U k ∈ Finset.range (Nat.succ n), A k) =
      (U k ∈ Finset.range n, A k) ∪ A n := by
      simp only [Finset.mem_range]
      exact Set.biUnion_lt_succ A n
    rw [heq]
    exact hunion

theorem finite_closed_connected_union_perfect_decomposition {X : Type*} [MetricSpace X] [SecondCountableTopology X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsClosed (A k)) →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ U j ∈ Finset.range k, A j).Nonempty) →
  (∃ x y : X, x ∈ U k ∈ Finset.range n, A k ∧ y ∈ U k ∈ Finset.range n, A k ∧ x ≠ y) →
  ∃ V D : Set X, V.Countable ∧ Perfect D ∧ D.Nonempty ∧
    (U k ∈ Finset.range n, A k) = V ∪ D := by
  classical
  intro hn hclosed hconn hattach htwo
  let C : Set X := U k ∈ Finset.range n, A k
  have hCclosed : IsClosed C := by
    dsimp [C]
    exact isClosed_biUnion_finset (fun k hk => hclosed k (Finset.mem_range.mp hk))
  have hCconn : IsConnected C := by
    exact finite_connected_union_predecessor A n hn hconn hattach
  have hCuncountable : ¬ C.Countable := by
    rcases htwo with (x, y, hx, hy, hxy)
    intro hcount
    have hsub : C.Subsingleton :=
      hcount.isTotallyDisconnected C (by intro z hz; exact hz) hCconn.isPreconnected
    exact hxy (hsub hx hy)
  obtain ⟨V, D, hV, hD, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed hCclosed
  have hDne : D.Nonempty := by
    by_contra hne
    have hDempty : D = ∅ := Set.not_nonempty_iff_eq_empty.mp hne
    apply hCuncountable
    rw [hdecomp, hDempty, union_empty]
    exact hV
  exact ⟨V, D, hV, hD, hDne, hdecomp⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__2746d5bb__x__Rudin_exercise_2_28__xe__e607282a

Fusion mechanism. invariant_transplant The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e607282a24cc0585

4.7 007. connected_image_of_overlapping_chain

The problem

Let (A_n) be connected subsets of a uniform space such that $A_n \cap A_{n+1} \neq \emptyset$ for every n . If f and g are uniformly continuous, show that $(g \circ f)(\bigcup_n A_n)$ is connected.

Lean statement

```
theorem connected_image_of_overlapping_chain
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected ((g ∘ f) '' (⋃ n, A n))
```

Parent 1. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent 1 in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

Parent 2. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent 2 in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: Parent 1 establishes connectedness of the indexed union, while Parent 2 supplies uniform continuity of the composite map, which is then used to prove connectedness of the image. Neither parent alone yields the conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses both parents: Parent 1 establishes connectedness of the overlapping union, while Parent 2 supplies uniform continuity of the composite, which is then transferred to continuous-image connectedness. The composite map is essential to the conclusion, so this is not parallel combination or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 establishes connectedness of the overlapping union, while Parent 2 supplies uniform continuity of the composite, which is then used to prove connectedness of its image. Neither parent alone yields the conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2143 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_image_of_overlapping_chain.extracted_1.1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {A : ℕ → Set α}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) : IsConnected (g ∘ f '' U n, A n)
```

Read back by a model that saw only the Lean. Let α , β , and γ be types equipped with uniform space structures. Let A assign to each natural number n a subset $A(n)$ of α , and let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ be uniformly continuous functions. If, for every natural number n , $A(n)$ is connected, and $A(n) \cap A(n + 1)$ is nonempty, then the image of $\bigcup n, A(n)$ under the composite function $g \circ f$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem connected_image_of_overlapping_chain
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected ((g ∘ f) '' (U n, A n)) := by
  classical
  have hAdj : ∀ n, (A n ∩ A (n + 1)).Nonempty := by
    intro n
    exact (Set.nonempty_iff_ne_empty).2 (hAn n)
  set R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hEdgeSucc : ∀ n, R n (n + 1) := by
    intro n
    simpa [R, Nat.succ_eq_add_one] using hAdj n
  have hEdgePred : ∀ n, R (n + 1) n := by
    intro n
    simpa [R, Nat.succ_eq_add_one, Set.inter_comm] using hAdj n
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    refine reflTransGen_of_succ R ?_ ?_
    · intro k hk
      exact hEdgeSucc k
    · intro k hk
      exact hEdgePred k
  have hUnion : IsConnected (U n, A n) := by
    simpa [R] using
```

```

    IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
have hComposite : UniformContinuous (g ∘ f) := by
  exact hg.comp hf
have hContinuous : ContinuousOn (g ∘ f) (U n, A n) := by
  exact hComposite.continuous.continuousOn
exact hUnion.image (g ∘ f) hContinuous

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__x__Rudin_exercise_4_12__xe__5c2bbc5c

Fusion mechanism. invariant_transplant Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5c2bbc5c5edd268b

4.8 008. fixed_point_locus_has_least

The problem

Let f be a continuous self-map of the unit interval $[0, 1]$. Prove that its fixed-point set has a least element.

Lean statement

```
theorem fixed_point_locus_has_least
  (f : Set.Icc (0 : ℝ) 1 → Set.Icc (0 : ℝ) 1)
  (hf : Continuous f) :
  ∃ x : Set.Icc (0 : ℝ) 1,
    f x = x ∧ ∀ y : Set.Icc (0 : ℝ) 1, f y = y → x ≤ y := by
```

Parent 1. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent 1 in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. View $\text{Fix}(f)$ as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

Parent 2. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent 2 in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. View $\text{Fix}(f)$ as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely combines Parent 1's fixed-point existence with Parent 2's closedness of the zero locus: together with interval compactness, these establish a nonempty compact fixed-point set and hence its least element. Neither parent's conclusion alone supplies the least-fixed-point result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely needs both parents: the fixed-point witness makes the locus nonempty, while the zero-preimage closedness argument makes it compact; only their combination yields a least fixed point.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses both parents to construct the fixed-point locus as a nonempty closed subset of the compact interval, then derives its least element. The existence and closedness results are jointly consumed by the compactness/order argument, not merely conjoined at the end.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2314 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fixed_point_locus_has_least.extracted_1_1 (f : ↑(Set.Icc 0 1) → ↑(Set.Icc 0 1)) (hf : Continuous f) :
  ∃ x, f x = x ∧ ∀ (y : ↑(Set.Icc 0 1)), f y = y → x ≤ y
```

Read back by a model that saw only the Lean. *Let f be a continuous function from the closed interval $[0,1]$ to itself. Then there exists a point x in $[0,1]$ such that $f(x)=x$ and, for every y in $[0,1]$, if $f(y)=y$, then $x \leq y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem fixed_point_locus_has_least
  (f : Set.Icc (0 : ℝ) 1 → Set.Icc (0 : ℝ) 1)
  (hf : Continuous f) :
  ∃ x : Set.Icc (0 : ℝ) 1,
    f x = x ∧ ∀ y : Set.Icc (0 : ℝ) 1, f y = y → x ≤ y := by
  classical
  let a : Set.Icc (0 : ℝ) 1 := ⟨0, ⟨le_rfl, zero_le_one⟩⟩
  let b : Set.Icc (0 : ℝ) 1 := ⟨1, ⟨zero_le_one, le_rfl⟩⟩
  let g : Set.Icc (0 : ℝ) 1 → ℝ := fun x => (f x : ℝ) - (x : ℝ)
  have hg : Continuous g := by
    have h₁ : Continuous (fun x : Set.Icc (0 : ℝ) 1 => (f x : ℝ)) :=
      continuous_subtype_val.comp hf
    exact h₁.sub continuous_subtype_val
  have hga : 0 ≤ g a := by
    have hmem : (f a : ℝ) ∈ Set.Icc (0 : ℝ) 1 := (f a).property
    have hlow : 0 ≤ (f a : ℝ) := (Set.mem_Icc.mp hmem).1
    simpa [g, a] using hlow
  have hgb : g b ≤ 0 := by
    have hmem : (f b : ℝ) ∈ Set.Icc (0 : ℝ) 1 := (f b).property
    have hupp : (f b : ℝ) ≤ 1 := (Set.mem_Icc.mp hmem).2
    have h : (f b : ℝ) - 1 ≤ 0 := sub_nonpos.mpr hupp
    simpa [g, b] using h
  have hzero : (0 : ℝ) ∈ Set.range g :=
    mem_range_of_exists_le_of_exists_ge hg ⟨b, hgb⟩ ⟨a, hga⟩
  have hfixed : ∃ x : Set.Icc (0 : ℝ) 1, f x = x := by
    rcases hzero with ⟨x, hx⟩
    have hreal : (f x : ℝ) = (x : ℝ) := by
      apply sub_eq_zero.mp
      simpa [g] using hx
    exact ⟨x, Subtype.ext hreal⟩
  let A : Set (Set.Icc (0 : ℝ) 1) := {x | f x = x}
  have hzero_closed : IsClosed (g ⁻¹' ({0} : Set ℝ)) :=
    isClosed_singleton.preimage hg
  have hA_eq : A = g ⁻¹' ({0} : Set ℝ) := by
    ext x
    constructor
    · intro hx
      have hreal : (f x : ℝ) = (x : ℝ) := by
        exact congrArg Subtype.val hx
      simpa [g, sub_eq_zero] using hreal
    · intro hx
      have hreal : (f x : ℝ) = (x : ℝ) := by
```

```

    apply sub_eq_zero.mp
    simp [g] using hx
    simp [A] using Subtype.ext hreal
have hA_closed : IsClosed A := by
  rw [hA_eq]
  exact hzero_closed
have hA_nonempty : A.Nonempty := by
  rcases hfixed with ⟨x, hx⟩
  exact ⟨x, by simp [A] using hx⟩
have hI_compact : IsCompact (Set.univ : Set (Set.Icc (0 : ℝ) 1)) := by
  exact (isCompact_iff_isCompact_univ (s := Set.Icc (0 : ℝ) 1)).mp isCompact_Icc
have hA_compact : IsCompact A :=
  hI_compact.of_isClosed_subset hA_closed (Set.subset_univ A)
have hleast : ∃ x, IsLeast A x := hA_compact.exists_isLeast hA_nonempty
rcases hleast with ⟨x, hx⟩
exact ⟨x, by simp [A] using hx.1, fun y hy => hx.2 (by simp [A] using hy)⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__x__Rudin_exercise_4_3__xe__fda27007

Fusion mechanism. goal_form_transplant View Fix(f) as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fda27007375c78cb

4.9 009. positive_fixed_point_normalized_stability

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous on $[0, 1]$, map $[0, 1]$ into itself, satisfy $f(x) > 0$ for every $x \in [0, 1]$, and have $x \mapsto f(x)/x$ monotone on $(0, \infty)$. Show that there exists $c \in [0, 1]$ such that $c > 0$, $f(c)/c = 1$, and $f(y) \leq y$ for every $y \in (0, c]$.

Lean statement

```
theorem positive_fixed_point_normalized_stability
  {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Icc (0 : ℝ) 1))
  (hmaps : Set.MapsTo f (Set.Icc (0 : ℝ) 1) (Set.Icc (0 : ℝ) 1))
  (hpositive : ∀ x ∈ Set.Icc (0 : ℝ) 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi (0 : ℝ))) :
  ∃ c ∈ Set.Icc (0 : ℝ) 1, 0 < c ∧ f c / c = 1 ∧
  ∀ y ∈ Set.Ioc (0 : ℝ) c, f y ≤ y
```

Parent 1. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent 1 in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Supplies the fixed-point witness c in the closed interval via continuity and self-mapping.

Parent 2. Suppose (a) f is continuous for $x \geq 0$, (b) $f'(x)$ exists for $x > 0$, (c) $f(0) = 0$, (d) f' is monotonically increasing. Put $g(x) = \frac{f(x)}{x}$ ($x > 0$) and prove that g is monotonically increasing.

Parent 2 in Lean

```
theorem Rudin_exercise_5_6
  {f : ℝ → ℝ}
  (hf1 : ContinuousOn f (Set.Ici 0))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf3 : f 0 = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi 0)) :
  MonotoneOn (λ x => f x / x) (Set.Ioi 0) := by
```

What it contributes. Supplies the monotonicity comparison for $f y / y \leq f c / c$; this is fused with the fixed-point equation through `hquot` and `hstable_bound`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines both parents: the interval fixed-point result supplies c with $f c = c$, while normalized monotonicity is applied at $y \leq c$ to derive $f y / y \leq f c / c = 1$ and hence $f y \leq y$. Neither parent can be discharged independently of the other in the final stability conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1 supplies the fixed point c , whose positivity is established using the positive self-map condition; Parent 2 supplies monotonicity of $f x / x$, which is then applied at y and c to derive $f y \leq y$. The fixed-point conclusion is therefore an actual input to the normalized-monotonicity argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed point supplied by Parent 1 provides the specific endpoint c that Parent 2's normalized monotonicity then consumes to derive $f y / y \leq f c / c = 1$ and hence $f y \leq y$. Neither parent alone supplies this stability conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1862 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_fixed_point_normalized_stability.extracted_1_1 {f : ℝ → ℝ} (hcont : ContinuousOn f (Set.Icc 0 1))
  (hmaps : Set.MapsTo f (Set.Icc 0 1) (Set.Icc 0 1)) (hpositive : ∀ x ∈ Set.Icc 0 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi 0)) :
  ∃ c ∈ Set.Icc 0 1, 0 < c ∧ f c / c = 1 ∧ ∀ y ∈ Set.Ioc 0 c, f y ≤ y
```

Read back by a model that saw only the Lean. Let f be a real-valued function on the real numbers. Assume that f is continuous on $[0,1]$, maps $[0,1]$ into $[0,1]$, satisfies $f(x) > 0$ for every $x \in [0,1]$, and the function $x \mapsto f(x)/x$ is monotone nondecreasing on $(0, \infty)$. Then there exists $c \in [0,1]$ such that $c > 0$, $f(c)/c = 1$, and for every y with $0 < y \leq c$, one has $f(y) \leq y$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem positive_fixed_point_normalized_stability
  {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Icc (0 : ℝ) 1))
  (hmaps : Set.MapsTo f (Set.Icc (0 : ℝ) 1) (Set.Icc (0 : ℝ) 1))
  (hpositive : ∀ x ∈ Set.Icc (0 : ℝ) 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi (0 : ℝ))) :
  ∃ c ∈ Set.Icc (0 : ℝ) 1, 0 < c ∧ f c / c = 1 ∧
    ∀ y ∈ Set.Ioc (0 : ℝ) c, f y ≤ y := by
  have hfixed : ∃ c ∈ Set.Icc (0 : ℝ) 1, Function.IsFixedPt f c :=
    exists_mem_Icc_isFixedPt_of_mapsTo hcont zero_le_one hmaps
  rcases hfixed with ⟨c, hc, hfc⟩
  have hfc' : f c = c := by
    simpa [Function.IsFixedPt] using hfc
  have hcpos : 0 < c := by
    have hpos := hpositive c hc
    linarith [hpos, hfc']
  have hquot : f c / c = 1 := by
    rw [hfc']
    exact div_self (ne_of_gt hcpos)
  refine ⟨c, hc, hcpos, hquot, ?_⟩
  intro y hy
  have hypos : 0 < y := hy.1
  have hy_mem : y ∈ Set.Ioi (0 : ℝ) := hypos
  have hc_mem : c ∈ Set.Ioi (0 : ℝ) := hcpos
  have hratio : f y / y ≤ f c / c := hmono hy_mem hc_mem hy.2
  have hstable_bound : f y / y ≤ 1 := by
    calc
      f y / y ≤ f c / c := hratio
      _ = 1 := hquot
```

```
have hscaled : f y ≤ 1 * y := (div_le_iff hypos).mp hstable_bound
simpa using hscaled
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__x__Rudin_exercise_5_6__xe__c5eb495c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c08d399978c8e429

4.10 010. closed_subspace_trace_basis_compact_refinement

The problem

Let A be a closed subset of a compact metric space K . Construct a countable family of ambient open sets whose traces form a basis for A , and such that every neighborhood of a point of A contains the compact closure of one of these ambient basis sets.

Lean statement

```
theorem closed_subspace_trace_basis_compact_refinement {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    IsCompact A ∧
    (∀ x : K, x ∈ A → ∀ U : Set K, IsOpen U → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U)
```

Parent 1. Let X be a Hausdorff locally compact space, and let $(B_n)_{n \in \mathbb{N}}$ be a countable open neighborhood basis at x . Show that for every open neighborhood U of x , some B_n has compact closure contained in U .

Parent 1 in Lean

```
theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℱ x, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

What it contributes. Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

Parent 2. Prove that every compact metric space K has a countable base.

Parent 2 in Lean

```
theorem Rudin_exercise_2.25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely fuses both parents: the compact-closure local refinement is combined with the ambient countable basis to select a basis element whose closure remains inside the prescribed neighborhood, while the basis is transported to the closed subspace via traces. Neither parent alone supplies this combined refinement-and-subspace-basis conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the ambient compact-metric countable basis with compact-closure local refinement: the same ambient family supplies both the subspace trace basis and, after selecting inside a compact-closure neighborhood, the refined sets. The closed-subspace compactness and trace-basis construction are also substantive consequences needed for the stated result.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the ambient countable basis with the compact-closure refinement invariant: a basis element is selected inside a locally compact neighborhood, then its closure is shown compact and contained in the prescribed open set. Neither parent alone supplies both the trace-basis construction and the refined basis property.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2899 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_trace_basis_compact_refinement.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    IsCompact A ∧
    ∀ x ∈ A, ∀ (U : Set K), IsOpen U → x ∈ U → ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let $A \subseteq K$ be closed. Then there exists a countable collection B of subsets of K such that every $U \in B$ is open, the collection of traces $\{A \cap U \mid U \in B\}$ is a topological basis for A with its subspace topology, A is compact, and for every $x \in A$ and every open set $U \subseteq K$ containing x , there exists $V \in B$ such that $x \in V$, the closure of V in K is compact, and the closure of V is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem closed_subspace_trace_basis_compact_refinement {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    IsCompact A ∧
    (∀ x : K, x ∈ A → ∀ U : Set K, IsOpen U → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U) := by
  classical
  let B : Set (Set K) := TopologicalSpace.countableBasis K
  have hCount : B.Countable := by
    simpa [B] using TopologicalSpace.countable_countableBasis K
  have hOpen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact TopologicalSpace.isOpen_of_mem_countableBasis (by simpa [B] using hU)
  have hBasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
    simpa [B] using
      (TopologicalSpace.IsTopologicalBasis.induced (Subtype.val : A → K))
```



```

    (TopologicalSpace.isBasis_countableBasis K))
have hRefine :  $\forall x : K, x \in A \rightarrow \forall U : \text{Set } K, \text{IsOpen } U \rightarrow x \in U \rightarrow$ 
   $\exists V \in B, x \in V \wedge \text{IsCompact } (\text{closure } V) \wedge \text{closure } V \subseteq U :=$  by
  intro x hxA U hU hxU
  have hcompact_x : IsCompact ({x} : Set K) := isCompact_singleton
  have hsingletonU : ({x} : Set K)  $\subseteq$  U := by
    rintro y rfl
    exact hxU
  obtain ⟨W, hWopen, hsingletonW, hclWsub, hclWcompact⟩ :=
    exists_open_between_and_isCompact_closure hcompact_x hU hsingletonU
  have hxW :  $x \in W :=$  hsingletonW (mem_singleton x)
  have hWnhds :  $W \in \mathcal{N} x :=$  hWopen.mem_nhds hxW
  have hAmbientBasis := TopologicalSpace.isBasis_countableBasis K
  obtain ⟨V, hVB, hxV, hVW⟩ := hAmbientBasis.mem_nhds_iff.mp hWnhds
  have hSelected :  $V \in B \wedge x \in V \wedge V \subseteq W :=$  by
    exact ⟨by simp [B] using hVB, hxV, hVW⟩
  have hclsub :  $\text{closure } V \subseteq \text{closure } W :=$  closure_mono hSelected.2.2
  have hclU :  $\text{closure } V \subseteq U :=$  hclsub.trans hclWsub
  have hclcompact : IsCompact (closure V) :=
    hclWcompact.of_isClosed_subset isClosed_closure hclsub
  exact ⟨V, hSelected.1, hSelected.2.1, hclcompact, hclU⟩
exact ⟨B, hCount, hOpen, hBasis, hA.isCompact, hRefine⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__cb92bce9__x__Rudin_exercise_2_25__xe__5ddc9a0c

Fusion mechanism. invariant_transplant Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 5ddc9a0ccb26edf3

4.11 011. compact_neighborhood_avoiding_common_level_locus

The problem

Let X be a locally compact metric space, let s be finite, and let $(f_i)_{i \in s}$ be continuous real-valued functions. If x is outside their common level locus and U is an open neighborhood of x , show that some open neighborhood V of x has compact closure contained in U and disjoint from the common level locus.

Lean statement

```
theorem compact_neighborhood_avoiding_common_level_locus
  {α : Type*} [MetricSpace α] [LocallyCompactSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (hcont : ∀ i, Continuous (f i)) (c : ℝ)
  (x : α)

  (hxL : x ∉ {y : α | ∀ i ∈ s, f i y = c})
  (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V : Set α, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧
    closure V ⊆ U ∧ Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
```

Parent 1. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent 1 in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

Parent 2. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent 2 in Lean

```
theorem finite_common_level_locus_closed
  {α : Type*} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely fuses both parents: continuity makes the common level locus closed, so its complement can be intersected with U to form the neighborhood to which local compactness is applied. The resulting compact closure is then shown to avoid the level locus, rather than merely combining two independent conclusions.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines Parent 2's closedness result into Parent 1's local compactness construction: the complement of the common level locus is combined with U to form an open neighborhood, whose compactly closed neighborhood then yields avoidance. Neither parent alone supplies the resulting disjointness conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines Parent 2's closedness of the common level locus into an open avoidance neighborhood, then uses Parent 1's local compactness construction inside that smaller neighborhood. The avoidance condition is carried through the compact closure, so neither parent is merely discharged in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2350 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_neighborhood_avoiding_common_level_locus.extracted_1_1.{u_2, u_1} {α : Type u_1} {ι : Type u_2}
  [inst : MetricSpace α] [inst_1 : LocallyCompactSpace α] (s : Finset ι) (f : ι → α → ℝ)
  (hcont : ∀ (i : ι), Continuous (f i)) (c : ℝ) (x : α) (hxlocal : ∃ N, x ∈ N ∧ IsOpen N ∧ ∃ K, N ⊆ K ∧ IsCompact K)
  (hxL : x ∉ {y | ∀ i ∈ s, f i y = c}) (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧ Disjoint (closure V) {y | ∀ i ∈ s, f i y = c}
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric-space structure and a locally compact-space structure, let ι be a type, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a continuous function $f(i) : \alpha \rightarrow \mathbb{R}$. Let $c \in \mathbb{R}$ and $x \in \alpha$. Assume that there exist sets N and K such that $x \in N$, N is open, $N \subseteq K$, and K is compact. Assume also that x does not belong to the common level set $\{y \in \alpha \mid \text{for every } i \in s, f(i)(y) = c\}$. For any open set $U \subseteq \alpha$ containing x , there exists an open set $V \subseteq \alpha$ such that $x \in V$, the closure of V is compact, the closure of V is contained in U , and the closure of V is disjoint from the common level set $\{y \in \alpha \mid \text{for every } i \in s, f(i)(y) = c\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem compact_neighborhood_avoiding_common_level_locus
  {α ι : Type*} [MetricSpace α] [LocallyCompactSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (hcont : ∀ i, Continuous (f i)) (c : ℝ)
  (x : α)

  (hxL : x ∉ {y : α | ∀ i ∈ s, f i y = c})
  (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V : Set α, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧
    closure V ⊆ U ∧ Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
  classical
  have hLclosed : IsClosed {y : α | ∀ i ∈ s, f i y = c} := by
    rw [show {y : α | ∀ i ∈ s, f i y = c} = ⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ) by
      ext y
      simp]
    exact isClosed_iInter (fun i => isClosed_iInter (fun hi =>
      isClosed_singleton.preimage (hcont i)))
  let W : Set α := U ∩ {y : α | ∀ i ∈ s, f i y = c}ᶜ
  have hWopen : IsOpen W := by
    dsimp [W]
    exact hU.inter hLclosed.isOpen_compl
  have hxW : x ∈ W := by
    exact ⟨hxU, hxL⟩
  have hWnhds : W ∈ nhds x := hWopen.mem_nhds hxW
  obtain ⟨K, hKnhd, hKW, hKcompact⟩ := local_compact_nhds hWnhds
```

```

have hKclosed : IsClosed K := hKcompact.isClosed
obtain ⟨V, hVK, hVopen, hxV⟩ := mem_nhds_iff.mp hKnhds
have hclosureK : closure V ⊆ K := by
  exact closure_minimal hVK hKclosed
have hclosureW : closure V ⊆ W := by
  intro y hy
  exact hKW (hclosureK hy)
have hclosureU : closure V ⊆ U := by
  intro y hy
  exact (hclosureW hy).1
have hclosurecompact : IsCompact (closure V) := by
  exact hKcompact.of_isClosed_subset isClosed_closure hclosureK
have hdisjoint : Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
  rw [Set.disjoint_left]
  intro y hyV hyL
  exact (hclosureW hyV).2 hyL
exact ⟨V, hVopen, hxV, hclosurecompact, hclosureU, hdisjoint⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__x__Rudin_exercise_4_3__me__fd57004e

Fusion mechanism. obstruction_as_lemma The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 43f7a0b0b99a56a0

4.12 012. irrational_checkpoint_uniform_composition

The problem

Let β and γ be uniform spaces, and let $f : \mathbb{R} \rightarrow \beta$ and $g : \beta \rightarrow \gamma$. If x is irrational, y is a nonzero rational, whenever xy is irrational f is uniformly continuous, and g is uniformly continuous, prove that $g \circ f$ is uniformly continuous.

Lean statement

```
theorem irrational_checkpoint_uniform_composition
  {β γ : Type*} [UniformSpace β] [UniformSpace γ]
  {f : ℝ → β} {g : β → γ}
  (x : ℝ) (y : ℚ) (h : y ≠ 0) (hx : Irrational x)
  (hf_if : Irrational (x * y) → UniformContinuous f)
  (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f)
```

Parent 1. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent 1 in Lean

```
theorem Rudin_exercise_1_1b
  (x : ℝ)
  (y : ℚ)
  (h : y ≠ 0)
  : (Irrational x) → Irrational (x * y) := by
```

What it contributes. pipeline input checkpoint/object/hypothesis

Parent 2. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent 2 in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. goal form or proof target that consumes the input

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: Parent 1 proves $\text{Irrational}(x * y)$, which is consumed as the premise of `hf_if` to obtain uniform continuity of f ; Parent 2 then composes that result with `hg`. Neither parent is discharged independently or merely combined at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1's irrationality result supplies the premise needed by `hf_if` to construct uniform continuity of f , which Parent 2 then consumes to prove uniform continuity of $g \circ f$. This is a genuine sequential pipeline rather than parallel discharge or conjunction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: Parent 1 derives Irrational ($x * y$), which is consumed by hf_if to construct UniformContinuous f; Parent 2 then consumes that result with hg to prove UniformContinuous ($g \circ f$).

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1994 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_checkpoint_uniform_composition.extracted_1_1.{u_2, u_1} {β : Type u_1} {γ : Type u_2}
  [inst : UniformSpace β] [inst_1 : UniformSpace γ] {f : ℝ → β} {g : β → γ} (x : ℝ) (y : ℚ) (h : y ≠ 0)
  (hx : Irrational x) (hf_if : Irrational (x * ↑y) → UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f)
```

Read back by a model that saw only the Lean. Let β and γ be types equipped with uniform structures, let $f : \mathbb{R} \rightarrow \beta$ and $g : \beta \rightarrow \gamma$, let x be a real number and y a nonzero rational number. Assume that x is irrational, that if the real number x multiplied by the real embedding of y is irrational then f is uniformly continuous, and that g is uniformly continuous. Then the composition $g \circ f : \mathbb{R} \rightarrow \gamma$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem irrational_checkpoint_uniform_composition
  {β γ : Type*} [UniformSpace β] [UniformSpace γ]
  {f : ℝ → β} {g : β → γ}
  (x : ℝ) (y : ℚ) (h : y ≠ 0) (hx : Irrational x)
  (hf_if : Irrational (x * y) → UniformContinuous f)
  (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
  have hxy : Irrational (x * y) := hx.mul_ratCast h
  have hf : UniformContinuous f := hf_if hxy
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  exact hcomp
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__x__Rudin_exercise_4_12__xe__cf5f0efc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-gerprint 139c1eae78687b59

4.13 013. closed_subset_compact_perfect_countable_basis

The problem

Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that the subspace P has a countable topological basis.

Lean statement

```
theorem closed_subset_compact_perfect_countable_basis (X : Type*) [MetricSpace X] [CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Perfect P ∧ Set.Countable C ∧
  ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B
```

Parent 1. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 1 in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

Parent 2. Prove that every compact metric space K has a countable base.

Parent 2 in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The decomposition parent supplies the specific perfect kernel P , and the basis construction is transferred to the subtype $\{x // x \in P\}$; the basis is therefore not an independent conclusion about X . Establishing P is closed and compact is the bridge that makes the second parent's compact-metric basis result applicable.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The decomposition parent supplies the specific perfect kernel P , and the basis parent is applied to the compact subtype of that constructed witness. The basis claim therefore depends on the decomposition's chosen P rather than being an independent conjunct.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The decomposition theorem supplies the specific perfect kernel P , whose closedness and compactness are then used to construct a countable basis on the subtype P . The basis construction therefore depends on the witness produced by the decomposition rather than being an independent conjunct.

Check

By

Result

statement_type_check
proof_accepted

lean
lean

type-checks
accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2238 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A requires a countable topological basis for the subspace P, whereas Statement B requires one for the ambient space X.

Goal round-trip: what Lean elaborated

```
theorem closed_subset_compact_perfect_countable_basis.extracted_1.1.{u.1} (X : Type u.1) [inst : MetricSpace X]
[inst_1 : CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧ ∃ B, B.Countable ∧ TopologicalSpace.IsTopologicalBasis B
```

Read back by a model that saw only the Lean. *For any compact metric space X and any closed subset A of X , there exist subsets P and C of X such that $A = P \cup C$, P is perfect, C is countable, and there exists a countable collection B that is a topological basis for X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem closed_subset_compact_perfect_countable_basis (X : Type*) [MetricSpace X] [CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Perfect P ∧ Set.Countable C ∧
    ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B := by
  classical
  have hSecondX : SecondCountableTopology X := inferInstance
  have hdecomp : ∃ V D : Set X, V.Countable ∧ Perfect D ∧ A = V ∪ D :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  obtain ⟨C, P, hCcount, hPperfect, hdecomp_eq⟩ := hdecomp
  have hPclosed : IsClosed P := hPperfect.closed
  have hPcompact : IsCompact P := hPclosed.isCompact
  have hPsep : TopologicalSpace.IsSeparable P := hPcompact.isSeparable
  have hPsecond : SecondCountableTopology P := hPsep.secondCountableTopology
  letI : SecondCountableTopology P := hPsecond
  have hWitnessBasis : ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B := by
    obtain ⟨B, hBcount, hBne, hBbasis⟩ := TopologicalSpace.exists_countable_basis P
    exact ⟨B, hBcount, hBbasis⟩
  refine ⟨P, C, ?_, hPperfect, hCcount, hWitnessBasis⟩
  simpa [Set.union_comm] using hdecomp_eq
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Rudin_exercise_2_25__xe__e7796e5c

Fusion mechanism. witness_exchange Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e7796e5ca5da6898

4.14 014. zero_constrained_positivity_closed

The problem

Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Lean statement

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

Parent 1. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent 1 in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

Parent 2. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent 2 in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely combines both parents: Parent 1 establishes that the zero locus is closed, then Parent 2's comparison-closedness result is applied within that locus to the condition $0 \leq g(x)$. The zero constraint therefore changes the domain on which the comparison theorem is used, rather than merely joining two independently proved facts.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses Parent 1 to establish that the zero locus is closed, then feeds that closed set into Parent 2's ordered-comparison lemma to constrain it by $0 \leq g(x)$. The positivity condition is therefore closed specifically within the zero locus, so neither parent can be discharged independently.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines the parents: Parent 1 establishes that the zero locus is closed, and Parent 2's comparison-closedness theorem is then applied relative to that locus for the condition $0 \leq g(x)$. The first result is an essential input to the second derivation, not a parallel conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2114 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem zero_constrained_positivity_closed.extracted_1.1.{u_1} {X : Type u_1} [inst : MetricSpace X] (f g : X → ℝ)
(hf : Continuous f) (hg : Continuous g) : IsClosed {x | f x = 0 ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. *Let X be a metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous functions. Then the set $\{x \in X \mid f(x) = 0 \text{ and } 0 \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_constrained_closed :
    IsClosed ({x ∈ {x | f x = 0} | (fun _ : X => 0) x ≤ g x}) := by
    exact h_zero_closed.isClosed_le continuous_const.continuousOn hg.continuousOn
  simpa using h_constrained_closed
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe__49c9c080

Fusion mechanism. invariant_transplant The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 49c9c08073317cd9

4.15 015. coupled_difference_isLeast

The problem

Suppose f attains its least value a on S , and g attains its greatest value b on T . Show that the least value of $f(x) - g(y)$ on $S \times T$ is $a - b$.

Lean statement

```
theorem coupled_difference_isLeast
  (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ)
  (hf : IsLeast (Set.image f S) a)
  (hg : IsGreatest (Set.image g T) b) :
  IsLeast (Set.image2 (fun x y : ℝ => f x - g y) S T) (a - b)
```

Parent 1. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 012.

Parent 1 in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

What it contributes. Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

Parent 2. What is the maximum value of $\frac{(2^t - 3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent 2 in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12 : ℝ) := by
```

What it contributes. Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely uses both parent roles: the first least-value witness and lower bound, and the second greatest-value witness and upper bound, jointly establish attainment and the lower bound for the coupled difference objective. Neither extremum alone can derive the stated least-value result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses both parent conclusions: the first supplies a minimizing witness and lower bound, while the second supplies a maximizing witness and upper bound. These bounds combine specifically through subtraction to establish both attainment and minimality of the coupled image.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines both extremum conclusions: the first supplies the lower bound and witness for f , while the second supplies the upper bound and witness for g , yielding both attainment and the coupled inequality for $f x - g y$. Neither parent alone establishes this difference-objective least-value theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2364 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_difference_isLeast.extracted_1_1 (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ) (hf : IsLeast (f '' S) a)
(hg : IsGreatest (g '' T) b) : IsLeast (Set.image2 (fun x y => f x - g y) S T) (a - b)
```

Read back by a model that saw only the Lean. For sets S and T of real numbers, functions f and g from $\mathbb{R}$ to $\mathbb{R}$, and real numbers a and b , if a is the least element of the set $\{f(x) \mid x \in S\}$ and b is the greatest element of the set $\{g(y) \mid y \in T\}$, then $a - b$ is the least element of the set $\{f(x) - g(y) \mid x \in S \text{ and } y \in T\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_difference_isLeast
  (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ)
  (hf : IsLeast (Set.image f S) a)
  (hg : IsGreatest (Set.image g T) b) :
  IsLeast (Set.image2 (fun x y : ℝ => f x - g y) S T) (a - b) := by
  rcases hf.1 with ⟨x, hx, hfx⟩
  rcases hg.1 with ⟨y, hy, hgy⟩
  have hattain : a - b ∈ Set.image2 (fun x y : ℝ => f x - g y) S T := by
    refine ⟨x, hx, y, hy, ?_⟩
    simp [hfx, hgy]
  have hbound : ∀ x ∈ S, ∀ y ∈ T, a - b ≤ f x - g y := by
    intro x hx y hy
    have hfx' : a ≤ f x := hf.2 ⟨x, hx, rfl⟩
    have hgy' : g y ≤ b := hg.2 ⟨y, hy, rfl⟩
    linarith
  constructor
  · exact hattain
  · intro z hz
    rcases hz with ⟨x, hx, y, hy, rfl⟩
    exact hbound x hx y hy
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__x__amc12b_2020_p22__xe__7194e753

Fusion mechanism. invariant_transplant Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7194e753bfc7d7d3

4.16 016. quadratic_reciprocal_specialization

The problem

Suppose $x^2 - 5x - 4 \leq 10$. Show that $\lfloor \frac{9}{13} \rfloor \leq \frac{2}{12} + \frac{2}{10-x} + \frac{2}{x+4}$.

Lean statement

```
theorem quadratic_reciprocal_specialization (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4)
```

Parent 1. Show that for any three positive real numbers x , y , and z , $9/(x+y+z) \leq 2/(x+y) + 2/(y+z) + 2/(z+x)$.

Parent 1 in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Its positive-three-variable reciprocal inequality is reproduced as `h_parent1` and explicitly instantiated at $(x+3, 9-x, 1)$.

Parent 2. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent 2 in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

What it contributes. Its quadratic interval checkpoint is derived as `hinterval` and feeds all affine positivity hypotheses used by the reciprocal application.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines both parents: the quadratic hypothesis yields the interval bounds needed to prove positivity of the affine triple, and Parent 1's reciprocal inequality is then instantiated and normalized to the target.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quadratic hypothesis is used to derive the interval bounds that certify positivity of the affine triple, and the reciprocal inequality is then obtained by applying parent 1 to that triple and normalizing. Both parents materially determine the proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quadratic hypothesis is used to derive bounds on x , which certify positivity of the affine triple $(x+3, 9-x, 1)$; the reciprocal inequality is then instantiated on that triple and normalized into the conclusion. Both parents materially interact through this specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2515 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_reciprocal_specialization.extracted_1_1 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4)
```

Read back by a model that saw only the Lean. *For a real number x satisfying $x^2 - 5x - 4 \leq 10$, we have $9/13 \leq 2/12 + 2/(10 - x) + 2/(x + 4)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem quadratic_reciprocal_specialization (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4) := by
  have hinterval : x ∈ Set.Icc (-2 : ℝ) 7 := by
    constructor
    · by_contra hx
      have hx' : x < -2 := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra hx
      have hx' : 7 < x := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  have hA : 0 < x + 3 := by
    linarith [hinterval.1]
  have hB : 0 < 9 - x := by
    linarith [hinterval.2]
  have hC : (0 : ℝ) < 1 := by
    linarith [hinterval.1, hinterval.2]
  have h_parent1 : ∀ a b c : ℝ, 0 < a ∧ 0 < b ∧ 0 < c →
    9 / (a + b + c) ≤ 2 / (a + b) + 2 / (b + c) + 2 / (c + a) := by
    intro a b c habc
    rcases habc with ⟨ha, hb, hc⟩
    have hab : 0 < a + b := by linarith
    have hbc : 0 < b + c := by linarith
    have hca : 0 < c + a := by linarith
    have habc' : 0 < a + b + c := by linarith
    have hnum :
      0 ≤ 2 * (a + b + c) *
        ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b)) -
        9 * (a + b) * (b + c) * (c + a) := by
      have h1 : 0 ≤ ((a + b) - (b + c)) ^ 2 := sq_nonneg _
      have h2 : 0 ≤ ((b + c) - (c + a)) ^ 2 := sq_nonneg _
      have h3 : 0 ≤ ((c + a) - (a + b)) ^ 2 := sq_nonneg _
      have h1' : 0 ≤ (a + b) * ((a + b) - (b + c)) ^ 2 :=
        mul_nonneg (le_of_lt hab) h1
      have h2' : 0 ≤ (b + c) * ((b + c) - (c + a)) ^ 2 :=
        mul_nonneg (le_of_lt hbc) h2
      have h3' : 0 ≤ (c + a) * ((c + a) - (a + b)) ^ 2 :=
        mul_nonneg (le_of_lt hca) h3
      nlinarith
    have hden : 0 < (a + b + c) * (a + b) * (b + c) * (c + a) := by
      positivity
    have hleft :
      9 / (a + b + c) =
        (9 * (a + b) * (b + c) * (c + a)) /
        ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
```

```

field_simp [ne_of_gt habc', ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
have hright :
  2 / (a + b) + 2 / (b + c) + 2 / (c + a) =
    (2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b))) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
field_simp [ne_of_gt habc', ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
rw [hleft, hright]
exact (div_le_div_iff_of_pos_right hden).2 (by nlinarith)
have hrec := h_parent1 (x + 3) (9 - x) 1 ⟨hA, hB, hC⟩
have hsum : (x + 3) + (9 - x) + 1 = (13 : ℝ) := by ring
have hab : (x + 3) + (9 - x) = (12 : ℝ) := by ring
have hbc : (9 - x) + 1 = 10 - x := by ring
have hca : (1 : ℝ) + (x + 3) = x + 4 := by ring
rw [hsum, hab, hbc, hca] at hrec
exact hrec

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__x__mathd_algebra_101__xe__7fee6821

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7fee68212b0d103c

4.17 017. weighted_energy_translated_square_min_unique

The problem

Suppose the four weighted-square equations for $n = 2, 4, 6, 8$ hold. If $a > 0$, show that the function $a(t - x)^2 + c$ on $[-6, 6]$ has minimum value c , attained uniquely at $t = x$.

Lean statement

```
theorem weighted_energy_translated_square_min_unique (x y z w a c : ℝ)
  (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x)
```

Parent 1. Show that no real numbers x, y, z, w satisfy the four weighted-square equations if $x \geq 7$.

Parent 1 in Lean

```
theorem weighted_square_infeasible_of_lower_bound (x y z w : ℝ)
  (hbound : 7 ≤ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  False
```

Parent 2. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent 2 in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The weighted-square equations are genuinely consumed to derive $x \in [-6, 6]$, which is necessary to attain the translated square's minimum on the interval. That checkpoint then feeds the minimum and uniqueness conclusion, so the child is a substantive pipeline rather than a parallel combination.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weighted-energy hypotheses are used to derive the essential bound $x^2 \leq 36$, which places the minimizer $t=x$ inside the interval and enables the translated-square minimum. The square-minimum argument alone cannot establish attainment on the restricted domain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weighted-square hypotheses from Parent 1 derive $x^2 \leq 36$ and hence $x \in [-6, 6]$, which is essential to exhibit c as the minimum over the restricted interval. Parent 2's translated-square minimum structure then uses that parent-derived domain membership; without it, the minimum claim can fail.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2704 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_energy_translated_square_min_unique.extracted_1_1 (x y z w a c : ℝ) (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q | ∃ t ∈ Set.Icc (-6) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6) 6, a * (t - x) ^ 2 + c = c ↔ t = x
```

Read back by a model that saw only the Lean. Let x, y, z, w, a , and c be real numbers with $a > 0$. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then c is the least element of the set of all values $a(t-x)^2+c$ as t ranges over the closed interval $[-6,6]$, and for every t in $[-6,6]$, $a(t-x)^2+c = c$ if and only if $t = x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem weighted_energy_translated_square_min_unique (x y z w a c : ℝ)
  (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x) := by
  norm_num at h₀ h₁ h₂ h₃
  have henergy : x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
    linarith
  have hx_sq : x ^ 2 ≤ 36 := by
    nlinarith [henergy, sq_nonneg y, sq_nonneg z, sq_nonneg w]
  have hinterval : x ∈ Set.Icc (-6 : ℝ) 6 := by
    constructor <|> nlinarith [sq_nonneg (x - 6), sq_nonneg (x + 6), hx_sq]
  have hmem : c ∈ {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} := by
    refine ⟨x, hinterval, ?_⟩
    ring
  have hlower : ∀ q ∈ {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c}, c ≤ q := by
    intro q hq
    rcases hq with ⟨t, ht, rfl⟩
    have hsq : 0 ≤ (t - x) ^ 2 := sq_nonneg (t - x)
    have hprod : 0 ≤ a * (t - x) ^ 2 := mul_nonneg (le_of_lt ha) hsq
    linarith
  have hleast : IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c := by
    exact ⟨hmem, hlower⟩
  have hunique : ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x) := by
    intro t ht
    constructor
    · intro heq
      have hzero : a * (t - x) ^ 2 = 0 := by
        linarith
      have hsquare : (t - x) ^ 2 = 0 := by
```

```
    exact (mul_eq_zero.mp hzero).resolve_left (ne_of_gt ha)
  nlinarith
  · intro htx
    subst t
    ring
  exact ⟨hleast, hunique⟩
```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__x__aime_1984_p15__xe.mh__483514d7__x__mathd_algebra_410__ms.xe__ac8bb441

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ac8bb4412ba21e23

4.18 018. entropy_mag_crossover

The problem

A geometric sequence begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$ and has a constant ratio. If a circuit has voltage $V = a_5 + i$ and impedance $Z = 2 - i$, find the current I satisfying $V = IZ$.

Lean statement

```
theorem entropy_mag_crossover
  (a : ℕ → ℚ)
  (d : ℚ)
  (v i z : ℂ)
  (hrec : ∀ n, a (n + 1) = a n * d)
  (h1 : a 0 = 27 / 125)
  (h2 : a 1 = 9 / 25)
  (h3 : a 2 = 3 / 5)
  (hcircuit : v = i * z)
  (hvoltage : v = (a 5 : ℂ) + Complex.I)
  (himpedance : z = 2 - Complex.I) :
  i = 41 / 45 + 43 / 45 * Complex.I
```

Parent 1. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent 1 in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h0 : ∀ n, a (n + 1) = a n * d)
  (h1 : a 0 = 27 / 125)
  (h2 : a 1 = 9 / 25)
  (h3 : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. The geometric-sequence value a_5 is cast into the complex voltage $V = (a_5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

Parent 2. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent 2 in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h0 : v = i * z) (h1 : v = 1 + Complex.I)
  (h2 : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

What it contributes. The geometric-sequence value a_5 is cast into the complex voltage $V = (a_5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The recurrence is genuinely used to derive $a(5)=25/9$, which becomes the real part of the voltage and changes the complex current determined by the circuit equation. Neither parent can be discharged independently or recalled without performing this witness exchange.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The recurrence is genuinely needed to derive the real voltage component $a(5)=25/9$, which is then substituted into the circuit equation; that fused voltage determines the new complex current. Neither parent can be discharged independently and combined only at the end.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The recurrence is genuinely used to derive $a(5)=25/9$, which becomes the real component of the circuit voltage; that value changes the solved complex current. The circuit algebra then depends on this derived sequence output, so the parents are fused rather than discharged in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2450 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B indexes the listed initial terms from $a(0)$, so $a(5)$ is the sixth term, whereas Statement A's a_5 denotes the fifth term of the sequence. Consequently, B obtains $a(5)=25/9$ and current $41/45+(43/45)i$, while A gives $a_5=5/3$ and current $7/15+(11/15)i$.

Goal round-trip: what Lean elaborated

```
theorem entropy_mag_crossover.extracted_1_1 (a : ℕ → ℚ) (d : ℚ) (v i z : ℂ) (hrec : ∀ (n : ℕ), a (n + 1) = a n * d)
(h1 : a 0 = 27 / 125) (h2 : a 1 = 9 / 25) (h3 : a 2 = 3 / 5) (hcircuit : v = i * z)
(hvoltage : v = ↑(a 5) + Complex.I) (himpedance : z = 2 - Complex.I) : i = 41 / 45 + 43 / 45 * Complex.I
```

Read back by a model that saw only the Lean. Let a be a sequence of rational numbers, let d be a rational number, and let v , i , and z be complex numbers. Suppose that for every natural number n , $a(n + 1) = a(n)d$; $a(0) = 27/125$; $a(1) = 9/25$; $a(2) = 3/5$; $v = iz$; $v = a(5) + \text{the imaginary unit } I$, where $a(5)$ is regarded as a complex number; and $z = 2 - I$. Then $i = 41/45 + (43/45)I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem entropy_mag_crossover
(a : ℕ → ℚ)
(d : ℚ)
(v i z : ℂ)
(hrec : ∀ n, a (n + 1) = a n * d)
(h1 : a 0 = 27 / 125)
(h2 : a 1 = 9 / 25)
(h3 : a 2 = 3 / 5)
(hcircuit : v = i * z)
(hvoltage : v = (a 5 : ℂ) + Complex.I)
(himpedance : z = 2 - Complex.I) :
i = 41 / 45 + 43 / 45 * Complex.I := by
have hd : d = 5 / 3 := by
have h := hrec 0
norm_num [h1, h2] at h
linarith
```

```

have h23 := hrec 2
norm_num [h3, hd] at h23
have h34 := hrec 3
norm_num [h23, hd] at h34
have h45 := hrec 4
norm_num [h34, hd] at h45
have ha5 : a 5 = 25 / 9 := by
  norm_num [h45]
have hFused : (25 / 9 : ℂ) + Complex.I = i * (2 - Complex.I) := by
  calc
    (25 / 9 : ℂ) + Complex.I = (a 5 : ℂ) + Complex.I := by norm_num [ha5]
    _ = v := by symm; exact hvoltage
    _ = i * z := hcircuit
    _ = i * (2 - Complex.I) := by rw [himpedance]
have hr := congrArg Complex.re hFused
have hm := congrArg Complex.im hFused
apply Complex.ext
· norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
  linarith
· norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
  linarith

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__x__mathd_algebra_313__xe__4b9136ba

Fusion mechanism. witness_exchange The geometric-sequence value $a\ 5$ is cast into the complex voltage $V = (a\ 5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b9136ba34508a07

4.19 019. latent_parameter_radical

The problem

Suppose $3 + \frac{1}{x} = \frac{7}{x}$ and $a = x^3$. Show that $(16\sqrt[3]{a^2})^{1/3} = 4$.

Lean statement

```
theorem latent_parameter_radical (x a : ℝ) (h₁ : 3 + 1 / x = 7 / x) (h₂ : a = x ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4
```

Parent 1. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

Parent 2. If $a = 8$, what is the value of $(16\sqrt[3]{a^2})^{1/3}$? Show that it is 4.

Parent 2 in Lean

```
theorem mathd_algebra_114 (a : ℝ) (h₀ : a = 8) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
```

What it contributes. The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines the parents: the reciprocal equation forces $x = 2$, which combines with $a = x^3$ to derive the radical theorem's required input $a = 8$. Neither parent conclusion is independently asserted or merely combined at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reciprocal equation is needed to derive $x = 2$, which is then used through $a = x^3$ to establish $a = 8$, the essential input for evaluating the radical expression. Neither parent can be discharged independently or combined only at the end.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reciprocal equation is needed to force $x = 2$, which via $a = x^3$ supplies $a = 8$; that value is then required to evaluate the radical expression. Neither parent conclusion can be discharged independently of the other.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2571 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem latent_parameter_radical.extracted_1_1 (x a : ℝ) (h1 : 3 + 1 / x = 7 / x) (h2 : a = x ^ 3) :
  (16 * (a ^ 2) ^ (1 / 3)) ^ (1 / 3) = 4
```

Read back by a model that saw only the Lean. For real numbers x and a , if $3 + 1/x = 7/x$ and $a = x^3$, then $(16(a^2)^{(1/3)})^{(1/3)} = 4$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem latent_parameter_radical (x a : ℝ) (h1 : 3 + 1 / x = 7 / x) (h2 : a = x ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
  have hx : x ≠ 0 := by
    intro hx
    subst x
    norm_num at h1
  have hx2 : x = 2 := by
    field_simp [hx] at h1
    linarith
  have ha : a = 8 := by
    calc
      a = x ^ 3 := h2
      _ = 8 := by rw [hx2]; norm_num
  have hc : (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 := by
    convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num
  have hinner : (a ^ 2) ^ ((1 : ℝ) / 3) = 4 := by
    calc
      (a ^ 2) ^ ((1 : ℝ) / 3) = (64 : ℝ) ^ ((1 : ℝ) / 3) := by
        rw [ha]
        norm_num
      _ = 4 := hc
  calc
    (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = (16 * 4) ^ ((1 : ℝ) / 3) := by rw [hinner]
    _ = (64 : ℝ) ^ ((1 : ℝ) / 3) := by norm_num
    _ = 4 := hc
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__x__mathd_algebra_114__xe__e0f06d2f

Fusion mechanism. witness_exchange The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e0f06d2fb129aad9

4.20 020. coupled_real_complex_unique

The problem

Show that the reciprocal condition $3 + 1/x = 7/x$ together with $V = I((x) - i)$, where $V = 1 + i$, determines a unique real parameter and complex current.

Lean statement

```
theorem coupled_real_complex_unique : ∃! p : ℝ × ℂ, 3 + 1 / p.1 = 7 / p.1 ∧ (1 + Complex.I : ℂ) = p.2 * ((p.1 : ℂ) - Complex.I)
```

Parent 1. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

Parent 2. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent 2 in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
(h₂ : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

What it contributes. The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The real reciprocal equation is used to force $x = 2$, which then specializes the complex impedance equation and is essential to proving uniqueness of the current. Neither parent can discharge the coupled uniqueness claim independently.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the real reciprocal equation to force $x = 2$, then substitutes that value into the complex impedance equation to uniquely determine the current. Neither parent alone establishes uniqueness of the coupled pair.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the reciprocal equation to force the real parameter $x = 2$, then substitutes that value into the complex impedance equation to establish uniqueness of the current. Neither parent can be discharged independently, and the cited siblings do not duplicate this same parent pair and coupling.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2446 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_real_complex_unique.extracted_1_1 :
  ∃! p, 3 + 1 / p.1 = 7 / p.1 ∧ 1 + Complex.I = p.2 * (↑p.1 - Complex.I)
```

Read back by a model that saw only the Lean. *There exists exactly one ordered pair (x, z) , where x is a real number and z is a complex number, such that $3 + 1/x = 7/x$ and $1 + i = z(x - i)$, with i denoting the imaginary unit.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_real_complex_unique : ∃! p : ℝ × ℂ, 3 + 1 / p.1 = 7 / p.1 ∧ (1 + Complex.I : ℂ) = p.2 * ((p.1 : ℂ) - Complex.I) := by
  let i₀ : ℂ := 1 / 5 + 3 / 5 * Complex.I
  have hroot : (3 : ℝ) + 1 / 2 = 7 / 2 := by
    norm_num
  have hcurrent : (1 + Complex.I : ℂ) = i₀ * ((2 : ℝ) - Complex.I) := by
    apply Complex.ext <|> norm_num [i₀, Complex.mul_re, Complex.mul_im]
  refine ⟨(2, i₀), ⟨hroot, hcurrent⟩, ?_⟩
  rintro ⟨x, i⟩ ⟨hxrec, hxc⟩
  change 3 + 1 / x = 7 / x at hxrec
  change (1 + Complex.I : ℂ) = i * ((x : ℂ) - Complex.I) at hxc
  have hxne : x ≠ 0 := by
    intro hx
    subst x
    norm_num at hxrec
  field_simp [hxne] at hxrec
  have hx : x = 2 := by
    linarith
  have hxc' := hxc
  rw [hx] at hxc'
  have hxr := congrArg Complex.re hxc'
  have hxi := congrArg Complex.im hxc'
  have hi : i = i₀ := by
    apply Complex.ext
    · norm_num [i₀, Complex.mul_re, Complex.mul_im] at hxr hxi
      linarith
    · norm_num [i₀, Complex.mul_re, Complex.mul_im] at hxr hxi
      linarith
  apply Prod.ext
  · exact hx
  · exact hi
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__x__mathd_algebra_313__xe__dbb2d393

Fusion mechanism. parameter_coupling The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint dbb2d393cb6de0b2

4.21 021. quadratic_finset_offset_mod_residue

The problem

Let S be the finite set of positive natural numbers x satisfying $x^2 + 4x + 4 < 20$. If $2n \equiv 15 + |S| \pmod{47}$ for a natural number n , prove that $n \equiv 32 \pmod{47}$.

Lean statement

```
theorem quadratic_finset_offset_mod_residue (S : Finset ℕ) (n : ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 + S.card [MOD 47]) : n ≡ 32 [MOD 47] := by
```

Parent 1. For how many positive integer values of x is the sum $x^2 + 4x + 4$ less than 20? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_405 (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) : S.card = 2 := by
```

What it contributes. The classification/count of the inequality-defined finite set fixes the additive offset applied to the parent's residue class modulo 47.

Parent 2. Solve the congruence $2n \equiv 15 \pmod{47}$, as a residue modulo 47. (Give an answer between 0 and 46.) Show that it is 31.

Parent 2 in Lean

```
theorem mathd_numbertheory_99 (n : ℕ) (h₀ : 2 * n ≡ 15 [MOD 47]) : n ≡ 31 [MOD 47] := by
```

What it contributes. The classification/count of the inequality-defined finite set fixes the additive offset applied to the parent's residue class modulo 47.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: Parent 1's quadratic finset characterization is used to derive $S.\text{card} = 2$, which changes Parent 2's modular premise to $2*n \equiv 17$ and yields the new residue $n \equiv 32$. Neither parent's conclusion alone proves the child's result.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines Parent 1 into Parent 2: the quadratic finset proof establishes $S.\text{card} = 2$, which is consumed to transform the modular hypothesis into $2*n \equiv 17 \pmod{47}$ before deriving $n \equiv 32 \pmod{47}$. The cardinality is an active offset in the second derivation, not a parallel closing conjunction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: the quadratic finset classification supplies $S.\text{card} = 2$, which is substituted into the modular hypothesis to obtain $2*n \equiv 17$ before solving for $n \equiv 32$. The cardinality is an essential input to the second derivation, not a parallel closing conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1798 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_finset_offset_mod_residue.extracted_1_1 (S : Finset ℕ) (n : ℕ)
  (h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 + S.card [MOD 47]) :
  n ≡ 32 [MOD 47]
```

Read back by a model that saw only the Lean. *Let S be a finite set of natural numbers and let n be a natural number. Suppose that for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < 20$. Suppose also that $2n$ is congruent modulo 47 to 15 plus the cardinality of S . Then n is congruent to 32 modulo 47.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem quadratic_finset_offset_mod_residue (S : Finset ℕ) (n : ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 +
  S.card [MOD 47]) : n ≡ 32 [MOD 47] := by
  have hS : S = {1, 2} := by
    ext x
    rw [h₀]
  constructor
  · rintro ⟨hx, hineq⟩
    have hxle : x ≤ 2 := by
      by_contra h
      have hx3 : 3 ≤ x := by omega
      have hsquare : 9 ≤ x ^ 2 := by
        simpa [pow_two] using Nat.mul_le_mul hx3 hx3
      omega
    interval_cases x <|> norm_num at hx ⊢
  · intro hx
    simp only [Finset.mem_insert, Finset.mem_singleton] at hx
    rcases hx with rfl | rfl <|> norm_num
  have hcard : S.card = 2 := by
    rw [hS]
  norm_num
  have hmod : 2 * n ≡ 17 [MOD 47] := by
    simp only [Nat.ModEq] at h₁ ⊢
    omega
  have hres : n ≡ 32 [MOD 47] := by
    simp only [Nat.ModEq] at hmod ⊢
    omega
  exact hres
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_405__x__mathd_numbertheory_99__xe__de242f87

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7e9d051f691873e2

4.22 022. incompatible_quadratic_extrema

The problem

Show that no real number x can both satisfy $x^2 - 6x + 13 = 4$ and minimize $x^2 - 14x + 3$ over all real numbers.

Lean statement

```
theorem incompatible_quadratic_extrema
  (x : ℝ)
  (h₁ : x ^ 2 - 6 * x + 13 = 4)
  (h₂ : IsMinOn (fun y : ℝ => y ^ 2 - 14 * y + 3) Set.univ x) :
  False
```

Parent 1. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent 1 in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

Parent 2. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent 2 in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x ^ 2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The two parent results interact through an incompatibility: the first condition forces $x = 3$, while global minimality of the second quadratic requires comparison with its vertex $x = 7$, yielding a contradiction. Neither parent can be discharged independently and combined only at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The first condition forces the shared input to be the first quadratic's vertex, $x = 3$. The second condition requires that input to globally minimize the other quadratic, but comparison with its vertex $x = 7$ yields a contradiction, so both parents interact through genuine incompatibility.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The first parent's minimum-level condition forces $x = 3$, while the second parent's global-minimizer condition requires the quadratic at x to be no larger than its value at 7; substituting $x = 3$ yields a contradiction. Both parent results interact through the shared input, producing a genuine incompatibility proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2674 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem incompatible_quadratic_extrema.extracted_1_1 (x : ℝ) (h1 : x ^ 2 - 6 * x + 13 = 4)
(h2 : IsMinOn (fun y => y ^ 2 - 14 * y + 3) Set.univ x) : False
```

Read back by a model that saw only the Lean. *Let x be a real number. Suppose that $x^2 - 6x + 13 = 4$ and that x is a global minimizer over all real numbers of the function $y \mapsto y^2 - 14y + 3$. Then a contradiction follows; that is, False.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem incompatible_quadratic_extrema
(x : ℝ)
(h1 : x ^ 2 - 6 * x + 13 = 4)
(h2 : IsMinOn (fun y : ℝ => y ^ 2 - 14 * y + 3) Set.univ x) :
False := by
have hx : x = 3 := by
  nlinarith [sq_nonneg (x - 3)]
have hglobal := h2
rw [isMinOn_univ_iff] at hglobal
have hbad := hglobal 7
rw [hx] at hbad
norm_num at hbad
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__x__mathd_algebra_113__xe__d052b38e

Fusion mechanism. obstruction_as_lemma The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d052b38e59cd7ec8

4.23 023. ternary_fibonacci_mod4_obstruction

The problem

Let $d \in \{1, 2\}$ and let a natural number have base-three representation $122d_3$. If (a_n) is the Fibonacci sequence, then the decoded number cannot have the same remainder modulo 4 as a_{100} , whose remainder is 3.

Lean statement

```
theorem ternary_fibonacci_mod4_obstruction
  (d x : ℕ)
  (a : ℕ → ℕ)
  (hd : d < 3)
  (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1])
  (h₀ : a 1 = 1)
  (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4
```

Parent 1. Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Parent 1 in Lean

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

What it contributes. The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

Parent 2. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent 2 in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

What it contributes. The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both results: the ternary decoding gives $x=d+51$ and $d \in \{1,2\}$, excluding $x \equiv 3 \pmod{4}$, while the Fibonacci recurrence gives $a(100) \equiv 3 \pmod{4}$. The obstruction then links these computed residues to prove the inequality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The ternary decoding derives $x = d + 51$ and, using $0 < d < 3$, proves $x \bmod 4 \neq 3$; the Fibonacci recurrence independently derives $a_{100} \bmod 4 = 3$, and these residues are combined to establish the obstruction. Neither parent can be discharged without affecting the final inequality.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The ternary decoding forces $x = d + 51$, and the positivity and bound on d then restrict x modulo 4 to 0 or 1. The Fibonacci recurrence independently forces a 100 modulo 4 to be 3, and these results interact to prove the obstruction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2535 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_fibonacci_mod4_obstruction.extracted_1_1 (d x : ℕ) (a : ℕ → ℕ) (hd : d < 3) (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1]) (h₀ : a 1 = 1) (h₁ : a 2 = 1) (h₂ : ∀ (n : ℕ), a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4
```

Read back by a model that saw only the Lean. For natural numbers d and x and a function a from natural numbers to natural numbers, suppose that d is less than 3, d is positive, the base-3 digit expansion of x is exactly $[d, 2, 2, 1]$, $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$. Then x modulo 4 is not equal to $a(100)$ modulo 4.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem ternary_fibonacci_mod4_obstruction
  (d x : ℕ)
  (a : ℕ → ℕ)
  (hd : d < 3)
  (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1])
  (h₀ : a 1 = 1)
  (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4 := by
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  have hvalue : x = d + 51 := by
    calc
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]
  have ha : ∀ n, a n = Nat.fib n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      by_cases h0 : n = 0
      · subst n
        have hr := h₂ 0
        norm_num [h₀, h₁, Nat.fib_zero] at hr ⊢
        omega
      by_cases h1 : n = 1
      · subst n
        simp [h₀, Nat.fib_one]
      have hn : 2 ≤ n := by omega
```

```

have hr := h₂ (n - 2)
have hi2 := ih (n - 2) (by omega)
have hi1 := ih (n - 1) (by omega)
have hf := Nat.fib_add_two (n := n - 2)
rw [show n - 2 + 2 = n by omega, show n - 2 + 1 = n - 1 by omega] at hr hf
omega
have hFib : a 100 % 4 = 3 := by
  rw [ha 100]
  norm_num [Nat.fib]
have hd_cases : d = 1 ∨ d = 2 := by omega
have hterminal_obstruction : x % 4 ≠ 3 := by
  rcases hd_cases with rfl | rfl
  · norm_num [hvalue]
  · norm_num [hvalue]
have hobstruction : x % 4 ≠ a 100 % 4 := by
  intro hxa
  apply hterminal_obstruction
  rw [hFib] at hxa
  exact hxa
exact hobstruction

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me__0fd2ae3f__x__mathd_numbertheory_483__xe__b7ac3c8d

Fusion mechanism. obstruction_as_lemma The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint b7ac3c8dbd47f8d9

4.24 024. weighted_binomial_eq_scaled_complementary_iff

The problem

For positive integers n and k with $0 < k \leq n$, the weighted sum $\sum_{j=1}^n j \binom{n}{j}$ equals $n2^{n-1} \binom{n}{n-k}$ if and only if $k = n$.

Lean statement

```
theorem weighted_binomial_eq_scaled_complementary_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔ k = n
```

Parent 1. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent 1 in Lean

```
theorem numbertheory_sumkmlncqnmul2pownm1 (n : ℕ) (hn : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

What it contributes. Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

Parent 2. For integers $n \geq 2$ and $0 < k \leq n$, classify all k such that the complementary binomial coefficient $\binom{n}{n-k}$ equals 1.

Parent 2 in Lean

```
theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose n (n - k) = 1 ↔ k = n
```

What it contributes. Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines parent 1's weighted-sum identity into parent 2's complementary-binomial characterization: positivity permits cancellation of the normalization factor, leaving exactly the condition that the complementary coefficient is 1. Both parent results are essential to the coupled iff.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: parent 1 rewrites the weighted sum to the nonzero normalization factor, whose cancellation reduces the equality to the complementary binomial coefficient being 1; parent 2 then characterizes that condition as $k=n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality first uses parent 1 to identify the weighted binomial sum with the nonzero factor $n2^{n-1}$, then cancels that factor and uses parent 2 to characterize when the complementary binomial coefficient is 1. Neither parent alone determines the child's iff.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2661 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B uses $\binom{n-k}{1}$ instead of Statement A's $\binom{n}{n-k}$, changing the conclusion.

Goal round-trip: what Lean elaborated

```
theorem weighted_binomial_eq_scaled_complementary_iff.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  ∑ j ∈ Finset.Icc 1 n, j * n.choose j = n * 2 ^ (n - 1) * n.choose (n - k) ↔ k = n
```

Read back by a model that saw only the Lean. For natural numbers n and k , if n is positive, k is positive, and $k \leq n$, then the sum from $j = 1$ to n of j times the binomial coefficient $\binom{n}{j}$ equals $(n \cdot 2^{n-1} \cdot \binom{n-k}{1})$ if and only if $k = n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators

theorem weighted_binomial_eq_scaled_complementary_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔ k = n := by
  have hset : Finset.range (n + 1) = insert 0 (Finset.Icc 1 n) := by
    rw [Finset.range_eq_Ico]
    rw [← Finset.insert_Ico_add_one_left_eq_Ico (a := 0) (b := n + 1) (by omega)]
    simp [Finset.Ico_add_one_right_eq_Icc]
  have hsum :
    (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) = n * 2 ^ (n - 1) := by
    calc
      (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
        ∑ j ∈ insert 0 (Finset.Icc 1 n), j * Nat.choose n j := by
          rw [Finset.sum_insert (by simp)]
          simp
        _ = ∑ j ∈ Finset.range (n + 1), j * Nat.choose n j := by
          rw [hset]
        _ = n * 2 ^ (n - 1) := Nat.sum_range_mul_choose n
  have hnorm : 0 < n * 2 ^ (n - 1) := by
    positivity
  have hcomp : Nat.choose n (n - k) = 1 ↔ k = n := by
    have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
      Nat.choose_symm hkn
    have hone : Nat.choose n k = 1 ↔ k = 0 ∨ n = k :=
      Nat.choose_eq_one_iff
    constructor
    · intro h
      have hcases : k = 0 ∨ n = k := hone.mp (hsymm.symm.trans h)
      rcases hcases with hk0 | hnk
      · omega
      · exact hnk.symm
    · intro h
      subst k
      simp
  have hbridge :
    (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
      (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔
      Nat.choose n (n - k) = 1 := by
    constructor
    · intro h
      have hmul :
```

```

      (n * 2 ^ (n - 1)) * 1 =
      (n * 2 ^ (n - 1)) * Nat.choose n (n - k) := by
calc
  (n * 2 ^ (n - 1)) * 1 = n * 2 ^ (n - 1) := by simp
  _ =  $\sum j \in \text{Finset.Icc } 1 \ n, j * \text{Nat.choose } n \ j := \text{hsum.symm}$ 
  _ = (n * 2 ^ (n - 1)) * Nat.choose n (n - k) := h
have hone : 1 = Nat.choose n (n - k) :=
  Nat.mul_left_cancel hnorm hmul
exact hone.symm
· intro h
  rw [hsum, h]
  simp
have hcoupled :
  ( $\sum j \in \text{Finset.Icc } 1 \ n, j * \text{Nat.choose } n \ j$ ) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k)  $\leftrightarrow$  k = n :=
  hbridge.trans hcomp
exact hcoupled

```

Operator. crossover / crossover_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_summulnckeqnmul2pownm1__x__numbertheory_nckeqnm1ckpnm1ckm1__ms.me.xe__e5dfcd7a

Fusion mechanism. parameter_coupling Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e5dfcd7afa017abc

4.25 025. finite_even_normal_subgroup_in_normalCore

The problem

Let N be a finite normal subgroup of a group G , and let H be a subgroup containing N . Prove that the normal core of H contains an element of order 2 when $|N|$ is even.

Lean statement

```
theorem finite_even_normal_subgroup_in_normalCore {G : Type*} [Group G]
  (N H : Subgroup G) [Fintype N] (hN : N.Normal)
  (hEven : Even (Fintype.card N)) (hNH : N ≤ H) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ H.normalCore
```

Parent 1. For a subgroup H of a group G , show that its normal core is normal and that, for every normal subgroup N , N is contained in H exactly when it is contained in the normal core of H .

Parent 1 in Lean

```
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H ↔ N ≤ H.normalCore)
```

What it contributes. The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

Parent 2. Prove that a finite group whose number of elements is divisible by 2 contains an element of order 2.

Parent 2 in Lean

```
theorem finite_group_order_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ x : G, orderOf x = 2
```

What it contributes. The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The proof genuinely uses both parents: the finite-even subgroup result supplies an order-two element inside N , while normality and $N \leq H$ are used to transport that same witness into $H.\text{normalCore}$. The final witness membership depends on the normal-core equivalence, so this is not parallel conjunction or decoration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses both parents: the finite-even subgroup result constructs an order-two element inside N , while normal-core maximality transports $N \leq H$ to $N \leq H.\text{normalCore}$, yielding the required membership for that same witness.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: the finite-even subgroup theorem supplies an order-two element in N , while normal-core maximality transports $N \leq H$ into $N \leq H.\text{normalCore}$, yielding that same witness in the core. Neither parent alone establishes the combined existence-and-membership conclusion.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2326 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_even_normal_subgroup_in_normalCore.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (N H : Subgroup G)
  [inst_1 : Fintype ιN] (hN : N.Normal) (hEven : Even (Fintype.card ιN)) (hNH : N ≤ H) :
  ∃ x, orderOf x = 2 ∧ x ∈ H.normalCore
```

Read back by a model that saw only the Lean. *Let G be a group and let N and H be subgroups of G . Assume that N is finite, N is normal in G , the cardinality of N is even, and N is contained in H . Then there exists an element x of G whose order is 2 and which belongs to the normal core of H in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem finite_even_normal_subgroup_in_normalCore {G : Type*} [Group G]
  (N H : Subgroup G) [Fintype N] (hN : N.Normal)
  (hEven : Even (Fintype.card N)) (hNH : N ≤ H) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ H.normalCore := by
  letI : N.Normal := hN
  obtain ⟨k, hk⟩ := hEven
  have h_two_dvd : 2 ∣ Fintype.card N := by
    exact ⟨k, by simpa [two_mul] using hk⟩
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  have hNcore : N ≤ H.normalCore :=
    (Subgroup.normal_le_normalCore (H := H) (N := N)).mpr hNH
  obtain ⟨x, hxorder⟩ := exists_prime_orderOf_dvd_card (G := N) (p := 2) h_two_dvd
  have hxFinal : ∃ y : G, orderOf y = 2 ∧ y ∈ H.normalCore := by
    refine ⟨(x : G), ?, hNcore x.property⟩
    rw [Subgroup.orderOf_coe, hxorder]
  exact hxFinal
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms__d6f85b6a__x__Artin_exercise_2_11_3__ms.xh__56ce9551

Fusion mechanism. witness_exchange The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 56ce9551c3474d9f

4.26 026. even_normal_intersection_has_conjugacy_stable_involution

The problem

Let a finite group have a nonempty family of normal subgroups whose intersection has even cardinality. Prove that the intersection contains an element of order 2 whose every conjugate remains in the intersection.

Lean statement

```
theorem even_normal_intersection_has_conjugacy_stable_involution {G : Type*} [Group G] [Fintype G]
  (I : Type*) [Nonempty I] (H : I → Subgroup G)
  (hH : ∀ i : I, (H i).Normal)
  (hEven : Even (Nat.card (⋂ i : I, H i)))) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ (⋂ i : I, H i) ∧
    ∀ g : G, g * x * g-1 ∈ (⋂ i : I, H i)
```

Parent 1. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent 1 in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ i : I, H i) := by
```

What it contributes. Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

Parent 2. Prove that a group of even order contains an element of order 2.

Parent 2 in Lean

```
theorem Artin_exercise_2.11.3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

What it contributes. Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines both parents: normality of the intersection is established from Parent 1, then Parent 2 is applied to that intersection to obtain an involution whose conjugates remain inside it. The conjugacy-stability conclusion depends essentially on the first parent and the involution witness on the second.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The even-cardinality theorem is applied to the intersection subgroup itself, producing an involution inside it; the independently established normality then transports that specific witness through every conjugation. Neither parent is merely appended or used only to supply a constant.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely instantiates the even-order involution theorem on the intersection subgroup, using the intersection's even cardinality to obtain the specific involution. It then uses the other parent's normality result to transfer that witness through every conjugation, so both parents materially interact.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2292 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the hypothesis that G is finite.

Goal round-trip: what Lean elaborated

```
theorem even_normal_intersection_has_conjugacy_stable_involution.extracted_1_1.{u_2, u_1} {G : Type u_1}
[inst : Group G] (I : Type u_2) [inst_1 : Nonempty I] (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal)
(hEven : Even (Nat.card (⋂ i, H i))) : ∃ x, orderOf x = 2 ∧ x ∈ ⋂ i, H i ∧ ∀ (g : G), g * x * g-1 ∈ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a nonempty type, and let H assign to each $i \in I$ a subgroup $H(i)$ of G . Suppose every $H(i)$ is a normal subgroup of G , and suppose the cardinality of the intersection $\bigcap_{i \in I} H(i)$ is even. Then there exists an element $x \in G$ such that the order of x is 2, x belongs to $\bigcap_{i \in I} H(i)$, and for every $g \in G$, the conjugate $g x g^{-1}$ also belongs to $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem even_normal_intersection_has_conjugacy_stable_involution {G : Type*} [Group G] [Fintype G]
(I : Type*) [Nonempty I] (H : I → Subgroup G)
(hH : ∀ i : I, (H i).Normal)
(hEven : Even (Nat.card (⋂ i : I, H i))) :
∃ x : G, orderOf x = 2 ∧ x ∈ (⋂ i : I, H i) ∧
  ∀ g : G, g * x * g-1 ∈ (⋂ i : I, H i) := by
  have hNormal : (⋂ i : I, H i).Normal := by
    exact Subgroup.normal_iInf_normal (fun i => hH i)
  let I : Fintype (⋂ i : I, H i) := Fintype.ofFinite _
  have hEvenK : Even (Fintype.card (⋂ i : I, H i)) := by
    simpa only [Nat.card_eq_fintype_card] using hEven
  obtain ⟨k, hk⟩ := hEvenK
  have h_two_dvd : 2 ∣ Fintype.card (⋂ i : I, H i) := by
    exact ⟨k, by simpa [two_mul] using hk⟩
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain ⟨x, hx⟩ : ∃ x : (⋂ i : I, H i), orderOf x = 2 :=
    exists_prime_orderOf_dvd_card 2 h_two_dvd
  refine ⟨(x : G), by simpa using hx, x.property, ?_⟩
  intro g
  exact hNormal.conj_mem (x : G) x.property g
```

Operator. crossover / crossover_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__x__Artin_exercise_2_11_3__xh__79d882c1

Fusion mechanism. witness_exchange Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 79d882c164f0b6f2

4.27 027. order_eleven_totient_bridge

The problem

Let $a > 1$ be an integer, and suppose that $n \mid \phi(a^n - 1)$ for every positive integer n . If a finite group has order 462, show that it contains an element x of order 11 such that $11 \mid \phi(a^{\text{ord}(x)} - 1)$.

Lean statement

```
theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 ∣ (a ^ orderOf x - 1).totient
```

Parent 1. If $a > 1$ is an integer, show that $n \mid \phi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent 1 in Lean

```
theorem Herstein_exercise_2_4_36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

Parent 2. Let G be a finite group of order 462. Prove that G contains an element of order 11.

Parent 2 in Lean

```
theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11
```

What it contributes. The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The group witness supplies an element with order exactly 11, and that equality is used to rewrite the exponent in the totient-divisibility claim obtained from the Herstein hypothesis. The arithmetic conclusion therefore depends on the specific witness rather than merely being conjoined with the independent existence result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The group parent supplies an element with order exactly 11, and that equality is used to rewrite the exponent in the totient-divisibility conclusion obtained from the Herstein family. Both parents are necessary: without the witness there is no order to substitute, and without the arithmetic family there is no divisibility claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The group parent supplies an element with exact order 11, and that equality is used to rewrite the exponent in the totient divisibility conclusion obtained from the Herstein hypothesis. The arithmetic claim is therefore indexed by the constructed witness rather than merely conjoined afterward.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2782 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_eleven_totient_bridge.extracted_1_1.{u_1} {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ (n : ℕ), a > 1 → n | (a ^ n - 1).totient) {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
  (hG : Fintype.card G = 462) : ∃ x, orderOf x = 11 ∧ 11 | (a ^ orderOf x - 1).totient
```

Read back by a model that saw only the Lean. For a natural number a greater than 1, suppose that for every natural number n , if $a > 1$ then n divides the Euler totient of $a^n - 1$. Let G be a finite group with exactly 462 elements. Then there exists an element x of G such that the order of x is exactly 11 and 11 divides the Euler totient of $a^{(\text{order of } x)} - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n | (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 | (a ^ orderOf x - 1).totient := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : 11 | Fintype.card G := by
    rw [hG]
    norm_num
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card 11 hcard
  have hfixed : 11 | (a ^ 11 - 1).totient := hHerstein 11 ha
  have hrewritten : 11 | (a ^ orderOf x - 1).totient := by
    rw [hx]
    exact hfixed
  exact ⟨x, hx, hrewritten⟩
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__x__Dummit_Foote_exercise_4_5_23__mh.xh__590497d8

Fusion mechanism. parameter_coupling The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 590497d834fffd5c

4.28 028. exists_central_involution_of_even_card

The problem

Let every subgroup of a finite group G be normal, and suppose that $|G|$ is even. Show that G contains a nonidentity element x with $x^2 = 1$ that commutes with every element of G .

Lean statement

```
theorem exists_central_involution_of_even_card
  {G : Type*} [Group G] [Fintype G]
  (hG : ∀ H : Subgroup G, H.Normal)
  (heven : Even (Fintype.card G)) :
  ∃ x : G, x ≠ 1 ∧ x * x = 1 ∧ ∀ b : G, b * x = x * b
```

Parent 1. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent 1 in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

What it contributes. Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in $\text{zpowers } x$ for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

Parent 2. Prove that a group of even order contains an element of order 2.

Parent 2 in Lean

```
theorem Artin_exercise_2_11_3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

What it contributes. Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in $\text{zpowers } x$ for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: even cardinality supplies a nontrivial element of order two, while universal subgroup normality puts each conjugate in its cyclic subgroup. Order preservation then rules out the identity power, forcing the conjugate to equal the element and establishing centrality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The order-two witness supplied by the even-cardinality parent is used as the generator of the cyclic subgroup in the normality argument. Its two-element structure then eliminates the identity alternative for each conjugate, yielding centrality; this is substantive witness exchange rather than a final conjunction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The order-two witness supplied by the even-cardinality theorem is instantiated into the conjugation-membership result, and its nontriviality eliminates the identity case, forcing every conjugate to equal the witness and hence proving centrality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2293 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_central_involution_of_even_card.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] [inst_1 : Fintype G]
(hG : ∀ (H : Subgroup G), H.Normal) (heven : Even (Fintype.card G)) :
  ∃ x, x ≠ 1 ∧ x * x = 1 ∧ ∀ (b : G), b * x = x * b
```

Read back by a model that saw only the Lean. *Let G be a finite group such that every subgroup of G is normal, and suppose that the cardinality of G is even. Then there exists an element $x \in G$ such that $x \neq 1$, $x \cdot x = 1$, and for every $b \in G$, $b \cdot x = x \cdot b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem exists_central_involution_of_even_card
  {G : Type*} [Group G] [Fintype G]
  (hG : ∀ H : Subgroup G, H.Normal)
  (heven : Even (Fintype.card G)) :
  ∃ x : G, x ≠ 1 ∧ x * x = 1 ∧ ∀ b : G, b * x = x * b := by
  classical
  rcases heven with ⟨k, hk⟩
  have htwo : 2 ∣ Fintype.card G := by
    refine ⟨k, ?_⟩
    omega
  letI : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain (x, hxorder) := exists_prime_orderOf_dvd_card 2 htwo
  have hxneq : x ≠ 1 := by
    intro hx
    rw [hx] at hxorder
    simp at hxorder
  have hxpow : x * x = 1 := by
    have hdiv : orderOf x ∣ 2 := by simp [hxorder]
    have hp : x ^ (2 : ℕ) = 1 := (orderOf_dvd_iff_pow_eq_one).mp hdiv
    simpa [pow_two] using hp
  refine ⟨x, hxneq, hxpow, ?_⟩
  intro b
  have hnormal : (Subgroup.zpowers x).Normal := hG (Subgroup.zpowers x)
  have hmem : b * x * b⁻¹ ∈ Subgroup.zpowers x := by
    exact hnormal.conj_mem x (Subgroup.mem_zpowers x) b
  have hconj_order : orderOf (b * x * b⁻¹) = 2 := by
    have hsem : SemiconjBy b⁻¹ (b * x * b⁻¹) x := by
      change b⁻¹ * (b * x * b⁻¹) = x * b⁻¹
    group
    calc
      orderOf (b * x * b⁻¹) = orderOf x := SemiconjBy.orderOf_eq b⁻¹ hsem
      _ = 2 := hxorder
  have hfused : b * x * b⁻¹ ∈ Subgroup.zpowers x ∧ orderOf (b * x * b⁻¹) = 2 :=
    ⟨hmem, hconj_order⟩
  obtain (n, hn) := Subgroup.mem_zpowers_iff.mp hfused.1
  have hreduce : x ^ n = x ^ (n % (2 : ℤ)) := by
    have hord : (orderOf x : ℤ) = 2 := by exact_mod_cast hxorder
    rw [← hord]
    exact (zpow_mod_orderOf x n).symm
```

```

have hcases : n % (2 : ℤ) = 0 ∨ n % (2 : ℤ) = 1 := by omega
have hconjne : b * x * b⁻¹ ≠ 1 := by
  intro h
  have ho : orderOf (b * x * b⁻¹) = 1 := by rw [h]; simp
  omega
rcases hcases with hz | ho
· exfalso
  apply hconjne
  rw [← hn, hreduce, hz]
  simp
· have heq : b * x * b⁻¹ = x := by
  rw [← hn, hreduce, ho]
  simp
calc
  b * x = (b * x * b⁻¹) * b := by group
  _ = x * b := by rw [heq]

```

Operator. crossover / crossover_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__297b115a__x__Artin_exercise_2_11_3__xh__405e03c7

Fusion mechanism. witness_exchange Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in z powers x for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 405e03c740bedf1c

4.29 029. restricted_comparison_perfect_countable

The problem

Let s be a closed subset of a second-countable metric space X , and let $f, g : X \rightarrow Y$ be continuous maps into a linearly ordered space with the order topology. Show that $\{x \in s \mid f(x) \leq g(x)\}$ is the union of a perfect set and a countable set.

Lean statement

```
theorem restricted_comparison_perfect_countable {X Y : Type*} [MetricSpace X] [SecondCountableTopology X]
[TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(s : Set X) (f g : X → Y) (hs : IsClosed s)
(hf : Continuous f) (hg : Continuous g) :
∃ P₁ P₂ : Set X,
  {x | x ∈ s ∧ f x ≤ g x} = P₁ ∪ P₂ ∧
  Perfect P₁ ∧ Set.Countable P₂
```

Parent 1. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 1 in Lean

```
theorem Rudin_exercise_2.28 (X : Type*) [MetricSpace X] [SeparableSpace X]
(A : Set X) (hA : IsClosed A) :
∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

Parent 2. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent 2 in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

What it contributes. Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines both parents: continuity and closedness establish that the restricted comparison locus is closed, then the Cantor–Bendixson decomposition is applied to that derived locus. The restriction by s makes this substantively different from decomposing the unrestricted comparison set.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: continuity and closedness of the restricted comparison locus are used to establish that it is closed, and that derived set is then fed into the perfect/countable decomposition theorem. Neither parent's conclusion is merely appended or used as a constant.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: continuity and closedness establish that the restricted comparison locus is closed, and that derived set is then fed into the Cantor–Bendixson decomposition theorem to obtain a perfect part and a countable remainder.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2789 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem restricted_comparison_perfect_countable.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : MetricSpace X] [inst_1 : SecondCountableTopology X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (s : Set X) (f g : X → Y) (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  ∃ P1 P2, {x | x ∈ s ∧ f x ≤ g x} = P1 ∪ P2 ∧ Perfect P1 ∧ P2.Countable
```

Read back by a model that saw only the Lean. *Let X be a metric space with a second-countable topology, and let Y be a topological space equipped with a linear order whose order topology is the given topology. For a subset s of X and continuous functions $f, g : X \rightarrow Y$, if s is closed, then there exist subsets P_1 and P_2 of X such that $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\} = P_1 \cup P_2$, where P_1 is perfect and P_2 is countable.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem restricted_comparison_perfect_countable {X Y : Type*} [MetricSpace X] [SecondCountableTopology X]
  [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (s : Set X) (f g : X → Y) (hs : IsClosed s)
  (hf : Continuous f) (hg : Continuous g) :
  ∃ P1 P2 : Set X,
    {x | x ∈ s ∧ f x ≤ g x} = P1 ∪ P2 ∧
    Perfect P1 ∧ Set.Countable P2 := by
  classical
  have hclosed : IsClosed {x | x ∈ s ∧ f x ≤ g x} := by
    exact hs.isClosed_le hf.continuousOn hg.continuousOn
  obtain ⟨N, P, hN, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X)
    (C := {x | x ∈ s ∧ f x ≤ g x}) hclosed
  exact ⟨P, N, hdecomp.trans (Set.union_comm N P), hP, hN⟩
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Munkres_exercise_18_8a__me.xh__d2849caf

Fusion mechanism. invariant_transplant Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d2849caffd5c506c

4.30 030. finite_iterated_neg_sum_transport

The problem

Let V be an additive commutative group, let f be a finite family of vectors, and let n be a non-negative integer. Show that the sum of the vectors obtained by applying negation n times to each $f(i)$ equals the original sum when n is even and its negative when n is odd.

Lean statement

```
theorem finite_iterated_neg_sum_transport {ι V : Type*} [Fintype ι] [AddCommGroup V]
  (n : ℕ) (f : ι → V) :
  Finset.univ.sum (fun i => Nat.rec (fun x : V => x)
    (fun _ g x => -g x) n (f i)) =
  if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the additive-group double-negation rewrite used in the parity induction step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes double negation to an arbitrary finite sum with n -fold negation, requiring induction on n , parity reasoning, and sum-negation transport. A solver recalling the parent's one-line 'simp' proof would not substantially solve this theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes double negation to an arbitrary iterated-negation parity law and transports it through a finite sum. Its proof requires induction and parity case analysis, which are not supplied by the parent's one-step simp proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes double negation to an arbitrary iterated-negation parity law and transports it through a finite sum. Its proof requires induction on the iteration count and a separate finite-sum negation argument, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2841 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_iterated_neg_sum_transport.extracted_1.1.{u_2, u_1} {ι : Type u_1} {V : Type u_2} [inst : Fintype ι]
  [inst.1 : AddCommGroup V] (n : ℕ) (f : ι → V) :
  ∑ i, Nat.rec (motive := fun x => V → V) (fun x => x) (fun x g x_1 => -g x_1) n (f i) =
  if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f
```

Read back by a model that saw only the Lean. For any finite type ι , any additive commutative group V , any natural number n , and any function $f : \iota \rightarrow V$, the sum over all $i \in \iota$ of the result of applying negation to $f(i)$ exactly n times equals the sum of f over ι if n is even, and equals the negation of that sum if n is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem finite_iterated_neg_sum_transport {ι V : Type*} [Fintype ι] [AddCommGroup V]
  (n : ℕ) (f : ι → V) :
  Finset.univ.sum (fun i => Nat.rec (fun x : V => x)
    (fun _ g x => -g x) n (f i)) =
  if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f := by
  classical
  have hiter : ∀ k : ℕ, ∀ x : V,
    Nat.rec (fun y : V => y) (fun _ g y => -g y) k x =
    if k % 2 = 0 then x else -x := by
    intro k
    induction k with
    | zero =>
      intro x
      simp
    | succ k ih =>
      intro x
      by_cases hk : k % 2 = 0
      · have hk' : (k + 1) % 2 ≠ 0 := by omega
        simp [ih, hk, hk']
      · have hk' : (k + 1) % 2 = 0 := by omega
        simp [ih, hk, hk']
  simp_rw [hiter n]
  split_ifs with hn
  · rfl
  · exact Finset.sum_neg_distrib f
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__me__171fb14c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 171fb14c01359541

4.31 031. submodule_neg_mem_iff

The problem

Prove that $-v \in U$ if and only if $v \in U$ for every submodule U of a vector space V over a field F and every $v \in V$.

Lean statement

```
theorem submodule_neg_mem_iff {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U := by
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the double-negation involution checkpoint used in the forward membership transport.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new submodule-membership equivalence, requiring both closure under negation and transport through double negation. The parent's identity is only one ingredient and does not supply the equivalence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a pointwise involution into a submodule-membership equivalence, requiring the genuinely new closure lemma 'U.neg_mem' and a two-direction proof. Although it uses double-negation, recalling the parent alone does not establish transport through submodule membership.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an equality into a submodule-membership equivalence and requires the new structural fact 'U.neg_mem' in both directions, using involutivity only as one component of the transport argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1928 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem submodule_neg_mem_iff.extracted_1_1.{u_2, u_1} {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U
```

Read back by a model that saw only the Lean. *Let F be a field, let V be an F -module that is also an additive commutative group, let U be a submodule of V over F , and let v be an element of V . Then the additive inverse $-v$ belongs to U if and only if v belongs to U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem submodule_neg_mem_iff {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U := by
  have h_invol : -(-v) = v := by
    simp
  constructor
  · intro hv
    have hdouble : -(-v) ∈ U := U.neg_mem hv
    rw [h_invol] at hdouble
    exact hdouble
  · intro hv
    exact U.neg_mem hv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__me__ba71430c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomsproptext, Quot.sound, fingerprint 69453865196dd89e

4.32 032. invariant_preimage_iInf_submodule

The problem

Let F be a field, let V and W be F -modules, and let $f : V \rightarrow W$, $h : V \rightarrow V$, and $g : W \rightarrow W$ be linear maps satisfying $f \circ h = g \circ f$. Given a family of submodules $(u_i)_{i \in I}$ of W such that $g(y) \in u_i$ whenever $y \in u_i$, there exists a submodule U of V equal to the preimage under f of $\bigcap_{i \in I} u_i$ such that $h(x) \in U$ for every $x \in U$.

Lean statement

```
theorem invariant_preimage_iInf_submodule {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W] (f : V → W) (h : V → V) (g : W → W) (u : ι → Submodule F W) (hcomm : f.comp h = g.comp f) (hpres : ∀ i y, y ∈ u i → g y ∈ u i) :
  ∃ U : Submodule F V, U = (⋂ i, u i).comap f ∧ ∀ x, x ∈ U → h x ∈ U
```

Parent. For a linear map, the preimage of an indexed intersection of submodules is a submodule, and membership in it is equivalent to satisfying every indexed constraint after applying the map.

Parent in Lean

```
theorem preimage_iInf_submodule_membership {F V W : Type*} [AddCommGroup V] [AddCommGroup W] [Field F] [Module F V] [Module F W] {ι : Type*} (f : V → W) (u : ι → Submodule F W) : ∃ U : Submodule F V, ∀ x : V, (x ∈ U ↔ f x ∈ (⋂ i, u i)) ∧ (x ∈ U ↔ ∀ i, f x ∈ u i)
```

What it contributes. Supplied the indexed infimum and linear-map comap construction and its membership viewpoint; the mutation adds a commuting endomorphism action and proves induced invariance.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine invariant-submodule argument: it transports membership through the commuting endomorphisms and applies preservation to every component of the infimum. The parent's preimage construction alone does not supply this closure proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-submodule claim: preservation of each u_i under g is transported through the commutation relation to preservation of the preimage under h . The parent's construction supplies only membership equivalences and does not establish this closure property.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-submodule conclusion: preservation of each u_i is transported through the commutation relation to show h preserves the preimage. This requires new reasoning beyond the parent's membership equivalences, despite reusing the same comap witness.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1939 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem invariant_preimage_iInf_submodule.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  {W : Type u_3} {ι : Type u_4} [inst : Field F] [inst_1 : AddCommGroup V] [inst_2 : AddCommGroup W]
  [inst_3 : Module F V] [inst_4 : Module F W] (f : V →ι[F] W) (h : V →ι[F] V) (g : W →ι[F] W) (u : ι → Submodule F W)
  (hcomm : f ∘ι h = g ∘ι f) (hpres : ∀ (i : ι), ∀ y ∈ u i, g y ∈ u i) :
  ∃ U, U = Submodule.comap f (⋂ i, u i) ∧ ∀ x ∈ U, h x ∈ U
```

Read back by a model that saw only the Lean. Let F be a field, let V and W be additive commutative groups equipped with F -module structures, let ι be a type, let $f : V \rightarrow W$, $h : V \rightarrow V$, and $g : W \rightarrow W$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of W . Assume that $f \circ h = g \circ f$, and that for every $i \in \iota$ and every $y \in u(i)$, we have $g(y) \in u(i)$. Then there exists a submodule U of V such that $U = \{x \in V \mid f(x) \in \bigcap_{i \in \iota} u(i)\}$ and, for every $x \in U$, $h(x) \in U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem invariant_preimage_iInf_submodule {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W] (f : V
  →ι[F] W) (h : V →ι[F] V) (g : W →ι[F] W) (u : ι → Submodule F W) (hcomm : f.comp h = g.comp f) (hpres : ∀ i y, y ∈ u i → g y ∈ u i) :
  ∃ U : Submodule F V, U = (⋂ i, u i).comap f ∧ ∀ x, x ∈ U → h x ∈ U := by
  refine ⟨(⋂ i, u i).comap f, rfl, ?_⟩
  intro x hx
  change f (h x) ∈ (⋂ i, u i)
  have hmem : f x ∈ (⋂ i, u i) := hx
  have hmem_all : ∀ i, f x ∈ u i := (Submodule.mem_iInf u).mp hmem
  have htransport : f (h x) = g (f x) := by
    have heval := congrArg (fun k : V →ι[F] W => k x) hcomm
    simpa using heval
  rw [htransport]
  apply (Submodule.mem_iInf u).mpr
  intro i
  exact hpres i (f x) (hmem_all i)
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__2e29dde1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 732c4a74bab5e882

4.33 033. finite_invariant_intersection_common_submodule

The problem

Let a finite nonempty family of subspaces be invariant under two bounded commuting linear maps. Show that their intersection is invariant under both maps, preserves their commutation relation, and is the greatest subspace contained in every member.

Lean statement

```
theorem finite_invariant_intersection_common_submodule {F V ι : Type*} [NormedField F] [NormedAddCommGroup V] [NormedSpace F V] [Fintype
  ι] [Nonempty ι] (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) (hg : ∀ i : ι, ∀ x : V, x ∈ u
  i → g x ∈ u i) (hfg : ∀ x : V, f (g x) = g (f x)) : ∃ U : Submodule F V, (∩ i : ι, (u i : Set V)) = (U : Set V) ∧ (∀ x : V, x ∈ U → f
  x ∈ U) ∧ (∀ x : V, x ∈ U → g x ∈ U) ∧ (∀ x : U, f (g (x : V)) = g (f (x : V))) ∧ (∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i)
```

Parent. Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Parent in Lean

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (∩ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

What it contributes. Supplied the indexed-infimum construction, intersection carrier equality, and greatest-lower-bound structure; the mutation adds invariance and commuting-map obligations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds genuinely new mathematics: proving the intersection is invariant under both continuous linear maps and inherits their commutation, using the family-wise invariance hypotheses. Although the greatest-lower-bound conjunct reuses the parent's infimum argument, the parent proof does not supply these preservation arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new reasoning to prove that the intersection inherits invariance under both endomorphisms, in addition to lifting the commuting relation to the restricted submodule. The parent's lattice-theoretic intersection proof alone does not supply these arguments.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: proving the intersection is invariant under both continuous endomorphisms and inherits their commutation relation. The parent's lattice-theoretic intersection proof does not supply these closure arguments, so recalling it is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2950 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_invariant_intersection_common_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  {ι : Type u_3} [inst : NormedField F] [inst_1 : NormedAddCommGroup V] [inst_2 : NormedSpace F V] [inst_3 : Nonempty ι]
  (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i)
  (hg : ∀ (i : ι), ∀ x ∈ u i, g x ∈ u i) (hfg : ∀ (x : V), f (g x) = g (f x)) :
  ∃ U,
    ∩ i, ↑(u i) = ↑U ∧
      (∀ x ∈ U, f x ∈ U) ∧
      (∀ x ∈ U, g x ∈ U) ∧
      (∀ (x : ιU), f (g ↑x) = g (f ↑x)) ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. Let F be a normed field, let V be a normed additive commutative group and a normed vector space over F , and let ι be a nonempty type. Given a family $u : \iota \rightarrow \text{Submodule } F V$ and continuous linear maps $f, g : V \rightarrow V$ such that, for every $i \in \iota$ and every $x \in u i$, $f(x) \in u i$ and $g(x) \in u i$, and such that $f(g(x)) = g(f(x))$ for every $x \in V$, there exists a submodule U of V over F satisfying: the underlying set of U is exactly the intersection of the underlying sets of all $u i$; f maps every element of U into U ; g maps every element of U into U ; for every $x \in U$, $f(g(x)) = g(f(x))$; and, for every submodule W of V over F , $W \leq U$ if and only if $W \leq u i$ for every $i \in \iota$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem finite_invariant_intersection_common_submodule {F V ι : Type*} [NormedField F] [NormedAddCommGroup V] [NormedSpace F V] [Fintype
  ι] [Nonempty ι] (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) (hg : ∀ i : ι, ∀ x : V, x ∈ u i →
  g x ∈ u i) (hfg : ∀ x : V, f (g x) = g (f x)) : ∃ U : Submodule F V, (∩ i : ι, (u i : Set V)) = (U : Set V) ∧ (∀ x : V, x ∈ U →
  f x ∈ U) ∧ (∀ x : V, x ∈ U → g x ∈ U) ∧ (∀ x : U, f (g (x : V)) = g (f (x : V))) ∧ (∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i)
  := by
  classical
  refine (∩ i, u i, (Submodule.coe_iInf u).symm, ?_, ?_, ?_, ?_)
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := by
      simpa only [Submodule.mem_iInf] using hx
    have hfx_all : ∀ i : ι, f x ∈ u i := by
      intro i
      exact hf i x (hx_all i)
    simpa only [Submodule.mem_iInf] using hfx_all
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := by
      simpa only [Submodule.mem_iInf] using hx
    have hgx_all : ∀ i : ι, g x ∈ u i := by
      intro i
      exact hg i x (hx_all i)
    simpa only [Submodule.mem_iInf] using hgx_all
  · intro x
    exact hfg (x : V)
  · intro W
    constructor
    · intro h i
      exact h.trans (iInf_le u i)
    · intro h
      exact le_iInf h
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__58cb856c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-

gerprint 58cb856c8665df89

4.34 034. composed_pullback_iInf_membership

The problem

Let F be a field, let V , W , and X be additive commutative groups equipped with F -module structures, let ι be an index type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be linear maps, and let $(U_i)_{i \in \iota}$ be a family of submodules of X . Then the preimage under f of the preimage under g of $\bigcap_{i \in \iota} U_i$ equals the preimage under $g \circ f$ of $\bigcap_{i \in \iota} U_i$, and for every $x \in V$, membership in this successive preimage is equivalent to $g(f(x)) \in U_i$ for every $i \in \iota$.

Lean statement

```
theorem composed_pullback_iInf_membership {F V W X : Type*} [AddCommGroup V] [AddCommGroup W] [AddCommGroup X] [Field F] [Module F V]
  [Module F W] [Module F X] {ι : Type*} (f : V → [F] W) (g : W → [F] X) (u : ι → Submodule F X) : ((⋂ i, u i).comap g).comap f = (⋂ i,
  u i).comap (g.comp f) ∧ ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ ∀ i, g (f x) ∈ u i
```

Parent. For a linear map, the preimage of an indexed intersection of submodules is a submodule, and membership in it is equivalent to satisfying every indexed constraint after applying the map.

Parent in Lean

```
theorem preimage_iInf_submodule_membership {F V W : Type*} [AddCommGroup V] [AddCommGroup W] [Field F] [Module F V] [Module F W] {ι :
  Type*} (f : V → [F] W) (u : ι → Submodule F W) : ∃ U : Submodule F V, ∀ x : V, (x ∈ U ↔ f x ∈ (⋂ i, u i)) ∧ (x ∈ U ↔ ∀ i, f x ∈ u i)
```

What it contributes. Supplied the indexed-infimum membership checkpoint and comap construction, extended here by explicitly composing two pullbacks.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely new composition law: it identifies successive submodule pullbacks with pullback along ‘ $g \circ f$ ’, then derives the indexed membership characterization through ‘ $\text{Submodule.mem_iInf}$ ’. This requires reasoning about linear-map composition in addition to the parent’s infimum-membership argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a genuine two-map pullback composition and proves comap associativity by extensionality, while separately using $\text{Submodule.mem_iInf}$ in both directions for the membership law. This is not obtainable by recalling the parent’s existential witness proof, and it follows the planned structure expansion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential membership statement into a two-map pullback composition law plus an explicit indexed membership characterization. Proving comap composition by extensionality and translating the infimum through $\text{Submodule.mem_iInf}$ requires reasoning not supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1929 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem composed_pullback_iInf_membership.extracted_1_1.{u_5, u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{W : Type u_3} {X : Type u_4} [inst : AddCommGroup V] [inst_1 : AddCommGroup W] [inst_2 : AddCommGroup X]
[inst_3 : Field F] [inst_4 : Module F V] [inst_5 : Module F W] [inst_6 : Module F X] {ι : Type u_5} (f : V →ι[F] W)
(g : W →ι[F] X) (u : ι → Submodule F X) :
Submodule.comap f (Submodule.comap g (⋂ i, u i)) = Submodule.comap (g ∘ι f) (⋂ i, u i) ∧
∀ (x : V), x ∈ Submodule.comap f (Submodule.comap g (⋂ i, u i)) ↔ ∀ (i : ι), g (f x) ∈ u i
```

Read back by a model that saw only the Lean. Let F be a field, let V, W , and X be additive commutative groups that are F -modules, let ι be a type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of X over F . Then the preimage under f of the preimage under g of the intersection of all the submodules $u(i)$ equals the preimage under the composite linear map $g \circ f$ of that intersection, and for every $x \in V$, x belongs to the preimage under f of the preimage under g of the intersection of all $u(i)$ if and only if, for every $i \in \iota$, $g(f(x))$ belongs to $u(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem composed_pullback_iInf_membership {F V W X : Type*} [AddCommGroup V] [AddCommGroup W] [AddCommGroup X] [Field F] [Module F V]
↔ [Module F W] [Module F X] {ι : Type*} (f : V →ι[F] W) (g : W →ι[F] X) (u : ι → Submodule F X) : ((⋂ i, u i).comap g).comap f = (⋂ i,
↔ u i).comap (g.comp f) ∧ ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ ∀ i, g (f x) ∈ u i := by
have h_iInf : ∀ y : X, y ∈ (⋂ i, u i) ↔ ∀ i, y ∈ u i := by
  intro y
  exact Submodule.mem_iInf u
have h_pullback : ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ g (f x) ∈ (⋂ i, u i) := by
  intro x
  rfl
have h_composed : ∀ x : V, x ∈ (⋂ i, u i).comap (g.comp f) ↔ g (f x) ∈ (⋂ i, u i) := by
  intro x
  simp [LinearMap.comp_apply]
constructor
· ext x
  constructor
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_pullback x).mp hx
    have hall : ∀ i, g (f x) ∈ u i := (h_iInf (g (f x))).mp hmid
    exact (h_composed x).mpr ((h_iInf (g (f x))).mpr hall)
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_composed x).mp hx
    exact (h_pullback x).mpr hmid
· intro x
  constructor
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_pullback x).mp hx
    exact (h_iInf (g (f x))).mp hmid
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_iInf (g (f x))).mpr hx
    exact (h_pullback x).mpr hmid
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__cb976168

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ce29fd684843f3f5

4.35 035. intersection_is_unique_greatest_invariant_submodule

The problem

Let a linear endomorphism preserve every subspace in a family. Prove that their intersection is the unique greatest invariant subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_unique_greatest_invariant_submodule {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (f : V
  → V) (u : ι → Submodule F V) (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) : ∃! U : Submodule F V, (∀ x : V, x ∈ U ↔ ∀ i : ι, x
  ∈ u i) ∧ (∀ x : V, x ∈ U → f x ∈ U) ∧ (∀ W : Submodule F V, (∀ x : V, x ∈ W → f x ∈ W) → (∀ i : ι, W ≤ u i) → W ≤ U)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ (i : ι), (u i).carrier) = ↑U := by
```

What it contributes. Uses the parent’s canonical indexed-infimum construction and its pointwise membership characterization, then strengthens the conclusion with invariance and the greatest-lower-bound universal property.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes mere existence of an intersection submodule into a unique characterization as the greatest f -invariant submodule below every family member. Its proof must establish invariance and the universal maximality property using the preservation hypothesis, which the parent’s proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential intersection statement into a uniqueness and maximality characterization, while additionally proving invariance under a linear endomorphism using the preservation hypothesis. This requires reasoning absent from the parent and is not recoverable by recalling its proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-subspace characterization: preservation under the endomorphism, greatestness among all invariant subspaces below every family member, and uniqueness. The parent supplies only existence of the intersection submodule and does not establish these new universal properties.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2190 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_unique_greatest_invariant_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (f : V →ι[F] V)
  (u : ι → Submodule F V) (hpres : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
  ∃! U,
    (∀ (x : V), x ∈ U ↔ ∀ (i : ι), x ∈ u i) ∧
    (∀ x ∈ U, f x ∈ U) ∧ ∀ (W : Submodule F V), (∀ x ∈ W, f x ∈ W) → (∀ (i : ι), W ≤ u i) → W ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, V an additive commutative group and an F -module, ι a type, $f: V \rightarrow V$ an F -linear map, and u a family of F -submodules of V indexed by ι . Assume that for every $i \in \iota$ and every $x \in u(i)$, we have $f(x) \in u(i)$. Then there exists a unique F -submodule U of V such that: for every $x \in V$, $x \in U$ if and only if x belongs to $u(i)$ for every $i \in \iota$; U is f -invariant, meaning that for every $x \in U$, $f(x) \in U$; and for every F -submodule W of V , if W is f -invariant and $W \leq u(i)$ for every $i \in \iota$, then $W \leq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_unique_greatest_invariant_submodule {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (f : V
  →ι[F] V) (u : ι → Submodule F V) (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) : ∃! U : Submodule F V, (∀ x : V, x ∈ U ↔ ∀ i : ι, x
  ∈ u i) ∧ (∀ x : V, x ∈ U → f x ∈ U) ∧ (∀ W : Submodule F V, (∀ x : V, x ∈ W → f x ∈ W) → (∀ i : ι, W ≤ u i) → W ≤ U) := by
  classical
  refine (⟨∏ i, u i, ?_, ?_⟩)
  · refine (⟨?_, ?_, ?_⟩)
  · intro x
    constructor
    · intro hx
      exact (Submodule.mem_iInf u).mp hx
    · intro hx
      exact (Submodule.mem_iInf u).mpr hx
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := (Submodule.mem_iInf u).mp hx
    have hfx_all : ∀ i : ι, f x ∈ u i := by
      intro i
      exact hpres i x (hx_all i)
    exact (Submodule.mem_iInf u).mpr hfx_all
  · intro W hW hle x hx
    exact (Submodule.mem_iInf u).mpr (fun i => hle i hx)
  · intro W hW
    apply Submodule.ext
    intro x
    have hxi : x ∈ (∏ i, u i) ↔ ∀ i : ι, x ∈ u i := by
      constructor
      · intro hx
        exact (Submodule.mem_iInf u).mp hx
      · intro hx
        exact (Submodule.mem_iInf u).mpr hx
    exact (hW.1 x).trans hxi.symm
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__244392f2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 244392f2e34dd943

4.36 036. kernel_of_linear_map_family

The problem

Given a family of linear maps, construct one map whose kernel consists exactly of the vectors sent to zero by every map in the family. Show that this kernel is the intersection of the individual kernels.

Lean statement

```
theorem kernel_of_linear_map_family {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W]
  (f : ι → V →F W) :
  ∃ g : V →F W, (ι → W),
  LinearMap.ker g = ⋂ i, LinearMap.ker (f i) ∧
  ∀ x : V, x ∈ LinearMap.ker g ↔ ∀ i, f i x = 0 := by
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child introduces a family of linear maps, constructs the bundled product map, and proves both the kernel intersection equality and the pointwise common-zero characterization. This requires new kernel and function-extensionality reasoning rather than recalling the parent's generic intersection packaging.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a bundled product-valued linear map and proves both its kernel intersection characterization and the pointwise common-zero condition. This requires new reasoning about function extensionality and pointwise evaluation, rather than recalling the parent's existence proof for an arbitrary intersection submodule.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a bundled linear map into the function space, proves its kernel equals the indexed infimum of the individual kernels, and establishes the pointwise common-zero characterization. This requires new kernel and pointwise-function reasoning rather than recalling the parent's canonical-intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2246 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem kernel_of_linear_map_family.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2} {W : Type u_3}
  {ι : Type u_4} [inst : Field F] [inst_1 : AddCommGroup V] [inst_2 : AddCommGroup W] [inst_3 : Module F V]
  [inst_4 : Module F W] (f : ι → V →ι[F] W) :
  ∃ g, g.ker = ⋂ i, (f i).ker ∧ ∀ (x : V), x ∈ g.ker ↔ ∀ (i : ι), (f i) x = 0
```

Read back by a model that saw only the Lean. Let F be a field, let V and W be additive commutative groups equipped with F -module structures, let ι be a type, and let f assign to each $i \in \iota$ an F -linear map from V to W . Then there exists an F -linear map g such that the kernel of g equals the intersection $\bigcap_{i \in \iota} \ker(f i)$, and for every $x \in V$, x belongs to $\ker(g)$ if and only if $f i(x) = 0$ for every $i \in \iota$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem kernel_of_linear_map_family {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W]
  (f : ι → V →ι[F] W) :
  ∃ g : V →ι[F] (ι → W),
    LinearMap.ker g = ⋂ i, LinearMap.ker (f i) ∧
      ∀ x : V, x ∈ LinearMap.ker g ↔ ∀ i, f i x = 0 := by
let g : V →ι[F] (ι → W) := LinearMap.pi f
refine ⟨g, ?_, ?_⟩
· ext x
  constructor
  · intro hx
    change g x = 0 at hx
    apply (Submodule.mem_iInf (fun i => LinearMap.ker (f i))).2
    intro i
    change f i x = 0
    have hi := congrFun hx i
    simpa [g] using hi
  · intro hx
    have hxi : ∀ i, x ∈ LinearMap.ker (f i) :=
      (Submodule.mem_iInf (fun i => LinearMap.ker (f i))).1 hx
    change g x = 0
    apply _root_.funext
    intro i
    have hi := hxi i
    change f i x = 0 at hi
    simpa [g] using hi
· intro x
  constructor
  · intro hx
    change g x = 0 at hx
    intro i
    have hi := congrFun hx i
    simpa [g] using hi
  · intro hx
    change g x = 0
    apply _root_.funext
    intro i
    simpa [g] using hx i
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__85841fac

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 85841fac89107b3b

4.37 037. simultaneous_restriction_domain

The problem

Let a finite nonempty family of subspaces be preserved by every map in a finite commuting family of linear endomorphisms. Show that their intersection is a common domain on which all maps restrict simultaneously and still commute.

Lean statement

```
theorem simultaneous_restriction_domain {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {ι κ : Type*} [Fintype ι] [Nonempty ι] [Fintype κ]
  (u : ι → Submodule F V) (T : κ → V →[F] V)
  (hpres : ∀ k i x, x ∈ u i → T k x ∈ u i)
  (hcomm : ∀ k l x, T k (T l x) = T l (T k x)) :
  ∃ U : Submodule F V,
    (↑U : Set V) = ⋂ i, (u i).carrier ∧
    ∃ r : κ → U →[F] U,
      (∀ k x, (r k x : V) = T k (x : V)) ∧
      (∀ k l x, r k (r l x) = r l (r k x))
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ (i : ι), (u i).carrier) = ↑U := by
```

What it contributes. Supplies the indexed-infimum realization of the set-theoretic intersection and the pointwise membership proof pattern.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child constructs the infimum intersection, proves it is preserved by every operator, and then builds simultaneous restricted endomorphisms whose commutation requires transporting the parent commutation law through subtypes. This is not recoverable by recalling the parent's bare intersection-existence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive mathematics beyond the parent: it proves the infimum intersection is invariant under every endomorphism, constructs simultaneous restricted endomorphisms, and establishes their commutativity. These obligations are not supplied by the parent's mere existence proof of the intersection submodule.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child extends mere existence of an intersection submodule to proving that the intersection is preserved by every endomorphism, constructing simultaneous restricted maps, and transferring commutativity. These restriction and commutation arguments are new mathematics not supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2240 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem simultaneous_restriction_domain.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} {κ : Type u_4} [inst_3 : Nonempty ι]
  (u : ι → Submodule F V) (T : κ → V →ι[F] V) (hpres : ∀ (k : κ) (i : ι), ∀ x ∈ u i, (T k) x ∈ u i)
  (hcomm : ∀ (k l : κ) (x : V), (T k) ((T l) x) = (T l) ((T k) x)) :
  ∃ U,
    ↑U = ⋂ i, (u i).carrier ∧
    ∃ r,
      (∀ (k : κ) (x : ιU), ↑((r k) x) = (T k) ↑x) ∧ ∀ (k l : κ) (x : ιU), (r k) ((r l) x) = (r l) ((r k) x)
```

Read back by a model that saw only the Lean. Let F be a field and V an additive commutative group that is an F -module. Let ι and κ be types, with ι nonempty. Suppose u assigns to each $i \in \iota$ a submodule $u(i)$ of V , and T assigns to each $k \in \kappa$ an F -linear endomorphism $T(k)$ of V . Assume that for every $k \in \kappa$, $i \in \iota$, and $x \in u(i)$, we have $T(k)(x) \in u(i)$, and assume that all the endomorphisms $T(k)$ commute pairwise: for every $k, l \in \kappa$ and $x \in V$, $T(k)(T(l)(x)) = T(l)(T(k)(x))$. Then there exists a submodule U of V whose underlying set is exactly the intersection $\bigcap_i u(i)$, and there exists a family r assigning to each $k \in \kappa$ an F -linear endomorphism $r(k)$ of U such that, for every $k \in \kappa$ and $x \in U$, the underlying element of $r(k)(x)$ equals $T(k)$ applied to the underlying element of x , and such that the endomorphisms $r(k)$ commute pairwise on U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem simultaneous_restriction_domain {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {ι κ : Type*} [Fintype ι] [Nonempty ι] [Fintype κ]
  (u : ι → Submodule F V) (T : κ → V →ι[F] V)
  (hpres : ∀ k i x, x ∈ u i → T k x ∈ u i)
  (hcomm : ∀ k l x, T k (T l x) = T l (T k x)) :
  ∃ U : Submodule F V,
    (↑U : Set V) = ⋂ i, (u i).carrier ∧
    ∃ r : κ → U →ι[F] U,
      (∀ k x, (r k x : V) = T k (x : V)) ∧
      (∀ k l x, r k (r l x) = r l (r k x)) := by
  classical
  let U : Submodule F V := ⋂ i, u i
  have hintersection : (↑U : Set V) = ⋂ i, (u i).carrier := by
    simpa [U] using (Submodule.coe_iInf u)
  have hmem : ∀ k x, x ∈ U → T k x ∈ U := by
    intro k x hx
    change T k x ∈ ⋂ i, u i
    apply (Submodule.mem_iInf u).2
    intro i
    apply hpres k i x
    exact (Submodule.mem_iInf u).1 (by simpa [U] using hx) i
  let r : κ → U →ι[F] U := fun k =>
    { toFun := fun x => ⟨T k (x : V), hmem k (x : V) x.property⟩
      map_add' := by
        intro x y
        apply Subtype.ext
        simp
      map_smul' := by
        intro c x
        apply Subtype.ext
        simp }
  }
```

```
refine (U, hintersection, r, ?_, ?_)
· intro k x
  rfl
· intro k l x
  apply Subtype.ext
  simpa [r] using hcomm k l (x : V)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__966dac6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 966dac6ffbe4624b

4.38 038. intersection_is_greatest_invariant_submodule

The problem

Let a family of subspaces of a vector space be preserved by a common linear endomorphism. Prove that their intersection is the greatest subspace preserved by that endomorphism and contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_invariant_submodule {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  (u : ι → Submodule F V) (f : V →1[F] V)
  (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (↑U : Set V) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    (∀ T : Submodule F V,
      (∀ x : V, x ∈ T → f x ∈ T) →
      (∀ i : ι, T ≤ u i) →
      T ≤ U ∧ ∀ x : V, x ∈ T → f x ∈ T)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Supplies the indexed-intersection-as-infimum construction and carrier characterization; the child extends it with a common endomorphism and a greatest invariant-submodule property.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine invariant-submodule argument: it proves the intersection is preserved by the linear map and characterizes it as containing every invariant submodule contained in each member of the family. The parent only establishes existence of the intersection submodule, so its proof does not supply the new preservation reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential intersection statement into a greatest-invariant-submodule characterization. It requires new reasoning that the common preservation hypothesis makes the intersection invariant and that every invariant submodule contained in each $u\ i$ lies below it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: proving the intersection is invariant under a common linear map and is greatest among invariant submodules contained in every $u\ i$. The parent proof only establishes existence and equality with the intersection, so recalling it does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_invariant_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{ι : Type u_3} [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] (u : ι → Submodule F V)
(f : V →ι[F] V) (hpres : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
  ∃ U,
    ∩ i, (u i).carrier = ↑U ∧
      (∀ x ∈ U, f x ∈ U) ∧
      ∀ (T : Submodule F V), (∀ x ∈ T, f x ∈ T) → (∀ (i : ι), T ≤ u i) → T ≤ U ∧ ∀ x ∈ T, f x ∈ T
```

Read back by a model that saw only the Lean. *Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, let u assign to each $i : \iota$ a submodule $u\ i$ of V , and let $f : V \rightarrow V$ be an F -linear map. Assume that for every $i : \iota$ and every $x \in u\ i$, we have $f(x) \in u\ i$. Then there exists a submodule U of V such that the underlying set of U is exactly the intersection $\bigcap i : \iota, u\ i$, U is f -invariant (that is, for every $x \in U$, $f(x) \in U$), and for every submodule T of V , if T is f -invariant and $T \leq u\ i$ for every $i : \iota$, then $T \leq U$ and T is f -invariant.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem intersection_is_greatest_invariant_submodule {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
(u : ι → Submodule F V) (f : V →ι[F] V)
(hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∩ i : ι, (u i).carrier) = (↑U : Set V) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    (∀ T : Submodule F V,
      (∀ x : V, x ∈ T → f x ∈ T) →
      (∀ i : ι, T ≤ u i) →
      T ≤ U ∧ ∀ x : V, x ∈ T → f x ∈ T) := by
  classical
  refine ⟨∩ i, u i, ?_, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := (Submodule.mem_iInf u).mp hx
    have hfx_all : ∀ i : ι, f x ∈ u i := by
      intro i
      exact hpres i x (hx_all i)
    exact (Submodule.mem_iInf u).mpr hfx_all
  · intro T hT hTcontained
    constructor
    · intro x hx
      exact (Submodule.mem_iInf u).mpr (fun i => hTcontained i hx)
    · exact hT
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__9f421d1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 9f421d1d63973b48

4.39 039. intersection_is_greatest_lower_bound

The problem

For an arbitrary family of subspaces, a vector belongs to their intersection exactly when it belongs to every member. Moreover, the intersection is the greatest subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  (∀ x : V, x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i) ∧
  (∀ W : Submodule F V, W ≤ ⋂ i, u i ↔ ∀ i, W ≤ u i)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i, u i).carrier = U := by
```

What it contributes. Uses the parent's indexed-intersection domain and membership reasoning, strengthening its existential carrier representation to a greatest-lower-bound characterization in submodule order.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the existential carrier-equality claim into a two-part characterization of the indexed infimum: pointwise membership and the universal greatest-lower-bound property. Its proof uses the submodule infimum APIs directly and is not obtained by recalling or applying the parent's existential construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a submodule representing the indexed intersection into a full greatest-lower-bound characterization, adding the nontrivial universal containment statement for arbitrary submodules. Its proof requires order-theoretic reasoning via 'le_trans' and 'le_iInf', so recalling the parent's witness-and-extensionality proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential carrier-equality claim into the full greatest-lower-bound characterization: both pointwise membership and submodule-order universality must be established. This is not recoverable by merely recalling the parent's witness-and-extensionality proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2243 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  (∀ (x : V), x ∈ ⋂ i, u i ↔ ∀ (i : ι), x ∈ u i) ∧ ∀ (W : Submodule F V), W ≤ ⋂ i, u i ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. *Let F be a field, V an additive commutative group and an F -module, let ι be a type, and let u assign to each $i : \iota$ a submodule $u(i)$ of V over F . Then (1) for every $x \in V$, x belongs to the intersection $\bigcap_{i : \iota} u(i)$ if and only if x belongs to $u(i)$ for every $i : \iota$; and (2) for every submodule W of V over F , W is contained in $\bigcap_{i : \iota} u(i)$ if and only if W is contained in $u(i)$ for every $i : \iota$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  (∀ x : V, x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i) ∧
  (∀ W : Submodule F V, W ≤ ⋂ i, u i ↔ ∀ i, W ≤ u i) := by
  constructor
  · intro x
    have h_membership : x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i := Submodule.mem_iInf u
    exact h_membership
  · intro W
    constructor
    · intro h i
      have h_component : W ≤ u i := le_trans h (iInf_le u i)
      exact h_component
    · intro h
      have h_lower_bound : ∀ i, W ≤ u i := h
      exact le_iInf h_lower_bound
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__b2253c94

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b2253c940d4a2c8c

4.40 040. intersection_linear_combination

The problem

Prove that any linear combination of two vectors in the intersection of a collection of subspaces again lies in that intersection.

Lean statement

```
theorem intersection_linear_combination {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) (a b : F) (x y : V)
  (hx : x ∈ (⋂ i : ι, (u i : Set V)))
  (hy : y ∈ (⋂ i : ι, (u i : Set V))) :
  a • x + b • y ∈ (⋂ i : ι, (u i : Set V))
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Uses the parent's indexed-intersection checkpoint: membership in the intersection is converted to membership in every subspace, where linear-combination closure is applied before rebuilding intersection membership.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new closure property: the intersection is closed under arbitrary linear combinations. The parent only establishes that the intersection is the carrier of some submodule and does not provide this closure argument directly, so recalling the parent does not solve the child.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existence claim into a closure property for the indexed intersection: every linear combination of two elements in the intersection remains in it. Proving this requires pointwise extraction of membership and applying each submodule's scalar and additive closure, which the parent's existence proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves closure of the indexed intersection under arbitrary linear combinations, requiring scalar- and addition-closure arguments for every component submodule. The parent's mere existence-and-carrier characterization does not supply this proof by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_linear_combination.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) (a b : F)
  (x y : V) (hx : x ∈ ⋂ i, ↑(u i)) (hy : y ∈ ⋂ i, ↑(u i)) : a • x + b • y ∈ ⋂ i, ↑(u i)
```

Read back by a model that saw only the Lean. Let F be a field, V an additive commutative group that is an F -module, and let u be a family of F -submodules of V indexed by a type ι . For scalars $a, b \in F$ and vectors $x, y \in V$, if x and y belong to every submodule $u(i)$, then $a \cdot x + b \cdot y$ belongs to every submodule $u(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_linear_combination {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) (a b : F) (x y : V)
  (hx : x ∈ (⋂ i : ι, (u i : Set V)))
  (hy : y ∈ (⋂ i : ι, (u i : Set V))) :
  a • x + b • y ∈ (⋂ i : ι, (u i : Set V)) := by
  have hx_all : ∀ i : ι, x ∈ u i := by
    intro i
    exact Set.mem_iInter.mp hx i
  have hy_all : ∀ i : ι, y ∈ u i := by
    intro i
    exact Set.mem_iInter.mp hy i
  have hcombined : ∀ i : ι, a • x + b • y ∈ u i := by
    intro i
    exact (u i).add_mem ((u i).smul_mem a (hx_all i)) ((u i).smul_mem b (hy_all i))
  exact Set.mem_iInter.mpr hcombined
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__bc501014

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint bc501014b392e5e2

4.41 041. linearMap_comap_iInf_composition

The problem

Let $f : V \rightarrow W$ and $g : W \rightarrow X$ be linear maps, and let $(U_i)_{i \in I}$ be subspaces of X . Show that the vectors $x \in V$ satisfying $g(f(x)) \in U_i$ for every i form the largest subspace with this property, and that pulling back the intersection in two stages equals pulling it back along $g \circ f$.

Lean statement

```
theorem linearMap_comap_iInf_composition {F V W X : Type*} [AddCommGroup V] [AddCommGroup W]
  [AddCommGroup X] [Field F] [Module F V] [Module F W] [Module F X]
  {ι : Type*} (f : V →[F] W) (g : W →[F] X) (u : ι → Submodule F X) :
  ∃ U : Submodule F V,
    U = (⋂ i, u i).comap g).comap f ∧
    U = (⋂ i, u i).comap (g.comp f) ∧
    (∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i) ∧
    ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i, u i).carrier = ↑U := by
```

What it contributes. Uses the parent's indexed-intersection-as-infimum construction, specializing it to the contravariant pullback of submodules along composable linear maps.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent by introducing two composable linear maps and proving that two-stage and composite pullbacks coincide, plus a greatest-submodule characterization. Its proof requires new comap/composition reasoning and an essential indexed-membership argument, not merely recalling the parent's intersection construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent from an arbitrary set intersection to a two-stage pullback along composable linear maps, adding both a pointwise characterization and a greatest-submodule property. Its proof requires comap composition and universal containment reasoning, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent from existence of an indexed intersection submodule to a two-stage versus one-stage pullback under composable linear maps. Its proof requires comap composition and a greatest-submodule argument, with indexed-intersection membership used essentially.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2224 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem linearMap_comap_iInf_composition.extracted_1.1.{u_5, u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{W : Type u_3} {X : Type u_4} [inst : AddCommGroup V] [inst_1 : AddCommGroup W] [inst_2 : AddCommGroup X]
[inst_3 : Field F] [inst_4 : Module F V] [inst_5 : Module F W] [inst_6 : Module F X] {ι : Type u_5} (f : V →ι[F] W)
(g : W →ι[F] X) (u : ι → Submodule F X) :
  ∃ U,
    U = Submodule.comap f (Submodule.comap g (⋂ i, u i)) ∧
    U = Submodule.comap (g ∘ι f) (⋂ i, u i) ∧
    (∀ (x : V), x ∈ U ↔ ∀ (i : ι), g (f x) ∈ u i) ∧
    ∀ (S : Submodule F V), (∀ x ∈ S, ∀ (i : ι), g (f x) ∈ u i) → S ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, let V, W , and X be F -modules whose additive structures are commutative groups, let ι be a type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of X . Then there exists a submodule U of V such that U is the inverse image under f of the inverse image under g of the intersection $\bigcap_{i \in \iota} u(i)$, U is also the inverse image under the composite $g \circ f$ of $\bigcap_{i \in \iota} u(i)$, every $x \in V$ belongs to U if and only if $g(f(x)) \in u(i)$ for every $i \in \iota$, and every submodule S of V satisfying $g(f(x)) \in u(i)$ for all $x \in S$ and all $i \in \iota$ is contained in U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem linearMap_comap_iInf_composition {F V W X : Type*} [AddCommGroup V] [AddCommGroup W]
[AddCommGroup X] [Field F] [Module F V] [Module F W] [Module F X]
{ι : Type*} (f : V →ι[F] W) (g : W →ι[F] X) (u : ι → Submodule F X) :
  ∃ U : Submodule F V,
    U = ((⋂ i, u i).comap g).comap f ∧
    U = (⋂ i, u i).comap (g.comp f) ∧
    (∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i) ∧
    ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U := by
  let U : Submodule F V := ((⋂ i, u i).comap g).comap f
  have hcomp : ((⋂ i, u i).comap g).comap f = (⋂ i, u i).comap (g.comp f) := by
    ext x
    rfl
  have hpoint : ∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i := by
    intro x
    change g (f x) ∈ (⋂ i, u i) ↔ ∀ i, g (f x) ∈ u i
    exact (Submodule.mem_iInf u)
  have hgreatest : ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U := by
    intro S hS x hx
    change g (f x) ∈ (⋂ i, u i)
    exact (Submodule.mem_iInf u).2 (fun i => hS x hx i)
  exact ⟨U, rfl, hcomp, hpoint, hgreatest⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__c12b47d6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c12b47d6b95350e1

4.42 042. intersection_is_greatest_lower_bound

The problem

Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Supplies the canonical indexed-infimum construction and the carrier/intersection membership checkpoint, which is expanded into the final greatest-lower-bound theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine greatest-lower-bound characterization: proving the universal equivalence for every submodule W requires the new lattice argument ‘ le_iInf_iff ’, beyond the parent’s carrier-equality proof. The added conjunct is substantive rather than decorative or forced.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine greatest-lower-bound characterization: it must prove the universal property of the infimum for every submodule W , not merely identify the intersection with some submodule. Although it reuses the canonical infimum construction, recalling the parent’s existence proof does not establish this added claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine universal-property characterization: the chosen intersection submodule must be the greatest lower bound of the family. The parent’s carrier-equality proof does not establish this order-theoretic claim, which requires the new ‘ le_iInf_iff ’ reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃ U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, and let u assign to each $i : \iota$ a submodule $u(i)$ of V over F . Then there exists a submodule U of V over F such that the intersection of the carrier sets of all $u(i)$ equals the carrier set of U , and for every submodule W of V over F , W is contained in U if and only if W is contained in $u(i)$ for every $i : \iota$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
  classical
  refine ⟨⋂ i, u i, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro W
    exact le_iInf_iff
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__d2ece878

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d2ece8786178579a

4.43 043. rotation_subgroup_reflections

The problem

Let R be the subgroup of rotations in a nontrivial dihedral group. Show that R is proper, every reflection lies outside R , and any subgroup containing R and one reflection contains every reflection and is the whole group.

Lean statement

```
theorem rotation_subgroup_reflections {n : ℕ} {hn : n ≠ 0}
  {R : Subgroup (DihedralGroup n)}
  (hR : R = Subgroup.closure {DihedralGroup.r (1 : ZMod n)}) :
  R ≠ ⊤ ∧
  (∀ i : ZMod n, DihedralGroup.sr i ∉ R) ∧
  (∀ K : Subgroup (DihedralGroup n), R ≤ K →
    (∃ i : ZMod n, DihedralGroup.sr i ∈ K) → K = ⊤)
```

Parent. Show that the subgroup of all rotations in a dihedral group is a maximal subgroup.

Parent in Lean

```
theorem Dummit_Foote_exercise_2_4_16b {n : ℕ} {hn : n ≠ 0}
  {R : Subgroup (DihedralGroup n)}
  (hR : R = Subgroup.closure {DihedralGroup.r 1}) :
  R ≠ ⊤ ∧
  ∀ K : Subgroup (DihedralGroup n), R ≤ K → K = R ∨ K = ⊤ := by
```

What it contributes. Supplies the rotation-subgroup presentation, zpowers characterization, rotation-membership checkpoint, and dihedral rotation/reflection case split.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is not a wrapper or decoration: it proves the genuinely new statement that every reflection is excluded and that any subgroup containing the rotation subgroup plus one reflection is the whole dihedral group. The explicit multiplication checkpoint and resulting generation of all reflections are not supplied by the parent's maximal-subgroup dichotomy.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not a recall wrapper: it proves every reflection is excluded from the rotation subgroup and establishes that any subgroup containing a reflection is top via an explicit reflection-product closure argument. This requires group multiplication and subgroup membership reasoning absent from the parent's maximal-subgroup dichotomy proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the substantive claim that every reflection lies outside the rotation subgroup and proves that any subgroup containing a reflection is the whole dihedral group. Its explicit reflection-reflection multiplication closure argument is not supplied by the parent's maximality statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2362 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rotation_subgroup_reflections.extracted_1_1 {n : ℕ} {hn : n ≠ 0} {R : Subgroup (DihedralGroup n)}
(hR : R = Subgroup.closure {DihedralGroup.r 1}) :
R ≠ τ ∧
(∀ (i : ZMod n), DihedralGroup.sr i ∉ R) ∧
∀ (K : Subgroup (DihedralGroup n)), R ≤ K → (∃ i, DihedralGroup.sr i ∈ K) → K = τ
```

Read back by a model that saw only the Lean. *Let n be a nonzero natural number, and let R be a subgroup of the dihedral group of order parameter n . Suppose that R is the subgroup generated by the rotation $r(1)$. Then R is not the whole dihedral group; for every i in $ZMod\ n$, the reflection $sr(i)$ does not belong to R ; and for every subgroup K of the dihedral group, if R is contained in K and K contains at least one reflection $sr(i)$, then K is the whole dihedral group.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem rotation_subgroup_reflections {n : ℕ} {hn : n ≠ 0}
{R : Subgroup (DihedralGroup n)}
(hR : R = Subgroup.closure {DihedralGroup.r (1 : ZMod n)}) :
R ≠ τ ∧
(∀ i : ZMod n, DihedralGroup.sr i ∉ R) ∧
(∀ K : Subgroup (DihedralGroup n), R ≤ K →
(∃ i : ZMod n, DihedralGroup.sr i ∈ K) → K = τ) := by
classical
letI : NeZero n := ⟨hn⟩
have hR_zpowers :
Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) = R := by
exact
(Subgroup.zpowers_eq_closure (DihedralGroup.r (1 : ZMod n))).trans hR.symm
have hr_mem : ∀ j : ZMod n, DihedralGroup.r j ∈ R := by
intro j
obtain ⟨m, hm⟩ := (ZMod.intCast_surjective (n := n)) j
have hmem :
(DihedralGroup.r (1 : ZMod n)) ^ m ∈
Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) :=
Subgroup.zpow_mem_zpowers _ m
have hmemR : (DihedralGroup.r (1 : ZMod n)) ^ m ∈ R := by
simp [hR_zpowers] using hmem
simp [DihedralGroup.r_one_zpow (n := n) (k := m), hm] using hmemR
have hreflection_not_mem : ∀ i : ZMod n, DihedralGroup.sr i ∉ R := by
intro i hi
have hzpow :
DihedralGroup.sr i ∈ Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) := by
simp [hR_zpowers] using hi
rcases Subgroup.mem_zpowers_iff.mp hzpow with ⟨m, hm⟩
have hrot :
DihedralGroup.r (m : ZMod n) = DihedralGroup.sr i := by
simp [DihedralGroup.r_one_zpow (n := n) (k := m)] using hm
simp using hrot
have hmul : ∀ j i : ZMod n,
DihedralGroup.sr j * DihedralGroup.sr i ∈ R := by
intro j i
rw [DihedralGroup.sr_mul_sr]
```

```

    exact hr_mem (i - j)
  refine (?, hreflection_not_mem, ?_)
  · intro htop
    have hmem : DihedralGroup.sr 0 ∈ R := by simp [htop]
    exact hreflection_not_mem 0 hmem
  · intro K hRK hcontains
    obtain ⟨i, hi⟩ := hcontains
    have hall : ∀ j : ZMod n, DihedralGroup.sr j ∈ K := by
      intro j
      have hp : DihedralGroup.sr j * DihedralGroup.sr i ∈ K :=
        hRK (hmul j i)
      have hinv : (DihedralGroup.sr i)-1 ∈ K := K.inv_mem hi
      have hprod :
        (DihedralGroup.sr j * DihedralGroup.sr i) *
        (DihedralGroup.sr i)-1 ∈ K := K.mul_mem hp hinv
      simpa using hprod
    apply top_unique
    intro x hx
    cases' x with j j
    · exact hRK (hr_mem j)
    · exact hall j

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_2_4_16b__me__20c0330b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20c0330bce06b0fd

4.44 044. normalClosure_union_eq_sup

The problem

Let S and T be subsets of a group. Prove that the normal closure of $S \cup T$ is the join of the normal closures of S and T .

Lean statement

```
theorem normalClosure_union_eq_sup {G : Type*} [Group G] (S T : Set G) :
  Subgroup.normalClosure (S ∪ T) =
    Subgroup.normalClosure S ∪ Subgroup.normalClosure T
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKnormal : K.Normal) :
  T ⊆ (K : Set G) ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
      Subgroup.normalClosure S ≤ K
```

What it contributes. Supplies the normal-closure universal-property checkpoint used for each inclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a distinct union-to-join law, requiring separate inclusion arguments and monotonicity for both sets; the parent's quantified universal-property equivalence does not supply this conclusion by recall.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a distinct normal-closure distributivity law, requiring union-case reasoning and combining monotonicity with the supremum property. The parent's universal characterization does not directly supply this proof, so recalling it would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different union-to-supremum equality, requiring antisymmetry, union case analysis, and monotonicity of normal closure. The parent's universal-property proof does not supply this argument, and the child does not embed or trivially reuse the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2983 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_union_eq_sup.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S T : Set G) :  
  normalClosure (S ∪ T) = normalClosure S ∪ normalClosure T
```

Read back by a model that saw only the Lean. For any group G and any subsets S and T of G , the normal closure of $S \cup T$ is equal to the supremum of the normal closures of S and T .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_union_eq_sup {G : Type*} [Group G] (S T : Set G) :  
  Subgroup.normalClosure (S ∪ T) =  
    Subgroup.normalClosure S ∪ Subgroup.normalClosure T := by  
  apply le_antisymm  
  · apply (Subgroup.normalClosure_subset_iff).mp  
    intro x hx  
    rcases hx with hx | hx  
    · exact (le_sup_left : Subgroup.normalClosure S ≤ Subgroup.normalClosure S ∪ Subgroup.normalClosure T)  
      (Subgroup.subset_normalClosure (s := S) hx)  
    · exact (le_sup_right : Subgroup.normalClosure T ≤ Subgroup.normalClosure S ∪ Subgroup.normalClosure T)  
      (Subgroup.subset_normalClosure (s := T) hx)  
  · apply sup_le  
    · exact Subgroup.normalClosure_mono Set.subset_union_left  
    · exact Subgroup.normalClosure_mono Set.subset_union_right
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me.me__c944fb1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c944fb1db4f020d0

4.45 045. subset_iInf_iff_forall_normalClosure_subset

The problem

Let T be a subset of a group and let (H_i) be a family of normal subgroups. Prove that T is contained in every H_i if and only if, for every subset S of the normal closure of T , the normal closure of S is contained in the intersection of all H_i .

Lean statement

```
theorem subset_iInf_iff_forall_normalClosure_subset {G : Type*} [Group G] {I : Type*}
  (T : Set G) (H : I → Subgroup G) (hNormal : ∀ i : I, (H i).Normal) :
  (∀ i : I, T ⊆ (H i : Set G)) ↔
  ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
    Subgroup.normalClosure S ≤ ⋂ i : I, H i
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKNormal : K.Normal) :
  T ⊆ (K : Set G) ↔
  ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
    Subgroup.normalClosure S ≤ K
```

What it contributes. Supplies the uniform-subset normal-closure pattern and the generator extraction through `sub_normalClosure`; its containment argument is generalized to an indexed normal-subgroup infimum.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child introduces an indexed family of normal subgroups and must prove the equivalence through normality and membership properties of their infimum. The parent's single-subgroup argument does not supply the needed `iInf` reasoning, and the hypothesis `hNormal` is essential.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the single normal subgroup into an indexed family and changes the target to an infimum. Its proof requires establishing normality of the infimum and reasoning with indexed subgroup membership, beyond the parent's single-subgroup argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single normal subgroup to an indexed family and must establish normality of the infimum, membership in the infimum, and extraction of each component bound. The parent's closure argument helps locally but does not prove this simultaneous-family formulation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2981 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem subset_iInf_iff_forall_normalClosure_subset.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G]
  {I : Type u_2} (T : Set G) (H : I → Subgroup G) (hNormal : ∀ (i : I), (H i).Normal) :
  (∀ (i : I), T ⊆ ↑(H i)) ↔ ∀ S ⊆ ↑(normalClosure T), normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, let T be a subset of G , and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Then the following are equivalent: (1) for every $i \in I$, T is contained in the underlying set of $H(i)$; (2) for every subset S of the underlying set of the normal closure of T in G , the normal closure of S is contained in the infimum subgroup $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem subset_iInf_iff_forall_normalClosure_subset {G : Type*} [Group G] {I : Type*}
  (T : Set G) (H : I → Subgroup G) (hNormal : ∀ i : I, (H i).Normal) :
  (∀ i : I, T ⊆ (H i : Set G)) ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
      Subgroup.normalClosure S ≤ ⋂ i : I, H i := by
  classical
  have hInfNormal : (⋂ i : I, H i).Normal :=
    Subgroup.normal_iInf_normal hNormal
  let I : (⋂ i : I, H i).Normal := hInfNormal
  constructor
  · intro hAll S hST
    have hTInf : T ⊆ (⋂ i : I, H i : Subgroup G) := by
      intro x hx
      exact Subgroup.mem_iInf.mpr (fun i => hAll i hx)
    have hClosureT : Subgroup.normalClosure T ≤ ⋂ i : I, H i :=
      Subgroup.normalClosure_le_normal hTInf
    have hSInf : S ⊆ (⋂ i : I, H i : Subgroup G) := by
      intro x hx
      exact hClosureT (hST hx)
    exact Subgroup.normalClosure_subset_iff.mpr hSInf
  · intro hUniform
    have hTGenerated : T ⊆ (Subgroup.normalClosure T : Set G) := by
      exact Subgroup.subset_normalClosure
    have hClosureInf : Subgroup.normalClosure T ≤ ⋂ i : I, H i := by
      exact hUniform T hTGenerated
    intro i x hx
    exact (iInf_le H i) (hClosureInf (hTGenerated hx))
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me.me__ecf2e464

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ecf2e4649340e958

4.46 046. normalClosure_family_containment_iff

The problem

Let S and T be subsets of a group, and let (N_i) be a family of normal subgroups. If each set is contained in the normal closure of the other, prove that the normal closure of S is contained in every N_i if and only if the normal closure of T is contained in every N_i .

Lean statement

```
theorem normalClosure_family_containment_iff {G : Type*} [Group G] {ι : Type*} (S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) (hTS : T ⊆ (Subgroup.normalClosure S : Set G)) : (∀ i, Subgroup.normalClosure S ≤ N i) ↔ (∀ i, Subgroup.normalClosure T ≤ N i)
```

Parent. Let S and T be subsets of a group, and let M be a normal subgroup containing the normal closure of T . If S is contained in the normal closure of T , prove that the normal closure of S is contained in M .

Parent in Lean

```
theorem normalClosure_le_of_subset_normalClosure {G : Type*} [Group G] (S T : Set G) (M : Subgroup G) (hMnormal : M.Normal) (hTM : (Subgroup.normalClosure T : Set G) ⊆ M) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) : Subgroup.normalClosure S ≤ M
```

What it contributes. Supplied the normal-closure containment mechanism: subset containment into a normal subgroup yields containment of the normal closure; this is applied separately for both transports.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine mutual-containment equivalence over an indexed family, requiring a fresh direction-by-direction argument using normality and the normal-closure universal property. The parent's one-way transport lemma does not directly supply either direction, and this is not the same characterization as the already-kept child.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an indexed-family equivalence in both directions, using mutual containment to transfer each normal closure into every normal subgroup. The parent only supplies a one-way containment result for a single target subgroup, so recalling its proof does not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2967 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_family_containment_iff.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2}
(S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ (i : ι), (N i).Normal) (hST : S ⊆ ↑(normalClosure T))
(hTS : T ⊆ ↑(normalClosure S)) : (∀ (i : ι), normalClosure S ≤ N i) ↔ ∀ (i : ι), normalClosure T ≤ N i
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be a type, let S and T be subsets of G , and let N assign to each $i \in \iota$ a subgroup $N(i)$ of G . Suppose that $N(i)$ is a normal subgroup of G for every $i \in \iota$, S is contained in the underlying set of the normal closure of T in G , and T is contained in the underlying set of the normal closure of S in G . Then, for every $i \in \iota$, the normal closure of S in G is contained in $N(i)$ if and only if, for every $i \in \iota$, the normal closure of T in G is contained in $N(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_family_containment_iff {G : Type*} [Group G] {ι : Type*} (S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ i, (N
↪ i).Normal) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) (hTS : T ⊆ (Subgroup.normalClosure S : Set G)) : (∀ i,
↪ Subgroup.normalClosure S ≤ N i) ↔ (∀ i, Subgroup.normalClosure T ≤ N i) := by
  constructor
  · intro hS i
    letI : (N i).Normal := hnormal i
    apply Subgroup.normalClosure_le_normal
    intro x hx
    exact hS i (hTS hx)
  · intro hT i
    letI : (N i).Normal := hnormal i
    apply Subgroup.normalClosure_le_normal
    intro x hx
    exact hT i (hST hx)
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me__c614d88a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c614d88a8bf7793d

4.47 047. indexed_normalClosure_le_iInf

The problem

Let $\{H_i\}_{i \in I}$ be normal subgroups of a group G . If $T \subseteq H_i$ for every i and $S \subseteq \langle\langle T \rangle\rangle$, prove that $\langle\langle S \rangle\rangle \subseteq \bigcap_{i \in I} H_i$.

Lean statement

```
theorem indexed_normalClosure_le_iInf {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S T : Set G)
  ↪ (hT : ∀ i : I, T ⊆ H i) (hS : S ⊆ Subgroup.normalClosure T) : Subgroup.normalClosure S ≤ ⋂ i, H i
```

Parent. Prove that the infimum of any set of normal subgroups of a group is normal.

Parent in Lean

```
theorem set_iInf_normal_subgroups {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ H ∈ S, H.Normal) : (sInf S : Subgroup G).Normal
```

What it contributes. Supplies the derived-target normality step for the indexed infimum, used as the local normal target for both closure transports.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different containment theorem: it must show that generators in $\$T\$$ lie in the indexed infimum, propagate containment through normal closure to $\$S\$$, and then close again under normal closure. The parent only establishes normality of an infimum and does not supply these closure and membership arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive closure-containment result: it first establishes normality and containment in the indexed infimum, then applies normal closure minimality twice. The parent's sInf-normality argument alone does not supply this reasoning, and all child hypotheses are used.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new normal-closure argument: it first establishes normality of the indexed infimum, proves containment of T there, lifts this through normalClosure, and then handles S . The parent only establishes normality of a set infimum and does not supply the closure-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2974 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem indexed_normalClosure_le_iInf.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) (S T : Set G) (hT : ∀ (i : I), T ⊆ ↑(H i))
  (hS : S ⊆ ↑(normalClosure T)) : normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Let S and T be subsets of G . Assume that T is contained in $H(i)$ for every $i \in I$, and that S is contained in the normal closure of T in G . Then the normal closure of S in G is contained in the infimum of the subgroups $H(i)$ over all $i \in I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem indexed_normalClosure_le_iInf {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal)
  (S T : Set G) (hT : ∀ i : I, T ⊆ H i)
  (hS : S ⊆ Subgroup.normalClosure T) :
  Subgroup.normalClosure S ≤ ⋂ i, H i := by
have hnormal : (⋂ i, H i).Normal := by
  apply Subgroup.normal_iInf_normal
  intro i
  exact hH i
have hTinf : ∀ x ∈ T, x ∈ (⋂ i, H i) := by
  intro x hx
  rw [Subgroup.mem_iInf]
  intro i
  exact hT i hx
let I : (⋂ i, H i).Normal := hnormal
have hTclosure : Subgroup.normalClosure T ≤ (⋂ i, H i) := by
  exact Subgroup.normalClosure_le_normal (fun x hx => hTinf x hx)
have hSinTarget : ∀ x ∈ S, x ∈ (⋂ i, H i) := by
  intro x hx
  exact hTclosure (hS hx)
exact Subgroup.normalClosure_le_normal (fun x hx => hSinTarget x hx)
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me__e215357d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e215357d58211e88

4.48 048. normalClosure_le_iInf_iff

The problem

Let S be a subset of a group and let $(N_i)_{i \in I}$ be a family of normal subgroups. Prove that the normal closure of S is contained in $\bigcap_{i \in I} N_i$ if and only if S is contained in every N_i .

Lean statement

```
theorem normalClosure_le_iInf_iff {G : Type*} [Group G] {ι : Type*} (S : Set G)
  (N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) :
  Subgroup.normalClosure S ≤ ⋂ i, N i ↔ ∀ i, S ≤ N i
```

Parent. Let S and T be subsets of a group. If S is contained in the normal closure of T , prove that the normal closure of S is contained in the normal closure of T .

Parent in Lean

```
theorem normalClosure_mono_of_subset {G : Type*} [Group G] (S T : Set G)
  (hST : S ⊆ (Subgroup.normalClosure T : Set G)) :
  Subgroup.normalClosure S ≤ Subgroup.normalClosure T
```

What it contributes. Supplied the normal-closure monotonicity and least-normal-overgroup checkpoints, generalized from one target normal closure to an arbitrary indexed family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from one normal-closure target to an arbitrary indexed infimum and requires both the infimum introduction/elimination arguments and pointwise normality. A solver recalling the parent's single-target proof would not directly solve the indexed equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single normal-closure inclusion to an indexed infimum and proves both directions using infimum bounds. The parent's monotonicity lemma supplies only the per-index step; it does not provide the new universal-property reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes containment in an indexed infimum, requiring both extraction through each infimum projection and reconstruction via 'le_iInf'; the parent only supplies the local normal-closure comparison. Its converse direction and indexed-family reasoning cannot be recovered by simply recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2954 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_le_iInf_iff.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2} (S : Set G)
(N : ι → Subgroup G) (hnormal : ∀ (i : ι), (N i).Normal) : normalClosure S ≤ ⋂ i, N i ↔ ∀ (i : ι), S ⊆ ↑(N i)
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be an index type, let S be a subset of G , and let N assign to each $i \in \iota$ a normal subgroup $N(i)$ of G . Then the normal closure of S in G is contained in the infimum of the subgroups $N(i)$ if and only if, for every $i \in \iota$, S is contained in $N(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_le_iInf_iff {G : Type*} [Group G] {ι : Type*} (S : Set G)
(N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) :
Subgroup.normalClosure S ≤ ⋂ i, N i ↔ ∀ i, S ⊆ N i := by
  constructor
  · intro h i x hx
    exact (iInf_le N i) (h (Subgroup.subset_normalClosure hx))
  · intro h
    refine le_iInf (fun i => ?_)
    letI : (N i).Normal := hnormal i
    exact Subgroup.normalClosure_le_normal (h i)
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me__3bcaece7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 3bcaece7d5ad25a8

4.49 049. normalClosure_conj_image_iInf_iff

The problem

Let G be a group and let (H_i) be any family of normal subgroups. For a subset $S \subseteq G$ and $g \in G$, the normal closure of S lies in $\bigcap_i H_i$ if and only if the normal closure of gSg^{-1} does.

Lean statement

```
theorem normalClosure_conj_image_iInf_iff {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S : Set G)
  ↔ (g : G) : normalClosure ((fun x : G => g * x * g⁻¹) '' S) ≤ (⋂ i, H i) ↔ normalClosure S ≤ (⋂ i, H i)
```

Parent. Prove that the intersection of any indexed family of normal subgroups is closed under conjugation, even when the index set is empty.

Parent in Lean

```
theorem indexed_inf_conjugation_closed {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : ∀ n, n ∈ (⋂
  i, H i) → ∀ g : G, g * n * g⁻¹ ∈ (⋂ i, H i)
```

What it contributes. Supplied the normality and conjugation-closure checkpoint for the indexed infimum, reused here as the transport mechanism for conjugated generators.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a pointwise conjugation-closure result into an equivalence about containment of normal closures of conjugated sets. Proving both directions requires lifting set images through normal closure and applying conjugation by the inverse, which is not supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine equivalence about normal closures under conjugating an entire generating set. It requires image unpacking, normal-closure containment, and conjugation by the inverse, none of which is supplied by the parent's direct membership proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an equivalence about containment of normal closures under conjugating an entire generating set, requiring both image unpacking and reverse conjugation via normality. The parent only establishes elementwise conjugation closure of the indexed infimum, so recalling it does not supply the child's normal-closure argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2962 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_conj_image_iInf_iff.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) (S : Set G) (g : G) :
  normalClosure ((fun x => g * x * g⁻¹) '' S) ≤ ⋂ i, H i ↔ normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, and let $H : I \rightarrow \text{Subgroup } G$ be a family of normal subgroups of G . For any subset S of G and any $g \in G$, the normal closure in G of the conjugate set $\{g x g^{-1} \mid x \in S\}$ is contained in the intersection of all the subgroups $H i$ if and only if the normal closure of S is contained in that intersection.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_conj_image_iInf_iff {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S : Set G)
  (g : G) : normalClosure ((fun x : G => g * x * g⁻¹) '' S) ≤ (⋂ i, H i) ↔ normalClosure S ≤ (⋂ i, H i) := by
  let I : (⋂ i, H i).Normal := Subgroup.normal_iInf_normal hH
  constructor
  · intro hConj
    apply Subgroup.normalClosure_le_normal
    intro x hx
    have hxConj : g * x * g⁻¹ ∈ (⋂ i, H i) := by
      apply hConj
      exact Subgroup.subset_normalClosure ⟨x, hx, rfl⟩
    have hxBack := (show (⋂ i, H i).Normal from inferInstance).conj_mem' (g * x * g⁻¹) hxConj g
    simpa [mul_assoc] using hxBack
  · intro hS
    apply Subgroup.normalClosure_le_normal
    intro x hx
    rcases hx with ⟨y, hy, rfl⟩
    have hyInf : y ∈ (⋂ i, H i) := hS (Subgroup.subset_normalClosure hy)
    exact (show (⋂ i, H i).Normal from inferInstance).conj_mem y hyInf g
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.ms.me__d98b1ebb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d98b1ebb84522093

4.50 050. normalClosure_inter_subset

The problem

For subsets S and T of a group, show that the normal subgroup generated by $S \cap T$ is contained in the intersection of the normal subgroups generated by S and T .

Lean statement

```
theorem normalClosure_inter_subset {G : Type*} [Group G] (S T : Set G) : Subgroup.normalClosure (S ∩ T) ≤ Subgroup.normalClosure S ∩ Subgroup.normalClosure T
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup G).Normal
```

What it contributes. Uses the parent's normality-of-intersections checkpoint in the bounded two-subgroup meet setting.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different inclusion using normal-closure minimality and normality of the meet; the parent's iInf -normality argument and closing lemma do not solve it by recall. It also introduces no redundant hypotheses or forced wrapper structure.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not obtainable by recalling the parent's iInf -normality argument: it requires constructing the inclusion of the intersection into the meet and applying the normal-closure universal property, together with normality of the meet. Its proof uses different mathematical checkpoints and no parent theorem is embedded.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new universal-property argument: it proves that the meet of two normal closures is normal, contains the intersection, and therefore bounds the normal closure of that intersection. The parent only establishes normality of an infimum of already-normal subgroups and does not supply this closure-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2828 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_inter_subset.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S T : Set G) :  
  normalClosure (S ∩ T) ≤ normalClosure S ∩ normalClosure T
```

Read back by a model that saw only the Lean. For any group G and any subsets S and T of G , the normal closure of $S \cap T$ is contained in the infimum (equivalently, the intersection) of the normal closures of S and T .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_inter_subset {G : Type*} [Group G] (S T : Set G) :  
  Subgroup.normalClosure (S ∩ T) ≤  
    Subgroup.normalClosure S ∩ Subgroup.normalClosure T := by  
  letI hSNormal : (Subgroup.normalClosure S).Normal :=  
    Subgroup.normalClosure_normal  
  letI hTNormal : (Subgroup.normalClosure T).Normal :=  
    Subgroup.normalClosure_normal  
  have hMeetNormal :  
    (Subgroup.normalClosure S ∩ Subgroup.normalClosure T).Normal :=  
    Subgroup.normal_inf_normal _ _  
  letI hMeetNormalInstance :  
    (Subgroup.normalClosure S ∩ Subgroup.normalClosure T).Normal := hMeetNormal  
  have hIntersection :  
    S ∩ T ≤ ↑(Subgroup.normalClosure S ∩ Subgroup.normalClosure T) := by  
    intro x hx  
    exact ⟨Subgroup.subset_normalClosure hx.1,  
      Subgroup.subset_normalClosure hx.2⟩  
  exact Subgroup.normalClosure_le_normal hIntersection
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__16f5f277

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16f5f2777b528b85

4.51 051. normalCore_iInf_eq_iInf_normalCore

The problem

Let G be a group and let $(H_i)_{i \in I}$ be a nonempty indexed family of subgroups. Prove that the normal core of their intersection is the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

What it contributes. Supplied the arbitrary-intersection normality checkpoint, generalized here to the family of normal cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely new indexed-infimum identity, requiring monotonicity, normality of an indexed infimum, and the universal property of the normal core. The parent's greatest-subgroup conjunction does not directly supply this proof, and no hypotheses are unused.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine meet-preservation law for normalCore over an arbitrary nonempty indexed family. Its proof requires lattice inequalities and establishing that the indexed infimum of normal subgroups is normal, reasoning not supplied by the parent's single-subgroup characterization proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive commutation law for normal cores over arbitrary nonempty indexed intersections. Its proof requires lattice-theoretic iInf reasoning and normality of an intersection, which is not supplied by recalling the parent's greatest-subgroup argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2309 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_eq_iInf_normalCore.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {I : Type u_2}
  [inst_1 : Nonempty I] (H : I → Subgroup G) : (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, and let H assign to each element i of I a subgroup $H(i)$ of G . Then the normal core of the infimum of all the subgroups $H(i)$ is equal to the infimum of their normal cores: $(\bigcap_i H(i)).\text{normalCore} = \bigcap_i H(i).\text{normalCore}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore := by
  apply le_antisymm
  · refine le_iInf ?_
    intro i
    exact Subgroup.normalCore_mono (iInf_le H i)
  · have hN_normal : (⋂ i, (H i).normalCore).Normal :=
      Subgroup.normal_iInf_normal (fun i => (H i).normalCore_normal)
    letI : (⋂ i, (H i).normalCore).Normal := hN_normal
    have hN_le : (⋂ i, (H i).normalCore) ≤ ⋂ i, H i := by
      refine le_iInf ?_
      intro i
      exact le_trans (iInf_le (fun j => (H j).normalCore) i) (Subgroup.normalCore_le (H i))
    exact Subgroup.normal_le_normalCore.mpr hN_le
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__58f2f6b7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 58f2f6b79940a756

4.52 052. normal_iInf_comap

The problem

Let G and K be groups, let I be a type, let $(H_i)_{i \in I}$ be a family of normal subgroups of K , and let $f : G \rightarrow K$ be a group homomorphism. Then the infimum of the subgroups $f^{-1}(H_i)$ is normal in G .

Lean statement

```
theorem normal_iInf_comap {G K : Type*} [Group G] [Group K] (I : Type*) (H : I → Subgroup K) (hH : ∀ i : I, Normal (H i)) (f : G →* K) :
  ⊆ Normal (⋂ i, (H i).comap f)
```

Parent. Prove that the intersection of an arbitrary indexed family of normal subgroups of a group is a normal subgroup, without assuming that the indexing type is nonempty.

Parent in Lean

```
theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i)
```

What it contributes. Supplied the indexed-infimum normality checkpoint, which is applied after each subgroup is replaced by its homomorphic comap.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine transport argument: it proves each comapped subgroup is normal via the homomorphism comap operation, then applies normality of arbitrary infima. This is not supplied by the parent's proof, even though both use the same library infimum lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transports normality through each subgroup comap under a homomorphism before taking the indexed infimum. This requires the new comap-normality lemma and is not obtainable by merely recalling the parent's empty/nonempty index proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transports normality through an arbitrary group homomorphism and then applies the indexed-infimum theorem to the comapped family. This requires the distinct mathematical lemma that comaps of normal subgroups are normal, so recalling the parent's empty-index case split and infimum argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1907 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_iInf_comap.extracted_1_1.{u_3, u_2, u_1} {G : Type u_1} {K : Type u_2} [inst : Group G]
[inst_1 : Group K] (I : Type u_3) (H : I → Subgroup K) (hH : ∀ (i : I), (H i).Normal) (f : G →* K) :
(⋂ i, Subgroup.comap f (H i)).Normal
```

Read back by a model that saw only the Lean. *Let G and K be groups, let I be a type, let H assign to each $i : I$ a subgroup $H(i)$ of K , and suppose $H(i)$ is normal in K for every i . For any group homomorphism $f : G \rightarrow K$, the infimum over i of the subgroups of G given by the preimages $f^{-1}(H(i))$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_iInf_comap {G K : Type*} [Group G] [Group K] (I : Type*)
(H : I → Subgroup K) (hH : ∀ i : I, Normal (H i)) (f : G →* K) :
Normal (⋂ i, (H i).comap f) := by
have hcomap : ∀ i : I, Normal ((H i).comap f) := by
intro i
exact (hH i).comap f
have hnormal : Normal (⋂ i, (H i).comap f) :=
Subgroup.normal_iInf_normal hcomap
simpa only using hnormal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__9abe72ec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b3a6ac04a0858410

4.53 053. finite_normalCore_greatest

The problem

Let H_i be a finite family of subgroups of a group G . The infimum of their normal cores is the greatest normal subgroup of G contained in every H_i .

Lean statement

```
theorem finite_normalCore_greatest {G : Type*} [Group G] {ι : Type*} [Fintype ι]
  (H : ι → Subgroup G) :
  (⋂ i, (H i).normalCore).Normal ∧
  (⋂ i, (H i).normalCore = (⋂ i, H i).normalCore) ∧
  (∀ i, (⋂ j, (H j).normalCore) ≤ H i) ∧
  ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ ⋂ i, (H i).normalCore
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

What it contributes. Uses the parent normal-core normality, containment, and maximality properties; these are lifted from one subgroup to a finite infimum and combined with the distribution equality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one subgroup to an indexed family and proves a new greatest-normal-subgroup characterization for the infimum, including a nontrivial commutation of normal core with infimum. Its proof requires family-level lattice reasoning and cannot be obtained by recalling the parent's single-subgroup argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new reasoning about indexed infima: proving normality, containment in every member, and the equality between the normal core of an infimum and the infimum of normal cores. Although it reuses the parent's normal-core maximality lemma, the finite-family lattice argument is not supplied by the parent and is not merely the already-kept equality theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely lifts the single-subgroup characterization to an indexed finite family: it must prove normality of an infimum, the normal-core/infimum equality, containment in every member, and the corresponding maximality property. These require new infimum and lattice reasoning beyond the parent's direct normal-core argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2353 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A restricts the family to be finite, whereas Statement B asserts the result for an arbitrary indexed family.

Goal round-trip: what Lean elaborated

```
theorem finite_normalCore_greatest.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2}
(H : ι → Subgroup G) :
(Π i, (H i).normalCore).Normal ∧
  Π i, (H i).normalCore = (Π i, H i).normalCore ∧
  (∀ (i : ι), Π j, (H j).normalCore ≤ H i) ∧
  ∀ (N : Subgroup G), N.Normal → (∀ (i : ι), N ≤ H i) → N ≤ Π i, (H i).normalCore
```

Read back by a model that saw only the Lean. Let G be a group, let ι be a type, and let H be a family of subgroups of G indexed by ι . Then the intersection (infimum) of the normal cores of the subgroups $H(i)$ is a normal subgroup of G ; it equals the normal core of the intersection (infimum) of all the $H(i)$; it is contained in $H(i)$ for every i in ι ; and every normal subgroup N of G that is contained in $H(i)$ for every i in ι is contained in the intersection of the normal cores of the $H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem finite_normalCore_greatest {G : Type*} [Group G] {ι : Type*} [Fintype ι]
(H : ι → Subgroup G) :
(Π i, (H i).normalCore).Normal ∧
  (Π i, (H i).normalCore = (Π i, H i).normalCore) ∧
  (∀ i, (Π j, (H j).normalCore) ≤ H i) ∧
  ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ Π i, (H i).normalCore := by
  have hnormal : (Π i, (H i).normalCore).Normal :=
    Subgroup.normal_iInf_normal (fun i => (H i).normalCore_normal)
  letI : (Π i, (H i).normalCore).Normal := hnormal
  have hdist : (Π i, H i).normalCore = Π i, (H i).normalCore := by
    apply le_antisymm
    · refine le_iInf (fun i => ?_)
      exact Subgroup.normalCore_mono (iInf_le H i)
    · apply Subgroup.normal_le_normalCore.mpr
      exact le_iInf (fun i => le_trans (iInf_le _ i) (H i).normalCore_le)
  have hcandidate : ∀ i, (Π j, (H j).normalCore) ≤ H i := by
    intro i
    exact le_trans (iInf_le _ i) (H i).normalCore_le
  have hmax : ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ Π i, (H i).normalCore := by
    intro N hN hNH
    letI : N.Normal := hN
    rw [← hdist]
    apply Subgroup.normal_le_normalCore.mpr
    exact le_iInf (fun i => hNH i)
  exact ⟨hnormal, hdist.symm, hcandidate, hmax⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3.1.22b__me.me__a240affc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a240affc44df70c8

4.54 054. normalCore_iInf_finset_eq_iSup

The problem

Let H_i be a finite family of subgroups of a group G . Prove that the normal core of their intersection is the supremum of all normal subgroups contained in every H_i .

Lean statement

```
theorem normalCore_iInf_finset_eq_iSup {G α : Type*} [Group G] (s : Finset α) (H : α → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore =
    ⋃ (N : Subgroup G) (N : Normal N) (N : ∀ i ∈ s, N ≤ H i), N
```

Parent. Let H_i be a finite nonempty family of subgroups of a group G . Prove that the normal core of their intersection equals the intersection of their normal cores.

Parent in Lean

```
theorem normalCore_iInf_finset {G α : Type*} [Group G] (s : Finset α) (H : α → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore
```

What it contributes. Supplies the finite-intersection normal-core checkpoint, reproduced as `hfinite_core` and then consumed to identify the target core with the intersection of individual cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the equality into a supremum characterization of the normal core, requiring the independent theorem ‘Subgroup.normalCore_eq_iSup’ and indexed supremum reasoning. Memorizing the parent’s infimum-preservation proof would not substantially help solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from commuting normalCore with a finite infimum to characterizing the normal core of that infimum as a supremum over all normal subgroups contained in every member. Its proof uses the independent supremum characterization and lattice bounds, so recalling the parent does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an equality of normal cores over a finite infimum to an iSup characterization of the normal core itself. Its proof requires lattice supremum reasoning and the normalCore_eq_iSup theorem; memorizing the parent proof does not substantially solve this claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2160 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_finset_eq_iSup.extracted_1_1.{u_2, u_1} {G : Type u_1} {ι : Type u_2} [inst : Group G]
  (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore = ⋃ N, ⋃ (_ : N.Normal), ⋃ (_ : ∀ i ∈ s, N ≤ H i), N
```

Read back by a model that saw only the Lean. For any group G , any type ι , any finite subset s of ι , and any family H assigning a subgroup $H(i)$ of G to each $i \in \iota$, the normal core of the infimum of the subgroups $H(i)$ for i in s equals the supremum, over all subgroups N of G , of N whenever N is normal in G and N is contained in $H(i)$ for every $i \in s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_finset_eq_iSup {G ι : Type*} [Group G] (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore =
    ⋃ (N : Subgroup G) (_ : Normal N) (_ : ∀ i ∈ s, N ≤ H i), N := by
  rw [Subgroup.normalCore_eq_iSup]
  apply le_antisymm
  · refine iSup_le fun N => ?_
    refine iSup_le fun hN => ?_
    refine iSup_le fun hNK => ?_
    refine le_iSup_of_le N ?_
    refine le_iSup_of_le hN ?_
    refine le_iSup_of_le (fun i hi => ?_) le_rfl
    exact hNK.trans (iInf_le_of_le i (iInf_le_of_le hi le_rfl))
  · refine iSup_le fun N => ?_
    refine iSup_le fun hN => ?_
    refine iSup_le fun hNcontained => ?_
    refine le_iSup_of_le N ?_
    refine le_iSup_of_le hN ?_
    refine le_iSup_of_le (le_iInf fun i => le_iInf fun hi => hNcontained i hi) le_rfl
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh.mh.me__cae6f81f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cae6f81f7639e060

4.55 055. normalCore_inf_of_normal

The problem

Let H be a subgroup of a group G , and let K be a normal subgroup. Prove that the normal core of $H \cap K$ equals the intersection of the normal core of H with K .

Lean statement

```
theorem normalCore_inf_of_normal {G : Type*} [Group G] (H K : Subgroup G) (hK : K.Normal) :
  (H ∩ K).normalCore = H.normalCore ∩ K
```

Parent. For a subgroup H of a group G , the normal core of H is normal. Moreover, every normal subgroup N is contained in the normal core exactly when it is contained in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup_reversed {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H.normalCore ↔ N ≤ H)
```

What it contributes. Supplied the normal-core normality and universal containment property used for both inclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new intersection identity requiring lattice reasoning and specialization to the normal subgroup K ; it is not obtained by recalling the parent's greatest-subgroup characterization alone. Its proof uses the underlying normal-core lemmas in a genuinely different argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different intersection identity: it must show both inclusions and use that a normal subgroup is its own normal core. The parent's greatest-subgroup characterization does not directly supply this equality or its lattice argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's greatest-subgroup characterization into a substantive interaction law for normal-Core and infimum. Its proof requires separate inclusions and uses normality of the specialized subgroup K ; recalling the parent's conjunction and universal property is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2345 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_inf_of_normal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
(hK : K.Normal) : (H ∩ K).normalCore = H.normalCore ∩ K
```

Read back by a model that saw only the Lean. For any group G and subgroups H and K of G , if K is a normal subgroup of G , then the normal core of the intersection $H \cap K$ is equal to the intersection of the normal core of H with K .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_inf_of_normal {G : Type*} [Group G] (H K : Subgroup G) (hK : K.Normal) :
(H ∩ K).normalCore = H.normalCore ∩ K := by
  letI : K.Normal := hK
  have hKcore : K.normalCore = K := Subgroup.normalCore_eq_self (H := K)
  rw [← hKcore]
  apply le_antisymm
  · have hN : (H ∩ K.normalCore).normalCore.Normal :=
      Subgroup.normalCore_normal (H ∩ K.normalCore)
    letI : (H ∩ K.normalCore).normalCore.Normal := hN
    have hH : (H ∩ K.normalCore).normalCore ≤ H.normalCore :=
      (Subgroup.normal_le_normalCore
        (H := H) (N := (H ∩ K.normalCore).normalCore)).2
      ((Subgroup.normalCore_le (H ∩ K.normalCore)).trans inf_le_left)
    have hK' : (H ∩ K.normalCore).normalCore ≤ K.normalCore :=
      (Subgroup.normalCore_le (H ∩ K.normalCore)).trans inf_le_right
    exact le_inf hH hK'
  · have hM : (H.normalCore ∩ K.normalCore).Normal := inferInstance
    letI : (H.normalCore ∩ K.normalCore).Normal := hM
    apply (Subgroup.normal_le_normalCore
      (H := H ∩ K.normalCore) (N := H.normalCore ∩ K.normalCore)).2
    exact inf_le_inf (Subgroup.normalCore_le H) le_rfl
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms.ms.me__8940ab06

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8940ab06c94c567f

4.56 056. infimum_normal_subgroups

The problem

Prove that the intersection of an arbitrary indexed family of normal subgroups of a group is a normal subgroup, without assuming that the indexing type is nonempty.

Lean statement

```
theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Retains the parent's indexed family of normal subgroups and reuses `Subgroup.normal_iInf_normal` in the nonempty branch, while adding the empty-index/top-subgroup branch.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes the Nonempty I assumption and handles the new empty-index case by showing the infimum is T and hence normal. The parent lemma remains useful only for the nonempty branch, while the empty case requires new reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Removing the nonemptiness assumption creates a genuine empty-index case, where the infimum is T and normality must be established separately. The parent theorem only handles the nonempty branch and does not supply this edge-case reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes to possibly empty index types. It must split on emptiness and prove that the empty infimum is T and hence normal, which the parent's nonempty-case proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1900 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem infimum_normal_subgroups.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) : (⋂ i, H i).Normal
```

Read back by a model that saw only the Lean. *Let G be a group, let I be an index type, and let H assign to each $i \in I$ a subgroup $H(i)$ of G . If $H(i)$ is a normal subgroup of G for every $i \in I$, then the infimum (intersection) of all the subgroups $H(i)$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
  classical
  cases isEmpty_or_nonempty I with
  | inl hI =>
    letI := hI
    have htop : (⋂ (i : I), H i) = (τ : Subgroup G) := by
      simp
    have hnormal : Normal (τ : Subgroup G) := by
      infer_instance
    rw [htop]
    exact hnormal
  | inr hI =>
    letI := hI
    have hnormal : Normal (⋂ (i : I), H i) :=
      Subgroup.normal_iInf_normal hH
    exact hnormal
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__1e63c082

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 29affb36027eda68

4.57 057. normalCore_greatest_normal_subgroup

The problem

For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Lean statement

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Uses the parent's arbitrary-intersection normality checkpoint via repeated applications of `Subgroup.normal_iInf_normal`, but consumes it inside the normal-core greatest-element characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine greatest-element characterization of the normal core, including normality, containment, and maximality among normal subgroups. This is substantially different mathematics from merely proving that an indexed intersection of normal subgroups is normal, so recalling the parent does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from proving normality of an arbitrary indexed intersection to characterizing the normal core via its normality, containment, and greatest-subgroup property. Its key final argument uses the normal-core universal property, which is not supplied by the parent's intersection-normality proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the task from proving normality of an indexed intersection to proving the normal core's full greatest-normal-subgroup characterization. Its proof uses normal-core-specific lemmas and a universal-property argument, so recalling the parent's exact proof does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propect, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2299 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_greatest_normal_subgroup.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore
```

Read back by a model that saw only the Lean. *For every group G and every subgroup H of G , the normal core $H.normalCore$ is a normal subgroup of G , $H.normalCore$ is contained in H , and for every subgroup N of G , if N is normal in G and N is contained in H , then N is contained in $H.normalCore$.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by  
  refine ⟨H.normalCore_normal, H.normalCore_le, ?_⟩  
  intro N hN hNH  
  letI : N.Normal := hN  
  exact Subgroup.normal_le_normalCore.mpr hNH
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__306825ab

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 306825ab3f3880b7

4.58 058. normal_core_characterization

The problem

For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Lean statement

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the arbitrary-intersection normality checkpoint, consumed for the constant family whose intersection is the normal core.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from closure of an arbitrary infimum of normal subgroups to the normal core's maximality characterization. Its proof requires the new containment and universal maximality arguments, so recalling the parent theorem is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality of an indexed infimum to the universal characterization of a subgroup's normal core. Although it reuses the parent's normal-infimum lemma for one conjunct, the containment and greatest-normal-subgroup argument require distinct reasoning, so recalling the parent does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine greatest-normal-subgroup characterization: the maximality clause requires the new normal-core containment lemma and is not supplied by the parent. Although it reuses the parent's normal-infimum lemma for one conjunct, recalling the parent does not solve the characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2109 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_core_characterization.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore
```

Read back by a model that saw only the Lean. *For a group G and a subgroup H of G , the normal core of H is a normal subgroup of G , is contained in H , and every normal subgroup N of G that is contained in H is also contained in the normal core of H .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore  $\wedge$  H.normalCore  $\leq$  H  $\wedge$ 
     $\forall$  N : Subgroup G, Normal N  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by
  have h_inter : Normal (H  $\cap$  H.normalCore) := by
    exact Subgroup.normal_iInf_normal (fun _ => H.normalCore_normal)
  have hcore_normal : Normal H.normalCore := by
    simpa using h_inter
  have hcore_contained : H.normalCore  $\leq$  H := by
    exact H.normalCore_le
  have hmax :  $\forall$  N : Subgroup G, Normal N  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by
    intro N hN hNH
    let I : Normal N := hN
    exact Subgroup.normal_le_normalCore.mpr hNH
  exact (hcore_normal, hcore_contained, hmax)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__55131fb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 55131fb71d8c9551

4.59 059. indexedInf_intersection_normal

The problem

Prove that the intersection of a nonempty indexed family of normal subgroups of a group with another normal subgroup is a normal subgroup.

Lean statement

```
theorem indexedInf_intersection_normal {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (K : Subgroup G)
  (hH : ∀ i : I, Normal (H i)) (hK : Normal K) :
  Normal ((⋂ (i : I), H i) ∩ K)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the indexed-infimum normality checkpoint via `Subgroup.normal_iInf_normal`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to normality of an intersection and must combine the indexed-infimum normality checkpoint with ‘hK’ to prove conjugation closure componentwise. Although it reuses the parent’s lemma for the required local checkpoint, the final intersection argument is genuinely new mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new intersection-normality claim, using the parent-style indexed-infimum checkpoint and separately combining it with normality of K via componentwise conjugation. The parent’s conclusion alone does not supply the final intersection argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving normality of an intersection, using conjugation closure for both the indexed infimum and K. Although it reuses the parent’s indexed-infimum checkpoint, the final intersection argument is new and satisfies the mutation_easy plan.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1893 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem indexedInf_intersection_normal.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  [inst_1 : Nonempty I] (H : I → Subgroup G) (K : Subgroup G) (hH : ∀ (i : I), (H i).Normal) (hK : K.Normal) :
  ((⋂ i, H i) ⋂ K).Normal
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, let H be a family of subgroups of G indexed by I , and let K be a subgroup of G . If every subgroup $H(i)$ is normal in G and K is normal in G , then the intersection of all the subgroups $H(i)$, intersected with K , is normal in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem indexedInf_intersection_normal {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (K : Subgroup G)
  (hH : ∀ i : I, Normal (H i)) (hK : Normal K) :
  Normal ((⋂ (i : I), H i) ⋂ K) := by
  have hInf : Normal (⋂ (i : I), H i) :=
    Subgroup.normal_iInf_normal (fun i => hH i)
  have hIntersection : Normal ((⋂ (i : I), H i) ⋂ K) := by
    constructor
    intro g hg h
    exact ⟨hInf.conj_mem g hg.1 h, hK.conj_mem g hg.2 h⟩
  exact hIntersection
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__bb03dfff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint ed5878fde5a25628

4.60 060. normal_inf_normal_specialization

The problem

If H and K are normal subgroups of a group G , prove that their intersection $H \cap K$ is normal in G .

Lean statement

```
theorem normal_inf_normal_specialization {G : Type*} [Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplied the arbitrary-family normality checkpoint used in hFamily.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine two-subgroup specialization: it constructs a Boolean-indexed family, proves its infimum is definitionally/equationally $H \cap K$, and only then applies the arbitrary-family normality lemma. Memorizing the parent proof does not supply the family construction or the indexed-infimum identification.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine two-subgroup specialization: it instantiates normality of an indexed infimum over Bool and separately proves that this infimum equals $H \cap K$. Recalling the parent's one-line application does not supply the nontrivial infimum identification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuine two-index encoding and a separate proof that the Boolean-indexed infimum equals $H \cap K$. The parent theorem supplies only normality of the indexed infimum, so recalling the parent proof does not finish the specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2931 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_inf_normal_specialization.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal
```

Read back by a model that saw only the Lean. *Let G be a group and let H and K be subgroups of G . If H and K are both normal subgroups of G , then their intersection $H \cap K$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_inf_normal_specialization {G : Type*} [Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal := by
  let F : Bool → Subgroup G := fun b => if b then H else K
  have hFamily : (Π b : Bool, F b).Normal :=
    Subgroup.normal_iInf_normal (fun b => by
      cases b <|> simp [F, hH, hK])
  have hTwo : (Π b : Bool, F b) = H ∩ K := by
    apply le_antisymm
    · refine le_inf ?_ ?_
      · exact iInf_le_of_le true (by simp [F])
      · exact iInf_le_of_le false (by simp [F])
    · refine le_iInf fun b => ?_
      cases b <|> simp [F]
  rw [← hTwo]
  exact hFamily
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__c5e2d40f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c5e2d40f0afcd6a

4.61 061. quotient_finset_disjoint_product_transport

The problem

For a commutative group A , a subgroup B , disjoint finite sets s, t of indices, and a function f from the index set to A , show that the product of the quotient images of $\prod_{i \in s} f(i)$ and $\prod_{i \in t} f(i)$ equals the quotient image of $\prod_{i \in s \cup t} f(i)$.

Lean statement

```
theorem quotient_finset_disjoint_product_transport {A ι : Type*} [CommGroup A] [DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj :
  Disjoint s t) (f : ι → A) :
  ((s.prod f : A) : A / B) * ((t.prod f : A) : A / B) =
  ((s ∪ t).prod f : A) : A / B
```

Parent. Let A be a commutative group, B a subgroup of A , s a finite set of indices, and f a function assigning an element of A to each index. Prove that the image in the quotient group A/B of the product of $f(i)$ over all $i \in s$ equals the product in A/B of the images of $f(i)$ over all $i \in s$.

Parent in Lean

```
theorem quotient_finset_product_transport {A ι : Type*} [CommGroup A] (B : Subgroup A) (s : Finset ι) (f : ι → A) :
  ((s.prod f : A) : A / B) =
  s.prod (fun i => (f i : A / B))
```

What it contributes. Supplies the quotient transport equalities `hs`, `ht`, and `hunions`; the new final obligation is the disjoint-union product decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from transporting one finite product to a disjoint-union decomposition, requiring the independent ‘`Finset.prod_union`’ argument in addition to quotient transport. The parent lemma supplies normalization but does not establish the decomposition itself.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine disjoint-index product decomposition, requiring `Finset.prod_union` in addition to quotient transport. Although it reuses the parent’s transport argument for each factor, recalling the parent alone does not establish the union identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a disjoint-union decomposition and requires the new mathematical step ‘`Finset.prod_union hdisj`’, beyond the parent’s single-set quotient transport. Although it reuses quotient transport for each factor, recalling the parent alone does not prove the union/product identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1986 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_finset_disjoint_product_transport.extracted_1_1.{u_2, u_1} {A : Type u_1} {ι : Type u_2}
  [inst : CommGroup A] [inst_1 : DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj : Disjoint s t) (f : ι → A) :
  ↑(s.prod f) * ↑(t.prod f) = ↑((s ∪ t).prod f)
```

Read back by a model that saw only the Lean. *Let A be a commutative group and ι a type with decidable equality. For any subgroup B of A , finite subsets s and t of ι that are disjoint, and any function $f : \iota \rightarrow A$, the product in the quotient group A/B of the s -indexed product of f and the t -indexed product of f equals the image in A/B of the $(s \cup t)$ -indexed product of f .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Subgroup Set Polynomial Ideal

theorem quotient_finset_disjoint_product_transport {A ι : Type*} [CommGroup A] [DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj : Disjoint s t) (f : ι → A) :
  ((s.prod f : A) : A / B) * ((t.prod f : A) : A / B) =
  (((s ∪ t).prod f : A) : A / B) := by
  have hs : ((s.prod f : A) : A / B) = s.prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B s (f := f))
  have ht : ((t.prod f : A) : A / B) = t.prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B t (f := f))
  have hunion : (((s ∪ t).prod f : A) : A / B) =
    (s ∪ t).prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B (s ∪ t) (f := f))
  rw [hs, ht, hunion]
  exact (Finset.prod_union hdisj).symm
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me.me.me.me__3331197a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 01b51bd9ea149c2d

4.62 062. quotient_finset_product_transport

The problem

Let A be a commutative group, B a subgroup of A , s a finite set of indices, and f a function assigning an element of A to each index. Prove that the image in the quotient group A/B of the product of $f(i)$ over all $i \in s$ equals the product in A/B of the images of $f(i)$ over all $i \in s$.

Lean statement

```
theorem quotient_finset_product_transport {A 1 : Type*} [CommGroup A] (B : Subgroup A) (s : Finset 1) (f : 1 → A) :
  ((s.prod f : A) : A / B) =
    s.prod (fun i => (f i : A / B))
```

Parent. For a commutative group and subgroup, the quotient projection of a finite list product equals the product of the quotient projections.

Parent in Lean

```
theorem quotient_list_product_transport {A : Type*} [CommGroup A] (B : Subgroup A) (l : List A) :
  ((l.prod : A) : A / B) =
    (l.map (fun a : A => (a : A / B))).prod
```

What it contributes. Generalizes the parent list-product transport law to a Finset-indexed product and uses the quotient projection/product compatibility checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes from ordered-list induction to a finite-set product law over an arbitrary index type, and its proof requires the quotient group's finset product homomorphism theorem. The parent's list induction proof does not directly supply this result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the transport domain from ordered lists to arbitrary finite sets and functions, proved via the finite-product quotient homomorphism lemma. A solver recalling the parent's list induction proof would not directly obtain this Finset statement or its library-based proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes list transport to arbitrary finite-indexed products, requiring the quotient homomorphism's Finset product theorem rather than recalling the parent's list induction. Its proof uses a distinct substantive lemma, not decoration or a forced wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1975 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_finset_product_transport.extracted_1_1.{u_2, u_1} {A : Type u_1} {ι : Type u_2} [inst : CommGroup A]
  (B : Subgroup A) (s : Finset ι) (f : ι → A) : ↑(s.prod f) = ∏ i ∈ s, ↑(f i)
```

Read back by a model that saw only the Lean. *For any commutative group A , subgroup B of A , finite set s of indices, and function $f : \iota \rightarrow A$, the coset of the product of $f i$ over all i in s equals the product, over all i in s , of the cosets of $f i$ in the quotient group A/B .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_finset_product_transport {A ι : Type*} [CommGroup A] (B : Subgroup A) (s : Finset ι) (f : ι → A) :
  ((s.prod f : A) : A / B) =
    s.prod (fun i => (f i : A / B)) := by
  simpa using (QuotientGroup.mk_prod B s (f := f))
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me.me.me__af6e130a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint cf624e07f7127f1b

4.63 063. quotient_list_product_reordering

The problem

For a commutative group quotient, finite products of quotient representatives are invariant under reordering.

Lean statement

```
theorem quotient_list_product_reordering {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ l l' : List A,
    l.Perm l' →
      (l.map (fun a : A => (a : A / B))).prod =
        (l'.map (fun a : A => (a : A / B))).prod
```

Parent. Let A be an abelian group and let B be a subgroup of A . Prove that A/B is abelian.

Parent in Lean

```
theorem Dummit_Foote_exercise_3_1_3a {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ a b : A / B, a * b = b * a := by
```

What it contributes. Uses the parent’s commutative-group quotient setting; the child upgrades binary commutativity to a representative-list product reordering law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes binary commutativity to invariance of arbitrary finite quotient-group products under list permutation. Its proof uses permutation mapping and ‘List.Perm.prod_eq’, requiring a genuinely different finite-product argument than the parent’s direct ‘mul_comm’ application.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely broader finite-product reordering law for lists via permutation transport and ‘List.Perm.prod_eq’, rather than reusing the parent’s binary commutativity proof. A solver remembering the parent would still need the distinct list-permutation/product lemma.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes binary quotient commutativity into a finite-product reordering law for arbitrary Perm-related lists. Its proof uses mapped-list permutation and ‘List.Perm.prod_eq’, requiring a different finite-product lemma rather than recalling the parent’s ‘mul_comm’ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1964 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_list_product_reordering.extracted_1_1 : ∀ {A : Type u_1} [inst : CommGroup A] (B : Subgroup A)
  (l l' : List A), l.Perm l' → (List.map (fun a => ↑a) l).prod = (List.map (fun a => ↑a) l').prod
```

Read back by a model that saw only the Lean. *For every type A equipped with a commutative group structure, every subgroup B of A , and all lists l and l' of elements of A , if l is a permutation of l' , then the products in the quotient group A/B of the images of the elements of l and l' are equal.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_list_product_reordering {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ l l' : List A,
    l.Perm l' →
      (l.map (fun a : A => (a : A / B))).prod =
        (l'.map (fun a : A => (a : A / B))).prod := by
  intro l l' h
  have hq : (l.map (fun a : A => (a : A / B))).Perm
    (l'.map (fun a : A => (a : A / B))) := h.map _
  exact hq.prod_eq
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me__09c322d9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a08ee45cfe6afbae

4.64 064. prime_order_element_of_dvd_card

The problem

Let G be a finite abelian group and let p be prime. If p divides the order of G , show that G contains an element of order p .

Lean statement

```
theorem prime_order_element_of_dvd_card {G : Type*} [CommGroup G] [Fintype G] {p : ℕ}
  (hp : Nat.Prime p) (hdiv : p ∣ Fintype.card G) :
  ∃ x : G, orderOf x = p
```

Parent. Use Cauchy's Theorem and induction to show that a finite abelian group has a subgroup of order n for each positive divisor n of its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_3.4.4 {G : Type*} [CommGroup G] [Fintype G] {n : ℕ}
  (hn : n ∣ (card G)) :
  ∃ (H : Subgroup G) (H_fin : Fintype H), @card H H_fin = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from constructing a subgroup of cardinality n to constructing an element of prime order p . Its proof uses Cauchy's prime-order theorem directly and does not recall or adapt the parent's induction or Sylow subgroup argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence of a subgroup of order n to existence of an element of prime order p , and proves it via the finite-group Cauchy theorem. This is independent of the parent's truncated induction and quotient/Sylow construction, satisfying the mutation_easy plan.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuinely different existence claim: it constructs an element of prime order using Cauchy's theorem, rather than constructing a subgroup of prescribed order via the parent's induction and Sylow argument. Its proof is independent of the parent and uses both prime-divisibility hypotheses substantively.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2200 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_order_element_of_dvd_card.extracted_1_1.{u_1} {G : Type u_1} [inst : CommGroup G] [inst_1 : Fintype G]
  {p : ℕ} (hp : Nat.Prime p) (hdiv : p ∣ card G) : ∃ x, orderOf x = p
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let p be a prime natural number, and suppose that p divides the cardinality of G . Then there exists an element x of G whose order is p .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem prime_order_element_of_dvd_card {G : Type*} [CommGroup G] [Fintype G] {p : ℕ}
  (hp : Nat.Prime p) (hdiv : p ∣ Fintype.card G) :
  ∃ x : G, orderOf x = p := by
  letI : Fact (Nat.Prime p) := ⟨hp⟩
  have hp_dvd_card : p ∣ Fintype.card G := hdiv
  have hresult : ∃ x : G, orderOf x = p :=
    exists_prime_orderOf_dvd_card (G := G) (p := p) hp_dvd_card
  exact hresult
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_4_4__me__e33d29b2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e33d29b28d212897

4.65 065. exists_order_two_subgroup_with_nontrivial_quotient

The problem

Every finite commutative group of order $2n$, where $2 \leq n \leq 10$, has a normal subgroup of order 2 such that the quotient group has order n and is nontrivial.

Lean statement

```
theorem exists_order_two_subgroup_with_nontrivial_quotient {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG :
  ↪ Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H)
```

Parent. Prove that every finite abelian group of order $2n$, where $2 \leq n \leq 10$, has a nontrivial proper normal subgroup.

Parent in Lean

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤
  ↪ 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ ⊤
```

What it contributes. Reuses the parent's finite counting and Sylow order-two subgroup construction, then consumes that subgroup in the quotient cardinality and nontriviality arguments.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child preserves the Sylow order-two construction but adds genuinely new quotient mathematics: it proves the exact quotient cardinality via the subgroup-cardinality formula and derives quotient nontriviality from properness. The parent's proof does not supply either quotient conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive quotient mathematics absent from the parent: it derives the exact quotient cardinality using the subgroup cardinality formula and proves quotient nontriviality from properness. Although it reuses the Sylow construction and normality argument, recalling the parent does not supply these quotient-level conclusions.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces genuinely new quotient mathematics: it derives the exact cardinality of G / H and proves quotient nontriviality, neither of which follows by recalling the parent's proof. The shared Sylow construction is only the setup for this new conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1929 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_order_two_subgroup_with_nontrivial_quotient.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst.1 : CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : card G = 2 * n) :
  ∃ H, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H)
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let n be a natural number such that $2 \leq n \leq 10$, and suppose that the cardinality of G is $2n$. Then there exists a normal subgroup H of G whose cardinality is 2, such that the quotient group G/H has cardinality n and is nontrivial.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_order_two_subgroup_with_nontrivial_quotient {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG :
  card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H) := by
  classical
  have hcount : Nat.card G = 2 * n := by
    simpa [Nat.card_eq_fintype_card] using hG
  letI : Fact (Nat.Prime 2) := ⟨by decide⟩
  have hpow : 2 ^ 1 ∣ Nat.card G := by
    rw [hcount]
    rcases hn with ⟨hnlo, hnhi⟩
    interval_cases n <|> norm_num
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) (p := 2) (n := 1) hpow
  have hHcard : Nat.card H = 2 := by
    simpa using hH
  have hnormal : H.Normal := by
    constructor
    intro x hx g
    rw [CommGroup.mul_comm g x]
    simpa [mul_assoc] using hx
  have hquotcard : Nat.card (G / H) = n := by
    have hmul := Subgroup.card_eq_card_quotient_mul_card_subgroup H
    rw [hcount, hHcard] at hmul
    omega
  have hproper : H ≠ ⊤ := by
    intro htop
    rw [htop] at hHcard
    simp at hHcard
    omega
  have hquotnontrivial : Nontrivial (G / H) := by
    exact QuotientGroup.nontrivial_iff.mpr hproper
  exact ⟨H, hnormal, hHcard, hquotcard, hquotnontrivial⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me.me__7c82a3d7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 88765ef7f846f66d

4.66 066. exists_proper_nontrivial_normal_subgroup_of_bounded_double_order

The problem

Prove that every finite abelian group of order $2n$, where $2 \leq n \leq 10$, has a nontrivial proper normal subgroup.

Lean statement

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ G
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
(hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Reuses the parent's finite-counting checkpoint and Sylow subgroup-counting surface, generalized from the fixed order 462 case to bounded orders $2*n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the theorem: it gives an explicit order-2 proper nontrivial normal subgroup for a bounded family of finite abelian groups, using Sylow existence and commutativity. This does not rely on recalling the parent's fixed-order 462 Sylow-11 obstruction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the group order and proves existence via a Sylow-2 subgroup, then uses commutativity to establish normality and cardinality-based nontriviality/properness. This is not recoverable by recalling the parent's fixed-order Sylow-11 nonsimplicity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded parameterized theorem with an explicit Sylow-2 subgroup witness, commutativity-based normality, and separate nontriviality/properness arguments. Its proof does not recall or reuse the parent's fixed-order Sylow-11 obstruction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1905 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order.extracted_1_1.{u_1} {G : Type u_1}
  [inst : Fintype G] [inst_1 : CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : card G = 2 * n) :
  ∃ H, H.Normal ∧ H ≠ 1 ∧ H ≠ τ
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let n be a natural number with $2 \leq n \leq 10$, and suppose that the cardinality of G is $2n$. Then there exists a normal subgroup H of G such that H is neither the trivial subgroup nor the whole group G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤
  <- 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ τ := by
  classical
  have hcount : Nat.card G = 2 * n := by
    simpa [Nat.card_eq_fintype_card] using hG
  haveI : Fact (Nat.Prime 2) := <by decide>
  have hpow : 2 ^ 1 ∣ Nat.card G := by
    rw [hcount]
    rcases hn with <hnlo, hnhi>
    interval_cases n <|> norm_num
  obtain (H, hH) := Sylow.exists_subgroup_card_pow_prime (G := G) (p := 2) (n := 1) hpow
  have hHcard : Nat.card H = 2 := by
    simpa using hH
  have hnormal : H.Normal := by
    constructor
    intro x hx g
    rw [CommGroup.mul_comm g x]
    simpa [mul_assoc] using hx
  have hnontrivial : H ≠ 1 := by
    intro hbot
    rw [hbot] at hHcard
    simp at hHcard
  have hproper : H ≠ τ := by
    intro htop
    rw [htop] at hHcard
    have hGcard : Nat.card G = 2 := by
      simpa using hHcard
    rw [hcount] at hGcard
    omega
  exact (H, hnormal, hnontrivial, hproper)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__2afb89cf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 92e34d98d1b53b3b

4.67 067. order_462_subgroup_gaussian_divisibility

The problem

Every finite group of order 462 contains a subgroup of odd order n such that $(1+i)^2$ divides $n^2 - 1$ in the Gaussian integers.

Lean statement

```
theorem order_462_subgroup_gaussian_divisibility {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, Odd (Nat.card H) ∧
    ((1, 1) : GaussianInt) ^ 2 ∣ (Nat.card H ^ 2 - 1 : GaussianInt)
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4.5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Reuses the parent's order-462 finite-group domain and cardinality normalization checkpoint, while changing the conclusion to an explicit Sylow subgroup and Gaussian divisibility target.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses Sylow existence directly to construct a subgroup of cardinality 11, then proves a new Gaussian-integer divisibility property by arithmetic. This is not supplied by the parent's contradiction and Sylow-counting argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses Sylow existence to construct a subgroup of cardinality 11, then proves oddness and a Gaussian-integer divisibility fact. None of this follows from or recalls the parent's non-simplicity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses Sylow subgroup existence directly to construct a subgroup of cardinality 11, then proves oddness and a new Gaussian-integer divisibility statement. This reasoning is not supplied by recalling the parent's non-simplicity conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2937 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_462_subgroup_gaussian_divisibility.extracted_1.1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst.1 : Group G] (hG : card G = 462) :
  ∃ H, Odd (Nat.card ↑H) ∧ { re := 1, im := 1 } ^ 2 | ↑(Nat.card ↑H) ^ 2 - 1
```

Read back by a model that saw only the Lean. *Let G be a finite group with $|G| = 462$. Then there exists a subgroup H of G such that the cardinality of H is odd and the Gaussian integer $(1 + i)^2$ divides $|H|^2 - 1$, where $|H|$ is viewed as a Gaussian integer.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem order_462_subgroup_gaussian_divisibility {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, Odd (Nat.card H) ∧
    ((1, 1) : GaussianInt) ^ 2 | (Nat.card H ^ 2 - 1 : GaussianInt) := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : Nat.card G = 462 := by
    simpa [Nat.card_eq_fintype_card] using hG
  have hdiv : 11 ^ 1 | Nat.card G := by
    rw [hcard]
    norm_num
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) 11 (n := 1) hdiv
  have h_cardH : Nat.card H = 11 := by
    simpa using hH
  refine ⟨H, ?_, ?_⟩
  · rw [h_cardH]
    norm_num
  · rw [h_cardH]
    have hsq : ((1, 1) : GaussianInt) ^ 2 = ⟨0, 2⟩ := by
      rw [pow_two]
      apply Zsqrtd.ext <|> norm_num [Zsqrtd.re_mul, Zsqrtd.im_mul]
    rw [hsq]
    refine ⟨⟨0, -60⟩, ?_⟩
    apply Zsqrtd.ext <|> norm_num [Zsqrtd.re_mul, Zsqrtd.im_mul]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__58a9e5ec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 58a9e5ecf3ba4f34

4.68 068. order_462_has_nontrivial_proper_normal_subgroup

The problem

Let G be a finite group of order 462. Prove that G has a nontrivial proper normal subgroup.

Lean statement

```
theorem order_462_has_nontrivial_proper_normal_subgroup {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ ⊤ := by
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Supplied the order-462 Sylow-11 counting strategy: divisibility of the Sylow count, congruence modulo 11, uniqueness, and normality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's contradiction-to-simplicity conclusion with an explicit Sylow-11 subgroup witness. Beyond counting Sylow subgroups, it proves normality from uniqueness and separately establishes non-triviality and properness using divisibility and the exact 11-adic factor of 462.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's contradiction-to-simplicity result with an explicit existential construction of a unique Sylow-11 subgroup and proves it is normal, nontrivial, and proper. These final structural properties require reasoning beyond merely recalling the parent's negated-simplicity conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's contradiction-to-nonsimplicity with an explicit Sylow-11 subgroup witness, requiring new proofs of uniqueness, normality, nontriviality, and properness. A memorized proof of the parent does not directly provide this existential structural conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2956 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_462_has_nontrivial_proper_normal_subgroup.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst_1 : Group G] (hG : card G = 462) : ∃ H, H.Normal ∧ H ≠ 1 ∧ H ≠ τ := by
```

Read back by a model that saw only the Lean. *For every finite group G with $|G| = 462$, there exists a normal subgroup H of G such that H is neither the trivial subgroup nor G itself.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem order_462_has_nontrivial_proper_normal_subgroup {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ τ := by
  classical
  have hcard : Nat.card G = 462 := by
    simpa [Nat.card_eq_fintype_card] using hG
  have hp : Nat.Prime 11 := by norm_num
  letI : Fact (11 : ℕ).Prime := ⟨hp⟩
  obtain ⟨P⟩ : Nonempty (Sylow 11 G) := Sylow.nonempty
  set n : ℕ := Nat.card (Sylow 11 G) with hn
  have hmod : n ≡ 1 [MOD 11] := by
    simpa [n] using (card_sylow_modEq_one 11 G)
  have hdiv_index : n ∣ P.index := by
    simpa [n] using (Sylow.card_dvd_index (p := 11) (G := G) P)
  have hindex_dvd : P.index ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
  have hdiv_card : n ∣ Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
  have hdiv_mul : n ∣ 11 * 42 := by
    rw [hcard] at hdiv_card
    simpa using hdiv_card
  have hnotdiv : ¬11 ∣ n := by
    simpa [n] using (not_dvd_card_sylow 11 G)
  have hcoprime : Nat.Coprime n 11 := by
    simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
  have hdiv42 : n ∣ 42 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
  have hnle : n ≤ 42 := Nat.le_of_dvd (by norm_num) hdiv42
  have hn_eq_one : n = 1 := by
    rw [Nat.ModEq] at hmod
    obtain ⟨k, hk⟩ := hdiv42
    have hnpos : 0 < n := by omega
    have hk_le : k ≤ 42 := by nlinarith [hk]
    interval_cases k <|> omega
  have hsub : Subsingleton (Sylow 11 G) := by
    exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
  have hnormal : P.Normal := by
    letI : Subsingleton (Sylow 11 G) := hsub
    exact Sylow.normal_of_subsingleton P
  have hdivF : 11 ∣ Fintype.card G := by
    rw [hG]
    norm_num
  have hdivF' : 11 ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using hdivF
  have hnebot : (P : Subgroup G) ≠ 1 := P.ne_bot_of_dvd_card hdivF'
  have hfac : Nat.factorization 462 11 = 1 := by
    rw [show (462 : ℕ) = 11 * 42 by norm_num,
      Nat.factorization_mul (by norm_num) (by norm_num)]
    rw [Nat.Prime.factorization (by norm_num)]
    simp [Nat.factorization_eq_zero_of_not_dvd]
  have hPcard : Nat.card (P : Subgroup G) = 11 := by
    have h := P.card_eq_multiplicity
    rw [hcard, hfac] at h
    simpa using h
  have hnetop : (P : Subgroup G) ≠ τ := by
    intro ht
```

```
have hcontra : Nat.card G = 11 := by
  simpa [ht, Nat.card_eq_fintype_card] using hPcard
  omega
exact ⟨(P : Subgroup G), hnormal, hnebot, hnetop⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__c9e0395d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c9e0395dd85c639f

4.69 069. idempotent_sum_commute

The problem

Let R be a ring and let $a, b \in R$. Suppose that $a^2 = a$, $b^2 = b$, and $(a + b)^2 = a + b$. Prove that $ab = ba$.

Lean statement

```
theorem idempotent_sum_commute {R : Type*} [Ring R] (a b : R) (ha : a ^ 2 = a) (hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
```

Parent. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a * b = b * a := by
```

What it contributes. The parent supplies the idempotent-ring expansion strategy and commutativity target; the child localizes it to hypotheses ha , hb , and hab and closes through multiplied mixed-term identities.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes idempotence to a , b , and $a+b$, then derives $a*b = b*a$ through mixed-term cancellation and multiplication by the idempotents. The parent's universal proof relies on deriving characteristic two from idempotence of arbitrary elements, which the child cannot use and does not reproduce.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses only local idempotence of a , b , and $a+b$, then derives the mixed-term cancellation and commutativity by multiplying and cancelling. The parent's global argument via idempotence of $-a$ is unavailable, so recalling the parent does not substantially solve this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes idempotence to a , b , and $a+b$ and derives commutativity through mixed-term cancellation and multiplication, without using the parent's global characteristic-two argument. Its proof is not recoverable by recalling the parent's theorem or final lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2000 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem idempotent_sum_commute.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R] (a b : R) (ha : a ^ 2 = a)
(hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
```

Read back by a model that saw only the Lean. *Let R be a ring and let a, b be elements of R . If $a^2 = a$, $b^2 = b$, and $(a + b)^2 = a + b$, then $ab = ba$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem idempotent_sum_commute {R : Type*} [Ring R] (a b : R) (ha : a ^ 2 = a) (hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
  <| a := by
  have haa : a * a = a := by
    simpa [pow_two] using ha
  have hbb : b * b = b := by
    simpa [pow_two] using hb
  have h_expand : (a + b) ^ 2 = a * a + a * b + b * a + b * b := by
    calc
      (a + b) ^ 2 = (a + b) * (a + b) := by simp [pow_two]
      _ = a * (a + b) + b * (a + b) := by rw [add_mul]
      _ = (a * a + a * b) + (b * a + b * b) := by rw [mul_add, mul_add]
      _ = a * a + a * b + b * a + b * b := by ac_rfl
  have h_sq : a * a + a * b + b * a + b * b = a + b := by
    calc
      a * a + a * b + b * a + b * b = (a + b) ^ 2 := h_expand.symm
      _ = a + b := hab
  have h_eq : a + a * b + b * a + b = a + b := by
    simpa [haa, hbb] using h_sq
  have h_mixed : a * b + b * a = 0 := by
    have h_cancel : a * b + b * a + b = b := by
      have h : a + (a * b + b * a + b) = a + b := by
        calc
          a + (a * b + b * a + b) = a + a * b + b * a + b := by ac_rfl
          _ = a + b := h_eq
      exact add_left_cancel h
    have h : (a * b + b * a) + b = 0 + b := by
      simpa using h_cancel
    exact add_right_cancel h
  have h_left : a * b + a * (b * a) = 0 := by
    simpa [mul_add, ← mul_assoc, haa] using congrArg (fun x : R => a * x) h_mixed
  have h_right : a * (b * a) + b * a = 0 := by
    simpa [add_mul, mul_assoc, haa] using congrArg (fun x : R => x * a) h_mixed
  apply sub_eq_zero.mp
  calc
    a * b - b * a = (a * b + a * (b * a)) - (a * (b * a) + b * a) := by abel
    _ = 0 - 0 := by rw [h_left, h_right]
    _ = 0 := by simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__me__09c50d0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint c0d16b1a74e46a74

4.70 070. universally_idempotent_frobenius

The problem

Let R be a ring such that $a^2 = a$ for every $a \in R$. Prove that $(a + b)^2 = a^2 + b^2$ for all $a, b \in R$.

Lean statement

```
theorem universally_idempotent_frobenius {R : Type*} [Ring R] (hR : ∀ a : R, a ^ 2 = a) : ∀ a b : R, (a + b) ^ 2 = a ^ 2 + b ^ 2
```

Parent. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a*b = b*a := by
```

What it contributes. Adapts the parent's sum-square expansion and additive cancellation checkpoints while changing the conclusion to the additive squaring law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from commutativity to Frobenius additivity and proves it by deriving mixed-term cancellation, expanding the sum square, and eliminating the cross terms. Although it shares the parent's preliminary idempotence consequences, it does not merely apply or wrap the parent's commutativity result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not a recalled wrapper or decoration: it proves Frobenius additivity by expanding the square and deriving cancellation of the mixed terms directly from idempotence of $a+b$. It reaches a different conclusion without applying the parent's commutativity theorem, satisfying the required semantic mutation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from commutativity to Frobenius additivity and explicitly derives mixed-term cancellation from idempotence of $a+b$ before completing the square expansion. Although it shares the sum-square technique with the parent, it is not a recalled proof of the parent's conclusion and satisfies the requested specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1986 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem universally_idempotent_frobenius.extracted_1_1 : ∀ {R : Type u_1} [inst : Ring R],  
  (∀ (a : R), a ^ 2 = a) → ∀ (a b : R), (a + b) ^ 2 = a ^ 2 + b ^ 2
```

Read back by a model that saw only the Lean. For every type R equipped with a ring structure, if every element a of R satisfies $a^2 = a$, then for all elements a and b of R , $(a + b)^2 = a^2 + b^2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem universally_idempotent_frobenius {R : Type*} [Ring R] (hR : ∀ a : R, a ^ 2 = a) : ∀ a b : R, (a + b) ^ 2 = a ^ 2 + b ^ 2 := by
  intro a b
  have ha : a * a = a := by
    simpa [pow_two] using hR a
  have hb : b * b = b := by
    simpa [pow_two] using hR b
  have h_expand : (a + b) ^ 2 = a * a + a * b + b * a + b * b := by
    calc
      (a + b) ^ 2 = (a + b) * (a + b) := by simp [pow_two]
      _ = a * (a + b) + b * (a + b) := by rw [add_mul]
      _ = (a * a + a * b) + (b * a + b * b) := by rw [mul_add, mul_add]
      _ = a * a + a * b + b * a + b * b := by simp [add_assoc]
  have h_sum : a * a + a * b + b * a + b * b = a + b := by
    calc
      a * a + a * b + b * a + b * b = (a + b) ^ 2 := h_expand.symm
      _ = a + b := hR (a + b)
  have h_cancel : a * b + b * a = 0 := by
    have h_sum' := h_sum
    rw [ha, hb] at h_sum'
    have h_left : a + ((a * b + b * a) + b) = a + b := by
      simpa [add_assoc] using h_sum'
    have h_middle : (a * b + b * a) + b = b := add_left_cancel h_left
    have h_right : (a * b + b * a) + b = 0 + b := by
      simpa using h_middle
    exact add_right_cancel h_right
  calc
    (a + b) ^ 2 = a * a + a * b + b * a + b * b := h_expand
    _ = a * a + (a * b + b * a) + b * b := by simp [add_assoc]
    _ = a * a + b * b := by rw [h_cancel, add_zero]
    _ = a ^ 2 + b ^ 2 := by simp [pow_two]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__me__82b7256f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropept, Quot.sound, fingerprint 5ff327b9de75ec42

4.71 071. boolean_ring_nsmul_period_two

The problem

Let R be a ring in which every element is unchanged when squared. Prove that for every natural number n and every $a \in R$, the $(n + 2)$ -fold sum of a equals the n -fold sum of a .

Lean statement

```
theorem boolean_ring_nsmul_period_two {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ n : ℕ, ∀ a : R, (n + 2) • a = n • a
```

Parent. In a ring where multiplying any quantity by itself leaves it unchanged, show that reversing the order of two factors does not change their product.

Parent in Lean

```
theorem boolean_ring_opposite_order {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ a b : R, b * a = a * b := by
```

What it contributes. Reuses the parent's Boolean-ring idempotence setup and substantive sum expansion pattern, but changes the final role to additive characteristic two and scalar periodicity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new universally quantified additive-scalar periodicity conclusion, requiring nsmul decomposition and simplification after deriving characteristic two. Although its characteristic-two lemma uses a related idempotence argument, the parent's commutativity conclusion does not supply the new scalar reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from multiplicative commutativity to additive scalar periodicity. Its proof independently derives characteristic two from idempotence and applies nsmul identities, so recalling the parent's commutativity argument does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different conclusion: additive scalar periodicity for every natural n and ring element a . Its characteristic-two lemma is derived directly from idempotence of $a+a$, then combined with nsmul arithmetic; it does not reuse the parent's multiplicative commutativity argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2824 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem boolean_ring_nsmul_period_two.extracted_1_1 : ∀ {R : Type u_1} [inst : Ring R],  
  (∀ (a : R), a * a = a) → ∀ (n : ℕ) (a : R), (n + 2) • a = n • a
```

Read back by a model that saw only the Lean. For every ring R , if every element a of R is idempotent, meaning $a \cdot a = a$, then for every natural number n and every element a of R , the $(n + 2)$ -fold sum of a equals the n -fold sum of a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem boolean_ring_nsmul_period_two {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ n : ℕ, ∀ a : R, (n + 2) • a = n • a := by
  have hchar : ∀ a : R, a + a = 0 := by
    intro a
    have hsq : (a + a) * (a + a) = a + a := hR (a + a)
    have hexpand : (a + a) * (a + a) = a * a + a * a + a * a + a * a := by
      noncomm_ring
    have hsum : a + a + a + a = a + a := by
      calc
        a + a + a + a = a * a + a * a + a * a + a * a := by rw [hR a]
        _ = (a + a) * (a + a) := hexpand.symm
        _ = a + a := hsq
    have hcancel : (a + a) + (a + a) = (a + a) + 0 := by
      calc
        (a + a) + (a + a) = a + a + a + a := by abel
        _ = a + a := hsum
        _ = (a + a) + 0 := by simp
    exact add_left_cancel hcancel
  intro n a
  have ha : a + a = 0 := hchar a
  calc
    (n + 2) • a = n • a + 2 • a := by rw [add_nsmul]
    _ = n • a + (a + a) := by rw [two_nsmul]
    _ = n • a := by rw [ha]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__ms.me__36389812

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropept, Quot.sound, fingerprint 36389812b52cf248

4.72 072. bounded_ideal_nilpotence

The problem

Let p be prime and $m > 0$. Prove that every ideal of $\mathbb{Z}/p^m\mathbb{Z}$ contained in the ideal generated by p is nilpotent.

Lean statement

```
theorem bounded_ideal_nilpotence {p m : ℕ} (I : Ideal (ZMod (p ^ m))) (hI : I ≤ (Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m))))) : IsNilpotent I := by
```

Parent. An ideal N is called nilpotent if N^n is the zero ideal for some $n \geq 1$. Prove that the ideal $p\mathbb{Z}/p^m\mathbb{Z}$ is a nilpotent ideal in the ring $\mathbb{Z}/p^m\mathbb{Z}$.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_3_37 {p m : ℕ} (_hp : p.Prime) :
  IsNilpotent (span ({p} : Set (ZMod (p ^ m)) : Ideal (ZMod (p ^ m))) := by
```

What it contributes. Supplies the principal ideal $J=(p)$ in $ZMod (p^m)$ and the power-vanishing argument via divisibility by p^m .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. Its proof requires the new monotonicity argument that powers of a subideal are bounded by powers of the principal ideal, beyond recalling the parent's nilpotence proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. Beyond reproducing the principal-ideal power calculation, it requires the new monotonicity argument that containment is preserved under ideal powers and transfers the vanishing power to the arbitrary ideal.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. It requires the new monotonicity argument that powers of a contained ideal lie below the corresponding power of the principal ideal, beyond the parent's principal-ideal computation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2262 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_ideal_nilpotence.extracted_1_1 {p m : ℕ} (hp : Nat.Prime p) (hm : 0 < m) (I : Ideal (ZMod (p ^ m)))  
(hI : I ≤ span {↑p}) : IsNilpotent I
```

Read back by a model that saw only the Lean. *Let p and m be natural numbers. If p is prime, m is positive, and I is an ideal of the ring $\text{ZMod}(p^m)$ such that I is contained in the ideal generated by the image of p in $\text{ZMod}(p^m)$, then I is nilpotent.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem bounded_ideal_nilpotence {p m : ℕ} (I : Ideal (ZMod (p ^ m))) (hI : I ≤ (Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m)))))) : IsNilpotent I := by
  let J : Ideal (ZMod (p ^ m)) := Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m)))
  have h_containment : ∀ K : Ideal (ZMod (p ^ m)), K ≤ J → IsNilpotent K := by
    intro K hK
    refine ⟨Nat.succ m, ?_⟩
    have hpowJ : J ^ Nat.succ m = 1 := by
      have hdiv : p ^ m ∣ p ^ Nat.succ m := by
        refine ⟨p, ?_⟩
        simp [Nat.pow_succ]
      have hzero : ((p : ZMod (p ^ m)) ^ Nat.succ m) = 0 := by
        have hcast : ((p ^ Nat.succ m : ℕ) : ZMod (p ^ m)) = 0 := by
          exact (ZMod.natCast_eq_zero_iff (a := p ^ Nat.succ m) (b := p ^ m)).2 hdiv
        simp [Nat.cast_pow] using hcast
      simp [J, Ideal.span_singleton_pow, hzero]
    have hpowK : K ^ Nat.succ m ≤ J ^ Nat.succ m := by
      apply pow_le_pow_left'
    exact hK
    have hzeroK : K ^ Nat.succ m ≤ 1 := by
      rw [hpowJ] at hpowK
      exact hpowK
    simp [Ideal.zero_eq_bot] using (le_antisymm hzeroK bot_le)
  exact h_containment I (by simp [J] using hI)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_3_37__me__e3db7d21

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 7925d0ab6e5efc6f

4.73 073. bounded_power_period_characterization

The problem

For natural numbers a , n , and t with $a > 1$, $0 < t$, and $t \leq n$, the power a^t is congruent to 1 modulo $a^n - 1$ if and only if $t = n$.

Lean statement

```
theorem bounded_power_period_characterization {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) : Nat.ModEq (a ^ n - 1) (a ^ t) 1 ↔ t = n
```

Parent. For natural numbers a , n , and t with $a > 1$ and $n \mid t$, show that $a^t \equiv 1 \pmod{a^n - 1}$.

Parent in Lean

```
theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) : Nat.ModEq (a ^ n - 1) (a ^ t) 1
```

What it contributes. Supplied the base congruence construction modulo $a^n - 1$ and the modular-power route, adapted here into an exact bounded characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: under $0 < t \leq n$, it proves that $a^t \equiv 1 \pmod{a^n - 1}$ forces $t = n$ using strict power bounds and remainder analysis. The parent only supplies the forward base congruence and does not provide this characterization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: under $0 < t \leq n$, it proves any congruence forces $t = n$ by bounding a^t below the modulus and comparing remainders. The parent only supplies the forward divisible-exponent congruence and does not provide this inequality-based exclusion of $t < n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse characterization: under $0 < t \leq n$, it proves the congruence forces $t = n$ using strict power bounds and remainder reasoning. Although it reuses the parent's base congruence for the easy forward direction, the parent's proof supplies no argument for this converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1888 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_period_characterization.extracted_1_1 {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) :  
  a ^ t ≡ 1 [MOD a ^ n - 1] ↔ t = n
```

Read back by a model that saw only the Lean. For natural numbers a , n , and t , if $a > 1$, $0 < t$, and $t \leq n$, then a raised to the t -th power is congruent to 1 modulo a raised to the n -th power minus 1 if and only if $t = n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem bounded_power_period_characterization {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) :  
  Nat.ModEq (a ^ n - 1) (a ^ t) 1 ↔ t = n := by  
  have hn : 0 < n := lt_of_lt_of_le ht htn  
  have hge : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)  
  have hpowgt : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha  
  have hbase : Nat.ModEq (a ^ n - 1) (a ^ n) 1 := by  
    change a ^ n % (a ^ n - 1) = 1 % (a ^ n - 1)  
  by_cases hm : a ^ n - 1 = 1  
  · rw [hm]  
    have hmod : a ^ n % 1 < 1 := Nat.mod_lt _ (by omega)  
    omega  
  · have hmgt : 1 < a ^ n - 1 := by omega  
    have hone : 1 % (a ^ n - 1) = 1 := Nat.mod_eq_of_lt (by omega)  
    calc  
      a ^ n % (a ^ n - 1) = (a ^ n - 1 + 1) % (a ^ n - 1) := by  
        rw [Nat.sub_add_cancel hge]  
      _ = 1 % (a ^ n - 1) := by  
        rw [Nat.add_mod]  
      simp [Nat.mod_eq_of_lt hmgt, hone]  
  constructor  
  · intro hcong  
    by_contra hne  
    have hlt : t < n := by omega  
    have hpowl : a ^ t < a ^ n := Nat.pow_lt_pow_right (by omega) hlt  
    have ht1 : t + 1 ≤ n := by omega  
    have hpowle : a ^ (t + 1) ≤ a ^ n := Nat.pow_le_pow_right (by omega) ht1  
    have htpowgt : 1 < a ^ t := Nat.one_lt_pow (n := t) (a := a) (by omega) ha  
    have htwo : 2 ≤ a ^ t := by omega  
    have hmul : a ^ t * 2 ≤ a ^ t * a := by  
      exact Nat.mul_le_mul_left (a ^ t) (by omega)  
    have hdouble : a ^ t + 2 ≤ a ^ t * 2 := by omega  
    have hstep2 : a ^ t + 2 ≤ a ^ (t + 1) := by  
      rw [pow_succ]  
      exact le_trans hdouble hmul  
    have hmodulus : a ^ t + 1 ≤ a ^ n - 1 := by  
      have : a ^ t + 2 ≤ a ^ n := le_trans hstep2 hpowle  
      omega  
    have hmod : a ^ t < a ^ n - 1 := by omega  
    have hgt : 1 < a ^ n - 1 := by  
      have : 1 ≤ a ^ t := Nat.one_le_pow t a (by omega)  
      omega  
    have heq : a ^ t = 1 := by  
      change a ^ t % (a ^ n - 1) = 1 % (a ^ n - 1) at hcong  
      rw [Nat.mod_eq_of_lt hmod, Nat.mod_eq_of_lt (by omega)] at hcong  
      exact hcong  
    omega  
  · intro h  
    subst t  
    exact hbase
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me__37be04a5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fin-

gerprint 8117fbe1560ccf55

4.74 074. power_shift_period_mod_sub_one

The problem

Let $a > 1$ and let k be a positive multiple of n . Show that raising a^k by one further block of n leaves the same residue modulo $a^n - 1$.

Lean statement

```
theorem power_shift_period_mod_sub_one {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ((a : ZMod (a ^ n - 1)) ^ (k + n) = (a : ZMod (a ^ n - 1)) ^ k)
```

Parent. Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Parent in Lean

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod (a ^ n - 1)) ^ k = 1) := by
```

What it contributes. Supplied the modular base-period checkpoint $a^n=1$ modulo a^n-1 and the divisible-exponent factorization pattern.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from evaluating a multiple-period power to proving a shift-period identity, requiring the new exponent decomposition via `pow_add`. Divisibility of k is used materially, and the result is not merely the parent theorem restated.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine shift-invariance consequence: after decomposing the divisible exponent, it uses ‘`pow_add`’ and the certified period fact to show $a^{k+n}=a^k$. This is not the parent’s value-at-multiples conclusion and requires new exponent algebra.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different periodic-shift identity, not merely that a divisible exponent yields value 1 . It uses divisibility to rewrite $k=nq$ and combines the parent’s base-period fact with ‘`pow_add`’; the case $k=0$ also distinguishes it from direct recall of the parent theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2271 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	MISMATCH: Statement A asserts congruence modulo a^n-1 , whereas Statement B asserts literal equality $a^{(k+n)}=a^k$.; Statement B allows $k=0$, while Statement A requires k to be positive (and implicitly n positive).

Goal round-trip: what Lean elaborated

```
theorem power_shift_period_mod_sub_one.extracted_1_1 {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ↑a ^ (k + n) = ↑a ^ k
```

Read back by a model that saw only the Lean. *For natural numbers a , n , and k , if $1 < a$ and n divides k , then the (coerced) number a raised to the power $k + n$ equals the (coerced) number a raised to the power k .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem power_shift_period_mod_sub_one {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ((a : ZMod (a ^ n - 1)) ^ (k + n) = (a : ZMod (a ^ n - 1)) ^ k) := by
  let m : ℕ := a ^ n - 1
  change ((a : ZMod m) ^ (k + n) = (a : ZMod m) ^ k)
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hcalc :
    ((a ^ n - 1 : ℕ) : ZMod m) + 1 = ((a ^ n : ℕ) : ZMod m) := by
      simpa [m] using congrArg (fun t : ℕ => (t : ZMod m)) (Nat.sub_add_cancel hle)
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = 1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = ((a ^ n - 1 : ℕ) : ZMod m) + 1 := hcalc.symm
      _ = 1 := by simp [m]
  have hbase : ((a : ZMod m) ^ n) = 1 := by
    simpa [Nat.cast_pow] using hbase_cast
  rcases hk with ⟨q, rfl⟩
  have hshift : n * q + n = n * q + n := rfl
  calc
    (a : ZMod m) ^ (n * q + n) = (a : ZMod m) ^ (n * q) * (a : ZMod m) ^ n := by
      rw [pow_add]
    _ = (a : ZMod m) ^ (n * q) := by rw [hbase]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me__74be0029

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 74be0029bb369cdf

4.75 075. parameterized_power_congruence

The problem

For natural numbers a , n , and t with $a > 1$ and $n \mid t$, show that $a^t \equiv 1 \pmod{a^n - 1}$.

Lean statement

```
theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) : Nat.ModEq (a ^ n - 1) (a ^ t) 1
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where φ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :  
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Supplied the domain condition $a > 1$ and the reusable checkpoint that $a^n - 1$ is the natural modulus associated with the exponent n ; the child changes the final role from totient divisibility to a parameterized power congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the totient divisibility claim into a parameterized modular-power statement for every exponent t divisible by n . Its proof requires establishing the base congruence modulo $a^n - 1$ and transporting it through powers, reasoning absent from the parent's totient argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the totient divisibility conclusion with a parameterized congruence for every exponent t divisible by n . Its proof establishes the base congruence directly and raises it to a symbolic quotient power, rather than recalling the parent's cyclic-group/totient argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1875 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_power_congruence.extracted_1_1 {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) :  
  a ^ t ≡ 1 [MOD a ^ n - 1]
```

Read back by a model that saw only the Lean. For natural numbers a , n , and t , if $a > 1$ and n divides t , then a raised to the power t is congruent to 1 modulo a raised to the power n minus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) :
  Nat.ModEq (a ^ n - 1) (a ^ t) 1 := by
  rcases hdiv with ⟨k, rfl⟩
  by_cases hn : n = 0
  · subst n
    simp
  have hge : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)
  have hpowgt : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) hn ha
  have hbase : Nat.ModEq (a ^ n - 1) (a ^ n) 1 := by
    change a ^ n % (a ^ n - 1) = 1 % (a ^ n - 1)
    by_cases hm : a ^ n - 1 = 1
    · rw [hm]
      have hmod : a ^ n % 1 < 1 := Nat.mod_lt _ (by omega)
      omega
    · have hmg : 1 < a ^ n - 1 := by omega
      have hone : 1 % (a ^ n - 1) = 1 := Nat.mod_eq_of_lt (by omega)
      calc
        a ^ n % (a ^ n - 1) = (a ^ n - 1 + 1) % (a ^ n - 1) := by
          rw [Nat.sub_add_cancel hge]
        _ = 1 % (a ^ n - 1) := by
          rw [Nat.add_mod]
          simp [Nat.mod_eq_of_lt hmg, hone]
  have hpow := hbase.pow k
  have hresult : Nat.ModEq (a ^ n - 1) (a ^ (n * k)) 1 := by
    simpa [pow_mul] using hpow
  exact hresult
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__443942c6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 4af7ce7980bde617

4.76 076. power_period_mod_sub_one

The problem

Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Lean statement

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ℤMod (a ^ n - 1)) ^ k = 1) := by
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where ϕ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2_4_36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Retains the parent's hypotheses and modulus $a^n - 1$, while replacing its totient-divisibility conclusion with an operational power-period congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent's intermediate fact $a^n = 1$ in $\mathbb{ZMod}(a^n - 1)$ to all positive exponents divisible by n , requiring a new divisibility decomposition and power-period argument. The round-trip concern is only a presentation mismatch: equality in 'ZMod' is the formal encoding of congruence, and positivity of n is unnecessary because positive k cannot be divisibl

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent's intermediate congruence to every positive exponent divisible by n . Its proof requires decomposing the exponent and applying the power-period argument, which is not supplied by recalling the parent's totient-divisibility conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantively generalizes the parent's base congruence to every positive exponent divisible by n . Its proof must decompose the divisibility witness and apply the power-period identity, so recalling the parent's totient-divisibility proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2261 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the congruence modulo $a^n - 1$ and instead asserts ordinary equality of a^k with 1.

Goal round-trip: what Lean elaborated

```
theorem power_period_mod_sub_one.extracted_1_1 : ∀ {a n : ℕ},  
  1 < a → 0 < n → ∀ (k : ℕ), 0 < k → n ∣ k → ↑a ^ k = 1
```

Read back by a model that saw only the Lean. For all natural numbers a and n , if $a > 1$ and $n > 0$, then for every natural number k , if $k > 0$ and n divides k , the k -th power of the coercion of a equals 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod (a ^ n - 1)) ^ k = 1) := by
  let m : ℕ := a ^ n - 1
  change ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod m) ^ k = 1)
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hcalc :
    ((a ^ n - 1 : ℕ) : ZMod m) + 1 = ((a ^ n : ℕ) : ZMod m) := by
      simpa [m] using congrArg (fun t : ℕ => (t : ZMod m)) (Nat.sub_add_cancel hle)
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = 1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = ((a ^ n - 1 : ℕ) : ZMod m) + 1 := hcalc.symm
      _ = 1 := by simp [m]
  have hbase : ((a : ZMod m) ^ n) = 1 := by
    simpa [Nat.cast_pow] using hbase_cast
  intro k _ hdiv
  rcases hdiv with ⟨q, rfl⟩
  have hexponent : n * q = n * q := rfl
  calc
    (a : ZMod m) ^ (n * q) = ((a : ZMod m) ^ n) ^ q := by rw [pow_mul]
    _ = 1 := by rw [hbase]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__a2c28210

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 5aba40bb18595ef0

4.77 077. bounded_totient_period

The problem

Let $a > 1$ and suppose that n divides the Euler totient of $a^n - 1$. Show that for every natural number k , the exponent $k\phi(a^n - 1)$ is a multiple of n , and $a^{k\phi(a^n - 1)} \equiv 1 \pmod{a^n - 1}$.

Lean statement

```
theorem bounded_totient_period {a n : ℕ} (ha : 1 < a) (hdiv : n ∣ (a ^ n - 1).totient) :
  ∀ k : ℕ, ∃ q : ℕ,
    k * (a ^ n - 1).totient = n * q ∧
    a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1]
```

Parent. If $a > 1$ is an integer, show that $n \mid \phi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Its totient-divisibility conclusion is retained as the explicit hypothesis `hdiv` and used to construct the multiple-of- n exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a new exponent-period argument: it converts the retained divisibility witness into a quotient and proves the congruence for every multiple of the totient. This is not obtained by recalling the parent's divisibility proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the parent divisibility witness to construct, for every k , an exponent quotient and then proves a new modular-periodicity conclusion. This requires reasoning about powers modulo $a^n - 1$ and is not supplied by recalling the parent's divisibility theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child retains the parent divisibility checkpoint but adds a genuinely new modular-period argument: it derives $a^n \equiv 1 \pmod{a^n - 1}$ and lifts this to every exponent $k \cdot \phi(a^n - 1)$ while constructing the corresponding quotient. A solver recalling only the parent's totient-divisibility proof would still need this exponentiation and congruence reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2801 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_totient_period.extracted_1_1 : ∀ {a n : ℕ},  
  1 < a →  
  0 < n →  
  n ∣ (a ^ n - 1).totient →  
  ∀ (k : ℕ), ∃ q, k * (a ^ n - 1).totient = n * q ∧ a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1]
```

Read back by a model that saw only the Lean. For all natural numbers a and n , if $a > 1$, $n > 0$, and n divides the Euler totient of $a^n - 1$, then for every natural number k there exists a natural number q such that k times the Euler totient of $a^n - 1$ equals n times q , and a raised to the power k times the Euler totient of $a^n - 1$ is congruent to 1 modulo $a^n - 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Set Real Ideal Polynomial  
open scoped BigOperators  
  
theorem bounded_totient_period {a n : ℕ} (ha : 1 < a) (hdiv : n ∣ (a ^ n - 1).totient) :  
  ∀ k : ℕ, ∃ q : ℕ,  
    k * (a ^ n - 1).totient = n * q ∧  
    a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1] := by  
  rcases hdiv with ⟨q, hq⟩  
  have hbase : a ^ n ≡ 1 [MOD a ^ n - 1] := by  
    rw [Nat.modEq_iff_dvd]  
    have hle : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)  
    change ((a ^ n - 1 : ℕ) : ℤ) ∣ (1 : ℤ) - (a ^ n : ℤ)  
    rw [show (1 : ℤ) - (a ^ n : ℤ) = -((a ^ n - 1 : ℕ) : ℤ) by  
      rw [Nat.cast_sub hle]  
      simp]  
    exact (dvd_neg).2 (dvd_refl _)  
  intro k  
  refine ⟨k * q, ?_, ?_⟩  
  · calc  
    k * (a ^ n - 1).totient = k * (n * q) := by rw [hq]  
    _ = n * (k * q) := by ac_rfl  
  · rw [hq]  
    convert hbase.pow (k * q) using 1 <|> simp [pow_mul, mul_assoc, mul_left_comm, mul_comm]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__e40788fd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ecdded62cf3a5cd33

4.78 078. conjugate_zpow_mem_zpowers

The problem

Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Lean statement

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

What it contributes. Supplies the cyclic-subgroup normality argument and the membership checkpoint for powers of a .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different quantified closure result: every conjugate of every integer power of a lies in the cyclic subgroup generated by a . Its proof requires showing a^n belongs to that subgroup and applying normality, rather than recalling the parent's existence-and-rearrangement argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes conjugation from the single element a to every integer power a^n . Its proof requires establishing $a^n \in \langle a \rangle$ via subgroup closure before applying normality, so recalling the parent does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes conjugation from a to every integer power a^n . It requires the new subgroup-membership step for integer powers and proves a universal membership statement rather than recalling the parent's existential commutation conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2286 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugate_zpow_mem_zpowers.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G],  
  (∀ (H : Subgroup G), H.Normal) → ∀ (a b : G) (n : ℤ), b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Read back by a model that saw only the Lean. For every type G equipped with a group structure, if every subgroup H of G is normal, then for all elements a, b of G and every integer n , the element $b a^n b^{-1}$ belongs to the subgroup of G consisting of the integer powers of a .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Set Real Ideal Polynomial  
open scoped BigOperators  
  
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]  
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :  
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a := by  
  classical  
  intro n  
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)  
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a  
  have han : a ^ n ∈ Subgroup.zpowers a := by  
    exact Subgroup.zpow_mem (Subgroup.zpowers a) ha n  
  have hconj : b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a := by  
    simpa using hHnormal.conj_mem (a ^ n) han b  
  exact hconj
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__297b115a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 297b115a6e38c837

4.79 079. bounded_conjugation_transport

The problem

For every group in which every subgroup is normal, a conjugation relation for an element extends to a bounded iterated transport law for powers of the conjugating element.

Lean statement

```
theorem bounded_conjugation_transport {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b ∧
  ∀ (N n : ℕ), n ≤ N → ∃ k : ℤ, b ^ n * a = a ^ k * b ^ n
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

What it contributes. Reuses the parent's normal cyclic-subgroup and integer-power extraction argument, while extending it inductively to conjugates by arbitrary powers of b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child genuinely proves transport for every natural power b^n by induction and repeated normality, which is reasoning absent from the parent's single-conjugation proof. The stated bound $n \leq N$ is unused, so the bounded aspect is superficial, but the iterated-conjugation generalization is substantive.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine iterated-conjugation law for every power of b , requiring an induction and repeated use of subgroup normality; the parent's single-step witness does not supply this reasoning. The bound $n \leq N$ is redundant, but the generalized transport claim remains substantive.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely stronger transport law for every power b^n , requiring induction to show all iterated conjugates remain in $\langle a \rangle$. This reasoning is not supplied by the parent's single-conjugation witness, despite retaining that witness in the conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1889 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_conjugation_transport.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j, b * a = a ^ j * b ∧ ∀ (N n : ℕ), n ≤ N → ∃ k, b ^ n * a = a ^ k * b ^ n
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal, and let a, b be elements of G . Then there exists a natural number j such that $b \cdot a = a^j \cdot b$, and for every pair of natural numbers N and n with $n \leq N$, there exists a natural number k such that $b^n \cdot a = a^k \cdot b^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem bounded_conjugation_transport {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b ∧
  ∀ (N n : ℕ), n ≤ N → ∃ k : ℤ, b ^ n * a = a ^ k * b ^ n := by
  classical
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hmem : ∀ n : ℕ, b ^ n * a * (b ^ n)⁻¹ ∈ Subgroup.zpowers a := by
    intro n
    induction n with
    | zero => simp using ha
    | succ n ih =>
      have h := hHnormal.conj_mem (b ^ n * a * (b ^ n)⁻¹) ih b
      simp [pow_succ', mul_assoc] using h
  obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp (hmem 1)
  have hstep : b * a = a ^ j * b := by
    have hj' : b * a * b⁻¹ = a ^ j := by
      simp using hj.symm
    calc
    b * a = (b * a * b⁻¹) * b := by simp [mul_assoc]
    _ = a ^ j * b := by rw [hj']
  refine ⟨j, hstep, ?_⟩
  intro N n hn
  obtain ⟨k, hk⟩ := Subgroup.mem_zpowers_iff.mp (hmem n)
  have htransport : b ^ n * a = a ^ k * b ^ n := by
    calc
    b ^ n * a = (b ^ n * a * (b ^ n)⁻¹) * b ^ n := by simp [mul_assoc]
    _ = a ^ k * b ^ n := by rw [hk]
  exact ⟨k, htransport⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__4a8d9215

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 95b5b05cc1c3ddd1

4.80 080. cyclic_subgroup_conjugation_automorphism

The problem

Let G be a group in which every subgroup is normal. For $a, b \in G$, conjugation by b restricts to an automorphism of the cyclic subgroup generated by a .

Lean statement

```
theorem cyclic_subgroup_conjugation_automorphism {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f : Subgroup.zpowers a → Subgroup.zpowers a,
    Function.Bijective f ∧
    ∀ x : Subgroup.zpowers a,
      ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a^j * b := by
```

What it contributes. Uses the parent's normality argument for the cyclic subgroup and its conjugation-membership witness, then consumes that witness in a new bijective homomorphism conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the existential power equation into a bijective group-homomorphism statement for the entire cyclic subgroup. Its proof must establish homomorphism, injectivity, and surjectivity via inverse conjugation, none of which is supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential power equation into a homomorphism-and-bijectivity claim for conjugation on the entire cyclic subgroup. Its proof must establish closure for every subgroup element, multiplicativity, injectivity, and surjectivity via the inverse conjugation map, none of which is supplied by the parent's single-element argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantially stronger and different claim: conjugation restricts to a bijective group homomorphism on the entire cyclic subgroup. This requires constructing the homomorphism and its inverse behavior, which the parent's single-element power witness does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2189 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_conjugation_automorphism.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f, Function.Bijective f ∧ ∀ (x : (Subgroup.zpowers a)), ↑(f x) = b * ↑x * b⁻¹
```

Read back by a model that saw only the Lean. *Let G be a group such that every subgroup of G is normal. For any elements a and b of G , there exists a bijective function f from the subgroup of G consisting of all integer powers of a to itself such that, for every element x of this subgroup, the element $f(x)$, viewed in G , equals $b \cdot x \cdot b^{-1}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cyclic_subgroup_conjugation_automorphism {G : Type*} [Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f : Subgroup.zpowers a →* Subgroup.zpowers a,
    Function.Bijective f ∧
      ∀ x : Subgroup.zpowers a,
        ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
  classical
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : ∀ x : Subgroup.zpowers a,
    b * (x : G) * b⁻¹ ∈ Subgroup.zpowers a := by
    intro x
    exact hHnormal.conj_mem (x : G) x.property b
  have hconj_inv : ∀ x : Subgroup.zpowers a,
    b⁻¹ * (x : G) * b ∈ Subgroup.zpowers a := by
    intro x
    simpa using hHnormal.conj_mem (x : G) x.property b⁻¹
  let f : Subgroup.zpowers a →* Subgroup.zpowers a :=
    { toFun := fun x => ⟨b * (x : G) * b⁻¹, hconj x⟩
      map_one' := by simp
      map_mul' := by
        intro x y
        apply Subtype.ext
        simp [mul_assoc] }
  have hf : ∀ x : Subgroup.zpowers a,
    ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
    intro x
    rfl
  have hinj : Function.Injective f := by
    intro x y hxy
    apply Subtype.ext
    have hxy' := congrArg (fun z : Subgroup.zpowers a => (z : G)) hxy
    simpa [f, mul_assoc] using hxy'
  have hsurj : Function.Surjective f := by
    intro y
    let x : Subgroup.zpowers a := ⟨b⁻¹ * (y : G) * b, hconj_inv y⟩
    refine ⟨x, ?_⟩
    apply Subtype.ext
    simp [f, x, mul_assoc]
    refine ⟨f, ⟨(hinj, hsurj), hf⟩⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__c4d2b752

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint c4d2b752bf56a5d0

4.81 081. conjugation_restriction_unique_exponent

The problem

Let G be a group in which every subgroup is normal, and let a have infinite order. For each $b \in G$, show that there is a unique integer j such that conjugation by b restricts to an automorphism of the cyclic subgroup generated by a , sending a to a^j .

Lean statement

```
theorem conjugation_restriction_unique_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ,
    ∃ f : Subgroup.zpowers a ≈* Subgroup.zpowers a,
      f ⟨a, Subgroup.mem_zpowers a⟩ =
        ⟨a ^ j, Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩⟩ ^
        ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b⁻¹
```

Parent. Let G be a group in which every subgroup is normal. Suppose that a has infinite order. Prove that for every $b \in G$, there is a unique integer j such that $ba = a^j b$.

Parent in Lean

```
theorem unique_commutation_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ, b * a = a ^ j * b
```

What it contributes. Supplied the normality-based existence of a unique commutation exponent and the infinite-order cancellation argument; these are integrated with the explicitly constructed restriction automorphism.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child materially changes the conclusion by requiring construction of a group equivalence realizing conjugation on the cyclic subgroup, including inverse and homomorphism proofs. Recalling the parent only supplies the exponent and uniqueness equation; it does not supply the restricted automorphism witness.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive conjugation-restriction automorphism of the cyclic subgroup and proves its generator action and behavior on every subgroup element. Constructing and verifying this bundled monoid isomorphism requires mathematics absent from the parent, beyond recalling the unique exponent argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive construction: it must define and verify a monoid isomorphism of the cyclic subgroup induced by conjugation, then connect its generator image to the unique exponent. The parent's commutation-and-uniqueness argument does not supply this restriction map or its inverse and homomorphism laws.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2221 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugation_restriction_unique_exponent.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a : G) (hinf : ∀ (n : ℤ), a ^ n = 1 → n = 0) (b : G) :
∃! j, ∃ f, f ⟨a, ...⟩ = ⟨a ^ j, ...⟩ ∧ ∀ (x : ↑(Subgroup.zpowers a)), ↑(f x) = b * ↑x * b-1
```

Read back by a model that saw only the Lean. *Let G be a group such that every subgroup of G is normal. Let a and b be elements of G , and assume that for every integer n , if $a^n = 1$ then $n = 0$. Then there exists a unique integer j such that there is a group homomorphism f from the subgroup of G consisting of all integer powers of a to itself satisfying $f(a) = a^j$ and, for every element x of this subgroup, the underlying element of $f(x)$ equals $b x b^{-1}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugation_restriction_unique_exponent {G : Type*} [Group G]
(hG : ∀ (H : Subgroup G), H.Normal)
(a : G)
(hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
(b : G) :
∃! j : ℤ,
  ∃ f : Subgroup.zpowers a ≈* Subgroup.zpowers a,
    f ⟨a, Subgroup.mem_zpowers a⟩ =
      ⟨a ^ j, Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩⟩ ∧
      ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b-1 := by
  classical
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : b * a * b-1 ∈ Subgroup.zpowers a := by
    simpa using hnormal.conj_mem a ha b
  obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp hconj
  have hcomm : b * a = a ^ j * b := by
    calc
      b * a = (b * a * b-1) * b := by simp [mul_assoc]
      _ = a ^ j * b := by rw [hj]
  have haj : a ^ j ∈ Subgroup.zpowers a := by
    exact Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩
  let f : Subgroup.zpowers a ≈* Subgroup.zpowers a :=
    { toFun := fun x => ⟨b * x * b-1, hnormal.conj_mem x.1 x.2 b⟩
      invFun := fun x => ⟨b-1 * x * b, hnormal.conj_mem' x.1 x.2 b⟩
      left_inv := by
        intro x
        apply Subtype.ext
        simp [mul_assoc]
      right_inv := by
        intro x
        apply Subtype.ext
        simp [mul_assoc]
      map_mul' := by
        intro x y
```

```

    apply Subtype.ext
    simp [mul_assoc] }
have hfgen : f ⟨a, ha⟩ = ⟨a ^ j, haj⟩ := by
  apply Subtype.ext
  simp [f, hj]
have hfres : ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b⁻¹ := by
  intro x
  rfl
refine ⟨j, ⟨f, hfgen, hfres⟩, ?_⟩
intro k hk
rcases hk with ⟨g, hggen, hgres⟩
have hgenval : (g ⟨a, ha⟩ : G) = a ^ k := by
  exact congrArg Subtype.val hggen
have hconjk : b * a * b⁻¹ = a ^ k := by
  calc
    b * a * b⁻¹ = (g ⟨a, ha⟩ : G) := (hgres ⟨a, ha⟩).symm
    _ = a ^ k := hgenval
have hcommk : b * a = a ^ k * b := by
  calc
    b * a = (b * a * b⁻¹) * b := by simp [mul_assoc]
    _ = a ^ k * b := by rw [hconjk]
have hpow : a ^ j = a ^ k := by
  exact mul_right_cancel (hcomm.symm.trans hcommk)
have hrel : a ^ (j - k) = 1 := by
  rw [zpow_sub, hpow]
  simp
have hzero : j - k = 0 := hinf (j - k) hrel
exact (sub_eq_zero.mp hzero).symm

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh.me__4b1a9d7c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 4b1a9d7cae1e5dfb

4.82 082. cubic_nilpotent_three_term_commutates

The problem

If $a^3 = 0$ in R , show that $a^2x + axa + xa^2$ commutes with a .

Lean statement

```
theorem cubic_nilpotent_three_term_commutates {R : Type*} [Ring R] (a x : R)
  (h : a ^ 3 = 0) :
  a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
  (a ^ 2 * x + a * x * a + x * a ^ 2) * a := by
```

Parent. If $a^2 = 0$ in R , show that $ax + xa$ commutes with a .

Parent in Lean

```
theorem Herstein_exercise_4_2_6 {R : Type*} [Ring R] (a x : R)
  (h : a ^ 2 = 0) : a * (a * x + x * a) = (a * x + x * a) * a := by
```

What it contributes. Supplied the noncommutative ring setting and commutation-by-expansion pattern; the child replaces the square-zero anticommutator with a cube-zero three-term expression and adds the cubic nilpotence expansion checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the square-zero anticommutator to a cubic-nilpotent three-term expression. Its proof must expand three summands and use the cubic relation to eliminate the endpoint terms, which is not supplied by the parent's square-zero argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from square-zero to cubic nilpotence and requires proving cancellation of the outer cubic terms, leaving matching two-sided products. The parent's anticommutator argument cannot supply this result because $a^3 = 0$ does not imply $a^2 = 0$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the commutation identity from square-zero to cubic-nilpotent elements and requires new three-term expansion plus cancellation using $a^3 = 0$. The parent's proof does not directly supply this cubic argument, so memorizing it would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1984 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_nilpotent_three_term_commutates.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R)
(h : a ^ 3 = 0) : a * (a ^ 2 * x + a * x * a + x * a ^ 2) = (a ^ 2 * x + a * x * a + x * a ^ 2) * a
```

Read back by a model that saw only the Lean. *Let R be a ring, and let a and x be elements of R . If $a^3 = 0$, then $a(a^2x + axa + xa^2) = (a^2x + axa + xa^2)a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cubic_nilpotent_three_term_commutates {R : Type*} [Ring R] (a x : R)
(h : a ^ 3 = 0) :
  a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
    (a ^ 2 * x + a * x * a + x * a ^ 2) * a := by
  have h_cube : a * a * a = 0 := by
    simpa [pow_succ, pow_two, mul_assoc] using h
  have h_left_cube : a * (a ^ 2 * x) = 0 := by
    calc
      a * (a ^ 2 * x) = (a * a * a) * x := by simp [pow_two, mul_assoc]
      _ = 0 := by rw [h_cube, zero_mul]
  have h_right_cube : (x * a ^ 2) * a = 0 := by
    calc
      (x * a ^ 2) * a = x * (a * a * a) := by simp [pow_two, mul_assoc]
      _ = 0 := by rw [h_cube, mul_zero]
  have h_expansion :
    a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
      a ^ 2 * x * a + a * x * a ^ 2 := by
    calc
      a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
        a * (a ^ 2 * x) + a * (a * x * a) + a * (x * a ^ 2) := by
          simp [mul_add]
      _ = a ^ 2 * x * a + a * x * a ^ 2 := by
        rw [h_left_cube]
        simp [pow_two, mul_assoc]
  have h_right_expansion :
    (a ^ 2 * x + a * x * a + x * a ^ 2) * a =
      a ^ 2 * x * a + a * x * a ^ 2 := by
    calc
      (a ^ 2 * x + a * x * a + x * a ^ 2) * a =
        (a ^ 2 * x) * a + (a * x * a) * a + (x * a ^ 2) * a := by
          simp [add_mul]
      _ = a ^ 2 * x * a + a * x * a ^ 2 := by
        rw [h_right_cube]
        simp [pow_two, mul_assoc]
  exact h_expansion.trans h_right_expansion.symm
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__me__70f5bb23

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 6e3358f449c14e20

4.83 083. anticommutator_commutes_with_a_iff_square_commutes

The problem

Let R be a ring and let $a, x \in R$. Prove that $ax + xa$ commutes with a if and only if a^2 commutes with x .

Lean statement

```
theorem anticommutator_commutes_with_a_iff_square_commutes {R : Type*} [Ring R] (a x : R) :
  ((a * x + x * a) * a = a * (a * x + x * a)) ↔ (a ^ 2 * x = x * a ^ 2)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with natural-number coefficients.

Parent in Lean

```
theorem anticommutator_commutes_with_nat_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℕ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim to an iff characterization in an arbitrary ring, removing the nilpotence hypothesis and requiring additive cancellation to relate anticommutator commutation to square commutation. The parent's polynomial-commutation induction does not supply this equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from commuting with a polynomial in a nilpotent element to an iff characterization in an arbitrary ring. Its proof requires additive cancellation and noncommutative ring normalization in both directions, none of which is supplied by recalling the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine general characterization in an arbitrary ring, with no nilpotence or polynomial hypothesis. Its proof uses bidirectional algebraic cancellation to relate anticommutator commutation to square commutation, which is not supplied by the parent's nilpotent polynomial-commutation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2924 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutes_with_a_iff_square_commutes.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R]
  (a x : R) : (a * x + x * a) * a = a * (a * x + x * a) ↔ a ^ 2 * x = x * a ^ 2
```

Read back by a model that saw only the Lean. For any elements a and x of a ring R , the product of the anticommutator $ax + xa$ with a equals the product of a with the anticommutator $ax + xa$ if and only if a squared times x equals x times a squared.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutes_with_a_iff_square_commutes {R : Type*} [Ring R] (a x : R) :
  ((a * x + x * a) * a = a * (a * x + x * a)) ↔ (a ^ 2 * x = x * a ^ 2) := by
  constructor
  · intro h
    have h_cancel : x * a ^ 2 = a ^ 2 * x := by
      calc
        x * a ^ 2 = (a * x + x * a) * a - a * x * a := by noncomm_ring
        _ = a * (a * x + x * a) - a * x * a := by rw [h]
        _ = a ^ 2 * x := by noncomm_ring
    exact h_cancel.symm
  · intro h
    calc
      (a * x + x * a) * a = a * x * a + x * a ^ 2 := by noncomm_ring
      _ = a * x * a + a ^ 2 * x := by rw [h]
      _ = a * (a * x + x * a) := by noncomm_ring
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh.me__fd705452

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint fd7054526db002b0

4.84 084. gaussian_mul_sq_eq_zero_iff

The problem

For a Gaussian integer a , multiplication by $(1 + i)^2$ gives zero exactly when $a = 0$.

Lean statement

```
theorem gaussian_mul_sq_eq_zero_iff (a : GaussianInt) : (((1, 1) : GaussianInt) ^ 2 * a = 0) ↔ a = 0
```

Parent. For every Gaussian integer a , show that the quotient of $2a$ by $(1 + i)^2$ exists uniquely and equals $a(-i)$.

Parent in Lean

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

What it contributes. Supplied the explicit factorization of $(1+i)^2$ and the nonzero checkpoint used in the cancellation argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to a zero-product characterization and requires applying the no-zero-divisors argument in both directions. Although it reuses the factorization and nonzero checkpoint, the parent's existence-and-uniqueness result does not directly prove this claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential scaled-factorization/uniqueness claim into a zero-product characterization. Although it reuses the factorization and nonzero checkpoint, the decisive cancellation argument via no-zero-divisors is new and the parent's conclusion does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new cancellation characterization: after reusing the factorization only to establish nonzeroness, it applies the zero-product argument to derive the iff statement. Recalling the parent's existence-and-uniqueness proof does not establish this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_mul_sq_eq_zero_iff.extracted_1_1 (a : GaussianInt) : { re := 1, im := 1 } ^ 2 * a = 0 ↔ a = 0
```

Read back by a model that saw only the Lean. For every Gaussian integer a , the product of a with the square of the Gaussian integer having real part 1 and imaginary part 1 is zero if and only if a is zero.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_mul_sq_eq_zero_iff (a : GaussianInt) : ((⟨1, 1⟩ : GaussianInt) ^ 2 * a = 0) ↔ a = 0 := by
  have hfactor : (⟨1, 1⟩ : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by
    ext <=> simp [pow_two]
  have hne : (⟨1, 1⟩ : GaussianInt) ^ 2 ≠ 0 := by
    intro hz
    have hzero : (0 : GaussianInt) = 2 := by
      calc
        (0 : GaussianInt) = (⟨1, 1⟩ : GaussianInt) ^ 2 * ⟨0, -1⟩ := by rw [hz]; simp
        _ = 2 := hfactor
    norm_num at hzero
  constructor
  · intro h
    rcases mul_eq_zero.mp h with hleft | ha
    · exact False.elim (hne hleft)
    · exact ha
  · intro ha
    rw [ha]
    simp
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh.me__e70e2bb2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e70e2bb2bcd97b04

4.85 085. gaussian_factorization_unique

The problem

Show that the quotient in the factorization of 2 by $(1 + i)^2$ exists, is unique, and equals $-i$.

Lean statement

```
theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem Ireland_Rosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 | 2 := by
```

What it contributes. Supplies the explicit Gaussian factorization bridge, reused as the named existence checkpoint hfactor.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness theorem: after establishing existence, it proves any Gaussian quotient equal to 2 must have real and imaginary coordinates matching $\langle 0, -1 \rangle$. The parent's divisibility witness proof does not supply this coordinate-based uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness claim: after constructing the quotient, it must prove that every Gaussian integer solution has the same real and imaginary parts. The parent's divisibility witness supplies existence but does not supply this coordinate-wise uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens divisibility to existence and uniqueness of the Gaussian quotient. The parent supplies only the witness equation; proving every quotient equals $\langle 0, -1 \rangle$ requires a new coordinate-level uniqueness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2754 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_factorization_unique.extracted_1_1 :
  ∃ q,
    { re := 1, im := 1 } ^ 2 * q = 2 ∧
    ∀ (q₁ : GaussianInt), { re := 1, im := 1 } ^ 2 * q₁ = 2 → q₁ = { re := 0, im := -1 }
```

Read back by a model that saw only the Lean. *There exists a Gaussian integer q such that $(1 + i)^2 q = 2$, and for every Gaussian integer q_1 , if $(1 + i)^2 q_1 = 2$, then $q_1 = -i$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) :
  GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
  have hfactor : ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by
    ext <|> simp [pow_two]
  refine ⟨⟨0, -1⟩, hfactor, ?_⟩
  intro q₁ hq
  have hqr := congrArg (fun z : GaussianInt => z.re) hq
  have hqi := congrArg (fun z : GaussianInt => z.im) hq
  ext
  · simp [pow_two] at hqi ⊢
    omega
  · simp [pow_two] at hqr ⊢
    omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me__34206d90

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 34206d90a1c2bdae

4.86 086. finite_family_union_isOpen_in_subspace

The problem

Let T be a subset of a topological space X , and let $(A_i)_{i < n}$ be a finite family of subsets of X . If for every $i < n$ and every $x \in T \cap A_i$ there is an open set $U \subseteq X$ with $x \in U$ and $U \cap T \subseteq A_i$, prove that $\bigcup_{i < n} A_i$ is open in the subspace topology on T .

Lean statement

```
theorem finite_family_union_isOpen_in_subspace (X : Type*) [TopologicalSpace X] (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ i x, x ∈ T
↪ → x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) : IsOpen ((fun x : T => (x : X)) ⁻¹' (U i : Fin n, A i))
```

Parent. A bounded indexed family of locally open sets has an open union.

Parent in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
(A : Fin n → Set X)
(h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
IsOpen (U i : Fin n, A i)
```

What it contributes. Supplied the finite indexed-union extraction and local-neighborhood transport pattern; the mutation adds an ambient subset T and subtype pullback witnesses.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the ambient topology: it proves openness of the union relative to an arbitrary subtype T , requiring a preimage construction and subtype-membership reasoning. The parent's proof in X does not supply this subspace argument, even though the local-union structure is shared.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to the subspace topology: it constructs an open neighborhood by taking the subtype preimage of U and uses the weaker relative containment $U \cap T \subseteq A_i$. This requires new preimage and subtype-membership reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the setting to a subtype and requires reasoning with the subtype topology, preimages under the continuous subtype inclusion, and intersected local neighborhoods. The parent's ambient-space local-openness proof does not supply these steps, and the child is not a forced wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1972 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_family_union_isOpen_in_subspace.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X]
  (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ (i : Fin n), ∀ x ∈ T, x ∈ A i → ∃ U, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) :
  IsOpen ((fun x => ↑x) ⁻¹' U i, A i)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let $T \subseteq X$, let n be a natural number, and let A be a family of n subsets of X indexed by $\text{Fin } n$. Assume that for every $i \in \text{Fin } n$ and every $x \in T$, if $x \in A_i$, then there exists an open set $U \subseteq X$ such that $x \in U$ and $U \cap T \subseteq A_i$. Then the union $\bigcup_{i \in \text{Fin } n} A_i$, regarded as a subset of the subspace T , is open in the subspace topology on T .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_family_union_isOpen_in_subspace (X : Type*) [TopologicalSpace X] (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ i x, x ∈ T
  ↪ → x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) : IsOpen ((fun x : T => (x : X)) ⁻¹' (U i : Fin n, A i)) := by
  refine isOpen_iff_forall_mem_open.mpr ?_
  intro x hx
  change (x : X) ∈ U i : Fin n, A i at hx
  have hx_index : ∃ i : Fin n, (x : X) ∈ A i := by
    simpa only [Set.mem_iUnion] using hx
  rcases hx_index with ⟨i, hxi⟩
  rcases h i (x : X) x.property hxi with ⟨U, hxU, hUopen, hUsub⟩
  let V : Set T := (fun y : T => (y : X)) ⁻¹' U
  have hVopen : IsOpen V := by
    dsimp [V]
    exact hUopen.preimage continuous_subtype_val
  have hxV : x ∈ V := by
    dsimp [V]
    exact hxU
  have hVsub : V ⊆ (fun y : T => (y : X)) ⁻¹' (U i : Fin n, A i) := by
    intro y hy
    change (y : X) ∈ U at hy
    change (y : X) ∈ U i : Fin n, A i
    apply Set.mem_iUnion.mpr
    refine ⟨i, ?_⟩
    exact hUsub ⟨hy, y.property⟩
  have hlocal : V ⊆ (fun y : T => (y : X)) ⁻¹' (U i : Fin n, A i) ∧ IsOpen V ∧ x ∈ V :=
    ⟨hVsub, hVopen, hxV⟩
  exact ⟨V, hlocal⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__me.me__1996e9f4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c62d0831393c4434

4.87 087. reindexed_iInf_topology_comparison

The problem

Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X , and let $f : J \rightarrow I$. Show that every topology on X contained in each $\mathcal{T}_i$ is contained in the largest topology contained in all the $\mathcal{T}_i$, and that this largest topology is contained in the largest topology contained in all the $\mathcal{T}_{f(j)}$.

Lean statement

```
theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⨅ i, T i) ∧
  (⨅ i, T i) ≤ ⨅ j, T (f j) := by
```

Parent. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

What it contributes. Uses the parent's greatest-common-lower-bound checkpoint for the infimum topology, reformulated with Mathlib's TopologicalSpace order and extended through a reindexing map.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child reexpresses the parent's universal lower-topology construction in the TopologicalSpace lattice and adds a genuinely new reindexing comparison, proving that the infimum over the full family lies below the infimum over the mapped subfamily. The reindexing argument is not supplied by the parent's uniqueness proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim to an order-theoretic universal property and a reindexing comparison for infima of topologies. Its proof relies on 'iInf_le' and 'le_iInf', not on the parent's explicit topology construction or uniqueness argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child re-encodes the common topology as an infimum in the lattice of topologies and proves a nontrivial reindexing comparison via 'iInf'/'order' reasoning. This is not obtainable by recalling the parent's set-membership proof, despite expressing the same universal construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1866 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reindexed_iInf_topology_comparison.extracted_1_1.{u_3, u_2, u_1} (X : Type u_1) (I : Type u_2) (J : Type u_3)
  (T : I → TopologicalSpace X) (f : J → I) :
  (∀ (U : TopologicalSpace X), (∀ (i : I), U ≤ T i) → U ≤ ⋂ i, T i) ∧ ⋂ i, T i ≤ ⋂ j, T (f j)
```

Read back by a model that saw only the Lean. For any types X, I , and J , any family T assigning a topological space on X to each $i \in I$, and any function $f: J \rightarrow I$, the following both hold: (1) for every topological space U on X , if $U \leq T(i)$ for every $i \in I$, then $U \leq$ the infimum of the topological spaces $T(i)$ over $i \in I$; and (2) the infimum of $T(i)$ over $i \in I$ is $\leq$ the infimum of $T(f(j))$ over $j \in J$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⋂ i, T i) ∧
  (⋂ i, T i) ≤ ⋂ j, T (f j) := by
  have h_original_lower : ∀ i : I, (⋂ i, T i) ≤ T i := by
    intro i
    exact iInf_le T i
  have h_original_universal : ∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⋂ i, T i := by
    intro U hU
    refine le_iInf ?_
    intro i
    exact hU i
  have h_reindexed_lower : ∀ j : J, (⋂ i, T i) ≤ T (f j) := by
    intro j
    exact h_original_lower (f j)
  have h_reindexed_universal : ∀ U : TopologicalSpace X, (∀ j, U ≤ T (f j)) → U ≤ ⋂ j, T (f j) := by
    intro U hU
    refine le_iInf ?_
    intro j
    exact hU j
  have h_comparison : (⋂ i, T i) ≤ ⋂ j, T (f j) := by
    exact h_reindexed_universal _ h_reindexed_lower
  exact ⟨h_original_universal, h_comparison⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__me__73a8a1e4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint 8cc68b9e3994adb1

4.88 088. quotient_descent_first_projection

The problem

Let $f : X \times Y \rightarrow Z$ be continuous and constant on every fiber of the first projection, with Y nonempty. If the first projection is open, show that there is a unique continuous map $g : X \rightarrow Z$ such that $f(x, y) = g(x)$ for all (x, y) .

Lean statement

```
theorem quotient_descent_first_projection {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
  (hY : Nonempty Y) (f : X × Y → Z) (hf : Continuous f)
  (hconst : ∀ x : X, ∀ y₁ y₂ : Y, f (x, y₁) = f (x, y₂))
  (hopen : IsOpenMap (Prod.fst : X × Y → X)) :
  ∃! g : X → Z, Continuous g ∧ ∀ p : X × Y, g (Prod.fst p) = f p
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Uses the parent's first-coordinate projection and its open-map property as the topological descent checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine quotient-descent theorem: constructing the descended map, proving continuity via openness of the projection, and establishing uniqueness. The parent only proves that coordinate projections are open and does not supply the fiberwise factorization or continuity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive existence-and-uniqueness theorem for a continuous descended map, using fiberwise constancy, a chosen point of Y , and openness of the projection. The parent's open-map result supplies only one hypothesis and does not provide the descent or continuity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine quotient-descent theorem: it constructs the descended map, establishes continuity via openness of the projection, and proves existence and uniqueness using fiber constancy. This mathematics is not supplied by the parent's separate openness result and cannot be recovered by recalling its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2823 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_descent_first_projection.extracted_1.1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] (hY : Nonempty Y)
  (f : X × Y → Z) (hf : Continuous f) (hconst : ∀ (x : X) (y₁ y₂ : Y), f (x, y₁) = f (x, y₂))
  (hopen : IsOpenMap Prod.fst) : ∃! g, Continuous g ∧ ∀ (p : X × Y), g p.1 = f p
```

Read back by a model that saw only the Lean. *Let X , Y , and Z be topological spaces, with Y nonempty. Let $f : X \times Y \rightarrow Z$ be continuous, and suppose that for every $x \in X$ and all $y_1, y_2 \in Y$, $f(x, y_1) = f(x, y_2)$. Assume that the first projection map $\text{Prod.fst} : X \times Y \rightarrow X$ is an open map. Then there exists a unique function $g : X \rightarrow Z$ such that g is continuous and, for every $p \in X \times Y$, $g(p.1) = f(p)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem quotient_descent_first_projection {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
  (hY : Nonempty Y) (f : X × Y → Z) (hf : Continuous f)
  (hconst : ∀ x : X, ∀ y₁ y₂ : Y, f (x, y₁) = f (x, y₂))
  (hopen : IsOpenMap (Prod.fst : X × Y → X)) :
  ∃! g : X → Z, Continuous g ∧ ∀ p : X × Y, g (Prod.fst p) = f p := by
  let y₀ : Y := Classical.choice hY
  let g : X → Z := fun x => f (x, y₀)
  have hg : Continuous g := by
    rw [continuous_def]
    intro v hv
    have hU : IsOpen (f ⁻¹' v) := hf.isOpen_preimage v hv
    have hImage : IsOpen (Prod.fst ⁻¹' (f ⁻¹' v)) := hopen _ hU
    have hEq : g ⁻¹' v = Prod.fst ⁻¹' (f ⁻¹' v) := by
      ext x
      constructor
      · intro hx
        refine ⟨(x, y₀), ?_, rfl⟩
        simp [g] using hx
      · rintro ⟨p, hp, hpx⟩
        rw [← hpx]
        change f (p.1, y₀) ∈ v
        rw [← hconst p.1 p.2 y₀]
        exact hp
    rw [hEq]
    exact hImage
  refine ⟨g, ⟨hg, ?_⟩, ?_⟩
  · intro p
    change f (p.1, y₀) = f p
    exact hconst p.1 y₀ p.2
  · intro g' hg'
    funext x
    have hx := hg'.2 (x, y₀)
    simp [g] using hx
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__6458c5ee

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6458c5ee2d242fa2

4.89 089. product_projections_are_quotient_maps

The problem

Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Lean statement

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Reused the parent's openness checkpoints for the first and second product projections; the new proof additionally establishes continuity and surjectivity to obtain quotient maps.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from openness to quotient-map status. Proving this requires continuity and surjectivity from the nonempty factor hypotheses in addition to the parent's open-map argument, so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to quotient-map status, requiring continuity and surjectivity in addition to the parent's open-map lemmas. The nonempty-factor hypotheses are used essentially to establish surjectivity, so recalling the parent proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to quotient-map status, requiring continuity and surjectivity in addition to the projection openness. The nonempty-factor hypotheses are mathematically necessary for surjectivity, so recalling the parent does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2812 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem product_projections_are_quotient_maps.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : Nonempty X] [inst_3 : Nonempty Y] (π₁ : X × Y → X)
  (π₂ : X × Y → Y) (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, each nonempty. Let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then π_1 and π_2 are both quotient maps.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂ := by
  have hopen1 : IsOpenMap π₁ := by
    simpa [h₁] using (isOpenMap_fst : IsOpenMap (Prod.fst : X × Y → X))
  have hopen2 : IsOpenMap π₂ := by
    simpa [h₂] using (isOpenMap_snd : IsOpenMap (Prod.snd : X × Y → Y))
  have hcont1 : Continuous π₁ := by
    simpa [h₁] using (continuous_fst : Continuous (Prod.fst : X × Y → X))
  have hcont2 : Continuous π₂ := by
    simpa [h₂] using (continuous_snd : Continuous (Prod.snd : X × Y → Y))
  have hsurj1 : Function.Surjective π₁ := by
    rw [h₁]
    intro x
    obtain ⟨y⟩ := ⟨Nonempty Y⟩
    exact ⟨(x, y), rfl⟩
  have hsurj2 : Function.Surjective π₂ := by
    rw [h₂]
    intro y
    obtain ⟨x⟩ := ⟨Nonempty X⟩
    exact ⟨(x, y), rfl⟩
  have hquot1 : Topology.IsQuotientMap π₁ :=
    hopen1.isQuotientMap hcont1 hsurj1
  have hquot2 : Topology.IsQuotientMap π₂ :=
    hopen2.isQuotientMap hcont2 hsurj2
  exact ⟨hquot1, hquot2⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__a3e56ed4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a3e56ed4a85fb36c

4.90 090. projection_fst_snd_open

The problem

Let X , Y , and Z be topological spaces. Show that the projection $(X \times Y) \times Z \rightarrow X \times Z$, $(x, y, z) \mapsto (x, z)$, is an open map.

Lean statement

```
theorem projection_fst_snd_open {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] :
  IsOpenMap (fun p : (X × Y) × Z => (p.1.1, p.2)) := by
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Uses the parent's binary projection openness checkpoint 'isOpenMap_snd', strengthened through product-map composition and reassociation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes to a three-factor product and projects onto non-adjacent factors. Its proof requires product-map composition through a product homeomorphism, which is not supplied by recalling the parent's separate two-factor projection proofs.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two-factor coordinate projections to a three-factor projection onto $X \times Z$. Its proof requires product-map composition with a product associativity homeomorphism, which is not supplied by the parent's direct projection lemmas.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes to a three-factor projection onto $X \times Z$, requiring product-map composition with a product-associativity homeomorphism. The parent's direct 'fst'/'snd' applications do not supply this construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2312 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem projection_fst_snd_open.extracted_1_1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] :
  IsOpenMap fun p => (p.1.1, p.2)
```

Read back by a model that saw only the Lean. *For topological spaces X , Y , and Z , the map from $(X \times Y) \times Z$ to $X \times Z$ defined by $((x, y), z) \mapsto (x, z)$ is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem projection_fst_snd_open {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] :
  IsOpenMap (fun p : (X × Y) × Z => (p.1.1, p.2)) := by
  have hprod : IsOpenMap (Prod.map (id : X → X) (Prod.snd : Y × Z → Z)) :=
    IsOpenMap.id.prodMap isOpenMap_snd
  have hassoc : IsOpenMap (Homeomorph.prodAssoc X Y Z : (X × Y) × Z ≅ X × (Y × Z)) :=
    (Homeomorph.prodAssoc X Y Z).isOpenMap
  have hcomp : IsOpenMap ((Prod.map (id : X → X) (Prod.snd : Y × Z → Z)) ∘ (Homeomorph.prodAssoc X Y Z)) :=
    hprod.comp hassoc
  simpa [Function.comp_def] using hcomp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__b7b89168

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b7b89168a7a88528

4.91 091. continuous_strict_order_open

The problem

Let Y be an ordered set in the order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) < g(x)\}$ is open in X .

Lean statement

```
theorem continuous_strict_order_open {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsOpen {x | f x < g x} := by
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the relation and topological predicate, proving openness of the strict-order locus via ‘isOpen_lt’; the parent’s closedness result via ‘isClosed_le’ does not supply this claim. The proof follows the plan directly and introduces no recall wrapper or trivial appended fact.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the relation and topological predicate from closedness of the non-strict order locus to openness of the strict-order locus. Its proof requires the distinct theorem ‘isOpen_lt’, so recalling the parent’s ‘isClosed_le’ proof does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the relation and topological predicate, from closedness of the non-strict-order locus to openness of the strict-order locus. Its proof uses the distinct strict-order continuity lemma ‘isOpen_lt’, so recalling the parent’s ‘isClosed_le’ proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2234 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_strict_order_open.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsOpen {x | f x < g x}
```

Read back by a model that saw only the Lean. *Let X be a topological space and let Y be a linearly ordered topological space equipped with its order topology. If $f : X \rightarrow Y$ and $g : X \rightarrow Y$ are continuous, then the set $\{x \in X \mid f(x) < g(x)\}$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_strict_order_open {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsOpen {x | f x < g x} := by
have hstrict : IsOpen {x | f x < g x} := by
  exact isOpen_lt hf hg
exact hstrict
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__20a4ae19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20a4ae1976535b65

4.92 092. finite_simultaneous_comparison_closed

The problem

Let Y be a linearly ordered topological space and let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a nonempty finite set. Show that the set of points satisfying $f_i(x) \leq g_i(x)$ for every i is closed in X .

Lean statement

```
theorem finite_simultaneous_comparison_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (f g : ι → X → Y) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Used as the local checkpoint ‘hlocal’, proving each indexed comparison locus is closed before forming the master intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one comparison to a simultaneous family and combines the pairwise closed loci via an intersection theorem. A memorized proof of the parent only supplies the local closedness step, not the family-level intersection argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one comparison to a finite simultaneous comparison, requiring identification with an indexed intersection and application of closedness under arbitrary intersections. The parent’s direct ‘isClosed_le’ proof does not supply this finite-family reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine finite-family generalization: it converts each pairwise comparison locus into a finite intersection and uses the closedness of arbitrary intersections. This requires the new intersection/set-extensionality argument and cannot be solved by recalling the parent’s direct ‘isClosed_le’ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2909 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_simultaneous_comparison_closed.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2} {Y : Type u_3}
  [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (f g : ι → X → Y) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ (i : ι), f i x ≤ g i x}
```

Read back by a model that saw only the Lean. *Let ι be a nonempty type, let X and Y be topological spaces, and suppose Y is linearly ordered with its order topology. For functions $f, g : \iota \rightarrow X \rightarrow Y$ such that, for every $i \in \iota$, the functions $x \mapsto f(i, x)$ and $x \mapsto g(i, x)$ are continuous, the set of all $x \in X$ satisfying $f(i, x) \leq g(i, x)$ for every $i \in \iota$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_simultaneous_comparison_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (f g : ι → X → Y) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
  have hlocal : ∀ i, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hset : (⋂ i, {x : X | f i x ≤ g i x}) = {x | ∀ i, f i x ≤ g i x} := by
    ext x
    simp
  rw [← hset]
  exact isClosed_iInter hlocal
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__292188c1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-
gerprint 292188c1730cb824

4.93 093. closed_restricted_le

The problem

Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Lean statement

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Supplies the closedness checkpoint for the set where two continuous functions satisfy an order inequality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child restricts the closed inequality set to a closed subset, requiring the new topological fact that intersections of closed sets are closed. The parent checkpoint supplies only the inequality-set closedness; it does not prove the restricted result by recall alone.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the parent by intersecting the closed inequality set with an arbitrary closed set, requiring an additional closed-set intersection argument. The parent's inequality result is only one subgoal, so recalling its proof alone does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the result by restricting the comparison set to an arbitrary closed subset. It uses the parent's closedness theorem as one checkpoint, then requires the new mathematical step that intersections of closed sets are closed.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2772 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_restricted_le.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) : IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. For a closed set $s \subseteq X$ and continuous functions $f, g : X \rightarrow Y$, the set $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x ≤ g x} := by
have hineq : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hmaster : IsClosed (s ∩ {x | f x ≤ g x}) := by
  exact hs.inter hineq
have hset : {x | x ∈ s ∧ f x ≤ g x} = s ∩ {x | f x ≤ g x} := by
  ext x
  rfl
rw [hset]
exact hmaster
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__6cceb54a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6cceb54a30e3621e

4.94 094. finite_joint_comparison_isClosed

The problem

Let X be a topological space and Y a linearly ordered topological space. Let $n \in \mathbb{N}$, and for each $i = 0, \dots, n-1$, let $f_i, g_i : X \rightarrow Y$ be continuous. Show that the set $\{x \in X \mid f_i(x) \leq g_i(x) \text{ for every } i = 0, \dots, n-1\}$ is closed in X .

Lean statement

```
theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Supplied the ordered-topology comparison-closedness checkpoint via `isClosed_le`, generalized from one pair of functions to every $i : \text{Fin } n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one comparison to a finite family and must combine the pointwise closed comparison sets using an arbitrary intersection argument. The parent's 'isClosed_le' application supplies only the individual cases, not the joint closedness proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single comparison to a finite family and requires proving the joint set is an indexed intersection, then applying closedness of finite intersections. The parent's direct 'isClosed_le' proof does not supply this intersection reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces one comparison with a finite family and proves closedness via an intersection of closed sets, requiring the additional finite-intersection argument and set extensionality. Recalling the parent's direct 'isClosed_le' proof does not establish the joint result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_joint_comparison_isClosed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {n : ℕ}
  {f g : Fin n → X → Y} (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  IsClosed {x | ∀ (i : Fin n), f i x ≤ g i x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is the given topology. Let n be a natural number, and let f and g be families of functions from X to Y indexed by $\text{Fin } n$. Assume that for every i in $\text{Fin } n$, the functions f_i and g_i are continuous. Then the set of all x in X such that $f_i(x) \leq g_i(x)$ for every i in $\text{Fin } n$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
  have hcomp : ∀ i : Fin n, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hjoint : IsClosed (⋂ i : Fin n, {x | f i x ≤ g i x}) := by
    exact isClosed_iInter hcomp
  have hrepr : {x | ∀ i, f i x ≤ g i x} = ⋂ i : Fin n, {x | f i x ≤ g i x} := by
    ext x
    simp
  rw [hrepr]
  exact hjoint
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__c65444ab

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ac793e1e6ac6c933

4.95 095. connected_preimage_of_closed_connected

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. If $C \subseteq Y$ is closed and connected, then $p^{-1}(C)$ is connected.

Lean statement

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' {y} : Set Y)) :
  IsConnected (p ⁻¹' C)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

What it contributes. Supplied the quotient-map and connected-fiber hypotheses; the global connected-codomain conclusion is generalized to a closed connected subset C .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the argument to an arbitrary closed connected subset C , proving connectedness of its preimage via the restricted quotient-map theorem and connected fibers. This is not obtainable by recalling the parent's whole-space conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from connectedness of the entire domain to connectedness of the preimage of an arbitrary closed connected subset, requiring the coinducing preimage theorem and genuinely parameterizing the codomain region. It is not recoverable by recalling the parent's proof, which only handles the whole codomain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes connectedness from the whole codomain to the preimage of an arbitrary closed connected subset. Its proof uses the coinducing restriction theorem and cannot be obtained by recalling the parent's whole-space open-cover argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2202 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_of_closed_connected.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p) {C : Set Y}
  (hCclosed : IsClosed C) (hCconn : IsConnected C) (hX : ∀ (y : Y), IsConnected (p ⁻¹' {y})) :
  IsConnected (p ⁻¹' C)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and let $C \subseteq Y$ be a closed and connected set. Assume that for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage $p^{-1}(C)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' C) := by
  have hcoind : IsCoinducing p := hq.isCoinducing
  have hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y)) := hX
  have hrestricted : IsConnected (p ⁻¹' C) :=
    hcoind.isConnected_preimage_of_isClosed hfib hCclosed hCconn
  exact hrestricted
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__6b34bd44

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 6b34bd44c4e33bff

4.96 096. finite_ordered_path_connected_union

The problem

Let $A_0, A_1, \dots, A_{N-1}$ be path-connected subspaces of a topological space, with $N > 0$. If each A_n for $n > 0$ meets the union of its predecessors, prove that their union is path connected.

Lean statement

```
theorem finite_ordered_path_connected_union {X : Type*} [TopologicalSpace X] (N : ℕ) (hN : 0 < N) (A : ℕ → Set X) :
  (∀ n, n < N → IsPathConnected (A n)) →
  (∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) →
  IsPathConnected (⋃ k < N, A k)
```

Parent. Let a finite family of path-connected subspaces have a distinguished root. If every nonroot member is attached to the root through a finite rooted relation and attached members intersect, prove that the union is path connected.

Parent in Lean

```
theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  ⇐ → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (⋃ i, A i)
```

What it contributes. Provides the finite-union induction checkpoint and the path-connected union closure pattern; the child replaces root attachment by ordered predecessor attachment and makes the stage carrier path connected at every index.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a genuinely different induction on the ordered bound: each new set attaches to the entire predecessor union, not to a fixed root as in the parent. The parent proof cannot be recalled or directly instantiated to establish this weaker, sequential attachment condition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-root attachment relation with predecessor-to-union attachments and proves the result by induction on the bound. This requires genuinely new reasoning about finite initial segments and union decomposition, so recalling the parent's proof is not sufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the rooted-star attachment condition with an ordered predecessor-union condition, so the parent theorem cannot be instantiated to discharge it. Its proof requires induction on the bound and stagewise gluing to the entire preceding union, which is distinct mathematics from attaching every set to a fixed root.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_ordered_path_connected_union.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] (N : ℕ)
(hN : 0 < N) (A : ℕ → Set X) :
(∀ n < N, IsPathConnected (A n)) →
(∀ n < N, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) → IsPathConnected (⋃ k < N, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let N be a positive natural number, and let A be a family of subsets of X indexed by the natural numbers. If every set A_n with $n < N$ is path-connected, and for every $n < N$ with $n \neq 0$, the intersection of A_n with the union of all A_k for $k < n$ is nonempty, then the union of all A_k for $k < N$ is path-connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_ordered_path_connected_union {X : Type*} [TopologicalSpace X] (N : ℕ) (hN : 0 < N) (A : ℕ → Set X) :
(∀ n, n < N → IsPathConnected (A n)) →
(∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) →
IsPathConnected (⋃ k < N, A k) := by
revert hN
induction N with
| zero =>
intro hN
omega
| succ N ih =>
intro hN hA hPred
by_cases hN0 : N = 0
· subst N
have heq : (⋃ k < Nat.succ 0, A k) = A 0 := by
ext x
constructor
· intro hx
rcases mem_iUnion.1 hx with ⟨k, hx⟩
rcases mem_iUnion.1 hx with ⟨hk, hx⟩
have hk0 : k = 0 := by omega
simp [hk0] using hx
· intro hx
exact mem_iUnion.2 ⟨0, mem_iUnion.2 ⟨by omega, hx⟩⟩
rw [heq]
exact hA 0 (by omega)
· have hprev : IsPathConnected (⋃ k < N, A k) := by
apply ih (by omega)
· intro k hk
exact hA k (by omega)
· intro k hk hk0
exact hPred k (by omega) hk0
have hinter : (A N ∩ ⋃ k < N, A k).Nonempty :=
hPred N (Nat.lt_succ_self N) (by omega)
have hstage : IsPathConnected (A N ∪ ⋃ k < N, A k) :=
(hA N (Nat.lt_succ_self N)).union hprev hinter
have heq : (⋃ k < Nat.succ N, A k) = A N ∪ ⋃ k < N, A k := by
ext x
constructor
· intro hx
rcases mem_iUnion.1 hx with ⟨k, hx⟩
rcases mem_iUnion.1 hx with ⟨hk, hx⟩
```

```

by_cases hkN : k = N
· exact Or.inl (hkN ▸ hx)
· right
  exact mem_iUnion.2 (k, mem_iUnion.2 (by omega, hx))
· intro hx
  rcases hx with hx | hx
  · exact mem_iUnion.2 (N, mem_iUnion.2 (Nat.lt_succ_self N, hx))
  · rcases mem_iUnion.1 hx with ⟨k, hx⟩
    rcases mem_iUnion.1 hx with ⟨hk, hx⟩
    exact mem_iUnion.2 (k, mem_iUnion.2 (by omega, hx))
rw [heq]
exact hstage

```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me.me__6c2d5cbc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6c2d5cbcbe61ce57

4.97 097. connected_product_image_iUnion

The problem

Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f and g are uniformly continuous, show that the union of the sets $\{(f(x), g(x)) : x \in A_n\}$ is connected.

Lean statement

```
theorem connected_product_image_iUnion
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected (⋃ n, (fun x => (f x, g x)) '' A n)
```

Parent. Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f , g , and h are uniformly continuous, show that the union of the traced sets $\{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Parent in Lean

```
theorem connected_adjacent_trace_iUnion
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n, (fun x => (g (f x), h (g (f x)))) '' A n)
```

What it contributes. Supplied the predecessor-overlap chain argument and the connected-union theorem pattern; the child changes the final map to a direct continuous product map and performs the mapping only after proving union connectedness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses a genuinely different product map and proves connectedness of the union before mapping, then establishes an explicit image-iUnion equality. This is not recoverable by applying the parent's composed-trace proof and satisfies the mutation plan.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different product map $\$(f(x), g(x))\$$ and proves connectedness of the preimage union before applying the continuous map, rather than reusing the parent's traced-image overlap argument. The explicit image-of-union equivalence and new continuity/product reasoning prevent recall of the parent's final proof from directly solving it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different product map $\$(f(x), g(x))\$$ and proves connectedness of the original union before mapping, then transports it via continuity and an explicit image-iUnion equality. This reasoning is not supplied by recalling the parent's composed trace map proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2188 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_product_image_iUnion.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hPred : ∀ (n : ℕ), n ≠ 0 → (A n ∩ ⋃ k, ⋃ (_ : k < n), A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) : IsConnected (U n, (fun x => (f x, g x)) '' A n)
```

Read back by a model that saw only the Lean. Let α , β , and γ be uniform spaces. Let A be a sequence of subsets of α , and let $f : \alpha \rightarrow \beta$ and $g : \alpha \rightarrow \gamma$. Assume that for every natural number n , $A n$ is connected; for every natural number $n \neq 0$, the intersection of $A n$ with the union of all $A k$ for $k < n$ is nonempty; and f and g are uniformly continuous. Then the union over all natural numbers n of the images of $A n$ under the map $x \mapsto (f x, g x)$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_product_image_iUnion
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected (U n, (fun x => (f x, g x)) '' A n) := by
  classical
  let R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hReach : ∀ n, Relation.ReflTransGen R 0 n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      cases n with
      | zero => exact Relation.ReflTransGen.refl
      | succ n =>
        obtain ⟨x, hxN, hxPred⟩ := hPred (n + 1) (by omega)
        rcases Set.mem_iUnion.mp hxPred with ⟨k, hxPred⟩
        rcases Set.mem_iUnion.mp hxPred with ⟨hk, hxk⟩
        have hedge : R k (n + 1) := ⟨x, hxk, hxN⟩
        exact (ih k hk).tail hedge
  have hSym : Symmetric R := by
    intro i j hij
    simpa [R, Set.inter_comm] using hij
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    exact (Relation.ReflTransGen.symmetric hSym (hReach i)).trans (hReach j)
  have hUnion : IsConnected (U n, A n) := by
    simpa [R] using IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
  let T : α → β × γ := fun x => (f x, g x)
  have hT : Continuous T := by
    dsimp [T]
    exact hf.continuous.prodMk hg.continuous
  have hImage : IsConnected (T '' (U n, A n)) :=
```

```

hUnion.image T hT.continuousOn
have hEq : T '' (U n, A n) = U n, T '' A n := by
  ext y
  constructor
  · rintro ⟨x, hx, rfl⟩
    rcases Set.mem_iUnion.mp hx with ⟨n, hxn⟩
    exact Set.mem_iUnion.mpr ⟨n, ⟨x, hxn, rfl⟩⟩
  · intro hy
    rcases Set.mem_iUnion.mp hy with ⟨n, ⟨x, hxn, rfl⟩⟩
    exact ⟨x, Set.mem_iUnion.mpr ⟨n, hxn⟩, rfl⟩
rw [← hEq]
simpa [T] using hImage

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me__1c1dc987

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1c1dc987ff5b5411

4.98 098. finite_path_connected_union_rooted_relation

The problem

Let a finite family of path-connected subspaces have a distinguished root. If every nonroot member is attached to the root through a finite rooted relation and attached members intersect, prove that the union is path connected.

Lean statement

```
theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (⋃ i, A i)
```

Parent. Let $A_0, A_1, \dots, A_{n-1}$ be preconnected subspaces of a topological space, with $n > 0$ and A_0 nonempty. If each A_k for $k > 0$ meets the union of its predecessors, prove that their union is connected.

Parent in Lean

```
theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. Generalizes the parent's finite predecessor-union induction and attachment-intersection checkpoint, while replacing preconnectedness/connectedness with path connectedness and an arbitrary finite rooted index scheme.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child is not a wrapper or decoration: it replaces predecessor attachments by a finite rooted attachment relation and proves path connectedness via finite-set induction and path-connected union. A memorized proof of the parent's Nat-indexed preconnectedness theorem would not directly solve it.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the natural predecessor induction with a finite rooted attachment argument and strengthens connectedness to path connectedness for path-connected pieces. Its proof requires new finite-set induction and path-connected union reasoning, so recalling the parent does not substantially solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces natural-number predecessor induction with finite-type finset induction and strengthens connectedness to path connectedness using path-connected pieces and rooted intersection attachments. Its proof requires a genuinely different union argument and cannot be obtained by recalling the parent's theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2852 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the finiteness hypothesis on the family/index type.; Statement B requires each nonroot member to relate directly to the root, whereas Statement A allows attachment through a finite rooted relation (potentially via intermediate members).

Goal round-trip: what Lean elaborated

```
theorem finite_path_connected_union_rooted_relation.extracted_1.1.{u_2, u_1} {X : Type u_1} {ι : Type u_2}
[inst : TopologicalSpace X] (A : ι → Set X) (root : ι) (R : ι → ι → Prop) :
(∀ (i : ι), i ≠ root → R i root) →
(∀ (i j : ι), R i j → (A i ∩ A j).Nonempty) →
(∀ (i : ι), IsPathConnected (A i)) → IsPathConnected (U i, A i)
```

Read back by a model that saw only the Lean. Let X be a topological space, let ι be an index type, let A assign to each $i \in \iota$ a subset $A i$ of X , let $\text{root} \in \iota$, and let R be a relation on ι . Assume that for every $i \in \iota$ with $i \neq \text{root}$, $R i \text{root}$ holds; whenever $R i j$ holds, the intersection $A i \cap A j$ is nonempty; and each $A i$ is path-connected. Then the union $\bigcup i, A i$ is path-connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (U i, A i) := by
  intro hroot hattach hpath
  classical
  have aux : ∀ s : Finset ι,
    IsPathConnected (A root ∪ U i ∈ s, A i) := by
    intro s
    induction s using Finset.induction_on with
    | empty =>
      simpa using hpath root
    | @insert a s ha ih =>
      by_cases har : a = root
      · subst a
        have heq : A root ∪ U i ∈ insert root s, A i =
          A root ∪ U i ∈ s, A i := by
          ext x
          simp [ha]
          rw [heq]
          exact ih
      · have hinter : (A a ∩ (A root ∪ U i ∈ s, A i)).Nonempty := by
          rcases hattach a root (hroot a har) with (x, hxa, hxr)
          exact (x, hxa, Or.inl hxr)
        have hu : IsPathConnected (A a ∪ (A root ∪ U i ∈ s, A i)) :=
          (hpath a).union ih hinter
        have heq : A root ∪ U i ∈ insert a s, A i =
          A a ∪ (A root ∪ U i ∈ s, A i) := by
          ext x
          simp [Finset.mem_insert, ha, or_assoc, or_left_comm, or_comm]
          rw [heq]
```



```

      exact hu
have h := aux Finset.univ
have heq : A root U U i, A i = U i, A i := by
  ext x
  constructor
  · intro hx
    rcases hx with hx | hx
    · exact mem_iUnion.2 ⟨root, hx⟩
    · exact hx
  · intro hx
    exact Or.inr hx
simpa using heq ▸ (by simpa using h)

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me__ba621362

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint ba621362ac60aafd

4.99 099. finite_connected_union_predecessor_preconnected

The problem

Let $A_0, A_1, \dots, A_{n-1}$ be preconnected subspaces of a topological space, with $n > 0$ and A_0 nonempty. If each A_k for $k > 0$ meets the union of its predecessors, prove that their union is connected.

Lean statement

```
theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $A_0, A_1, \dots, A_{N-1}$ be connected subspaces of a topological space, where $N > 0$. If each A_n with $n > 0$ meets the union of the preceding subspaces, prove that their union is connected.

Parent in Lean

```
theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ k < N, A k)
```

What it contributes. Retains the finite predecessor-attachment decomposition and induction, while replacing stage connectedness with preconnectedness and adding a nonempty base stage.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens each finite-stage hypothesis from connectedness to preconnectedness and separately supplies nonemptiness of the base stage. Its proof must establish nonemptiness and use preconnected union reasoning, so recalling the parent's connected-union proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens each piece from connectedness to preconnectedness and adds a base nonemptiness condition, requiring separate nonemptiness propagation and a preconnected-union argument. The parent theorem cannot be recalled directly because its connectedness hypotheses are unavailable.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces connectedness of each piece with preconnectedness and supplies nonemptiness only at the base stage. The proof must reconstruct connectedness from nonemptiness plus preconnectedness and use the preconnected union lemma, so recalling the parent's connected-union argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2839 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_predecessor_preconnected.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a sequence of subsets of X indexed by the natural numbers, and let n be a natural number. If n is positive, A_0 is nonempty, every A_k with $k < n$ is preconnected, and for every natural number k with $0 < k < n$, the set A_k intersects the union of the sets A_j for $j < k$, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hbase hpre hattach
    by_cases hzero : n = 0
    · subst n
      have hne : (⋃ k ∈ Finset.range 1, A k).Nonempty := by
        simpa [Finset.mem_range] using hbase
      have hpc : IsPreconnected (⋃ k ∈ Finset.range 1, A k) := by
        simpa [Finset.mem_range] using hpre 0 (by omega)
      exact ⟨hne, hpc⟩
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (⋃ k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · exact hbase
        · intro k hk
          exact hpre k (by omega)
        · intro k hkpos hkl1
          exact hattach k hkpos (by omega)
      have hlast : IsPreconnected (A n) := hpre n (by omega)
      have hinter : ((⋃ k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        simpa [Set.inter_comm] using hattach n (by omega) (by omega)
      have hunion : IsPreconnected ((⋃ k ∈ Finset.range n, A k) ∪ A n) :=
        IsPreconnected.union' hinter hprev.isPreconnected hlast
      have heq : (⋃ k ∈ Finset.range (Nat.succ n), A k) =
```

```

      (U k ∈ Finset.range n, A k) ∪ A n := by
      simp only [Finset.mem_range]
      exact Set.biUnion_lt_succ A n
  have hfull : ((U k ∈ Finset.range n, A k) ∪ A n).Nonempty := by
    rcases hprev.nonempty with ⟨x, hx⟩
    exact ⟨x, Or.inl hx⟩
  rw [heq]
  exact ⟨hfull, hunion⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__3e7b61e1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3e7b61e100165ae6

4.100 100. connected_preimage_selected_finite_order_locus

The problem

Let $p : X \rightarrow Y$ be a quotient map with connected fibers. For a finite selected family of continuous real-valued functions f_i, g_i on Y , suppose the set of points satisfying $f_i(y) \leq g_i(y)$ for every selected index i is connected. Prove that its inverse image under p is connected.

Lean statement

```
theorem connected_preimage_selected_finite_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] (p : X → Y) (hq :
  Topology.IsQuotientMap p) (s : Finset I) (f g : I → Y → ℝ) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) (hc :
  IsConnected {y : Y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) : IsConnected (p ⁻¹' {y : Y | ∀ i ∈
  s, f i y ≤ g i y})
```

Parent. Let $p: X \rightarrow Y$ be a quotient map with connected fibers. For finitely many continuous real-valued functions f_i and g_i on Y , suppose the intersection of the loci $\{y | f_i(y) \leq g_i(y)\}$ is connected. Prove that its inverse image under p is connected.

Parent in Lean

```
theorem connected_preimage_finite_intersection_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Fintype I]
  (p : X → Y) (hq : Topology.IsQuotientMap p)
  (f g : I → Y → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (hc : IsConnected (⋂ i : I, {y : Y | f i y ≤ g i y}))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (⋂ i : I, {y : Y | f i y ≤ g i y}))
```

What it contributes. Retains the quotient-map, connected-fiber, continuous-inequality, closed-locus, and connected-preimage argument, while replacing the full index family by a selected finite subtype and explicitly consuming the point-wise/intersection representation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the index structure from a globally finite type to an arbitrary finite selected subfamily, requiring a new subtype-indexed intersection representation before the quotient-map argument applies. The parent proof cannot be recalled verbatim because it lacks this representation and does not cover arbitrary I .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from an entire fintype-indexed family to an arbitrary finite selected subfamily, requiring a subtype-indexed intersection representation and new equivalence reasoning. A memorized proof of the parent does not directly solve the altered statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the index family from an entire finite type to an arbitrary selected finite subset, requiring a subtype intersection representation and transport of connectedness through that representation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2859 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_selected_finite_order_locus.extracted_1_1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2}
{I : Type u_3} [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : Topology.IsQuotientMap p)
(s : Finset I) (f g : I → Y → ℝ) (hf : ∀ (i : I), Continuous (f i)) (hg : ∀ (i : I), Continuous (g i))
(hC : IsConnected {y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ (y : Y), IsConnected (p ⁻¹' {y})) :
IsConnected (p ⁻¹' {y | ∀ i ∈ s, f i y ≤ g i y})
```

Read back by a model that saw only the Lean. Let X, Y , and I be types, with topological space structures on X and Y . Let $p : X \rightarrow Y$ be a quotient map. Let s be a finite subset of I , and let $f, g : I \rightarrow Y \rightarrow \mathbb{R}$ be families of functions such that, for every $i \in I$, both $f(i)$ and $g(i)$ are continuous. Suppose the subset $\{y \in Y \mid \text{for every } i \in s, f(i)(y) \leq g(i)(y)\}$ is connected, and suppose that for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage under p of this subset, namely $\{x \in X \mid \text{for every } i \in s, f(i)(p(x)) \leq g(i)(p(x))\}$, is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem connected_preimage_selected_finite_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] (p : X → Y) (hq :
  Topology.IsQuotientMap p) (s : Finset I) (f g : I → Y → ℝ) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) (hC :
  IsConnected {y : Y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ y : Y, IsConnected (p ⁻¹' {y} : Set Y)) : IsConnected (p ⁻¹' {y : Y | ∀ i ∈
  s, f i y ≤ g i y}) := by
  have hrepr : {y : Y | ∀ i ∈ s, f i y ≤ g i y} = ⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y} := by
    ext y
    simp only [Set.mem_setOf_eq, Set.mem_iInter]
  constructor
  · intro hy i
    exact hy i.1 i.2
  · intro hy i hi
    exact hy ⟨i, hi⟩
  have hclosed_i : ∀ i : {i // i ∈ s}, IsClosed {y : Y | f i.1 y ≤ g i.1 y} := by
    intro i
    have hcont : Continuous (fun y : Y => f i.1 y - g i.1 y) := (hf i.1).sub (hg i.1)
    have heq : {y : Y | f i.1 y ≤ g i.1 y} = (fun y : Y => f i.1 y - g i.1 y) ⁻¹' Set.Iic 0 := by
      ext y
      simp [sub_nonpos]
    rw [heq]
    exact isClosed_Iic.preimage hcont
  have hclosed : IsClosed (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y}) := by
    exact isClosed_iInter hclosed_i
  have hc' : IsConnected (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y}) := by
    rw [← hrepr]
    exact hc
  have hresult : IsConnected (p ⁻¹' (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y})) := by
    exact hq.isCoinducing.isConnected_preimage_of_isClosed hfib hclosed hc'
  have hfinal : IsConnected (p ⁻¹' {y : Y | ∀ i ∈ s, f i y ≤ g i y}) := by
    rw [hrepr]
    exact hresult
  exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__5097ab74__x__Rudin_exercise_4_3__me.mh.xe.ms.me__d0014abf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint d0014abfd8565d9b

4.101 101. finite_connected_union_rooted

The problem

Let $A_0, \dots, A_{n-1}$ be connected subspaces of X . Suppose each A_k with $k > 0$ meets an earlier set $A_{r(k)}$, where $r(k) < k$. Show that their union is connected.

Lean statement

```
theorem finite_connected_union_rooted {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Parent in Lean

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. Provides the finite-union induction invariant and connected-union assembly pattern; the predecessor-range attachment is replaced by the explicit rooted map r .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces intersection with the entire predecessor union by an explicit rooted attachment map and must convert the chosen root intersection into membership in the finite union. This is not a forced wrapper or decoration; the new predecessor-map reasoning is absent from the parent proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces intersection with the entire predecessor union by an explicit rooted attachment map and must prove that the chosen root lies in the finite union. This requires a new membership construction and is not recoverable by simply recalling the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces attachment to the entire predecessor union with an explicit rooted attachment map, and its proof must convert the root intersection into membership in the finite union using the bound $r(k) < k$. This is genuine new reasoning beyond the parent's attachment hypothesis, not a forced wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2254 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_rooted.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] (A : ℕ → Set X)
  (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) → IsConnected (U k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a family of subsets of X indexed by the natural numbers, let n be a natural number, and let r be a function from the natural numbers to the natural numbers. If n is positive, every set A_k with $k < n$ is connected, and for every natural number k with $0 < k < n$, one has $r(k) < k$ and the intersection $A_k \cap A_{r(k)}$ is nonempty, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_rooted {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) →
  IsConnected (U k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hconn hattach
    by_cases hzero : n = 0
    · subst n
      simpa [Finset.mem_range] using hconn 0 (by omega)
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (U k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · intro k hk
          exact hconn k (by omega)
        · intro k hkpos hkl
          exact hattach k hkpos (by omega)
      have hlast : IsConnected (A n) := hconn n (by omega)
      have hroot : r n < n ∧ (A n ∩ A (r n)).Nonempty :=
        hattach n (by omega) (by omega)
      have hinter : ((U k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        rcases hroot.2 with ⟨x, hxlast, hxroot⟩
        refine ⟨x, ?_, hxlast⟩
        simp only [Set.mem_iUnion]
        exact ⟨r n, ⟨by simpa [Finset.mem_range] using hroot.1, hxroot⟩⟩
      have hunion : IsConnected ((U k ∈ Finset.range n, A k) ∪ A n) :=
        IsConnected.union hinter hprev hlast
      have heq : (U k ∈ Finset.range (Nat.succ n), A k) =
        (U k ∈ Finset.range n, A k) ∪ A n := by
        simp only [Finset.mem_range]
        exact Set.biUnion_lt_succ A n
      simpa only [heq] using hunion
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__9f6221c7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9f6221c77d66fd83

4.102 102. cumulative_overlap_connected_union

The problem

Let $\{A_n\}$ be a sequence of connected subspaces of a topological space X . Suppose that $A_{n+1} \cap \bigcup_{k \leq n} A_k \neq \emptyset$ for every n . Show that $\bigcup_n A_n$ is connected.

Lean statement

```
theorem cumulative_overlap_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hCum : ∀ n, (⋃ k ≤ n, A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Retains the parent’s connected-union strategy and `IsConnected.iUnion_of_chain` infrastructure, while replacing adjacent-set overlap with an initial-union invariant and cumulative-overlap induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens consecutive overlap to cumulative overlap and must inductively prove connectedness of each finite initial union before applying exhaustion. The parent’s adjacency-graph argument does not supply this finite-union induction or its cumulative-to-consecutive reduction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces pairwise consecutive overlap with cumulative overlap and proves connectedness through the new bounded-union sequence B , an induction, and an exhaustion equality. This proof structure is not supplied by recalling the parent’s direct intersection-chain argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens consecutive overlap to cumulative overlap and proves connectedness by induction on bounded unions, followed by an explicit exhaustion/equality argument. This requires new union decomposition and connected-chain reasoning rather than recalling the parent’s adjacent-intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1942 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cumulative_overlap_connected_union.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} (hA : ∀ (n : ℕ), IsConnected (A n)) (hCum : ∀ (n : ℕ), (⋃ k, ⋃ (_ : k ≤ n), A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space, and let $(A_n)_{n \in \mathbb{N}}$ be a sequence of connected subsets of X . Suppose that for every $n \in \mathbb{N}$, the union of A_k over all $k \leq n$ has nonempty intersection with A_{n+1} . Then the union $\bigcup_{n \in \mathbb{N}} A_n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem cumulative_overlap_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hCum : ∀ n, (⋃ k ≤ n, A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
  classical
  let B : ℕ → Set X := fun n => ⋃ k ≤ n, A k
  have hStep : ∀ n, B (n + 1) = B n ∪ A (n + 1) := by
    intro n
    ext x
    simp only [B, mem_iUnion, mem_setOf_eq, mem_union]
  constructor
  · rintro ⟨k, hk, hx⟩
    by_cases hkn : k ≤ n
    · exact Or.inl ⟨k, hkn, hx⟩
    · have hkeq : k = n + 1 := by omega
      exact Or.inr (hkeq ▸ hx)
  · rintro ⟨⟨k, hk, hx⟩ | hx⟩
    · exact ⟨k, Nat.le_trans hk (Nat.le_succ n), hx⟩
    · exact ⟨n + 1, le_rfl, hx⟩
  have hB : ∀ n, IsConnected (B n) := by
    intro n
    induction n with
    | zero =>
      simpa [B] using hA 0
    | succ n ih =>
      rw [hStep n]
      apply IsConnected.union
      · exact (Set.nonempty_iff_ne_empty).2 (hCum n)
      · exact ih
      · exact hA (n + 1)
  have hChain : ∀ n, (B n ∩ B (n + 1)).Nonempty := by
    intro n
    rw [hStep n]
    rcases (Set.nonempty_iff_ne_empty).2 (hCum n) with ⟨x, hxB, hxA⟩
    exact ⟨x, hxB, Or.inr hxA⟩
  have hAll : IsConnected (⋃ n, B n) :=
    IsConnected.iUnion_of_chain hB (by
      intro n
      simpa [Nat.succ_eq_add_one] using hChain n)
  have hEq : (⋃ n, B n) = ⋃ n, A n := by
    ext x
    simp only [B, mem_iUnion]
  constructor
  · rintro ⟨n, k, hk, hx⟩
    exact ⟨k, hx⟩
  · rintro ⟨k, hx⟩
    exact ⟨k, k, le_rfl, hx⟩
  rw [hEq] at hAll
```

exact hAll

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__03b7f6c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 88ca384e0f975c8c

4.103 103. finite_connected_union_predecessor

The problem

Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Lean statement

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Generalizes the connected-union argument from consecutive overlaps in an infinite sequence to a finite family whose each noninitial member meets the union of all predecessors.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces an infinite chain with a finite family whose each new set intersects the entire predecessor union, requiring induction and a connected-union argument absent from the parent's consecutive-overlap proof. It is not recoverable by recalling the parent theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the infinite consecutive-overlap argument with a finite induction in which each new set intersects the entire predecessor union. This requires a new connected-union induction and is not obtainable by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces consecutive overlaps in an infinite family with bounded attachment to the entire predecessor union and proves the result by finite induction. This requires a genuinely new induction-and-union argument not supplied by the parent's relation-chain proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2237 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_predecessor.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a sequence of subsets of X indexed by the natural numbers, and let n be a natural number. If n is positive, every set A_k with $k < n$ is connected, and for every natural number k with $0 < k < n$, the set A_k has a nonempty intersection with the union of the sets A_j for $j < k$, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hconn hattach
    by_cases hzero : n = 0
    · subst n
      simpa [Finset.mem_range] using hconn 0 (by omega)
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (⋃ k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · intro k hk
          exact hconn k (by omega)
        · intro k hkpos hklt
          exact hattach k hkpos (by omega)
      have hlast : IsConnected (A n) := hconn n (by omega)
      have hinter : ((⋃ k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        simpa [Set.inter_comm] using hattach n (by omega) (by omega)
      have hunion : IsConnected ((⋃ k ∈ Finset.range n, A k) ∪ A n) :=
        IsConnected.union hinter hprev hlast
      have heq : (⋃ k ∈ Finset.range (Nat.succ n), A k) =
        (⋃ k ∈ Finset.range n, A k) ∪ A n := by
        simp only [Finset.mem_range]
        exact Set.biUnion_lt_succ A n
      rw [heq]
      exact hunion
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__2746d5bb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2746d5bb611de8d4

4.104 104. bounded_predecessor_connected_iUnion

The problem

Let (A_n) be connected subspaces of X . Suppose that for every $n > 0$, A_n intersects the union of its predecessors $\text{igcup}_{k < n} A_k$. Show that $\text{igcup}_n A_n$ is connected.

Lean statement

```
theorem bounded_predecessor_connected_iUnion {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ n, A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Retains the parent's connected indexed-union and overlap-relation framework, while replacing consecutive overlap with cumulative predecessor overlap and supplying a strong-induction bridge.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes consecutive overlap to intersection with the entire bounded predecessor union. Its proof requires strong induction and extracting an arbitrary predecessor index before chaining reachability, which is not supplied by the parent's successor-chain argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes consecutive overlap to intersection with any bounded predecessor union. Its proof requires strong induction to establish reachability and a new graph-connectivity argument, which the parent's adjacent-chain proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces consecutive overlap with bounded-predecessor overlap and proves reachability by strong induction, introducing reasoning absent from the parent's successor-chain argument. The changed hypothesis and proof are mathematically substantive rather than decorative or recall-based.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2142 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_predecessor_connected_iUnion.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} (hA : ∀ (n : ℕ), IsConnected (A n))
  (hPred : ∀ (n : ℕ), n ≠ 0 → (A n ∩ ⋃ k, A k).Nonempty) : IsConnected (⋃ n, A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let $(A_n)_{n \in \mathbb{N}}$ be a family of subsets of X . Assume that A_n is connected for every natural number n . Assume also that for every natural number $n \neq 0$, the intersection of A_n with the union of all A_k for natural numbers $k < n$ is nonempty. Then the union of all A_n over $n \in \mathbb{N}$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_predecessor_connected_iUnion {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ n, A n) := by
  classical
  let R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hReach : ∀ n, Relation.ReflTransGen R 0 n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      cases n with
      | zero => exact Relation.ReflTransGen.refl
      | succ n =>
        obtain ⟨x, hxN, hxPred⟩ := hPred (n + 1) (by omega)
        rcases Set.mem_iUnion.mp hxPred with ⟨k, hxPred⟩
        rcases Set.mem_iUnion.mp hxPred with ⟨hk, hxk⟩
        have hlt : k < n + 1 := hk
        have hedge : R k (n + 1) := ⟨x, hxk, hxN⟩
        exact (ih k hlt).tail hedge
  have hSym : Symmetric R := by
    intro i j hij
    simpa [R, Set.inter_comm] using hij
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    exact (Relation.ReflTransGen.symmetric hSym (hReach i)).trans (hReach j)
  have hUnion : IsConnected (⋃ n, A n) := by
    simpa [R] using IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
  exact hUnion
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__7edaafef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7edaafef58792997

4.105 105. bounded_finite_predecessor_connected_union

The problem

Let $A_0, A_1, \dots, A_{N-1}$ be connected subspaces of a topological space, where $N > 0$. If each A_n with $n > 0$ meets the union of the preceding subspaces, prove that their union is connected.

Lean statement

```
theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ k < N, A k)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Supplies the connected-union topology argument and the `IsConnected.union` checkpoint; the mutation replaces consecutive overlap with finite predecessor-union attachment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine finite ordered-attachment generalization: each later set may intersect any earlier union rather than only its immediate predecessor, so the parent's successor-chain argument does not apply. Its induction must establish connectedness of successive finite unions and prove the corresponding union decomposition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces adjacent overlap on an infinite sequence with predecessor-overlap for a finite ordered family. Its proof requires induction on finite unions and explicitly rewriting the union decomposition, reasoning not supplied by the parent's relation-chain argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces infinite adjacent overlaps with a finite ordered attachment scheme: each noninitial set must meet the entire earlier union. Its induction must establish connectedness incrementally and prove the finite-union decomposition, which the parent's successor-chain argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2830 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_finite_predecessor_connected_union.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} {N : ℕ} (hN : 0 < N) (hA : ∀ (n : ℕ), IsConnected (A n))
  (hPred : ∀ n < N, n ≠ 0 → (A n ∩ U k, U ( _ : k < n), A k).Nonempty) : IsConnected (U k, U ( _ : k < N), A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let $(A_n)_{n \in \mathbb{N}}$ be a family of subsets of X , and let $N \in \mathbb{N}$ with $N > 0$. Suppose that every A_n is connected, and that for every $n < N$ with $n \neq 0$, the set A_n intersects the union of all A_k with $k < n$. Then the union of all A_k with $k < N$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ U k < n, A k).Nonempty) :
  IsConnected (U k < N, A k) := by
  classical
  have hUnion : ∀ m, 0 < m → m ≤ N → IsConnected (U k < m, A k) := by
    intro m
    induction m with
    | zero =>
      intro hm hle
      omega
    | succ n ih =>
      intro hm hle
      by_cases hn : n = 0
      · subst n
        simpa using hA 0
      · have hPrev : IsConnected (U k < n, A k) := ih (by omega) (by omega)
        have hOverlap : ((U k < n, A k) ∩ A n).Nonempty := by
          simpa [Set.inter_comm] using hPred n (by omega) hn
        have hStep : IsConnected ((U k < n, A k) ∪ A n) :=
          IsConnected.union hOverlap hPrev (hA n)
        have hEq : (U k < n + 1, A k) = (U k < n, A k) ∪ A n := by
          ext x
          simp only [Set.mem_iUnion, Set.mem_union]
        constructor
        · rintro ⟨k, hk, hx⟩
          by_cases hkn : k < n
          · exact Or.inl ⟨k, hkn, hx⟩
          · have hkeq : k = n := by omega
            exact Or.inr (hkeq ▸ hx)
        · rintro ⟨⟨k, hk, hx⟩ | hx⟩
          · exact ⟨k, by omega, hx⟩
          · exact ⟨n, by omega, hx⟩
      rw [hEq]
      exact hStep
  exact hUnion N hN le_refl
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__9a77ee70

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9a77ee70d2ecef65

4.106 106. bounded_prefix_connected

The problem

Let X be a topological space and let $\{A_n\}$ be a sequence of connected subspaces of X such that $A_n \cap A_{n+1} \neq \emptyset$ for every n . Show that for every natural number N , the union $A_0 \cup A_1 \cup \dots \cup A_N$ is connected.

Lean statement

```
theorem bounded_prefix_connected {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
  IsConnected (⋃ n ∈ Finset.range (N + 1), A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Uses the parent's connected-set and consecutive-overlap structure, specialized to an explicit finite prefix and consumed through induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves connectedness for each finite prefix by induction, explicitly decomposing the prefix union and establishing overlap with the next set. This bounded exhaustion argument is mathematically distinct from the parent's direct connectedness proof for the full infinite union.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new finite-prefix connectedness theorem by induction, constructing the prefix-to-next-set overlap and rewriting the finite union at each step. This reasoning is not supplied by the parent's full-union argument and is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded-prefix theorem by induction, explicitly decomposing each finite union and establishing overlap with the next set. This reasoning is not supplied by recalling the parent's full-union connectedness proof, and it directly satisfies the planned bounded generalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1938 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prefix_connected.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
  IsConnected (U n ∈ Finset.range (N + 1), A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let A be a sequence of subsets of X indexed by the natural numbers. Suppose that each set $A(n)$ is connected, and that for every natural number n , the intersection $A(n) \cap A(n+1)$ is nonempty. Then, for every natural number N , the union of the sets $A(0), A(1), \dots, A(N)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_prefix_connected {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
  IsConnected (U n ∈ Finset.range (N + 1), A n) := by
  induction N with
  | zero =>
    simpa using hA 0
  | succ N ih =>
    change IsConnected (U n ∈ Finset.range (N + 2), A n)
    have hPrefix : A N ⊆ U n ∈ Finset.range (N + 1), A n := by
      intro x hx
      refine Set.mem_iUnion.2 ⟨N, ?_⟩
      refine Set.mem_iUnion.2 ⟨by simp, hx⟩
    have hOverlap :
      ((U n ∈ Finset.range (N + 1), A n) ∩ A (N + 1)).Nonempty := by
      have hAdj : (A N ∩ A (N + 1)).Nonempty :=
        (Set.nonempty_iff_ne_empty).2 (hAn N)
      rcases hAdj with ⟨x, hxN, hxNext⟩
      exact ⟨x, hPrefix hxN, hxNext⟩
    have hUnion :
      (U n ∈ Finset.range (N + 2), A n) =
        (U n ∈ Finset.range (N + 1), A n) ∪ A (N + 1) := by
      ext x
      constructor
      · intro hx
        rcases Set.mem_iUnion.1 hx with ⟨n, hx⟩
        rcases Set.mem_iUnion.1 hx with ⟨hn, hxn⟩
        by_cases h : n = N + 1
        · right
          simpa [h] using hxn
        · left
          refine Set.mem_iUnion.2 ⟨n, ?_⟩
          refine Set.mem_iUnion.2 ⟨?_, hxn⟩
          have hn' : n < N + 2 := Finset.mem_range.1 hn
          exact Finset.mem_range.2 (by omega)
      · intro hx
        rcases hx with hx | hx
        · rcases Set.mem_iUnion.1 hx with ⟨n, hx⟩
          rcases Set.mem_iUnion.1 hx with ⟨hn, hxn⟩
          refine Set.mem_iUnion.2 ⟨n, ?_⟩
          refine Set.mem_iUnion.2 ⟨?_, hxn⟩
          have hn' : n < N + 1 := Finset.mem_range.1 hn
          exact Finset.mem_range.2 (by omega)
        · refine Set.mem_iUnion.2 ⟨N + 1, ?_⟩
          refine Set.mem_iUnion.2 ⟨?_, hx⟩
```

```
exact Finset.mem_range.2 (by omega)
have hConnected := IsConnected.union hOverlap ih (hA (N + 1))
rw [hUnion]
exact hConnected
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__9f0e11a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ed46bedda43616df

4.107 107. two_sided_integer_connected_union

The problem

Let X be a topological space and let $\{B_z\}_{z \in \mathbb{Z}}$ be a family of connected subspaces of X such that $B_z \cap B_{z+1} \neq \emptyset$ for every $z \in \mathbb{Z}$. Show that $\bigcup_{z \in \mathbb{Z}} B_z$ is connected.

Lean statement

```
theorem two_sided_integer_connected_union {X : Type*} [TopologicalSpace X]
  {B : ℤ → Set X} (hB : ∀ z, IsConnected (B z))
  (hAdj : ∀ z : ℤ, (B z ∩ B (z + 1)).Nonempty) :
  IsConnected (⋃ z : ℤ, B z)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23.2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Reuse the connected-chain union strategy on each of the two $\mathbb{N}$ -indexed rays, while adding an explicit integer split and zero gluing step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the one-sided natural chain to a two-sided integer-indexed chain. Its proof requires splitting positive and negative indices into two connected rays, proving their intersection is nonempty, and establishing the union reindexing equality before gluing them.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new two-sided integer reindexing: it decomposes the chain into positive and negative natural rays, proves each ray connected, and glues them across the $-1/0$ intersection. This reasoning is not supplied by the parent's one-sided chain proof and is not a forced-witness or decorative wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the one-sided natural chain to a two-sided integer-indexed chain, requiring decomposition into positive and negative rays, reindexing, and a nontrivial gluing argument. This reasoning is not supplied by recalling the parent's natural-number union proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1935 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem two_sided_integer_connected_union.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {B : ℤ → Set X}
(hB : ∀ (z : ℤ), IsConnected (B z)) (hAdj : ∀ (z : ℤ), (B z ∩ B (z + 1)).Nonempty) : IsConnected (U z, B z)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let B be a family of subsets of X indexed by the integers. Suppose that for every integer z , the set $B(z)$ is connected, and that for every integer z , the intersection $B(z) \cap B(z + 1)$ is nonempty. Then the union $\bigcup_{z \in \mathbb{Z}} B(z)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem two_sided_integer_connected_union {X : Type*} [TopologicalSpace X]
{B : ℤ → Set X} (hB : ∀ z, IsConnected (B z))
(hAdj : ∀ z : ℤ, (B z ∩ B (z + 1)).Nonempty) :
IsConnected (U z : ℤ, B z) := by
classical
let P : ℕ → Set X := fun n => B (n : ℤ)
let N : ℕ → Set X := fun n => B (-(n : ℤ) - 1)
have hPconn : ∀ n, IsConnected (P n) := by
  intro n
  simpa [P] using hB (n : ℤ)
have hNconn : ∀ n, IsConnected (N n) := by
  intro n
  simpa [N] using hB (-(n : ℤ) - 1)
have hPedge : ∀ n, (P n ∩ P (n + 1)).Nonempty := by
  intro n
  simpa [P] using hAdj (n : ℤ)
have hNedge : ∀ n, (N n ∩ N (n + 1)).Nonempty := by
  intro n
  have h := hAdj (-(n : ℤ) - 2)
  simpa [N, Set.inter_comm, sub_eq_add_neg, add_assoc, add_comm, add_left_comm] using h
have hPos : IsConnected (U n : ℕ, P n) := by
  exact IsConnected.iUnion_of_chain hPconn hPedge
have hNeg : IsConnected (U n : ℕ, N n) := by
  exact IsConnected.iUnion_of_chain hNconn hNedge
have hGluue : ((U n : ℕ, P n) ∩ (U n : ℕ, N n)).Nonempty := by
  obtain ⟨x, hx⟩ := hAdj (-1)
  rcases hx with ⟨hxneg, hxzero⟩
  refine ⟨x, Set.mem_iUnion.2 ⟨0, ?_⟩, Set.mem_iUnion.2 ⟨0, ?_⟩⟩
  · simpa [P] using hxzero
  · simpa [N] using hxneg
have hRayUnion : IsConnected ((U n : ℕ, P n) ∪ (U n : ℕ, N n)) := by
  exact IsConnected.union hGluue hPos hNeg
have hEq : (U z : ℤ, B z) = (U n : ℕ, P n) ∪ (U n : ℕ, N n) := by
  ext x
  constructor
  · intro hx
    rcases Set.mem_iUnion.1 hx with ⟨z, hxz⟩
    cases z with
    | ofNat n =>
      exact Or.inl (Set.mem_iUnion.2 ⟨n, by simpa [P] using hxz⟩)
    | negSucc n =>
      have hz : (Int.negSucc n : ℤ) = -(n : ℤ) - 1 := by omega
      apply Or.inr
      refine Set.mem_iUnion.2 ⟨n, ?_⟩
      change x ∈ B (-(n : ℤ) - 1)
      rw [← hz]
      exact hxz
  · intro hx
```



```

rcases hx with hx | hx
· rcases Set.mem_iUnion.1 hx with ⟨n, hxn⟩
  exact Set.mem_iUnion.2 ⟨(n : ℤ), by simp [P] using hxn⟩
· rcases Set.mem_iUnion.1 hx with ⟨n, hxn⟩
  exact Set.mem_iUnion.2 ⟨-(n : ℤ) - 1, by simp [N] using hxn⟩
rw [hEq]
exact hRayUnion

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__b8d24dc9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 813e5feaffbc950b

4.108 108. parameterized_closed_interval_fixed_point

The problem

Let $a, b \in \mathbb{R}$ with $a < b$, and let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous and map $[a, b]$ into itself. Show that there exists $t \in [0, 1]$ such that $\frac{f(a+(b-a)t)-a}{b-a} = t$ and $f(a + (b - a)t) = a + (b - a)t$.

Lean statement

```
theorem parameterized_closed_interval_fixed_point (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f) (hmaps : MapsTo f (Set.Icc a b)
  ↪ (Set.Icc a b)) : ∃ t ∈ Set.Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Retains the parent's continuous self-map fixed-point structure, while replacing the fixed unit interval with an explicitly parameterized interval and proving the affine transfer obligations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-point result to an arbitrary nondegenerate interval and requires constructing and proving properties of the affine transfer map and normalized self-map. The affine algebraic transfer back to a fixed point is new reasoning not supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the interval and constructs an affine transfer map, then proves that its normalized fixed point yields a fixed point of f . The division by $b-a$ and interval-mapping argument require reasoning absent from the fixed $[0,1]$ parent instance.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the interval and transfers the self-map through the affine map $\varphi(t)=a+(b-a)t$, requiring new continuity, mapping, positivity, and division reasoning. Its proof uses a different real-valued fixed-point construction rather than recalling the subtype proof for $[0,1]$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1860 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_closed_interval_fixed_point.extracted_1_1 (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f)
(hmaps : MapsTo f (Icc a b) (Icc a b)) :
  ∃ t ∈ Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t
```

Read back by a model that saw only the Lean. For real numbers a and b with $a < b$, and a continuous function $f: \mathbb{R} \rightarrow \mathbb{R}$ that maps the closed interval $[a, b]$ into itself, there exists $t \in [0, 1]$ such that $(f(a + (b - a)t) - a)/(b - a) = t$ and $f(a + (b - a)t) = a + (b - a)t$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem parameterized_closed_interval_fixed_point (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f) (hmaps : MapsTo f (Set.Icc a b)
  ↪ (Set.Icc a b)) : ∃ t ∈ Set.Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t := by
  let φ : ℝ → ℝ := fun t => a + (b - a) * t
  let g : ℝ → ℝ := fun t => (f (φ t) - a) / (b - a)
  have hφ_cont : Continuous φ := by
    dsimp [φ]
    fun_prop
  have hg_cont : Continuous g := by
    dsimp [g]
    fun_prop
  have hg_maps : MapsTo g (Set.Icc 0 1) (Set.Icc 0 1) := by
    intro t ht
    have hφ_mem : φ t ∈ Set.Icc a b := by
      dsimp [φ]
      constructor <|> nlinarith [ht.1, ht.2, hab]
    have hf_mem : f (φ t) ∈ Set.Icc a b := hmaps hφ_mem
    constructor
    · apply (le_div_iff₀ (sub_pos.mpr hab)).2
      nlinarith [hf_mem.1]
    · apply (div_le_iff₀ (sub_pos.mpr hab)).2
      nlinarith [hf_mem.2]
  obtain ⟨t, ht, hfixed⟩ := exists_mem_Icc_isFixedPt_of_mapsTo hg_cont.continuousOn (by norm_num) hg_maps
  have htransfer : (f (φ t) - a) / (b - a) = t := by
    simp [g] using hfixed
  have htransfer' : f (φ t) - a = t * (b - a) := by
    calc
      f (φ t) - a = ((f (φ t) - a) / (b - a)) * (b - a) := by
        field_simp [ne_of_gt (sub_pos.mpr hab)]
      _ = t * (b - a) := by rw [htransfer]
  have hfixed_f : f (φ t) = φ t := by
    dsimp [φ]
    nlinarith [htransfer']
  refine ⟨t, ht, ?_, ?_⟩
  · simp [φ] using htransfer
  · simp [φ] using hfixed_f
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__me__df176385

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 324ad392b04ea85f

4.109 109. localized_compact_trace_basis

The problem

Let K be a compact metric space, let A be a closed subset containing x , and let U be an open neighborhood of x . Show that some open neighborhood V of x has compact closure contained in U , and that $A \cap \overline{V}$ has a countable topological basis.

Lean statement

```
theorem localized_compact_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) (x : K)
  (U : Set K) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set K) (B : Set (Set {y : K // y ∈ A ∩ closure V})),
    IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧
    IsClosed (A ∩ closure V) ∧ Set.Countable B ∧
    TopologicalSpace.IsTopologicalBasis B
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\overline{V}$ is compact and $\overline{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Supplied the compact local-core proof skeleton: choose a neighborhood core, pass to an open interior, control its closure by a compact closed set, and retain containment in U .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires proving compactness of the localized subtype and constructing a countable topological basis for it, obligations absent from the parent's compact-neighborhood construction. Its proof materially consumes the closed intersection and subtype compactness rather than merely reusing the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: it proves the localized intersection $A \cap \text{closure } V$ is closed and compact, then constructs a countable topological basis for its subtype. Its proof does not recall or reuse the parent's separation argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine compactness argument for the localized intersection $A \cap \overline{V}$: it proves closedness, derives compactness of the corresponding subtype, and constructs a countable topological basis. These obligations are absent from and cannot be discharged by recalling the parent's compact-neighborhood proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2921 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B replaces the conclusion that $A \cap \text{closure}(V)$ has a countable topological basis with the weaker assertion that $A \cap \text{closure}(V)$ is closed; its separate countable basis is for K , not explicitly for the intersection.

Goal round-trip: what Lean elaborated

```
theorem localized_compact_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] (A : Set K) (hA : IsClosed A) (x : K) (hxA : x ∈ A) (U : Set K) (hU : IsOpen U)
  (hxU : x ∈ U) :
  ∃ V B,
    IsOpen V ∧
    x ∈ V ∧
    IsCompact (closure V) ∧ closure V ⊆ U ∧ IsClosed (A ∩ closure V) ∧ B.Countable ∧ IsTopologicalBasis B
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, let A be a closed subset of K , let $x \in A$, and let U be an open subset of K containing x . Then there exist an open set $V \subseteq K$ and a countable topological basis B for K such that $x \in V$, the closure of V is compact, the closure of V is contained in U , and $A \cap \text{closure}(V)$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem localized_compact_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) (x : K)
  (U : Set K) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set K) (B : Set (Set {y : K // y ∈ A ∩ closure V})),
    IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧
    IsClosed (A ∩ closure V) ∧ Set.Countable B ∧
    TopologicalSpace.IsTopologicalBasis B := by
  classical
  obtain ⟨C, hCcompact, hxC, hCU⟩ := exists_compact_subset hU hxU
  let V : Set K := interior C
  have hVopen : IsOpen V := isOpen_interior
  have hxV : x ∈ V := hxC
  have hclosureV : closure V ⊆ C :=
    closure_minimal interior_subset hCcompact.isClosed
  have hclosureCompact : IsCompact (closure V) :=
    hCcompact.of_isClosed_subset isClosed_closure hclosureV
  have hclosureU : closure V ⊆ U := hclosureV.trans hCU
  have hclosedTrace : IsClosed (A ∩ closure V) :=
    hA.inter isClosed_closure
  have hcompactTrace : IsCompact (A ∩ closure V) :=
    hclosureCompact.of_isClosed_subset hclosedTrace interior_subset_right
  let I : CompactSpace {y : K // y ∈ A ∩ closure V} :=
    isCompact_iff_compactSpace.mp hcompactTrace
  have hcount : (countableBasis {y : K // y ∈ A ∩ closure V}).Countable :=
    countable_countableBasis _
  have hbasis :
    TopologicalSpace.IsTopologicalBasis
      (countableBasis {y : K // y ∈ A ∩ closure V}) :=
    isBasis_countableBasis _
  exact ⟨V, countableBasis _, hVopen, hxV, hclosureCompact, hclosureU,
    hclosedTrace, hcount, hbasis⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__8d1abbd9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint dc45fa6cf5c8fe6c

4.110 110. localCompactAt_iff_exists_compact_closed_interior

The problem

Show that, in a Hausdorff space, the point x is locally compact exactly when there is a compact closed set K whose interior contains x .

Lean statement

```
theorem localCompactAt_iff_exists_compact_closed_interior {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X) :
  (∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K)) ↔
  ∃ K : Set X, IsCompact K ∧ IsClosed K ∧ x ∈ interior K
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from producing a relatively compact neighborhood inside an arbitrary open set to a two-way characterization via compact closed sets with x in their interior. Its proof uses interior and neighborhood equivalences in both directions, rather than recalling the parent's separation-and-shrinking argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine two-way characterization, including the reverse construction using interior K , rather than reproducing the parent's closure-shrinking argument. It is not substantially helped by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's compact-closure neighborhood conclusion into a genuine two-way characterization. Its proof uses a distinct direct interior/neighborhood argument in each direction, with compactness supplying closedness, rather than recalling the parent's separation and closure construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2916 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem localCompactAt_iff_exists_compact_closed_interior.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  [inst_1 : T2Space X] (x : X) :
  (∃ U, x ∈ U ∧ IsOpen U ∧ ∃ K, U ⊆ K ∧ IsCompact K) ↔ ∃ K, IsCompact K ∧ IsClosed K ∧ x ∈ interior K
```

Read back by a model that saw only the Lean. *Let X be a topological space satisfying the T_2 (Hausdorff) separation axiom, and let x be a point of X . Then the following are equivalent: (1) there exists a set U such that x belongs to U , U is open, and there exists a compact set K containing U ; (2) there exists a compact, closed set K such that x belongs to the interior of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem localCompactAt_iff_exists_compact_closed_interior {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X) :
  (∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K)) ↔
  ∃ K : Set X, IsCompact K ∧ IsClosed K ∧ x ∈ interior K := by
  constructor
  · rintro ⟨U, hxU, hU, K, hUK, hKcompact⟩
    have hKclosed : IsClosed K := hKcompact.isClosed
    have hUnhds : U ∈ nhds x := hU.mem_nhds hxU
    have hKnhds : K ∈ nhds x := Filter.mem_of_superset hUnhds hUK
    have hxinterior : x ∈ interior K := (mem_interior_iff_mem_nhds).2 hKnhds
    exact ⟨K, hKcompact, hKclosed, hxinterior⟩
  · rintro ⟨K, hKcompact, hKclosed, hxinterior⟩
    have hinterior_open : IsOpen (interior K) := isOpen_interior
    have hinterior_subset : interior K ⊆ K := interior_subset
    exact ⟨interior K, hxinterior, hinterior_open, K, hinterior_subset, hKcompact⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__90c38c4d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 90c38c4d709643ee

4.111 111. compact_set_nested_open_shrink

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space. Show that there are open sets V and W such that $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ is compact, and $\overline{W} \subseteq U$.

Lean statement

```
theorem compact_set_nested_open_shrink {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\overline{V}$ is compact and $\overline{V} \subseteq U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Retains the parent's compact-closure neighborhood-shrink principle, but upgrades the protected object from a single point to a compact set and makes the second uniform shrink the final obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new nested refinement: it first obtains a compact-closure neighborhood W of K inside U , then independently shrinks again to V with $\text{closure } V \subseteq W$. The parent's point-separation argument supplies neither compact-set shrinking nor this second containment obligation.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a nested refinement: after obtaining W with compact closure inside U , it must shrink $\text{closure } V$ again so that $\text{closure } V$ is contained in W . This second application and the resulting two-level inclusion are not supplied by the parent's pointwise construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from a single point to an arbitrary compact set and requires a genuinely nested refinement: first compactly shrinking into U , then shrinking again so that $\text{closure } V$ is contained in W while $\text{closure } W$ remains compact and contained in U . Its proof uses the compact-set shrinking theorem twice and does not recall or reuse the parent's point-separation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2327 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_set_nested_open_shrink.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  [inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧ IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U
```

Read back by a model that saw only the Lean. *Let X be a locally compact Hausdorff topological space, and let K and U be subsets of X . Suppose that K is compact, U is open, and $K \subseteq U$. Then there exist subsets V and W of X such that V is open, $K \subseteq V$, the closure of V is contained in W , W is open, the closure of W is compact, and the closure of W is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem compact_set_nested_open_shrink {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U := by
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hU hKU
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧ closure W ⊆ U ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hU hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsub⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__d4a2b15e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d4a2b15e8b809b20

4.112 112. prescribed_pair_unit_orthogonal

The problem

Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates. Prove that there is a unit vector supported only on i and j and orthogonal to x .

Lean statement

```
theorem prescribed_pair_unit_orthogonal
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

Parent. Let $n > 1$, let $x \in \mathbb{R}^n$, and choose distinct coordinates i and j . Prove that there is a nonzero vector supported only on these two coordinates and orthogonal to x .

Parent in Lean

```
theorem prescribed_pair_orthogonal
  (n : ℕ)

  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    y ≠ 0 ∧ ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

What it contributes. Supplies the prescribed-pair witness construction, its coordinate nonzeroness checkpoint, orthogonality calculation, and support argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine unit-norm requirement: it constructs the prescribed-pair orthogonal witness, proves its norm is nonzero, and transports orthogonality and support through normalization. This requires reasoning beyond recalling the parent's existential conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new normalization argument: it must prove the constructed supported orthogonal witness is nonzero, establish its norm is nonzero, and transport orthogonality and support through scalar multiplication. Memorizing the parent only supplies the unnormalized witness, not the unit-norm conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive normalization argument: it constructs the prescribed-pair orthogonal witness, proves its norm is nonzero, and transports orthogonality and support through scalar multiplication to obtain unit norm. Recalling the parent only supplies the unnormalized witness and does not solve the new claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2348 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_unit_orthogonal.extracted_1.1 (n : ℕ) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n)
(hij : i ≠ j) : ∃ y, ‖y‖ = 1 ∧ ⟨x, y⟩_R = 0 ∧ ∀ (k : Fin n), k ≠ i → k ≠ j → y.ofLp k = 0
```

Read back by a model that saw only the Lean. *For any natural number n , any vector x in the Euclidean space $\mathbb{R}^n$ indexed by $\text{Fin } n$, and any distinct indices $i, j \in \text{Fin } n$, there exists a vector y in the same space such that $\|y\| = 1$, the real inner product of x and y is 0, and for every index $k \in \text{Fin } n$ distinct from both i and j , the k -th coordinate of y is 0.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_unit_orthogonal
(n : ℕ)
(x : EuclideanSpace ℝ (Fin n))
(i j : Fin n)
(hij : i ≠ j) :
∃ y : EuclideanSpace ℝ (Fin n),
‖y‖ = 1 ∧
⟨x, y⟩_R = 0 ∧
∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
classical
have normalize : ∀ z : EuclideanSpace ℝ (Fin n),
z ≠ 0 →
⟨x, z⟩_R = 0 →
(∀ k : Fin n, k ≠ i → k ≠ j → z k = 0) →
∃ y : EuclideanSpace ℝ (Fin n),
‖y‖ = 1 ∧
⟨x, y⟩_R = 0 ∧
∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
intro z hz horth hsupport
let y := (‖z‖ : ℝ)⁻¹ • z
have hn : ‖z‖ ≠ 0 := norm_ne_zero_iff.mpr hz
have hnorm : ‖y‖ = 1 := by
simp [y] using (norm_smul_inv_norm hz)
have hyorth : ⟨x, y⟩_R = 0 := by
change ⟨x, (‖z‖ : ℝ)⁻¹ • z⟩_R = 0
rw [real_inner_smul_right, horth, mul_zero]
have hysupport : ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
intro k hki hkj
simp [y, hsupport k hki hkj]
exact ⟨y, hnorm, hyorth, hysupport⟩
by_cases hxi : x i = 0
· let z := EuclideanSpace.single i (1 : ℝ)
have hz : z ≠ 0 := by
intro hzero
have hi := congrArg (fun v => v i) hzero
simp [z] at hi
```

```

have horth :  $\langle x, z \rangle_R = 0 :=$  by
  simp [z, EuclideanSpace.inner_single_right, hxi]
have hsupport :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow z \cdot k = 0 :=$  by
  intro k hki hkj
  simp [z, EuclideanSpace.single_apply, hki]
exact normalize z hz horth hsupport
· let z := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
  have hji :  $j \neq i :=$  Ne.symm hij
  have hzj :  $z \cdot j = -x \cdot i :=$  by
    simp [z, EuclideanSpace.single_apply, hji]
  have hz :  $z \neq 0 :=$  by
    intro hzero
    have hz' := congrArg (fun v => v j) hzero
    have hzero_i :  $x \cdot i = 0 :=$  by
      simpa [hzj] using hz'
    exact hxi hzero_i
  have horth :  $\langle x, z \rangle_R = 0 :=$  by
    simp [z, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]
  have hsupport :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow z \cdot k = 0 :=$  by
    intro k hki hkj
    simp [z, EuclideanSpace.single_apply, hki, hkj]
  exact normalize z hz horth hsupport

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me.me__ecf6bb19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ecf6bb190714a02a

4.113 113. prescribed_pair_orthogonal

The problem

Let $n > 1$, let $\mathbf{x} \in \mathbb{R}^n$, and choose distinct coordinates i and j . Prove that there is a nonzero vector supported only on these two coordinates and orthogonal to $\mathbf{x}$.

Lean statement

```
theorem prescribed_pair_orthogonal
  (n : ℕ)

  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

Parent. If $k \geq 2$ and $\mathbf{x} \in \mathbb{R}^k$, prove that there exists $\mathbf{y} \in \mathbb{R}^k$ such that $\mathbf{y} \neq 0$ but $\mathbf{x} \cdot \mathbf{y} = 0$

Parent in Lean

```
theorem Rudin_exercise_1_18a
  (n : ℕ)
  (h : n > 1)
  (x : EuclideanSpace ℝ (Fin n)) -- ℝ^n
  : ∃ (y : EuclideanSpace ℝ (Fin n)), y ≠ 0 ∧ ⟨x, y⟩_R = 0 := by
```

What it contributes. Adapts the parent's two-coordinate orthogonal witness construction, replacing the parent's existentially chosen pair with an arbitrary prescribed distinct pair and adding an explicit zero-coordinate case split plus support conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uniformly handles an arbitrary prescribed distinct pair of coordinates, rather than choosing a pair adaptively from $\mathbf{x}$. Its zero/nonzero case split and support verification require reasoning not supplied by the parent's existential construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing an orthogonal vector supported on an arbitrary prescribed pair, using the two selected coordinates and a nonzero-coordinate case split. The parent only chooses coordinates existentially and does not supply this uniform prescribed-support argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the result to every prescribed distinct coordinate pair and requires the orthogonal vector to be supported on that pair. Its proof must analyze the two selected coordinates directly, unlike the parent's adaptive choice of coordinates from a nonzero component.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2343 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_orthogonal.extracted_1_1 (n : ℕ) (h : n > 1) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n)
(hij : i ≠ j) : ∃ y, y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧ ∀ (k : Fin n), k ≠ i → k ≠ j → y.ofLp k = 0
```

Read back by a model that saw only the Lean. *For every natural number n greater than 1, every vector x in the real Euclidean space indexed by $\text{Fin } n$, and every pair of distinct indices $i, j \in \text{Fin } n$, there exists a nonzero vector y in the same space such that the real inner product $\langle x, y \rangle$ is zero, and for every index $k \in \text{Fin } n$ distinct from both i and j , the k -th coordinate of y is zero.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_orthogonal
(n : ℕ)

(x : EuclideanSpace ℝ (Fin n))
(i j : Fin n)
(hij : i ≠ j) :
∃ y : EuclideanSpace ℝ (Fin n),
y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧
∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
classical
by_cases hxi : x i = 0
· by_cases hxj : x j = 0
· let y := EuclideanSpace.single i (1 : ℝ)
have hy : y ≠ 0 := by
intro hz
have hz' := congrArg (fun z => z i) hz
simp [y, EuclideanSpace.single_apply] at hz'
have ho : ⟨x, y⟩_R = 0 := by
simp [y, EuclideanSpace.inner_single_right, hxi]
have hs : ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
intro k hki hkj
simp [y, EuclideanSpace.single_apply, hki]
exact ⟨y, hy, ho, hs⟩
· let y := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
have hji : j ≠ i := Ne.symm hij
have hyj : y j = -x i := by
simp [y, EuclideanSpace.single_apply, hji]
have hy : y ≠ 0 := by
```

```

intro hz
have hz' := congrArg (fun z => z j) hz
have : x i = 0 := by
  simp [hyj] using hz'
exact hxi this
have ho :  $\langle x, y \rangle_R = 0$  := by
  simp [y, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]
have hs :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow y k = 0$  := by
  intro k hki hkj
  simp [y, EuclideanSpace.single_apply, hki, hkj]
exact  $\langle y, hy, ho, hs \rangle$ 

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me__58f629fc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f57f8e5f7e32f73a

4.114 114. irrational_affine_rat

The problem

If q and r are rational numbers with $q \neq 0$ and x is irrational, prove that $xq + r$ is irrational.

Lean statement

```
theorem irrational_affine_rat (x : ℝ) (q r : ℚ) (hq : q ≠ 0) : Irrational x → Irrational (x * q + r)
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

What it contributes. Supplies the nonzero rational scaling checkpoint via `hx.mul_ratCast hq`; the new proof adds a rational translation contradiction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely extends irrationality preservation from nonzero rational multiplication to a rational affine map, requiring a new contradiction argument that solves for x via subtraction and division by q . The parent's multiplication lemma alone does not supply translation invariance.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes multiplication into a rational affine embedding and proves irrationality by deriving a rational representation of x via the inverse affine transformation. This requires new witness construction and field reasoning rather than recalling the parent's multiplication lemma.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a rational translation and requires an inverse affine argument: from a rational representation of $xq+r$ it constructs $(s-r)/q$ and derives x is rational. This is not obtained by recalling the parent's direct multiplication lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2015 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_affine_rat.extracted_1_1 (x : ℝ) (q r : ℚ) (hq : q ≠ 0) :  
  Irrational x → Irrational (x * ↑q + ↑r)
```

Read back by a model that saw only the Lean. *For a real number x and rational numbers q and r , if q is nonzero, then whenever x is irrational, the real number x multiplied by q and then increased by r is irrational.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_affine_rat (x : ℝ) (q r : ℚ) (hq : q ≠ 0) : Irrational x → Irrational (x * q + r) := by  
  intro hx hrat  
  apply hx  
  rcases hrat with ⟨s, hs⟩  
  refine ⟨(s - r) / q, ?_⟩  
  rw [Rat.cast_div, Rat.cast_sub]  
  field_simp [hq]  
  linarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__me__ebdca03d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f712fecdd06d0e5a

4.115 115. rational_affine_action_free

The problem

Let x be irrational, and let r, s be rational numbers and y, z nonzero rational numbers. If $y(x + r) = z(x + s)$, prove that $y = z$ and $r = s$.

Lean statement

```
theorem rational_affine_action_free
(x : ℝ)
(hx : Irrational x)
(r s y z : ℚ)

(hz : z ≠ 0)
(h : y * (x + r) = z * (x + s)) :
y = z ∧ r = s
```

Parent. For a real number x , a rational number r , and a nonzero rational number y , prove that $y(x + r)$ is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_rat_translate_mul_iff
(x : ℝ)
(r y : ℚ)
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x
```

What it contributes. Supplies the translation checkpoint via `Irrational.add_ratCast` and the nonzero-scaling checkpoint via `Irrational.mul_ratCast`; these establish irrationality of both affine images before the new parameter-recovery obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different freeness/uniqueness theorem: equality of two rational affine images forces both rational parameters to agree. Its proof requires coefficient comparison and an irrationality contradiction, none of which is supplied by the parent's preservation equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new freeness/uniqueness theorem: irrationality forces equality of the rational coefficients, after which nonzero scaling forces equality of translations. The parent's equivalence proof does not supply this coefficient-comparison argument, and the child does not recall or embed the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine freeness/uniqueness theorem: irrationality forces equality of both rational scale and translation parameters. Its proof requires a new contradiction and rational-expression argument, not recall or reuse of the parent's irrationality equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2766 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_affine_action_free.extracted_1_1 (x : ℝ) (hx : Irrational x) (r s y z : ℚ) (hy : y ≠ 0) (hz : z ≠ 0)
(h : ↑y * (x + ↑r) = ↑z * (x + ↑s)) : y = z ∧ r = s
```

Read back by a model that saw only the Lean. *Let x be an irrational real number, and let r, s, y , and z be rational numbers with $y \neq 0$ and $z \neq 0$. If $y(x + r) = z(x + s)$, where the rational numbers are viewed as real numbers, then $y = z$ and $r = s$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem rational_affine_action_free
(x : ℝ)
(hx : Irrational x)
(r s y z : ℚ)
(hz : z ≠ 0)
(h : y * (x + r) = z * (x + s)) :
y = z ∧ r = s := by
  have hyz : y = z := by
    by_contra hne
    have hden : y - z ≠ 0 := sub_ne_zero.mpr hne
    have hdenR : (y : ℝ) - (z : ℝ) ≠ 0 := by
      exact_mod_cast hden
    have hlin : ((y : ℝ) - (z : ℝ)) * x =
      (z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ) := by
      linear_combination h
    have hxratt : x =
      ((z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ)) /
      ((y : ℝ) - (z : ℝ)) := by
      apply (eq_div_iff hdenR).2
      nlinarith [hlin]
    have hratt : x = (((z * s - y * r) / (y - z) : ℚ) : ℝ) := by
      calc
        x = ((z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ)) /
          ((y : ℝ) - (z : ℝ)) := hxratt
        _ = (((z * s - y * r) / (y - z) : ℚ) : ℝ) := by
          norm_num [Rat.cast_div]
    apply hx
    exact ⟨(z * s - y * r) / (y - z), hratt.symm⟩
  have hsum : (z : ℝ) * ((r : ℝ) - (s : ℝ)) = 0 := by
    rw [hyz] at h
    nlinarith [h]
  have hzR : (z : ℝ) ≠ 0 := by
    exact_mod_cast hz
  have hrsR : (r : ℝ) - (s : ℝ) = 0 :=
    (mul_eq_zero.mpr hsum).resolve_left hzR
  have hrs : r = s := by
    exact_mod_cast sub_eq_zero.mpr hrsR
  exact ⟨hyz, hrs⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.me.me__4b96916a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c9cb24bccdc1d4b3

4.116 116. irrational_rat_translate_mul_iff

The problem

For a real number x , a rational number r , and a nonzero rational number y , prove that $y(x + r)$ is irrational if and only if x is irrational.

Lean statement

```
theorem irrational_rat_translate_mul_iff
(x : ℝ)
(r y : ℚ)
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x
```

Parent. If r is a nonzero rational number and x is real, prove that x is irrational if and only if rx is irrational.

Parent in Lean

```
theorem irrational_mul_rat_iff
(x : ℝ)
(y : ℚ)
(h : y ≠ 0) : Irrational (x * y) ↔ Irrational x
```

What it contributes. Uses the parent's nonzero rational scaling checkpoint via `mul_ratCast` and explicitly consumes its inverse in the reverse-recovery direction; rational translation is inserted before scaling.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a symbolic rational translation, requiring irrationality invariance under rational addition and cancellation by the rational multiplier; the parent's scaling equivalence alone does not supply this argument. The proof is not a wrapper, decoration, or recall of the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new argument: irrationality must first be preserved under rational translation and then under nonzero rational scaling. The parent's scaling equivalence alone cannot prove the translated case.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves invariance of irrationality under both rational translation and nonzero rational scaling, requiring additive closure and an inverse-scaling argument absent from the parent. The parent's multiplicative equivalence alone does not substantially help prove the translated expression.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2753 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_rat_translate_mul_iff.extracted_1_1 (x : ℝ) (r y : ℚ) (hy : y ≠ 0) :  
  Irrational (↑y * (x + ↑r)) ↔ Irrational x
```

Read back by a model that saw only the Lean. For every real number x and rational numbers r and y with $y \neq 0$, the real number $y(x+r)$ is irrational if and only if x is irrational.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_rat_translate_mul_iff  
(x : ℝ)  
(r y : ℚ)  
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x := by  
  constructor  
  · intro hxy  
    have hscaled : Irrational ((x + r) * y) := by  
      simpa [mul_comm] using hxy  
    have hinv : Irrational (((x + r) * y) * y⁻¹) := by  
      exact hscaled.mul_ratCast (inv_ne_zero hy)  
    have htranslated := hinv.add_ratCast (q := -r)  
    simpa [mul_assoc, hy] using htranslated  
  · intro hx  
    have htranslated : Irrational (x + r) := by  
      exact hx.add_ratCast (q := r)  
    have hscaled : Irrational ((x + r) * y) := by  
      exact htranslated.mul_ratCast hy  
    simpa [mul_comm] using hscaled
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.me__1ceef025

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1ceef02531900e66

4.117 117. irrational_affine_quotient_rat_iff

The problem

Let x be real, let a and c be nonzero rational numbers, and let b be rational. Prove that $((x/a)+b)c$ is irrational if and only if x is irrational.

Lean statement

```
theorem irrational_affine_quotient_rat_iff
  (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :
  Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) ↔ Irrational x
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

What it contributes. Supplies the quotient irrationality normalization, used through `irrational_div_ratCast_iff` for the parameter `a`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the transformation by adding an arbitrary rational translation and an independent nonzero rational scale. Its proof requires new irrationality-preservation steps for rational addition and multiplication, so recalling the parent's division equivalence alone does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new irrationality-invariance arguments for adding and multiplying by independent rational parameters, beyond the parent's quotient equivalence. Both nonzero hypotheses are materially used, so recalling the parent proof does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds an independent rational translation and scaling, requiring irrationality transport through addition and multiplication by nonzero rational casts. The parent's division-only argument does not supply these checkpoints, and the new parameters are substantively used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2785 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_affine_quotient_rat_iff.extracted_1_1 (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :  
  Irrational ((x / ↑a + ↑b) * ↑c) ↔ Irrational x
```

Read back by a model that saw only the Lean. For every real number x and rational numbers a , b , and c , if $a \neq 0$ and $c \neq 0$, then $(x/a + b)c$ is irrational if and only if x is irrational, where the rational numbers are regarded as real numbers.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_affine_quotient_rat_iff  
  (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :  
    Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) ↔ Irrational x := by  
  constructor  
  · intro h  
    have htranslated : Irrational ((x / (a : ℝ)) + (b : ℝ)) := by  
      exact h.of_mul_ratCast  
    have hquotient : Irrational (x / (a : ℝ)) := by  
      exact htranslated.of_add_ratCast  
    have hnormalized : a ≠ 0 ∧ Irrational x := by  
      exact irrational_div_ratCast_iff.mp hquotient  
      exact hnormalized.2  
  · intro hx  
    have hquotient : Irrational (x / (a : ℝ)) := by  
      exact irrational_div_ratCast_iff.mpr ⟨ha, hx⟩  
    have htranslated : Irrational ((x / (a : ℝ)) + (b : ℝ)) := by  
      exact hquotient.add_ratCast b  
    have hscaled : Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) := by  
      exact htranslated.mul_ratCast hc  
    exact hscaled
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.me__97127643

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 97127643b9327c7c

4.118 118. rational_division_representation

The problem

Let x be real and y a nonzero rational number. Prove that x/y is rational exactly when x is a rational multiple of y .

Lean statement

```
theorem rational_division_representation
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  (∃ q : ℚ, x / (y : ℝ) = (q : ℝ)) ↔
  (∃ q : ℚ, x = (q : ℝ) * (y : ℝ))
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from preservation of irrationality to an existential characterization of rational quotients. Its proof requires explicit field cancellation and constructing witnesses, none of which is supplied by the parent's irrationality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from preservation of irrationality to an existential representation equivalence. Its proof requires clearing a nonzero rational denominator and constructing witnesses in both directions, which the parent's irrationality lemma does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from irrationality preservation to an existential characterization of rational divisibility. Its proof requires introducing a rational witness and using nonzero-field cancellation, which is not supplied by recalling the parent's irrationality equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2787 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_division_representation.extracted_1_1 (x : ℝ) (y : ℚ) (h : y ≠ 0) :  
  (∃ q, x / ↑y = ↑q) ↔ ∃ q, x = ↑q * ↑y
```

Read back by a model that saw only the Lean. For every real number x and rational number y with $y \neq 0$, the quotient x divided by y is equal to some rational number if and only if there exists a rational number q such that $x = q \cdot y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem rational_division_representation  
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :  
    (∃ q : ℚ, x / (y : ℝ) = (q : ℝ)) ↔  
    (∃ q : ℚ, x = (q : ℝ) * (y : ℝ)) := by  
  have hy : (y : ℝ) ≠ 0 := by  
    intro hy0  
    apply h  
    exact_mod_cast hy0  
  constructor  
  · rintro ⟨q, hq⟩  
    refine ⟨q, ?_⟩  
    have hmul : x = (x / (y : ℝ)) * (y : ℝ) := by  
      field_simp [hy]  
    calc  
      x = (x / (y : ℝ)) * (y : ℝ) := hmul  
      _ = (q : ℝ) * (y : ℝ) := by rw [hq]  
  · rintro ⟨q, hq⟩  
    refine ⟨q, ?_⟩  
    calc  
      x / (y : ℝ) = ((q : ℝ) * (y : ℝ)) / (y : ℝ) := by rw [hq]  
      _ = (q : ℝ) := by field_simp [hy]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.me__c46ab9d2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c46ab9d2198a9861

4.119 119. supplied_basis_trace_basis

The problem

Let K be a compact metric space, A a closed subset of K , and B a countable topological basis for K . Show that the traces of the sets in B on A form a countable topological basis for A .

Lean statement

```
theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Uses the parent's closed-subspace trace construction, specialized to a fixed supplied ambient basis rather than an existentially chosen family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine induced-topology result for an arbitrary countable ambient basis, using the basis-induction lemma and image countability; the parent's existential construction and closedness argument do not supply this proof. All explicit child hypotheses are used, and the absence of a closedness hypothesis makes the result strictly more general.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential construction into a general theorem about any supplied countable ambient basis, and its proof relies on the induced-topology basis lemma rather than recalling the parent's closed-subspace construction. Compactness and closedness are no longer needed, so this is not decoration or a forced wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the ambient basis: it proves that any supplied countable topological basis traces to a countable basis on the subtype. The parent only constructs one suitable basis existentially, so recalling its proof does not establish this arbitrary-basis result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2349 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem supplied_basis_trace_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K] [inst_1 : CompactSpace K]
  {A : Set K} (hA : IsClosed A) (B : Set (Set K)) (hBcount : B.Countable) (hBbasis : IsTopologicalBasis B) :
  ((fun U => Subtype.val ⁻¹' U) '' B).Countable ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, let $A \subseteq K$ be closed, and let B be a countable topological basis for K . Then the family of traces $\{U \cap A \mid U \in B\}$, regarded as subsets of the subspace A , is countable and is a topological basis for A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  have hcount : Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) :=
    hBcount.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U)
  have hbasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) :=
    hBbasis.induced (Subtype.val : A → K)
  exact ⟨hcount, hbasis⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.me__86310cc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 291534e956a8f28a

4.120 120. compact_closed_refined_trace_family

The problem

Let K be a compact second-countable metric space and let A be a closed subset of K . Show that there is a countable family of open subsets of K whose traces form a basis for A , separate distinct points of A , and admit local refinements whose closures remain inside the original member.

Lean statement

```
theorem compact_closed_refined_trace_family {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) ∧
    (∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U)
```

Parent. Let K be a second-countable metric space and $A \subseteq K$. Show that some countable family of open subsets of K has traces forming a basis for A and separates every two distinct points of A .

Parent in Lean

```
theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Supplied the countable ambient basis, induced trace-basis checkpoint, and point-separation construction; the child adds a consumed closure-refinement obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine closure-refinement property, proved using metric-ball shrinking and closedness of A ; this mathematics is absent from the parent's basis and separation proof. The added compactness assumption is not needed for the refinement itself, but the new conclusion is substantive and cannot be obtained by recalling the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added relative-closure refinement is a substantive compactness/metric shrinking claim, requiring closedness and a new closure-containment argument beyond the parent's basis and separation proof. Recalling the parent cannot establish this extra conjunct.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new refinement theorem: each basis neighborhood must admit a basis neighborhood whose relative closure in the closed compact subspace remains inside the original neighborhood. Proving this requires metric-ball and closure containment reasoning absent from the parent, so recalling the parent alone does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2790 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_closed_refined_trace_family.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : SecondCountableTopology K] [inst_2 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    (∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U) ∧
    ∀ U ∈ B, ∀ x ∈ A, x ∈ U → ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U
```

Read back by a model that saw only the Lean. *Let K be a compact metric space with a second-countable topology, and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that: every member of B is open in K ; the collection of their preimages under the inclusion map $A \rightarrow K$ is a topological basis for A with its subspace topology; for any two distinct points x and y of A , some $U \in B$ contains x and does not contain y ; and for every $U \in B$ and every $x \in A$ with $x \in U$, there exists $V \in B$ such that $x \in V$ and the closure in K of $A \cap V$ is contained in $A \cap U$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_closed_refined_trace_family {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) ∧
    (∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U) := by
  classical
  obtain ⟨B, hBcount, hBne, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hBopen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact hBbasis.isOpen hU
  have hTraceBasis :
```

```

TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
exact hBbasis.induced (Subtype.val : A → K)
have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
  x ∈ (Subtype.val : A → K) ⁻¹' U ∧
  y ∉ (Subtype.val : A → K) ⁻¹' U := by
intro x y hxy
have hpos : 0 < dist (x : K) (y : K) := dist_pos.mpr (by
  intro h
  apply hxy
  exact Subtype.ext h)
have hxball : (x : K) ∈ Metric.ball (x : K) (dist (x : K) (y : K)) := by
  rw [Metric.mem_ball]
  simpa using hpos
have hyball : (y : K) ∉ Metric.ball (x : K) (dist (x : K) (y : K)) := by
  intro hy
  have hlt : dist (x : K) (y : K) < dist (x : K) (y : K) := by
    simpa [Metric.mem_ball, dist_comm] using hy
  exact (lt_irrefl _ hlt)
obtain ⟨U, hU, hxU, hUsub⟩ :=
  hBbasis.exists_subset_of_mem_open hxball Metric.isOpen_ball
refine ⟨U, hU, hxU, ?_⟩
intro hyU
exact hyball (hUsub hyU)
have hAcompact : IsCompact A := hA.isCompact
have hAclosed : IsClosed A := hAcompact.isClosed
have hRefine : ∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
  ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U := by
  intro U hU x hxA hxU
  rcases (Metric.isOpen_iff.mp (hBopen U hU) x hxU) with ⟨ε, hε, hball⟩
  have hxsmall : x ∈ Metric.ball x (ε / 2) := by
    rw [Metric.mem_ball]
    simp [hε]
  obtain ⟨V, hV, hxV, hVsub⟩ :=
    hBbasis.exists_subset_of_mem_open hxsmall Metric.isOpen_ball
  refine ⟨V, hV, hxV, ?_⟩
  have hclosed : IsClosed (A ∩ Metric.closedBall x (ε / 2)) :=
    hAclosed.inter Metric.isClosed_closedBall
  have hsub : A ∩ V ⊆ A ∩ Metric.closedBall x (ε / 2) := by
    intro z hz
    refine ⟨hz.1, ?_⟩
    rw [Metric.mem_closedBall]
    exact le_of_lt (Metric.mem_ball.mp (hVsub hz.2))
  have hcl : closure (A ∩ V) ⊆ A ∩ Metric.closedBall x (ε / 2) :=
    closure_minimal hsub hclosed
  intro z hz
  have hz' := hcl hz
  refine ⟨hz'.1, hball ?_⟩
  rw [Metric.mem_ball]
  calc
    dist z x ≤ ε / 2 := Metric.mem_closedBall.mp hz'.2
    _ < ε := by linarith
exact ⟨B, hBcount, hBopen, hTraceBasis, hSeparate, hRefine⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me.me__7c9f9ef2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7c9f9ef2278dc87a

4.121 121. second_countable_subspace_trace_basis

The problem

Let K be a second-countable metric space and $A \subseteq K$. Show that some countable family of open subsets of K has traces forming a basis for A and separates every two distinct points of A .

Lean statement

```
theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

Parent. Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Parent in Lean

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Retained the parent's countable ambient-open family, trace-basis construction, and metric-ball separation checkpoint, while replacing compact closed subspaces with arbitrary subspaces of second-countable metric spaces.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from compact closed subspaces to arbitrary subspaces of a second-countable metric space, and its separation component requires a new positive-distance ball and basis-refinement argument. The parent's compactness-based countable basis construction does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from compact closed subspaces to arbitrary subspaces of a second-countable metric space. Its proof uses a new ambient countable-basis construction and induced-topology argument, while separately deriving metric separation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces compactness and closedness with ambient second countability, then derives a countable trace basis by inducing an ambient basis and proves separation via metric balls and basis refinement. This reasoning does not rely on recalling the parent's compact-subspace basis construction.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem second_countable_subspace_trace_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : SecondCountableTopology K] {A : Set K} :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U
```

Read back by a model that saw only the Lean. *Let K be a metrizable, second-countable topological space, and let A be a subset of K . Then there exists a countable collection B of open subsets of K such that the collection of their preimages under the subtype inclusion map $A \rightarrow K$ forms a topological basis for A with its subspace topology, and such that for any two distinct points x and y of A , there is some $U \in B$ with $x \in U$ (as a point of the subspace A) and $y \notin U$ (as a point of the subspace A).*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
[SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) := by
  classical
  obtain ⟨B, hBcount, hBne, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hBopen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact hBbasis.isOpen hU
  have hTraceBasis :
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
    exact hBbasis.induced (Subtype.val : A → K)
  have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
    x ∈ (Subtype.val : A → K) ⁻¹' U ∧
    y ∉ (Subtype.val : A → K) ⁻¹' U := by
    intro x y hxy
    have hpos : 0 < dist (x : K) (y : K) := dist_pos.mpr (by
      intro h
      apply hxy
      exact Subtype.ext h)
    have hxball : (x : K) ∈ Metric.ball (x : K) (dist (x : K) (y : K)) := by
      rw [Metric.mem_ball]
    simp using hpos
    have hyball : (y : K) ∉ Metric.ball (x : K) (dist (x : K) (y : K)) := by
      intro hy
```

```

have hlt : dist (x : K) (y : K) < dist (x : K) (y : K) := by
  simpa [Metric.mem_ball, dist_comm] using hy
exact (lt_irrefl _ hlt)
obtain ⟨U, hU, hxU, hUsub⟩ :=
  hBbasis.exists_subset_of_mem_open hxball Metric.isOpen_ball
refine ⟨U, hU, hxU, ?_⟩
intro hyU
exact hyball (hUsub hyU)
exact ⟨B, hBcount, hBopen, hTraceBasis, hSeparate⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me__129dda3a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 129dda3a21fafe47

4.122 122. closed_subspace_countable_trace_basis_union

The problem

Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A , and that every open subset of A is the union of the traces contained in it.

Lean statement

```
theorem closed_subspace_countable_trace_basis_union {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ V : Set A, IsOpen V →
      V = ⋃₀ {T : Set A |
        T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V})
```

Parent. Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Parent in Lean

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Supplied the compact-subtype countable basis construction, ambient open representatives, and trace-basis identification; the child adds the explicit open-set union representation as the new final obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's point-separation conclusion with an explicit union representation for every open subset of the subtype. Proving this requires the basis characterization and a new set-theoretic argument, so recalling the parent's final separation proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive basis characterization: every open subset of the closed subtype is explicitly represented as the union of contained traces. Proving this final obligation requires new membership and subset reasoning beyond the parent's separation argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's point-separation conclusion with an explicit representation of every open subset as a union of contained trace basis elements. Its final obligation uses the basis characterization and requires new membership and union reasoning, so recalling the parent's separation argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2767 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_trace_basis_union.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (V : Set ↑A), IsOpen V → V = ⋃₀ {T | T ∈ (fun U => Subtype.val ⁻¹' U) '' B ∧ T ⊆ V}
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that every U in B is open in K , the collection of traces $\{U \cap A : U \in B\}$ is a topological basis for the subspace A , and every open subset V of A is the union of all basis elements T from this collection that are contained in V .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_trace_basis_union {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ V : Set A, IsOpen V →
      V = ⋃₀ {T : Set A |
        T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V}) := by
  classical
  letI : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  obtain ⟨C, hCcount, _hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧
    (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  have hBasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
```

```

    rw [htrace]
    exact hCbasis
have hUnion : ∀ V : Set A, IsOpen V →
  V = ⋃ {T : Set A |
    T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V} := by
  intro V hV
  ext x
  constructor
  · intro hx
    obtain ⟨c, hc, hxc, hsub⟩ := hCbasis.exists_subset_of_mem_open hx hV
    have hcD : c ∈ {T : Set A |
      T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V} := by
      refine ⟨?_, hsub⟩
      rw [htrace]
      exact hc
    exact Set.mem_sUnion.mpr ⟨c, hcD, hxc⟩
  · intro hx
    obtain ⟨T, hTD, hxT⟩ := Set.mem_sUnion.mp hx
    exact hTD.2 hxT
exact ⟨B, hBcount, hBopen, hBasis, hUnion⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me__409c12da

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 409c12da748b86fc

4.123 123. compact_core_countable_trace_basis

The problem

In a compact metric space, every neighborhood of a point contains a compact neighborhood whose relative neighborhoods at that point are refined by traces of a countable ambient basis.

Lean statement

```
theorem compact_core_countable_trace_basis {X : Type*} [MetricSpace X] [CompactSpace X]
  (x : X) (U : Set X) (hU : U ∈ nhds x) :
  ∃ V : Set X, V ∈ nhds x ∧ IsCompact V ∧ V ⊆ U ∧
  ∃ B : Set (Set X), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B ∧
  ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2.25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplies the second-countability/countable-basis checkpoint via `inferInstance` and `TopologicalSpace.exists_countable_basis`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds genuinely new local compactness reasoning: it extracts a smaller closed-ball neighborhood inside U and proves its compactness, then establishes a relative trace refinement for every neighborhood. The parent's global countable-basis existence alone cannot supply these claims.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: constructing a compact closed-ball neighborhood inside U and proving its relative neighborhood trace property. The parent only supplies a countable topological basis, so recalling its proof does not establish the compact-core construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: it constructs a compact neighborhood contained in an arbitrary neighborhood and proves a relative trace-basis refinement property. The parent's global countable-basis existence supplies only the basis component and does not provide the metric compact-core argument or local containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2914 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_core_countable_trace_basis.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X]
[inst_1 : CompactSpace X] (x : X) (U : Set X) (hU : U ∈ ℳ x) :
  ∃ V ∈ ℳ x,
    IsCompact V ∧ V ⊆ U ∧ ∃ B, B.Countable ∧ IsTopologicalBasis B ∧ ∀ W ∈ ℳ x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V
```

Read back by a model that saw only the Lean. *Let X be a type equipped with a metric and a compact-space structure, let x be a point of X , and let U be a set containing a neighborhood of x . Then there exists a set V containing a neighborhood of x such that V is compact and V is contained in U , and there exists a countable topological basis B for X such that, for every neighborhood W of x , there is some b in B with $x \in b$ and $b \cap V \subseteq W \cap V$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_core_countable_trace_basis {X : Type*} [MetricSpace X] [CompactSpace X]
(x : X) (U : Set X) (hU : U ∈ nhds x) :
  ∃ V : Set X, V ∈ nhds x ∧ IsCompact V ∧ V ⊆ U ∧
  ∃ B : Set (Set X), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B ∧
  ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V := by
  classical
  rcases Metric.mem_nhds_iff.mp hU with ⟨r, hr, hball⟩
  let V : Set X := Metric.closedBall x (r / 2)
  have hVsub : V ⊆ U := by
    intro y hy
    apply hball
    change dist y x ≤ r / 2 at hy
    change dist y x < r
    linarith
  have hVnhds : V ∈ nhds x := by
    refine Filter.mem_of_superset (show Metric.ball x (r / 2) ∈ nhds x from Metric.ball_mem_nhds x (by linarith)) ?_
    intro y hy
    change dist y x < r / 2 at hy
    change dist y x ≤ r / 2
    exact le_of_lt hy
  have hVcompact : IsCompact V := by
    change IsCompact (Metric.closedBall x (r / 2))
    exact ProperSpace.isCompact_closedBall x (r / 2)
  have hSecond : SecondCountableTopology X := inferInstance
  obtain ⟨B, hBcount, -, hBbasis⟩ := TopologicalSpace.exists_countable_basis (α := X)
  have hTrace : ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V := by
    intro W hW
    rcases hBbasis.mem_nhds_iff.mp hW with ⟨b, hbB, hxb, hbW⟩
    exact ⟨b, hbB, hxb, fun y hy => ⟨hbW hy.1, hy.2⟩⟩
  exact ⟨V, hVnhds, hVcompact, hVsub, B, hBcount, hBbasis, hTrace⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__71368725

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-printer 71368725e7401bae

4.124 124. localized_compact_core_basis

The problem

Let X be a compact metric space, $x \in X$, and U an open neighborhood of x . Show that some compact neighborhood core $K \subseteq U$ has a countable basis whose members are traces of ambient subsets contained in K .

Lean statement

```
theorem localized_compact_core_basis {X : Type*} [MetricSpace X] [CompactSpace X] (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K : Set X, IsCompact K ∧ x ∈ interior K ∧ K ⊆ U ∧
    ∃ C : Set (Set K), Set.Countable C ∧ TopologicalSpace.IsTopologicalBasis C ∧
      ∃ r : Set K → Set X, ∀ s ∈ C,
        r s ⊆ K ∧ {y : K | (y : X) ∈ r s} = s
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplied the countable-basis checkpoint, instantiated on the compact-core subtype after consuming the compactness witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires constructing a compact neighborhood core inside the given open set and then transporting a countable basis from the compact subtype to ambient representatives. The parent only supplies a countable basis for the whole compact space and does not provide either the localized core or the subtype trace argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new compact-neighborhood construction inside the open set and an explicit subtype-to-ambient representative argument. The parent's countable-basis result alone does not supply either localization or the representation equality, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantially stronger localized compact-core theorem, including an induced-subtype basis and ambient representatives. The parent only supplies an abstract countable basis for the whole compact metric space and does not provide the compact refinement or localization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2908 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem localized_compact_core_basis.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X] [inst_1 : CompactSpace X]
  (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K,
    IsCompact K ∧
    x ∈ interior K ∧
    K ⊆ U ∧ ∃ C, C.Countable ∧ IsTopologicalBasis C ∧ ∃ r, ∀ s ∈ C, r s ⊆ K ∧ {y | ↑y ∈ r s} = s
```

Read back by a model that saw only the Lean. *Let X be a compact metric space, let x be a point of X , and let U be an open subset of X containing x . Then there exist a compact subset K of X and a countable collection C of subsets of the subspace K such that x lies in the interior of K , K is contained in U , C is a topological basis for K , and there exists an assignment r sending each $s \in C$ to a subset $r(s)$ of X with $r(s) \subseteq K$ and such that s is exactly the set of points y of K whose images in X belong to $r(s)$, i.e. $\{y \mid \uparrow y \in r(s)\} = s$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem localized_compact_core_basis {X : Type*} [MetricSpace X] [CompactSpace X] (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K : Set X, IsCompact K ∧ x ∈ interior K ∧ K ⊆ U ∧
    ∃ C : Set (Set K), Set.Countable C ∧ TopologicalSpace.IsTopologicalBasis C ∧
      ∃ r : Set K → Set X, ∀ s ∈ C,
        r s ⊆ K ∧ {y : K | (y : X) ∈ r s} = s := by
  classical
  obtain ⟨K, hK, hxK, hKU⟩ := exists_compact_subset hU hxU
  let I : CompactSpace K := isCompact_iff_compactSpace.mp hK
  obtain ⟨C, hC, hCempty, hBasis⟩ := ∃ C : Set (Set K), Set.Countable C ∧ ∅ ∉ C ∧ TopologicalSpace.IsTopologicalBasis C :=
    TopologicalSpace.exists_countable_basis K
  refine ⟨K, hK, hxK, hKU, C, hC, hBasis, ?_⟩
  let r : Set K → Set X := fun t => Set.image (fun y : K => (y : X)) t
  refine ⟨r, ?_⟩
  intro s hs
  refine ⟨?, ?_⟩
  · rintro z ⟨y, hy, rfl⟩
    exact y.property
  · ext y
    constructor
    · intro hyPre
      obtain ⟨z, hz, hzy⟩ := hyPre
      have heq : z = y := Subtype.ext hzy
      simpa [heq] using hz
    · intro hyMem
      exact ⟨y, hyMem, rfl⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__c765a259

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c765a2597f9807ad

4.125 125. closed_subspace_countable_trace_basis

The problem

Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Lean statement

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplies the compact-metric countable-basis checkpoint, transferred here to the closed subtype A and then lifted through ambient-open traces.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent to a closed subspace: it constructs ambient open representatives and proves their traces form a basis on the subtype. This requires induced-topology reasoning and lifting basis elements, so recalling the parent's ambient-basis proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the object from an ambient countable basis on K to a countable family of open subsets of K whose traces form a basis on the closed subspace A . Its proof requires compactness and second-countability of A plus representing induced-open sets by ambient opens, which is not supplied by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the ambient space by a closed subspace and requires constructing countably many ambient open representatives whose traces form a basis. This needs induced-topology reasoning and lifting basis elements, which the parent's direct second-countable-basis packaging does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2740 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst.1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B, B.Countable ∧ (∀ U ∈ B, IsOpen U) ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let $A \subseteq K$ be closed. Then there exists a countable collection B of open subsets of K such that the collection of traces $\{A \cap U \mid U \in B\}$, viewed as subsets of the subspace A , is a topological basis for A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  classical
  let I : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  have hSecond : SecondCountableTopology A := inferInstance
  obtain ⟨C, hCcount, hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧ (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  refine ⟨B, hBcount, hBopen, ?_⟩
  simpa [htrace] using hCbasis
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__ee014913

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ee01491334364c9b

4.126 126. uniformContinuousOn_bounded_chain

The problem

Let $(f_k)_{k \geq 0}$ be a sequence of self-maps of a uniform space and let S be a subset of that space. If each f_k is uniformly continuous on the image of S after the preceding k stages, prove that the composite of the first n stages is uniformly continuous on S .

Lean statement

```
theorem uniformContinuousOn_bounded_chain
  {α : Type*} [UniformSpace α]
  {s : Set α} (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ k, UniformContinuousOn (f k)
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) '' s)) :
  UniformContinuousOn
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s
```

Parent. Let f , g , and h be successive maps, and let S be a subset of the source space. If each map is uniformly continuous on the image relevant to S , prove that $h \circ g \circ f$ is uniformly continuous on S .

Parent in Lean

```
theorem uniformContinuousOn_three_stage_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {s : Set α} {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s))
  (hh : UniformContinuousOn h ((g ∘ f) '' s)) :
  UniformContinuousOn (h ∘ g ∘ f) s
```

What it contributes. Generalizes the parent's image-local composition step from three maps to an inductively indexed finite chain, retaining its MapsTo and UniformContinuousOn.comp checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed three-stage composition to an arbitrary finite chain and requires an induction proving the successive-image invariant. A solver recalling the parent would still need to construct and manage this recursive invariant.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed three-stage composition to an arbitrary finite chain and requires induction to maintain the successive-image invariant. A memorized proof of the parent does not supply this inductive argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed three-stage composition to an arbitrary finite chain and requires induction to establish the successive-image invariant. The parent's direct two-step composition proof does not supply this inductive reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2900 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_bounded_chain.extracted_1_1.{u_1} {α : Type u_1} [inst : UniformSpace α] {s : Set α}
  (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ (k : ℕ), UniformContinuousOn (f k) (Nat.rec (motive := fun x => α → α) id (fun j g => f j ∘ g) k ' ' s)) :
  UniformContinuousOn (Nat.rec (motive := fun x => α → α) id (fun j g => f j ∘ g) n) s
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a uniform space, let $s \subseteq \alpha$, let f be a sequence of functions $f_k : \alpha \rightarrow \alpha$ indexed by natural numbers, and let n be a natural number. Assume that for every natural number k , the function f_k is uniformly continuous on the image of s under the function obtained by recursively composing the functions $f_0, f_1, \dots, f_{k-1}$, starting from the identity function. Then the function obtained by recursively composing the first n functions in this way is uniformly continuous on s .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_bounded_chain
  {α : Type*} [UniformSpace α]
  {s : Set α} (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ k, UniformContinuousOn (f k)
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) ' ' s)) :
  UniformContinuousOn
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s := by
  induction n with
  | zero =>
    have hid : UniformContinuousOn (id : α → α) s :=
      uniformContinuous_id.uniformContinuousOn
    simpa using hid
  | succ n ih =>
    have hmap : Set.Mapsto
      (Nat.rec (id : α → α) (fun j g => f j ∘ g) n)
      s
      ((Nat.rec (id : α → α) (fun j g => f j ∘ g) n) ' ' s) := by
      intro x hx
      exact ⟨x, hx, rfl⟩
    have hstep : UniformContinuousOn
      (f n ∘ Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s := by
      exact (hcont n).comp ih hmap
    simpa [Function.comp_def] using hstep
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me.mh.me__47be696d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 47be696d25904cea

4.127 127. three_stage_uniformContinuousOn_comp

The problem

Let f , g , and h be three successive maps. If f is uniformly continuous, g is uniformly continuous on the image of f , and h is uniformly continuous on the image of $g \circ f$, prove that $h \circ g \circ f$ is uniformly continuous.

Lean statement

```
theorem three_stage_uniformContinuousOn_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f))
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (h ∘ g ∘ f)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Generalizes binary composition to a three-stage composition while replacing the later global hypotheses with uniform continuity on the successive images.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes global uniform continuity to uniform continuity on the successive ranges, requiring MapsTo obligations and UniformContinuousOn composition through the intermediate image. The parent's direct 'hg.comp hf' argument does not supply this restricted-domain reasoning, and all hypotheses are essential.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes the composition theorem: it derives global uniform continuity from local uniform continuity of the later maps on the successive ranges. This requires range mapping and UniformContinuousOn composition reasoning that the parent's direct global composition lemma does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes continuity to successive image sets and requires composing UniformContinuousOn hypotheses with explicit range-mapping arguments. The parent's global composition lemma does not supply this restricted-domain reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem three_stage_uniformContinuousOn_comp.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f)
(hg : UniformContinuousOn g (Set.range f)) (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (h ∘ g ∘ f)
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, and let $f: \alpha \rightarrow \beta$, $g: \beta \rightarrow \gamma$, and $h: \gamma \rightarrow \delta$. Suppose f is uniformly continuous, g is uniformly continuous on the range of f , and h is uniformly continuous on the range of $g \circ f$. Then the composition $h \circ g \circ f: \alpha \rightarrow \delta$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem three_stage_uniformContinuousOn_comp
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f)
(hg : UniformContinuousOn g (Set.range f))
(hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (h ∘ g ∘ f) := by
have hmap_f : Set.MapsTo f Set.univ (Set.range f) := by
  intro x hx
  exact ⟨x, rfl⟩
have hgf : UniformContinuousOn (g ∘ f) Set.univ := by
  exact hg.comp hf.uniformContinuousOn hmap_f
have hmap_gf : Set.MapsTo (g ∘ f) Set.univ (Set.range (g ∘ f)) := by
  intro x hx
  exact ⟨x, rfl⟩
have hfinal_on : UniformContinuousOn (h ∘ (g ∘ f)) Set.univ := by
  exact hh.comp hgf hmap_gf
apply uniformContinuousOn_univ.mp
simpa [Function.comp_def] using hfinal_on
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me__a36d04eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a36d04eb5dda70f9

4.128 128. uniformContinuous_paired_two_stage_comp

The problem

If f is uniformly continuous, g is uniformly continuous on a set t containing the range of f , and h is uniformly continuous on the range of $g \circ f$, prove that $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_paired_two_stage_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g t)
  (hft : Set.range f ⊆ t)
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (fun x => (g (f x), h (g (f x))))
```

Parent. If f is uniformly continuous, g is uniformly continuous on a set containing the range of f , and h is uniformly continuous on the range of $g \circ f$, then $h \circ (g \circ f)$ is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_two_stage_comp_of_on_ambient
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g t)
  (hft : Set.range f ⊆ t)
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (h ∘ (g ∘ f))
```

What it contributes. Supplied the ambient-range composition checkpoints for deriving $\text{UniformContinuous } (g \circ f)$ and $\text{UniformContinuous } (h \circ (g \circ f))$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target to a product-valued uniform-continuity claim. It must derive both intermediate maps and use them together via ‘prodMk’, which the parent’s proof does not establish.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a product-valued uniform-continuity conclusion: both $\text{UniformContinuous } (g \circ f)$ and $\text{UniformContinuous } (h \circ (g \circ f))$ are derived and then combined via prodMk. The parent’s final composition proof alone does not supply this product-map step.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely proves a product-valued uniform-continuity statement: it derives uniform continuity of $g \circ f$, independently derives $h \circ (g \circ f)$, and combines both via prodMk. The parent’s final composition proof does not supply the product-map step or make the first-stage map a separate required component.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2020 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_paired_two_stage_comp.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β} (hf : UniformContinuous f)
(hg : UniformContinuousOn g t) (hft : Set.range f ⊆ t) (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous fun x => (g (f x), h (g (f x)))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$. Suppose f is uniformly continuous, g is uniformly continuous on a set $t \subseteq \beta$, the range of f is contained in t , and h is uniformly continuous on the range of $g \circ f$. Then the map from α to $\gamma \times \delta$ defined by $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_paired_two_stage_comp
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
(hf : UniformContinuous f)
(hg : UniformContinuousOn g t)
(hft : Set.range f ⊆ t)
(hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (fun x => (g (f x), h (g (f x)))) := by
have hgf_on : UniformContinuousOn (g ∘ f) Set.univ := by
  refine hg.comp hf.uniformContinuousOn ?_
  intro x hx
  exact hft ⟨x, rfl⟩
have hgf : UniformContinuous (g ∘ f) := uniformContinuousOn_univ.mp hgf_on
have hcomp_on : UniformContinuousOn (h ∘ (g ∘ f)) Set.univ := by
  refine hh.comp hgf.uniformContinuousOn ?_
  intro x hx
  exact ⟨x, rfl⟩
have hcomp : UniformContinuous (h ∘ (g ∘ f)) := uniformContinuousOn_univ.mp hcomp_on
have hpair : UniformContinuous (fun x => ((g ∘ f) x, (h ∘ (g ∘ f)) x)) := by
  exact hgf.prodMk hcomp
simpa [Function.comp_def] using hpair
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me.me__50d0cc9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b14ec291665fff5e

4.129 129. iterate_cauchy

The problem

Let (u_i) be a Cauchy net and let f be a uniformly continuous self-map. Show that for every natural number n , the net obtained by applying f n times to each u_i is Cauchy.

Lean statement

```
theorem iterate_cauchy
  {α ι : Type*} [UniformSpace α] [Preorder ι]
  {f : α → α} {u : ι → α}
  (hf : UniformContinuous f) (hu : CauchySeq u) :
  ∀ n : ℕ, CauchySeq ((f^[n]) ∘ u)
```

Parent. Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Parent in Lean

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

What it contributes. Reuses the parent's uniform-continuity composition and Cauchy-sequence transport atoms, while adding induction over an arbitrary iterate depth.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the composition depth: it must induct on n , prove uniform continuity of each iterate, and transport the Cauchy property at every stage. The parent's one-step composition proof does not by itself supply this induction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the iterate depth and proves the result by induction, establishing uniform continuity and Cauchy preservation at every iterate. This requires reasoning beyond the parent's one-step composition argument and is not recoverable by simply recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the iterate depth and requires induction to prove uniform continuity and Cauchy preservation for every iterate. The parent's two-map composition proof does not supply this induction or the iterate-recurrence reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2203 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem iterate_cauchy.extracted_1_1 : ∀ {α : Type u_1} {ι : Type u_2} [inst : UniformSpace α] [inst_1 : Preorder ι]
  {f : α → α} {u : ι → α}, UniformContinuous f → CauchySeq u → ∀ (n : ℕ), CauchySeq (f^[n] ∘ u)
```

Read back by a model that saw only the Lean. *For any type α equipped with a uniform space structure, any type ι equipped with a preorder, any function $f : \alpha \rightarrow \alpha$, and any function $u : \iota \rightarrow \alpha$, if f is uniformly continuous and u is a Cauchy sequence, then for every natural number n , the sequence obtained by applying the n -fold iterate of f to each term of u is also a Cauchy sequence.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem iterate_cauchy
  {α ι : Type*} [UniformSpace α] [Preorder ι]
  {f : α → α} {u : ι → α}
  (hf : UniformContinuous f) (hu : CauchySeq u) :
  ∀ n : ℕ, CauchySeq ((f^[n]) ∘ u) := by
  have hmain : ∀ n : ℕ, UniformContinuous (f^[n]) ∧ CauchySeq ((f^[n]) ∘ u) := by
    intro n
    induction n with
    | zero =>
      constructor
      · simpa using (uniformContinuous_id : UniformContinuous (id : α → α))
      · simpa using hu
    | succ n ih =>
      have hfu : CauchySeq (f ∘ u) := hf.comp_cauchySeq hu
      have hc : CauchySeq ((f^[n]) ∘ (f ∘ u)) :=
        ih.1.comp_cauchySeq hfu
      constructor
      · exact ih.1.comp hf
      · simpa [Function.comp_def, Function.iterate_succ_apply] using hc
  intro n
  exact (hmain n).2
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me__3a1560d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint 3a1560d5a16ba49d

4.130 130. uniformContinuousOn_comp_image

The problem

If f is uniformly continuous on s and g is uniformly continuous on the image of s under f , then $g \circ f$ is uniformly continuous on s .

Lean statement

```
theorem uniformContinuousOn_comp_image
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s)) :
  UniformContinuousOn (g ∘ f) s
```

Parent. If f is uniformly continuous and g is uniformly continuous on the range of f , then $g \circ f$ is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f)
```

What it contributes. Supplied the composition-on-range checkpoint generalized from `Set.range f` to the image $f '' s$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the composition result from global uniform continuity and `Set.range f` to uniform continuity on an arbitrary domain s and the image $f '' s$. Its proof uses the restricted composition theorem and explicitly supplies the image witness $\langle x, hx, rfl \rangle$.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely localizes the composition theorem: it composes uniform continuity on s with continuity of g on the image $f '' s$, and constructs the required image witness $\langle x, hx, rfl \rangle$. The parent's global-domain proof does not supply this restricted-domain reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely bounded composition law on an arbitrary set s , using `UniformContinuousOn` composition and the nontrivial image witness $\langle x, hx, rfl \rangle$. The parent only handles global uniform continuity with g restricted to `Set.range f`, so recalling it does not directly solve this local-image theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2021 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_comp_image.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s) (hg : UniformContinuousOn g (f '' s)) : UniformContinuousOn (g ∘ f) s
```

Read back by a model that saw only the Lean. *Let α , β , and γ be uniform spaces, let s be a subset of α , let $f : \alpha \rightarrow \beta$, and let $g : \beta \rightarrow \gamma$. If f is uniformly continuous on s and g is uniformly continuous on the image $f(s) = \{f(x) \mid x \in s\}$, then the composition $g \circ f : \alpha \rightarrow \gamma$ is uniformly continuous on s .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_comp_image
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s)) :
  UniformContinuousOn (g ∘ f) s := by
  apply hg.comp hf
  intro x hx
  exact ⟨x, hx, rfl⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me__c686e5f7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 74443254879dfcd9

4.131 131. agreement_iff_three_stage_agreement

The problem

Let three uniformly continuous maps be composed, and let the first map be replaced by another map on a set. If the latter two maps are injective, then the two first maps agree on the set exactly when their three-stage outputs agree there.

Lean statement

```
theorem agreement_iff_three_stage_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}

  (hginj : Function.Injective g) (hhinj : Function.Injective h)
  (s : Set α) (m : α → β) :
  (∀ x, x ∈ s → m x = f x) ↔
  (∀ x, x ∈ s → h (g (m x)) = h (g (f x)))
```

Parent. Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Parent in Lean

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to an agreement characterization and requires a genuinely new injectivity-cancellation argument through h and g. Its proof does not recall or reuse the parent's uniform-continuity proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the continuity conclusion into an iff characterization and requires genuinely new cancellation reasoning through injective h and g. Its forward direction is congruence, while the reverse direction cannot be obtained by recalling the parent's continuity proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to an iff characterization and adds genuinely new injectivity-cancellation reasoning: equality after h and g is cancelled to recover $m x = f x$. This is not supplied by the parent's continuity proof and is not recall or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2183 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem agreement_iff_three_stage_agreement.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) (hginj : Function.Injective g) (hhinj : Function.Injective h) (s : Set α) (m : α → β) :
(∀ x ∈ s, m x = f x) ↔ ∀ x ∈ s, h (g (m x)) = h (g (f x))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous, with g and h injective. For any subset s of α and any function $m : \alpha \rightarrow \beta$, the following are equivalent: for every $x \in s$, $m(x) = f(x)$; and for every $x \in s$, $h(g(m(x))) = h(g(f(x)))$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem agreement_iff_three_stage_agreement
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}

(hginj : Function.Injective g) (hhinj : Function.Injective h)
(s : Set α) (m : α → β) :
(∀ x, x ∈ s → m x = f x) ↔
(∀ x, x ∈ s → h (g (m x)) = h (g (f x))) := by
  constructor
  · intro hm x hx
    rw [hm x hx]
  · intro houtputs x hx
    have hmiddle : g (m x) = g (f x) := hhinj (houtputs x hx)
    exact hginj hmmiddle
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me.me__f7815124

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint ec1531061a228b3a

4.132 132. uniformContinuous_adjacent_stage_pair

The problem

If f , g , and h are uniformly continuous maps between uniform spaces, then the map $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_adjacent_stage_pair
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => (g (f x), h (g (f x))))
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

What it contributes. Uses the parent's composition checkpoint pattern, extending it so the shared composite serves both product coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from uniform continuity of the final composite to uniform continuity of a paired function with intermediate and final outputs. Its proof requires the product-preservation lemma 'prodMk', so recalling the parent's composition proof alone does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composition to a product-valued map and requires proving uniform continuity via 'prodMk', which the parent's composition proof does not supply. It is not a forced wrapper, decoration, or repeat of the retained fold-composition theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a single final-stage output to continuity of a paired adjacent-stage map. It requires the new product-continuity argument via 'prodMk', which the parent proof does not supply, so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2153 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator goal_roundtrip	lean model	statement, axioms and kernel acceptance independently replayed MISMATCH: Statement B gives the codomain as $\beta \times \delta$, but the first component $g(f(x))$ has type γ , so the natural codomain is $\gamma \times \delta$.

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_adjacent_stage_pair.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) : UniformContinuous fun x => (g (f x), h (g (f x)))
```

Read back by a model that saw only the Lean. *Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Then the function from α to $\beta \times \delta$ defined by $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_adjacent_stage_pair
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous (fun x => (g (f x), h (g (f x)))) := by
have hfg : UniformContinuous (g ∘ f) := hg.comp hf
have houter : UniformContinuous (h ∘ (g ∘ f)) := hh.comp hfg
have hpair : UniformContinuous (fun x => ((g ∘ f) x, (h ∘ (g ∘ f)) x)) :=
hfg.prodMk houter
simpa [Function.comp_def] using hpair
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me__1381c1ba

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1381c1bab0e73398

4.133 133. uniformContinuousOn_three_stage_local_agreement

The problem

Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Lean statement

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

What it contributes. Supplied the three-stage uniform-continuity composition checkpoints; the mutation adds local agreement and UniformContinuousOn.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the claim to uniform continuity on a subset after replacing the first map by an arbitrary map that only agrees there. Its proof must use the agreement hypothesis to establish local continuity of the replacement before composing, which the parent's global composition proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new local-agreement argument: it proves uniform continuity of m only on s via congruence, then composes locally. The parent's global composition proof cannot be recalled to establish this specialized conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new local reasoning: it transfers uniform continuity from f to an arbitrary map m using agreement on s , then composes via UniformContinuousOn. The parent's global composition proof does not supply this congruence or restricted-domain argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2167 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_three_stage_local_agreement.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) (s : Set α) (m : α → β) (hm : ∀ x ∈ s, m x = f x) :
UniformContinuousOn (fun x => h (g (m x))) s
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions, let s be a subset of α , and let $m : \alpha \rightarrow \beta$ satisfy $m(x) = f(x)$ for every $x \in s$. Then the function $x \mapsto h(g(m(x)))$ is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_three_stage_local_agreement
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
(s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
UniformContinuousOn (fun x => h (g (m x))) s := by
have hmOn : UniformContinuousOn m s :=
  hf.uniformContinuousOn.congr (fun x hx => (hm x hx).symm)
have hgmOn : UniformContinuousOn (g ∘ m) s :=
  (show UniformContinuousOn g (Set.range m) from hg.uniformContinuousOn).comp hmOn (by
    intro x hx
    exact ⟨x, rfl⟩)
have hfinalOn : UniformContinuousOn (h ∘ (g ∘ m)) s :=
  (show UniformContinuousOn h (Set.range (g ∘ m)) from hh.uniformContinuousOn).comp hgmOn (by
    intro x hx
    exact ⟨x, rfl⟩)
have hrewrite : (fun x => h (g (m x))) = h ∘ (g ∘ m) := by
  funext x
  rfl
rw [hrewrite]
exact hfinalOn
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me__7c32138e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7c32138eabe4522b

4.134 134. uniformContinuous_three_stage

The problem

If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Lean statement

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

Parent. A uniformly continuous map followed by another uniformly continuous map is uniformly continuous, even when the intermediate map is named separately and agrees pointwise with the first.

Parent in Lean

```
theorem uniformContinuous_named_intermediate
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g)
  (m : α → β) (hm : ∀ x, m x = f x) :
  UniformContinuous (fun x => g (m x))
```

What it contributes. Supplied the uniform-continuity composition principle used to construct the intermediate composite and final composition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from two composable uniformly continuous maps to three, requiring a new intermediate composite and a second composition argument. It is not a wrapper, decoration, or recall of the parent's exact proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the two-map composition law to three composable uniformly continuous maps, requiring a new nested composition step involving the final map. It has no forced witness, decoration, or unused hypothesis, and recalling the parent's proof does not directly solve the new theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law from two maps to three: it must first establish uniform continuity of $g \circ f$ and then compose with h . A solver recalling the parent still needs this additional composition step and the new associativity shape.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2146 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_three_stage.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
{δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous fun x => h (g (f x))
```

Read back by a model that saw only the Lean. *Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Then the function from α to δ defined by $x \mapsto h(g(f(x)))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_three_stage
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous (fun x => h (g (f x))) := by
have hfg : UniformContinuous (g ∘ f) := hg.comp hf
have hfinal : UniformContinuous (h ∘ (g ∘ f)) := hh.comp hfg
have hrewrite : (fun x => h (g (f x))) = h ∘ (g ∘ f) := by
  funext x
  rfl
rw [hrewrite]
exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me__adb7544c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint adb7544cb1cbe0c9

4.135 135. continuous_preimage_finite_common_pointwise_order_closed

The problem

Let $h : \beta \rightarrow X$ be continuous, and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the inverse image under h of the set where every first function is at most its paired second function is closed in β .

Lean statement

```
theorem continuous_preimage_finite_common_pointwise_order_closed
  {β X ι : Type} [TopologicalSpace β] [TopologicalSpace X]
  (h : β → X) (hh : Continuous h)
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → X → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed (h ⁻¹' {x : X | ∀ i ∈ s, f i x ≤ g i x})
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

What it contributes. Supplies the finite-induction and intersection argument for common pointwise-order regions; the child adds composition with the continuous map h and proves closedness of the resulting preimage.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent to a continuous preimage: it must compose every inequality with h and prove closedness on β , then identify the resulting region with the preimage. The parent's induction alone does not supply the continuity-composition and preimage reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to continuous preimages over arbitrary topological spaces and requires composing every constraint with h before proving closedness. Although the finite-intersection induction is shared, the parent proof does not supply the continuity-composition and preimage reasoning needed here.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from metric spaces to arbitrary topological spaces and adds a continuous preimage. Its proof must compose each inequality with the map and establish closedness in the domain, which the parent's proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_preimage_finite_common_pointwise_order_closed.extracted_1_1 {β X ι : Type}
  [inst : TopologicalSpace β] [inst_1 : TopologicalSpace X] (h : β → X) (hh : Continuous h) (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → X → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed (h ⁻¹' {x | ∀ i ∈ s, f i x ≤ g i x})
```

Read back by a model that saw only the Lean. Let β , X , and ι be types equipped with topological spaces, let $h : \beta \rightarrow X$ be continuous, and let s be a nonempty finite set of indices from ι . Suppose f and g assign to each index $i \in \iota$ continuous real-valued functions on X . Then the preimage under h of the set of points $x \in X$ such that $f(i, x) \leq g(i, x)$ for every $i \in s$ is closed in β .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_preimage_finite_common_pointwise_order_closed
  {β X ι : Type} [TopologicalSpace β] [TopologicalSpace X]
  (h : β → X) (hh : Continuous h)
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → X → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed (h ⁻¹' {x : X | ∀ i ∈ s, f i x ≤ g i x}) := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp at hs
  | @insert i s hi ih =>
    have h_i : IsClosed {y : β | f i (h y) ≤ g i (h y)} := by
      exact isClosed_le ((hf i).comp hh) ((hg i).comp hh)
    have h_s : IsClosed (h ⁻¹' {x : X | ∀ j ∈ s, f j x ≤ g j x}) := by
      rcases s.eq_empty_or_nonempty with rfl | hs'
      · simp
      · exact ih hs'
    have h_inter : IsClosed ({y : β | f i (h y) ≤ g i (h y)} ∩
      (h ⁻¹' {x : X | ∀ j ∈ s, f j x ≤ g j x})) := h_i.inter h_s
    simp [Finset.mem_insert, and_assoc, and_left_comm, and_comm] using h_inter
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.me__85d2a477

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 85d2a4777bc98bb7

4.136 136. finite_closed_constraints_compact

The problem

Let K be a compact subset of a metric space, let s be finite, and let each f_i be continuous. If each C_i is closed, prove that the set of points $x \in K$ satisfying $f_i(x) \in C_i$ for every $i \in s$ is compact.

Lean statement

```
theorem finite_closed_constraints_compact
  {α β ι : Type} [MetricSpace α] [MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι)
  (f : ι → α → β) (h : ∀ i, Continuous (f i))
  (C : ι → Set β) (hC : ∀ i, IsClosed (C i)) :
  IsCompact (K ∩ {x : α | ∀ i ∈ s, f i x ∈ C i})
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. Reused the parent's closed-preimage and finite-intersection checkpoints, replacing singleton equality levels with arbitrary closed target sets and using the resulting closed locus to obtain compactness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from finite common level sets to arbitrary closed target constraints and adds compactness of their intersection with a compact carrier. Its proof requires a new compact-intersection argument beyond the parent's closedness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a common real level set to compactness of its intersection with an arbitrary compact carrier and finitely many closed target constraints. The proof requires a new compactness-intersection argument and is not recoverable by merely recalling the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a finite common level set to compactness of its intersection with an arbitrary compact carrier and arbitrary closed target constraints. Proving compactness requires the additional compact-intersection argument, so recalling the parent does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2331 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_closed_constraints_compact.extracted_1_1 {α β ι : Type} [inst : MetricSpace α] [inst_1 : MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι) (f : ι → α → β) (h : ∀ (i : ι), Continuous (f i)) (C : ι → Set β)
  (hC : ∀ (i : ι), IsClosed (C i)) : IsCompact (K ∩ {x | ∀ i ∈ s, f i x ∈ C i})
```

Read back by a model that saw only the Lean. *Let α and β be metric spaces, let K be a compact subset of α , let s be a finite set of indices ι , let f assign to each $i \in \iota$ a continuous function $f_i : \alpha \rightarrow \beta$, and let C_i be a closed subset of β for each $i \in \iota$. Then the set of all $x \in K$ such that $f_i(x) \in C_i$ for every $i \in s$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_closed_constraints_compact
  {α β ι : Type} [MetricSpace α] [MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι)
  (f : ι → α → β) (h : ∀ i, Continuous (f i))
  (C : ι → Set β) (hC : ∀ i, IsClosed (C i)) :
  IsCompact (K ∩ {x : α | ∀ i ∈ s, f i x ∈ C i}) := by
  have h_preimage : ∀ i, IsClosed (f i ⁻¹' C i) := by
    intro i
    exact IsClosed.preimage (h i) (hC i)
  have h_constraint : IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i} := by
    have h_inter : IsClosed (⋂ i ∈ s, f i ⁻¹' C i) := by
      exact isClosed_biInter (fun i hi => h_preimage i)
    have h_eq :
      {x : α | ∀ i ∈ s, f i x ∈ C i} =
      ⋂ i ∈ s, f i ⁻¹' C i := by
      ext x
      simp
      rw [h_eq]
      exact h_inter
    exact hK.inter_right h_constraint
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__7c585f69

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7c585f69234fbb03

4.137 137. finite_indexed_closed_target_locus_closed

The problem

Let X be a metric space, let s be a finite index set, let each $f_i : X \rightarrow \mathbb{R}$ be continuous, and let $C_i \subseteq \mathbb{R}$ be closed. Prove that $\{x \in X : f_i(x) \in C_i \text{ for every } i \in s\}$ is closed.

Lean statement

```
theorem finite_indexed_closed_target_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ)
  (hcont : ∀ i, Continuous (f i))
  (C : 1 → Set ℝ) (hclosed : ∀ i, IsClosed (C i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i}
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. Reuses the parent’s continuity, closed-preimage, finite-intersection, and set-extensionality checkpoints while replacing the common singleton target with an index-dependent closed set.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes singleton level sets to an indexed family of arbitrary closed target sets. Its proof requires recognizing and applying the general closed-preimage theorem, rather than recalling the parent’s singleton-preimage argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed common level to an indexed family of closed target sets, so each constraint requires its own closed-set preimage argument. The parent theorem cannot be recalled to prove arbitrary sets C_i .

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the common singleton target to an indexed family of arbitrary closed sets, requiring the preimage-closedness argument to depend on each target set. This is substantive bounded generalization, not a trivial wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2340 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_indexed_closed_target_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (hcont : ∀ (i : ι), Continuous (f i)) (C : ι → Set ℝ) (hclosed : ∀ (i : ι), IsClosed (C i)) :
  IsClosed {x | ∀ i ∈ s, f i x ∈ C i}
```

Read back by a model that saw only the Lean. Let α be a metric space, let ι be a type, and let s be a finite set of indices in ι . For each $i \in \iota$, let $f_i : \alpha \rightarrow \mathbb{R}$ be continuous, and let $C_i \subseteq \mathbb{R}$ be closed. Then the set of all $x \in \alpha$ such that $f_i(x) \in C_i$ for every $i \in s$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_indexed_closed_target_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ)
  (hcont : ∀ i, Continuous (f i))
  (C : ι → Set ℝ) (hclosed : ∀ i, IsClosed (C i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i} := by
  have h_pre : ∀ i, IsClosed (f i ⁻¹' C i) := by
    intro i
    exact IsClosed.preimage (hcont i) (hclosed i)
  have h_int : IsClosed (⋂ i ∈ s, f i ⁻¹' C i) := by
    exact isClosed_biInter (fun i hi => h_pre i)
  have h_eq :
    {x : α | ∀ i ∈ s, f i x ∈ C i} =
    ⋂ i ∈ s, f i ⁻¹' C i := by
    ext x
    simp
  rw [h_eq]
  exact h_int
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__b11ade90

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b11ade90a5b15d33

4.138 138. finite_common_pointwise_order_closed

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Lean statement

```
theorem finite_common_pointwise_order_closed
  {α : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Parent in Lean

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

What it contributes. Generalizes the parent from constant interval bounds for one continuous function to finite intersections of variable pairwise continuous order loci.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes a single continuous function's interval preimage to a finite intersection of pointwise inequality loci. Its induction over the finite family and use of 'isClosed_le' require reasoning absent from the parent's interval-preimage proof, so recalling the parent is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single interval preimage to the common locus of a finite family of pointwise inequalities. Its proof requires induction over the finite index set and repeated intersections of closed sets, which is not supplied by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from one interval preimage to the finite intersection of pointwise order loci. Its proof requires finite-set induction and repeated closed-set intersections, reasoning not supplied by the parent's single preimage argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2846 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_pointwise_order_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space, let ι be a type, and let s be a nonempty finite subset of ι . Let f and g be functions from ι to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \iota$, the function $f(i) : \alpha \rightarrow \mathbb{R}$ is continuous, and for every $i \in \iota$, the function $g(i) : \alpha \rightarrow \mathbb{R}$ is continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) \leq g(i)(x)$ for every $i \in s$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp at hs
  | @insert i s hi ih =>
    have h_i : IsClosed {x : α | f i x ≤ g i x} := by
      exact isClosed_le (hf i) (hg i)
    have h_s : IsClosed {x : α | ∀ j ∈ s, f j x ≤ g j x} := by
      rcases s.eq_empty_or_nonempty with rfl | hs'
      · simp
      · exact ih hs'
    have h_inter := h_i.inter h_s
    simpa [Finset.mem_insert, and_assoc, and_left_comm, and_comm] using h_inter
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__c0f67f60

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c0f67f60f7f40288

4.139 139. finite_pointwise_tolerance_locus_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that, for any $\varepsilon \in \mathbb{R}$, the set of points where $|f_i(x) - g_i(x)| \leq \varepsilon$ for every i is closed.

Lean statement

```
theorem finite_pointwise_tolerance_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ) (ε : ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, |f i x - g i x| ≤ ε}) : IsClosed z
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Supplied the continuous-difference, preimage-of-a-closed-set, and indexed-intersection method; the child expands the order locus to a uniform absolute-error tolerance locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a genuinely new tolerance condition involving absolute values and proves continuity of the composed absolute-value map before taking preimages of a closed ray. The parent's order-locus proof does not provide this absolute-value continuity and preimage argument, so recalling it would not substantially solve the child.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a symbolic tolerance and absolute-value locus, requiring continuity of the absolute-valued difference and a new preimage argument; recalling the parent's order-locus proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2844 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_tolerance_locus_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (ε : ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ (i : Fin n), |f i x - g i x| ≤ ε}) : IsClosed z
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric space structure, let n be a natural number, let $f, g : \text{Fin } n \rightarrow \alpha \rightarrow \mathbb{R}$, and let $\varepsilon \in \mathbb{R}$. Assume that for every $i \in \text{Fin } n$, the functions $x \mapsto f i x$ and $x \mapsto g i x$ are continuous. If $z \subseteq \alpha$ is exactly the set $\{x \in \alpha \mid \text{for every } i \in \text{Fin } n, |f i x - g i x| \leq \varepsilon\}$, then z is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_tolerance_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ) (ε : ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, |f i x - g i x| ≤ ε}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | |f i x - g i x| ≤ ε} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have habs : Continuous (fun x => |q x|) := hq.abs
    have hpre : IsClosed ((fun x => |q x|) ⁻¹' Set.Iic ε) :=
      IsClosed.preimage habs isClosed_Iic
    have heq : {x : α | |f i x - g i x| ≤ ε} =
      (fun x => |q x|) ⁻¹' Set.Iic ε := by
      ext x
      simp [q]
    rw [heq]
    exact hpre
  have hcone : IsClosed (⋂ i : Fin n, {x : α | |f i x - g i x| ≤ ε}) :=
    isClosed_iInter hclosed
  have hlocus : z = ⋂ i : Fin n, {x : α | |f i x - g i x| ≤ ε} := by
    rw [hz]
    ext x
    simp
  simpa only [hlocus] using hcone
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__000e7c9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 000e7c9a13984189

4.140 140. finite_pointwise_order_locus_as_continuous_zero_set

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Show that the locus where $f_i(x) \leq g_i(x)$ for every i is exactly the zero set of a continuous nonnegative function formed by summing the finitely many violations.

Lean statement

```
theorem finite_pointwise_order_locus_as_continuous_zero_set
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  ∃ A : α → ℝ,
    (∀ x, A x = (Finset.univ.sum (fun i => max (f i x - g i x) 0))) ∧
    Continuous A ∧
    (∀ x, 0 ≤ A x) ∧
    (∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x) ∧
    ({x | ∀ i, f i x ≤ g i x} = A⁻¹' ({0} : Set ℝ))
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Provides the finite family of continuous functions and the pointwise order-locus domain; the mutation replaces the closure conclusion with a continuous nonnegative zero-locus representation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the closed-locus conclusion into an explicit construction of a continuous nonnegative finite-sum aggregate and characterizes its zero set. Its proof uses finite-sum nonnegativity and zero-sum reasoning, not the parent's closed-preimage/intersection argument, so recalling the parent does not substantially help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to constructing a specific continuous nonnegative finite sum whose zero set exactly characterizes the order locus. Its proof requires new reasoning about nonnegative sums and zero terms, so recalling the parent's closed-intersection proof would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to constructing a continuous nonnegative finite-sum aggregate whose zero set exactly characterizes the pointwise order locus. Its proof requires nonnegativity, zero-sum reasoning, and continuity of the max-based aggregate, none of which can be recovered by recalling the parent's preimage/intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2842 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_order_locus_as_continuous_zero_set.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  ∃ A,
    (∀ (x : α), A x = ∑ i, max (f i x - g i x) 0) ∧
    Continuous A ∧
    (∀ (x : α), 0 ≤ A x) ∧
    (∀ (x : α), A x = 0 ↔ ∀ (i : Fin n), f i x ≤ g i x) ∧ {x | ∀ (i : Fin n), f i x ≤ g i x} = A-1{0}
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric-space structure, let n be a natural number, and let $f, g : \text{Fin } n \rightarrow \alpha \rightarrow \mathbb{R}$ be functions such that, for every $i \in \text{Fin } n$, the functions $x \mapsto f(i, x)$ and $x \mapsto g(i, x)$ are continuous. Then there exists a function $A : \alpha \rightarrow \mathbb{R}$ such that, for every $x \in \alpha$, $A(x) = \sum_{i \in \text{Fin } n} \max(f(i, x) - g(i, x), 0)$; A is continuous; for every $x \in \alpha$, $0 \leq A(x)$; for every $x \in \alpha$, $A(x) = 0$ if and only if, for every $i \in \text{Fin } n$, $f(i, x) \leq g(i, x)$; and the set $\{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f(i, x) \leq g(i, x)\}$ equals the preimage $A^{-1}(\{0\})$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem finite_pointwise_order_locus_as_continuous_zero_set
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  ∃ A : α → ℝ,
    (∀ x, A x = (Finset.univ.sum (fun i => max (f i x - g i x) 0))) ∧
    Continuous A ∧
    (∀ x, 0 ≤ A x) ∧
    (∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x) ∧
    ({x | ∀ i, f i x ≤ g i x} = A-1{0} : Set ℝ) := by
  let A : α → ℝ := fun x => Finset.univ.sum (fun i => max (f i x - g i x) 0)
  have hcont : Continuous A := by
    dsimp [A]
    fun_prop
  have hnonneg : ∀ x, 0 ≤ A x := by
    intro x
    dsimp [A]
    exact Finset.sum_nonneg (fun i hi => le_max_right _ _)
  have hiff : ∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x := by
    intro x
    constructor
    · intro hi
      have hle : max (f i x - g i x) 0 ≤ A x := by
        dsimp [A]
        exact Finset.single_le_sum
          (fun (j : Fin n) _ => le_max_right (f j x - g j x) 0)
          (Finset.mem_univ i)
      have htermle : max (f i x - g i x) 0 ≤ 0 := by
        simpa [hi] using hle
      have htermzero : max (f i x - g i x) 0 = 0 :=
```

```

    le_antisymm htermle (le_max_right _ _)
  have hdiff : f i x - g i x ≤ 0 := by
    have hleft := le_max_left (f i x - g i x) 0
    rw [htermzero] at hleft
    exact hleft
  linarith
· intro hall
  dsimp [A]
  apply Finset.sum_eq_zero
  intro i hi
  apply max_eq_right
  linarith [hall i]
refine (A, ?_, hcont, hnonneg, hiff, ?_)
· intro x
  rfl
· ext x
  simpa only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff] using (hiff x).symm

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__4d879715

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4d879715e2757247

4.141 141. continuous_cross_equalizer_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let g be a continuous real-valued function on a metric space Y . Prove that the set of pairs (x, y) satisfying $f(x) = g(y)$ is closed.

Lean statement

```
theorem continuous_cross_equalizer_closed
  {α β : Type} [MetricSpace α] [MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {p : α × β | f p.1 = g p.2} := by
```

Parent. Let f and g be continuous real-valued functions on a metric space X . Prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Parent in Lean

```
theorem continuous_equalizer_closed
  {α : Type} [MetricSpace α]
  (f g : α → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

What it contributes. Reuses the parent's continuous-difference and closed-zero-preimage method, with product projections added to handle distinct coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the equalizer from one domain to two independent metric spaces and requires continuity of the composed projections on the product. The parent proof does not supply those projection and product-space arguments directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the equalizer from one shared domain to a product of two metric spaces. Its proof must establish continuity after composing each function with the first and second projections, which is mathematics not supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from an equalizer on one space to a cross-equalizer on a product of two spaces. Its proof must establish continuity through the first and second projections, which is reasoning not supplied by recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2856 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_cross_equalizer_closed.extracted_1_1 {α β : Type} [inst : MetricSpace α] [inst_1 : MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) : IsClosed {p | f p.1 = g p.2}
```

Read back by a model that saw only the Lean. *Let α and β be metric spaces, let $f: \alpha \rightarrow \mathbb{R}$ and $g: \beta \rightarrow \mathbb{R}$ be continuous functions. Then the set of pairs $(a,b) \in \alpha \times \beta$ such that $f(a) = g(b)$ is closed in $\alpha \times \beta$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_cross_equalizer_closed
  {α β : Type} [MetricSpace α] [MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {p : α × β | f p.1 = g p.2} := by
  have hfp : Continuous (fun p : α × β => f p.1) := by
    exact hf.comp continuous_fst
  have hgp : Continuous (fun p : α × β => g p.2) := by
    exact hg.comp continuous_snd
  have hdif : Continuous (fun p : α × β => f p.1 - g p.2) := by
    exact hfp.sub hgp
  have hzero : IsClosed ({0} : Set ℝ) := by
    exact isClosed_singleton
  have hpreimage : IsClosed ((fun p : α × β => f p.1 - g p.2)⁻¹' ({0} : Set ℝ)) := by
    exact IsClosed.preimage hdif hzero
  have hequalizer : {p : α × β | f p.1 = g p.2} =
    (fun p : α × β => f p.1 - g p.2)⁻¹' ({0} : Set ℝ) := by
    ext p
    simp only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff]
  constructor
  · intro h
    exact sub_eq_zero.mpr h
  · intro h
    exact sub_eq_zero.mp h
  rw [hequalizer]
  exact hpreimage
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__d9523552

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d9523552a0127411

4.142 142. finite_pointwise_strict_order_locus_open

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) < g_i(x)$ for every i is open.

Lean statement

```
theorem finite_pointwise_strict_order_locus_open
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (u : Set α) (hu : u = {x | ∀ i, f i x < g i x}) : IsOpen u
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Supplies the finite-index pointwise order-locus setup and the continuous-difference checkpoint; the mutation changes the closed non-strict locus into an open strict locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the mathematical claim from a closed non-strict inequality locus to an open strict inequality locus. Its proof requires the open ray preimage argument and finite indexed intersection, rather than merely wrapping or decorating the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the mathematical claim from a finite non-strict-order locus being closed to a finite strict-order locus being open. It requires the open ray and finite intersection arguments, so recalling the parent's closed-set proof does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different openness theorem for strict inequalities, requiring open-ray preimages and finite intersections of open sets. The parent's closed non-strict argument does not directly close this goal, despite the analogous proof structure.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2838 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_strict_order_locus_open.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (u : Set α)
  (hu : u = {x | ∀ (i : Fin n), f i x < g i x}) : IsOpen u
```

Read back by a model that saw only the Lean. *Let α be a metric space, let n be a natural number, and let f and g be functions from $\text{Fin } n$ to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \text{Fin } n$, the function $f\ i$ is continuous and the function $g\ i$ is continuous. If $u \subseteq \alpha$ satisfies $u = \{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f\ i(x) < g\ i(x)\}$, then u is open.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_strict_order_locus_open
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (u : Set α) (hu : u = {x | ∀ i, f i x < g i x}) : IsOpen u := by
  have hopen : ∀ i : Fin n, IsOpen {x : α | f i x < g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hpre : IsOpen (q ⁻¹' Set.Iio (0 : ℝ)) :=
      IsOpen.preimage hq isOpen_Iio
    have heq : {x : α | f i x < g i x} = q ⁻¹' Set.Iio (0 : ℝ) := by
      ext x
      change (f i x < g i x) ↔ q x < 0
      constructor <=> intro h <=> dsimp [q] <=> linarith
    rw [heq]
    exact hpre
  have hinter : IsOpen (⋂ i : Fin n, {x : α | f i x < g i x}) :=
    isOpen_iInter_of_finite hopen
  have hu' : u = ⋂ i : Fin n, {x : α | f i x < g i x} := by
    rw [hu]
    ext x
    simp
  rw [hu']
  exact hinter
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__fc50be9d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint fc50be9d98793731

4.143 143. continuous_interval_preimage_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Lean statement

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Generalizes the parent's continuous-preimage-of-a-closed-set checkpoint from the singleton $\{0\}$ to the closed interval $[a,b]$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the singleton preimage to the preimage of an arbitrary closed interval. Its proof requires establishing closedness of the interval via 'isClosed_Icc', which is not supplied by recalling the parent's singleton argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the singleton preimage to the preimage of an arbitrary closed interval. Proving closedness of the interval and identifying the set as its preimage requires reasoning not supplied by the parent's singleton argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the closed singleton zero-level set to the inverse image of an arbitrary closed interval. It requires establishing interval closedness and identifying the set as a preimage, so recalling the parent does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_interval_preimage_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (f : α → ℝ)
  (h : Continuous f) (a b : ℝ) (hab : a ≤ b) : IsClosed {x | a ≤ f x ∧ f x ≤ b}
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric-space structure, and let $f : \alpha \rightarrow \mathbb{R}$ be continuous. For real numbers a and b with $a \leq b$, the set of all x in α such that $a \leq f(x)$ and $f(x) \leq b$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
  have h_interval_closed : IsClosed (Set.Icc a b) := by
    exact isClosed_Icc
  have h_preimage_closed : IsClosed (f ⁻¹' Set.Icc a b) := by
    exact IsClosed.preimage h h_interval_closed
  have h_set_eq : {x : α | a ≤ f x ∧ f x ≤ b} = f ⁻¹' Set.Icc a b := by
    ext x
    simp [Set.mem_Icc]
  rw [h_set_eq]
  exact h_preimage_closed
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__18b2d420

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 45c40ee9bb3f29e2

4.144 144. finite_common_level_locus_closed

The problem

Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Lean statement

```
theorem finite_common_level_locus_closed
  {α : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Generalizes the parent's continuous preimage-of-a-closed-singleton argument from one zero set to a finite intersection of level sets.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from one zero-level set to the common level locus of a finite family of continuous functions. Its proof requires finite intersections of closed preimages, which is new mathematics not supplied by the parent's single-preimage argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero-level locus to the common level locus of a finite family, requiring a finite-intersection argument and set extensionality in addition to continuity preimages. Memorizing the parent does not supply the central common-locus reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent from one zero-level preimage to the finite intersection of level sets for a family of continuous functions. Its proof requires finite-intersection reasoning and cannot be obtained by simply recalling the parent's singleton-preimage argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2311 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_level_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (h : ∀ (i : ι), Continuous (f i)) (c : ℝ) : IsClosed {x | ∀ i ∈ s, f i x = c}
```

Read back by a model that saw only the Lean. *Let α be a metric space, let ι be a type, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a continuous function $f_i : \alpha \rightarrow \mathbb{R}$. For any real number c , the set of points $x \in \alpha$ such that $f_i(x) = c$ for every $i \in s$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_level_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
  have h_level : ∀ i, IsClosed (f i ⁻¹' ({c} : Set ℝ)) := by
    intro i
    exact IsClosed.preimage (h i) (by simp)
  have h_common :
    IsClosed (⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ)) := by
    exact isClosed_biInter (fun i hi => h_level i)
  have h_eq :
    {x : α | ∀ i ∈ s, f i x = c} =
    ⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ) := by
    ext x
    simp
  rw [h_eq]
  exact h_common
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__517e1096

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 517e109641faffc9

4.145 145. bounded_simultaneous_equalizer_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every i is closed.

Lean statement

```
theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Used the parent's closed-preimage checkpoint for each continuous difference function, generalized from one zero locus to a bounded simultaneous equalizer.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one continuous function's zero locus to the simultaneous equalizer of a finite family. Its proof requires new reasoning via pointwise differences and a finite intersection of closed sets, so recalling the parent's preimage argument alone is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero locus to the simultaneous equalizer locus of a finite family. Its proof requires constructing pairwise closed equalizer sets via continuous differences and combining them with a finite intersection, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero locus to a finite simultaneous equalizer locus. Its proof requires constructing continuous differences and combining their closed equalizer sets via a finite intersection, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_simultaneous_equalizer_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (z : Set α)
  (hz : z = {x | ∀ (i : Fin n), f i x = g i x}) : IsClosed z
```

Read back by a model that saw only the Lean. *Let α be a metric space, let n be a natural number, and let f and g be families of real-valued functions on α indexed by $\text{Fin } n$. Suppose that for every $i : \text{Fin } n$, $f i$ and $g i$ are continuous. If z is the set of all x in α such that $f i x = g i x$ for every $i : \text{Fin } n$, then z is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | f i x = g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hzero : IsClosed (q ⁻¹' ({0} : Set ℝ)) :=
      IsClosed.preimage hq (by simp)
    have heq : {x : α | f i x = g i x} = q ⁻¹' ({0} : Set ℝ) := by
      ext x
      simp [q, sub_eq_zero]
    rw [heq]
    exact hzero
  have hfinite : IsClosed (⋂ i : Fin n, {x : α | f i x = g i x}) :=
    isClosed_iInter hclosed
  have hz' : z = ⋂ i : Fin n, {x : α | f i x = g i x} := by
    rw [hz]
    ext x
    simp
    rw [hz']
  exact hfinite
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__872e7f6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 872e7f6f479bc1d7

4.146 146. common_nonnegative_locus_closed

The problem

Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every $f_i(x) \geq 0$ is closed.

Lean statement

```
theorem common_nonnegative_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, 0 ≤ f i x} := by
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

What it contributes. Retains the metric-space, finite-family, and continuity setup while replacing the common zero locus with a common inequality locus; its aggregate argument motivates the family-wise closed-set checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the zero-locus conclusion to a common nonnegativity locus and proves it via individual preimages of the closed ray $[0, \infty)$ followed by an intersection, rather than recalling the parent's sum-of-squares argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the zero-locus into the common nonnegativity locus and proves it through closed half-line preimages and a finite intersection. This requires a different topological argument and cannot be solved by recalling the parent's sum-of-squares proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different closed-set claim using individual preimages of the closed ray $[0, \infty)$ and a finite intersection, rather than recalling the parent's sum-of-squares zero-locus argument. All hypotheses and intermediate closedness facts are used in the final proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2154 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem common_nonnegative_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (f : ι → α → ℝ)
  (h : ∀ (i : ι), Continuous (f i)) : IsClosed {x | ∀ (i : ι), 0 ≤ f i x}
```

Read back by a model that saw only the Lean. *Let α and ι be types, let α be equipped with a metric-space structure, and let f be a family of real-valued functions on α indexed by ι . Suppose that for every $i \in \iota$, the function $x \mapsto f(i, x)$ is continuous. Then the set of all $x \in \alpha$ such that $f(i, x) \geq 0$ for every $i \in \iota$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem common_nonnegative_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, 0 ≤ f i x} := by
  have h_i : ∀ i, IsClosed {x : α | 0 ≤ f i x} := by
    intro i
    change IsClosed ((f i) ⁻¹' Set.Ici (0 : ℝ))
    exact isClosed_Ici.preimage (h i)
  have h_locus :
    {x : α | ∀ i, 0 ≤ f i x} = ⋂ i, {x : α | 0 ≤ f i x} := by
    ext x
    simp
  rw [h_locus]
  exact isClosed_iInter h_i
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.me__1122ba60

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1122ba60417540d3

4.147 147. common_zero_locus_closed_normed

The problem

Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous maps from X into a normed real vector space. Prove that the set of points where every f_i vanishes is closed.

Lean statement

```
theorem common_zero_locus_closed_normed
  {α ι E : Type*} [MetricSpace α] [Fintype ι]
  [NormedAddCommGroup E] [NormedSpace ℝ E]
  (f : ι → α → E) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0}
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

What it contributes. Supplied the finite-family simultaneous-zero-locus strategy and the finite sum-of-squares continuity checkpoint; generalized the codomain from $\mathbb{R}$ to a normed real vector space.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the codomain from $\mathbb{R}$ to a real normed vector space and must use norm-square vanishing to recover each zero. The parent's scalar sum-of-squares proof cannot be recalled verbatim to solve this statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from real-valued functions to maps into a real normed vector space and requires the new norm-vanishing argument via 'norm_eq_zero'. Although the proof skeleton is similar, recalling the parent does not supply the codomain-generalized aggregate or its zero characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the codomain from $\mathbb{R}$ to an arbitrary real normed vector space and replaces scalar squares with squared norms. It additionally proves vanishing of each vector from norm-square vanishing, so recalling the parent's scalar proof does not directly solve the new goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2161 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A assumes the index family is finite, whereas Statement B allows an arbitrary index type ι .

Goal round-trip: what Lean elaborated

```
theorem common_zero_locus_closed_normed.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {ι : Type u_2} {E : Type u_3}
  [inst : MetricSpace α] [inst_1 : NormedAddCommGroup E] (f : ι → α → E) (h : ∀ (i : ι), Continuous (f i)) :
  IsClosed {x | ∀ (i : ι), f i x = 0}
```

Read back by a model that saw only the Lean. *Let α be a metric space, E a normed additive commutative group, and ι any type. Let f be a family of functions $f_i : \alpha \rightarrow E$ indexed by ι , and suppose each f_i is continuous. Then the set of all $x \in \alpha$ such that $f_i(x) = 0$ for every $i \in \iota$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem common_zero_locus_closed_normed
  {α ι E : Type*} [MetricSpace α] [Fintype ι]
  [NormedAddCommGroup E] [NormedSpace ℝ E]
  (f : ι → α → E) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
  have h_aggregate :
    Continuous (fun x : α => ∑ i, ‖f i x‖ ^ 2) := by
    fun_prop
  have h_zero_closed : IsClosed ({0} : Set ℝ) := by
    simp
  have h_aggregate_closed :
    IsClosed ((fun x : α => ∑ i, ‖f i x‖ ^ 2) ^{-1'} ({0} : Set ℝ)) :=
    IsClosed.preimage h_aggregate h_zero_closed
  have h_locus :
    {x : α | ∀ i, f i x = 0} =
      (fun x : α => ∑ i, ‖f i x‖ ^ 2) ^{-1'} ({0} : Set ℝ) := by
    ext x
    simp only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff]
  constructor
  · intro hx
    simp [hx]
  · intro hx
    have hsq : ∀ i, ‖f i x‖ ^ 2 = 0 := by
      intro i
      have hnonneg : ∀ j ∈ (Finset.univ : Finset ι), 0 ≤ ‖f j x‖ ^ 2 := by
        intro j hj
        positivity
      have hall := (Finset.sum_eq_zero_iff_of_nonneg hnonneg).mp hx
      exact hall i (Finset.mem_univ i)
    intro i
    have hnorm : ‖f i x‖ = 0 := by
      nlinarith [hsq i]
    exact norm_eq_zero.mpr hnorm
  rw [h_locus]
  exact h_aggregate_closed
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.me__4d450bdc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 4d450bdc5f309318

4.148 148. finite_zero_constrained_nonneg_closed

The problem

Let X be a metric space, let $f : X \rightarrow \mathbb{R}$ and let $(g_i)_{i \in I} : X \rightarrow \mathbb{R}$ be continuous, where I is finite. Prove that the set of points satisfying $f(x) = 0$ and $g_i(x) \geq 0$ for every $i \in I$ is closed.

Lean statement

```
theorem finite_zero_constrained_nonneg_closed
  {X ι : Type*} [MetricSpace X] [Fintype ι]
  (f : X → ℝ) (g : ι → X → ℝ)
  (hf : Continuous f) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} := by
```

Parent. Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Parent in Lean

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

What it contributes. Supplies the zero-locus checkpoint and the closed comparison/superlevel-set argument, generalized from one inequality to a finite indexed intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one nonnegativity constraint to an indexed family and requires proving closure via an intersection of closed sets, which the parent's proof does not supply. The finite index assumption is unused because arbitrary intersections of closed sets are closed, making the result stronger rather than a recall or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one continuous nonnegativity constraint to a finite family indexed by ι . Its proof requires establishing closedness of an indexed intersection and combining it with the zero locus, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one nonnegativity constraint to an indexed family and requires proving closedness via an intersection of closed sets, rather than directly reusing the parent's single-constraint argument. The finite-index assumption is unnecessary, but the indexed-family reasoning is substantive and the family hypotheses are used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2145 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the hypothesis that the index type ι is finite, allowing infinitely many inequalities; arbitrary intersections of closed sets need not be closed.

Goal round-trip: what Lean elaborated

```
theorem finite_zero_constrained_nonneg_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {ι : Type u_2}
  [inst : MetricSpace X] (f : X → ℝ) (g : ι → X → ℝ) (hf : Continuous f) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ (i : ι), 0 ≤ g i x}
```

Read back by a model that saw only the Lean. Let X be a metric space, let ι be a type, let $f : X \rightarrow \mathbb{R}$ and $g : \iota \rightarrow X \rightarrow \mathbb{R}$ be functions. Assume that f is continuous and that, for every $i \in \iota$, the function $x \mapsto g i x$ is continuous. Then the set of all $x \in X$ such that $f(x) = 0$ and $g(i, x) \geq 0$ for every $i \in \iota$ is closed in X .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem finite_zero_constrained_nonneg_closed
  {X ι : Type*} [MetricSpace X] [Fintype ι]
  (f : X → ℝ) (g : ι → X → ℝ)
  (hf : Continuous f) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_ineq_closed : IsClosed (⋂ i : ι, {x | 0 ≤ g i x}) := by
    exact isClosed_iInter (fun i => isClosed_le continuous.const (hg i))
  have h_combined : IsClosed ({x | f x = 0} ∩ (⋂ i : ι, {x | 0 ≤ g i x})) := by
    exact h_zero_closed.inter h_ineq_closed
  have h_set : {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} =
    {x | f x = 0} ∩ (⋂ i : ι, {x | 0 ≤ g i x}) := by
    ext x
    simp
  rw [h_set]
  exact h_combined
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe.me__f4046ed9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint f4046ed933be35da

4.149 149. bounded_shift_forward_difference

The problem

Suppose f is differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. If h is nonnegative and bounded above by B , prove that $f(x + h) - f(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Lean statement

```
theorem bounded_shift_forward_difference
  {f : ℝ → ℝ} {h B : ℝ}
  (hh0 : 0 ≤ h) (hhB : h ≤ B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Filter.Tendsto (deriv f) Filter.atTop (ℳ 0)) :
  Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℳ 0)
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x + 1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℳ 0) := by
```

What it contributes. Reuses the parent’s derivative-at-infinity estimate, eventual lower-bound extraction, positivity domain restriction, and mean-value theorem checkpoint, with the unit interval replaced by the bounded interval $[x, x+h]$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the unit shift by a bounded nonnegative shift, requiring a zero-shift case and a new derivative bound scaled by $B+1$. The parent’s fixed-shift proof does not directly establish this generalized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces the unit shift with an arbitrary bounded nonnegative shift. It requires new reasoning for the zero-shift case and scales the derivative bound using the symbolic upper bound B , so recalling the parent proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the unit shift by a bounded nonnegative real h , requiring a zero-shift case and a new derivative bound scaled by $B+1$ before applying the mean value theorem. This is not recoverable by simply recalling the parent’s unit-interval proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2763 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_forward_difference.extracted_1_1 {f : ℝ → ℝ} {h B : ℝ} (hh0 : 0 ≤ h) (hhB : h ≤ B)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ 0)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0)
```

Read back by a model that saw only the Lean. *Let f be a real-valued function on the real numbers, and let h and B be real numbers. Suppose that $0 \leq h$, $h \leq B$, f is differentiable on the open interval $(0, \infty)$, and the derivative $f'(x)$ tends to 0 as x tends to $+\infty$. Then $f(x + h) - f(x)$ tends to 0 as x tends to $+\infty$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_forward_difference
{f : ℝ → ℝ} {h B : ℝ}
(hh0 : 0 ≤ h) (hhB : h ≤ B)
(hfd : DifferentiableOn ℝ f (Ioi 0))
(hf : Tendsto (deriv f) atTop (ℳ 0)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0) := by
  by_cases hzero : h = 0
  · subst h
    simpa using (tendsto_const_nhds : Tendsto (fun _ : ℝ => (0 : ℝ)) atTop (ℳ 0))
  · have hpos : 0 < h := lt_of_le_of_ne hh0 (Ne.symm hzero)
    refine Metric.tendsto_atTop.2 ?_
    intro ε εpos
    have hB1 : 0 < B + 1 := by linarith
    have hδpos : 0 < ε / (B + 1) := div_pos εpos hB1
    have hderiv_small : ∀ f y : ℝ in atTop, |deriv f y| < ε / (B + 1) := by
      have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hδpos)
      refine hball.mono ?_
      intro y hy
      simpa [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
    obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
    refine ⟨max N 1, ?_⟩
    intro x hx
    have hxN : N ≤ x := le_trans (le_max_left N 1) hx
    have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
    have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
    have hxlt : x < x + h := lt_add_of_pos_right x hpos
    have hsubset_cc : Icc x (x + h) ⊆ Ioi (0 : ℝ) := by
      intro y hy
      exact lt_of_lt_of_le hxpos hy.1
    have hsubset_oo : Ioo x (x + h) ⊆ Ioi (0 : ℝ) := by
      intro y hy
      exact lt_trans hxpos hy.1
    have hcont_restrict : ContinuousOn f (Icc x (x + h)) :=
      (hfd.continuousOn).mono hsubset_cc
    have hdiff_restrict : DifferentiableOn ℝ f (Ioo x (x + h)) :=
      hfd.mono hsubset_oo
    obtain ⟨c, hc_mem, hc_deriv⟩ :=
      exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
    have hxden : (x + h) - x = h := by ring
    have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
    have hbound : |deriv f c| < ε / (B + 1) := hN c hc_ge
    have hmul : (B + 1) * |deriv f c| < ε := by
      calc
```

```

(B + 1) * |deriv f c| = |deriv f c| * (B + 1) := by ring
_ < (ε / (B + 1)) * (B + 1) := mul_lt_mul_of_pos_right hbound hB1
_ = ε := by field_simp [ne_of_gt hB1]
have hle : h * |deriv f c| ≤ (B + 1) * |deriv f c| := by
  exact mul_le_mul_of_nonneg_right (by linarith [hhB]) (abs_nonneg _)
have hdiffbound : h * |deriv f c| < ε := lt_of_le_of_lt hle hmul
have hdiff_eq : f (x + h) - f x = h * deriv f c := by
  rw [hc_deriv, hxdn]
  field_simp [hzero]
have hgoal : |f (x + h) - f x| < ε := by
  rw [hdiff_eq, abs_mul, abs_of_pos hpos]
  exact hdiffbound
simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__me__1e023b0f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1e023b0fde6347d9

4.150 150. variable_increment_tendsto

The problem

Suppose f is differentiable for every $x > 0$ and $f'(x) \rightarrow L$ as $x \rightarrow +\infty$. For a fixed positive increment a , prove that $f(x + a) - f(x) \rightarrow aL$ as $x \rightarrow +\infty$.

Lean statement

```
theorem variable_increment_tendsto
  {f : ℝ → ℝ} {a L : ℝ}
  (ha : 0 < a)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Tendsto (fun x => f (x + a) - f x) Filter.atTop (ℳ (a * L))
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x + 1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℳ 0) := by
```

What it contributes. Supplies the differentiability-on- $(0, \infty)$ setup, atTop derivative convergence strategy, interval restriction, and mean-value-theorem checkpoint; the child carries an arbitrary positive increment and arbitrary derivative limit through the final estimate.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment and derivative limit, requiring a scaled epsilon bound and a nontrivial mean-value/algebraic argument to obtain the limit aL . The fixed unit-shift, zero-limit parent proof does not directly establish this conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment and the derivative limit. Its proof must scale the derivative error by ε/a , use positivity of arbitrary a to apply the mean value theorem on $[x, x+a]$, and algebraically derive convergence to aL ; the parent's unit-zero specialization does not supply these obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2249 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem variable_increment_tendsto.extracted_1_1 {f : ℝ → ℝ} {a L : ℝ} (ha : 0 < a)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ L)) :
Tendsto (fun x => f (x + a) - f x) atTop (ℳ (a * L))
```

Read back by a model that saw only the Lean. Let f be a real-valued function on the real numbers, and let a and L be real numbers. Assume that a is positive, that f is differentiable on the open interval $(0, \infty)$, and that the derivative of f tends to L as x tends to $+\infty$. Then, as x tends to $+\infty$, the difference $f(x + a) - f(x)$ tends to aL .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem variable_increment_tendsto
  {f : ℝ → ℝ} {a L : ℝ}
  (ha : 0 < a)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Tendsto (fun x => f (x + a) - f x) Filter.atTop (ℳ (a * L)) := by
  refine Metric.tendsto_atTop.2 ?_
  intro ε hε
  have hderiv_small : ∀ε y : ℝ in Filter.atTop, |deriv f y - L| < ε / a := by
    have hball := hf.eventually (Metric.ball_mem_nhds L (div_pos hε ha))
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N 1, ?_⟩
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left N 1) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hsubset_cc : Set.Icc x (x + a) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + a) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + a)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + a)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + a := lt_add_of_pos_right x ha
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + a)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
  have hden : (x + a) - x = a := by ring
  have hc_deriv' : deriv f c = (f (x + a) - f x) / a := by
    simp [hden] using hc_deriv
  have hdiff_eq : f (x + a) - f x = a * deriv f c := by
    calc
      f (x + a) - f x = ((f (x + a) - f x) / a) * a := by
        field_simp [ne_of_gt ha]
      _ = deriv f c * a := by rw [← hc_deriv']
      _ = a * deriv f c := by ring
  have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
  have hbound : |deriv f c - L| < ε / a := hN c hc_ge
  have hgoal : |(f (x + a) - f x) - a * L| < ε := by
    rw [hdiff_eq]
    calc
```



```

|a * deriv f c - a * L| = a * |deriv f c - L| := by
  rw [show a * deriv f c - a * L = a * (deriv f c - L) by ring]
  rw [abs_mul, abs_of_pos ha]
_ < a * (ε / a) := mul_lt_mul_of_pos_left hbound ha
_ = ε := by field_simp [ne_of_gt ha]
have hgoal' : |f (x + a) - (a * L + f x)| < ε := by
  convert hgoal using 1 <|> ring
simp [Real.dist_eq] using hgoal'

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__me__71e4f964

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 71e4f964bc061904

4.151 151. shifted_secant_slope_monotone

The problem

Let f be continuous on $[a, \infty)$, differentiable on (a, ∞) , satisfy $f(a) = 0$, and have an increasing derivative there. Prove that the secant slope $f(x)/(x - a)$ is increasing for $x > a$.

Lean statement

```
theorem shifted_secant_slope_monotone
  {f : ℝ → ℝ} (a : ℝ)
  (hf1 : ContinuousOn f (Set.Ici a))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi a))
  (hf3 : f a = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
  MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a)
```

Parent. Suppose (a) f is continuous for $x \geq 0$, (b) $f'(x)$ exists for $x > 0$, (c) $f(0) = 0$, (d) f' is monotonically increasing. Put $g(x) = \frac{f(x)}{x}$ ($x > 0$) and prove that g is monotonically increasing.

Parent in Lean

```
theorem Rudin_exercise_5_6
  {f : ℝ → ℝ}
  (hf1 : ContinuousOn f (Set.Ici 0))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf3 : f 0 = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi 0)) :
  MonotoneOn (λ x => f x / x) (Set.Ioi 0) := by
```

What it contributes. Generalizes the parent secant-slope monotonicity argument from basepoint 0 to an arbitrary basepoint a , preserving the convexity and chord-comparison checkpoints while introducing translated domains and denominators.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the basepoint and translates the positive ray, requiring the convexity and secant comparison to be rebuilt with expressions involving $x-a$ and $y-a$. The arbitrary shift is not a decorative wrapper and the proof cannot be applied from the parent without substantive adaptation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the basepoint: the convexity argument and secant comparison must be translated from 0 to an arbitrary a , with new positivity, affine-combination, and denominator algebra. The parent's proof supplies a strategy but does not directly prove the shifted conclusion by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2172 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_secant_slope_monotone.extracted_1_1 {f : ℝ → ℝ} (a : ℝ) (hf1 : ContinuousOn f (Set.Ici a))
(hf2 : DifferentiableOn ℝ f (Set.Ioi a)) (hf3 : f a = 0) (hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a)
```

Read back by a model that saw only the Lean. *Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $a \in \mathbb{R}$. Assume that f is continuous on $[a, \infty)$, differentiable on (a, ∞) , $f(a) = 0$, and the derivative f' is nondecreasing on (a, ∞) . Then the function $x \mapsto f(x)/(x - a)$ is nondecreasing on (a, ∞) .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem shifted_secant_slope_monotone
{f : ℝ → ℝ} (a : ℝ)
(hf1 : ContinuousOn f (Set.Ici a))
(hf2 : DifferentiableOn ℝ f (Set.Ioi a))
(hf3 : f a = 0)
(hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a) := by
have hfconv : ConvexOn ℝ (Set.Ici a) f := by
  have hdiff : DifferentiableOn ℝ f (interior (Set.Ici a)) := by
    simpa [interior_Ici] using hf2
  have hmono : MonotoneOn (deriv f) (interior (Set.Ici a)) := by
    simpa [interior_Ici] using hf4
  exact hmono.convexOn_of_deriv (convex_Ici a) hf1 hdiff
refine fun x hx y hy hxy => ?_
have hx_pos : a < x := hx
have hy_pos : a < y := hy
have hxden : 0 < x - a := sub_pos.mpr hx_pos
have hyden : 0 < y - a := sub_pos.mpr hy_pos
have hy_mem : y ∈ Set.Ici a := le_of_lt hy_pos
have ha_mem : a ∈ Set.Ici a := by simp
have hy_ne : y - a ≠ 0 := ne_of_gt hyden
set θ : ℝ := (x - a) / (y - a)
have hθ_nonneg : 0 ≤ θ := by
  simpa [θ] using div_nonneg hxden.le hyden.le
have hnum : x - a ≤ y - a := by linarith
have hθ_le_one : θ ≤ 1 := by
  simpa [θ] using div_le_one_of_le hnum hyden.le
have hone_minus_nonneg : 0 ≤ 1 - θ := sub_nonneg.mpr hθ_le_one
have hcomb : θ * y + (1 - θ) * a = x := by
  dsimp [θ]
  field_simp [hy_ne] <|> ring
have hconv_app := hfconv.2 hy_mem ha_mem hθ_nonneg hone_minus_nonneg (by ring)
have hfxle : f x ≤ θ * f y := by
  calc
    f x = f (θ * y + (1 - θ) * a) := by rw [hcomb]
    _ ≤ θ * f y + (1 - θ) * f a := hconv_app
    _ = θ * f y := by rw [hf3]; ring
have hcross : f x * (y - a) ≤ f y * (x - a) := by
  calc
    f x * (y - a) ≤ (θ * f y) * (y - a) :=
      mul_le_mul_of_nonneg_right hfxle hyden.le
    _ = f y * (x - a) := by
      dsimp [θ]
```

```
field_simp [hy_ne] <|> ring
exact (div_le_div_iff hxden hyden).2 hcross
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_6__me__a17d08de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a17d08de5eac9a69

4.152 152. bounded_coefficient_phase_series_converges

The problem

Let $|z| = 1$. If (b_n) is uniformly bounded by B and (a_n) is dominated by a summable nonnegative sequence (g_n) , prove that the partial sums of $\sum z^{n+1} b_n a_n$ converge.

Lean statement

```
theorem bounded_coefficient_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hb :
  ↪ ∀ n, ‖b n‖ ≤ B) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * (b i * a i))
  ↪ : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  ↪ : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retains the parent's unit-phase construction, norm domination, comparison test, and partial-sum convergence argument, while inserting the essential bounded coefficient family and scaled summable majorant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely introduces a parameterized bounded coefficient sequence and must establish the new majorant $\|b_n * a_n\| \leq B * g_n$ and summability of the scaled majorant. This requires reasoning beyond recalling the parent's direct domination argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded coefficient family introduces genuinely new reasoning: the proof must establish nonnegativity of the majorant, bound the product by $B \cdot g$, and prove that scaled series is summable. The parent's direct norm domination does not supply this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded coefficient family requires new reasoning: deriving nonnegativity of the majorant and proving the product bound $\|b_n a_n\| \leq B g_n$, then establishing summability of the scaled majorant. This is not supplied by recalling the parent's direct domination proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2207 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_coefficient_phase_series_converges.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (hB : 0 ≤ B)
(a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hb : ∀ (n : ℕ), ‖b n‖ ≤ B) (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n)
(hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * (b i * a i)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. Let z be a complex number with $\|z\| = 1$, let $b : \mathbb{N} \rightarrow \mathbb{C}$, let B be a real number with $0 \leq B$, let $a : \mathbb{N} \rightarrow \mathbb{C}$, let $g : \mathbb{N} \rightarrow \mathbb{R}$, and let $s : \mathbb{N} \rightarrow \mathbb{C}$. Suppose that for every natural number n , $\|b n\| \leq B$; for every natural number n , $\|a n\| \leq g n$; the series $\sum_{n=0}^{\infty} g n$ is summable; and for every natural number n , $s n = \sum_{i \in \text{range } n} z^{i+1} (b i \cdot a i)$. Then there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem bounded_coefficient_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
(hb : ∀ n, ‖b n‖ ≤ B) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
(h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * (b i * a i)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * (b n * a n)
  have hg : ∀ n, 0 ≤ g n := by
    intro n
    exact le_trans (norm_nonneg (a n)) (hdom n)
  have hmajor : ∀ n, ‖f n‖ ≤ B * g n := by
    intro n
    have hprod : ‖b n‖ * ‖a n‖ ≤ B * g n := by
      nlinarith [hb n, hdom n, hg n, norm_nonneg (b n), norm_nonneg (a n)]
    calc
      ‖f n‖ = ‖b n‖ * ‖a n‖ := by simp [f, norm_mul, norm_pow, hz]
      _ ≤ B * g n := hprod
  have hsum_major : Summable (fun n => B * g n) := by
    simpa [smul_eq_mul] using hsummable.mul_left B
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsum_major hmajor
  have hconv : Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  refine ⟨∑' k, f k, ?_⟩
  simpa [h, f] using hconv
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me__263cd10d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 09cad9c3a6353e88

4.153 153. summable_majorant_complex_partial_sums

The problem

Let (a_n) be a sequence of complex numbers. If there is a summable nonnegative majorant (g_n) with $\|a_n\| \leq g_n$ for every n , prove that the partial sums of $\sum a_n$ converge.

Lean statement

```
theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℳ y)
```

Parent. Let (a_n) be a bounded sequence of complex numbers. If the series with terms $\|a_n\|/(n+1)^2$ converges, prove that the partial sums of $\sum a_n/(n+1)^2$ converge.

Parent in Lean

```
theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
  (hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retains the parent's complex-series-to-partial-sum convergence checkpoint while replacing the reciprocal-square bridge with an arbitrary summable majorant hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely replaces the reciprocal-square argument with a general summable majorant and uses norm domination to establish summability of the complex sequence. Its proof requires a different convergence principle and is not recoverable by recalling the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the reciprocal-square construction with a genuinely general summable-majorant argument and applies a different domination theorem to establish summability of the complex sequence. The parent's denominator-specific norm calculation and bridge hypothesis cannot be recalled to solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the reciprocal-square construction with an arbitrary summable majorant and uses norm domination to prove summability of the complex sequence. This requires a different convergence argument and cannot be solved by recalling the parent's weighted-series proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2140 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem summable_majorant_complex_partial_sums.extracted_1_1 (a : ℕ → ℂ) (g : ℕ → ℝ) (hmajorant : Summable g)
(hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (s : ℕ → ℂ) (h : s = fun n => ∑ i ∈ range n, a i) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. *Let a be a sequence of complex numbers and g a sequence of real numbers. Assume that the series $\sum_n g(n)$ is summable, that for every natural number n , the complex norm $\|a(n)\|$ is at most $g(n)$, and that s is the sequence of complex partial sums defined by $s(n) = \sum_{i < n} a(i)$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
(hmajorant : Summable g)
(hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℳ y) := by
  have hsum : Summable a := by
    exact Summable.of_norm_bounded hmajorant hdom
  refine ⟨∑' n, a n, ?_⟩
  have hpartial :
    Tendsto (fun n : ℕ => ∑ i ∈ Finset.range n, a i)
      atTop (ℳ (∑' n, a n)) :=
    hsum.hasSum.tendsto_sum_nat
  simpa [h] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me__938439ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 938439efcfd451c7

4.154 154. reciprocal_power_partial_sums_converge

The problem

Prove that for every natural number $p \geq 2$, every complex number z with $|z| = 1$, and every sequence s defined by $s_n = \sum_{i=0}^{n-1} \frac{z^{i+1}}{(i+1)^p}$, there exists a complex number y such that s_n converges to y .

Lean statement

```
theorem reciprocal_power_partial_sums_converge (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ) (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / ((i + 1 : ℕ) : ℂ) ^ p)) : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Generalizes the parent's unit-modulus complex partial-sum argument while replacing the fixed square exponent with $p \geq 2$ and explicitly proving the parameterized summability obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal-square exponent to an arbitrary natural $p \geq 2$. Its summability argument requires using hp to establish $p > 1$ and applying the p -series theorem at variable exponent p , which the parent's fixed-exponent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-square exponent to an arbitrary natural $p \geq 2$. Its proof must establish p -series summability from hp and transport the resulting norm estimate, which the parent's fixed-exponent argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-square exponent to arbitrary $p \geq 2$. Its convergence proof must establish summability for the variable exponent using hp and transport that result through the norm estimate, so recalling the parent's fixed-exponent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1934 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_power_partial_sums_converge.extracted_1_1 (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / ↑(i + 1) ^ p) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For every natural number p with $2 \leq p$, every complex number z with $\|z\| = 1$, and every sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s(n) = \sum_{i \in \{0, \dots, n-1\}} z^{i+1} / (i+1)^p$ for all $n \in \mathbb{N}$, there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem reciprocal_power_partial_sums_converge (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ) (h : s = (fun n => ∑ i ∈
  ↪ Finset.range n, z ^ (i + 1) / ((i + 1 : ℕ) : ℂ) ^ p)) : ∃ y, Tendsto s atTop (ℳ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / ((n + 1 : ℕ) : ℂ) ^ p
  have h_norm : (fun n : ℕ => ‖f n‖) = (fun n : ℕ => (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p) := by
    funext n
    have hzpow : ‖z ^ (n + 1)‖ = 1 := by
      simp [norm_pow, hz]
    have hden : ‖((n + 1 : ℕ) : ℂ)‖ = ((n + 1 : ℕ) : ℝ) := by
      simp using (Complex.norm_natCast (n + 1))
    calc
      ‖f n‖ = ‖z ^ (n + 1)‖ / ‖((n + 1 : ℕ) : ℂ) ^ p‖ := by simp [f]
      _ = 1 / ‖((n + 1 : ℕ) : ℂ) ^ p‖ := by rw [hzpow]
      _ = (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p := by rw [norm_pow, hden]
  have h_base : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p) := by
    apply (Real.summable_one_div_nat_pow (p := p)).mpr
  omega
  have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p) := by
    simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    rw [h_norm]
    exact h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine ⟨∑' n, f n, ?_⟩
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  have hpartial : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℳ (∑' n, f n)) := by
    exact hy.tendsto_sum_nat
  have htransport : Tendsto s atTop (ℳ (∑' n, f n)) := by
    simp [h, f] using hpartial
  exact htransport
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__15f3a09f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c10f3235f84352e8

4.155 155. dominated_phase_series_converges

The problem

Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Lean statement

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  ↪ : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Retains the unit-modulus complex phase, partial-sum encoding, and HasSum-based convergence conclusion while replacing reciprocal-square coefficients with a general dominated summable family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces the fixed coefficients $(n+1)^{-2}$ with an arbitrary complex sequence dominated by a summable real majorant. Its proof requires a new norm-comparison and summability argument, so recalling the parent's p-series proof is insufficient.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed reciprocal-square coefficients to an arbitrary complex sequence dominated by a summable majorant. Its proof requires a norm comparison and the summable comparison theorem, which the parent's proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed $1/(n+1)^2$ coefficients to an arbitrary complex sequence dominated by a summable real majorant. Its proof requires norm domination and a general summability theorem, so recalling the parent's p-series argument does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2172 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_converges.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm 1, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$, $g : \mathbb{N} \rightarrow \mathbb{R}$, and $s : \mathbb{N} \rightarrow \mathbb{C}$, suppose that for every natural number n , the norm of $a(n)$ is at most $g(n)$, that the real series $\sum_n g(n)$ is summable, and that for every natural number n , $s(n) = \sum_{i \in \{0, \dots, n-1\}} z^{(i+1)} a(i)$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    simpa [f, norm_mul, norm_pow, hz] using hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  refine ⟨∑' k, f k, ?_⟩
  simpa [h, f] using hconv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__2f08af31

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2f08af312633ecb1

4.156 156. phase_weighted_sum_converges

The problem

Let (a_n) be a complex coefficient family and let (c_n) be a summable nonnegative majorant with $\|a_n\| \leq c_n$. Show that for every complex z on the unit circle, the phase-weighted partial sums $\sum_{k < n} a_k z^{k+1}$ converge, with limit $\sum_{n \geq 0} a_n z^{n+1}$.

Lean statement

```
theorem phase_weighted_sum_converges
  (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)

  (hc : Summable c)
  (hbound : ∀ n, ‖a n‖ ≤ c n)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, a i * z ^ (i + 1))) :
  ∃ y, Tendsto s atTop (ℳ y) ∧ y = ∑' n, a n * z ^ (n + 1)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Retains the complex phase family, finite partial-sum encoding, and HasSum-to-Tendsto endpoint, while replacing the reciprocal-square coefficients with a parameterized majorant-controlled family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from the fixed coefficients $1/(n+1)^2$ to an arbitrary coefficient family dominated by an explicitly summable majorant. Its proof requires norm comparison and summability of the weighted series, rather than recalling the parent's p-series argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed coefficients $1/(n+1)^2$ to an arbitrary complex coefficient family dominated by a summable real majorant, and additionally identifies the limit as the infinite sum. Proving summability via norm comparison is new reasoning not supplied by the parent's specific p-series argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the parent from the specific coefficients $1/(n+1)^2$ to an arbitrary complex coefficient family dominated by a summable real majorant. Its proof requires a norm comparison and summability argument, rather than recalling the parent's p-series proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2739 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem phase_weighted_sum_converges.extracted_1.1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)
(hc_nonneg : ∀ (n : ℕ), 0 ≤ c n) (hc : Summable c) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ c n) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, a i * z ^ (i + 1)) :
  ∃ y, Tendsto s atTop (λ y) λ y = ∑' (n : ℕ), a n * z ^ (n + 1)
```

Read back by a model that saw only the Lean. For a complex number z with norm $\|z\| = 1$, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$ and $c : \mathbb{N} \rightarrow \mathbb{R}$ such that $c n \geq 0$ for every n , the series $\sum' n, c n$ is summable, and $\|a n\| \leq c n$ for every n , and for a sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s n = \sum_{i \in \{0, \dots, n-1\}} a i z^{(i+1)}$ for every n , there exists a complex number y such that s tends to y as n tends to infinity and $y = \sum' n, a n z^{(n+1)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem phase_weighted_sum_converges
  (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)

  (hc : Summable c)
  (hbound : ∀ n, ‖a n‖ ≤ c n)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, a i * z ^ (i + 1))) :
  ∃ y, Tendsto s atTop (λ y) λ y = ∑' n, a n * z ^ (n + 1) := by
  classical
  let f : ℕ → ℂ := fun n => a n * z ^ (n + 1)
  have h_norm_bound : ∀ n, ‖f n‖ ≤ c n := by
    intro n
    simp [f, norm_mul, norm_pow, hz] using hbound n
  have hfSummable : Summable f := by
    exact Summable.of_norm_bounded hc h_norm_bound
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  refine (∑' n, f n, ?_, ?_)
  · simp [h, f] using hy.tendsto_sum_nat
  · simp [f]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__3eeca7f4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20152abf32d46eef

4.157 157. bounded_reciprocal_square_convergence

The problem

Let (a_n) be a bounded sequence of complex numbers. If the series with terms $\|a_n\|/(n+1)^2$ converges, prove that the partial sums of $\sum a_n/(n+1)^2$ converge.

Lean statement

```
theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
  (hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Preserves the parent's reciprocal-square complex partial-sum and HasSum-to-Tendsto structure, while replacing the unit-modulus numerator with a bounded coefficient sequence and an explicit norm-summability bridge.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child replaces the fixed unit-modulus coefficient sequence with an arbitrary complex sequence and must establish convergence through absolute summability of its norm terms. Although the bridge hypothesis supplies that summability directly, the parent's power-norm argument cannot simply be recalled to solve the new statement.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces the fixed unit-modulus phase with an arbitrary bounded coefficient sequence and proves convergence by establishing norm-summability from the new bridge hypothesis, then applying absolute convergence. The parent's p-series argument does not supply this reasoning, and the added hypotheses are used materially.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed unit-modulus phase sequence with an arbitrary complex coefficient sequence and proves convergence via a supplied norm-summability bridge, including the nontrivial derivation that the bound constant is nonnegative. The parent's p-series argument cannot be recalled to solve this generalized statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2120 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_reciprocal_square_convergence.extracted_1_1 (a : ℕ → ℂ) (C : ℝ) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ C)
(hbridge : 0 ≤ C → Summable fun n => ‖a n‖ / (↑n + 1) ^ 2) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, a i / (↑i + 1) ^ 2) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. *Let a be a sequence of complex numbers, let C be a real number, and suppose that for every natural number n , the norm of $a(n)$ is at most C . Suppose also that if C is nonnegative, then the series with terms $\|a(n)\|/(n+1)^2$ is summable. Let s be a sequence of complex numbers satisfying, for every natural number n , $s(n) = \sum_{i < n} a(i)/(i+1)^2$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Complex Filter Function Metric Finset

theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
(hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
(hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
∃ y, Tendsto s atTop (ℕ y) := by
classical
set f : ℕ → ℂ := fun n => a n / (n + 1 : ℂ) ^ 2 with hf
have hC_nonneg : 0 ≤ C :=
le_trans (norm_nonneg (a 0)) (hbound 0)
have hbridge_used :
Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2) :=
hbridge hC_nonneg
have hnorm : ∀ n : ℕ, ‖f n‖ = ‖a n‖ / (n + 1 : ℝ) ^ 2 := by
intro n
have hden : ‖(n + 1 : ℂ) ^ 2‖ = (n + 1 : ℝ) ^ 2 := by
rw [norm_pow]
have hn : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
rw [hn]
calc
‖f n‖ = ‖a n‖ / ‖(n + 1 : ℂ) ^ 2‖ := by
rw [hf, norm_div]
_ = ‖a n‖ / (n + 1 : ℝ) ^ 2 := by rw [hden]
have hnorm_summable : Summable (fun n : ℕ => ‖f n‖) := by
rw [show (fun n : ℕ => ‖f n‖) =
(fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2) by
funext n
exact hnorm n]
exact hbridge_used
have hfSummable : Summable f := Summable.of_norm hnorm_summable
refine ⟨∑' n, f n, ?_⟩
have hpartial :
Tendsto (fun n : ℕ => ∑ k ∈ Finset.range n, f k)
atTop (ℕ (∑' n, f n)) :=
hfSummable.hasSum.tendsto_sum_nat
simp [h, f] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__8adda5dd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

4.158 158. cubic_denominator_linear_growth_convergence

The problem

Let (a_n) be a complex sequence and suppose there is a real constant C such that $|a_n| \leq C(n+1)$ for every n . Prove that the series $\sum_{n=0}^{\infty} a_n/(n+1)^3$ converges.

Lean statement

```
theorem cubic_denominator_linear_growth_convergence (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ n : ℕ, ‖a n‖ ≤ C * (n + 1 : ℝ))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 3)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Reuses the parent’s reciprocal-square p-series checkpoint and partial-sum convergence pattern, while replacing the fixed phase numerator with a variable sequence and a cubic-denominator majorant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from unit-modulus powers and quadratic denominators to arbitrary complex coefficients with linear norm growth and cubic denominators. Its proof requires a new norm comparison against a shifted reciprocal-square majorant and does not reuse the parent’s argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: it replaces the unit-modulus numerator with arbitrary coefficients having linear norm growth, then proves convergence via cubic-denominator normalization and a shifted reciprocal-square majorant. This requires comparison and norm-growth reasoning absent from the parent’s fixed reciprocal-square proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the unit-modulus reciprocal-square series with a genuinely different convergence theorem: linearly growing coefficients over cubic denominators are reduced to a shifted reciprocal-square majorant. Its comparison and normalization argument is not supplied by the parent’s fixed-coefficient proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2898 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_denominator_linear_growth_convergence.extracted_1_1 (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ (n : ℕ), ‖a n‖ ≤ C * (n + 1)) (s : ℕ → ℂ) (h : s = fun n => ∑ i ∈ range n, a i / (i + 1) ^ 3) :
  ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For a sequence $a : \mathbb{N} \rightarrow \mathbb{C}$ and a real constant C , suppose that for every natural number n , the complex norm of $a(n)$ is at most $C(n+1)$. Let $s : \mathbb{N} \rightarrow \mathbb{C}$ be defined by $s(n) = \sum_{i=0}^{n-1} \{a(i)/(i+1)^3\}$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem cubic_denominator_linear_growth_convergence (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ n : ℕ, ‖a n‖ ≤ C * (n + 1 : ℝ))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 3)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
  classical
  let f : ℕ → ℂ := fun n => a n / (n + 1 : ℂ) ^ 3
  have hbase : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have hn : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
      (Real.summable_one_div_nat_pow (p := 2)).2 (by norm_num)
    simpa [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).mpr hn
  have hmajorant : Summable (fun n : ℕ => C / (n + 1 : ℝ) ^ 2) := by
    simpa [div_eq_mul_inv, mul_comm] using hbase.mul_left C
  have hnorm_bound : ∀ n : ℕ, ‖f n‖ ≤ C / (n + 1 : ℝ) ^ 2 := by
    intro n
    have hden : ‖(n + 1 : ℂ) ^ 3‖ = (n + 1 : ℝ) ^ 3 := by
      rw [norm_pow]
    have hn : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
      simpa [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
    rw [hn]
  calc
    ‖f n‖ = ‖a n‖ / (n + 1 : ℝ) ^ 3 := by
      simp [f, norm_div, hden]
    _ ≤ (C * (n + 1 : ℝ)) / (n + 1 : ℝ) ^ 3 := by
      exact div_le_div_of_nonneg_right (ha n) (by positivity)
    _ = C / (n + 1 : ℝ) ^ 2 := by
      field_simp
  have hfSummable : Summable f :=
    Summable.of_norm_bounded hmajorant hnorm_bound
  refine (Σ' n, f n, ?_)
  have hpartial :
    Tendsto (fun n : ℕ => ∑ k ∈ Finset.range n, f k)
      atTop (ℕ (Σ' n, f n)) :=
    hfSummable.hasSum.tendsto_sum_nat
  simpa [h, f] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__8feb093a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 8feb093a58671b9d

4.159 159. reciprocal_power_partial_sums_converge

The problem

Let $p \geq 2$. Prove that the power series $\sum z^n/n^p$ converges at every point z on the unit circle.

Lean statement

```
theorem reciprocal_power_partial_sums_converge (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Preserves the parent's unit-circle norm argument and partial-sum convergence route, while replacing exponent 2 with p and adding reciprocal-square domination.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the reciprocal-square series to every integer exponent $p \geq 2$. Its proof requires a new comparison argument showing $(n+1)^2 \leq (n+1)^p$ and transferring summability, which is not supplied by the parent's fixed-exponent proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-power exponent and must prove summability for every integer $p \geq 2$ by comparing the p -series to the square series. This reasoning is not supplied by the parent's fixed-exponent proof, and no hypotheses are redundant.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2116 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_power_partial_sums_converge.extracted_1_1 (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (i + 1) ^ p) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z and a natural number p with $2 \leq p$, if $\|z\| = 1$, and if $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfies $s(n) = \sum_{i \in \text{range } n} z^{i+1} / ((i : \mathbb{C}) + 1)^p$ for every natural number n , then there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem reciprocal_power_partial_sums_converge (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ p
  have h_norm : (fun n : ℕ => ‖f n‖) =
    (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) := by
    funext n
    have hbase : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
      simp using (Complex.norm_natCast (n + 1))
    simp [f, hz, hbase]
  have h_square : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have hs : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
      (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
    simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 hs
  have h_compare : ∀ n : ℕ, 0 ≤ (1 : ℝ) / (n + 1 : ℝ) ^ p ∧
    (1 : ℝ) / (n + 1 : ℝ) ^ p ≤ (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
    intro n
    let x : ℝ := n + 1
    have hx : 1 ≤ x := by
      dsimp [x]
      exact_mod_cast (Nat.succ_le_succ (Nat.zero_le n))
    have hpow : x ^ 2 ≤ x ^ p := pow_le_pow_right hx hp
    have hpos : 0 < x ^ 2 := pow_pos (by linarith) _
    constructor
    · positivity
    · exact one_div_le_one_div_of_le hpos hpow
  have h_p : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) :=
    Summable.of_nonneg_of_le (fun n => (h_compare n).1) (fun n => (h_compare n).2) h_square
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    rw [h_norm]
    exact h_p
  have hf : Summable f := Summable.of_norm h_norm_summable
  refine (∑' n, f n, ?_)
  have hy : HasSum f (∑' n, f n) := hf.hasSum
  have h_partial : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℳ (∑' n, f n)) :=
    hy.tendsto_sum_nat
  simp [h, f] using h_partial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__9361aa77

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9361aa775d5ad8a3

4.160 160. scaled_unit_circle_p_series_converges

The problem

Let c be a fixed complex number. Prove that the series obtained by multiplying z^n/n^2 by c converges at every point z on the unit circle.

Lean statement

```
theorem scaled_unit_circle_p_series_converges
  (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ Finset.range n,
    c * (z ^ (i + 1) / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
```

What it contributes. Uses the parent’s unit-circle norm estimate, shifted p-series summability checkpoint, and partial-sum convergence architecture; mutation adds a fixed scalar multiplier and explicitly consumes the base summability through ‘Summable.mul_left’.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The scalar parameter c is genuinely used through `Summable.mul_left`, so the parent’s unscaled summability argument does not directly supply the child’s proof. The child also explicitly derives norm summability before transporting it through scalar multiplication and identifying the partial sums.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the series by an arbitrary complex scalar and must establish summability through the new ‘`Summable.mul_left c`’ step. The parent’s proof does not supply this scalar-dependent argument by recall alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an arbitrary complex scalar and must establish summability of the scaled sequence via ‘`Summable.mul_left c`’ before identifying its partial sums. This is not solvable by simply recalling the parent’s unscaled convergence proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_unit_circle_p_series_converges.extracted_1_1 (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, c * (z ^ (i + 1) / (i + 1) ^ 2)) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For complex numbers c and z with $\|z\| = 1$, and a sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s(n) = \sum_{i < n} c \cdot (z^{i+1} / (i+1)^2)$ for every natural number n , there exists a complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem scaled_unit_circle_p_series_converges
  (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ Finset.range n,
    c * (z ^ (i + 1) / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ 2
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
  funext n
  have hzpow : ‖z ^ (n + 1)‖ = 1 := by
    rw [norm_pow, hz]
  norm_num
  have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
    simp using (Complex.norm_natCast (n + 1))
  calc
    ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ 2‖ := by
      simp [f]
    _ = 1 / ‖(n + 1 : ℂ) ^ 2‖ := by rw [hzpow]
    _ = (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
      rw [norm_pow, hden]
  have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have h_base :
      Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
        (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
    simp [Nat.cast_add, Nat.cast_one] using
      (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable f := by
    apply Summable.of_norm
    rw [h_norm]
    exact h_pSeries
  have h_scaled : Summable (fun n : ℕ => c * f n) := by
    exact h_norm_summable.mul_left c
  refine ⟨∑' n, c * f n, ?_⟩
  have hy : HasSum (fun n : ℕ => c * f n) (∑' n, c * f n) :=
    h_scaled.hasSum
  simp [h, f] using hy.tendsto_sum_nat
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__eb1129bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint eb1129bff69b752e

4.161 161. bounded_unit_circle_power_series_hasSum

The problem

Let p be a natural number greater than 1, let z lie on the unit circle, and let (a_n) be a bounded complex sequence. Prove that the series $\sum_{n \geq 0} a_n z^{n+1} / (n+1)^p$ has a sum.

Lean statement

```
theorem bounded_unit_circle_power_series_hasSum
  (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p)
  (a : ℕ → ℂ) (B : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ B) :
  ∃ y : ℂ, HasSum (fun n : ℕ => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p) y
```

Parent. Let p be an integer greater than 1. Prove that the power series $\sum z^n / n^p$ has a sum at every point z on the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b_mutation (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p) :
  ∃ y : ℂ, HasSum (fun n : ℕ => z ^ (n + 1) / (n + 1 : ℂ) ^ p) y := by
```

What it contributes. Supplied the unit-circle norm reduction and shifted reciprocal-power summability checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the numerator to an arbitrary uniformly bounded complex family and requires a norm-majorant comparison argument. The parent's exact norm computation cannot be recalled to solve this without the new bounded-coefficient reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the numerator to an arbitrary bounded complex family and must establish a new norm majorization using the bound. The parent proof only handles the constant numerator and cannot be recalled to solve this claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the constant numerator to an arbitrary bounded complex coefficient family. Its proof requires a new norm-majorization argument and uses the bound hypothesis essentially, so recalling the parent's norm equality proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2751 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_unit_circle_power_series_hasSum.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p) (a : ℕ → ℂ)
  (B : ℝ) (hB : 0 ≤ B) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ B) :
  ∃ y, HasSum (fun n => a n * z ^ (n + 1) / (↑n + 1) ^ p) y
```

Read back by a model that saw only the Lean. For every complex number z with norm $\|z\| = 1$, every natural number p with $1 < p$, every function $a : \mathbb{N} \rightarrow \mathbb{C}$, and every real number $B \geq 0$ such that $\|a(n)\| \leq B$ for all natural numbers n , there exists a complex number y such that the series $\sum_{n=0}^{\infty} a(n) z^{n+1} / (n+1)^p$ converges to y in the sense of `HasSum`.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem bounded_unit_circle_power_series_hasSum
  (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p)
  (a : ℕ → ℂ) (B : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ B) :
  ∃ y : ℂ, HasSum (fun n : ℕ => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p) y := by
classical
let f : ℕ → ℂ := fun n => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p
have h_base : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p) :=
  (Real.summable_one_div_nat_pow (p := p)).mpr hp
have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) := by
  simpa [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
have h_majorant : Summable (fun n : ℕ => B * ((1 : ℝ) / (n + 1 : ℝ) ^ p)) :=
  Summable.mul_left B h_pSeries
have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
  apply h_majorant.of_nonneg_of_le
  · intro n
    exact norm_nonneg _
  · intro n
    have hzpow : ‖z ^ (n + 1)‖ = 1 := by
      simp [hz, norm_pow]
    have hn1 : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
      simpa using (Complex.norm_natCast (n + 1))
    have hden : ‖(n + 1 : ℂ) ^ p‖ = (n + 1 : ℝ) ^ p := by
      rw [norm_pow, hn1]
    calc
      ‖f n‖ = ‖a n‖ / ‖(n + 1 : ℂ) ^ p‖ := by simp [f, hz]
      _ = ‖a n‖ * (1 / (n + 1 : ℝ) ^ p) := by rw [hden]; ring
      _ ≤ B * (1 / (n + 1 : ℝ) ^ p) := by
        exact mul_le_mul_of_nonneg_right (hbound n) (by positivity)
have hfSummable : Summable f := Summable.of_norm h_norm_summable
refine (Σ' n, f n, ?_)
have hsum : HasSum f (Σ' n, f n) := hfSummable.hasSum
simpa [f] using hsum
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh.me__b7093508

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 756af0834f5f75eb

4.162 162. bounded_rational_recurrence_transfer

The problem

Let t and u be sequences of rational numbers satisfying $t_1 = u_1 = 20$, $t_2 = u_2 = 21$, and, for every integer $n \geq 3$, $t_n = \frac{5t_{n-1}+1}{25t_{n-2}}$ and $u_n = \frac{5u_{n-1}+1}{25u_{n-2}}$. If the denominator of t_{2020} plus its numerator equals 626, prove that $t_n = u_n$ for every $1 \leq n \leq 2020$, and that the denominator of u_{2020} plus its numerator also equals 626.

Lean statement

```
theorem bounded_rational_recurrence_transfer (t u : ℕ → ℚ)
  (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)))
  (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2)))
  (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ n, 1 ≤ n → n ≤ 2020 → t n = u n) ∧
  ↑(u 2020).den + (u 2020).num = 626
```

Parent. Define a sequence recursively by $t_1 = 20$, $t_2 = 21$, and $t_n = \frac{5t_{n-1}+1}{25t_{n-2}}$ for all $n \geq 3$. Then t_{2020} can be expressed as $\frac{p}{q}$, where p and q are relatively prime positive integers. Find $p + q$. Show that it is 626.

Parent in Lean

```
theorem aimeII_2020_p6 (t : ℕ → ℚ) (h₀ : t 1 = 20) (h₁ : t 2 = 21)
  (h₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2))) :
  ↑(t 2020).den + (t 2020).num = 626 := by
```

What it contributes. Supplies the rational recurrence, initial values, bounded horizon 2020, and endpoint numerator-denominator invariant transferred here to a second admissible sequence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive bounded uniqueness theorem: strong induction proves $t_n = u_n$ for every index from the shared initial data and recurrence, then transfers the endpoint invariant to u . The parent's explicit five-step cycle proof does not supply this sequence-comparison argument, and memorizing it offers little help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded uniqueness law by strong induction, showing $t_n = u_n$ from shared initial values and recurrence, then transfers the endpoint invariant. The parent's five-cycle computation does not supply this sequence-to-sequence uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded uniqueness law by strong induction, comparing both sequences at each predecessor and predecessor-two recurrence step. The endpoint transfer then uses this newly established equality, so the parent's five-cycle computation is not recalled or embedded.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1735 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_rational_recurrence_transfer.extracted_1_1 (t u : ℕ → ℚ) (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2))) (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2))) (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ (n : ℕ), 1 ≤ n → n ≤ 2020 → t n = u n) ∧ ↑(u 2020).den + (u 2020).num = 626
```

Read back by a model that saw only the Lean. Let t and u be functions from the natural numbers to the rational numbers. Suppose $t(1)=20$, $t(2)=21$, and for every natural number n with $n \geq 3$, $t(n)=(5t(n-1)+1)/(25t(n-2))$, where the subtractions are natural-number subtractions. Suppose likewise that $u(1)=20$, $u(2)=21$, and for every natural number n with $n \geq 3$, $u(n)=(5u(n-1)+1)/(25u(n-2))$. Suppose also that the natural-number denominator of $t(2020)$, coerced to a rational number, plus the numerator of $t(2020)$, equals 626. Then, for every natural number n with $1 \leq n \leq 2020$, $t(n)=u(n)$, and the denominator of $u(2020)$, coerced to a rational number, plus the numerator of $u(2020)$, equals 626.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_rational_recurrence_transfer (t u : ℕ → ℚ)
  (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)))
  (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2)))
  (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ n, 1 ≤ n → n ≤ 2020 → t n = u n) ∧
  ↑(u 2020).den + (u 2020).num = 626 := by
  have huniq : ∀ n : ℕ, 1 ≤ n → n ≤ 2020 → t n = u n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn hN
      by_cases h1 : n = 1
      · subst n
        exact ht₀.trans hu₀.symm
      by_cases h2 : n = 2
      · subst n
        exact ht₁.trans hu₁.symm
      have hn3 : 3 ≤ n := by omega
      have hp1 : t (n - 1) = u (n - 1) :=
        ih (n - 1) (by omega) (by omega) (by omega)
      have hp2 : t (n - 2) = u (n - 2) :=
        ih (n - 2) (by omega) (by omega) (by omega)
      have ht := ht₂ n (by omega)
      have hu := hu₂ n (by omega)
      calc
        t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)) := ht
        _ = (5 * u (n - 1) + 1) / (25 * u (n - 2)) := by rw [hp1, hp2]
        _ = u n := hu.symm
  constructor
  · exact huniq
  · rw [← huniq 2020 (by norm_num) (by norm_num)]
    exact hInv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aimeII_2020_p6__me__ce035b37

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 15ad3ed09766e4d3

4.163 163. parameterized_radical_root_product

The problem

For $0 \leq t \leq 1$, let the distinct real roots of $x^2 + 18x + 30 + t = 2\sqrt{x^2 + 18x + 45}$ be collected. Prove that their product is $28 + t - 2\sqrt{16 - t}$.

Lean statement

```
theorem parameterized_radical_root_product (t : ℝ) (f : ℝ → ℝ)
  (ht₁ : t ≤ 1)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30 + t) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) :
  (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 28 + t - 2 * Real.sqrt (16 - t) := by
```

Parent. What is the product of the $\mathbb{R}$ roots of the equation $x^2 + 18x + 30 = 2\sqrt{x^2 + 18x + 45}$? Show that it is 020.

Parent in Lean

```
theorem aime_1983_p3 (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) : (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 20 := by
```

What it contributes. Retains the parent's radical substitution, nonnegativity checkpoint, exact root-set characterization, finite-set product normalization, and root-product computation, while replacing the fixed coefficients by the bounded parameter t and adding explicit extraneous-branch exclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The parameter t genuinely changes the radical equation and forces a new quadratic-branch analysis, including admissibility of the negative branch and a parameter-dependent root product. The parent's fixed constant argument does not directly establish the child's symbolic formula.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The parameter t remains symbolic and changes the admissible quadratic branch, requiring new domain and positivity arguments before deriving the roots and their product. The parent's fixed- $t=0$ proof does not directly establish this parameterized theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The parameter t changes the quadratic branch structure: proving the admissible branch requires using $t \leq 1$ to show $\sqrt{16-t} > 1$ and exclude the extraneous solution. The resulting symbolic root product is not obtainable by simply recalling the parent's $t=0$ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2699 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_radical_root_product.extracted_1_1 (t : ℝ) (f : ℝ → ℝ) (ht₁ : t ≤ 1)
  (h₀ : ∀ (x : ℝ), f x = x ^ 2 + (18 * x + 30 + t) - 2 * √(x ^ 2 + (18 * x + 45))) (h₁ : Fintype ↑(f ⁻¹' {0})) :
  ∏ x ∈ (f ⁻¹' {0}).toFinset, x = 28 + t - 2 * √(16 - t)
```

Read back by a model that saw only the Lean. Let t be a real number with $t \leq 1$, and let $f: \mathbb{R} \rightarrow \mathbb{R}$ satisfy, for every real x , $f(x) = x^2 + (18x + 30 + t) - 2\sqrt{x^2 + (18x + 45)}$. Assume that the set of real roots of f , namely $f^{-1}(\{0\})$, is finite. Then the product of all real numbers x such that $f(x) = 0$ is $28 + t - 2\sqrt{16 - t}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem parameterized_radical_root_product (t : ℝ) (f : ℝ → ℝ)
  (ht₁ : t ≤ 1)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30 + t) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) :
  (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 28 + t - 2 * Real.sqrt (16 - t) := by
  letI := h₁
  let r : ℝ := Real.sqrt (16 - t)
  let q : ℝ := Real.sqrt (36 + (1 + r) ^ 2)
  have hrt : 0 ≤ 16 - t := by linarith
  have hr : 0 ≤ r := by
    dsimp [r]
    exact Real.sqrt_nonneg _
  have hr2 : r ^ 2 = 16 - t := by
    dsimp [r]
    nlinarith [Real.sq_sqrt hrt]
  have hro : 1 < r := by
    nlinarith [hr2]
  have hqarg : 0 ≤ (36 : ℝ) + (1 + r) ^ 2 := by
    nlinarith [sq_nonneg (1 + r)]
  have hq : q ^ 2 = 36 + (1 + r) ^ 2 := by
    dsimp [q]
    nlinarith [Real.sq_sqrt hqarg]
  have hchar : ∀ x : ℝ, f x = 0 ↔ x = -9 - q ∨ x = -9 + q := by
    intro x
    constructor
    · intro hx
      have he := h₀ x
      rw [hx] at he
      let z : ℝ := x ^ 2 + (18 * x + 45)
      have hez : 0 = z - 15 + t - 2 * Real.sqrt z := by
        dsimp [z]
        nlinarith [he]
      have hz : 0 ≤ z := by
        nlinarith [hez, Real.sqrt_nonneg z]
      have hv : (Real.sqrt z) ^ 2 - 2 * Real.sqrt z + t - 15 = 0 := by
        nlinarith [hez, Real.sq_sqrt hz]
      have hbranches :
        (Real.sqrt z - (1 + r)) * (Real.sqrt z - (1 - r)) = 0 := by
          nlinarith [hv, hr2]
      have hvalid : Real.sqrt z = 1 + r := by
        rcases mul_eq_zero.mp hbranches with hplus | hminus
        · linarith
        · ex falso
          nlinarith [Real.sqrt_nonneg z, hro]
      have hzs : z = (1 + r) ^ 2 := by
```

```

    nlinarith [Real.sq_sqrt hz, hvalid]
have hfactor : (x + 9 - q) * (x + 9 + q) = 0 := by
  dsimp [z] at hzs
  nlinarith [hq, hzs]
rcases mul_eq_zero.mp hfactor with hleft | hright
· right
  linarith
· left
  linarith
· intro hx
  have hy : 0 ≤ 1 + r := by linarith
  have hsq : Real.sqrt ((1 + r) ^ 2) = 1 + r := by
    have hs := Real.sq_sqrt (sq_nonneg (1 + r))
    nlinarith [Real.sqrt_nonneg ((1 + r) ^ 2)]
  rcases hx with rfl | rfl
  · have harg : (-9 - q) ^ 2 + (18 * (-9 - q) + 45) = (1 + r) ^ 2 := by
      nlinarith [hq]
      rw [h₀, harg, hsq]
      nlinarith [hq, hr2]
  · have harg : (-9 + q) ^ 2 + (18 * (-9 + q) + 45) = (1 + r) ^ 2 := by
      nlinarith [hq]
      rw [h₀, harg, hsq]
      nlinarith [hq, hr2]
have hset : f ⁻¹' {0} = ({-9 - q, -9 + q} : Set ℝ) := by
  ext x
  simp only [Set.mem_preimage, Set.mem_singleton_iff, Set.mem_insert_iff,
    Set.mem_singleton_iff]
  exact hchar x
have hfin : (f ⁻¹' {0}).toFinset = ({-9 - q, -9 + q} : Finset ℝ) := by
  ext x
  simp [hset]
rw [hfin]
rw [Finset.prod_insert]
· rw [Finset.prod_singleton]
  change (-9 - q) * (-9 + q) = 28 + t - 2 * r
  nlinarith [hq, hr2]
· intro h
  simp only [Finset.mem_singleton] at h
  nlinarith [hq]

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p3__me__c0629d00

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3925eaedea382347

4.164 164. aime_1984_p15_x_squared_scaled

The problem

Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = r$, where $r \geq 0$. Show that $x^2 = \frac{11025}{1024} r$.

Lean statement

```
theorem aime_1984_p15_x_squared_scaled (x y z w r : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = (11025 / 1024 : ℝ) * r
```

Parent. Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Parent in Lean

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

What it contributes. Retains the parent's four weighted equations and normalization/elimination method, while replacing the fixed right-hand side 1 with the shared nonnegative parameter r .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the common right-hand side, requiring the solver to derive the symbolic scaling relation $x^2 = (11025/1024)r$ rather than the parent's fixed numerical value. The proof uses the same linear structure but the parent's conclusion cannot simply be recalled, and no hypothesis is redundant.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the common right-hand side and proves the resulting scaling law for x^2 , rather than merely decorating the fixed-value conclusion. The parameter r remains mathematically essential and is used in the derived equation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the shared right-hand side and derives the scaled value of x^2 , requiring the coefficient relation to remain symbolic in r . It is not a wrapper or decoration, and the parent's fixed-value conclusion cannot be recalled verbatim.

Check

statement_type_check

By

lean

Result

type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2724 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_squared_scaled.extracted_1_1 (x y z w r : ℝ) (hr : 0 ≤ r)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = 11025 / 1024 * r
```

Read back by a model that saw only the Lean. For real numbers x, y, z, w , and r with $r \geq 0$, if $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = r$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = r$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = r$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = r$, then $x^2 = (11025/1024)r$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_squared_scaled (x y z w r : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = (11025 / 1024 : ℝ) * r := by
  norm_num at h₀ h₁ h₂ h₃
  have hlinear : 1024 * x ^ 2 = 11025 * r := by
    linarith [h₀, h₁, h₂, h₃]
  have hscale : x ^ 2 = (11025 / 1024 : ℝ) * r := by
    linarith [hlinear]
  exact hscale
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms.me__bebfb24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint a5628f834d130201

4.165 165. aime_1984_p15_x

The problem

Suppose x, y, z, w are real numbers satisfying the four weighted-square equations above, and $x \geq 0$. Determine x .

Lean statement

```
theorem aime_1984_p15_x (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
  (hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

Parent. Determine $x^2 + y^2 + z^2 + w^2$ if $\frac{x^2}{2^2-1} + \frac{y^2}{2^2-3^2} + \frac{z^2}{2^2-5^2} + \frac{w^2}{2^2-7^2} = 1$ $\frac{x^2}{4^2-1} + \frac{y^2}{4^2-3^2} + \frac{z^2}{4^2-5^2} + \frac{w^2}{4^2-7^2} = 1$ $\frac{x^2}{6^2-1} + \frac{y^2}{6^2-3^2} + \frac{z^2}{6^2-5^2} + \frac{w^2}{6^2-7^2} = 1$ $\frac{x^2}{8^2-1} + \frac{y^2}{8^2-3^2} + \frac{z^2}{8^2-5^2} + \frac{w^2}{8^2-7^2} = 1$ Show that it is 036.

Parent in Lean

```
theorem aime_1984_p15 (x y z w : ℝ)
  (h₀ :
    x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
    w ^ 2 / (2 ^ 2 - 7 ^ 2) =
    1)
  (h₁ :
    x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
    w ^ 2 / (4 ^ 2 - 7 ^ 2) =
    1)
  (h₂ :
    x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
    w ^ 2 / (6 ^ 2 - 7 ^ 2) =
    1)
  (h₃ :
    x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
    w ^ 2 / (8 ^ 2 - 7 ^ 2) =
    1) :
  x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
```

What it contributes. Retains the parent's full four-equation weighted-square system and its normalization/elimination checkpoint, while specializing with $x \geq 0$ and identifying the coordinate rather than the total energy.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from the total squared norm to a specific coordinate and uses the new non-negativity hypothesis to select the positive root. The parent's proof only derives the aggregate sum and does not determine x .

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from the total sum of squares to a uniquely determined coordinate value. It must first isolate x^2 from the four equations and then use nonnegativity to select the positive square root, reasoning not supplied by the parent's concluding equality.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining the total sum of squares to determining the unique nonnegative value of x . The parent's linear elimination only yields the aggregate result; the child additionally derives x^2 and uses nonnegativity with nonlinear reasoning, so recalling the parent proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2696 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
(hx : 0 ≤ x) : x = 105 / 32
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers satisfying $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. If x is nonnegative, then $x = 105/32$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
(hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1984_p15__me__3f82f5b1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3f82f5b1b213b0d1

4.166 166. squared_coordinate_classification

The problem

The four quadratic normalization identities uniquely determine the squared coordinates: $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean statement

```
theorem squared_coordinate_classification (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = (11025 : ℝ) / 1024 ∧
y ^ 2 = (10395 : ℝ) / 1024 ∧
z ^ 2 = (9009 : ℝ) / 1024 ∧
w ^ 2 = (6435 : ℝ) / 1024
```

Parent. Determine $x^2 + y^2 + z^2 + w^2$ if $\frac{x^2}{2^2-1} + \frac{y^2}{2^2-3^2} + \frac{z^2}{2^2-5^2} + \frac{w^2}{2^2-7^2} = 1$
 $\frac{y^2}{4^2-3^2} + \frac{z^2}{4^2-5^2} + \frac{w^2}{4^2-7^2} = 1$
 $\frac{x^2}{6^2-1} + \frac{y^2}{6^2-3^2} + \frac{z^2}{6^2-5^2} + \frac{w^2}{6^2-7^2} = 1$
 $\frac{x^2}{8^2-1} + \frac{y^2}{8^2-3^2} + \frac{z^2}{8^2-5^2} + \frac{w^2}{8^2-7^2} = 1$ Show that it is 036.

Parent in Lean

```
theorem aime_1984_p15 (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) =
1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) =
1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) =
1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) =
1) :
x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
```

What it contributes. Retains the original variables and all four quadratic normalization identities, while replacing the aggregate-energy target with the uniquely determined squared-coordinate vector.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child replaces the parent's single total-energy consequence with a full classification of all four squared coordinates. Although the same normalization and linear-arithmetic method applies, the parent proof does not supply these individual values, so this is a substantive change of shape.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the conclusion from the total squared-energy sum to a full classification of each squared coordinate. Proving four individual values requires solving the underlying four-equation linear system; the parent's final linarith step for the aggregate identity does not supply those coefficients.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a single total-energy sum to a full classification of all four squared coordinates. Proving each coordinate requires solving the entire linear system, which is not supplied by the parent's direct total-sum 'linarith' proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1820 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem squared_coordinate_classification.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem squared_coordinate_classification (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = (11025 : ℝ) / 1024 ∧
y ^ 2 = (10395 : ℝ) / 1024 ∧
z ^ 2 = (9009 : ℝ) / 1024 ∧
```

```

w ^ 2 = (6435 : ℝ) / 1024 := by
norm_num at h₀ h₁ h₂ h₃
have hclass :
  x ^ 2 = (11025 : ℝ) / 1024 ∧
  y ^ 2 = (10395 : ℝ) / 1024 ∧
  z ^ 2 = (9009 : ℝ) / 1024 ∧
  w ^ 2 = (6435 : ℝ) / 1024 := by
constructor
· linarith [h₀, h₁, h₂, h₃]
constructor
· linarith [h₀, h₁, h₂, h₃]
constructor
· linarith [h₀, h₁, h₂, h₃]
· linarith [h₀, h₁, h₂, h₃]
exact hclass

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1984_p15__me__8fb70bef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b1915c90a2b9f7a

4.167 167. cubic_affine_evaluation_injective

The problem

For complex numbers a , b , and c , if the affine coefficient $a^2 - (b + c)a + cb$ is nonzero, then the induced affine evaluation map is injective.

Lean statement

```
theorem cubic_affine_evaluation_injective (a b c : ℂ)
  (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective
    (fun x : ℂ =>
      -((a ^ 2 - (b + c) * a + c * b) * x) +
      (a ^ 2 - (b + c) * a + c * b) * a)
```

Parent. For complex numbers a, b, c, d, e , if $(a - d)(a - c)(a - b) = -((a^2 - (b + c)a + cb)d) + (a^2 - (b + c)a + cb)a$ and $(a - e)(a - c)(a - b) = -((a^2 - (b + c)a + cb)e) + (a^2 - (b + c)a + cb)a$, with $a - c \neq 0$, then the two evaluations are equal if and only if $d = e$ or $a = b$.

Parent in Lean

```
theorem cubic_factored_evaluation_collision (a b c d e : ℂ)
  (hfacd : (a - d) * (a - c) * (a - b) =
    -((a ^ 2 - (b + c) * a + c * b) * d) +
    (a ^ 2 - (b + c) * a + c * b) * a)
  (hfcae : (a - e) * (a - c) * (a - b) =
    -((a ^ 2 - (b + c) * a + c * b) * e) +
    (a ^ 2 - (b + c) * a + c * b) * a)
  (hnd : a - c ≠ 0) :
  (-((a ^ 2 - (b + c) * a + c * b) * d) +
    (a ^ 2 - (b + c) * a + c * b) * a =
    -((a ^ 2 - (b + c) * a + c * b) * e) +
    (a ^ 2 - (b + c) * a + c * b) * a) ↔
  d = e ∨ a = b
```

What it contributes. Reuses the parent's complex affine evaluation expression while promoting its pairwise collision analysis to a function-level injectivity theorem with an explicit nonzero-slope hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the collision disjunction into injectivity of an affine map and strengthens nondegeneracy to a nonzero slope. Its proof requires direct cancellation of that symbolic coefficient, rather than reusing the parent's two-point transport argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the collision characterization into injectivity of an affine map and proves it by deriving a nonzero slope and cancelling it. This is not a recalled application of the parent's disjunction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the two-point collision equivalence into injectivity of the affine evaluation map and proves it by explicit nonzero-slope cancellation. This is not a recall wrapper around the parent's disjunctive factorization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1858 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_affine_evaluation_injective.extracted_1_1 (a b c : ℂ) (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective fun x => -((a ^ 2 - (b + c) * a + c * b) * x) + (a ^ 2 - (b + c) * a + c * b) * a
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c , if $a^2 - (b + c)a + cb \neq 0$, then the function from $\mathbb{C}$ to $\mathbb{C}$ defined by $x \mapsto -(a^2 - (b + c)a + cb)x + (a^2 - (b + c)a + cb)a$ is injective.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem cubic_affine_evaluation_injective (a b c : ℂ)
  (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective
    (fun x : ℂ =>
      -((a ^ 2 - (b + c) * a + c * b) * x) +
      (a ^ 2 - (b + c) * a + c * b) * a) := by
  have hslope : -(a ^ 2 - (b + c) * a + c * b) ≠ 0 := by
    exact neg_ne_zero.mpr hs
  intro x y hxy
  have hdiff :
    (-(a ^ 2 - (b + c) * a + c * b)) * x =
    (-(a ^ 2 - (b + c) * a + c * b)) * y := by
    linear_combination hxy
  exact mul_left_cancel₀ hslope hdiff
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__me.mh.me__7de9810f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7a676a66f66e1f02

4.168 168. cubic_zero_locus_exact

The problem

Let a, b , and c be pairwise distinct complex numbers. Show that the zeros of $(x - a)(x - b)(x - c)$ are exactly a, b , and c , with each zero different from the other two.

Lean statement

```
theorem cubic_zero_locus_exact (a b c : ℂ) (hab : a ≠ b) (hac : a ≠ c) (hbc : b ≠ c) :
  ∀ x : ℂ, (x - a) * (x - b) * (x - c) = 0 ↔
    ((x = a ∧ x ≠ b ∧ x ≠ c) ∨
     (x = b ∧ x ≠ a ∧ x ≠ c) ∨
     (x = c ∧ x ≠ a ∧ x ≠ b))
```

Parent. Show that for any complex numbers a, b, c, d , $(a-d)(a-c)(a-b) = -(((a^2 - (b+c)a) + cb)d) + (a^2 - (b+c)a + cb)a$.

Parent in Lean

```
theorem algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta (a b c d : ℂ) :
  (a - d) * (a - c) * (a - b) =
  -((a ^ 2 - (b + c) * a + c * b) * d) + (a ^ 2 - (b + c) * a + c * b) * a := by
```

What it contributes. Reuses the parent cubic factorization surface, but changes the conclusion to an exact zero-locus classification with converse and distinctness exclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an algebraic identity into an exact zero-locus characterization under pairwise-distinctness hypotheses. Its proof requires factoring a zero product, case-splitting over complex factors, and deriving uniqueness inequalities, none of which follows from recalling the parent's ring proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an exact zero-locus characterization using pairwise distinctness and case analysis on vanishing factors. The parent's purely algebraic ring identity supplies none of this reasoning, so recalling it does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine zero-locus characterization requiring factor-by-factor case analysis and use of pairwise distinctness hypotheses. The parent is only a ring identity and provides none of this reasoning, so recalling its proof does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2729 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_zero_locus_exact.extracted_1_1 : ∀ (a b c : ℂ),
  a ≠ b →
  a ≠ c →
  b ≠ c →
  ∀ (x : ℂ),
    (x - a) * (x - b) * (x - c) = 0 ↔
    x = a ∧ x ≠ b ∧ x ≠ c ∨ x = b ∧ x ≠ a ∧ x ≠ c ∨ x = c ∧ x ≠ a ∧ x ≠ b
```

Read back by a model that saw only the Lean. For all complex numbers a , b , and c such that $a \neq b$, $a \neq c$, and $b \neq c$, and for every complex number x , $(x - a)(x - b)(x - c) = 0$ if and only if either $x = a$ and $x \neq b$ and $x \neq c$, or $x = b$ and $x \neq a$ and $x \neq c$, or $x = c$ and $x \neq a$ and $x \neq b$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem cubic_zero_locus_exact (a b c : ℂ) (hab : a ≠ b) (hac : a ≠ c) (hbc : b ≠ c) :
  ∀ x : ℂ, (x - a) * (x - b) * (x - c) = 0 ↔
    ((x = a ∧ x ≠ b ∧ x ≠ c) ∨
     (x = b ∧ x ≠ a ∧ x ≠ c) ∨
     (x = c ∧ x ≠ a ∧ x ≠ b)) := by
  intro x
  constructor
  · intro hx
    have hfactor : ((x - a) * (x - b)) * (x - c) = 0 := by
      simpa [mul_assoc] using hx
    rcases mul_eq_zero.mp hfactor with h_ab | h_c
    · rcases mul_eq_zero.mp h_ab with h_a | h_b
      · left
        have hxa : x = a := sub_eq_zero.mp h_a
        refine ⟨hxa, ?_, ?_⟩
        · intro hxb
          exact hab (hxa.symm.trans hxb)
        · intro hxc
          exact hac (hxa.symm.trans hxc)
      · right
        left
        have hxb : x = b := sub_eq_zero.mp h_b
        refine ⟨hxb, ?_, ?_⟩
        · intro hxa
          exact hab (hxa.symm.trans hxb)
        · intro hxc
          exact hbc (hxb.symm.trans hxc)
    · right
      right
      have hxc : x = c := sub_eq_zero.mp h_c
      refine ⟨hxc, ?_, ?_⟩
      · intro hxa
        exact hac (hxa.symm.trans hxc)
      · intro hxb
        exact hbc (hxb.symm.trans hxc)
  · intro hx
    rcases hx with hxa | hxb | hxc
    · rcases hxa with ⟨rfl, _, _⟩
      simp
    · rcases hxb with ⟨rfl, _, _⟩
      simp
    · rcases hxc with ⟨rfl, _, _⟩
      simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__me__6f44f522

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6f44f522c277cf76

4.169 169. ternary_coupled_reciprocal_strict

The problem

Let $d \in \{0, 1, 2\}$, and suppose a natural number has base-three representation $122d_3$. If positive reals a, b, c have sum equal to that number and are not all equal, show that $\frac{9}{d+51} < \frac{2}{a+b} + \frac{2}{b+c} + \frac{2}{c+a}$.

Lean statement

```
theorem ternary_coupled_reciprocal_strict
  (d n : ℕ) (a b c : ℝ)
  (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c)
  (hne : ¬ (a = b ∧ b = c))
  (htotal : a + b + c = (n : ℝ)) :
  9 / ((d : ℝ) + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a)
```

Parent. Show that for any three positive real numbers x, y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (ha : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Supplies the reciprocal-inequality clearing strategy, positivity checkpoints, common-denominator rewrites, and weighted squared-difference certificate; the child strengthens it to strictness and couples the total to the decoded parameter.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine strict strengthening: the non-equality hypothesis requires proving a positive squared-difference contribution, not merely replaying the parent's non-strict certificate. The digit coupling also forces the parameterized denominator through a new base-3 decoding argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent's non-strict inequality: the non-equality hypothesis is used to make one squared-difference term strictly positive, yielding a strict numerator certificate before clearing denominators. The digit decoding and total constraint also require additional reasoning, and the parent's proof alone cannot establish the strict conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent inequality to a strict one and must use the non-equality hypothesis to prove a squared-difference term is positive. The digit decoding and total constraint also replace the parent's symbolic denominator with a parameterized expression, so recalling the parent's non-strict proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2543 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_coupled_reciprocal_strict.extracted_1_1 (d n : ℕ) (a b c : ℝ) (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c) (hne : ¬(a = b ∧ b = c)) (htotal : a + b + c = ↑n) :
  9 / (↑d + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a)
```

Read back by a model that saw only the Lean. *Let d and n be natural numbers, and let a , b , and c be real numbers. Suppose that the base-3 digit list of n is exactly $[d, 2, 2, 1]$, that a , b , and c are all positive, that it is not the case that $a = b$ and $b = c$, and that $a + b + c = n$. Then $9/(d + 51) < 2/(a + b) + 2/(b + c) + 2/(c + a)$, where d and n are regarded as real numbers in the fractions.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_coupled_reciprocal_strict
  (d n : ℕ) (a b c : ℝ)
  (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c)
  (hne : ¬(a = b ∧ b = c))
  (htotal : a + b + c = (n : ℝ)) :
  9 / ((d : ℝ) + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a) := by
  rcases hpos with ⟨ha, hb, hc⟩
  have hvalue : n = d + 51 := by
    have hdecode : n = Nat.ofDigits 3 (Nat.digits 3 n) :=
      (Nat.ofDigits_digits 3 n).symm
    calc
      n = Nat.ofDigits 3 (Nat.digits 3 n) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [hdigits]
      _ = d + 51 := by norm_num [Nat.ofDigits]
  have hsum : a + b + c = (d : ℝ) + 51 := by
    calc
      a + b + c = (n : ℝ) := htotal
      _ = (d : ℝ) + 51 := by rw [hvalue]; norm_num
  have hab : 0 < a + b := by linarith
  have hbc : 0 < b + c := by linarith
  have hca : 0 < c + a := by linarith
  have habc : 0 < a + b + c := by linarith
  have h1 : 0 ≤ ((a + b) - (b + c)) ^ 2 := sq_nonneg _
  have h2 : 0 ≤ ((b + c) - (c + a)) ^ 2 := sq_nonneg _
  have h3 : 0 ≤ ((c + a) - (a + b)) ^ 2 := sq_nonneg _
  have h1' : 0 ≤ (a + b) * ((a + b) - (b + c)) ^ 2 :=
    mul_nonneg (le_of_lt hab) h1
  have h2' : 0 ≤ (b + c) * ((b + c) - (c + a)) ^ 2 :=
    mul_nonneg (le_of_lt hbc) h2
  have h3' : 0 ≤ (c + a) * ((c + a) - (a + b)) ^ 2 :=
    mul_nonneg (le_of_lt hca) h3
  have hneq : a ≠ b ∨ b ≠ c := by
    by_cases hab' : a = b
    · right
      intro hbc'
      exact hne ⟨hab', hbc'⟩
    · exact Or.inl hab'
  have hstrictnum :
    0 < 2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b)) -
      9 * (a + b) * (b + c) * (c + a) := by
    rcases hneq with hab' | hbc'
    · have hdiff : (b + c) - (c + a) ≠ 0 := by
```

```

    intro hz
    apply hab'
    linarith
have hs : 0 < ((b + c) - (c + a)) ^ 2 := by
  have hm : 0 < ((b + c) - (c + a)) * ((b + c) - (c + a)) :=
    mul_self_pos.mpr hdiff
  simp [pow_two] using hm
have hs' : 0 < (b + c) * ((b + c) - (c + a)) ^ 2 :=
  mul_pos hbc hs
nlinarith
· have hdiff : (c + a) - (a + b) ≠ 0 := by
  intro hz
  apply hbc'
  linarith
have hs : 0 < ((c + a) - (a + b)) ^ 2 := by
  have hm : 0 < ((c + a) - (a + b)) * ((c + a) - (a + b)) :=
    mul_self_pos.mpr hdiff
  simp [pow_two] using hm
have hs' : 0 < (c + a) * ((c + a) - (a + b)) ^ 2 :=
  mul_pos hca hs
nlinarith
have hden : 0 < (a + b + c) * (a + b) * (b + c) * (c + a) := by
  positivity
have hleft :
  9 / (a + b + c) =
    (9 * (a + b) * (b + c) * (c + a)) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
    field_simp [ne_of_gt habc, ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
have hright :
  2 / (a + b) + 2 / (b + c) + 2 / (c + a) =
    (2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b))) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
    field_simp [ne_of_gt habc, ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
rw [← hsum, hleft, hright]
exact (div_lt_div_iff_of_pos_right hden).2 (by nlinarith [hstrictnum])

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__me__7dc1d005

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7dc1d00539fc0f42

4.170 170. scaled_reciprocal_amgm

The problem

Let λ and x be positive real numbers. Show that $2 - \sqrt{2\lambda} \geq 2 - x - \frac{\lambda}{2x}$, with equality exactly when $x = \frac{\sqrt{2\lambda}}{2}$.

Lean statement

```
theorem scaled_reciprocal_amgm :
  ∀ (lam x : ℝ), lam > 0 → x > 0 →
    (2 - Real.sqrt (2 * lam) ≥ 2 - x - lam / (2 * x)) ∧
    ((2 - Real.sqrt (2 * lam) = 2 - x - lam / (2 * x)) ↔
     x = Real.sqrt (2 * lam) / 2) := by
```

Parent. Let x be a positive real number. Show that $2 - \sqrt{2} \geq 2 - x - \frac{1}{2x}$.

Parent in Lean

```
theorem algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x :
  ∀ x > 0, 2 - Real.sqrt 2 ≥ 2 - x - 1 / (2 * x) := by
```

What it contributes. Generalizes the parent's reciprocal AM-GM denominator-clearing strategy from fixed scale 1 to the parameterized scale λ , while adding an equality characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the scale and adds a sharp iff equality characterization, requiring symbolic square expansion and a separate equality argument. A memorized proof of the fixed-scale parent does not supply the generalized equality locus.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal inequality by a positive scale and adds a nontrivial equality characterization. Its proof must establish both directions of the equality case using the symbolic square identity, so recalling the parent's fixed-constant inequality is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the constant by introducing positive λ and additionally proves the sharp equality locus $x = \sqrt{(2\lambda)}/2$. Its proof requires symbolic square-root algebra and an equality characterization not supplied by the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2735 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_reciprocal_amgm.extracted_1_1 : ∀ (lam x : ℝ),
  lam > 0 →
  x > 0 →
  2 - √(2 * lam) ≥ 2 - x - lam / (2 * x) ∧ (2 - √(2 * lam) = 2 - x - lam / (2 * x) ↔ x = √(2 * lam) / 2)
```

Read back by a model that saw only the Lean. For all real numbers λ and x , if $\lambda > 0$ and $x > 0$, then $2 - \sqrt{(2\lambda)} \geq 2 - x - \lambda/(2x)$, and equality holds if and only if $x = \sqrt{(2\lambda)}/2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem scaled_reciprocal_amgm :
  ∀ (lam x : ℝ), lam > 0 → x > 0 →
    (2 - Real.sqrt (2 * lam) ≥ 2 - x - lam / (2 * x)) ∧
    ((2 - Real.sqrt (2 * lam) = 2 - x - lam / (2 * x)) ↔
      x = Real.sqrt (2 * lam) / 2) := by
  intro lam x hlam hx
  have hs : (Real.sqrt (2 * lam)) ^ 2 = 2 * lam := by
    have hnonneg : 0 ≤ 2 * lam := by positivity
    simpa using Real.sq_sqrt hnonneg
  have hmul :
    2 * x * Real.sqrt (2 * lam) ≤ 2 * x * x + lam := by
    have hsq : 0 ≤ (2 * x - Real.sqrt (2 * lam)) ^ 2 :=
      sq_nonneg (2 * x - Real.sqrt (2 * lam))
    nlinarith [hsq, hs]
  have hden : 0 < 2 * x := by positivity
  have hdiv :
    Real.sqrt (2 * lam) ≤ (2 * x * x + lam) / (2 * x) := by
    apply (le_div_iff₀ hden).2
    nlinarith [hmul]
  have hid :
    x + lam / (2 * x) = (2 * x * x + lam) / (2 * x) := by
    field_simp [ne_of_gt hx] <|> ring
  have hroot : Real.sqrt (2 * lam) ≤ x + lam / (2 * x) := by
    rw [hid]
    exact hdiv
  constructor
  · linarith
  · constructor
    · intro hEq
      have hsum : x + lam / (2 * x) = Real.sqrt (2 * lam) := by
        linarith [hEq]
      have hfrac :
        (2 * x * x + lam) / (2 * x) = Real.sqrt (2 * lam) := by
        rw [← hid]
        exact hsum
      have hb := (div_eq_iff (ne_of_gt hden)).mp hfrac
      have hbalance :
        2 * x * x + lam = 2 * x * Real.sqrt (2 * lam) := by
        nlinarith [hb]
      have hz : (2 * x - Real.sqrt (2 * lam)) ^ 2 = 0 := by
        nlinarith [sq_nonneg (2 * x - Real.sqrt (2 * lam)), hbalance, hs]
      nlinarith [hz]
    · intro hxeq
      subst x
      have hr : 0 < Real.sqrt (2 * lam) := by positivity
      field_simp [ne_of_gt hr] <|> nlinarith [hs]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x__me__2a3e888a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2a3e888a0ab283e4

4.171 171. bilinear_iff_square_eq_neg_one

The problem

For complex numbers a , b , and c satisfying $a + b = 2c$, prove that $ac + bc = -2$ if and only if $c^2 = -1$.

Lean statement

```
theorem bilinear_iff_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) : a * c + b * c = -2 ↔ c * c = -1
```

Parent. Given that $a + b = 2c$ and $c^2 = -1$ for complex numbers a , b , and c , show that $ac + bc = -2$.

Parent in Lean

```
theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
```

What it contributes. Reuses the parent's distributive factorization and sum substitution checkpoints, while strengthening the conclusion to an equivalence and explicitly deriving the square condition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way implication into a genuine iff: besides the parent direction, it must recover $c * c = -1$ by cancelling the nonzero scalar 2 from the bilinear equality. The shared factorization is useful, but the reverse algebraic characterization is new mathematics rather than a trivial wrapper.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: under the sum constraint, the bilinear value determines $c*c$ via invertibility of 2. The parent supplies only the forward implication from the square constraint, so memorizing its proof does not solve the iff.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reverse implication is a genuine new obligation: from the bilinear value it must recover $c * c = -1$ by using the factorization and cancellation by the nonzero scalar 2. The parent only proves the forward consequence and does not supply this characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1754 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bilinear_iff_square_eq_neg_one.extracted_1_1 (a b c : ℂ) (hsum : a + b = 2 * c) :  
  a * c + b * c = -2 ↔ c * c = -1
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c satisfying $a + b = 2c$, the equation $ac + bc = -2$ holds if and only if $c^2 = -1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bilinear_iff_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) : a * c + b * c = -2 ↔ c * c = -1 := by
  have hfactor : a * c + b * c = 2 * (c * c) := by
    calc
      a * c + b * c = (a + b) * c := by ring
      _ = (2 * c) * c := by rw [hsum]
      _ = 2 * (c * c) := by ring
  constructor
  · intro hbilinear
    calc
      c * c = (1 / 2 : ℂ) * (2 * (c * c)) := by ring
      _ = (1 / 2 : ℂ) * (a * c + b * c) := by rw [hfactor]
      _ = -1 := by rw [hbilinear]; norm_num
  · intro hsq
    calc
      a * c + b * c = 2 * (c * c) := hfactor
      _ = -2 := by rw [hsq]; norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. algebra_manipexpr_apbeq2cceqiacpbceqm2__me.me__dbede63d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ef375b1eca9c3081

4.172 172. bilinear_of_square_eq_neg_one

The problem

Given that $a + b = 2c$ and $c^2 = -1$ for complex numbers a , b , and c , show that $ac + bc = -2$.

Lean statement

```
theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
```

Parent. Given that $a + b = 2c$ and $c = \text{Im}(1)$, show that $ac + bc = -2$.

Parent in Lean

```
theorem algebra_manipexpr_apbeq2cceqiacpbceqm2 (a b c : ℂ) (h₀ : a + b = 2 * c)
(h₁ : c = Complex.I) : a * c + b * c = -2 := by
```

What it contributes. Preserves the parent sum relation and bilinear factoring checkpoint while replacing $c = \text{Complex.I}$ with the explicit square-root-of-negative hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the concrete hypothesis $c = \text{Complex.I}$ with the strictly more general algebraic condition $c * c = -1$. The bilinear rearrangement is shared, but the final simplification must use the new square-root condition directly rather than the parent's imaginary-unit lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the concrete witness $c = \text{Complex.I}$ with the algebraic condition $c * c = -1$, so the final step must use a new hypothesis rather than Complex.I_mul_I . The bilinear rearrangement is shared, but the parent's proof does not directly supply the generalized square-root argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the hypothesis from the specific value $c = \text{Complex.I}$ to the algebraic condition $c^2 = -1$, so the conclusion must be derived from a general square-root-of-negative-one property rather than the concrete imaginary-unit lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1740 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bilinear_of_square_eq_neg_one.extracted_1_1 (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) :
a * c + b * c = -2
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c , if $a + b = 2c$ and $c^2 = -1$, then $ac + bc = -2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
  have hbilinear : a * c + b * c = (a + b) * c := by
    ring
  have hsum_mul : (a + b) * c = (2 * c) * c := by
    rw [hsum]
  have hscaled : (2 * c) * c = 2 * (c * c) := by
    ring
  calc
    a * c + b * c = (a + b) * c := hbilinear
    _ = (2 * c) * c := hsum_mul
    _ = 2 * (c * c) := hscaled
    _ = -2 := by rw [hsq]; norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_manipexpr_apbeq2cceqiacpbceqm2__me__1fd261c9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f6782ae7e80f9e5b

4.173 173. bounded_subtractive_balance_quadratic_negative_between_roots

The problem

Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 < 0$ for every integer z strictly between $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_balance_quadratic_negative_between_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (p : ℤ) - 1 < z →
      z < (q : ℤ) - 1 →
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 < 0
```

Parent. Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 > 0$ for every integer $z > q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

What it contributes. Reuses the parent's integer balance conversion and factorization checkpoint, then changes the sign analysis from outside the larger root to opposite factor signs inside the roots.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from positivity outside the larger root to strict negativity between both roots, requiring a new opposite-sign argument and the lemma 'mul_neg_of_pos_of_neg'. Although it reuses the factorization derivation as instructed, the parent proof does not supply this interval-sign reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the domain from points right of the larger root to points strictly between both roots, requiring an opposite-sign argument and strict negativity via multiplication. The parent's positive-factor proof does not establish this conclusion by recall alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from positivity to strict negativity between the two roots, requiring a new sign argument using one positive and one negative factor and 'mul_neg_of_pos_of_neg'. Recalling the parent's proof does not establish this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2464 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_balance_quadratic_negative_between_roots.extracted_1.1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
  p * q - (p + q) = t → ∀ (z : ℤ), ↑p - 1 < z → z < ↑q - 1 → z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1 < 0
```

Read back by a model that saw only the Lean. For every natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $p < q$, and $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for every integer z satisfying $p - 1 < z < q - 1$, we have $z^2 - (p + q - 2)z + t + 1 < 0$, where the natural numbers p , q , and t are viewed as integers in this expression.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_balance_quadratic_negative_between_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (p : ℤ) - 1 < z →
      z < (q : ℤ) - 1 →
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 < 0 := by
  intro t p q hdom hbal z hpz hqz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, hpq⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hprod : (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) < 0 := by
    have hleft : 0 < z - ((p : ℤ) - 1) := by
      linarith
    have hright : z - ((q : ℤ) - 1) < 0 := by
      linarith
    exact mul_neg_of_pos_of_neg hleft hright
  calc
    z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 =
      (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by
        calc
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 =
            z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by
              nlinarith [hfactor]
          _ = (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by ring
    _ < 0 := hprod
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh.me__0bc05ceb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

gerprint 0bc05ceb87eb7b15

4.174 174. amc12_2000_p6_prime_products

The problem

Which natural numbers are products of two distinct primes between 4 and 18? Show that they are exactly 35, 55, 65, 77, 85, 91, 119, 143, 187, and 221.

Lean statement

```
theorem amc12_2000_p6_prime_products :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
      n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces five existence/nonexistence claims about $p*q-(p+q)$ with a full biconditional characterizing all products of two distinct bounded primes. Its forward finite enumeration and ten explicit reverse witnesses require reasoning not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from ruling out specific product-minus-sum values to a full biconditional characterization of all products of two distinct bounded primes. Its proof requires a two-variable finite enumeration and ten explicit witness constructions, which the parent's one-variable contradiction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new biconditional characterizing all products of two distinct primes in the interval, including a converse with ten explicit witnesses. The parent's contradiction from the single product-minus-sum equation does not supply this enumeration or converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2404 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_prime_products.extracted_1_1 : ∀ (n : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
    n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221
```

Read back by a model that saw only the Lean. *For every natural number n , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q equals n , if and only if n is one of 35, 55, 65, 77, 85, 91, 119, 143, 187, or 221.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_prime_products :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
      n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221 := by
  intro n
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
    interval_cases p <|> interval_cases q <|>
      simp_all (config := {decide := true})
  · intro h
    rcases h with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨13, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__2dc5d985

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2dc5d98568c75d91

4.175 175. bounded_prime_pair_power_residue

The problem

Let $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that there is a unique ordered pair of distinct primes $p < q$ in $[4, 18]$ satisfying $pq - (p + q) = 119$, and that for this pair $A^{6q+1} - B^{6q+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem bounded_prime_pair_power_residue (A B : Z)
  (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q : N,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧
    p ≠ q ∧ p < q ∧ p * q - (p + q) = 119 ∧
    (∀ r s : N,
      Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
      r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7]
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? **(A)** 22 **(B)** 60 **(C)** 119 **(D)** 180 **(E)** 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Supplied the bounded prime-pair domain and the explicit witness $(11,13)$ satisfying the value 119; its bounded-check style is reused for uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses the bounded prime-pair classification to fix $q=13$, then proves a genuinely new modular exponentiation claim: $A^{79} - B^{79} \equiv 3 \pmod{7}$ from the variable congruence hypotheses. Recalling the parent supplies the witness and interval classification but does not solve the residue argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing and uniquely classifying the ordered pair $(11,13)$, then using the larger coordinate to prove a separate modular exponentiation law from hypotheses on A and B . The parent's contradiction-and-interval proof does not supply either the uniqueness argument or the residue computation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving uniqueness of the bounded ordered prime pair and deriving a modular power-residue law from arbitrary congruence hypotheses. The parent only eliminates the first impossible candidate by bounded case analysis and supplies neither classification nor modular exponent reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2491 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_pair_power_residue.extracted_1_1 (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q,
    Nat.Prime p ∧
    Nat.Prime q ∧
    4 ≤ p ∧
    p ≤ 18 ∧
    4 ≤ q ∧
    q ≤ 18 ∧
    p ≠ q ∧
    p < q ∧
    p * q - (p + q) = 119 ∧
    (∀ (r s : ℕ),
      Nat.Prime r ∧
      Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧ r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers A and B , if A is congruent to 1 modulo 7 and B is congruent to 5 modulo 7, then there exist natural numbers p and q such that p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $p < q$, and $pq - (p + q) = 119$; moreover, these p and q are unique among all natural numbers r and s satisfying the same primality, bounds, distinctness, ordering, and equation conditions, in the sense that any such r and s satisfy $r = p$ and $s = q$; and $A^{(6q+1)} - B^{(6q+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_prime_pair_power_residue (A B : ℤ)
  (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧
    p ≠ q ∧ p < q ∧ p * q - (p + q) = 119 ∧
    (∀ r s : ℕ,
      Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
      r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7] := by
  have hpair :
    Nat.Prime 11 ∧ Nat.Prime 13 ∧ 4 ≤ 11 ∧ 11 ≤ 18 ∧ 4 ≤ 13 ∧ 13 ≤ 18 ∧
    11 ≠ 13 ∧ 11 < 13 ∧ 11 * 13 - (11 + 13) = 119 := by
```

```

norm_num
have huniq : ∀ r s : ℕ,
  Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
  r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
  r = 11 ∧ s = 13 := by
  intro r s h
  rcases h with ⟨hr, hs, hr4, hr18, hs4, hs18, hrs, hlt, heq⟩
  interval_cases r <=> interval_cases s <=> norm_num at *
  refine ⟨11, 13, hpair.1, hpair.2.1, hpair.2.2.1, hpair.2.2.2.1,
    hpair.2.2.2.2.1, hpair.2.2.2.2.2.1, hpair.2.2.2.2.2.2.1,
    hpair.2.2.2.2.2.2.2.1, hpair.2.2.2.2.2.2.2.2, ?_, ?_⟩
  · intro r s hs
    exact huniq r s hs
  · change A ^ 79 - B ^ 79 ≡ 3 [ZMOD 7]
    have hdiff : A ^ 79 - B ^ 79 ≡ (1 : ℤ) ^ 79 - 5 ^ 79 [ZMOD 7] := by
      exact (hA.pow 79).sub (hB.pow 79)
    have hnum : (1 : ℤ) ^ 79 - 5 ^ 79 ≡ 3 [ZMOD 7] := by
      norm_num [Int.ModEq]
    exact hdiff.trans hnum

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__4312632b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4312632b71c131d7

4.176 176. bounded_prime_product_minus_sum_characterization

The problem

For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p-1)(q-1) = t+1$ for such primes.

Lean statement

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? **(A)** 22 **(B)** 60 **(C)** 119 **(D)** 180 **(E)** 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Retains the parent's bounded distinct-prime domain and product-minus-sum expression, while replacing its fixed target list with a parameterized factorization characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes five finite attainability checks into an iff characterization for every t , requiring symbolic handling of natural-number subtraction and the factorization identity $(p-1)(q-1)=pq-p-q+1$. The parent's bounded interval enumeration cannot supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child parameterizes the target value and proves a two-way factorization equivalence, requiring algebraic manipulation in both directions rather than recalling the parent's finite contradiction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an arbitrary parameter and proves a genuine two-way algebraic characterization via $(p-1)(q-1) = t+1$. The parent's finite contradiction and interval case split do not supply this general equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2401 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_minus_sum_characterization.extracted_1.1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = t) ↔
  ∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ (p - 1) * (q - 1) = t + 1
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $p \cdot q - (p + q) = t$, if and only if there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $(p - 1)(q - 1) = t + 1$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1) := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, ht⟩
    have hle : p + q ≤ p * q := by
      nlinarith
    have ht_z : ((p * q - (p + q) : ℕ) : ℤ) = (t : ℤ) := by
      exact_mod_cast ht
    rw [Nat.cast_sub hle] at ht_z
    norm_num [Nat.cast_mul, Nat.cast_add] at ht_z
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
        Nat.cast_sub (by omega : 1 ≤ q)]
      nlinarith [ht_z]
    have hfactor : (p - 1) * (q - 1) = t + 1 := by
      exact_mod_cast hfac_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
    have hle : p + q ≤ p * q := by
      nlinarith
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      exact_mod_cast hfactor
    norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
      Nat.cast_sub (by omega : 1 ≤ q)] at hfac_z
    have ht_z : ((p * q - (p + q) : ℕ) : ℤ) = (t : ℤ) := by
      rw [Nat.cast_sub hle]
      norm_num [Nat.cast_mul, Nat.cast_add]
      nlinarith [hfac_z]
    have hprod : p * q - (p + q) = t := by
      exact_mod_cast ht_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hprod⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__c3538557

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c35385572f20d6e9

4.177 177. arbitrary_parameter_shifted_roots

The problem

Let p and q be distinct natural numbers with $p, q \geq 2$, and let $t = pq - (p + q)$. Show that $(p - 1)(q - 1) = t + 1$, and that the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$ are exactly p and q .

Lean statement

```
theorem arbitrary_parameter_shifted_roots :
  ∀ p q t : ℕ,
    2 ≤ p → 2 ≤ q → p ≠ q →
    p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Parent. Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Parent in Lean

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

What it contributes. Generalized the parent's prime-and-bounded setup to arbitrary natural parameters at least 2, while retaining its algebraic factorization and integer-root checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes the primality and finite bounds, requiring a new general argument that $p, q \geq 2$ implies $p + q \leq pq$; the resulting algebraic identity and exact integer-root characterization then hold for arbitrary distinct natural parameters. This is the planned bounded generalization and is not recoverable by simply recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from bounded distinct primes to arbitrary distinct naturals at least 2. Its proof replaces finite interval checking with the symbolic inequality $(p-2)(q-2) \geq 0$, so recalling the parent does not supply the needed argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from bounded distinct primes to arbitrary distinct natural parameters. The parent's finite interval-case argument no longer applies; the child requires a general nonnegativity argument and symbolic algebra to establish the factorization and root characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2543 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem arbitrary_parameter_shifted_roots.extracted_1_1 : ∀ (p q t : ℕ),
  2 ≤ p →
  2 ≤ q →
  p ≠ q →
  p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧ ∀ (x : ℤ), x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = 0 ↔ x = ↑p ∨ x = ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p , q , and t , if $2 \leq p$, $2 \leq q$, $p \neq q$, and $p \cdot q - (p + q) = t$, then $(p - 1)(q - 1) = t + 1$, and for every integer x , $x^2 - (p + q)x + (t + p + q) = 0$ if and only if $x = p$ or $x = q$, where p , q , and t are regarded as integers in the latter equation.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem arbitrary_parameter_shifted_roots :
  ∀ p q t : ℕ,
    2 ≤ p → 2 ≤ q → p ≠ q →
    p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
  intro p q t hp hq hne hrel
  have hp_repr : p = (p - 1) + 1 := by
    omega
  have hq_repr : q = (q - 1) + 1 := by
    omega
  have hsum_le : p + q ≤ p * q := by
    have hnonneg : 0 ≤ (p - 2) * (q - 2) := Nat.zero_le _
    nlinarith
  have hprod : p * q = t + (p + q) := by
    exact (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hfac : (p - 1) * (q - 1) = t + 1 := by
    nlinarith [hprod, hp_repr, hq_repr]
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q : ℕ) : ℤ) := by
      exact_mod_cast hprod
    simpa [Nat.cast_add, add_assoc] using hprodZ
  have hpoly : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
    (x - p) * (x - q) := by
    intro x
    nlinarith [hrelZ]
  have hroots : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
    x = p ∨ x = q := by
    intro x
    constructor
    · intro hx
```

```

have hzero : (x - p) * (x - q) = 0 := by
  rw [← hpoly x]
  exact hx
rcases mul_eq_zero.mp hzero with hzero | hzero
· left
  omega
· right
  omega
· intro hx
  rcases hx with rfl | rfl
  · simp [hpoly]
  · simp [hpoly]
exact ⟨hfacs, hroots⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.me__11c1e1ee

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 11c1e1eef1d5f3e9

4.178 178. bounded_threshold_mutation

The problem

Let a sequence begin with 4, 7, and let each later term be the units digit of the sum of the two preceding terms. If S_n is the sum of the first n terms, show that 2 is the least n such that $S_n > 10$ and $n \leq 3$.

Lean statement

```
theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sₙ 2
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10000}) :
  IsLeast Sₙ 1999 := by
```

What it contributes. Used the recurrence and finite partial-sum derivation pattern, adapting the threshold and replacing the unbounded minimum with a bounded IsLeast classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a valid bounded least-element characterization with a different threshold and an essential finite case analysis for indices below 2. Its proof derives $u\ 2$ and the partial sums explicitly, then excludes earlier candidates, so recalling the parent's 1999-threshold monotonicity argument does not solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded-threshold characterization: it computes the new local values, establishes membership at 2, and excludes earlier indices by finite case analysis. The parent's large-sum monotonicity argument and IsLeast result do not provide this proof.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2713 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_threshold_mutation.extracted_1_1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ k ∈ Finset.range n, u k) (Sn : Set ℕ)
  (h₄ : Sn = {n | S n > 10 ∧ n ≤ 3}) : IsLeast Sn 2
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers. Assume $u(0)=4$, $u(1)=7$, and for every natural number n with $n \geq 2$, $u(n)=(u(n-2)+u(n-1))$ modulo 10. Assume also that for every natural number n , $S(n)=\sum_{k \in \{0,1,\dots,n-1\}} u(k)$. Let S_n be a set of natural numbers, and assume $S_n=\{n \in \mathbb{N} \mid S(n) > 10 \text{ and } n \leq 3\}$. Then 2 is the least element of S_n ; that is, $2 \in S_n$ and every element of S_n is greater than or equal to 2.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sn : Set ℕ)
  (h₄ : Sn = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sn 2 := by
  have hu2 : u 2 = 1 := by
    have h := h₂ 2 (by omega)
    norm_num [h₀, h₁] at h ⊢
    exact h
  have hS0 : S 0 = 0 := by
    rw [h₃]
    simp
  have hS1 : S 1 = 4 := by
    rw [h₃]
    simp [h₀]
  have hS2 : S 2 = 11 := by
    rw [h₃]
    norm_num [Finset.sum_range_succ, h₀, h₁, hu2]
  constructor
  · rw [h₄]
    change S 2 > 10 ∧ 2 ≤ 3
    constructor <|> omega
  · intro n hn
    rw [h₄] at hn
    change S n > 10 ∧ n ≤ 3 at hn
    rcases hn with ⟨hSn, hn3⟩
    interval_cases n <|> simp_all [hS0, hS1, hS2]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__me__a407401f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a407401f0770c6be

4.179 179. least_threshold_residue_from_period_block

The problem

Let a sequence have period p and block sum b . If every partial sum before $kp + r$ is at most T , the sum at $kp + r$ exceeds T , and that sum equals b^{2q+1} for some $b \equiv 2 \pmod{3}$, prove that $kp + r$ is the least index whose partial sum exceeds T and is congruent to 2 modulo 3.

Lean statement

```
theorem least_threshold_residue_from_period_block
  (u : ℕ → ℕ)
  (p b T k r q : ℕ)
  (hp : 0 < p)

  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1))
  (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r)
```

Parent. Let a sequence satisfy the same units-digit recurrence, and suppose it has period p with one-period sum b . If every partial block sum before $kp + r$ is at most T , while the sum at $kp + r$ exceeds T , prove that $kp + r$ is the least index whose partial sum exceeds T .

Parent in Lean

```
theorem least_threshold_from_period_block
  (u : ℕ → ℕ)

  (p b T k r : ℕ)
  (hp : 0 < p)
  (hperiod : ∀ n, u (n + p) = u n)
  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
  IsLeast {n | S n > T} (k * p + r) := by
```

What it contributes. Supplied the period-block decomposition, candidate threshold checkpoint, quotient/remainder decomposition, and earlier-index bound used to establish leastness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new residue argument: it must prove that an odd power of a number congruent to 2 modulo 3 is itself congruent to 2 modulo 3, then establish leastness in the intersected threshold-and-residue set. The parent supplies only threshold leastness and cannot certify the new conjunct.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to leastness within a residue-refined threshold set and must prove the candidate's residue via an independent odd-power modular induction. This reasoning is absent from the parent's proof, while the threshold-minimality portion is appropriately reused.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to leastness within a residue-refined threshold set and must prove the candidate's residue from an odd-power congruence via a new induction. Although the threshold-minimality argument is reused, the parent cannot establish membership in the refined set.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem least_threshold_residue_from_period_block.extracted_1_1 (u : ℕ → ℕ) (p b T k r q : ℕ) (hp : 0 < p)
  (hperiod : ∀ (n : ℕ), u (n + p) = u n) (hblock : ∀ (j s : ℕ), S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ (j s : ℕ), j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1)) (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r)
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers, and let p, b, T, k, r , and q be natural numbers. Assume that p is positive; for every natural number n , $u(n + p) = u(n)$; for all natural numbers j and s , $S(jp + s) = jb + \sum_{i \in \text{Finset.range}(s)} u(i)$; $kb + \sum_{i \in \text{Finset.range}(r)} u(i) > T$; for all natural numbers j and s , if $jp + s < kp + r$, then $jb + \sum_{i \in \text{Finset.range}(s)} u(i) \leq T$; $S(kp + r) = b^{2q+1}$; and $b \equiv 2 \pmod{3}$. Then $kp + r$ is the least natural number n such that $S(n) > T$ and $S(n) \equiv 2 \pmod{3}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem least_threshold_residue_from_period_block
  (u : ℕ → ℕ)
  (p b T k r q : ℕ)
  (hp : 0 < p)

  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1))
  (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r) := by
  have hodd : ∀ m : ℕ, b ^ (2 * m + 1) % 3 = 2 := by
    intro m
    induction m with
    | zero => simp [hb]
    | succ m ih =>
      rw [show 2 * Nat.succ m + 1 = (2 * m + 1) + 2 by omega, pow_add]
      have hb2 : b ^ 2 % 3 = 1 := by
        simp [pow_two, Nat.mul_mod, hb]
      simp [Nat.mul_mod, ih, hb2]
  have hthreshold : S (k * p + r) > T := by
```

```

    rw [hblock]
    exact hcross
have hresidue : S (k * p + r) % 3 = 2 := by
  rw [hbridge]
  exact hodd q
constructor
· exact (hthreshold, hresidue)
· intro n hn
  change S n > T ∧ S n % 3 = 2 at hn
  by_contra hnot
  have hnlt : n < k * p + r := Nat.lt_of_not_ge hnot
  let j := n / p
  let s := n % p
  have hslt : s < p := Nat.mod_lt n hp
  have hdecomp : j * p + s = n := by
    dsimp [j, s]
    simpa [Nat.mul_comm] using Nat.div_add_mod n p
  have harg : j * p + s < k * p + r := by
    simpa [hdecomp] using hnlt
  have hsum : j * b + ∑ i ∈ Finset.range s, u i ≤ T := hbound j s harg
  have hSn : S n = j * b + ∑ i ∈ Finset.range s, u i := by
    rw [← hdecomp]
    exact hblock j s
  have hle : S n ≤ T := by
    rw [hSn]
    exact hsum
  omega

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh.me__0c83ce1f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5bb41ad5ca1bc9d3

4.180 180. bounded_recurrence_window_unique

The problem

Let N be a nonnegative integer, let p and q be real numbers, and let a and b be real sequences such that, for every nonnegative integer n , $a_{n+2} - a_{n+1} = a_{n+1} - a_n$ and $b_{n+2} - b_{n+1} = b_{n+1} - b_n$. If $a_1 = b_1 = p$ and $a_2 = b_2 = 9$, prove that $a_n = b_n$ for every integer n with $1 \leq n \leq N$.

Lean statement

```
theorem bounded_recurrence_window_unique (N : ℕ) (p q : ℝ) (a b : ℕ → ℝ)
  (ha : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (hb : ∀ n, b (n + 2) - b (n + 1) = b (n + 1) - b n)
  (h1 : a 1 = p) (h2 : a 2 = 9)

  (hb1 : b 1 = p) (hb2 : b 2 = 9) :
  ∀ n, 1 ≤ n → n ≤ N → a n = b n
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the 2010th term of this sequence? Show that it is 8041.

Parent in Lean

```
theorem amc12a_2010_p10 (p q : ℝ) (a : ℕ → ℝ) (h0 : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h1 : a 1 = p) (h2 : a 2 = 9) (h3 : a 3 = 3 * p - q) (h4 : a 4 = 3 * p + q) : a 2010 = 8041 := by
```

What it contributes. Retains the parent recurrence, domain, fixed initial values, and auxiliary evaluations while replacing the fixed endpoint conclusion with bounded recurrence uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different uniqueness theorem: two sequences satisfying the same second-difference recurrence and matching at indices 1 and 2 agree throughout any bounded window. Its strong-induction argument is independent of the parent's parameter-solving and endpoint evaluation, and the extra hypotheses on a are unused rather than carrying over the parent's proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine uniqueness theorem for two recurrence sequences via strong induction, using matching initial values and recurrence propagation. This is a different proof obligation and route from the parent's fixed-endpoint arithmetic-progression evaluation, so recalling the parent does not substantially solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine two-sequence uniqueness theorem by strong induction from matching initial values and equal second differences. This reasoning does not rely on the parent's derivation of $p = 5$ or its fixed endpoint evaluation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed h_3 , h_4

Check	By	Result
corpus_dedup	hash	unique against 1694 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_recurrence_window_unique.extracted_1_1 : ∀ (N : ℕ) (p : ℝ) (a b : ℕ → ℝ),
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →
  (∀ (n : ℕ), b (n + 2) - b (n + 1) = b (n + 1) - b n) →
  a 1 = p → a 2 = 9 → b 1 = p → b 2 = 9 → ∀ (n : ℕ), 1 ≤ n → n ≤ N → a n = b n
```

Read back by a model that saw only the Lean. For every natural number N , every real number p , and every pair of functions $a, b: \mathbb{N} \rightarrow \mathbb{R}$, if for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$, and for every natural number n , $b(n+2) - b(n+1) = b(n+1) - b(n)$, and $a(1) = p$, $a(2) = 9$, $b(1) = p$, and $b(2) = 9$, then for every natural number n such that $1 \leq n$ and $n \leq N$, we have $a(n) = b(n)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_recurrence_window_unique (N : ℕ) (p q : ℝ) (a b : ℕ → ℝ)
  (h₀a : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₀b : ∀ n, b (n + 2) - b (n + 1) = b (n + 1) - b n)
  (h₁ : a 1 = p) (h₂ : a 2 = 9)

  (hb₁ : b 1 = p) (hb₂ : b 2 = 9) :
  ∀ n, 1 ≤ n → n ≤ N → a n = b n := by
  intro n hn₁ hnN
  revert hn₁ hnN
  induction n using Nat.strong_induction_on with
  | h n ih =>
    intro hn₁ hnN
    by_cases h_one : n = 1
    · subst n
      exact h₁.trans hb₁.symm
    by_cases h_two : n = 2
    · subst n
      exact h₂.trans hb₂.symm
    have hn₃ : 3 ≤ n := by omega
    have hprev₁ : a (n - 1) = b (n - 1) := by
      apply ih (n - 1)
      · omega
      · omega
      · omega
    have hprev₂ : a (n - 2) = b (n - 2) := by
      apply ih (n - 2)
      · omega
      · omega
      · omega
    have ha := h₀a (n - 2)
    have hb := h₀b (n - 2)
    have hidx₁ : n - 2 + 2 = n := by omega
    have hidx₂ : n - 2 + 1 = n - 1 := by omega
    rw [hidx₁, hidx₂] at ha hb
    linarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2010_p10__me__8f91749d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7045cbe3a1ce5b8d

4.181 181. bounded_interval_factorization

The problem

For integers $0 \leq m \leq n$, show that $\prod_{k=m}^n (2^{2^k} + 3^{2^k}) = 3^{2^{n+1}} - 2^{2^{n+1}}$.

Lean statement

```
theorem bounded_interval_factorization :
  ∀ m n : ℕ, m ≤ n →
    (∏ k ∈ Finset.range (n + 1),
      if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
      ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
```

Parent. For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Generalizes the parent's initial-segment telescoping induction to a bounded lower-index interval, retaining its exponent-doubling and ring-factorization checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the lower endpoint and proves a bounded interval factorization, requiring a new induction with a boundary case for $m = n + 1$. The parent's initial-range telescoping proof does not directly supply this interval handling, and no hypotheses are redundant.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the lower endpoint and proves a bounded-interval factorization, requiring a new induction split on whether the lower bound has entered the range. The parent's initial-range telescoping proof does not supply this shifted-boundary reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely shifts the product start from 0 to an arbitrary m and must handle the new boundary case $m = n + 1$, where the product is empty. The parent's induction only establishes the initial-range identity and does not supply this interval factorization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2412 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_interval_factorization.extracted_1_1 : ∀ (m n : ℕ),
  m ≤ n →
  (∏ k ∈ Finset.range (n + 1), if m ≤ k then 2 ^ 2 ^ k + 3 ^ 2 ^ k else 1) * (3 ^ 2 ^ m - 2 ^ 2 ^ m) =
  3 ^ 2 ^ (n + 1) - 2 ^ 2 ^ (n + 1)
```

Read back by a model that saw only the Lean. For all natural numbers m and n , if $m \leq n$, then the product over all natural numbers k in the range from 0 through n of the factor $2^{(2^k)} + 3^{(2^k)}$ when $m \leq k$ and the factor 1 otherwise, multiplied by $3^{(2^m)} - 2^{(2^m)}$, equals $3^{(2^{(n+1)})} - 2^{(2^{(n+1)})}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem bounded_interval_factorization :
  ∀ m n : ℕ, m ≤ n →
    (∏ k ∈ Finset.range (n + 1),
      if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
      ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
    intro m n hmn
    induction n generalizing m with
    | zero =>
      have hm : m = 0 := Nat.eq_zero_of_le_zero hmn
      subst m
      norm_num
    | succ n ih =>
      by_cases hmn' : m ≤ n
      · rw [Finset.prod_range_succ]
        rw [if_pos (Nat.le_succ_of_le hmn')]
        have hi := ih m hmn'
        have hpow : (2 : ℕ) ^ (n + 1 + 1) = (2 : ℕ) ^ (n + 1) * 2 := by
          rw [pow_succ]
        have h3 : (3 : ℤ) ^ (2 ^ (n + 1 + 1)) = ((3 : ℤ) ^ (2 ^ (n + 1))) ^ 2 := by
          rw [hpow, pow_mul]
        have h2 : (2 : ℤ) ^ (2 ^ (n + 1 + 1)) = ((2 : ℤ) ^ (2 ^ (n + 1))) ^ 2 := by
          rw [hpow, pow_mul]
        calc
          ((∏ k ∈ Finset.range (n + 1),
            if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1)))) *
            ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
          ((∏ k ∈ Finset.range (n + 1),
            if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
            ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m))) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1))) := by ring
          = ((3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1))) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1))) := by rw [hi]
          = (3 : ℤ) ^ (2 ^ (n + 1 + 1)) - (2 : ℤ) ^ (2 ^ (n + 1 + 1)) := by
            rw [h3, h2]
            ring
          = (3 : ℤ) ^ (2 ^ (Nat.succ n + 1)) - (2 : ℤ) ^ (2 ^ (Nat.succ n + 1)) := by
            simp [Nat.succ_eq_add_one, Nat.add_assoc]
      · have hm : m = n + 1 := by omega
        subst m
        rw [Finset.prod_range_succ]
        have hprod :
```

```

      (∏ k ∈ Finset.range (n + 1),
        if n + 1 ≤ k then ((2 : Z) ^ (2 ^ k) + (3 : Z) ^ (2 ^ k)) else 1) = 1 := by
    apply Finset.prod_eq_one
    intro k hk
    have hk' : k ≤ n := by simpa using (Finset.mem_range.mp hk)
    simp [show ¬ n + 1 ≤ k by omega]
    rw [hprod]
    simp only [if_pos (le_refl (n + 1)), one_mul]
    have hpow : (2 : N) ^ (n + 1 + 1) = (2 : N) ^ (n + 1) * 2 := by
      rw [pow_succ]
    have h3 : (3 : Z) ^ (2 ^ (n + 1 + 1)) = ((3 : Z) ^ (2 ^ (n + 1))) ^ 2 := by
      rw [hpow, pow_mul]
    have h2 : (2 : Z) ^ (2 ^ (n + 1 + 1)) = ((2 : Z) ^ (2 ^ (n + 1))) ^ 2 := by
      rw [hpow, pow_mul]
    rw [h3, h2]
    ring

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__2706e7a7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2706e7a74c656432

4.182 182. dyadic_product_difference_scaled_bounded

The problem

For all natural numbers m and n with $n \leq 8$, prove $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^m - 2^m$.

Lean statement

```
theorem dyadic_product_difference_scaled_bounded :
  ∀ m n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
      ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      (3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n)
```

Parent. For every integer n with $0 \leq n \leq 8$, prove that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Supplied the bounded induction structure, product-range expansion, and exponent-doubling check-point; the child introduces the scale parameter m and the necessary factor $3^m - 2^m$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the exponent scale with an essential natural number m and carries the factor $(3^m - 2^m)$ through the induction. Its successor step requires transporting $m \cdot 2^{(n+1)}$ to $(m \cdot 2^n) \cdot 2$ and applying the difference-of-squares argument, so recalling the parent does not directly solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the identity to every natural scale m and introduces the factor $(3^m - 2^m)$, so the parent's fixed-scale theorem cannot solve it. The induction must preserve this symbolic scale through the exponent-doubling factorization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the exponent scale by an arbitrary natural number m . The induction and difference-of-squares argument must now establish symbolic exponent identities involving m , so recalling the parent's fixed-scale theorem does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2627 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_product_difference_scaled_bounded.extracted_1_1 : ∀ (m n : ℕ),
  n ≤ 8 →
  (∏ k ∈ Finset.range n, (2 ^ (m * 2 ^ k) + 3 ^ (m * 2 ^ k))) * (3 ^ m - 2 ^ m) =
  3 ^ (m * 2 ^ n) - 2 ^ (m * 2 ^ n)
```

Read back by a model that saw only the Lean. *For all natural numbers m and n , if n is at most 8, then the product over all natural numbers k less than n of (2 raised to the power $m \cdot 2^k$ plus 3 raised to the power $m \cdot 2^k$), multiplied by ($3^m - 2^m$), equals 3 raised to the power $m \cdot 2^n$ minus 2 raised to the power $m \cdot 2^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_product_difference_scaled_bounded :
  ∀ m n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
      ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      (3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n) := by
  intro m n hn
  induction n with
  | zero =>
    simp
  | succ n ih =>
    have ih' := ih (by omega)
    have h3 :
      (3 : ℤ) ^ (m * 2 ^ (Nat.succ n)) =
      ((3 : ℤ) ^ (m * 2 ^ n)) ^ 2 := by
      rw [show m * 2 ^ (Nat.succ n) = (m * 2 ^ n) * 2 by
        simp [pow_succ, Nat.mul_assoc, Nat.mul_comm, Nat.mul_left_comm]]
      rw [pow_mul]
    have h2 :
      (2 : ℤ) ^ (m * 2 ^ (Nat.succ n)) =
      ((2 : ℤ) ^ (m * 2 ^ n)) ^ 2 := by
      rw [show m * 2 ^ (Nat.succ n) = (m * 2 ^ n) * 2 by
        simp [pow_succ, Nat.mul_assoc, Nat.mul_comm, Nat.mul_left_comm]]
      rw [pow_mul]
    rw [Finset.prod_range_succ]
    calc
      ((∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n))) *
        ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      ((∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
        ((3 : ℤ) ^ m - (2 : ℤ) ^ m)) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n)) := by ring
      = ((3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n)) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n)) := by rw [ih']
      = (3 : ℤ) ^ (m * 2 ^ (Nat.succ n)) -
        (2 : ℤ) ^ (m * 2 ^ (Nat.succ n)) := by
        rw [h3, h2]
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me.me__820eb7ea

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 820eb7eabf23c1db

4.183 183. parameterized_dyadic_difference_of_squares

The problem

For integers x, y and every nonnegative integer n , prove $\prod_{k=0}^{n-1} (x^{2^k} + y^{2^k}) (x-y) = x^{2^n} - y^{2^n}$.

Lean statement

```
theorem parameterized_dyadic_difference_of_squares :
  ∀ (x y : ℤ) (n : ℕ),
    (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y) =
      x ^ (2 ^ n) - y ^ (2 ^ n) := by
```

Parent. For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Generalizes the parent's dyadic finite-product induction and exponent-doubling checkpoints from fixed bases 2 and 3 to arbitrary integer bases, adding the necessary initial-difference factor.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes both bases and makes the initial difference factor explicit, requiring a genuinely symbolic difference-of-squares induction rather than replaying the fixed-base proof. Its proof must establish the generalized algebraic invariant for arbitrary integers x and y .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both fixed bases and makes the initial difference factor explicit, proving a universal difference-of-squares product law over arbitrary integers. Although the induction structure is shared, the parent's fixed-coefficient result does not itself establish the symbolic two-variable identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both bases and makes the initial difference factor explicit, requiring the general difference-of-squares identity for arbitrary integers rather than the parent's fixed coefficients. Although the induction skeleton is shared, the parent does not prove the child by specialization or recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2445 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_dyadic_difference_of_squares.extracted_1_1 : ∀ (x y : ℤ) (n : ℕ),
  (∏ k ∈ Finset.range n, (x ^ 2 ^ k + y ^ 2 ^ k)) * (x - y) = x ^ 2 ^ n - y ^ 2 ^ n
```

Read back by a model that saw only the Lean. *For all integers x and y and every natural number n , the product over $k = 0, 1, \dots, n - 1$ of $(x^{2^k} + y^{2^k})$, multiplied by $(x - y)$, equals $x^{2^n} - y^{2^n}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_dyadic_difference_of_squares :
  ∀ (x y : ℤ) (n : ℕ),
    (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y) =
      x ^ (2 ^ n) - y ^ (2 ^ n) := by
  intro x y n
  induction n with
  | zero =>
    norm_num
  | succ n ih =>
    rw [Finset.prod_range_succ]
    have hpow : (2 : ℕ) ^ (n + 1) = (2 : ℕ) ^ n * 2 := by
      simp [pow_succ]
    have hx : x ^ (2 ^ (n + 1)) = (x ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    have hy : y ^ (2 ^ (n + 1)) = (y ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    calc
      (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) *
        (x ^ (2 ^ n) + y ^ (2 ^ n)) * (x - y) =
        ((∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y)) *
          (x ^ (2 ^ n) + y ^ (2 ^ n)) := by ring
      = (x ^ (2 ^ n) - y ^ (2 ^ n)) *
        (x ^ (2 ^ n) + y ^ (2 ^ n)) := by rw [ih]
      = x ^ (2 ^ (n + 1)) - y ^ (2 ^ (n + 1)) := by
        rw [hx, hy]
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__e421c102

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e421c1028fbd8f18

4.184 184. dyadic_subinterval_endpoint_factorization

The problem

For natural numbers $a \leq b$, prove that $\left[\left(\prod_{k=a}^b \left(2^{2^k} + 3^{2^k} \right) \right) \left(3^{2^a} - 2^{2^a} \right) \right] = 3^{2^{b+1}} - 2^{2^{b+1}}$.

Lean statement

```
theorem dyadic_subinterval_endpoint_factorization :
  ∀ a b : ℕ, a ≤ b →
    (∏ k ∈ Finset.Icc a b,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
      ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
      (3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1))
```

Parent. For every integer n with $0 \leq n \leq 8$, prove that $\left[\prod_{k=1}^n \left(2^{2^{k-1}} + 3^{2^{k-1}} \right) \right] \left(3^{2^n} - 2^{2^n} \right)$.

Parent in Lean

```
theorem dyadic_product_difference_interval :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.Icc 1 n,
      ((2 : ℤ) ^ (2 ^ (k - 1)) + (3 : ℤ) ^ (2 ^ (k - 1)))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)
```

What it contributes. Supplied the dyadic product structure and exponent-doubling factorization checkpoint; the child generalizes it to an arbitrary lower endpoint and consumes that endpoint factor in the induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from the fixed interval starting at 1 to every nonempty bounded interval $[a, b]$. Its proof requires a new induction over the upper endpoint and an endpoint factorization argument; the parent's finite interval_cases proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the identity to arbitrary nonempty bounded subintervals and proves it by induction, using a symbolic endpoint factorization absent from the parent's bounded finite check. Recalling the parent's proof does not establish the child's unrestricted interval law.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the parent's bounded initial interval to arbitrary nonempty subintervals and requires induction plus endpoint factorization. Its proof does not reuse the parent's finite case split or conclusion as a recalled lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2636 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_subinterval_endpoint_factorization.extracted_1_1 : ∀ (a b : ℕ),
  a ≤ b →
  (∏ k ∈ Finset.Icc a b, (2 ^ 2 ^ k + 3 ^ 2 ^ k)) * (3 ^ 2 ^ a - 2 ^ 2 ^ a) =
  3 ^ 2 ^ (b + 1) - 2 ^ 2 ^ (b + 1)
```

Read back by a model that saw only the Lean. *For all natural numbers a and b with $a \leq b$, the product over all natural numbers k in the interval $[a, b]$ of $(2^{2^k} + 3^{2^k})$, multiplied by $(3^{2^a} - 2^{2^a})$, equals $3^{2^{b+1}} - 2^{2^{b+1}}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_subinterval_endpoint_factorization :
  ∀ a b : ℕ, a ≤ b →
    (∏ k ∈ Finset.Icc a b,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
      ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
      (3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1)) := by
  have hfac : ∀ n : ℕ,
    ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) *
      ((3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
    intro n
    rw [show 2 ^ (n + 1) = 2 ^ n + 2 ^ n by
      rw [pow_succ]
      ring]
    rw [pow_add, pow_add]
    ring
  intro a b hab
  induction b generalizing a with
  | zero =>
    have ha : a = 0 := by omega
    subst a
    norm_num
  | succ b ih =>
    by_cases h : a ≤ b
    · have hprev := ih a h
      have hprod :
        (∏ k ∈ Finset.Icc a (b + 1),
          ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
          (∏ k ∈ Finset.Icc a b,
            ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
            ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1)))) := by
          rw [Finset.prod_Icc_succ_top (by omega)]
        calc
          (∏ k ∈ Finset.Icc a (b + 1),
            ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
            ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
            ((∏ k ∈ Finset.Icc a b,
              ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
              ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1)))) *
              ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) := by rw [hprod]
            = ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1))) *
              ((∏ k ∈ Finset.Icc a b,
                ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
                ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a))) := by ring
```

```

    _ = ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1))) *
      ((3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1))) := by rw [hprev]
    _ = (3 : ℤ) ^ (2 ^ (b + 1 + 1)) - (2 : ℤ) ^ (2 ^ (b + 1 + 1)) := by simpa [Nat.add_assoc] using hfac (b + 1)
  · have ha : a = b + 1 := by omega
    subst a
    simpa using hfac (b + 1)

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me.ms.me__a0719f65

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a0719f6522070116

4.185 185. dyadic_product_difference_bounded

The problem

For every integer n with $0 \leq n \leq 8$, prove that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Lean statement

```
theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128} - 2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Retains the parent's dyadic product and difference-of-powers structure while promoting the fixed seven-stage identity to a bounded parameterized law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the fixed seven-factor computation with a parameterized product identity and proves it by induction, using the factorization step uniformly across stages. Memorizing the parent's norm_num proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uniformly parameterizes the product length and proves the product-to-difference-of-powers identity by induction, using the doubling exponent law and an algebraic factorization. This reasoning is not supplied by the parent's fixed seven-term 'norm_num' computation, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed seven-factor identity to a variable stage and requires induction plus symbolic power and ring reasoning. The parent's direct normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2595 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_product_difference_bounded.extracted_1_1 : ∀ n ≤ 8,
  ∏ k ∈ Finset.range n, (2 ^ 2 ^ k + 3 ^ 2 ^ k) = 3 ^ 2 ^ n - 2 ^ 2 ^ n
```

Read back by a model that saw only the Lean. *For every natural number n with $n \leq 8$, the product over all natural numbers k in the range $0 \leq k < n$ of $(2^{2^k} + 3^{2^k})$ equals $3^{2^n} - 2^{2^n}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
  intro n hn
  induction n with
  | zero =>
    simp
  | succ n ih =>
    have ih' := ih (by omega)
    rw [Finset.prod_range_succ, ih']
    have h3 :
      (3 : ℤ) ^ (2 ^ (n + 1)) = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by
      calc
        (3 : ℤ) ^ (2 ^ (n + 1)) = (3 : ℤ) ^ (2 ^ n * 2) := by
          rw [show 2 ^ (n + 1) = 2 ^ n * 2 by simp [pow_succ]]
        _ = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by rw [pow_mul]
    have h2 :
      (2 : ℤ) ^ (2 ^ (n + 1)) = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by
      calc
        (2 : ℤ) ^ (2 ^ (n + 1)) = (2 : ℤ) ^ (2 ^ n * 2) := by
          rw [show 2 ^ (n + 1) = 2 ^ n * 2 by simp [pow_succ]]
        _ = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by rw [pow_mul]
    rw [h3, h2]
  ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__6082dd30

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6082dd3056b8dc69

4.186 186. power_sum_product_telescopes

The problem

For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Lean statement

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128} - 2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k : ℕ) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Generalizes the parent's seven-factor power-difference identity to an arbitrary bounded product length while retaining its telescoping algebra.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed seven-factor identity to every natural index and changes the codomain to integers. Its induction and algebraic telescoping argument cannot be obtained by recalling the parent's finite 'norm_num' proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor computation with a genuine induction over arbitrary n , requiring the telescoping factorization at each successor step. This proof cannot be obtained by recalling the parent's norm_num computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor natural-number identity with a universal integer-valued telescoping theorem. Its induction and algebraic power-doubling argument are not supplied by the parent's one-shot norm_num proof, so recalling the parent does not substantially solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2388 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_sum_product_teslescopes.extracted_1_1 : ∀ (n : ℕ),
  ∏ k ∈ Finset.range n, (2 ^ 2 ^ k + 3 ^ 2 ^ k) = 3 ^ 2 ^ n - 2 ^ 2 ^ n
```

Read back by a model that saw only the Lean. *For every natural number n , the product over all natural numbers k in the range $0 \leq k < n$ of $(2^{2^k} + 3^{2^k})$ equals $3^{2^n} - 2^{2^n}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem power_sum_product_teslescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
  intro n
  induction n with
  | zero =>
    norm_num
  | succ n ih =>
    rw [Finset.prod_range_succ]
    have hpow : (2 : ℕ) ^ (n + 1) = (2 : ℕ) ^ n * 2 := by
      simp [pow_succ]
    have h3 : (3 : ℤ) ^ (2 ^ (n + 1)) = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    have h2 : (2 : ℤ) ^ (2 ^ (n + 1)) = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    calc
      (∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
        ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) =
        ((3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)) *
        ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) := by
          rw [ih]
      _ = (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
          rw [h3, h2]
      _ =
        ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__e0c02869

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e0c028695321139b

4.187 187. bounded_radix_geometric_product

The problem

For every natural number q with $q = 2$ or $q = 3$, all integers a and b , and every natural number n , there exists a sequence of natural numbers $e(k)$ with $e(0) = 1$ and $e(k + 1) = qe(k)$ for every k , such that
$$(b - a) \prod_{k=0}^{n-1} \left(\sum_{j=0}^{q-1} b^{(q-1-j)e(k)} a^{je(k)} \right) = b^{e(n)} - a^{e(n)}.$$

Lean statement

```
theorem bounded_radix_geometric_product :
  ∀ (q : ℕ), q = 2 ∨ q = 3 → ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = q * e k) ∧
    (∀ k ∈ Finset.range n,
      (∑ j ∈ Finset.range q,
        b ^ ((q - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
      b ^ e n - a ^ e n
```

Parent. For every pair of integers a and b and every natural number n , there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $(b - a) \prod_{k=0}^{n-1} (b^{e_k} + a^{e_k}) = b^{e_n} - a^{e_n}$.

Parent in Lean

```
theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∀ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n
```

What it contributes. Supplies the exponent-recurrence and product-telescoping induction pattern; the proof extends the binary factorization to the $q=3$ cubic factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the telescoping identity from doubling to radix 2 or 3, requiring separate geometric-factor expansions and inductive factorization for the cubic case. The parent's direct 'pow_two_pow_sub_pow_two_pow' application does not supply the new finite-sum factorization or bounded-radix case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the telescoping product to $q=3$, requiring a new cubic geometric-factor identity and an induction argument; the parent only supplies the $q=2$ square case. The bounded q split is used to construct distinct radix-specific exponent sequences.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The $q = 3$ branch requires a new cubic geometric-factor identity and an induction over the product, which the parent's doubling-specific lemma does not provide. The bounded q case split therefore demands genuinely different reasoning rather than recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1827 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_radix_geometric_product.extracted_1.1 : ∀ (q : ℕ),
  q = 2 ∨ q = 3 →
  ∀ (a b : ℤ) (n : ℕ),
    ∃ e,
      e 0 = 1 ∧
      (∀ (k : ℕ), e (k + 1) = q * e k) ∧
      (∏ k ∈ Finset.range n, ∑ j ∈ Finset.range q, b ^ ((q - 1 - j) * e k) * a ^ (j * e k)) * (b - a) =
        b ^ e n - a ^ e n
```

Read back by a model that saw only the Lean. For every natural number q , if $q = 2$ or $q = 3$, then for every pair of integers a and b and every natural number n , there exists a sequence e of natural numbers such that $e(0) = 1$, $e(k + 1) = q \cdot e(k)$ for every natural number k , and $(\prod_{k=0}^{n-1} \sum_{j=0}^{q-1} b^{(q-1-j) \cdot e(k)} \cdot a^{j \cdot e(k)}) \cdot (b-a) = b^{e(n)} - a^{e(n)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem bounded_radix_geometric_product :
  ∀ (q : ℕ), q = 2 ∨ q = 3 → ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = q * e k) ∧
    (∏ k ∈ Finset.range n,
      (∑ j ∈ Finset.range q,
        b ^ ((q - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
      b ^ e n - a ^ e n := by
  intro q hq a b n
  rcases hq with rfl | rfl
  · let e : ℕ → ℕ := fun k => 2 ^ k
    refine ⟨e, ?_, ?_, ?_⟩
    · simp [e]
    · intro k
      simp [e, pow_succ, Nat.mul_comm]
    · induction n with
      | zero => simp [e]
      | succ n ih =>
        have he : e (n + 1) = 2 * e n := by
          simp [e, pow_succ, Nat.mul_comm]
        have hb2 : b ^ (2 * e n) = (b ^ e n) ^ 2 := by
          rw [show 2 * e n = e n * 2 by omega, pow_mul]
        have ha2 : a ^ (2 * e n) = (a ^ e n) ^ 2 := by
          rw [show 2 * e n = e n * 2 by omega, pow_mul]
        have hfactor :
          (b ^ e n - a ^ e n) *
            (∑ j ∈ Finset.range 2,
              b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) =
              b ^ (2 * e n) - a ^ (2 * e n) := by
          rw [hb2, ha2]
```

```

norm_num [Finset.sum_range_succ]
ring
calc
  (Π k ∈ Finset.range (Nat.succ n),
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
    ((Π k ∈ Finset.range n,
      (Σ j ∈ Finset.range 2,
        b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) *
      (Σ j ∈ Finset.range 2,
        b ^ ((2 - 1 - j) * e n) * a ^ (j * e n))) * (b - a) := by
      rw [Finset.prod_range_succ]
  - = ((Π k ∈ Finset.range n,
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) * (b - a)) *
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) := by ring
  - = (b ^ e n - a ^ e n) *
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) := by rw [ih]
  - = b ^ (2 * e n) - a ^ (2 * e n) := hfactor
  - = b ^ e (Nat.succ n) - a ^ e (Nat.succ n) := by rw [he]
· let e : ℕ → ℕ := fun k => 3 ^ k
refine (e, ?_, ?_, ?_)
· simp [e]
· intro k
  simp [e, pow_succ, Nat.mul_comm]
· induction n with
| zero => simp [e]
| succ n ih =>
  have he : e (n + 1) = 3 * e n := by
    simp [e, pow_succ, Nat.mul_comm]
  have hb3 : b ^ (3 * e n) = (b ^ e n) ^ 3 := by
    rw [show 3 * e n = e n * 3 by omega, pow_mul]
  have ha3 : a ^ (3 * e n) = (a ^ e n) ^ 3 := by
    rw [show 3 * e n = e n * 3 by omega, pow_mul]
  have hfactor :
    (b ^ e n - a ^ e n) *
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) =
    b ^ (3 * e n) - a ^ (3 * e n) := by
    rw [hb3, ha3]
  norm_num [Finset.sum_range_succ]
  ring
calc
  (Π k ∈ Finset.range (Nat.succ n),
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
    ((Π k ∈ Finset.range n,
      (Σ j ∈ Finset.range 3,
        b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) *
      (Σ j ∈ Finset.range 3,
        b ^ ((3 - 1 - j) * e n) * a ^ (j * e n))) * (b - a) := by
      rw [Finset.prod_range_succ]
  - = ((Π k ∈ Finset.range n,
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) * (b - a)) *
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) := by ring
  - = (b ^ e n - a ^ e n) *
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) := by rw [ih]
  - = b ^ (3 * e n) - a ^ (3 * e n) := hfactor
  - = b ^ e (Nat.succ n) - a ^ e (Nat.succ n) := by rw [he]

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh.me.me__3967729c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 59bc3742bbb4ce64

4.188 188. symbolic_doubling_power_product

The problem

For every pair of integers a and b and every natural number n , there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $(b - a) \prod_{k=0}^{n-1} (b^{e_k} + a^{e_k}) = b^{e_n} - a^{e_n}$.

Lean statement

```
theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n
```

Parent. For every natural number n , show that there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $\prod_{k=0}^{n-1} (2^{e_k} + 3^{e_k}) = 3^{e_n} - 2^{e_n}$.

Parent in Lean

```
theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n
```

What it contributes. Generalizes the parent's fixed bases 2 and 3 to arbitrary integer bases a and b , retaining the doubling exponent sequence while exposing the symbolic difference-of-squares step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both bases and requires the doubling-exponent factorization for arbitrary integers a and b ; the parent only establishes the fixed-base case. Although the witness sequence is reused, the parent theorem cannot directly prove the symbolic identity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the bases and introduces the symbolic factor ' $(b - a)$ ', requiring a general power-difference telescoping lemma over $\mathbb{Z}$. The parent's specialized induction for bases 2 and 3 does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the telescoping identity from fixed bases 2 and 3 to arbitrary integer bases a and b , introducing the factor $(b - a)$. Its proof requires a general power-difference lemma rather than recalling the parent's fixed-base induction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1820 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem symbolic_doubling_power_product.extracted_1.1 : ∀ (a b : ℤ) (n : ℕ),
  ∃ e,
    e 0 = 1 ∧
    (∀ (k : ℕ), e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) = b ^ e n - a ^ e n
```

Read back by a model that saw only the Lean. For every pair of integers a and b and every natural number n , there exists a function e from the natural numbers to the natural numbers such that $e(0) = 1$, for every natural number k we have $e(k + 1) = 2e(k)$, and $(\prod_{k=0}^{n-1} (b^{e(k)} + a^{e(k)}))(b - a) = b^{e(n)} - a^{e(n)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n := by
  intro a b n
  let e : ℕ → ℕ := fun k => 2 ^ k
  refine ⟨e, ?_, ?_, ?_⟩
  · simp [e]
  · intro k
    simp [e, pow_succ, Nat.mul_comm]
  · simpa [e] using
    (pow_two_pow_sub_pow_two_pow (R := ℤ) (x := b) (y := a) n).symm
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh.me__ddd6340a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 014c3b0057680c73

4.189 189. natural_iterate_period

The problem

Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Lean statement

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

Parent. Let a be a positive real number and f be a real function such that $\forall x \in \mathbb{R}, f(x+a) = \frac{1}{2} + \sqrt{f(x) - f(x)^2}$. Show that there exists a positive real number b such that $\forall x \in \mathbb{R}, f(x+b) = f(x)$.

Parent in Lean

```
theorem imo_1968_p5_1 (a : ℝ) (f : ℝ → ℝ) (h₀ : 0 < a)
  (h₁ : ∀ x, f (x + a) = 1 / 2 + Real.sqrt (f x - f x ^ 2)) :
  ∃ b > 0, ∀ x, f (x + b) = f x := by
```

What it contributes. The parent supplies the positive-period checkpoint: its existential conclusion provides a suitable p and $hperiod$ for this reusable natural-iterate law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a new induction argument to lift a one-step period to every natural multiple, including the arithmetic normalization of successive translates. This reasoning is not supplied by the parent's proof, which only derives a single period from the recurrence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an existential period conclusion into a natural-indexed translation law and requires induction to propagate the period across arbitrary multiples. The parent's analytic argument only derives one period and does not supply this iterative reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a reusable natural-number iterate law from an arbitrary period, requiring induction and transport of the casted successor. The parent's proof only derives one specific positive period and supplies no iteration argument, so recalling it does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2487 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem natural_iterate_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p : ℝ),
  0 < p → (∀ (x : ℝ), f (x + p) = f x) → ∀ (n : ℕ) (x : ℝ), f (x + ↑n * p) = f x
```

Read back by a model that saw only the Lean. *For every function f from the real numbers to the real numbers and every real number p , if p is positive and $f(x + p) = f(x)$ for every real number x , then for every natural number n and every real number x , $f(x + n p) = f(x)$, where n is regarded as a real number.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
  intro n
  induction n with
  | zero =>
    intro x
    simp
  | succ n ih =>
    intro x
    have hstep : f ((x + (n : ℝ) * p) + p) = f (x + (n : ℝ) * p) :=
      hperiod (x + (n : ℝ) * p)
    have hprev : f (x + (n : ℝ) * p) = f x := ih x
    calc
      f (x + (Nat.succ n : ℝ) * p) = f ((x + (n : ℝ) * p) + p) := by
        congr 1
      norm_num [Nat.cast_succ]
      ring
      _ = f (x + (n : ℝ) * p) := hstep
      _ = f x := hprev
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me__04dc59b8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c91ed953a9e4530c

4.190 190. coupled_configuration_impossible

The problem

Let $p < q < r$ be integers greater than 1 such that $(p-1)(q-1)(r-1)$ divides $pqr-1$. If r is one of the two possible classified values 8 or 15, show that $r \neq 10^n - (-1)^n$ for every natural number n .

Lean statement

```
theorem coupled_configuration_impossible (p q r : ℤ) (n : ℕ)

(hclassified : r = 8 ∨ r = 15)
(hcouple : r = (10 : ℤ) ^ n - (-1 : ℤ) ^ n) : False
```

Parent. Let p, q, r be integers with $1 < p < q < r$. Show that if $(p-1) * (q-1) * (r-1)$ divides $p * q * r - 1$, then $(p, q, r) = (2, 4, 8)$ or $(p, q, r) = (3, 5, 15)$.

Parent in Lean

```
theorem imo_1992_p1 (p q r : ℤ) (h₀ : 1 < p ∧ p < q ∧ q < r)
(h₁ : (p - 1) * (q - 1) * (r - 1) ∣ p * q * r - 1) :
(p, q, r) = (2, 4, 8) ∨ (p, q, r) = (3, 5, 15) := by
```

What it contributes. Supplies the classification checkpoint that the possible third coordinate is 8 or 15, represented here by `hclassified`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child uses the certified classification $r = 8$ or 15 but introduces genuinely new induction proving 11 divides $10^n - (-1)^n$, then derives contradictions for both classified values. A memorized proof of the parent would not supply this divisibility argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the parent-derived classification of r but requires a new induction proving $11 \mid 10^n - (-1)^n$, then derives a contradiction for both classified values. Recalling the parent does not supply this divisibility argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not solvable by recalling the parent's classification proof: it independently proves that 11 divides $10^n - (-1)^n$ by induction, then contradicts the classified values 8 and 15 . This is a smaller executable child using the available classification checkpoint and introduces genuinely different mathematics.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_configuration_impossible.extracted_1_1 (p q r : ℤ) (n : ℕ) (h₀ : 1 < p ∧ p < q ∧ q < r)
(h₁ : (p - 1) * (q - 1) * (r - 1) ∣ p * q * r - 1) (hclassified : r = 8 ∨ r = 15) (hcouple : r = 10 ^ n - (-1) ^ n) :
False
```

Read back by a model that saw only the Lean. *Let p, q , and r be integers and let n be a natural number. Suppose that $1 < p < q < r$, that $(p-1)(q-1)(r-1)$ divides $pqr-1$, that r is either 8 or 15, and that $r = 10^n - (-1)^n$. Then a contradiction follows; equivalently, False holds.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem coupled_configuration_impossible (p q r : ℤ) (n : ℕ)

  (hclassified : r = 8 ∨ r = 15)
  (hcouple : r = (10 : ℤ) ^ n - (-1 : ℤ) ^ n) : False := by
have hpow : ∀ m : ℕ, (11 : ℤ) ∣ (10 : ℤ) ^ m - (-1 : ℤ) ^ m := by
  intro m
  induction m with
  | zero => norm_num
  | succ m ih =>
    rcases ih with ⟨k, hk⟩
    refine ⟨10 * k + (-1 : ℤ) ^ m, ?_⟩
    simp only [pow_succ]
    calc
      10 ^ m * 10 - (-1) ^ m * (-1) =
        10 * (10 ^ m - (-1) ^ m) + 11 * (-1) ^ m := by ring
      _ = 10 * (11 * k) + 11 * (-1) ^ m := by rw [hk]
      _ = 11 * (10 * k + (-1) ^ m) := by ring
have hdiv : (11 : ℤ) ∣ r := by
  rw [hcouple]
  exact hpow n
rcases hclassified with h8 | h15
· rw [h8] at hdiv
  norm_num at hdiv
· rw [h15] at hdiv
  norm_num at hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. imo_1992_p1__me__0afbec7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 4f2b9338b2a23037

4.191 191. bounded_residue_power_mod11

The problem

If an integer base is congruent to 10 modulo 11, then every natural power of that base is congruent to the corresponding signed power of -1 modulo 11.

Lean statement

```
theorem bounded_residue_power_mod11 (b : ℤ) (h : Int.ModEq 11 b 10) : ∀ n : ℕ, Int.ModEq 11 (b ^ n) ((-1 : ℤ) ^ n)
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Supplied the signed-power invariant and modulus/base residue pattern; the child parameterizes the base and proves the modular congruence directly.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: from an arbitrary residue-class hypothesis $b \equiv 10 \pmod{11}$, it proves all powers $b^n \equiv (-1)^n$ by induction and multiplicativity of `Int.ModEq`. The parent's fixed-base divisibility lemma does not supply this argument for arbitrary b .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and changes divisibility into a modular-congruence statement. Its proof must transfer $b \equiv 10 \equiv -1 \pmod{11}$ and propagate that relation through multiplication and induction, which the parent's single fixed-base divisibility lemma does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base 10 result to every integer base congruent to 10 modulo 11. Its induction uses modular-equivalence transitivity and multiplicativity, reasoning not supplied by the parent's direct divisibility lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1874 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_residue_power_mod11.extracted_1_1 : ∀ (b : ℤ),  
  b ≡ 10 [ZMOD 11] → ∀ (n : ℕ), b ^ n ≡ (-1) ^ n [ZMOD 11]
```

Read back by a model that saw only the Lean. *For every integer b , if b is congruent to 10 modulo 11, then for every natural number n , b raised to the n th power is congruent to (-1) raised to the n th power modulo 11.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_residue_power_mod11 (b : ℤ) (h : Int.ModEq 11 b 10) : ∀ n : ℕ, Int.ModEq 11 (b ^ n) ((-1 : ℤ) ^ n) := by  
  intro n  
  induction n with  
  | zero =>  
    simp [Int.ModEq]  
  | succ n ih =>  
    have hten : Int.ModEq 11 (10 : ℤ) (-1 : ℤ) := by  
      norm_num [Int.ModEq]  
    have hstep : Int.ModEq 11 b (-1 : ℤ) := h.trans hten  
    have hmul : Int.ModEq 11 (b ^ n * b) ((-1 : ℤ) ^ n * (-1 : ℤ)) := ih.mul hstep  
    simp [pow_succ] using hmul
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__0077ec87

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 35afb0214da0a4fd

4.192 192. bounded_mod11_product

The problem

For any finite list of natural numbers, the difference between the product of the corresponding powers of 10 and the product of the corresponding powers of -1 is divisible by 11.

Lean statement

```
theorem bounded_mod11_product (xs : List ℕ) :
  (11 : ℤ) ∣ (xs.map (fun n => (10 : ℤ) ^ n)).prod -
  (xs.map (fun n => (-1 : ℤ) ^ n)).prod
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Supplies the modulo-11 divisibility checkpoint for the aggregate exponent via the standard difference-of-powers argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine aggregate generalization: it requires induction to identify each product with a power indexed by the list sum, then applies the modulo-11 law. Recalling the parent only supplies the final divisibility step and does not solve the new product-to-power reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new induction proving that both list products collapse to powers indexed by the sum of exponents, then combines these identities with the single-exponent divisibility law. Recalling the parent only supplies the final divisibility step and cannot solve the aggregate statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires an induction to identify each finite product with a power at the aggregate exponent, then applies the parent law. This aggregate normalization is new reasoning beyond recalling the parent's single-exponent divisibility proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2707 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	MISMATCH: Statement B uses a natural-number interpretation for the difference (and potentially for -1), whereas Statement A uses integer arithmetic; for example, with $xs=[1]$, A gives $10-(-1)=11$, but B's natural interpretation gives $10-0=10$, which is not divisible by 11.

Goal round-trip: what Lean elaborated

```
theorem bounded_mod11_product.extracted_1_1 (xs : List ℕ) :
  11 ∣ (List.map (fun n => 10 ^ n) xs).prod - (List.map (fun n => (-1) ^ n) xs).prod
```

Read back by a model that saw only the Lean. *For every list xs of natural numbers, 11 divides the natural-number difference between the product of 10^n over all n in xs and the product of $(-1)^n$ over all n in xs .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_mod11_product (xs : List ℕ) :
  (11 : ℤ) ∣ (xs.map (fun n => (10 : ℤ) ^ n)).prod -
    (xs.map (fun n => (-1 : ℤ) ^ n)).prod := by
  have hagg :
    (xs.map (fun n => (10 : ℤ) ^ n)).prod = (10 : ℤ) ^ xs.sum ∧
    (xs.map (fun n => (-1 : ℤ) ^ n)).prod = (-1 : ℤ) ^ xs.sum := by
    induction xs with
    | nil => simp
    | cons a xs ih =>
      constructor
      · simpa [ih.1, pow_add]
      · simpa [ih.2, pow_add]
  have hdiv : (11 : ℤ) ∣ (10 : ℤ) ^ xs.sum - (-1 : ℤ) ^ xs.sum := by
    simpa using (sub_dvd_pow_sub_pow (10 : ℤ) (-1 : ℤ) xs.sum)
  rw [hagg.1, hagg.2]
  exact hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__16570d78

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16570d7852a5e928

4.193 193. alternating_power_divisibility_iff

The problem

For an integer a , prove that 11 divides $a^n - (-1)^n$ for every natural number n if and only if a is congruent to -1 modulo 11.

Lean statement

```
theorem alternating_power_divisibility_iff (a : ℤ) : (∀ n : ℕ, (11 : ℤ) ∣ a ^ n - (-1 : ℤ) ^ n) ↔ a ≡ (-1 : ℤ) [ZMOD 11]
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonm1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Generalizes the parent's alternating-power divisibility argument from the fixed base 10 to a parameter a , consuming the congruence-derived divisor through the power-difference lemma.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the fixed divisibility claim into an exact iff characterization over arbitrary integers. It requires both extracting the congruence from the $n = 1$ case and proving the universal law from an arbitrary congruence, so recalling the parent does not suffice.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes all integers satisfying the divisibility law modulo 11. Its reverse direction uses the power-difference divisibility lemma parametrically, while the forward direction extracts the congruence from the exponent-one case; recalling the parent's fixed-base proof does not solve this equivalence.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the task from proving one fixed-base divisibility law to characterizing exactly which integers satisfy the law for every exponent. It requires a new reverse-direction argument via the $n = 1$ case and modular equivalence, so recalling the parent's single application of the power-difference lemma is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2759 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem alternating_power_divisibility_iff.extracted_1_1 (a : Z) :  
  (∀ (n : N), 11 ∣ a ^ n - (-1) ^ n) ↔ a ≡ -1 [ZMOD 11]
```

Read back by a model that saw only the Lean. For an integer a , 11 divides $a^n - (-1)^n$ for every natural number n if and only if a is congruent to -1 modulo 11.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem alternating_power_divisibility_iff (a : Z) : (∀ n : N, (11 : Z) ∣ a ^ n - (-1 : Z) ^ n) ↔ a ≡ (-1 : Z) [ZMOD 11] := by  
  constructor  
  · intro h  
    have h1 : (11 : Z) ∣ a - (-1 : Z) := by  
      simpa using h 1  
    have hneg : (11 : Z) ∣ (-1 : Z) - a := by  
      simpa using (dvd_neg.mpr h1)  
    rw [Int.modEq_iff_dvd]  
    exact hneg  
  · intro h  
    have hdiff : (11 : Z) ∣ a - (-1 : Z) := by  
      exact h.symm.dvd  
    intro n  
    have hexp : a - (-1 : Z) ∣ a ^ n - (-1 : Z) ^ n := by  
      exact sub_dvd_pow_sub_pow a (-1 : Z) n  
    exact dvd_trans hdiff hexp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__6e895ca9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 6e895ca941e9e1a4

4.194 194. parameterized_11_dvd_pow_sub_pow

The problem

Let b be an integer such that $11 \mid b^2 - 1$ but $11 \nmid b - 1$. Prove that $11 \mid b^n - (-1)^n$ for every natural number n .

Lean statement

```
theorem parameterized_11_dvd_pow_sub_pow (b : ℤ) (hquad : (11 : ℤ) ∣ b ^ 2 - 1) (hne : ¬ (11 : ℤ) ∣ b - 1) : ∀ n : ℕ, (11 : ℤ) ∣ b ^ n - (-1 : ℤ) ^ n
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Reuses the parent's difference-of-powers exponent law while replacing the fixed base 10 with a parameter whose residue is classified from the quadratic hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires genuinely new reasoning: factoring the quadratic, using primality of 11 and the nondivisibility hypothesis to classify b modulo 11, then transporting that result to all powers. The parent is a direct application of `sub_dvd_pow_sub_pow` and does not supply this classification argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and requires new reasoning: factoring the quadratic, using primality of 11 and the nondivisibility hypothesis to classify b modulo 11, then transferring that divisibility to all powers. The parent's direct fixed-base application cannot solve this classification step.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and must derive the residue classification from the quadratic divisibility and nondivisibility hypotheses before applying the power-difference lemma. The parent's direct proof for base 10 does not supply that classification or the required divisibility transfer.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propect, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2736 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_11_dvd_pow_sub_pow.extracted_1_1 : ∀ (b : ℤ),  
  11 ∣ b ^ 2 - 1 → ¬11 ∣ b - 1 → ∀ (n : ℕ), 11 ∣ b ^ n - (-1) ^ n
```

Read back by a model that saw only the Lean. For every integer b , if 11 divides $b^2 - 1$ and 11 does not divide $b - 1$, then for every natural number n , 11 divides $b^n - (-1)^n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem parameterized_11_dvd_pow_sub_pow (b : ℤ) (hquad : (11 : ℤ) ∣ b ^ 2 - 1) (hne : ¬ (11 : ℤ) ∣ b - 1) : ∀ n : ℕ, (11 : ℤ) ∣ b ^ n -  
  (-1 : ℤ) ^ n := by
  have hfactor : b ^ 2 - 1 = (b - 1) * (b + 1) := by
    ring
  have hprod : (11 : ℤ) ∣ (b - 1) * (b + 1) := by
    rw [← hfactor]
    exact hquad
  have hp11 : Prime (11 : ℤ) := by
    norm_num
  have hcases : (11 : ℤ) ∣ b - 1 ∨ (11 : ℤ) ∣ b + 1 :=
    (hp11.dvd_mul).mp hprod
  have hplus : (11 : ℤ) ∣ b + 1 := by
    cases hcases with
    | inl hleft => exact False.elim (hne hleft)
    | inr hright => exact hright
  have hbase : (11 : ℤ) ∣ b - (-1 : ℤ) := by
    simpa using hplus
  intro n
  have hpow : b - (-1 : ℤ) ∣ b ^ n - (-1 : ℤ) ^ n := by
    exact sub_dvd_pow_sub_pow b (-1 : ℤ) n
  exact dvd_trans hbase hpow
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__a4706044

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a4706044089b5e7d

4.195 195. odd_power_geometric_factorization

The problem

For natural numbers b , n , and q with $b = 3q + 2$, prove that $(b^{2n+1} + 1) = (b + 1) \sum_{k=0}^{2n} (-1)^k b^{2n-k}$ and $b + 1 = 3(q + 1)$, with these equalities interpreted over the integers.

Lean statement

```
theorem odd_power_geometric_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ((b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k)) ∧ (b : ℤ) + 1 = 3 * (q + 1)
```

Parent. For natural numbers b , n , and q , if $b = 3q + 2$, prove that there exists a natural number s such that $b^{2n+1} + 1 = 3(q + 1)s$.

Parent in Lean

```
theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1) * s
```

What it contributes. Uses the parent decomposition $b = 3q + 2$ and its checkpoint $b + 1 = 3(q + 1)$, while replacing the existential divisibility witness with a symbolic geometric-factorization identity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces existential divisibility with an explicit alternating geometric-factorization identity over $\mathbb{Z}$. Proving this requires a new induction on the exponent and sign analysis; the parent's divisibility lemma and final witness construction do not supply that reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces existential divisibility with an explicit integer geometric-factorization identity, proved by induction over the exponent, while separately deriving the factor relation from the decomposition hypothesis. This requires genuinely new reasoning and cannot be solved by recalling the parent's divisibility proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces existential divisibility with an explicit integer geometric-factorization identity, whose proof requires a new induction and ring algebra absent from the parent. The appended base equality is shared, but the substantive factorization is not recoverable by recalling the parent's divisibility argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2066 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_geometric_factorization.extracted_1_1 (b n q : ℕ) (hdecomp : b = 3 * q + 2) :
  ↑b ^ (2 * n + 1) + 1 = (↑b + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1) ^ k * ↑b ^ (2 * n - k) ∧
  ↑b + 1 = 3 * (↑q + 1)
```

Read back by a model that saw only the Lean. For natural numbers b , n , and q , if $b = 3q + 2$, then both of the following hold: (1) the integer b raised to the natural-number exponent $2n + 1$, plus 1, equals $(b + 1)$ times the sum over k in $\text{Finset.range } (2n + 1)$ of $(-1)^k$ times b raised to the natural-number exponent $2n - k$; and (2) $b + 1 = 3(q + 1)$, where b and q are regarded as integers via the natural-number embedding.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_power_geometric_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ((b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈
  ↪ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k)) ∧ (b : ℤ) + 1 = 3 * (q + 1) := by
  have hgeom : ∀ m : ℕ, ((b : ℤ) + 1) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) = (b : ℤ) ^ m - (-1 : ℤ) ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp [Finset.sum_range_succ, Nat.succ_sub_succ_eq_sub] at ih ⊢
      have hs :
        (∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - k)) =
          (b : ℤ) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) := by
            rw [Finset.mul_sum]
            apply Finset.sum_congr rfl
            intro k hk
            have hkm : k < m := Finset.mem_range.mp hk
            have he : m - k = (m - (k + 1)) + 1 := by omega
            rw [he, pow_succ]
            ring
      calc
      ((b : ℤ) + 1) *
        ((∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - k)) + (-1 : ℤ) ^ m) =
        ((b : ℤ) + 1) *
          ((b : ℤ) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) + (-1 : ℤ) ^ m) := by rw [hs]
      _ = (b : ℤ) * (((b : ℤ) + 1) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1))) + ((b : ℤ) + 1) * (-1 : ℤ) ^ m :=
        ↪ by ring
      _ = (b : ℤ) * ((b : ℤ) ^ m - (-1 : ℤ) ^ m) + ((b : ℤ) + 1) * (-1 : ℤ) ^ m := by rw [ih]
      _ = (b : ℤ) ^ (m + 1) - (-1 : ℤ) ^ (m + 1) := by
        rw [pow_succ, pow_succ]
        ring
      have hsign : (-1 : ℤ) ^ (2 * n + 1) = -1 := by
        induction n with
        | zero => norm_num
        | succ n ih =>
          rw [show 2 * (Nat.succ n) + 1 = (2 * n + 1) + 2 by omega, pow_add]
          norm_num [ih]
      have hfactor : (b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k) := by
        simpa [Nat.add_sub_add_right, hsign] using (hgeom (2 * n + 1)).symm
      have hbase : (b : ℤ) + 1 = 3 * (q + 1) := by
        omega
      exact ⟨hfactor, hbase⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.me.me__3f8b13d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3f8b13d5913ed6df

4.196 196. residue_power_minus_one_iff_odd

The problem

For every natural base b and exponent m , if $b \equiv 2 \pmod{3}$, then 3 divides $b^m + 1$ over the integers if and only if m is odd.

Lean statement

```
theorem residue_power_minus_one_iff_odd (b m : ℕ) (hmod : b % 3 = 2) : (3 : ℤ) ∣ (b : ℤ) ^ m + 1 ↔ Odd m
```

Parent. For every natural bound B , base b , and exponent parameter n , if $b \leq B$ and $b \equiv 2 \pmod{3}$, then 3 divides $b^{(2n+1)} + 1$ over the integers.

Parent in Lean

```
theorem bounded_residue_odd_power (B b n : ℕ) (hb : b ≤ B) (hmod : b % 3 = 2) : 3 ∣ (b : ℤ) ^ (2 * n + 1) + 1
```

What it contributes. Supplies the residue decomposition, divisibility of $b+1$ by 3, and the odd-power sum factorization used in the odd direction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child upgrades the parent's one-way odd-exponent divisibility result to an iff characterization. Its converse requires a new modular argument showing even powers are congruent to 1 modulo 3, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's one-way odd-exponent result into an exact iff characterization. Its reverse direction requires a new even/odd case split and modular induction, so recalling the parent cannot solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine iff characterization: besides the parent's odd-exponent direction, it establishes that even exponents cannot yield the congruence, using parity decomposition and an inductive residue argument. This converse is not supplied by recalling the parent's proof, and the hypothesis is materially used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1770 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem residue_power_minus_one_iff_odd.extracted_1_1 (b m : ℕ) (hmod : b % 3 = 2) : 3 ∣ b ^ m + 1 ↔ Odd m
```

Read back by a model that saw only the Lean. For natural numbers b and m , if b modulo 3 equals 2, then 3 divides the integer $b^m + 1$ if and only if m is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem residue_power_minus_one_iff_odd (b m : ℕ) (hmod : b % 3 = 2) : (3 : ℤ) ∣ (b : ℤ) ^ m + 1 ↔ Odd m := by
  have hdecomp : b = 3 * (b / 3) + 2 := by
    omega
  have hdecompZ : (b : ℤ) = 3 * (↑(b / 3) : ℤ) + 2 := by
    exact_mod_cast hdecomp
  have hbase : (3 : ℤ) ∣ (b : ℤ) + 1 := by
    refine ⟨(↑(b / 3) : ℤ) + 1, ?_⟩
    rw [hdecompZ]
    ring
  have hbsq : b ^ 2 % 3 = 1 := by
    norm_num [pow_two, Nat.mul_mod, hmod]
  have h_even_mod : ∀ k : ℕ, b ^ (2 * k) % 3 = 1 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [Nat.mul_succ, Nat.pow_add, Nat.mul_mod, ih, hbsq]
  constructor
  · intro hd
    have hdNat : 3 ∣ b ^ m + 1 := by
      exact_mod_cast hd
    rcases Nat.even_or_odd m with he | ho
    · rcases he with (k, hk)
      have hzero : (b ^ m + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hdNat
      have hkm : k + k = 2 * k := by omega
      rw [hk, hkm, Nat.add_mod, h_even_mod] at hzero
      norm_num at hzero
    · exact ho
  · intro ho
    have hpow : (b : ℤ) + 1 ∣ (b : ℤ) ^ m + (1 : ℤ) ^ m := by
      exact Odd.add_dvd_pow_add_pow (b : ℤ) 1 ho
    have htarget : (b : ℤ) + 1 ∣ (b : ℤ) ^ m + 1 := by
      simpa using hpow
    rcases hbase with ⟨c, hc⟩
    rcases htarget with ⟨d, hd⟩
    refine ⟨c * d, ?_⟩
    rw [hd, hc]
    ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__bd3f2cd6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 469f52043806a3be

4.197 197. power_mod_three_if

The problem

Let b and k be natural numbers with $b \equiv 2 \pmod{3}$. Show that the remainder of b^k upon division by 3 is 1 when k is even and 2 when k is odd.

Lean statement

```
theorem power_mod_three_if (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reuses the parent's explicit base-class condition modulo 3 and its odd-power residue mechanism, generalized here through exponent induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from odd exponents and divisibility to an exact two-branch residue characterization for every exponent. Its induction tracks parity and multiplication modulo 3, requiring reasoning not supplied by the parent's divisibility argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from odd exponents of the form $2n+1$ to every exponent and characterizes both residue branches modulo 3. Its induction and parity case split supply genuinely new reasoning beyond the parent's odd-power divisibility argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from odd exponents to every natural exponent and gives the complete parity-dependent residue formula modulo 3. Its induction and parity case split require reasoning not supplied by the parent's divisibility argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2757 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_mod_three_if.extracted_1_1 (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2
```

Read back by a model that saw only the Lean. For natural numbers b and k , if b leaves remainder 2 upon division by 3, then b raised to the k -th power leaves remainder 1 modulo 3 when k is even, and remainder 2 modulo 3 when k is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_mod_three_if (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2 := by
  induction k with
  | zero => simp
  | succ k ih =>
    rw [pow_succ, Nat.mul_mod]
    by_cases hk : k % 2 = 0
    · have hk1 : (Nat.succ k) % 2 = 1 := by omega
      rw [ih]
      simp [hk, hk1, hb]
    · have hk1 : k % 2 = 1 := by omega
      have hk2 : (Nat.succ k) % 2 = 0 := by omega
      rw [ih]
      simp [hk, hk1, hk2, hb]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__dd5e8f57

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint dd5e8f579d17e3eb

4.198 198. divisibility_iff_odd_exponent_mod_three

The problem

Let b and k be natural numbers with $b \equiv 2 \pmod{3}$. Prove that $3 \mid (b^k + 1)$ if and only if k is odd, including the case $k = 0$.

Lean statement

```
theorem divisibility_iff_odd_exponent_mod_three (b k : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ k + 1 ↔ Odd k
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Supplies the base divisibility $3 \mid b + 1$ and odd-power sum-divisibility argument; the child adds the reverse parity implication and the explicit $k=0$ case.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: divisibility forces oddness by handling zero and proving all even exponents give residue 1 modulo 3. Recalling the parent only supplies the odd-to-divisibility direction, not the parity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes all exponents: beyond reusing the parent's odd-power argument, it proves the converse by deriving the modular behavior of even powers and handling the zero exponent separately. This reverse direction is new mathematics and cannot be obtained by recalling the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-way divisibility lemma into a full iff characterization. Besides reusing the parent mechanism for odd exponents as required, it introduces genuinely new parity decomposition and modular-periodicity reasoning to exclude even exponents, including the $k = 0$ case.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2724 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem divisibility_iff_odd_exponent_mod_three.extracted_1_1 (b k : ℕ) (hb : b % 3 = 2) :  
  3 ∣ b ^ k + 1 ↔ Odd k
```

Read back by a model that saw only the Lean. For natural numbers b and k satisfying b modulo 3 equals 2, 3 divides $b^k + 1$ if and only if k is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem divisibility_iff_odd_exponent_mod_three (b k : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ k + 1 ↔ Odd k := by
  have hzero : (¬ 3 ∣ b ^ 0 + 1) ∧ ¬ Odd 0 := by
    simp [Nat.dvd_iff_mod_eq_zero]
  have h_two : ∀ n : ℕ, 2 ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      rw [show 2 * (n + 1) = 2 * n + 2 by omega, pow_add]
      rw [Nat.mul_mod, ih]
      norm_num
  have h_even_mod : ∀ n : ℕ, b ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      rw [show 2 * (n + 1) = 2 * n + 2 by omega, pow_add]
      rw [Nat.mul_mod, Nat.pow_mod, Nat.pow_mod]
      simp [h_two n, hb, Nat.pow_mod]
  constructor
  · intro hd
    by_cases hk0 : k = 0
    · subst k
      exact False.elim (hzero.1 (by simp using hd))
    · rcases Nat.even_or_odd k with he | ho
      · rcases he with ⟨n, hn⟩
        have hm : (b ^ k + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hd
        have hn2 : k = 2 * n := by omega
        rw [hn2] at hm
        rw [Nat.add_mod, h_even_mod n] at hm
        omega
      · exact ho
  · intro hk
    by_cases hk0 : k = 0
    · subst k
      exact False.elim (hzero.2 hk)
    · rcases hk with ⟨n, hn⟩
      rw [hn]
      have hbase : 3 ∣ b + 1 := by
        have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
        rw [hb] at hdecomp
        use b / 3 + 1
        omega
      have hodd : Odd (2 * n + 1) := by
        refine ⟨n, ?_⟩
        omega
      have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
        exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
      have hpow2 : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
        simp using hpow
      have hfinal : 3 ∣ b ^ (2 * n + 1) + 1 := dvd_trans hbase hpow2
      exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.me__e1a43802

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e1a438021c39f610

4.199 199. odd_power_plus_one_parameterized_factorization

The problem

For natural numbers b , n , and q , if $b = 3q + 2$, prove that there exists a natural number s such that $b^{2n+1} + 1 = 3(q + 1)s$.

Lean statement

```
theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1)
  ↪ * s
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reuses the parent's odd-exponent power-sum divisibility checkpoint, while strengthening the conclusion by retaining the symbolic factor $q+1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the quotient factor: it derives $b + 1 = 3(q + 1)$, constructs the odd-power cofactor, and transports that cofactor into an explicit three-factor equality. This is not recoverable by merely recalling the parent's divisibility proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the factorization: it derives the exact identity $b+1=3(q+1)$ and carries the symbolic cofactor through to an explicit three-factor equality. The parent only supplies divisibility by 3 and does not provide the stronger quotient-sensitive conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the decomposition and proves the stronger exact factorization with the non-trivial factor $q+1$. A memorized proof of mere divisibility by 3 does not supply this quotient construction or the rewritten three-factor equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2057 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_plus_one_parameterized_factorization.extracted_1_1 (b n q : ℕ) (hdecomp : b = 3 * q + 2) :  
  ∃ s, b ^ (2 * n + 1) + 1 = 3 * (q + 1) * s
```

Read back by a model that saw only the Lean. For natural numbers b , n , and q , if $b = 3q + 2$, then there exists a natural number s such that $b^{(2n+1)} + 1 = 3(q+1)s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1)
  <= * s := by
  have hbase : b + 1 = 3 * (q + 1) := by
    omega
  have hodd : Odd (2 * n + 1) := by
    refine ⟨n, ?_⟩
    omega
  have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
    have h := Odd.nat_add_dvd_pow_add_pow b 1 hodd
    simpa using h
  rcases hpow with ⟨s, hs⟩
  refine ⟨s, ?_⟩
  calc
    b ^ (2 * n + 1) + 1 = (b + 1) * s := hs
    _ = 3 * (q + 1) * s := by rw [hbase]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__f9d6e83f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f9d6e83f43f91234

4.200 200. odd_exponent_divisibility_of_base_mod_three

The problem

Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Lean statement

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

Parent. For a natural number n , show that $3 \mid (2^{2n+1} + 1)$.

Parent in Lean

```
theorem induction_divisibility_3div2tooddp1 (n : ℕ) : 3 ∣ 2 ^ (2 * n + 1) + 1 := by
```

What it contributes. Generalizes the parent's odd-exponent divisibility checkpoint from base 2 to an arbitrary base residue class modulo 3, replacing induction with the reusable odd-power divisibility template.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed base 2 to an arbitrary natural base congruent to 2 modulo 3. Its proof requires deriving $3 \mid b+1$ from the modular hypothesis and applying an odd-exponent sum-of-powers divisibility lemma, which is not supplied by the parent's induction argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base from 2 to every natural number congruent to 2 modulo 3. Its proof uses modular decomposition and the odd-exponent factorization theorem, rather than recalling the parent's induction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base 2 to arbitrary natural bases congruent to 2 modulo 3. Its proof requires deriving divisibility of $b+1$ and applying an odd-power divisibility lemma, so the parent's induction proof cannot simply be recalled.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2050 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_exponent_divisibility_of_base_mod_three.extracted_1_1 (b n : ℕ) (hb : b % 3 = 2) :
  3 ∣ b ^ (2 * n + 1) + 1
```

Read back by a model that saw only the Lean. For natural numbers b and n , if the remainder when b is divided by 3 is 2, then 3 divides b raised to the power $2n + 1$ plus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1 := by
  have hbase : 3 ∣ b + 1 := by
    have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
    rw [hb] at hdecomp
    use b / 3 + 1
    omega
  have hodd : Odd (2 * n + 1) := by
    refine ⟨n, ?_⟩
    omega
  have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
    exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
  have hpow2 : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
    simpa using hpow
  exact dvd_trans hbase hpow2
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me__46cb0a18

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46cb0a1847bea7fd

4.201 201. odd_power_divisibility_iff_base_congruent_two

The problem

For natural numbers b and n , prove that $3 \mid b^{2n+1} + 1$ if and only if b leaves remainder 2 upon division by 3.

Lean statement

```
theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

Parent. For a natural number n , show that $3 \mid (2^{2n+1} + 1)$.

Parent in Lean

```
theorem induction_divisibility_3div2tooddp1 (n : ℕ) : 3 ∣ 2 ^ (2 * n + 1) + 1 := by
```

What it contributes. Generalizes the parent's odd-exponent divisibility mechanism from the fixed base 2 to an arbitrary base, while adding a converse residue classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed-base divisibility fact into an iff characterization, requiring a new forward residue analysis to exclude bases congruent to 0 or 1 modulo 3. Its reverse direction also generalizes odd-power divisibility to an arbitrary base, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed divisibility fact into an iff characterization for arbitrary bases. Its forward direction requires modular residue elimination, and its reverse direction uses a general odd-power divisibility argument that the parent's induction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine converse residue characterization for arbitrary bases: the forward direction excludes residues 0 and 1 modulo 3, while the reverse direction uses odd-power divisibility from $3 \mid b + 1$. The parent only handles the fixed base 2 by induction and supplies neither the residue analysis nor the converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2472 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_divisibility_iff_base_congruent_two.extracted_1_1 (b n : ℕ) :  
  3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

Read back by a model that saw only the Lean. *For natural numbers b and n , 3 divides b raised to the power $2n + 1$ plus 1 if and only if the remainder when b is divided by 3 is 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2 := by
  have hepos : 0 < 2 * n + 1 := by omega
  constructor
  · intro hd
    have hzero : (b ^ (2 * n + 1) + 1) % 3 = 0 :=
      Nat.dvd_iff_mod_eq_zero.mp hd
    have hlt : b % 3 < 3 := Nat.mod_lt b (by norm_num)
    have hcases : b % 3 = 0 ∨ b % 3 = 1 ∨ b % 3 = 2 := by omega
    rcases hcases with hres | hres | hres
    · have hbmod : Nat.ModEq 3 b 0 := by
        simp [Nat.ModEq] using hres
      have hp := hbmod.pow (2 * n + 1)
      have hpm0 : b ^ (2 * n + 1) % 3 = 0 := by
        simp [Nat.ModEq, hepos] using hp
      rw [Nat.add_mod, hpm0] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 3 b 1 := by
        simp [Nat.ModEq] using hres
      have hp := hbmod.pow (2 * n + 1)
      have hpm1 : b ^ (2 * n + 1) % 3 = 1 := by
        simp [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm1] at hzero
      norm_num at hzero
    · exact hres
  · intro hres
    have hbase : 3 ∣ b + 1 := by
      refine ⟨b / 3 + 1, ?_⟩
      have hdecomp := Nat.mod_add_div b 3
      omega
    have hodd : Odd (2 * n + 1) := by
      exact ⟨n, by omega⟩
    have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
      exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
    have hpow' : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
      simp using hpow
    exact dvd_trans hbase hpow'
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me__acf5ec57

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint acf5ec572f5a01a5

4.202 202. pow_mod_three_period_two

The problem

For every natural base b with $b \equiv 2 \pmod{3}$ and every natural number n , shifting the exponent by two preserves its residue modulo 3.

Lean statement

```
theorem pow_mod_three_period_two (b n : ℕ) (hb : b % 3 = 2) : Nat.ModEq 3 (b ^ (n + 2)) (b ^ n) := by
```

Parent. For every natural number n , shifting the exponent of 2 by two preserves its residue modulo 3.

Parent in Lean

```
theorem pow_two_mod_three_period_two (n : ℕ) : Nat.ModEq 3 (2 ^ (n + 2)) (2 ^ n) := by
```

What it contributes. Supplied the exponent-shift and modular-product strategy, generalized from the fixed base 2 to an arbitrary residue-class base b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: it must derive that every b with $b \% 3 = 2$ has $b^2 \% 3 = 1$, then use that residue law for arbitrary powers. The parent's fixed-base calculation does not supply this argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: proving period two for every b with $b \% 3 = 2$ requires using the hypothesis to establish $b^2 \equiv 1 \pmod{3}$. This reasoning is not supplied by the fixed-base parent proof and is not a recall or decoration variant.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the period-two congruence from the fixed base 2 to every base satisfying $b \equiv 2 \pmod{3}$. Its proof requires transporting congruence through powers and establishing $b^2 \equiv 1$, rather than recalling the parent's fixed-base multiplication argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1769 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem pow_mod_three_period_two.extracted_1_1 (b n : ℕ) (hb : b % 3 = 2) : b ^ (n + 2) ≡ b ^ n [MOD 3]
```

Read back by a model that saw only the Lean. For natural numbers b and n , if b has remainder 2 upon division by 3,

then b raised to the power $n + 2$ is congruent modulo 3 to b raised to the power n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem pow_mod_three_period_two (b n : ℕ) (hb : b % 3 = 2) : Nat.ModEq 3 (b ^ (n + 2)) (b ^ n) := by
  have hbmod : Nat.ModEq 3 b 2 := hb
  have hsq : Nat.ModEq 3 (b ^ 2) (2 ^ 2) := hbmod.pow 2
  have hbase : b ^ 2 % 3 = 1 := by
    simpa [Nat.ModEq] using hsq
  change b ^ (n + 2) % 3 = b ^ n % 3
  rw [pow_add, Nat.mul_mod, hbase]
  simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__mh.me__7de29617

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint c7cdcab0799d9197

4.203 203. quadratic_strict_sublevel_unique_minimizer

The problem

For every positive radius r , there exists a unique real number x such that $x^2 - 5x - 4 < r^2 - \frac{41}{4}$ and, for every real number y satisfying $y^2 - 5y - 4 < r^2 - \frac{41}{4}$, we have $x^2 - 5x - 4 \leq y^2 - 5y - 4$.

Lean statement

```
theorem quadratic_strict_sublevel_unique_minimizer (r : ℝ) (hr : 0 < r) : ∃! x : ℝ, x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧ ∀ y : ℝ, y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 → x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4
```

Parent. For every positive radius and every real number x , the strict quadratic sublevel condition $x^2 - 5x - 4 < r^2 - 41/4$ holds exactly when x lies in the centered open interval $(5/2 - r, 5/2 + r)$.

Parent in Lean

```
theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) : x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

What it contributes. The parent's positive-radius centered-interval characterization supplies the geometric feasibility checkpoint for the vertex; the child changes the target to unique minimum attainment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine existence-and-uniqueness statement, requiring a new completed-square argument to establish that the vertex minimizes the quadratic over the strict sublevel set. The parent's interval characterization does not supply the minimization or uniqueness proof, and the child does not recall or embed the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine unique-minimizer statement and proves uniqueness via completing the square and nonnegativity, with feasibility requiring the positive-radius hypothesis. It does not recall or reuse the parent's interval-equivalence argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine unique-minimizer statement, requiring quadratic completion, nonnegativity, feasibility of the vertex, and an equality argument; the parent only characterizes the strict sublevel interval. Although it does not use the interval characterization as specified, it is not recall or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1784 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_strict_sublevel_unique_minimizer.extracted_1.1 (r : ℝ) (hr : 0 < r) :
  ∃! x,
    x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧
    ∀ (y : ℝ), y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 → x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4
```

Read back by a model that saw only the Lean. *For every real number r with $r > 0$, there exists exactly one real number x such that $x^2 - 5x - 4 < r^2 - 41/4$, and such that for every real number y satisfying $y^2 - 5y - 4 < r^2 - 41/4$, one has $x^2 - 5x - 4 \leq y^2 - 5y - 4$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_strict_sublevel_unique_minimizer (r : ℝ) (hr : 0 < r) :
  ∃! x : ℝ,
    x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧
    ∀ y : ℝ,
      y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 →
      x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4 := by
  have hcompleted_vertex :
    (5 / 2 : ℝ) ^ 2 - 5 * (5 / 2) - 4 = -41 / 4 := by
    ring
  have hvertex_feasible :
    (5 / 2 : ℝ) ^ 2 - 5 * (5 / 2) - 4 < r ^ 2 - 41 / 4 := by
    have hr2 : 0 < r ^ 2 := sq_pos_of_pos hr
    linarith [hcompleted_vertex]
  have hcompleted : ∀ y : ℝ,
    y ^ 2 - 5 * y - 4 = (y - 5 / 2) ^ 2 - 41 / 4 := by
    intro y
    ring
  refine (5 / 2, ⟨hvertex_feasible, ?_⟩, ?_)
  · intro y hy
    have hnonneg : 0 ≤ (y - 5 / 2) ^ 2 := sq_nonneg (y - 5 / 2)
    rw [hcompleted y, hcompleted_vertex]
    linarith
  · intro z hz
    rcases hz with ⟨hzfeasible, hzmin⟩
    have hmin_at_vertex := hzmin (5 / 2) hvertex_feasible
    rw [hcompleted z, hcompleted_vertex] at hmin_at_vertex
    have hnonneg : 0 ≤ (z - 5 / 2) ^ 2 := sq_nonneg (z - 5 / 2)
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_101__me.mh.me__e8655936

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e19e254c4bf2c1e0

4.204 204. circle_x_projection_noncentral

The problem

Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$ at a point whose y -coordinate is not 3? Show that they are exactly the numbers in $(-9, 1)$.

Lean statement

```
theorem circle_x_projection_noncentral (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔
  (-9 < x ∧ x < 1)
```

Parent. Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Parent in Lean

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

What it contributes. Reuses the parent circle equation and completed-square algebra, while strengthening the projection bounds through the hypothesis $y \neq 3$ and an explicit positive-square boundary obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes the circle projection by excluding the central point, changing the endpoint bounds from non-strict to strict. Its proof requires strict positivity of $(y-3)^2$ and of the square root, which the parent proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added condition $y \neq 3$ changes the projection from a closed interval to its interior. The proof genuinely needs strict square positivity and a strictly positive square-root witness, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes the projection to noncentral points, forcing strict inequalities and a strictly positive square-root witness. The parent only establishes the closed interval and does not supply the endpoint exclusion or noncentrality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2629 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_x_projection_noncentral.extracted_1_1 (x : ℝ) :  
  (∃ y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔ -9 < x ∧ x < 1
```

Read back by a model that saw only the Lean. For every real number x , there exists a real number y such that $x^2 + 8x + y^2 - 6y = 0$ and $y \neq 3$ if and only if $-9 < x$ and $x < 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_x_projection_noncentral (x : ℝ) :  
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔  
    (-9 < x ∧ x < 1) := by  
  constructor  
  · rintro ⟨y, hy, hne⟩  
    have hsq : 0 < (y - 3) ^ 2 := sq_pos_of_ne_zero (sub_ne_zero.mpr hne)  
    have hquad : (x + 4) ^ 2 < 25 := by  
      nlinarith [hy, hsq]  
    constructor  
    · by_contra h  
      have hx : x ≤ -9 := le_of_not_gt h  
      have hp : 0 ≤ (x + 9) * (x - 1) := by  
        apply mul_nonneg_of_nonpos_of_nonpos  
        · linarith  
        · linarith  
      nlinarith [hp, hquad]  
    · by_contra h  
      have hx : 1 ≤ x := le_of_not_gt h  
      have hp : 0 ≤ (x + 9) * (x - 1) := by  
        apply mul_nonneg  
        · linarith  
        · linarith  
      nlinarith [hp, hquad]  
  · rintro ⟨hx₁, hx₂⟩  
    have hprod : 0 < (x + 9) * (1 - x) := by  
      apply mul_pos  
      · linarith  
      · linarith  
    have hd : 0 < 25 - (x + 4) ^ 2 := by  
      nlinarith [hprod]  
    have hspos : 0 < Real.sqrt (25 - (x + 4) ^ 2) := Real.sqrt_pos.2 hd  
    refine ⟨3 + Real.sqrt (25 - (x + 4) ^ 2), ?_, ?_⟩  
    · have hs : (Real.sqrt (25 - (x + 4) ^ 2)) ^ 2 = 25 - (x + 4) ^ 2 := by  
      exact Real.sq_sqrt (le_of_lt hd)  
      nlinarith [hs]  
    · nlinarith [hspos]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.me__b2ecfb10

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b2ecfb10dcb7e680

4.205 205. quadratic_y_solvable_parameterized

The problem

For real numbers x , b , and t , the equation $x^2 + 8x + y^2 - 2by = t$ has a real solution exactly when $x^2 + 8x - b^2 \leq t$.

Lean statement

```
theorem quadratic_y_solvable_parameterized (x b t : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔
  x ^ 2 + 8 * x - b ^ 2 ≤ t
```

Parent. For real numbers x and t , the equation $x^2 + 8x + y^2 - 6y = t$ determines a unique real value of y exactly when $x^2 + 8x - 9 = t$.

Parent in Lean

```
theorem circle_unique_y_coordinate_level (x t : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

What it contributes. Uses the parent's completed-square structure and real-parameter level equation, while changing the final obligation from uniqueness to solvability.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces uniqueness at the completed-square level with a genuine real-solvability characterization, parameterizes the linear coefficient by b , and requires a square-root construction for the reverse direction. This is not recoverable by recalling the parent's reflection-and-uniqueness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes unique solvability at the completed-square minimum into unrestricted real solvability, parameterizes the linear coefficient by b , and requires a square-root witness plus nonnegativity reasoning. The parent's reflection and uniqueness argument does not supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes unique solvability at the fixed completed-square level into a parameterized existence characterization for arbitrary b and t . Its reverse direction requires constructing a square-root witness and proving the radicand is nonnegative, reasoning not supplied by the parent's reflection-and-uniqueness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2673 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_y_solvable_parameterized.extracted_1_1 (x b t : ℝ) :  
  (∃ y, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔ x ^ 2 + 8 * x - b ^ 2 ≤ t
```

Read back by a model that saw only the Lean. For all real numbers x , b , and t , there exists a real number y such that $x^2 + 8x + y^2 - 2by = t$ if and only if $x^2 + 8x - b^2 \leq t$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_y_solvable_parameterized (x b t : ℝ) :  
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔  
    x ^ 2 + 8 * x - b ^ 2 ≤ t := by  
  constructor  
  · rintro ⟨y, hy⟩  
    have hnonneg : 0 ≤ (y - b) ^ 2 := sq_nonneg (y - b)  
    nlinarith [hy, hnonneg]  
  · intro hlevel  
    have hradicand : 0 ≤ t - (x ^ 2 + 8 * x - b ^ 2) := by  
      nlinarith  
    have hsqrt : (Real.sqrt (t - (x ^ 2 + 8 * x - b ^ 2))) ^ 2 =  
      t - (x ^ 2 + 8 * x - b ^ 2) := by  
      exact Real.sq_sqrt hradicand  
    refine ⟨b + Real.sqrt (t - (x ^ 2 + 8 * x - b ^ 2)), ?_⟩  
    nlinarith [hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh.me.me__8589c85d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 8589c85d6ee74ec5

4.206 206. circle_unique_y_coordinate_level

The problem

For real numbers x and t , the equation $x^2 + 8x + y^2 - 6y = t$ determines a unique real value of y exactly when $x^2 + 8x - 9 = t$.

Lean statement

```
theorem circle_unique_y_coordinate_level (x t : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

Parent. For which real values of x does the equation $x^2 + 8x + y^2 - 6y = 0$ determine a unique real value of y ? Show that these values are $x = -9$ and $x = 1$.

Parent in Lean

```
theorem circle_unique_y_coordinate (x : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

What it contributes. Reuses the parent's reflection argument and unique-y checkpoint, generalized from level 0 to an arbitrary real parameter t .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the level and replaces the parent's fixed-root characterization with the completed-square condition $x^2 + 8x - 9 = t$. The parameter t is essential, and the proof must establish the new equivalence rather than merely reuse the parent's factorization.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The level parameter t changes the conclusion from two isolated x -values to an exact completed-square condition, and the reverse direction must derive uniqueness from $(y-3)^2 = 0$ under the new level equation. This is a genuine parameterized generalization rather than a wrapper or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the level t and proves the unique- y condition via the new completed-square equation $x^2 + 8x - 9 = t$. Although it reuses the reflection idea, the fixed-level parent does not directly supply the generalized characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2654 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_unique_y_coordinate_level.extracted_1_1 (x t : ℝ) :  
  (∃! y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

Read back by a model that saw only the Lean. For real numbers x and t , there exists exactly one real number y satisfying $x^2 + 8x + y^2 - 6y = t$ if and only if $x^2 + 8x - 9 = t$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_unique_y_coordinate_level (x t : ℝ) :  
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t := by  
  constructor  
  · rintro ⟨y, hy, huniq⟩  
    have hy_reflected : x ^ 2 + 8 * x + (6 - y) ^ 2 - 6 * (6 - y) = t := by  
      nlinarith [hy]  
    have heq : 6 - y = y := huniq (6 - y) hy_reflected  
    have hy3 : y = 3 := by  
      linarith  
    nlinarith [hy, hy3]  
  · intro hlevel  
    refine ⟨3, ?_, ?_⟩  
    · nlinarith [hlevel]  
    · intro y hy  
      have hcompleted : (y - 3) ^ 2 = 0 := by  
        nlinarith [hy, hlevel]  
      nlinarith [hcompleted]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh.me__d1798a1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1798a1d31fc74f9

4.207 207. circle_x_projection

The problem

Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Lean statement

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

Parent. Find the radius of the circle with equation $x^2 + 8x + y^2 - 6y = 0$. Show that it is 5.

Parent in Lean

```
theorem mathd_algebra_107
  (x y : ℝ)
  (h₀ : x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) :
  ∃ a b r : ℝ, 0 < r ∧ (x + a) ^ 2 + (y + b) ^ 2 = r ^ 2 ∧ r = 5 := by
```

What it contributes. Uses the parent's completed-square radius-five identity, extending it to an exact existential characterization of the x -coordinate projection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an existential circle-membership witness into an exact characterization of the entire x -coordinate projection, requiring both interval necessity and a square-root construction for sufficiency. The parent's fixed-center/radius witness proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential circle witness into an exact characterization of the x -coordinate projection. Its necessity argument bounds a quadratic and its sufficiency constructs a square-root witness, neither of which is supplied by the parent's existential parameter choice.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's existential circle-witness claim into an exact characterization of all possible x -coordinates. Its proof requires completing the square, bounding a quadratic, and constructing a witness with a square root, so recalling the parent's short substitution proof is not substantially helpful.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2614 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_x_projection.extracted_1_1 (x : ℝ) : (∃ y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ -9 ≤ x ∧ x ≤ 1
```

Read back by a model that saw only the Lean. For every real number x , there exists a real number y such that $x^2 + 8x + y^2 - 6y = 0$ if and only if $-9 \leq x$ and $x \leq 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1) := by
  constructor
  · rintro ⟨y, hy⟩
    have hsq : 0 ≤ (y - 3) ^ 2 := sq_nonneg (y - 3)
    have hquad : (x + 4) ^ 2 ≤ 25 := by
      nlinarith [hy, hsq]
    constructor
    · by_contra h
      have hx : x < -9 := lt_of_not_ge h
      have hp : 0 < (x + 9) * (x - 1) := by
        apply mul_pos_of_neg_of_neg
        · linarith
        · linarith
      nlinarith [hp, hquad]
    · by_contra h
      have hx : x > 1 := lt_of_not_ge h
      have hp : 0 < (x + 9) * (x - 1) := by
        apply mul_pos
        · linarith
        · linarith
      nlinarith [hp, hquad]
  · intro hx
    have hprod : 0 ≤ (x + 9) * (1 - x) := by
      apply mul_nonneg
      · linarith
      · linarith
    have hd : 0 ≤ 25 - (x + 4) ^ 2 := by
      nlinarith [hprod]
    refine ⟨3 + Real.sqrt (25 - (x + 4) ^ 2), ?_⟩
    have hs : (Real.sqrt (25 - (x + 4) ^ 2)) ^ 2 = 25 - (x + 4) ^ 2 := by
      exact Real.sq_sqrt hd
    nlinarith [hs]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_107__me__68485970

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 68485970de78008b

4.208 208. scaled_shifted_square_range

The problem

For real numbers c, a, b with $c > 0$, let $f(x) = c(x - a)^2 + b$. For every real y , there exists a real x such that $f(x) = y$ if and only if $b \leq y$.

Lean statement

```
theorem scaled_shifted_square_range
  (c a b : ℝ)
  (hc : 0 < c)
  (f : ℝ → ℝ)
  (hf : f = fun x => c * (x - a)^2 + b) :
  ∀ y : ℝ, (∃ x : ℝ, f x = y) ↔ b ≤ y
```

Parent. Let $a \in \mathbb{R}$ and $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = (x - a)^2$ for all $x \in \mathbb{R}$. Prove that for every $y \in \mathbb{R}$, y is a global minimizer of f if and only if $y = a$.

Parent in Lean

```
theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a
```

What it contributes. Preserved the shifted-square minimization checkpoint: the square is minimized at its center and is nonnegative, then extended it to a scaled and translated attained-range characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes minimization into a full range characterization and introduces positive scaling and vertical translation. Its reverse direction requires constructing a preimage using division and `Real.sqrt`, mathematics absent from the parent's minimizer proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from identifying the unique minimizer to characterizing the entire range, while introducing symbolic positive scaling and vertical translation. Its proof requires constructing a square-root witness and handling division by c , reasoning not supplied by the parent's minimizer proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the range characterization to a positively scaled and vertically translated square. Proving the converse requires constructing a square-root witness and using positivity of the scale, reasoning absent from the parent's minimizer proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1755 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_shifted_square_range.extracted_1_1 : ∀ (c a b : ℝ),
  0 < c → ∀ (f : ℝ → ℝ), (f = fun x => c * (x - a) ^ 2 + b) → ∀ (y : ℝ), (∃ x, f x = y) ↔ b ≤ y
```

Read back by a model that saw only the Lean. For all real numbers c , a , and b , if c is positive, then for every function f from the real numbers to the real numbers such that $f(x) = c(x - a)^2 + b$ for all real x , and for every real number y , there exists a real number x with $f(x) = y$ if and only if $b \leq y$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem scaled_shifted_square_range
  (c a b : ℝ)
  (hc : 0 < c)
  (f : ℝ → ℝ)
  (hf : f = fun x => c * (x - a)^2 + b) :
  ∀ y : ℝ, (∃ x : ℝ, f x = y) ↔ b ≤ y := by
  intro y
  rw [hf]
  constructor
  · rintro ⟨x, hx⟩
    have hs : 0 ≤ (x - a)^2 := sq_nonneg (x - a)
    nlinarith
  · intro hy
    have hnonneg : 0 ≤ y - b := sub_nonneg.mpr hy
    have hq : 0 ≤ (y - b) / c := div_nonneg hnonneg (le_of_lt hc)
    have hsqrt : (Real.sqrt ((y - b) / c))^2 = (y - b) / c := Real.sq_sqrt hq
    have hdiv : c * ((y - b) / c) = y - b := by
      field_simp [ne_of_gt hc]
    refine ⟨a + Real.sqrt ((y - b) / c), ?_⟩
    nlinarith [hsqrt, hdiv]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__me.me__a47a2f74

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-gerprint 93d83fb32a180755

4.209 209. shifted_square_unique_minimizer

The problem

Let $a \in \mathbb{R}$ and $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = (x - a)^2$ for all $x \in \mathbb{R}$. Prove that for every $y \in \mathbb{R}$, y is a global minimizer of f if and only if $y = a$.

Lean statement

```
theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. Generalizes the parent’s completion-of-the-square global-minimum argument from fixed vertex 7 to a bounded real parameter and strengthens minimum existence to an iff uniqueness characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes the vertex and changes a single minimum claim into a characterization of all minimizers, requiring both uniqueness and the converse minimization argument. The boundedness hypotheses are unused, but the core proof demands reasoning beyond the parent’s fixed-instance minimum.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed minimizer to an arbitrary real shift and proves the converse uniqueness direction, requiring evaluation at the candidate minimizer and a separate case split. This is not recoverable by simply recalling the parent’s proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the vertex and strengthens the conclusion to a full unique-minimizer characterization. Its reverse direction requires an equality-case argument using evaluation at the parameter, which is not supplied by the parent’s one-sided minimum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed ha_lower, ha_upper
corpus_dedup	hash	unique against 1742 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_square_unique_minimizer.extracted_1_1 : ∀ (a : ℝ) (f : ℝ → ℝ),
  (f = fun x => (x - a) ^ 2) → ∀ (y : ℝ), IsMinOn f Set.univ y ↔ y = a
```

Read back by a model that saw only the Lean. *For every real number a and every function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = (x - a)^2$ for all real x , then for every real number y , y is a global minimizer of f on $\mathbb{R}$ if and only if $y = a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a := by
  intro y
  rw [hf, isMinOn_univ_iff]
  constructor
  · intro hy
    have hya : (y - a)^2 ≤ (a - a)^2 := hy a
    nlinarith [sq_nonneg (y - a)]
  · intro hya
    subst y
    intro x
    nlinarith [sq_nonneg (x - a)]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__me__a9461eb6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7743b44fb2139a32

4.210 210. radical_difference_gcd_candidate_impossible

The problem

Let b be a positive natural number and let k be divisible by both 2 and 3. If the positive integer y is the common-multiple radical difference and its greatest common divisor with 40 is greater than y , show that this is impossible.

Lean statement

```
theorem radical_difference_gcd_candidate_impossible
  (b k y : ℕ)

  (hy : (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3))
  (hidentity : Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hpos : 0 < (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hg : Nat.gcd 40 y > y) :
  False
```

Parent. For natural numbers b, k with $1 \leq b$, $1 \leq k \leq 600$, and k divisible by both 2 and 3, show that $\sqrt{b^k} - \sqrt[3]{b^k} = b^{k/2} - b^{k/3}$, where the quotients are natural-number quotients.

Parent in Lean

```
theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
```

What it contributes. Supplied the common-multiple radical identity checkpoint, used as `hidentity` to rewrite the candidate before the gcd obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child does not require reproving or recalling the parent identity: it uses that identity only as a hypothesis, then derives positivity of the natural candidate and applies the independent fact that a positive natural number's gcd with 40 cannot exceed it. The proof therefore introduces genuinely different gcd reasoning, although the radical setup overlaps an already-kept family.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the certified radical identity only to transfer positivity to the natural-number candidate, then applies the independent fact that a gcd is at most its right argument and derives a contradiction. This gcd-bound argument is not supplied by the parent's radical decomposition proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not proved by replaying the parent: it assumes the radical identity and instead combines positivity with the independent fact that $\gcd(40, y) \leq y$ for positive y . All hypotheses contribute to deriving the contradiction.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2536 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem radical_difference_gcd_candidate_impossible.extracted_1_1 (b k y : ℕ) (hb : 0 < b) (h2 : 2 ∣ k) (h3 : 3 ∣ k)
(hy : ↑y = √(↑b ^ k) - (↑b ^ k) ^ (1 / 3)) (hidentity : √(↑b ^ k) - (↑b ^ k) ^ (1 / 3) = ↑b ^ (k / 2) - ↑b ^ (k / 3))
(hpos : 0 < ↑b ^ (k / 2) - ↑b ^ (k / 3)) (hg : Nat.gcd 40 y > y) : False
```

Read back by a model that saw only the Lean. For natural numbers b, k , and y , assume that b is positive, k is divisible by 2 and by 3, y (viewed as a real number) equals the square root of b^k minus $(b^k)^{1/3}$, and this difference also equals $b^{k/2}$ minus $b^{k/3}$. Assume further that $b^{k/2}$ minus $b^{k/3}$ is positive and that the greatest common divisor of 40 and y is greater than y . Then a contradiction follows; that is, False.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem radical_difference_gcd_candidate_impossible
  (b k y : ℕ)

  (hy : (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3))
  (hidentity : Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hpos : 0 < (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hg : Nat.gcd 40 y > y) :
  False := by
  have hcandidate : (y : ℝ) = (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
    calc
      (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) := hy
      _ = (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := hidentity
  have hypos_real : (0 : ℝ) < y := by
    linarith [hpos, hcandidate]
  have hypos : 0 < y := by
    exact_mod_cast hypos_real
  have hgcd : Nat.gcd 40 y ≤ y := by
    exact Nat.gcd_le_right 40 hypos
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_208__me.me.me__0b049e9b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 164222055c3632b8

4.211 211. bounded_common_multiple_radical_identity

The problem

For natural numbers b, k with $1 \leq b$, $1 \leq k \leq 600$, and k divisible by both 2 and 3, show that $\sqrt{b^k} - \sqrt[3]{b^k} = b^{k/2} - b^{k/3}$, where the quotients are natural-number quotients.

Lean statement

```
theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
  (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
```

Parent. For a natural number b with $1 \leq b \leq 100$, show that $\sqrt{b^6} - \sqrt[3]{b^6} = b^3 - b^2$.

Parent in Lean

```
theorem bounded_radical_identity (b : ℕ) (hb₁ : 1 ≤ b) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) =
  (b : ℝ) ^ 3 - (b : ℝ) ^ 2 := by
```

What it contributes. Reuses the parent's positive perfect-power checkpoints for the square root and cube-root, generalized from exponent 6 to a bounded exponent divisible by both 2 and 3.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the sixth-power identity to every exponent divisible by both 2 and 3, requiring new quotient and divisibility reasoning to derive the square-root and cube-root exponents. The parent's fixed-exponent proof cannot simply be recalled to establish this parameterized result.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from 6 to an arbitrary common multiple of 2 and 3. Its proof must extract divisibility witnesses, establish the quotient identities, and apply generalized square-root and cube-root reasoning, so recalling the parent proof is insufficient.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed sixth-power identity to an arbitrary exponent divisible by both 2 and 3. Its proof introduces divisibility witnesses, derives quotient identities with arithmetic reasoning, and applies the radical arguments parametrically rather than recalling the parent's fixed-exponent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2514 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_common_multiple_radical_identity.extracted_1_1 (b k : ℕ) (hb : 1 ≤ b) (hk : 1 ≤ k) (hk_upper : k ≤ 600)
  (h2 : 2 ∣ k) (h3 : 3 ∣ k) : √(↑b ^ k) - (↑b ^ k) ^ (1 / 3) = ↑b ^ (k / 2) - ↑b ^ (k / 3)
```

Read back by a model that saw only the Lean. For natural numbers b and k , if $1 \leq b$, $1 \leq k$, $k \leq 600$, 2 divides k , and 3 divides k , then the square root of the real number b^k minus the real one-third power of b^k equals $b^{k/2}$ minus $b^{k/3}$, where the exponents $k/2$ and $k/3$ are natural-number quotients.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
  rcases h2 with ⟨a, ha⟩
  rcases h3 with ⟨c, hc⟩
  have hka : k / 2 = a := by
    omega
  have hkc : k / 3 = c := by
    omega
  have hsqrt : Real.sqrt ((b : ℝ) ^ k) = (b : ℝ) ^ a := by
    calc
      Real.sqrt ((b : ℝ) ^ k) = Real.sqrt ((b : ℝ) ^ (2 * a)) := by rw [ha]
      _ = Real.sqrt (((b : ℝ) ^ a) ^ 2) := by
        congr 1
        simp [Nat.mul_comm] using (pow_mul (b : ℝ) a 2)
      _ = |(b : ℝ) ^ a| := by rw [Real.sqrt_sq_eq_abs]
      _ = (b : ℝ) ^ a := by rw [abs_of_nonneg] <|> positivity
  have hcuberoot : ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) = (b : ℝ) ^ c := by
    calc
      ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) = (((b : ℝ) ^ c) ^ 3) ^ ((1 : ℝ) / 3) := by
        congr 1
        rw [hc]
        simp [Nat.mul_comm] using (pow_mul (b : ℝ) c 3)
      _ = (b : ℝ) ^ c := by
        have hroot := Real.pow_rpow_inv_natCast (x := (b : ℝ) ^ c) (n := 3)
        (by positivity) (by norm_num)
        convert hroot using 1 <|> norm_num
      _ = (b : ℝ) ^ c := by
        rw [hsqrt, hcuberoot, hka, hkc]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_208__me__9013b35e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 3a92415ad8f828ba

4.212 212. factorization_identity

The problem

Show that $(\sqrt{1,000,000} - \sqrt[3]{1,000,000})(\sqrt{1,000,000} + \sqrt[3]{1,000,000}) = 990,000$.

Lean statement

```
theorem factorization_identity : (Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3)) * (Real.sqrt 1000000 + 1000000 ^ ((1 : ℝ) / 3)) = 990000
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires independently evaluating both the square root and real cube-root power, then proving a new product identity. The parent's difference equation alone does not determine the needed sum factor.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves separate square-root and real cube-root evaluations, then establishes a new difference-of-squares factorization. The parent's single evaluated difference does not supply the individual values or the product obligation, so recalling its proof does not substantially solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves separate evaluations of the square root and real cube-root power, then establishes a new product identity; the parent's difference equation alone does not supply this factorization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1713 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem factorization_identity.extracted_1_1 :
  (√1000000 - 1000000 ^ (1 / 3)) * (√1000000 + 1000000 ^ (1 / 3)) = 990000
```

Read back by a model that saw only the Lean. *The product of the difference and sum of the square root of 1,000,000 and 1,000,000 raised to the power 1/3 equals 990,000.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem factorization_identity : (Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3)) * (Real.sqrt 1000000 + 1000000 ^ ((1 : ℝ) / 3)) = 990000
↪ := by
  have hsqrt : Real.sqrt (1000000 : ℝ) = 1000 := by
    norm_num
  have hpow : (1000000 : ℝ) ^ ((1 : ℝ) / 3) = 100 := by
    norm_num [Real.rpow_def_of_pos]
  rw [hsqrt, hpow]
  norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__me__dab0e446

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint dab0e446d87a110c

4.213 213. mathd_algebra_214_bounded_height_fiber

The problem

For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Lean statement

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Parent. For a quadratic real function attaining an extremum at 2 with $f 2 = 3$ and $f 4 = 4$, the equality $f x = 3$ holds exactly when $x = 2$.

Parent in Lean

```
theorem mathd_algebra_214_extremum_eq_three
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x : ℝ, f x = 3 ↔ x = 2
```

What it contributes. Supplied the extremum-based derivation of $b = -4a$, the parameter recovery $a = 1/4$ and $c = 4$, and the completed-square normalization used before solving the generalized height fiber.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely characterizes every level set of the quadratic, requiring derivation of the explicit vertex-centered identity and algebraic equivalence for arbitrary h . This is not recoverable by merely recalling the parent's fixed-level equivalence, and h is materially used.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed level 3 to an arbitrary real height and classifies every fiber via $(x-2)^2=4(h-3)$. Proving this requires deriving the full completed-square form and comparing with an arbitrary h , not merely reusing the parent's equality-at-the-vertex conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child classifies every level set of the quadratic via the squared displacement from the vertex, requiring new algebra beyond the parent's single-level equality claim. The arbitrary real height h makes it stronger than the planned bounded version, not a recall or decorative wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2082 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_bounded_height_fiber.extracted_1_1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
    IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (h x : ℝ), f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c and every function f from the real numbers to the real numbers, if for every real number x we have $f(x) = ax^2 + bx + c$, if 2 is an extremum of f on the set of all real numbers, if $f(2) = 3$, and if $f(4) = 4$, then for every pair of real numbers h and x , $f(x) = h$ if and only if $(x - 2)^2 = 4(h - 3)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3) := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra h
        have ha' : a = 0 := by nlinarith
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let x : ℝ := 2 - (4 * a + b) / (2 * a)
      have hx : f 2 ≤ f x := by simpa using hmin (Set.mem_univ x)
      rw [h₀ x, h₂] at hx
      have hpoly :
        a * x ^ 2 + b * x + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
          -(4 * a + b) ^ 2 / (4 * a) := by
        dsimp [x]
        field_simp [ha_ne]
        ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hx, e2, hpoly]
```

```

have hprod :  $-(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 :=$  by
  field_simp [ha_ne]
have hd :  $(4 * a + b) ^ 2 = 0 :=$  by
  have hp := mul_nonneg hdiff (le_of_lt (by nlinarith :  $0 < 4 * a$ ))
    nlinarith [hprod]
nlinarith
• have h1 :  $f \ 1 \leq f \ 2 :=$  by simp using hmax (Set.mem_univ (1 :  $\mathbb{R}$ ))
have h3' :  $f \ 3 \leq f \ 2 :=$  by simp using hmax (Set.mem_univ (3 :  $\mathbb{R}$ ))
rw [h0 1, h2] at h1
rw [h0 3, h2] at h3'
have e2 := h0 2
have e4 := h0 4
rw [h2] at e2
rw [h3] at e4
have ha :  $a < 0 :=$  by
  by_contra h
  have ha' :  $a = 0 :=$  by nlinarith
  nlinarith [h1, h3', e2, e4]
have ha_ne :  $a \neq 0 :=$  ne_of_lt ha
let x :  $\mathbb{R} := 2 - (4 * a + b) / (2 * a)$ 
have hx :  $f \ x \leq f \ 2 :=$  by simp using hmax (Set.mem_univ x)
rw [h0 x, h2] at hx
have hpoly :
   $a * x ^ 2 + b * x + c - (a * (2 : \mathbb{R}) ^ 2 + b * 2 + c) =$ 
   $-(4 * a + b) ^ 2 / (4 * a) :=$  by
  dsimp [x]
  field_simp [ha_ne]
  ring
have hdiff :  $-(4 * a + b) ^ 2 / (4 * a) \leq 0 :=$  by
  nlinarith [hx, e2, hpoly]
have hprod :  $-(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 :=$  by
  field_simp [ha_ne]
have hd :  $(4 * a + b) ^ 2 = 0 :=$  by
  have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith :  $4 * a < 0$ ))
    nlinarith [hprod]
nlinarith
have ha :  $a = (1 : \mathbb{R}) / 4 :=$  by
  have e2 := h0 2
  have e4 := h0 4
  rw [h2] at e2
  rw [h3] at e4
  nlinarith [e2, e4, hb]
have hc :  $c = 4 :=$  by
  have e2 := h0 2
  rw [h2] at e2
  nlinarith [e2, ha, hb]
have hform :  $\forall x : \mathbb{R}, f \ x = (x - 2) ^ 2 / 4 + 3 :=$  by
  intro x
  rw [h0 x, hb, ha, hc]
  ring
intro x
rw [hform x]
constructor
• intro hx
  nlinarith
• intro hx
  nlinarith

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me__5125f923

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c64bc860433ce35c

4.214 214. quadratic_bounded_inverse

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = 3 + (x - 2)^2/4$ for every real x . For every real h with $3 \leq h \leq 4$, there exists a unique $x \in [2, 4]$ such that $f(x) = h$.

Lean statement

```
theorem quadratic_bounded_inverse
  (f : ℝ → ℝ)
  (hf : ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4) :
  ∀ h : ℝ, 3 ≤ h → h ≤ 4 →
    ∃! x : ℝ, x ∈ Set.Icc 2 4 ∧ f x = h
```

Parent. For every real x , the quadratic function satisfies $f(x) = 3 + (x - 2)^2/4$.

Parent in Lean

```
theorem quadratic_completed_square
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4
```

What it contributes. Supplies the completed-square law used for both witness construction and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to a bounded range-and-inverse characterization: it constructs the unique right-branch preimage using square roots and proves uniqueness from the interval restriction. This is genuinely different reasoning from the parent's completed-square identity and is not a forced wrapper or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a bounded range-and-inverse theorem, requiring construction via square roots and proving uniqueness on the right branch. The parent's completed-square derivation does not supply this existence-and-injectivity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a quadratic to proving bounded range coverage and uniqueness of a preimage. Its proof requires constructing the inverse with a square root and proving branch uniqueness, mathematics not supplied by the parent's completed-square argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2091 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_bounded_inverse.extracted_1_1 : ∀ (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = 3 + (x - 2) ^ 2 / 4) → ∀ (h : ℝ), 3 ≤ h → h ≤ 4 → ∃! x, x ∈ Set.Icc 2 4 ∧ f x = h
```

Read back by a model that saw only the Lean. *For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ satisfying $f(x) = 3 + (x - 2)^2/4$ for every real number x , and for every real number h with $3 \leq h \leq 4$, there exists exactly one real number x such that $2 \leq x \leq 4$ and $f(x) = h$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_bounded_inverse
  (f : ℝ → ℝ)
  (hf : ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4) :
  ∀ h : ℝ, 3 ≤ h → h ≤ 4 →
  ∃! x : ℝ, x ∈ Set.Icc 2 4 ∧ f x = h := by
  intro h hh3 hh4
  have hnonneg : 0 ≤ h - 3 := by
    linarith
  let y : ℝ := Real.sqrt (h - 3)
  have hy0 : 0 ≤ y := by
    dsimp [y]
    exact Real.sqrt_nonneg _
  have hy_sq : y ^ 2 = h - 3 := by
    dsimp [y]
    exact Real.sq_sqrt hnonneg
  have hy1 : y ≤ 1 := by
    nlinarith [hy_sq]
  let x : ℝ := 2 + 2 * y
  have hxmem : x ∈ Set.Icc 2 4 := by
    constructor
    · dsimp [x]
      nlinarith
    · dsimp [x]
      nlinarith
  have hxeq : f x = h := by
    rw [hf x]
    dsimp [x]
    nlinarith [hy_sq]
  refine ⟨x, ⟨hxmem, hxeq⟩, ?_⟩
  intro z hz
  have hzmem : z ∈ Set.Icc 2 4 := hz.1
  have hzeq : f z = h := hz.2
  have hzsquare : (z - 2) ^ 2 = 4 * (h - 3) := by
    rw [hf z] at hzeq
    nlinarith [hzeq]
  have hzleft : 0 ≤ z - 2 := by
    exact sub_nonneg.mpr hzmem.1
  dsimp [x]
  nlinarith [hsquare, hy_sq]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_214__mh.me__20a18ab2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20a18ab2df106de0

4.215 215. bounded_geometric_term_law

The problem

A geometric sequence begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$. Determine its common ratio and show that, for every index n with $0 \leq n \leq 5$, the n th term is $\frac{27}{125} \left(\frac{5}{3}\right)^n$.

Lean statement

```
theorem bounded_geometric_term_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  d = 5 / 3 ∧ ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. Retains the parent sequence, recurrence, and initial values; generalizes the parent's fixed sixth-term computation into a bounded quantified law, with the ratio determination as an explicit checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces the isolated computation of a_5 with a bounded universal formula for every $n \leq 5$. Its induction over n is genuinely new reasoning, and recalling the parent's sequential calculation does not directly establish the law.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the isolated computation of a_5 with a bounded law for every $n \leq 5$, proved by induction from the initial terms. Although determining d reuses the parent's initial algebra, the induction argument and omission of the parent's a_2 hypothesis require new reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the isolated sixth-term computation with a bounded law for every term through index 5. Proving that law requires induction and is not obtained by recalling the parent's finite chain of calculations.

Check

By

Result

statement_type_check

lean

type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2673 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_geometric_term_law.extracted_1_1 (a : ℕ → ℚ) (d : ℚ) (h₀ : ∀ (n : ℕ), a (n + 1) = a n * d)
(h₁ : a 0 = 27 / 125) (h₂ : a 1 = 9 / 25) : d = 5 / 3 ∧ ∀ n ≤ 5, a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. *Let a be a function from the natural numbers to the rational numbers, and let d be a rational number. Suppose that for every natural number n , $a(n + 1) = a(n) \cdot d$, that $a(0) = 27/125$, and that $a(1) = 9/25$. Then $d = 5/3$, and for every natural number n with $n \leq 5$, $a(n) = (27/125) \cdot (5/3)^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_geometric_term_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  d = 5 / 3 ∧ ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  have hlaw : ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
    intro n hn
    induction n with
    | zero =>
      norm_num [h₁]
    | succ n ih =>
      have hs := h₀ n
      rw [hd] at hs
      rw [ih (by omega)] at hs
      simpa [pow_succ, mul_assoc, mul_left_comm, mul_comm] using hs
  exact ⟨hd, hlaw⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__360cecbc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ebb5b4e6ea5a5c76

4.216 216. bounded_recurrence_transfer

The problem

Let a and b be rational sequences and let d be rational. If for every natural number n , $a(n+1) = a(n)d$ and $b(n+1) = b(n)d$, and $a(0) = b(0)$, prove that $a(n) = b(n)$ for every natural number $n \leq 5$.

Lean statement

```
theorem bounded_recurrence_transfer
  (a b : ℕ → ℚ)
  (d : ℚ)
  (ha : ∀ n, a (n + 1) = a n * d)
  (hb : ∀ n, b (n + 1) = b n * d)
  (h₀ : a 0 = b 0) :
  ∀ n, n ≤ 5 → a n = b n
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{25}{9}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. Preserves the parent's rational multiplicative recurrence setting while changing the proof role from one-sequence endpoint evaluation to bounded uniqueness for two sequences.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a fixed endpoint computation with a genuine transfer theorem for two arbitrary rational sequences. Its induction proves equality is preserved by a shared recurrence, reasoning not supplied by the parent's ratio computation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a fixed endpoint computation with a transfer theorem for two arbitrary rational recurrences. Its induction propagates equality from the shared initial value, requiring reasoning absent from the parent's coefficient calculation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single fixed endpoint computation with a transfer theorem for two arbitrary rational recurrences. Its induction argument establishes equality through the bound from a shared initial value and recurrence, which is not supplied by the parent's ratio computation.

Check

statement_type_check

By

lean

Result

type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2046 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_recurrence_transfer.extracted_1_1 : ∀ (a b : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) → (∀ (n : ℕ), b (n + 1) = b n * d) → a 0 = b 0 → ∀ n ≤ 5, a n = b n
```

Read back by a model that saw only the Lean. *For all functions a and b from the natural numbers to the rational numbers and every rational number d , if for every natural number n , $a(n+1) = a(n) \cdot d$, if for every natural number n , $b(n+1) = b(n) \cdot d$, and if $a(0) = b(0)$, then for every natural number n with $n \leq 5$, $a(n) = b(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_recurrence_transfer
  (a b : ℕ → ℚ)
  (d : ℚ)
  (ha : ∀ n, a (n + 1) = a n * d)
  (hb : ∀ n, b (n + 1) = b n * d)
  (h₀ : a 0 = b 0) :
  ∀ n, n ≤ 5 → a n = b n := by
  intro n
  induction n with
  | zero =>
    intro _
    exact h₀
  | succ n ih =>
    intro hn
    have hprev : a n = b n := ih (by omega)
    have ha_step : a (Nat.succ n) = a n * d := by
      simpa [Nat.succ_eq_add_one] using ha n
    have hb_step : b (Nat.succ n) = b n * d := by
      simpa [Nat.succ_eq_add_one] using hb n
    calc
      a (Nat.succ n) = a n * d := ha_step
      _ = b n * d := by rw [hprev]
      _ = b (Nat.succ n) := by symm; exact hb_step
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__40e78e5c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 40e78e5c41e17eb3

4.217 217. bounded_geometric_law

The problem

Prove that if (a_n) is a sequence of rational numbers and d is rational such that $a_{n+1} = a_n d$ for every natural number n , with $a_0 = \frac{27}{125}$ and $a_1 = \frac{9}{25}$, then for every natural number $n \leq 5$, $a_n = \frac{27}{125} \left(\frac{5}{3}\right)^n$.

Lean statement

```
theorem bounded_geometric_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 1 / 25 := by
```

What it contributes. Retains the parent recurrence and rational initial data, using its ratio checkpoint $d = 5 / 3$ while generalizing the terminal computation to a bounded quantified law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces a single terminal value with a bounded closed-form law and requires induction over the index. Although the ratio derivation is shared with the parent, the parent's finite unrolling does not supply the universal recurrence argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's single terminal computation with a bounded closed-form law and proves it by induction over the index. The parent's finite unfolding argument does not supply this universal statement directly, and the hypotheses are substantively used to determine the ratio.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's single terminal evaluation with a bounded closed-form law and requires induction over the index, while deriving the ratio from the initial two values. This is not recall of the parent's forward three-step calculation and is mathematically distinct from the kept recurrence-transfer child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2040 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_geometric_law.extracted_1_1 : ∀ (a : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) → a 0 = 27 / 125 → a 1 = 9 / 25 → ∀ n ≤ 5, a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. For every function a from the natural numbers to the rational numbers and every rational number d , if for every natural number n we have $a(n + 1) = a(n) \cdot d$, if $a(0) = 27/125$, and if $a(1) = 9/25$, then for every natural number n with $n \leq 5$, $a(n) = (27/125) \cdot (5/3)^n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_geometric_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  intro n hn
  induction n with
  | zero =>
    norm_num [h₁]
  | succ n ih =>
    have hnle : n ≤ 5 := Nat.le_of_succ_le hn
    rw [h₀ n, ih hnle, hd]
    simp [pow_succ]
    ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__8bb838c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46495cc8b496d3a7

4.218 218. reciprocal_equation_unique_solution

The problem

Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Lean statement

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

Parent. Let a, b be positive real numbers. If $a + \frac{1}{x} = \frac{b}{x}$, prove that $x = \frac{b-1}{a}$; in particular, when $a = 3$ and $b = 7$, $x = 2$.

Parent in Lean

```
theorem reciprocal_parameter_uniqueness (a b x : ℝ) (ha : 0 < a) (h : a + 1 / x = b / x) : x = (b - 1) / a ∧ ((a = 3 ∧ b = 7) → x = 2)
  ↪ := by
```

What it contributes. Retains the parent's reciprocal-equation algebra and denominator-clearing argument, while changing the target to an existence-and-uniqueness characterization for arbitrary nonzero a .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the parent's conditional solution formula into an existence-and-uniqueness characterization, including proving that no nonzero solution exists when $b=1$. This requires new reasoning beyond recalling the parent's derivation, while retaining the parameterized coefficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the claim to an existence-and-uniqueness characterization and weakens the coefficient hypothesis from positivity to nonzeroness. Proving existence, nonzero witness validity, and uniqueness requires reasoning not supplied by the parent's direct solution formula.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine existence-and-uniqueness characterization: for arbitrary nonzero a , a nonzero solution exists exactly when $b \neq 1$, and uniqueness must be proved. The parent only derives a formula from an already-given solution under $a > 0$, so its proof does not supply the child's converse or witness construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2453 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equation_unique_solution.extracted_1_1 (a b : ℝ) (ha : a ≠ 0) :  
  (b ≠ 1 ↔ ∃! x, x ≠ 0 ∧ a + 1 / x = b / x) ∧ (b ≠ 1 → ∀ (x : ℝ), x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Read back by a model that saw only the Lean. For real numbers a and b , assuming a is nonzero, b is not equal to 1 if and only if there exists exactly one nonzero real number x satisfying $a + 1/x = b/x$; moreover, if b is not equal to 1, then every nonzero real number x satisfying $a + 1/x = b/x$ is equal to $(b - 1)/a$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :  
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧  
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by  
  have hcharacterization : b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x := by  
    constructor  
    · intro hb  
      have hnum : b - 1 ≠ 0 := sub_ne_zero.mpr hb  
      have hx0 : (b - 1) / a ≠ 0 := div_ne_zero hnum ha  
      have heq : a + 1 / ((b - 1) / a) = b / ((b - 1) / a) := by  
        field_simp [hx0, ha]  
      ring  
      refine ⟨(b - 1) / a, ⟨hx0, heq⟩, ?_⟩  
      intro y hy  
      rcases hy with ⟨hy0, hyeq⟩  
      have hyeq' := hyeq  
      field_simp [hy0] at hyeq'  
      field_simp [ha]  
      nlinarith [hyeq']  
    · intro hex hb  
      rcases hex with ⟨x, ⟨hx0, hxeq⟩, huniq⟩  
      have hxeq' := hxeq  
      field_simp [hx0] at hxeq'  
      rw [hb] at hxeq'  
      have hax : a * x = 0 := by  
        nlinarith [hxeq']  
      have haz : a = 0 := (mul_eq_zero.mp hax).resolve_right hx0  
      exact ha haz  
  have hformula : b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a := by  
    intro hb x hx  
    rcases hx with ⟨hx0, hxeq⟩  
    have hxeq' := hxeq  
    field_simp [hx0] at hxeq'  
    field_simp [ha]  
    nlinarith [hxeq']  
  exact ⟨hcharacterization, hformula⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me__3c0a1ee0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3c0a1ee08cd89089

4.219 219. coupled_geometric_reciprocal_parameters

The problem

A geometric sequence begins with nonzero real terms p, q and has common ratio d . If a, b satisfy $a + \frac{1}{d} = \frac{b}{d}$, prove that $b = 1 + \frac{q}{p}a$.

Lean statement

```
theorem coupled_geometric_reciprocal_parameters (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0) (h0 : u 0 = p) (h1 : u 1 = q) (hrec : ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + (q / p) * a
```

Parent. A geometric sequence begins with 3, 5 and has common ratio d . If positive real numbers a, b satisfy $a + \frac{1}{d} = \frac{b}{d}$, prove that $b = 1 + \frac{5a}{3}$.

Parent in Lean

```
theorem coupled_geometric_reciprocal_coefficients (a b d : ℝ) (u : ℕ → ℝ) (h0 : u 0 = 3) (h1 : u 1 = 5) (hrec : ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + 5 * a / 3
```

What it contributes. Generalizes the parent's first-step ratio recovery and reciprocal-equation simplification from fixed initial terms 3 and 5 to symbolic nonzero parameters p and q .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the initial geometric terms: it must derive the symbolic ratio $d = q/p$ and establish $d \neq 0$ before transforming the reciprocal equation. This proof cannot be obtained by recalling the parent's fixed 3-to-5 calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed values 3 and 5 are replaced by symbolic nonzero parameters, requiring derivation of $d = q/p$ and separately establishing $d \neq 0$ before transforming the reciprocal equation. This is not recoverable by recalling the parent's specialized proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the initial geometric terms and must derive the symbolic ratio $d = q/p$, establish $d \neq 0$, and solve the reciprocal equation algebraically. The parent's fixed-value normalization does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2465 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_geometric_reciprocal_parameters.extracted_1_1 (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0)
  (h0 : u 0 = p) (h1 : u 1 = q) (hrec : ∀ (n : ℕ), u (n + 1) = d * u n) (h : a + 1 / d = b / d) :
  b = 1 + q / p * a
```

Read back by a model that saw only the Lean. For real numbers a, b, d, p , and q , and a sequence u of real numbers indexed by the natural numbers, suppose that p is nonzero, q is nonzero, $u(0) = p$, $u(1) = q$, and for every natural number n , $u(n + 1) = d \cdot u(n)$. If $a + 1/d = b/d$, then $b = 1 + (q/p) \cdot a$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_geometric_reciprocal_parameters (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0) (h0 : u 0 = p) (h1 : u 1 = q) (hrec :
  ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + (q / p) * a := by
  have hstep : u 1 = d * u 0 := by
    simp using hrec 0
  have hratio : d = q / p := by
    apply (eq_div_iff hp).2
    rw [← h1, ← h0]
    exact hstep.symm
  have hd : d ≠ 0 := by
    intro hd0
    have hzero : u 1 = 0 := by
      calc
        u 1 = d * u 0 := hstep
        _ = 0 := by rw [hd0]; simp
    apply hq
    rw [← h1]
    exact hzero
  have hcoeff : b = 1 + a * d := by
    field_simp [hd] at h
    linarith
  calc
    b = 1 + a * d := hcoeff
    _ = 1 + (q / p) * a := by rw [hratio]; ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_251__me__80e0f764__x__mathd_algebra_234__xe.me__0801ad48

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0801ad48a199c288

4.220 220. unique_reciprocal_solution

The problem

For real numbers a, b, c with $a \neq 0$ and $c - b \neq 0$, the equation $a + \frac{b}{x} = \frac{c}{x}$ has exactly one nonzero real solution. Show that this solution is $x = \frac{c-b}{a}$.

Lean statement

```
theorem unique_reciprocal_solution (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :
  (∃! x : ℝ, x ≠ 0 ∧ a + b / x = c / x) ∧
  ∀ x : ℝ, x ≠ 0 → a + b / x = c / x → x = (c - b) / a
```

Parent. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h1 : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. Retains the parent's reciprocal-equation strategy of establishing a nonzero denominator, clearing denominators, and finishing by linear arithmetic, while parameterizing all constants and adding existence and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal equation and proves a symbolic nonzero candidate, existence, uniqueness, and the universal solution formula. Its denominator clearing and use of the essential hypotheses require reasoning beyond recalling the parent's isolated numerical proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the isolated numerical equation to arbitrary real coefficients and proves both existence and uniqueness of the nonzero solution, including the symbolic formula. Its candidate nonzeroness and uniqueness derivation are substantive obligations not supplied by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation and proves both existence and uniqueness of the symbolic nonzero solution. It requires candidate nonzeroness, verification, denominator clearing, and a general algebraic uniqueness argument beyond recalling the parent's isolated calculation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2569 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_reciprocal_solution.extracted_1_1 (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :  
  (∃! x, x ≠ 0 ∧ a + b / x = c / x) ∧ ∀ (x : ℝ), x ≠ 0 → a + b / x = c / x → x = (c - b) / a
```

Read back by a model that saw only the Lean. For real numbers a , b , and c , if a is nonzero and $c - b$ is nonzero, then there exists exactly one nonzero real number x such that $a + b/x = c/x$; moreover, every real number x satisfying $x \neq 0$ and $a + b/x = c/x$ is equal to $(c - b)/a$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_reciprocal_solution (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :  
  (∃! x : ℝ, x ≠ 0 ∧ a + b / x = c / x) ∧  
    ∀ x : ℝ, x ≠ 0 → a + b / x = c / x → x = (c - b) / a := by  
  have hcandidate : (c - b) / a ≠ 0 := div_ne_zero hcb ha  
  have hverify : a + b / ((c - b) / a) = c / ((c - b) / a) := by  
    field_simp [ha, hcb] <=> ring  
  constructor  
  · refine ⟨(c - b) / a, ⟨hcandidate, hverify⟩, ?_⟩  
    intro y hy  
    rcases hy with ⟨hy0, hyEq⟩  
    have hcleared := hyEq  
    field_simp [hy0] at hcleared  
    have hmul : a * y = c - b := by  
      linarith  
    apply (eq_div_iff ha).2  
    nlinarith [hmul]  
  · intro x hx hEq  
    have hcleared := hEq  
    field_simp [hx] at hcleared  
    have hmul : a * x = c - b := by  
      linarith  
    apply (eq_div_iff ha).2  
    nlinarith [hmul]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__me__84df140b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 84df140bbd03007e

4.221 221. bounded_quadratic_parameter_characterization

The problem

For all real numbers b and c with $1 \leq b \leq 5$, the equation $2x^2 + bx + c = 0$ has a real solution if and only if $c \leq \frac{b^2}{8}$.

Lean statement

```
theorem bounded_quadratic_parameter_characterization :
  ∀ b c : ℝ, 1 ≤ b → b ≤ 5 →
    ((∃ x : ℝ, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8) := by
```

Parent. Show that $2x^2 + 5x + c = 0$ has at least one real solution if and only if $c \leq \frac{25}{8}$.

Parent in Lean

```
theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
```

What it contributes. Supplied the quadratic solvability pattern, sharp-square obstruction, and explicit square-root witness; the fixed coefficient 5 was parameterized to b over a bounded positive range.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the linear coefficient: the threshold becomes the symbolic discriminant bound $c \leq b^2/8$, and the proof constructs a root using $\sqrt{(b^2 - 8c)}$. This requires carrying b through the completed-square and root-construction arguments rather than merely reusing the fixed-coefficient conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and proves the symbolic discriminant threshold for arbitrary b , requiring the completed-square argument to survive symbolically rather than merely instantiating the parent at $b = 5$. The bounds on b are unused but do not undermine the substantive generalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and proves the symbolic discriminant threshold using a variable completed square and square root construction. The parent's fixed-coefficient proof does not directly supply this generalized argument, and the bounds are harmlessly redundant rather than the source of the result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1747 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_parameter_characterization.extracted_1_1 : ∀ (b c : ℝ),
  1 ≤ b → b ≤ 5 → ((∃ x, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8)
```

Read back by a model that saw only the Lean. *For all real numbers b and c , if $1 \leq b$ and $b \leq 5$, then there exists a real number x satisfying $2x^2 + bx + c = 0$ if and only if $c \leq b^2/8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_parameter_characterization :
  ∀ b c : ℝ, 1 ≤ b → b ≤ 5 →
    ((∃ x : ℝ, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8) := by
  intro b c hb_lower hb_upper
  have hb_pos : 0 < b := by
    linarith
  constructor
  · rintro ⟨x, hx⟩
    have hsquare : 0 ≤ (4 * x + b) ^ 2 := by
      exact sq_nonneg (4 * x + b)
    nlinarith
  · intro hc
    have hdisc : 0 ≤ b ^ 2 - 8 * c := by
      nlinarith
    have hsqrt : (Real.sqrt (b ^ 2 - 8 * c)) ^ 2 = b ^ 2 - 8 * c := by
      exact Real.sq_sqrt hdisc
    refine ⟨(-b + Real.sqrt (b ^ 2 - 8 * c)) / 4, ?_⟩
    nlinarith [hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__37417bdf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint df675ab28c6ab34d

4.222 222. quadratic_unique_root_iff

The problem

For real numbers a, b, c with $a > 0$, prove that $ax^2 + bx + c = 0$ has exactly one real solution if and only if $c = \frac{b^2}{4a}$.

Lean statement

```
theorem quadratic_unique_root_iff (a b c : ℝ) (ha : 0 < a) :
  (∃! x : ℝ, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a)
```

Parent. For a real number a with $0 < a \leq 10$, what is the largest number c such that $ax^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_real_root_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | ∃ x : ℝ, a * x ^ 2 + 5 * x + c = 0}) : IsGreatest S (25 / (4 * a))
```

What it contributes. Generalizes the parent's positive-leading-coefficient quadratic and reuses its completed-square/vertex argument, while changing the conclusion to a unique-root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely changes the conclusion from a greatest-element characterization to an iff characterization of unique real roots, while also parameterizing the fixed linear coefficient 5 as arbitrary b . Its proof requires a distinct symmetry/uniqueness argument and is not recoverable by recalling the parent.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the extremal-value claim into a unique-real-root characterization. Its proof requires a new symmetry argument showing any unique root must be the vertex, plus a separate uniqueness proof from square nonnegativity; the parent's greatest-element proof does not supply this reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine change of shape: it characterizes the discriminant-zero case by existence and uniqueness of a real root. Its proof uses the reflected second root and proves uniqueness algebraically, rather than recalling the parent's IsGreatest argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2606 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_unique_root_iff.extracted_1_1 (a b c : ℝ) (ha : 0 < a) :  
  (∃! x, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a)
```

Read back by a model that saw only the Lean. For real numbers a , b , and c with $a > 0$, the quadratic equation $ax^2 + bx + c = 0$ has exactly one real solution if and only if $c = b^2/(4a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_unique_root_iff (a b c : ℝ) (ha : 0 < a) :  
  (∃! x : ℝ, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a) := by
  have hane : a ≠ 0 := ne_of_gt ha
  constructor
  · rintro ⟨x, hx, huniq⟩
    have hreflect : a * (-b / a - x) ^ 2 + b * (-b / a - x) + c = 0 := by
      have hid : a * (-b / a - x) ^ 2 + b * (-b / a - x) + c =  
        a * x ^ 2 + b * x + c := by
          field_simp [hane]
          ring
      rw [hid]
      exact hx
    have heq : -b / a - x = x := huniq (-b / a - x) hreflect
    have hvertex : 2 * a * x + b = 0 := by
      field_simp [hane] at heq
      nlinarith [heq]
    have hb : b = -2 * a * x := by
      linarith [hvertex]
    rw [hb] at hx
    apply (eq_div_iff (by nlinarith [ha])).2
    rw [hb]
    nlinarith [hx]
  · intro hc
    refine ⟨-b / (2 * a), ?_, ?_⟩
    · rw [hc]
      field_simp [hane]
      ring
    · intro y hy
      rw [hc] at hy
      have hs : (2 * a * y + b) ^ 2 = 0 := by
        calc
          (2 * a * y + b) ^ 2 =  
            4 * a * (a * y ^ 2 + b * y + b ^ 2 / (4 * a)) := by
              field_simp [hane]
              ring
          _ = 0 := by rw [hy]; ring
      have hlin : 2 * a * y + b = 0 := by
        nlinarith [hs]
      apply (eq_div_iff (by nlinarith [ha])).2
      nlinarith [hlin]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__fce4aeba

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fce4aeba45e34408

4.223 223. quadratic_parameterized_extremum

The problem

For positive real numbers a and b , let S be the set of constants c for which $ax^2 + bx + c = 0$ has a real solution. Show that S has greatest element $\frac{b^2}{4a}$, and that at this value the equation has the unique root $-\frac{b}{2a}$.

Lean statement

```
theorem quadratic_parameterized_extremum (a b : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | ∃ x : ℝ, a * x ^ 2 + b * x + c = 0}) :
  ↪ IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a)
```

Parent. For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

What it contributes. Supplied the discriminant/completed-square extremality pattern; the mutation replaces the fixed coefficient 5 by the positive parameter b and adds root uniqueness at the extremal constant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the fixed constant 25 into b^2 and changes the feasible-set description to existence of a real quadratic root, then proves the additional uniqueness characterization at the extremal constant. Its proof requires completing the square and deriving the zero-square condition, which is not supplied by recalling the parent's direct inequality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both coefficients and changes the set characterization from a linear discriminant inequality to existence of a real quadratic root. Its proof requires completing-square reasoning, constructing a witness, and proving uniqueness of the extremal root, none of which is supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and changes the feasible-set definition from a discriminant inequality to existence of a real root. Its proof requires constructing a root, completing the square, and proving uniqueness via a zero-square argument, none of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2624 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_parameterized_extremum.extracted_1_1 (a b : ℝ) (ha : 0 < a) (hb : 0 < b) (S : Set ℝ)
(hS : S = {c | ∃ x, a * x ^ 2 + b * x + c = 0}) :
IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ (x : ℝ), a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a)
```

Read back by a model that saw only the Lean. For real numbers a and b with $0 < a$ and $0 < b$, and for a set S of real numbers satisfying $S = \{c \in \mathbb{R} \mid \text{there exists } x \in \mathbb{R} \text{ such that } ax^2 + bx + c = 0\}$, the number $b^2/(4a)$ is the greatest element of S , and for every real number x , $ax^2 + bx + b^2/(4a) = 0$ if and only if $x = -b/(2a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_parameterized_extremum (a b : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | ∃ x : ℝ, a * x ^ 2 + b * x + c = 0}) :
  IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a) := by
  rw [hS]
  constructor
  · constructor
    · change ∃ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0
      refine ⟨-b / (2 * a), ?_⟩
      field_simp [ha.ne']
      ring
    · intro c hc
      change ∃ x : ℝ, a * x ^ 2 + b * x + c = 0 at hc
      rcases hc with ⟨x, hx⟩
      have hpos : 0 < 4 * a := by
        nlinarith [ha]
      apply (le_div_iff hpos).2
      nlinarith [sq_nonneg (2 * a * x + b), hx]
  · intro x
    constructor
    · intro hx
      have hclean : 4 * a * (a * x ^ 2 + b * x) + b ^ 2 = 0 := by
        calc
          4 * a * (a * x ^ 2 + b * x) + b ^ 2 = 4 * a * (a * x ^ 2 + b * x + b ^ 2 / (4 * a)) := by
            field_simp [ha.ne']
          _ = 0 := by rw [hx]; ring
      have hsq : (2 * a * x + b) ^ 2 = 0 := by
        nlinarith [hclean]
      have hzero : 2 * a * x + b = 0 := by
        nlinarith [hsq]
      apply (eq_div_iff (by nlinarith [ha] : (2 * a) ≠ 0)).2
      nlinarith [hzero]
    · intro hx
      rw [hx]
      field_simp [ha.ne']
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms.me__1f5d0186

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b1ffb242803e785a

4.224 224. quadratic_parameter_characterization

The problem

Show that $2x^2 + 5x + c = 0$ has at least one real solution if and only if $c \leq \frac{25}{8}$.

Lean statement

```
theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8 : ℝ) := by
```

What it contributes. Supplies the sharp bound $25/8$ and the completed-square inequality used in the necessity direction; the same bound is used to establish discriminant nonnegativity for the converse.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child characterizes the entire parameter range, requiring a constructive real-root argument via the discriminant and square root for every $c \leq 25/8$. The parent only proves that $25/8$ is the greatest attainable value and does not establish this converse.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the greatest-element claim into a full range characterization, requiring the new converse that every $c \leq 25/8$ admits a real root. Its explicit square-root construction is not supplied by the parent's upper-bound argument, so recalling the parent does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's greatest-element claim into an exact characterization of S , requiring a new converse construction of a real root using the discriminant and square root. The parent proof only establishes the upper bound and one extremal witness, so recalling it does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1726 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_parameter_characterization.extracted_1_1 : ∀ (S : Set ℝ),
  S = {c | ∃ x, 2 * x ^ 2 + 5 * x + c = 0} → ∀ (c : ℝ), c ∈ S ↔ c ≤ 25 / 8
```

Read back by a model that saw only the Lean. *For every set S of real numbers, if S is exactly the set of real numbers c for which there exists a real number x satisfying $2x^2 + 5x + c = 0$, then for every real number c , c belongs to S if and only if $c \leq 25/8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
  rw [hS]
  intro c
  simp only [Set.mem_setOf_eq]
  have sharp_bound : ∀ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0 → c ≤ (25 / 8 : ℝ) := by
    intro x hx
    nlinarith [sq_nonneg (4 * x + 5)]
  have root_construction : c ≤ (25 / 8 : ℝ) → ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0 := by
    intro hc
    have hdisc : 0 ≤ 25 - 8 * c := by
      nlinarith
    have hsqrt : (Real.sqrt (25 - 8 * c)) ^ 2 = 25 - 8 * c := by
      exact Real.sq_sqrt hdisc
    refine ((-5 + Real.sqrt (25 - 8 * c)) / 4, ?_)
    nlinarith [hsqrt]
  constructor
  · rintro ⟨x, hx⟩
    exact sharp_bound x hx
  · intro hc
    exact root_construction hc
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__770a1b59

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b17ecd8fe38642b9

4.225 225. ap_quadratic_unique_double_root

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. Prove that the equation $a_4x^2 + 5x + \frac{25}{68} = 0$ has exactly one real solution.

Lean statement

```
theorem ap_quadratic_unique_double_root
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∃! x : ℝ, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the greatest real number c for which $a_4x^2 + 5x + c = 0$ has a real solution? Show that it is

$\frac{25}{68}$.

Parent in Lean

```
theorem coupled_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hS : S = {c | ∃ x : ℝ, a 4 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a 4) : ℝ) := by
```

What it contributes. Uses the parent's AP recurrence derivation to identify $p = 5$, $q = 2$, and the terminal coefficient $a_4 = 17$; replaces the parent's extremal-set conclusion with unique double-root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-set extremum to an explicit existence-and-uniqueness statement for the double root. Uniqueness requires a separate square-nonnegativity argument, so the parent's proof does not directly solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-element characterization of attainable constants to uniqueness of the quadratic root at the extremal coefficient. After deriving the AP coefficient, it requires a distinct global uniqueness argument via completing the square; the parent's greatest-set proof does not directly supply this.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the extremum/set-membership conclusion into an explicit unique-root theorem. Proving uniqueness via the completed-square argument is new reasoning not supplied by the parent's greatest-element proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2642 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ap_quadratic_unique_double_root.extracted_1.1 (p q : ℝ) (a : ℕ → ℝ)
  (h₀ : ∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) : ∃! x, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0
```

Read back by a model that saw only the Lean. Let p and q be real numbers, and let a be a sequence of real numbers indexed by the natural numbers. Suppose that for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$, that $a(1) = p$, that $a(2) = 9$, that $a(3) = 3p - q$, and that $a(4) = 3p + q$. Then there exists exactly one real number x such that $a(4)x^2 + 5x + 25/68 = 0$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem ap_quadratic_unique_double_root
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∃! x : ℝ, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0 := by
  have hA := h₀ 1
  have hB := h₀ 2
  norm_num at hA hB
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hcoef : a 4 = 17 := by
    rw [h₄, hp, hq]
    norm_num
  refine (-5 / 34, ?_, ?_)
  · rw [hcoef]
    norm_num
  · intro y hy
    rw [hcoef] at hy
    nlinarith [sq_nonneg (34 * y + 5)]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.me__06a52718

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 06a5271847415a28

4.226 226. bounded_ap_quadratic

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. If $k \in \{4, 5\}$, what is the greatest real number c for which $a_k x^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a_k}$.

Lean statement

```
theorem bounded_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (k : ℕ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hk : 4 ≤ k ∧ k ≤ 5)
  (hS : S = {c | ∃ x : ℝ, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k)) : ℝ
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the greatest real number c for which $a_4 x^2 + 5x + c = 0$ has a real solution? Show that it is

rac2568.

Parent in Lean

```
theorem coupled_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hS : S = {c | ∃ x : ℝ, a 4 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a 4)) : ℝ := by
```

What it contributes. Uses the parent's arithmetic-progression deductions and quadratic-extremum structure, extended to the bounded adjacent index $k \in \{4, 5\}$ with a new recurrence-derived fifth-term obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bounded index forces a genuine case split between $k=4$ and $k=5$. The $k=5$ branch requires deriving $a(5)=21$ from the arithmetic-progression recurrence and proving a new quadratic extremal case, so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient index from fixed a_4 to bounded a_k and must handle the new case $k=5$, derive $a_5=21$, and split on the index. The parent proof does not supply this case analysis or the additional recurrence computation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded index introduces a genuine new case, $k=5$, requiring derivation of $a(5)=21$ and a separate extremal argument; recalling the parent only handles the $k=4$ branch. This matches the planned bounded generalization and is not merely decorative.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2599 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_ap_quadratic.extracted_1_1 (S : Set ℝ) (p q : ℝ) (a : ℕ → ℝ) (k : ℕ)
  (h₀ : ∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) (hk : 4 ≤ k ∧ k ≤ 5) (hS : S = {c | ∃ x, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k))
```

Read back by a model that saw only the Lean. Let S be a subset of the real numbers, let p and q be real numbers, let a be a sequence of real numbers indexed by the natural numbers, and let k be a natural number. Assume that for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$; that $a(1) = p$; that $a(2) = 9$; that $a(3) = 3p - q$; that $a(4) = 3p + q$; that $4 \leq k$ and $k \leq 5$; and that S is exactly the set of real numbers c for which there exists a real number x satisfying $a(k)x^2 + 5x + c = 0$. Then $25/(4a(k))$ is the greatest element of S .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem bounded_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (k : ℕ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hk : 4 ≤ k ∧ k ≤ 5)
  (hS : S = {c | ∃ x : ℝ, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k) : ℝ) := by
  have hA := h₀ 1
  have hB := h₀ 2
  have hC := h₀ 3
  norm_num at hA hB hC
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  rw [h₄, h₃] at hC
  have hp : p = 5 := by
    linarith [hA, hB]
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hcoef4 : a 4 = 17 := by
    rw [h₄, hp, hq]
```

```

norm_num
have hcoef5 : a 5 = 21 := by
  rw [hp, hq] at hC
  linarith [hC, hcoef4]
have hk_cases : k = 4 ∨ k = 5 := by
  omega
rcases hk_cases with rfl | rfl
· have hpos : 0 < a 4 := by
  rw [hcoef4]
  norm_num
have hquad_bound : ∀ x c : ℝ,
  a 4 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / (4 * a 4) := by
  intro x c hx
  rw [hcoef4] at hx ⊢
  nlinarith [sq_nonneg (34 * x + 5), hpos]
rw [hS]
constructor
· refine ⟨-5 / (2 * a 4), ?_⟩
  rw [hcoef4]
  norm_num
· intro c hc
  rcases hc with ⟨x, hx⟩
  exact hquad_bound x c hx
· have hpos : 0 < a 5 := by
  rw [hcoef5]
  norm_num
have hquad_bound : ∀ x c : ℝ,
  a 5 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / (4 * a 5) := by
  intro x c hx
  rw [hcoef5] at hx ⊢
  nlinarith [sq_nonneg (42 * x + 5), hpos]
rw [hS]
constructor
· refine ⟨-5 / (2 * a 5), ?_⟩
  rw [hcoef5]
  norm_num
· intro c hc
  rcases hc with ⟨x, hx⟩
  exact hquad_bound x c hx

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.me__1f523ee9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1f523ee96233b1a2

4.227 227. classify_terminal_index_bounded

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. If $4 \leq N$, show that among the terms with indices from 1 through N , the value 17 occurs exactly at index 4.

Lean statement

```
theorem classify_terminal_index_bounded
  (p q : ℝ)
  (a : ℕ → ℝ)
  (N : ℕ)

  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 N → (a n = 17 ↔ n = 4)
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. Determine exactly which index among 1, 2, 3, 4 has value 17.

Parent in Lean

```
theorem classify_terminal_index
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 4 → (a n = 17 ↔ n = 4)
```

What it contributes. Supplied the initial arithmetic-progression hypotheses and the endpoint equations used to solve $p=5$ and $q=2$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the conclusion from indices through 4 to an arbitrary endpoint N . Its proof must derive the full arithmetic-sequence formula and solve the resulting equation, rather than recalling the parent's finite case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the classification to arbitrary N and must derive the full affine formula for a by induction. This reasoning is not supplied by the parent's finite interval case split, so recalling the parent would not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the conclusion to an arbitrary bounded endpoint N and must derive the entire arithmetic progression by induction, rather than relying on the parent's finite four-case enumeration. The added N is used substantively, and the proof is not recoverable by simply recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2643 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem classify_terminal_index_bounded.extracted_1.1 : ∀ (p q : ℝ) (a : ℕ → ℝ) (N : ℕ),
  4 ≤ N →
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →
  a 1 = p → a 2 = 9 → a 3 = 3 * p - q → a 4 = 3 * p + q → ∀ n ∈ Finset.Icc 1 N, a n = 17 ↔ n = 4
```

Read back by a model that saw only the Lean. For all real numbers p and q , all functions a from the natural numbers to the real numbers, and all natural numbers N , if $4 \leq N$, if for every natural number n we have $a(n+2) - a(n+1) = a(n+1) - a(n)$, and if $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$, then for every natural number n with $1 \leq n \leq N$, $a(n) = 17$ if and only if $n = 4$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem classify_terminal_index_bounded
  (p q : ℝ)
  (a : ℕ → ℝ)
  (N : ℕ)

  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 N → (a n = 17 ↔ n = 4) := by
  have hA := h₀ 1
  have hB := h₀ 2
  norm_num at hA hB
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith [hA, hB]
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hlaw : ∀ n : ℕ, a (n + 1) = 5 + 4 * (n : ℝ) := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      by_cases hn : n = 0
      · subst n
        rw [h₁, hp]
        norm_num
      · obtain ⟨k, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hn
        by_cases hk : k = 0
        · subst k
          norm_num [h₂]
        · obtain ⟨j, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hk
          have hprev : a (j + 1) = 5 + 4 * (j : ℝ) := ih j (by omega)
          have hcurr : a (j + 2) = 5 + 4 * ((j + 1) : ℝ) := by
            simpa [Nat.add_assoc] using ih (j + 1) (by omega)
          have hr : a (j + 3) - a (j + 2) = a (j + 2) - a (j + 1) := by
```



```

      simpa [Nat.add_assoc] using h₀ (j + 1)
    push_cast at hcurr ⊢
      linarith [hr, hprev, hcurr]
intro n hn
have hn' : 1 ≤ n ∧ n ≤ N := by
  simpa [Finset.mem_Icc] using hn
have hn0 : n ≠ 0 := by omega
obtain ⟨k, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hn0
have hk := hlaw k
constructor
· intro heq
  have hkval : (k : ℝ) = 3 := by
    linarith [heq, hk]
  have hkN : k = 3 := by
    exact_mod_cast hkval
  exact congrArg Nat.succ hkN
· intro heq
  have hk3 : k = 3 := by omega
  subst k
  have h := hlaw 3
  norm_num at h ⊢
  exact h

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.mh.me__87e1cfc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint c69284efcceb600e

4.228 228. bounded_abs_interval

The problem

Prove that for real numbers a and c satisfying $0 < c \leq 3$, the inequality $\frac{1}{5}|9 + ca| < 1$ holds if and only if a lies in the open interval $\left(-\frac{14}{c}, -\frac{4}{c}\right)$.

Lean statement

```
theorem bounded_abs_interval (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :
  (1 / 5 * abs (9 + c * a) < 1) ↔ a ∈ Set.Ioo (-14 / c) (-4 / c)
```

Parent. Solve for a : $\frac{1}{5}|9 + 2a| < 1$. Express your answer in interval notation. Show that it is $(-7, -2)$.

Parent in Lean

```
theorem mathd_algebra_327
  (a : ℝ) :
  (1 / 5 * abs (9 + 2 * a) < 1) ↔ a ∈ Set.Ioo (-7:ℝ) (-2) := by
```

What it contributes. Generalizes the parent's absolute-value-to-open-interval proof by replacing the fixed coefficient 2 with a bounded positive parameter c and using positive-denominator interval scaling in both directions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The coefficient 2 is genuinely replaced by a symbolic positive c , and proving the scaled interval requires division by c and explicit positivity reasoning absent from the parent proof. The upper bound $c \leq 3$ is unnecessary but does not make the child a recall or decoration mutation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the coefficient of $\$a\$$ and must use the new positivity hypothesis on $\$c\$$ to divide inequalities and derive the scaled interval. This is not recoverable by simply recalling the parent's fixed-coefficient proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the coefficient and must use the new positivity hypothesis to divide inequalities by c and derive scaled interval endpoints. The parent's fixed-coefficient linear arithmetic does not directly supply this symbolic division argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2076 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_abs_interval.extracted_1_1 (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :  
  1 / 5 * |9 + c * a| < 1 ↔ a ∈ Set.Ioo (-14 / c) (-4 / c)
```

Read back by a model that saw only the Lean. For real numbers a and c satisfying $0 < c$ and $c \leq 3$, one has $(1/5)$ times the absolute value of $9 + ca$ is less than 1 if and only if a lies in the open interval $(-14/c, -4/c)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_abs_interval (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :  
  (1 / 5 * abs (9 + c * a) < 1) ↔ a ∈ Set.Ioo (-14 / c) (-4 / c) := by  
  constructor  
  · intro h  
    have habs : abs (9 + c * a) < 5 := by  
      nlinarith  
    rw [abs_lt] at habs  
    have hlow : -14 / c < a := by  
      apply (div_lt_iff₀ hc.1).2  
      nlinarith [habs.1]  
    have hupp : a < -4 / c := by  
      apply (lt_div_iff₀ hc.1).2  
      nlinarith [habs.2]  
    exact ⟨hlow, hupp⟩  
  · intro h  
    have hlow : -14 < c * a := by  
      have h' := (div_lt_iff₀ hc.1).1 h.1  
      nlinarith [h']  
    have hupp : c * a < -4 := by  
      have h' := (lt_div_iff₀ hc.1).1 h.2  
      nlinarith [h']  
    have hinterval : -5 < 9 + c * a ∧ 9 + c * a < 5 := by  
      constructor <|> nlinarith  
    rw [← abs_lt] at hinterval  
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_327__me__c47e671f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c47e671f35455c30

4.229 229. unique_positive_quadratic_above_threshold

The problem

For every natural cutoff c with $9 < c \leq 10$, there exists exactly one positive natural number x such that $x^2 + 4x + 4 < c$.

Lean statement

```
theorem unique_positive_quadratic_above_threshold (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) : ∃! x : ℕ, 0 < x ∧ x ^ 2 + 4 * x + 4 < c
```

Parent. For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Parent in Lean

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :  
  S.card = if c ≤ 9 then 0 else 1
```

What it contributes. Reuses the parent's bounded cutoff and positive quadratic predicate, while replacing its cardinality conclusion with an existence-and-uniqueness characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the cardinality conclusion into an explicit uniqueness statement and must prove that every positive witness equals 1. Although it shares the cutoff arithmetic with the parent, the parent's finite-set cardinality proof does not directly supply this $\exists!$ characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the cardinality conclusion into a genuine uniqueness statement and proves that any positive witness must satisfy $y \leq 1$, then $y = 1$. Although the bound $c = 10$ and inequality argument parallel the parent, the parent's finset-cardinality proof does not directly establish this existential uniqueness claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine change of shape from a cardinality equality to an existence-and-uniqueness characterization. Its proof must establish a witness and prove arbitrary positive candidates equal it, rather than merely recalling the parent's Finset-cardinality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1819 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_positive_quadratic_above_threshold.extracted_1_1 (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) :  
  ∃! x, 0 < x ∧ x ^ 2 + 4 * x + 4 < c
```

Read back by a model that saw only the Lean. *For a natural number c satisfying $c \leq 10$ and $9 < c$, there exists exactly one natural number x such that $0 < x$ and $x^2 + 4x + 4 < c$.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_positive_quadratic_above_threshold (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) : ∃! x : ℕ, 0 < x ∧ x ^ 2 + 4 * x + 4 < c := by  
  have hc10 : c = 10 := by omega  
  have h_one : 0 < 1 ∧ 1 ^ 2 + 4 * 1 + 4 < c := by  
    norm_num [hc10]  
  refine ⟨1, h_one, ?_⟩  
  intro y hy  
  have hypos : 0 < y := hy.1  
  have hyineq : y ^ 2 + 4 * y + 4 < c := hy.2  
  have hyle : y ≤ 1 := by  
    by_contra h  
    have hy2 : 2 ≤ y := by omega  
    have hysq : 4 ≤ y ^ 2 := by  
      simpa [pow_two] using Nat.mul_le_mul hy2 hy2  
    omega  
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_405__me.me__9834fd26

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a3852ffd0f770e73

4.230 230. bounded_quadratic_cutoff_cardinality

The problem

For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Lean statement

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S.card = if c ≤ 9 then 0 else 1
```

Parent. For how many positive integer values of x is the sum $x^2 + 4x + 4$ less than 20? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_405 (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) :
  S.card = 2 := by
```

What it contributes. Reused the parent's membership-biconditional encoding and finite-set extensional classification strategy, generalized to a bounded cutoff with a threshold case split.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the cutoff and classifies two threshold regimes, requiring separate arguments for emptiness when $c \leq 9$ and the singleton case $c = 10$. The parent's fixed $c = 20$ proof does not supply this bounded case analysis or its cardinality classification.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the cutoff and classifies both threshold cases, requiring a new case split and cardinality analysis for $c \leq 9$ versus $c = 10$. The parent's fixed-cutoff proof at 20 does not supply this reasoning or conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the cutoff and classifies two threshold regimes, requiring new reasoning for $c \leq 9$ and $c = 10$. The parent's fixed-cutoff proof does not supply this case analysis or the resulting cardinality classification.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1803 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_cutoff_cardinality.extracted_1_1 (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ)
(h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) : S.card = if c ≤ 9 then 0 else 1
```

Read back by a model that saw only the Lean. Let c be a natural number with $c \leq 10$, and let S be a finite set of natural numbers such that, for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < c$. Then the cardinality of S equals 0 if $c \leq 9$, and equals 1 otherwise.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
S.card = if c ≤ 9 then 0 else 1 := by
  by_cases hcut : c ≤ 9
  · have hS : S = ∅ := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hsquare : 1 ≤ x ^ 2 := by
          simpa [pow_two] using Nat.mul_le_mul hx hx
        omega
      · intro hx
        simp at hx
    rw [hS]
    simp [hcut]
  · have hc10 : c = 10 := by omega
    have hS : S = {1} := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hxle : x ≤ 1 := by
          by_contra h
          have hx2 : 2 ≤ x := by omega
          have hsquare : 4 ≤ x ^ 2 := by
            simpa [pow_two] using Nat.mul_le_mul hx2 hx2
          omega
        have hxone : x = 1 := by omega
        simpa [hxone]
      · intro hx
        have hxone : x = 1 := by simpa using hx
        subst x
        norm_num [hc10]
    rw [hS]
    simp [hcut]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_405__me__71e06a20

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e507191f044f7129

4.231 231. translated_square_bounded_min_unique

The problem

For real numbers a, b, c, l, u with $a > 0$ and $l \leq b \leq u$, the function $x \mapsto a(x - b)^2 + c$ has least value c on $[l, u]$, attained uniquely at $x = b$.

Lean statement

```
theorem translated_square_bounded_min_unique (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u,
  ↪ y = a * (x - b) ^ 2 + c} c ∧ ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b)
```

Parent. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Parent in Lean

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↪ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

What it contributes. Reuses the parent's completed-square lower-bound argument and positive-coefficient reasoning, while consuming interval hypotheses to prove a restricted minimum and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child restricts the range to a closed interval and additionally proves that equality at the minimum uniquely characterizes the vertex. The uniqueness direction requires using positivity of a and vanishing of the square, which is not supplied by the parent's `IsLeast` proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness characterization: equality at the minimum requires proving the square vanishes using positivity of a , while also restricting the range to an interval containing the vertex. The parent's least-value result does not supply this uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine unique-minimizer characterization: equality at the minimum requires proving that the positive coefficient forces the square to vanish, then deriving $x = b$. The interval hypotheses are also used materially to establish that the vertex lies in the bounded domain, so recalling the parent's least-range proof does not supply the full argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_bounded_min_unique.extracted_1_1 (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) :  
  IsLeast {y | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} c ∧  
  ∀ x ∈ Set.Icc l u, a * (x - b) ^ 2 + c = c ↔ x = b
```

Read back by a model that saw only the Lean. For real numbers a, b, c, l , and u , if $a > 0$, $l \leq b$, and $b \leq u$, then c is the least element of the set of all real numbers y for which there exists an $x \in [l, u]$ such that $y = a(x - b)^2 + c$; moreover, for every $x \in [l, u]$, $a(x - b)^2 + c = c$ if and only if $x = b$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_bounded_min_unique (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u,  
  ↪ y = a * (x - b) ^ 2 + c} c ∧ ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b) := by  
  have hvertex : b ∈ Set.Icc l u := ⟨hl, hu⟩  
  have hmem : c ∈ {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} := by  
    refine ⟨b, hvertex, ?_⟩  
    ring  
  have hlower : ∀ y ∈ {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c}, c ≤ y := by  
    intro y hy  
    rcases hy with ⟨x, hx, rfl⟩  
    have hsq : 0 ≤ (x - b) ^ 2 := sq_nonneg (x - b)  
    have hprod : 0 ≤ a * (x - b) ^ 2 := mul_nonneg (le_of_lt ha) hsq  
    linarith  
  have hleast : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} c := by  
    exact ⟨hmem, hlower⟩  
  have hunique : ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b) := by  
    intro x hx  
    constructor  
    · intro heq  
      have hzero : a * (x - b) ^ 2 = 0 := by  
        linarith  
      have hsquare : (x - b) ^ 2 = 0 := by  
        exact (mul_eq_zero.mp hzero).resolve_left (ne_of_gt ha)  
      nlinarith  
    · intro hxb  
      subst x  
      ring  
      exact ⟨hleast, hunique⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__me.me__c2bdca76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f1638f917bd7a73d

4.232 232. translated_completed_square_range

The problem

For real numbers a, b, c with $a > 0$, if $f(x) = a(x - b)^2 + c$ for all real x , prove that the range of f is exactly $[c, \infty)$; that is, every value of f is at least c , and every real number at least c is attained.

Lean statement

```
theorem translated_completed_square_range (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : Set.range f =
  ↪ Set.Ici c
```

Parent. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Parent in Lean

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

What it contributes. Supplies the positivity and square-nonnegativity lower-bound argument, adapted from least-value membership to global range containment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes least-value existence into a full range characterization. Its reverse inclusion requires constructing a preimage using a symbolic square root and proving the scaled square equals $y - c$, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the entire range, requiring construction of a preimage using a square root and division by the positive scale factor. The parent only proves a lower bound and provides no surjectivity argument for every value at least c .

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a least-value claim into a full range characterization, requiring a new reverse-inclusion argument that constructs a preimage using a square root; the parent's lower-bound proof does not supply this.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1777 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_completed_square_range.extracted_1_1 (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a)
  (h₀ : f = fun x => a * (x - b) ^ 2 + c) : Set.range f = Set.Ici c
```

Read back by a model that saw only the Lean. Let f be a function from $\mathbb{R}$ to $\mathbb{R}$, and let a , b , and c be real numbers. Assume that $a > 0$ and that for every real number x , $f(x) = a(x - b)^2 + c$. Then the range of f is the set of all real numbers greater than or equal to c .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_completed_square_range (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x => a * (x - b) ^ 2 + c) : Set.range f =
  ↪ Set.Ici c := by
  subst f
  ext y
  constructor
  · rintro ⟨x, rfl⟩
    change c ≤ a * (x - b) ^ 2 + c
    have hparent_lower_bound : 0 ≤ a * (x - b) ^ 2 :=
      mul_nonneg (le_of_lt ha) (sq_nonneg (x - b))
    linarith
  · intro hy
    change c ≤ y at hy
    have hyc : 0 ≤ y - c := by linarith
    have ht : 0 ≤ (y - c) / a := div_nonneg hyc (le_of_lt ha)
    have hsqr : (Real.sqrt ((y - c) / a)) ^ 2 = (y - c) / a :=
      Real.sq_sqrt ht
    have hdiv : a * ((y - c) / a) = y - c := by
      field_simp [ne_of_gt ha]
    have hcalc : a * (Real.sqrt ((y - c) / a)) ^ 2 = y - c := by
      rw [hsqr, hdiv]
    refine ⟨b + Real.sqrt ((y - c) / a), ?_⟩
    dsimp
    nlinarith [hcalc]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__c6571295

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b03f0c818f06c94

4.233 233. translated_completed_square_least

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Lean statement

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Generalizes the parent's completed-square witness and lower-bound argument from fixed coefficients to symbolic a , b , and c .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the quadratic and requires proving that the symbolic value c is attained at $x = b$ and is a lower bound using $a > 0$. The parent's fixed completing-square argument at 3 does not supply this generalized reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and must prove the lower bound using positivity of an arbitrary coefficient and nonnegativity of a square. This requires the new multiplicative nonnegativity argument rather than recalling the parent's fixed completed-square proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic: it must use the symbolic minimizer b and establish nonnegativity of $a \cdot (x - b)^2$ from the new positivity hypothesis. The parent's fixed witness and square argument do not directly supply this generalized reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_completed_square_least.extracted_1_1 (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a)
(h₀ : f = fun x => a * (x - b) ^ 2 + c) : IsLeast (Set.range f) c
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$. Assume that $a > 0$ and that for every real number x , $f(x) = a(x - b)^2 + c$. Then c is the least element of the range of f ; that is, c belongs to the range of f and $c \leq y$ for every y in the range of f .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x => a * (x - b) ^ 2 + c) : IsLeast
  (Set.range f) c := by
  subst f
  constructor
  · refine ⟨b, ?_⟩
    simp
  · rintro y ⟨x, rfl⟩
    have hprod : 0 ≤ a * (x - b) ^ 2 := mul_nonneg (le_of_lt ha) (sq_nonneg (x - b))
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__3cb02b9d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 93b50f44115c430f

4.234 234. quadratic_range_characterization

The problem

For the function $f(x) = x^2 - 6x + 13$, characterize its range. Show that a real number y lies in the range of f exactly when $y \geq 4$.

Lean statement

```
theorem quadratic_range_characterization
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  ∀ y : ℝ, y ∈ Set.range f ↔ 4 ≤ y := by
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Uses the parent quadratic and its lower-bound completion-of-the-square argument, extending it with an explicit witness for every admissible range value.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the minimum assertion into a full range characterization. Its converse direction requires constructing the explicit witness $3 + \sqrt{y-4}$ and proving it realizes every admissible value, which is not supplied by the parent's lower-bound argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum assertion into a full range characterization. Its reverse implication requires constructing the explicit witness using $\sqrt{y-4}$, reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child characterizes the entire range, including a new reverse-direction construction using the explicit witness $3 + \sqrt{y-4}$. The parent's minimum proof supplies only attainment at 4 and a lower bound, so recalling it does not solve the child's substantive existence argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2665 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_range_characterization.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 6 * x + 13) → ∀ (y : ℝ), y ∈ Set.range f ↔ 4 ≤ y
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = x^2 - 6x + 13$ for all real x , then for every real number y , y belongs to the range of f if and only if $4 \leq y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_range_characterization  
  (f : ℝ → ℝ)  
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :  
  ∀ y : ℝ, y ∈ Set.range f ↔ 4 ≤ y := by  
  subst f  
  intro y  
  constructor  
  · rintro ⟨x, rfl⟩  
    have hsquare : 0 ≤ (x - 3) ^ 2 := sq_nonneg (x - 3)  
    nlinarith  
  · intro hy  
    have hnonneg : 0 ≤ y - 4 := by  
      linarith  
    have hsq : (Real.sqrt (y - 4)) ^ 2 = y - 4 := by  
      simpa using Real.sq_sqrt hnonneg  
    refine ⟨3 + Real.sqrt (y - 4), ?_⟩  
    nlinarith [hsq]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__b16e00b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b16e00b9f30acda3

4.235 235. translated_square_strict_positivity

The problem

For real numbers c, d, x , prove that $d \leq (x - c)^2 + d$, with equality if and only if $x = c$.

Lean statement

```
theorem translated_square_strict_positivity (c d x : ℝ) :
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c)
```

Parent. For real numbers c, d and $r \geq 0$, the minimum of $(x - c)^2 + d$ on the interval $[c - r, c + r]$ is d , attained exactly at $x = c$.

Parent in Lean

```
theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c)
```

What it contributes. Uses the parent's translated-square and equality-characterization checkpoints, generalized from interval-restricted minimization to a pointwise lower-bound characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuinely new universal pointwise characterization: it removes the interval restriction and `IsLeast` image claim, requiring the lower bound and equality condition for every real x . Its proof directly uses square nonnegativity and zero-square elimination, so recalling the parent's interval-minimum theorem does not supply the result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's interval-image minimum statement into a pointwise inequality and equality characterization for arbitrary real x . Its proof directly derives nonnegativity and zero-square elimination, without relying on the parent's interval or `IsLeast` argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the interval and `IsLeast` structure, proving a global pointwise lower-bound and equality characterization for arbitrary real x . Its proof independently derives nonnegativity and zero-square elimination, so recalling the parent theorem is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2727 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_strict_positivity.extracted_1_1 (c d x : ℝ) :  
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c)
```

Read back by a model that saw only the Lean. For all real numbers c , d , and x , d is less than or equal to $(x - c)^2 + d$, and $(x - c)^2 + d$ equals d if and only if x equals c .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_strict_positivity (c d x : ℝ) :  
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c) := by  
  have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)  
  have hbound : d ≤ (x - c) ^ 2 + d := by  
    linarith  
  have heq : (x - c) ^ 2 + d = d ↔ x = c := by  
    constructor  
    · intro h  
      have hzero : x - c = 0 := by  
        nlinarith  
      linarith  
    · intro hxc  
      rw [hxc]  
      ring  
  exact ⟨hbound, heq⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me.me__d0a0cd76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d0a0cd76d20e84ec

4.236 236. translated_square_range_iff

The problem

For real numbers c , d , and y , the number y is in the range of $(x - c)^2 + d$ exactly when $d \leq y$.

Lean statement

```
theorem translated_square_range_iff (c d y : ℝ) :
  y ∈ Set.range (fun x : ℝ => (x - c) ^ 2 + d) ↔ d ≤ y
```

Parent. For real numbers c and d , the least value of $(x - c)^2 + d$ over all real x is d .

Parent in Lean

```
theorem translated_square_minimum_uniform (c d : ℝ) :
  IsLeast (Set.range (fun x : ℝ => (x - c) ^ 2 + d)) d
```

What it contributes. Supplied the lower-bound argument via nonnegativity of the translated square; the mutation changes the minimum claim into a full range characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from identifying a minimum to characterizing the entire range. Its reverse direction requires constructing an arbitrary preimage with a square-root witness, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a minimum-existence result into a full range characterization. Although the lower-bound direction resembles the parent, the reverse direction requires constructing an explicit square-root witness and applying `Real.sq_sqrt`, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum assertion into a full range characterization. Its reverse direction requires constructing the explicit witness $c + \sqrt{y - d}$ and using `Real.sq_sqrt`, which the parent proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2721 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_range_iff.extracted_1_1 (c d y : ℝ) :
  (y ∈ Set.range fun x => (x - c) ^ 2 + d) ↔ d ≤ y
```

Read back by a model that saw only the Lean. For real numbers c , d , and y , y belongs to the range of the function that maps each real number x to $(x - c)^2 + d$ if and only if $d \leq y$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_range_iff (c d y : ℝ) :
  y ∈ Set.range (fun x : ℝ => (x - c) ^ 2 + d) ↔ d ≤ y := by
  constructor
  · intro hy
    rcases hy with ⟨x, hx⟩
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)
    nlinarith
  · intro hdy
    have hnonneg : 0 ≤ y - d := sub_nonneg.mpr hdy
    have hsquare : (Real.sqrt (y - d)) ^ 2 = y - d := Real.sq_sqrt hnonneg
    have hwitness : ((c + Real.sqrt (y - d)) - c) ^ 2 + d = y := by
      nlinarith
    exact ⟨c + Real.sqrt (y - d), hwitness⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me__3cf2d608

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3cf2d608076357ba

4.237 237. translated_square_minimum_interval

The problem

For real numbers c, d and $r \geq 0$, the minimum of $(x - c)^2 + d$ on the interval $[c - r, c + r]$ is d , attained exactly at $x = c$.

Lean statement

```
theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c)
```

Parent. For real numbers c and d , the least value of $(x - c)^2 + d$ over all real x is d .

Parent in Lean

```
theorem translated_square_minimum_uniform (c d : ℝ) :
  IsLeast (Set.range (fun x : ℝ => (x - c) ^ 2 + d)) d
```

What it contributes. Reuses the translated-square attainment and nonnegative-square lower-bound reasoning, strengthened to an interval image and an equality-locus conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the domain to a bounded interval and adds a genuine equality-locus characterization, requiring proof that equality occurs exactly at $x = c$. This is not recoverable by merely recalling the parent's minimum proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine equality-locus characterization, requiring a new argument that a square vanishes only when $x = c$. The parent's minimum proof does not establish this claim, and the interval restriction is correctly handled by proving that c lies in the interval.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine equality-locus characterization on a restricted interval, requiring proving that a square vanishes exactly at the center. This is not supplied by recalling the parent's minimum proof, and the interval restriction also requires establishing center membership.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2719 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_minimum_interval.extracted_1_1 (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x ∈ Set.Icc (c - r) (c + r), (x - c) ^ 2 + d = d ↔ x = c
```

Read back by a model that saw only the Lean. For real numbers c, d , and r with $0 \leq r$, the number d is the least element of the set of values $\{(x - c)^2 + d \mid c - r \leq x \leq c + r\}$; moreover, for every x satisfying $c - r \leq x \leq c + r$, $(x - c)^2 + d = d$ if and only if $x = c$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c) := by
  have hcenter : c ∈ Set.Icc (c - r) (c + r) := by
    constructor <=> linarith
  constructor <=> linarith
  have h_attain : d ∈ ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) := by
    refine ⟨c, hcenter, ?_⟩
  ring
  have h_lower : ∀ {y : ℝ},
    y ∈ ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) → d ≤ y := by
    intro y hy
    rcases hy with ⟨x, hx, rfl⟩
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)
    nlinarith
  have h_eq : ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c) := by
    intro x hx
    rcases hx with ⟨hl, hu⟩
    constructor
    · intro hval
      have hzero : x - c = 0 := by
        nlinarith [sq_nonneg (x - c), hl, hu]
      linarith
    · intro hxc
      rw [hxc]
      ring
  constructor
  · exact ⟨h_attain, h_lower⟩
  · exact h_eq
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me__fe304b63

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fe304b6318eac288

4.238 238. completed_square_level_right

The problem

For $y = (x - 3)^2 + 4$, show that every level $t \geq 4$ is attained at exactly one point $x \geq 3$.

Lean statement

```
theorem completed_square_level_right
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4)
  (t : ℝ)
  (ht : 4 ≤ t) :
  ∃! x : ℝ, 3 ≤ x ∧ f x = t := by
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Retains the parent's completed-square representation and nonnegativity checkpoint, while replacing the fixed minimum claim with a parameterized inverse-level existence and uniqueness theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes the attained level and proves existence and uniqueness on the right-hand ray using square-root construction and sign elimination. The parent only establishes the minimum value and supplies neither the level witness nor uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the attained level and proves existence and uniqueness on the ray $x \geq 3$, requiring square-root construction and exclusion of the negative root. The parent's minimum argument does not supply this characterization or its uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the unique right-hand point attaining every level $t \geq 4$, requiring square-root construction and a two-factor uniqueness argument beyond the parent's minimum proof. It follows the planned parameterized bounded generalization and is not solvable by recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2695 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem completed_square_level_right.extracted_1.1 (f : ℝ → ℝ) (h : f = fun x => (x - 3) ^ 2 + 4) (t : ℝ) (ht : 4 ≤ t) :
  ∃! x, 3 ≤ x ∧ f x = t
```

Read back by a model that saw only the Lean. Let f be a function from the real numbers to the real numbers such that $f(x) = (x - 3)^2 + 4$ for every real number x . For any real number t with $4 \leq t$, there exists exactly one real number x such that $3 \leq x$ and $f(x) = t$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem completed_square_level_right
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4)
  (t : ℝ)
  (ht : 4 ≤ t) :
  ∃! x : ℝ, 3 ≤ x ∧ f x = t := by
  subst f
  have hnonneg : 0 ≤ t - 4 := by
    linarith
  have hsqrt : 0 ≤ Real.sqrt (t - 4) := Real.sqrt_nonneg _
  have hsquare : (Real.sqrt (t - 4)) ^ 2 = t - 4 := by
    rw [Real.sq_sqrt hnonneg]
  refine (3 + Real.sqrt (t - 4), ?_, ?_)
  · constructor
    · linarith
    · nlinarith
  · intro y hy
    rcases hy with ⟨hy3, hfy⟩
    have hy_sq : (y - 3) ^ 2 = t - 4 := by
      nlinarith [hfy]
    have hprod : ((y - 3) - Real.sqrt (t - 4)) * ((y - 3) + Real.sqrt (t - 4)) = 0 := by
      nlinarith [hy_sq, hsquare]
    rcases mul_eq_zero.mp hprod with hdiff | hsum
    · nlinarith
    · nlinarith [hy3, hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me__0ee00499

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 0ee00499097b34fd

4.239 239. translated_square_minimum

The problem

For any real number c , the least value of $(x - c)^2 + 4$ over all real x is 4.

Lean statement

```
theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Generalized the parent's completed-square range-minimum argument by replacing the fixed vertex 3 with an arbitrary real translation parameter c .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the vertex: the range witness must change from the fixed value 3 to the arbitrary real c , requiring symbolic ring reasoning rather than direct normalization. No hypothesis is unused, and the parent proof cannot be copied unchanged.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the vertex: the least-value witness changes from 3 to the symbolic parameter c , and the lower-bound argument must remain symbolic. A solver recalling the parent's fixed-3 proof cannot apply it unchanged.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed vertex 3 is genuinely replaced by an arbitrary real parameter c , and the proof must establish attainment at the symbolic point $x=c$ while preserving the lower-bound argument. This is the requested parameterization, not a decorative wrapper or unused hypothesis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2700 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_minimum.extracted_1_1 (c : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Read back by a model that saw only the Lean. *Let c be a real number and let f be a function from the real numbers to the real numbers such that, for every real number x , $f(x) = (x - c)^2 + 4$. Then 4 is the least element of the range of f : 4 belongs to the range of f , and for every y in the range of f , $4 \leq y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
  subst f
  constructor
  · use c
    ring
  · rintro y ⟨x, rfl⟩
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me__2590095a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 2590095a9abbd33b

4.240 240. positive_base_cube_logb

The problem

For a real number $b > 1$, evaluate $\log_b(b^3)$. Show that it is 3.

Lean statement

```
theorem positive_base_cube_logb (b : ℝ) (hb : 1 < b) : Real.logb b (b ^ 3) = 3
```

Parent. Evaluate $\log_3 27$. Show that it is 3.

Parent in Lean

```
theorem mathd_algebra_484 : Real.logb (3 : ℝ) (27 : ℝ) = 3 := by
```

What it contributes. Generalized the parent's concrete logb evaluation pattern from base 3 and input 27 to the parameterized cube input b^3 , retaining the logarithm nonzero obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-base evaluation: it must derive ' $\text{Real.log } b \neq 0$ ' from the symbolic hypothesis ' $1 < b$ ' and then establish the cube law for every such base. The parent's concrete normalization cannot be reused without this variable-dependent argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed-base computation to arbitrary real $b > 1$. Its proof must derive nonzero $\log b$ from the symbolic positivity hypothesis, so the parent's numerical normalization is insufficient by itself.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and proves the cube law for every real $b > 1$. Its proof requires deriving $\log b \neq 0$ from positivity and applying Real.log_pow symbolically, so recalling the fixed-base parent does not supply the argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1708 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_base_cube_logb.extracted_1_1 (b : ℝ) (hb : 1 < b) : logb b (b ^ 3) = 3
```

Read back by a model that saw only the Lean. For a real number b with $1 < b$, the logarithm base b of b cubed is equal to 3.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem positive_base_cube_logb (b : ℝ) (hb : 1 < b) : Real.logb b (b ^ 3) = 3 := by
  have hlogpos : 0 < Real.log b := Real.log_pos hb
  have hlog : Real.log b ≠ 0 := ne_of_gt hlogpos
  rw [Real.logb, Real.log_pow]
  field_simp [hlog]
  norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_484__me__a80c851b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8fd9ebcfe21a83f9

4.241 241. gcd_lcm_set_empty

The problem

Let x be a positive integer. Then the set of integers b such that the greatest common divisor of b and x is $x + 3$ and the least common multiple of b and x is $x(x + 3)$ is empty.

Lean statement

```
theorem gcd_lcm_set_empty
  (x : ℤ)
  (h₀ : 0 < x)
  (Sa : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)}) :
  Sa = ∅
```

Parent. The greatest common divisor of two integers is $(x + 3)$ and their least common multiple is $x(x + 3)$, where x is a positive integer. If one of the integers is 40, what is the smallest possible value of the other one? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_126
  (x a : ℤ)
  (h₀ : 0 < x)
  (h₁ : x = 40 ∨ a = 40)
  (Sa Sx : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)})
  (hSx : Sx = {y | 0 < y ∧ Int.gcd a y = y + 3 ∧ Int.lcm a y = y * (y + 3)})
  (h₂ : Nonempty Sa)
  (h₃ : Nonempty Sx) :
  IsLeast Sa 8 ∨ IsLeast Sx 8 := by
```

What it contributes. Uses the parent's gcd-divides-right checkpoint and the same symbolic gcd/lcm set definition, while changing the final obligation to set emptiness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's fixed $x = 40$ contradiction to arbitrary positive x : $x + 3$ divides x , which is impossible by positivity and multiplier sign cases. Its set-emptiness conclusion and proof structure are materially different from the parent's disjunctive least-element argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's fixed $x = 40$ contradiction to arbitrary positive x , proving that $x + 3$ cannot divide x via a sign and multiplier argument. Its set-emptiness conclusion and proof are not recoverable by recalling the parent's disjunctive least-element proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's vacuous disjunction with a symbolic emptiness theorem and generalizes the divisor contradiction from $x = 40$ to arbitrary positive x . Its proof requires a new sign-and-divisibility argument, so recalling the parent's case split and final contradiction is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1793 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_lcm_set_empty.extracted_1_1 (x : ℤ) (h₀ : 0 < x) (Sa : Set ℤ)
  (hSa : Sa = {b | ↑(b.gcd x) = x + 3 ∧ ↑(b.lcm x) = x * (x + 3)}) : Sa = ∅
```

Read back by a model that saw only the Lean. *For an integer x with $0 < x$, and a set Sa of integers satisfying $Sa = \{b \in \mathbb{Z} \mid \gcd(b, x) = x + 3 \text{ and } \text{lcm}(b, x) = x(x + 3)\}$, the set Sa is empty.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem gcd_lcm_set_empty
  (x : ℤ)
  (h₀ : 0 < x)
  (Sa : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)}) :
  Sa = ∅ := by
  rw [hSa]
  ext b
  constructor
  · intro hb
    have hd : (Int.gcd b x : ℤ) | x := Int.gcd_dvd_right b x
    have heq : (Int.gcd b x : ℤ) = x + 3 := by
      simpa using hb.1
    rw [heq] at hd
    rcases hd with ⟨k, hk⟩
    have hpos : 0 < x + 3 := by omega
    rcases le_or_gt k 0 with hk_nonpos | hk_pos
    · have hmul : (x + 3) * k ≤ 0 :=
        mul_nonpos_of_nonneg_of_nonpos (by omega) hk_nonpos
      omega
    · have hdiff : 0 ≤ (x + 3) * (k - 1) :=
        mul_nonneg (by omega) (by omega)
      have hbound : x + 3 ≤ (x + 3) * k := by
        nlinarith
      omega
  · intro hb
    simp at hb
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__me__b6f20679

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 14622aac7d81ca1c

4.242 242. bounded_offset_gcd_lcm_sign

The problem

Let y be positive and let the integer offset c satisfy $-10 \leq c \leq 10$. If the greatest common divisor of 40 and y is $y + c$, while their least common multiple is $y(y + c)$, prove that $c \leq 0$.

Lean statement

```
theorem bounded_offset_gcd_lcm_sign
  (y c : ℤ)
  (hy : 0 < y)

  (hg : (Int.gcd 40 y : ℤ) = y + c)
  :
  c ≤ 0
```

Parent. There is no positive integer y and positive integer offset $c \leq 10$ such that the greatest common divisor of 40 and y is $y + c$ while their least common multiple is $y(y + c)$.

Parent in Lean

```
theorem bounded_offset_gcd_lcm_impossible
  (y c : ℤ)
  (hy : 0 < y)
  (hc : 0 < c)

  (hg : (Int.gcd 40 y : ℤ) = y + c)
  (hl : (Int.lcm 40 y : ℤ) = y * (y + c)) :
  False
```

What it contributes. Uses the parent's gcd divisibility checkpoint and the same gcd/lcm configuration, while changing the conclusion from impossibility to the signed characterization $c \leq 0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child drops the positivity and lcm hypotheses and proves a new sign conclusion from gcd divisibility alone. Its proof uses a divisibility and quotient-sign argument, so recalling the parent's contradiction proof does not substantially help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded-sign characterization: from the gcd divisibility it derives that a positive offset would make a positive divisor larger than y , without using the parent's lcm identity or square contradiction. It therefore cannot be solved by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different sign characterization: from the gcd equation alone, it derives that a positive offset would force the positive divisor $y+c$ to divide the smaller positive y . It drops the lcm contradiction entirely and requires a new divisibility and quotient-sign argument, so recalling the parent proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2545 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_offset_gcd_lcm_sign.extracted_1_1 (y c : ℤ) (hy : 0 < y) (hbound : -10 ≤ c ∧ c ≤ 10)
(hg : ↑(Int.gcd 40 y) = y + c) (hl : ↑(Int.lcm 40 y) = y * (y + c)) : c ≤ 0
```

Read back by a model that saw only the Lean. *For integers y and c , if y is positive, c lies between -10 and 10 inclusive, the greatest common divisor of 40 and y equals $y + c$, and the least common multiple of 40 and y equals $y(y + c)$, then $c \leq 0$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_offset_gcd_lcm_sign
(y c : ℤ)
(hy : 0 < y)

(hg : (Int.gcd 40 y : ℤ) = y + c)
:
c ≤ 0 := by
by_contra hnot
have hc : 0 < c := by omega
have hgc : (Int.gcd 40 y : ℤ) | y :=
Int.gcd_dvd_right 40 y
have hdiv : y + c | y := by
rw [← hg]
exact hgc
rcases hdiv with ⟨k, hk⟩
have hsum : 0 < y + c := by omega
have hk_nonneg : 0 ≤ k := by
by_contra hkn
have hkneg : k < 0 := by omega
have hprodneg : (y + c) * k < 0 :=
mul_neg_of_pos_of_neg hsum hkneg
nlinarith [hk, hprodneg, hy]
have hkpos : 0 < k := by
by_contra hknot
have hkzero : k = 0 := by omega
subst k
norm_num at hk
omega
have hkone : 1 ≤ k := by omega
have hmul_nonneg : 0 ≤ (y + c) * (k - 1) := by
apply mul_nonneg
· omega
· omega
have hle : y + c ≤ (y + c) * k := by
nlinarith [hmul_nonneg]
nlinarith [hk, hle, hc]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me.me__77c02b0d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1d9ab47563e631cc

4.243 243. gcd_strictly_exceeds_candidate_impossible

The problem

There is no positive integer y whose greatest common divisor with 40 is greater than y and whose least common multiple with 40 equals the product of y and that greatest common divisor.

Lean statement

```
theorem gcd_strictly_exceeds_candidate_impossible
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) > y)
  :
  False
```

Parent. There is no positive integer y whose greatest common divisor with 40 is $y + 3$ and whose least common multiple with 40 is $y(y + 3)$.

Parent in Lean

```
theorem mathd_numbertheory_126_specialized
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) = y + 3)
  (hl : (Int.lcm 40 y : ℤ) = y * (y + 3)) :
  False
```

What it contributes. Retains the positive integer and gcd/lcm configuration and reuses the parent's gcd-divisibility check-point, while replacing the offset equation with a strict gcd-versus-candidate obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's fixed-offset gcd/lcm argument with a genuinely different contradiction: since $\text{gcd}(40, y)$ divides positive y , it cannot exceed y . The lcm hypothesis and the parent's product identity are entirely unnecessary.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed gcd/lcm equations with a strict inequality and requires a new argument that the gcd divides y , hence cannot exceed positive y . The parent's equation-based proof does not provide this reasoning, and the child is not a wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed gcd offset and lcm coupling with the independent condition that $\text{gcd}(40, y) > y$, then proves impossibility directly from $\text{gcd}(40, y)$ dividing y and $y > 0$. This requires a different divisibility-and-bound argument and does not recall the parent's product identity or case analysis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2525 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_strictly_exceeds_candidate_impossible.extracted_1_1 (y : Z) (hy : 0 < y) (hg : ↑(Int.gcd 40 y) > y)
(hl : ↑(Int.lcm 40 y) = y * ↑(Int.gcd 40 y)) : False
```

Read back by a model that saw only the Lean. *Let y be an integer such that $0 < y$. Suppose that the integer cast of $\text{gcd}(40, y)$ is greater than y , and that the integer cast of $\text{lcm}(40, y)$ equals y multiplied by the integer cast of $\text{gcd}(40, y)$. Then a contradiction follows; that is, False.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem gcd_strictly_exceeds_candidate_impossible
  (y : Z)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : Z) > y)
  :
  False := by
  have hd : (Int.gcd 40 y : Z) | y :=
    Int.gcd_dvd_right 40 y
  have hnonneg : 0 ≤ (Int.gcd 40 y : Z) := by
    exact_mod_cast (Nat.zero_le (Int.gcd 40 y))
  have hbound : (Int.gcd 40 y : Z) ≤ y := by
    rcases hd with ⟨k, hk⟩
    have hk_pos : 0 < k := by
      by_contra hkp
      have hk_nonpos : k ≤ 0 := le_of_not_gt hkp
      have hprod : (Int.gcd 40 y : Z) * k ≤ 0 :=
        mul_nonpos_of_nonneg_of_nonpos hnonneg hk_nonpos
      have hy_nonpos : y ≤ 0 := by
        calc
          y = (Int.gcd 40 y : Z) * k := hk
            _ ≤ 0 := hprod
      omega
    have hk_one : (1 : Z) ≤ k := by omega
    have hmul : (Int.gcd 40 y : Z) * 1 ≤ (Int.gcd 40 y : Z) * k :=
      mul_le_mul_of_nonneg_left hk_one hnonneg
    calc
      (Int.gcd 40 y : Z) = (Int.gcd 40 y : Z) * 1 := by ring
        _ ≤ (Int.gcd 40 y : Z) * k := hmul
        _ = y := by symm; exact hk
    exact (not_lt_of_ge hbound) hg
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me__a7839e6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9116a430db6e9204

4.244 244. odd_power_sum_last_digit

The problem

Show that for every natural number n , the sum $19^{2n+1} + 99^{2n+1}$ has last digit 8.

Lean statement

```
theorem odd_power_sum_last_digit : ∀ n : ℕ, (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

What it contributes. The parent supplies the fixed two-base last-digit pattern; this mutation generalizes it to all explicitly odd exponents and proves periodicity for both bases.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponents to all odd exponents and requires an induction showing multiplication by the square preserves the last digit. The parent's 'norm_num' computation supplies no such parametric argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent to all odd exponents and requires an induction showing each power remains 9 modulo 10. The parent's one-shot norm_num computation provides no proof of this law, so recalling it does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponents to all odd exponents and requires an induction showing multiplication by the square preserves the last digit. The parent's one-shot norm_num computation cannot be recalled to supply this universal law.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1867 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_sum_last_digit.extracted_1_1 : ∀ (n : ℕ), (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8
```

Read back by a model that saw only the Lean. For every natural number n , the remainder when 19 raised to the power $2n+1$ plus 99 raised to the power $2n+1$ is divided by 10 is 8.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_sum_last_digit : ∀ n : ℕ, (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8 := by
  intro n
  have h19 : ∀ k : ℕ, 19 ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have h99 : ∀ k : ℕ, 99 ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have h19n := h19 n
  have h99n := h99 n
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__me__2aac1b65

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint e434ffffdaed70cc

4.245 245. gcd_odd_power_sum

The problem

For every natural number n , the greatest common divisor of $19^{2n+1} + 99^{2n+1}$ and 10 is 2.

Lean statement

```
theorem gcd_odd_power_sum (n : ℕ) : Nat.gcd (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) 10 = 2 := by
```

Parent. Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Parent in Lean

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from one fixed modular computation to a universal exact gcd characterization. It generalizes the modulo-10 periodicity to arbitrary odd exponents and uses that residue to determine the gcd, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponents to every odd exponent and changes the conclusion to an exact gcd characterization. Its proof derives a reusable modulo-10 power pattern, establishes the gcd through a nontrivial modular reduction, and cannot be obtained by recalling the parent's one-shot norm_num proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent congruence to all odd exponents and changes the conclusion to an exact gcd characterization. Its proof must establish divisibility by 2 and exclusion of divisibility by 5, rather than recalling the parent's single numerical normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2762 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_odd_power_sum.extracted_1_1 (n : ℕ) : (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)).gcd 10 = 2
```

Read back by a model that saw only the Lean. For every natural number n , the greatest common divisor of $19^{(2n+1)} + 99^{(2n+1)}$ and 10 is equal to 2.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem gcd_odd_power_sum (n : ℕ) : Nat.gcd (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) 10 = 2 := by
  let x := 19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)
  change Nat.gcd x 10 = 2
  have h19 : 19 ≡ 9 [MOD 10] := by
    norm_num [Nat.ModEq]
  have h99 : 99 ≡ 9 [MOD 10] := by
    norm_num [Nat.ModEq]
  have hbase : 9 ^ 2 ≡ 1 [MOD 10] := by
    norm_num [Nat.ModEq]
  have hpow : 9 ^ (2 * n) ≡ 1 [MOD 10] := by
    rw [pow_mul]
    simpa using hbase.pow n
  have hodd : 9 ^ (2 * n + 1) ≡ 9 [MOD 10] := by
    rw [pow_add]
    simpa using hpow.mul (by norm_num [Nat.ModEq] : 9 ^ 1 ≡ 9 [MOD 10])
  have hsum : x ≡ 9 ^ (2 * n + 1) + 9 ^ (2 * n + 1) [MOD 10] := by
    dsimp [x]
    exact (h19.pow (2 * n + 1)).add (h99.pow (2 * n + 1))
  have hmod : x % 10 = 8 := by
    have ht : x ≡ 9 + 9 [MOD 10] := hsum.trans (hodd.add hodd)
    norm_num [Nat.ModEq] at ht
    exact ht
  have hxmod : x ≡ 8 [MOD 10] := by
    simpa [Nat.ModEq] using hmod
  calc
    Nat.gcd x 10 = Nat.gcd 8 10 := Nat.ModEq.gcd_eq hxmod
    _ = 2 := by norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms.me__a5ada452

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a5ada4520e402853

4.246 246. bounded_first_period_exponent

The problem

Let n be a positive integer with $n \leq 6$. Show that 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Lean statement

```
theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplies the modular-power checkpoint pattern, specialized to the first period representative $5^6 \equiv 1$ modulo 7.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child characterizes all positive exponents up to 6 for which $5^n \equiv 1 \pmod{7}$, requiring a case analysis and establishing the minimal period 6. The parent only verifies the isolated exponent 30 and supplies none of this bounded characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single fixed residue computation into a bounded first-period characterization. It requires ruling out exponents 1 through 5 and separately proving the exponent-6 case, which the parent's computation of $5^{30} \equiv 1$ does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed residue computation into a bounded characterization of the first positive exponent, requiring an explicit case split over $n = 1$ through 6 and exclusion of the first five cases. The parent's proof of $5^{30} \equiv 1$ does not supply this minimality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_first_period_exponent.extracted_1_1 (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) :
  5 ^ n ≡ 1 [ZMOD 7] ↔ n = 6
```

Read back by a model that saw only the Lean. For a natural number n with $0 < n$ and $n \leq 6$, 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
  have hperiod : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  constructor
  · intro h
    have hcases : n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4 ∨ n = 5 ∨ n = 6 := by
      omega
    rcases hcases with h1 | h2 | h3 | h4 | h5 | h6
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · exact h6
  · intro h
    subst n
    exact hperiod
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me__a3c3ca0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 9a5af6fc5c1aa54d

4.247 247. mathd_numbertheory_229_mutation

The problem

For every integer base whose residue modulo 7 is nonzero, its sixth power is congruent to 1 modulo 7.

Lean statement

```
theorem mathd_numbertheory_229_mutation (n : ℤ) (h : (n : ℤMod 7) ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Generalizes the parent’s modulo-7 power computation from base 5 and exponent 30 to an arbitrary nonzero residue and the underlying exponent-six period.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a single numerical computation with the general Fermat-law claim that every nonzero residue modulo 7 has sixth power 1. Its proof requires reasoning in $\mathbb{ZMod}\ 7$ and translating that result back to integer congruence, none of which is supplied by recalling the parent’s normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed computation to every integer base nonzero modulo 7, requiring Fermat-style reasoning in $\mathbb{ZMod}\ 7$ and conversion back to integer congruence. The parent’s direct numerical normalization supplies none of this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed computation to every integer nonzero modulo 7 and exponent 6, requiring Fermat’s little theorem and a conversion back to integer congruence. The parent’s direct normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2667 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_229_mutation.extracted_1_1 (n : ℤ) (h : ↑n ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7]
```


Read back by a model that saw only the Lean. For an integer n whose residue modulo 7 is nonzero, the sixth power n^6 is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_229_mutation (n : ℤ) (h : (n : ℤMod 7) ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7] := by
  letI : Fact (Nat.Prime 7) := ⟨by norm_num⟩
  have hn : (n : ℤMod 7) ^ 6 = 1 := by
    simpa using (ℤMod.pow_card_sub_one_eq_one (p := 7) h)
  apply (Int.modEq_iff_dvd).2
  have hz : ((1 - n ^ 6 : ℤ) : ℤMod 7) = 0 := by
    norm_num [Int.cast_sub, Int.cast_pow]
    exact sub_eq_zero.mpr hn.symm
  exact (ℤMod.intCast_zmod_eq_zero_iff_dvd (1 - n ^ 6) 7).mp hz
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me__c5ab2543

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c5ab2543f0de475e

4.248 248. bounded_five_power_add_one_gcd_iff

The problem

For an integer k with $0 \leq k \leq 6$, show that the greatest common divisor of 7 and $5^k + 1$ equals 7 exactly when $k = 3$.

Lean statement

```
theorem bounded_five_power_add_one_gcd_iff (k : ℕ) (hbound : k ≤ 6) :
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3
```

Parent. For an integer k with $0 \leq k \leq 6$, show that 7 divides $5^k + 1$ exactly when $k = 3$.

Parent in Lean

```
theorem bounded_five_power_add_one_dvd_iff (k : ℕ) (hbound : k ≤ 6) : 7 ∣ (5 ^ k + 1 : ℕ) ↔ k = 3 := by
```

What it contributes. Supplies the bounded iff characterization that 7 divides 5^k+1 exactly when $k=3$; this is consumed as `hdiv` and converted into the new `gcd` target.

Review

Every judgement this row received. Admission required `strong` from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine `gcd-divisibility` equivalence, requiring `divisibility antisymmetry` and `gcd properties` before applying the bounded power characterization. Recalling the parent supplies only one checkpoint and does not prove the new `gcd` statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes `divisibility` into a genuine `gcd` characterization and proves the nontrivial bridge between `gcd equality` and `divisibility`. Although it rederives the bounded `divisibility` checkpoint, the parent's proof does not supply the `gcd` argument, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from `divisibility` to a `gcd equality` and proves a genuine bridge between the two claims via `gcd divisibility properties`. Although it reproves the bounded `divisibility` checkpoint by case analysis, the parent proof does not supply the new `gcd` argument, so recalling the parent alone is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2432 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_five_power_add_one_gcd_iff.extracted_1_1 (k : ℕ) (hbound : k ≤ 6) :  
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3
```

Read back by a model that saw only the Lean. For a natural number k satisfying $k \leq 6$, the greatest common divisor of 7 and $5^k + 1$ is equal to 7 if and only if $k = 3$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_five_power_add_one_gcd_iff (k : ℕ) (hbound : k ≤ 6) :  
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3 := by  
  have hdiv : 7 ∣ (5 ^ k + 1 : ℕ) ↔ k = 3 := by  
    interval_cases k <|> norm_num  
  have hrel : Nat.gcd 7 (5 ^ k + 1) = 7 ↔ 7 ∣ (5 ^ k + 1 : ℕ) := by  
    constructor  
    · intro heq  
      have hg : Nat.gcd 7 (5 ^ k + 1) ∣ (5 ^ k + 1 : ℕ) :=  
        Nat.gcd_dvd_right 7 (5 ^ k + 1)  
      rw [heq] at hg  
      exact hg  
    · intro hd  
      apply Nat.dvd_antisymm  
      · exact Nat.gcd_dvd_left 7 (5 ^ k + 1)  
      · exact Nat.dvd_gcd (dvd_refl 7) hd  
  exact hrel.trans hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__ms.mh.mh.me__e1eec4bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint e1eec4bc2fd5b65f

4.249 249. fibonacci_mod_four_period

The problem

Let (a_n) begin with $a_1 = a_2 = 1$ and satisfy $a_{n+2} = a_{n+1} + a_n$. Prove that for every nonnegative integer k , the remainders of $a_{6k}, a_{6k+1}, \dots, a_{6k+5}$ upon division by 4 are respectively 0, 1, 1, 2, 3, 1.

Lean statement

```
theorem fibonacci_mod_four_period (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ k : ℕ,
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧
    a (6 * k + 3) % 4 = 2 ∧
    a (6 * k + 4) % 4 = 3 ∧
    a (6 * k + 5) % 4 = 1
```

Parent. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

What it contributes. Preserves the parent's Fibonacci recurrence setup and initial-value derivation, while generalizing the isolated residue computation into a six-class periodic induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a complete six-class modulo-4 periodic classification via recurrence-based induction, requiring modular transport across all six residues. This is substantive new reasoning rather than recalling the parent's identification of the sequence with `Nat.fib` and evaluating one fixed index.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a complete six-class modulo-4 classification for all indices, requiring a new induction that propagates six consecutive residues through six recurrence steps. This is genuine bounded generalization and cannot be solved by recalling the parent's isolated computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's isolated computation at index 100 with a complete six-class modulo-4 classification for every index block. Its induction derives and transports six new recurrence residues, so recalling the parent's Fibonacci-identification proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2556 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fibonacci_mod_four_period.extracted_1_1 : ∀ (a : ℕ → ℕ),
  a 1 = 1 →
  a 2 = 1 →
  (∀ (n : ℕ), a (n + 2) = a (n + 1) + a n) →
  ∀ (k : ℕ),
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧ a (6 * k + 3) % 4 = 2 ∧ a (6 * k + 4) % 4 = 3 ∧ a (6 * k + 5) % 4 = 1
```

Read back by a model that saw only the Lean. *For every function a from the natural numbers to the natural numbers, if $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$, then for every natural number k , the following congruences hold modulo 4: $a(6k)$ has remainder 0; $a(6k + 1)$ has remainder 1; $a(6k + 2)$ has remainder 1; $a(6k + 3)$ has remainder 2; $a(6k + 4)$ has remainder 3; and $a(6k + 5)$ has remainder 1.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fibonacci_mod_four_period (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ k : ℕ,
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧
    a (6 * k + 3) % 4 = 2 ∧
    a (6 * k + 4) % 4 = 3 ∧
    a (6 * k + 5) % 4 = 1 := by
  have ha0 : a 0 = 0 := by
    have h := h₂ 0
    norm_num [h₀, h₁] at h
    omega
  have ha3 : a 3 = 2 := by
    have h := h₂ 1
    norm_num [h₀, h₁] at h
    exact h
  have ha4 : a 4 = 3 := by
    have h := h₂ 2
    norm_num [h₁, ha3] at h
    exact h
  have ha5 : a 5 = 5 := by
    have h := h₂ 3
    norm_num [ha3, ha4] at h
    exact h
  intro k
  induction k with
  | zero =>
    norm_num [ha0, h₀, h₁, ha3, ha4, ha5]
  | succ k ih =>
    rcases ih with ⟨h₀, h₁, h₂, h₃, h₄, h₅⟩
    have r6 : a (6 * k + 6) = a (6 * k + 5) + a (6 * k + 4) := by
      have h := h₂ (6 * k + 4)
      convert h using 1 <|> omega
    have r7 : a (6 * k + 7) = a (6 * k + 6) + a (6 * k + 5) := by
      have h := h₂ (6 * k + 5)
      convert h using 1 <|> omega
    have r8 : a (6 * k + 8) = a (6 * k + 7) + a (6 * k + 6) := by
```

```

have h := h2 (6 * k + 6)
convert h using 1 <|> omega
have r9 : a (6 * k + 9) = a (6 * k + 8) + a (6 * k + 7) := by
  have h := h2 (6 * k + 7)
  convert h using 1 <|> omega
have r10 : a (6 * k + 10) = a (6 * k + 9) + a (6 * k + 8) := by
  have h := h2 (6 * k + 8)
  convert h using 1 <|> omega
have r11 : a (6 * k + 11) = a (6 * k + 10) + a (6 * k + 9) := by
  have h := h2 (6 * k + 9)
  convert h using 1 <|> omega
have h6 : a (6 * k + 6) % 4 = 0 := by omega
have h7 : a (6 * k + 7) % 4 = 1 := by omega
have h8 : a (6 * k + 8) % 4 = 1 := by omega
have h9 : a (6 * k + 9) % 4 = 2 := by omega
have h10 : a (6 * k + 10) % 4 = 3 := by omega
have h11 : a (6 * k + 11) % 4 = 1 := by omega
refine {?_, ?_, ?_, ?_, ?_, ?_}
· convert h6 using 1 <|> omega
· convert h7 using 1 <|> omega
· convert h8 using 1 <|> omega
· convert h9 using 1 <|> omega
· convert h10 using 1 <|> omega
· convert h11 using 1 <|> omega

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_483__me__d29b0035

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d29b0035a34bd2fc

4.250 250. four_consecutive_residue_law

The problem

For every natural number n , prove that there is a natural number k such that $n + (n + 1) + (n + 2) + (n + 3) = 4k + 2$ and the remainder when this sum is divided by 4 is 2.

Lean statement

```
theorem four_consecutive_residue_law (n : ℕ) : ∃ k : ℕ, n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧ (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2 := by
```

Parent. Find the remainder when $91145 + 91146 + 91147 + 91148$ is divided by 4. Show that it is 2.

Parent in Lean

```
theorem mathd_numbertheory_640 : (91145 + 91146 + 91147 + 91148) % 4 = 2 := by
```

What it contributes. Retains the four-consecutive-natural-number sum and modulo-4 conclusion while replacing the fixed numerical instance with a symbolic initial term and explicit algebraic decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child establishes the symbolic identity $n + (n + 1) + (n + 2) + (n + 3) = 4(n + 1) + 2$ for arbitrary n , then derives the residue law. The parent's numerical norm_num proof provides no reasoning for this parameterized decomposition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed residue computation: it proves symbolically that every four-consecutive-integer sum is $4(n+1)+2$, then derives the modulus law. The parent's numerical norm_num proof provides no reusable argument for this universal statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed numerical computation into a law for arbitrary n , requiring symbolic arithmetic and an explicit decomposition. The parent's norm_num proof offers no reusable argument for this general case.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1851 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem four_consecutive_residue_law.extracted_1_1 (n : ℕ) :  
  ∃ k, n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧ (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2
```

Read back by a model that saw only the Lean. *For every natural number n , there exists a natural number k such that $n + (n + 1) + (n + 2) + (n + 3) = 4k + 2$ and the remainder when $n + (n + 1) + (n + 2) + (n + 3)$ is divided by 4 is 2.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem four_consecutive_residue_law (n : ℕ) :  
  ∃ k : ℕ,  
    n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧  
    (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2 := by  
  refine ⟨n + 1, ?_, ?_⟩  
  · omega  
  · have hdecomp : n + (n + 1) + (n + 2) + (n + 3) = 4 * (n + 1) + 2 := by  
      omega  
    rw [hdecomp]  
    omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_640__me__625ab733

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 8ccadd1d869afcb4

4.251 251. modular_power_difference_general

The problem

Let a and b be the residues of integers A and B modulo 7. Show that for every natural number t , $A^{6t+1} - B^{6t+1} \equiv a - b \pmod{7}$.

Lean statement

```
theorem modular_power_difference_general (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7]) (hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7]
```

Parent. For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Parent in Lean

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Supplied the period-reduction argument modulo 7 and the congruence-subtraction checkpoint; the fixed residues 1 and 5 were generalized to arbitrary residues a and b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed residues to arbitrary symbolic residues a and b , so the final congruence must be transported symbolically rather than concluding the parent's fixed value 3. It also independently uses the exponent-period reduction and has no redundant hypotheses.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed residues 1 and 5 to arbitrary residues a and b , requiring both congruence hypotheses to be transported through the reduced powers and yielding the symbolic difference $a-b$. This is not decoration or recall because the parent cannot establish the generalized conclusion with its fixed-residue hypotheses.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed residues 1 and 5 to arbitrary symbolic residues a and b , requiring both congruence hypotheses to be transported through the reduced powers and combined symbolically. The parent theorem cannot be recalled to close this generalized conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2500 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_difference_general.extracted_1_1 (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7])  
(hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t and integers A, B, a , and b , if A is congruent to a modulo 7 and B is congruent to b modulo 7, then A raised to the power $6t+1$ minus B raised to the power $6t+1$ is congruent to a minus b modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_difference_general (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7]) (hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7] := by
  have hperiodA : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiodB : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hAred : A ^ (6 * t + 1) ≡ a [ZMOD 7] := by
    simpa using hperiodA.trans (hA.pow 1)
  have hBred : B ^ (6 * t + 1) ≡ b [ZMOD 7] := by
    simpa using hperiodB.trans (hB.pow 1)
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7] := by
    exact hAred.sub hBred
  exact hdifference
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.me__ec6d6429

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ec6d6429c3fc1b30

4.252 252. modular_power_difference_characterization

The problem

For integers A , B , and C , and any natural number t , show that $A^{6t+1} - B^{6t+1} \equiv C \pmod{7}$ if and only if $A - B \equiv C \pmod{7}$.

Lean statement

```
theorem modular_power_difference_characterization (t : ℕ) (A B C : ℤ) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7]
```

Parent. Two integer readings differ from 1 and 5 by multiples of 7, respectively. Show that the difference of their $(6t+1)$ st powers leaves remainder 3 upon division by 7.

Parent in Lean

```
theorem power_difference_residue_law_divisibility (t : ℕ) (A B : ℤ) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Reuses the parent's modulus-7 exponent reduction checkpoint, generalized from fixed residue representatives to arbitrary integers and a latent target residue C .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's fixed residue hypotheses and proves a reusable congruence equivalence for arbitrary A , B , and latent target C . Its forward and reverse directions require establishing and using the symbolic difference congruence, so the parent's fixed-residue conclusion cannot be recalled directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the fixed residue hypotheses and proves a general latent-target equivalence. Its key periodicity argument applies to arbitrary A and B , then uses transitivity in both directions; the parent's fixed-residue specialization cannot directly supply this characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the fixed residue hypotheses and proves a reusable equivalence for arbitrary latent target C . Its proof must establish the general power reduction and both directions of the residue characterization, which the parent's fixed-residue transitivity proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2530 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_difference_characterization.extracted_1_1 (t : ℕ) (A B C : ℤ) :  
  A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7]
```

Read back by a model that saw only the Lean. *For every natural number t and integers A , B , and C , the difference A raised to the power $6t+1$ minus B raised to the power $6t+1$ is congruent to C modulo 7 if and only if A minus B is congruent to C modulo 7.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_difference_characterization (t : ℕ) (A B C : ℤ) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7] := by
  have hAperiod : A ^ (6 * t + 1) ≡ A [ZMOD 7] := by
    have h : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
      · norm_num
      · use t
      · omega
      · omega
      · omega
    simp using h
  have hBperiod : B ^ (6 * t + 1) ≡ B [ZMOD 7] := by
    have h : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
      · norm_num
      · use t
      · omega
      · omega
      · omega
    simp using h
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ A - B [ZMOD 7] := by
    exact hAperiod.sub hBperiod
  constructor
  · intro h
    exact hdifference.symm.trans h
  · intro h
    exact hdifference.trans h
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.ms.me__cb5235b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint cb5235b5b834e1eb

4.253 253. power_difference_residue_law_mod_class

The problem

Let n be a natural number congruent to 1 modulo 6. If integers A and B differ from 1 and 5 by multiples of 7, respectively, show that $A^n - B^n \equiv 3 \pmod{7}$.

Lean statement

```
theorem power_difference_residue_law_mod_class (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ n - B ^ n ≡ 3 [ZMOD 7]
```

Parent. Two integer readings differ from 1 and 5 by multiples of 7, respectively. Show that the difference of their $(6t+1)$ st powers leaves remainder 3 upon division by 7.

Parent in Lean

```
theorem power_difference_residue_law_divisibility (t : ℕ) (A B : ℤ) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Reuses the parent's modulo-7 period reduction, congruence transport, subtraction, and residue computation, with the exponent independently quantified by $n \% 6 = 1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent from the explicit form $6t+1$ to an arbitrary n satisfying $n \bmod 6 = 1$. Its proof must derive the quotient decomposition and divisibility of $n-1$ before applying periodicity, so the parent proof cannot be directly recalled.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent: it must derive a representation of any bounded-congruence exponent n as $6(n/6)+1$ before applying periodicity. This is not supplied by recalling the parent, whose exponent already has the required form.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from the syntactic form $6t+1$ to an arbitrary exponent satisfying $n \bmod 6 = 1$. Its proof must derive the decomposition and divisibility needed for periodicity, so recalling the parent's proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2527 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_low_mod_class.extracted_1_1 (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : 7 ∣ 1 - A)
(hB : 7 ∣ 5 - B) : A ^ n - B ^ n ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For a natural number n and integers A and B , if n leaves remainder 1 upon division by 6, 7 divides $1 - A$, and 7 divides $5 - B$, then A raised to the n th power minus B raised to the n th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem power_difference_residue_low_mod_class (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ n -
  B ^ n ≡ 3 [ZMOD 7] := by
  have hAcon : A ≡ 1 [ZMOD 7] := (Int.modEq_iff_dvd).2 hA
  have hBcon : B ≡ 5 [ZMOD 7] := (Int.modEq_iff_dvd).2 hB
  have hdecomp : n = 6 * (n / 6) + 1 := by
    omega
  have hdiv : 6 ∣ n - 1 := by
    use n / 6
    omega
  have hpos : 0 < n := by
    omega
  have hle : 1 ≤ n := by
    omega
  have hperiodA : A ^ n ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := n) (y := 1)
    · norm_num
    · exact hdiv
    · exact hle
    · norm_num
  have hperiodB : B ^ n ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := n) (y := 1)
    · norm_num
    · exact hdiv
    · exact hle
    · norm_num
  have hAred : A ^ n ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hperiodA.trans (hAcon.pow 1)
  have hBred : B ^ n ≡ (5 : ℤ) [ZMOD 7] := by
    simpa using hperiodB.trans (hBcon.pow 1)
  have hdifference : A ^ n - B ^ n ≡ (1 : ℤ) - 5 [ZMOD 7] := by
    exact hAred.sub hBred
  calc
    A ^ n - B ^ n ≡ (1 : ℤ) - 5 [ZMOD 7] := hdifference
    _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.ms.me__f1a7f310

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f1a7f3100b6b19a6

4.254 254. power_difference_residue_law

The problem

For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Parent. For every natural number t with $t \leq 100$, show that $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Parent in Lean

```
theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
```

What it contributes. Generalizes the parent's period-6 power reduction and subtraction checkpoints from fixed bases to arbitrary integers constrained to the same residue classes.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both bases and requires transporting their modular residues through the power-period argument before subtracting. Its hypotheses are essential, so recalling the parent's concrete proof does not directly solve the generalized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both concrete bases and must use the hypotheses that they lie in residue classes 1 and 5 modulo 7. The parent's proof does not supply this symbolic residue-transfer step, so recalling it is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both concrete bases and requires using the new residue hypotheses to derive the final difference congruence. Although the periodicity argument is inherited, the parent proof cannot be applied directly because its concrete bases and residue computation no longer match.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2481 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_law.extracted_1_1 (t : ℕ) (ht : t ≤ 100) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7])  
(hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t with $t \leq 100$ and all integers A and B such that A is congruent to 1 modulo 7 and B is congruent to 5 modulo 7, the difference $A^{(6t+1)} - B^{(6t+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡  
  3 [ZMOD 7] := by
  have hperiodA : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiodB : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hAred : A ^ (6 * t + 1) ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hperiodA.trans (hA.pow 1)
  have hBred : B ^ (6 * t + 1) ≡ (5 : ℤ) [ZMOD 7] := by
    simpa using hperiodB.trans (hB.pow 1)
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : ℤ) - 5 [ZMOD 7] := by
    exact hAred.sub hBred
  have hresidue : (1 : ℤ) - 5 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  calc
    A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : ℤ) - 5 [ZMOD 7] := hdifference
    _ ≡ 3 [ZMOD 7] := hresidue
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me__6dbad337

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint ee3078c8502a4e6b

4.255 255. coupled_ap_modular_term

The problem

Let an arithmetic sequence have first four terms p , 9 , $3p - q$, and $3p + q$. If n is congruent modulo 7 to $29^{13} - 5^{13}$, show that the $(n + 1)$ st term is congruent to 3 modulo 7.

Lean statement

```
theorem coupled_ap_modular_term (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ k : ℕ, a (k + 2) - a (k + 1) = a (k + 1) - a k)
  (h1 : a 1 = p) (h2 : a 2 = 9) (h3 : a 3 = 3 * p - q)
  (h4 : a 4 = 3 * p + q)
  (hidx : (n : ℤ) ≡ (29 : ℤ)^13 - 5^13 [ZMOD 7]) :
  a (n + 1) ≡ 3 [ZMOD 7]
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Supplies the modulus-7 computation that $29^{13} - 5^{13}$ is congruent to 3, consumed through the explicit index hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires deriving the arithmetic progression explicitly, forcing $p = 5$ and $a(k) = 4k + 1$, then combining that formula with the modular condition on n . This is genuinely new reasoning and is not recoverable by recalling the parent's direct numerical normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires deriving the entire arithmetic progression from the recurrence and initial constraints, then coupling its index modulo 7 to the parent residue. A solver recalling the parent's fixed modular computation would not obtain this progression argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires deriving the entire arithmetic progression from the recurrence and initial constraints, then coupling its index modulo 7 to the parent residue. The parent's direct normalization of a fixed modular expression supplies none of this reasoning, and all hypotheses are materially used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2366 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_ap_modular_term.extracted_1_1 (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ (k : ℕ), a (k + 2) - a (k + 1) = a (k + 1) - a k) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) (hidx : ↑n ≡ 29 ^ 13 - 5 ^ 13 [ZMOD 7]) : a (n + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers p and q , an integer-valued sequence a indexed by the natural numbers, and a natural number n , suppose that for every natural number k , $a(k + 2) - a(k + 1) = a(k + 1) - a(k)$, that $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$. If the integer n is congruent modulo 7 to $29^{13} - 5^{13}$, then $a(n + 1)$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem coupled_ap_modular_term (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ k : ℕ, a (k + 2) - a (k + 1) = a (k + 1) - a k)
  (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hidx : (n : ℤ) ≡ (29 : ℤ)^13 - 5^13 [ZMOD 7]) :
  a (n + 1) ≡ 3 [ZMOD 7] := by
  have hA := hrec 1
  have hB := hrec 2
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith [hA, hB]
  have hzero := hrec 0
  rw [h₂, h₁] at hzero
  have ha0 : a 0 = 1 := by
    linarith [hzero, hp]
  have hform : ∀ k : ℕ, a k = 4 * (k : ℤ) + 1 := by
    intro k
    induction k using Nat.strong_induction_on with
    | h k ih =>
      by_cases hk0 : k = 0
      · subst k
        rw [ha0]
        norm_num
      · by_cases hk1 : k = 1
        · subst k
          rw [h₁, hp]
          norm_num
        · have hk2 : 2 ≤ k := by omega
          let j := k - 2
          have hk : j + 2 = k := by
            dsimp [j]
            omega
          rw [← hk]
          have hj := ih j (by omega)
          have hj1 := ih (j + 1) (by omega)
          have hr := hrec j
          calc
            a (j + 2) = a (j + 1) + (a (j + 1) - a j) := by
              linarith [hr]
            _ = 4 * (↑(j + 2) : ℤ) + 1 := by
              rw [hj1, hj]
              norm_num [Nat.cast_add] <|> ring
  rw [hform]
  norm_num [Int.ModEq] at hidx
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me__43ecb415

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

4.256 256. mathd_numbertheory_728_mutation

The problem

For every natural number t with $t \leq 100$, show that $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Lean statement

```
theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Retains the parent's modulo-7 power-difference target and residue computation, while adding a bounded exponent parameter and explicit period checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed exponent is genuinely parameterized to $6t+1$, requiring periodicity arguments for both powers and modular subtraction; the parent's direct normalization proof does not supply this reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6t+1$. Its proof requires periodicity and modular-power reasoning, which the parent's direct norm_num proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2475 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_728_mutation.extracted_1_1 (t : ℕ) (ht : t ≤ 100) :
  29 ^ (6 * t + 1) - 5 ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t with $t \leq 100$, the natural-number difference $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
  have hperiod29 : (29 : ℤ) ^ (6 * t + 1) ≡ 29 ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiod5 : (5 : ℤ) ^ (6 * t + 1) ≡ 5 ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hdifference : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 29 ^ 1 - 5 ^ 1 [ZMOD 7] := by
    exact hperiod29.sub hperiod5
  have hresidue : ((29 : ℤ) ^ 1 - 5 ^ 1) ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdifference.trans hresidue
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me__e5a2f50a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8112b513089cfe57

4.257 257. affine_power_residue

The problem

For integers r, s and a natural number k , prove that $(29 + 7r)^{6k+1} - (5 + 7s)^{6k+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem affine_power_residue (r s : ℤ) (k : ℕ) : (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Uses the parent's congruence-power transport and subtraction structure, strengthened to explicit affine base parameters and a variable exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent to every $6k+1$ and requires proving the new periodicity $(5)^6 \equiv 1 \pmod{7}$, while also deriving the base congruences from affine parameterizations. This is not recoverable by recalling the parent's fixed-exponent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The variable exponent $6k+1$ requires a new periodicity argument, including proving $5^6 \equiv 1 \pmod{7}$ and transporting powers through multiplication and addition. The parent's fixed-exponent numerical normalization cannot establish this generalized residue.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both bases to independent multiples of seven and the exponent to $6k+1$, requiring periodicity reasoning for the powered residue of 5. The parent's fixed-exponent normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2419 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_power_residue.extracted_1_1 (r s : ℤ) (k : ℕ) :
  (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For all integers r and s and every natural number k , $(29 + 7r)^{(6k+1)} - (5 + 7s)^{(6k+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem affine_power_residue (r s : ℤ) (k : ℕ) : (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
  have ha : (29 + 7 * r : ℤ) ≡ 29 [ZMOD 7] := by
    norm_num [Int.ModEq, Int.add_emod, Int.mul_emod]
  have hb : (5 + 7 * s : ℤ) ≡ 5 [ZMOD 7] := by
    norm_num [Int.ModEq, Int.add_emod, Int.mul_emod]
  have haPow : (29 + 7 * r : ℤ) ^ (6 * k + 1) ≡ 29 ^ (6 * k + 1) [ZMOD 7] := ha.pow (6 * k + 1)
  have hbPow : (5 + 7 * s : ℤ) ^ (6 * k + 1) ≡ 5 ^ (6 * k + 1) [ZMOD 7] := hb.pow (6 * k + 1)
  have hdiff : (29 + 7 * r : ℤ) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 29 ^ (6 * k + 1) - 5 ^ (6 * k + 1) [ZMOD 7] := haPow.sub
    <| hbPow
  have h29 : (29 : ℤ) ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h29Pow : (29 : ℤ) ^ (6 * k + 1) ≡ 1 [ZMOD 7] := by
    simpa using h29.pow (6 * k + 1)
  have h5six : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h5mult : (5 : ℤ) ^ (6 * k) ≡ 1 [ZMOD 7] := by
    simpa [pow_mul] using h5six.pow k
  have h5Pow : (5 : ℤ) ^ (6 * k + 1) ≡ 5 [ZMOD 7] := by
    calc
      (5 : ℤ) ^ (6 * k + 1) = 5 ^ (6 * k) * 5 := by rw [pow_add]; simp
      _ = (1 : ℤ) * 5 [ZMOD 7] := h5mult.mul (Int.ModEq.refl 5)
      _ = 5 := by simp
  have hfixed : (29 : ℤ) ^ (6 * k + 1) - 5 ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * k + 1) - 5 ^ (6 * k + 1) ≡ 1 - 5 [ZMOD 7] := h29Pow.sub h5Pow
      _ = 3 [ZMOD 7] := by norm_num [Int.ModEq]
  exact hdiff.trans hfixed
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.me__641f9146

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 641f91465ff59538

4.258 258. power_difference_mod_seven_periodic

The problem

For every natural number t , if integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{6t+1} - b^{6t+1}$ leaves remainder 3 modulo 7.

Lean statement

```
theorem power_difference_mod_seven_periodic (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7] := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Retains the parent’s residue hypotheses and subtraction-of-powered-congruences checkpoint, while replacing the fixed exponent with the six-period parameterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent to an unbounded parameter and requires a new period-six power reduction modulo 7 before transporting and subtracting the congruences. This reasoning is absent from the parent’s direct fixed-exponent computation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent to arbitrary $6t+1$ and requires a new period-six modular argument to reduce both powers before subtracting. The parent’s fixed thirteenth-power calculation does not supply this parameterized reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent to arbitrary unbounded t and requires a period-six modular-power argument to reduce both powers. The parent’s direct numerical normalization for exponent 13 does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2096 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_mod_seven_periodic.extracted_1_1 (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7])  
(hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For all integers a and b and every natural number t , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then $a^{6t+1} - b^{6t+1}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_mod_seven_periodic (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7] := by
  have hperiod : 6 ∣ (6 * t + 1) - 1 := by
    refine ⟨t, ?_⟩
    omega
  have hle : 1 ≤ 6 * t + 1 := by
    omega
  have h29period : (29 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) ^ 1 [ZMOD 7] := by
    exact Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1) (by norm_num) hperiod hle (by norm_num) 29
  have h5period : (5 : ℤ) ^ (6 * t + 1) ≡ (5 : ℤ) ^ 1 [ZMOD 7] := by
    exact Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1) (by norm_num) hperiod hle (by norm_num) 5
  have ha_period : a ^ (6 * t + 1) ≡ (29 : ℤ) ^ (6 * t + 1) [ZMOD 7] := ha.pow (6 * t + 1)
  have hb_period : b ^ (6 * t + 1) ≡ (5 : ℤ) ^ (6 * t + 1) [ZMOD 7] := hb.pow (6 * t + 1)
  have hdiff : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) [ZMOD 7] := ha_period.sub hb_period
  have hbase : (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) - 5 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) ^ 1 - (5 : ℤ) ^ 1 [ZMOD 7] := h29period.sub h5period
      _ ≡ (29 : ℤ) - 5 [ZMOD 7] := by norm_num
  have hnum : (29 : ℤ) - 5 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdiff.trans (hbase.trans hnum)
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.me__a73264dd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a73264ddf683dbf4

4.259 259. generalized_residue_divisibility

The problem

Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that, for every nonnegative integer k , 7 divides $a^{6k+1} - b^{6k+1} - 3$.

Lean statement

```
theorem generalized_residue_divisibility (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3) := by
```

Parent. Two sensors report integers a and b , with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that 7 divides $a^{13} - b^{13} - 3$.

Parent in Lean

```
theorem variable_base_residue_divisibility (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ 13 - b ^ 13 - 3) := by
```

What it contributes. Supplied the power-transport, subtraction, fixed-residue, congruence-to-divisibility, and witness-negation checkpoints; the exponent is generalized to $6k+1$ and the periodic power checkpoint is proved inductively.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6k+1$. Its proof must establish the exponent periodicity modulo 7, especially for 5^{6k+1} , which is not supplied by recalling the parent's fixed-exponent calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6k+1$. Its proof must establish periodicity of powers modulo 7 and derive the residue for the symbolic exponent, which is not supplied by the parent's numerical normalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the isolated exponent 13 to every exponent of the form $6k+1$, requiring a new periodicity argument for $5^{6k+1} \equiv 5 \pmod{7}$. The parent's direct normalization of fixed powers does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2421 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_residue_divisibility.extracted_1_1 (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  7 ∣ a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3
```

Read back by a model that saw only the Lean. For integers a and b and a natural number k , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then 7 divides $a^{6k+1} - b^{6k+1} - 3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem generalized_residue_divisibility (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3) := by
  have ha_exp : a ^ (6 * k + 1) ≡ (29 : ℤ) ^ (6 * k + 1) [ZMOD 7] := ha.pow (6 * k + 1)
  have hb_exp : b ^ (6 * k + 1) ≡ (5 : ℤ) ^ (6 * k + 1) [ZMOD 7] := hb.pow (6 * k + 1)
  have h29base : (29 : ℤ) ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h29 : (29 : ℤ) ^ (6 * k + 1) ≡ 1 [ZMOD 7] := by
    simpa using h29base.pow (6 * k + 1)
  have h5 : (5 : ℤ) ^ (6 * k + 1) ≡ 5 [ZMOD 7] := by
    have hdiv : 6 ∣ (6 * k + 1 - 1) := by omega
    have hle : 1 ≤ 6 * k + 1 := by omega
    simpa using
      (Int.ModEq.pow_eq_pow (p := 7) (x := 6 * k + 1) (y := 1)
        (by norm_num : Nat.Prime 7) hdiv hle (by norm_num : 0 < (1 : ℕ)) (5 : ℤ)))
  have hdiff : a ^ (6 * k + 1) - b ^ (6 * k + 1) ≡ (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) [ZMOD 7] := ha_exp.sub hb_exp
  have hconst : (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) ≡ 1 - 5 [ZMOD 7] := h29.sub h5
      _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
  have hres : a ^ (6 * k + 1) - b ^ (6 * k + 1) ≡ 3 [ZMOD 7] := hdiff.trans hconst
  rw [Int.modEq_iff_dvd] at hres
  rcases hres with ⟨m, hm⟩
  refine ⟨-m, ?_⟩
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.ms.me__ad3f0fdf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ad3f0fdcf255c771

4.260 260. ternary_prefix_decode

The problem

Let n be a positive natural number. If a natural number has ternary digit list consisting of a terminal digit d , followed by n copies of 2 and then 1, prove that its positional value is $d + 3 \left(2 \sum_{i=0}^{n-1} 3^i + 3^n \right)$; in particular, the terminal digit is retained.

Lean statement

```
theorem ternary_prefix_decode
  (n d x : ℕ)

  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n)
```

Parent. Let n be positive with $n \leq 3$. If a natural number has ternary digit list consisting of a terminal digit d , followed by n copies of 2 and then 1, prove its positional value is $d + 3 \left(2 \sum_{i=0}^{n-1} 3^i + 3^n \right)$; in particular, the terminal digit is retained.

Parent in Lean

```
theorem ternary_bounded_prefix_decode
  (n d x : ℕ)
  (hn : 0 < n)
  (hbound : n ≤ 3)
  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n)
```

What it contributes. Retains the parent's ternary digit-list encoding and terminal-digit positional-decoding checkpoint, while replacing its finite case split with induction over an arbitrary prefix length.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the finite bound on n and proves the positional decoding law uniformly by induction, requiring a genuinely new recursive argument rather than the parent's finite case split. Its proof does not recall or reuse the parent's bounded conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the bound on n and proves the decoding identity uniformly by induction, requiring a geometric-sum lemma and recursive evaluation of `ofDigits`. This is genuine reasoning absent from the parent's finite case split and is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite bound and replaces the parent's three-case computation with a genuine induction proving the positional decoding law for every prefix length. A solver recalling the parent's bounded case split would not supply the new uniform induction argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2548 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_prefix_decode.extracted_1_1 (n d x : ℕ) (hn : 0 < n)
  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * ∑ i ∈ Finset.range n, 3 ^ i + 3 ^ n)
```

Read back by a model that saw only the Lean. For natural numbers n , d , and x , if n is positive and the base-3 digit list of x is d followed by n copies of 2 and then 1, then $x = d + 3(2\sum_{i \in \{0, \dots, n-1\}} 3^i + 3^n)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_prefix_decode
  (n d x : ℕ)

  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n) := by
  have hgeom : ∀ m : ℕ,
    2 * Finset.sum (Finset.range m) (fun i => 3 ^ i) + 1 = 3 ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp only [Finset.sum_range_succ, pow_succ]
      omega
  have hsum : ∀ m : ℕ,
    Nat.ofDigits 3 (List.replicate m 2 ++ [1]) =
      2 * Finset.sum (Finset.range m) (fun i => 3 ^ i) + 3 ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp [List.replicate_succ, Nat.ofDigits_cons, ih,
        Finset.sum_range_succ, pow_succ]
      have hg := hgeom m
      omega
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  rw [h] at hdecode
  rw [Nat.ofDigits_cons, hsum n] at hdecode
  exact hdecode
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me.me.me__7899a23f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c6c3a577dd4c7ba1

4.261 261. ternary_terminal_digit_decode

The problem

Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Lean statement

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

What it contributes. Uses the parent's `Nat.ofDigits_digits` recovery and positional evaluation approach, generalized from the fixed least-significant digit 2 to $d < 3$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the terminal ternary digit and requires deriving the symbolic decoding formula $x = d + 51$; the fixed-value parent proof does not supply that algebra. The specialization $x = 53$ is only a consequence of the new general law.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading digit and proves the symbolic positional-decoding law $x = d + 51$, rather than merely replaying the fixed computation. The specialization implication is immediate from this new general result, but the main generalized claim requires symbolic decoding absent from the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the terminal digit and proves the new symbolic decoding law $x = d + 51$; the parent only supplies the fixed case $d = 2$. The parent's proof cannot be recalled verbatim to establish the generalized arithmetic identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2511 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_terminal_digit_decode.extracted_1_1 (d x : ℕ) (hd : d < 3) (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Read back by a model that saw only the Lean. For natural numbers d and x , if $d < 3$ and the base-3 digit expansion of x , listed from least significant to most significant, is $[d, 2, 2, 1]$, then $x = d + 51$, and if $d = 2$, then $x = 53$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53) := by
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  have hvalue : x = d + 51 := by
    calc
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]
  constructor
  · exact hvalue
  · intro hdigit
    rw [hvalue, hdigit]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me__0fd2ae3f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 46d57d77b1203e42

4.262 262. choose_translated_square_level_iff

The problem

For positive integers n and k , show that $\left[\left(x - \binom{n}{k} \right)^2 - \binom{n}{k} \right] = \binom{n-1}{k} + \binom{n-1}{k-1}$ if and only if $x = \binom{n}{k}$, for real x .

Lean statement

```
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  ((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
  x = (Nat.choose n k : ℝ)
```

Parent. For a positive integer n and $0 \leq k \leq n$, show that $\binom{n}{k} = 1$ when $k = 0$, while for $k > 0$, $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

What it contributes. Supplied the positive-index Pascal recurrence, used as the load-bearing rewrite of the vertical offset.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child retains the Pascal recurrence but adds a genuine translated-square classification: the solver must normalize the recursive level and prove that a real square vanishes exactly at its center. Memorizing the parent's boundary-case proof does not supply this argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a Pascal identity into a real-valued level-set classification. It must cast the recursive identity and prove that a translated square equals zero exactly at its center, reasoning not supplied by the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child combines Pascal recursion with a genuinely new real-algebraic classification: it reduces the translated-square equation to a zero square and proves an iff statement. Memorizing the parent's boundary-case Pascal proof would not supply the square-equation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2415 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_translated_square_level_iff.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :  
(x - ↑(n.choose k)) ^ 2 + ↑(n.choose k) = ↑((n - 1).choose k + (n - 1).choose (k - 1)) ↔ x = ↑(n.choose k)
```

Read back by a model that saw only the Lean. For natural numbers n and k with $0 < n$ and $0 < k$, and for a real number x , the equation $(x - C(n,k))^2 + C(n,k) = C(n-1,k) + C(n-1,k-1)$ holds if and only if $x = C(n,k)$, where $C(a,b)$ denotes the binomial coefficient and subtraction of natural numbers is truncated subtraction.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :  
((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔  
x = (Nat.choose n k : ℝ) := by  
have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by  
  simpa [Nat.add_comm] using Nat.choose_eq_choose_pred_add hn hk  
have hoff : (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by  
  exact_mod_cast hpascal  
constructor  
· intro h  
  nlinarith [sq_nonneg (x - (Nat.choose n k : ℝ))]  
· intro hx  
  rw [hx]  
  simp [hoff]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me.me__ae9ead38

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ae9ead389b8cfd91

4.263 263. choose_boundary_pascal

The problem

For a positive integer n and $0 \leq k \leq n$, show that $\binom{n}{k} = 1$ when $k = 0$, while for $k > 0$, $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Lean statement

```
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

Parent. Show that for positive integers n and k with $k \leq n$, we have $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem numbertheory_nckeqnm1ckpm1ckm1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
```

What it contributes. Supplies the positive-index Pascal recurrence via `Nat.choose_eq_choose_pred_add`; the new zero-index branch is handled separately.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds the genuinely new boundary case $k=0$, where Pascal's positive-index recurrence does not apply, and removes the unnecessary bound $k \leq n$. Its proof requires a case split and separately establishes the base value.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the recurrence to the boundary case $k = 0$, requiring a case split and a separate proof that the choose value is 1. The parent's positive-index lemma cannot establish this boundary branch, and the child also removes the unnecessary bound $k \leq n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the recurrence to the boundary case $k=0$, requiring a case split and removal of the parent's positivity hypothesis on k . The parent's proof only handles the positive-index case and does not establish the new boundary behavior.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2406 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_boundary_pascal.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : k ≤ n) :  
  n.choose k = if k = 0 then 1 else (n - 1).choose k + (n - 1).choose (k - 1)
```

Read back by a model that saw only the Lean. For natural numbers n and k , if n is positive and k is at most n , then the binomial coefficient n choose k equals 1 when $k = 0$, and otherwise equals the sum of $(n - 1)$ choose k and $(n - 1)$ choose $(k - 1)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :  
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by  
  by_cases hzero : k = 0  
  · subst k  
    simp  
  · have hkpos : 0 < k := Nat.pos_of_ne_zero hzero  
    have hpascal : Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k :=  
      Nat.choose_eq_choose_pred_add hn hkpos  
    rw [if_neg hzero]  
    calc  
      Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := hpascal  
      _ = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := Nat.add_comm _ _
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me__ab239085

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint c0f43117760bc230

4.264 264. complementary_choose_eq_one_iff

The problem

For integers $n \geq 2$ and $0 < k \leq n$, classify all k such that the complementary binomial coefficient $\binom{n}{n-k}$ equals 1.

Lean statement

```
theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose n (n - k) = 1 ↔ k = n
```

Parent. For positive integers n and k with $k \leq n$, show that $\binom{n}{n-k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

What it contributes. Reused the parent’s complementary-index symmetry checkpoint, while changing the conclusion to a bounded classification of unit-valued coefficients.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the Pascal identity into a classification theorem for when a complementary binomial coefficient equals one. Its proof requires the characterization `Nat.choose_eq_one_iff` and excludes the zero-index branch using `hk`; recalling the parent does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from a Pascal recurrence to a classification of when a complementary binomial coefficient equals 1. Its proof requires the distinct theorem ‘`Nat.choose_eq_one_iff`’ and a case split using the positivity hypothesis, so recalling the parent’s Pascal proof does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the Pascal recurrence into a classification theorem for when the complementary binomial coefficient equals one. Although it reuses binomial symmetry, its essential argument relies on the distinct characterization `Nat.choose n k = 1` and a case split, so recalling the parent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2629 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_choose_eq_one_iff.extracted_1_1 (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :  
  n.choose (n - k) = 1 ↔ k = n
```

Read back by a model that saw only the Lean. For natural numbers n and k satisfying $2 \leq n$, $0 < k$, and $k \leq n$, the binomial coefficient " n choose $(n - k)$ " equals 1 if and only if $k = n$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :  
  Nat.choose n (n - k) = 1 ↔ k = n := by  
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=  
    Nat.choose_symm hkn  
  have hone : Nat.choose n k = 1 ↔ k = 0 ∨ n = k :=  
    Nat.choose_eq_one_iff  
  constructor  
  · intro h  
    have hcases : k = 0 ∨ n = k := hone.mp (hsymm.symm.trans h)  
    rcases hcases with hk0 | hnk  
    · omega  
    · exact hnk.symm  
  · intro h  
    subst k  
    simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.me__6b4f9b19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 803ae78f47813c8c

4.265 265. prime_dvd_iff_coupled_product_uniform_offset

The problem

For a prime number p and a positive exponent e , characterize the integer offsets m for which, for every integer n , p divides n exactly when $(n(n + m))^e \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_coupled_product_uniform_offset
  (m : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e) :
  ((∀ n : ℤ, (↑p : ℤ) ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD p]) ↔
   (↑p : ℤ) ∣ m)
```

Parent. For a prime number p , integers n, k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n(n + kp))^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p]
```

What it contributes. Supplied the prime-modulus, positive-power, natAbs product-splitting, and divisibility-cancellation proof checkpoints; the fixed multiple $k \cdot p$ was replaced by the active hypothesis $p \mid m$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine offset characterization: its forward direction specializes the universal criterion at $n = -m$ to derive $p \mid m$, while the reverse direction uses $p \mid m$ to recover the coupled-product criterion. This introduces arbitrary offsets and a new necessity argument beyond the parent's fixed multiple-of- p translate.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: it characterizes exactly which offsets work uniformly, adding a new forward argument by specializing at $n = -m$. The reverse direction generalizes the parent's divisibility transfer, but recalling the parent alone does not establish the offset characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed translated-product equivalence into a uniform offset characterization. Its forward direction requires the new specialization $n = -m$ to derive $p \mid m$, so recalling the parent cannot solve the theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2635 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_coupled_product_uniform_offset.extracted_1_1 (m : ℤ) (p e : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) : (∀ (n : ℤ), ↑p ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD ↑p]) ↔ ↑p ∣ m
```

Read back by a model that saw only the Lean. *Let m be an integer and let p and e be natural numbers such that p is prime and e is positive. Then the following are equivalent: for every integer n , p divides n if and only if the e -th power of $n(n + m)$ is congruent to 0 modulo p ; and p divides m .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_coupled_product_uniform_offset
  (m : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e) :
  ((∀ n : ℤ, (↑p : ℤ) ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD p]) ↔
   (↑p : ℤ) ∣ m) := by
  simp only [Int.modEq_zero_iff_dvd]
  constructor
  · intro h
    have hneg : (↑p : ℤ) ∣ -m := by
      apply (h (~m)).mpr
      simp [Nat.ne_of_gt he_pos]
    obtain ⟨a, ha⟩ := hneg
    refine ⟨-a, ?_⟩
    calc
      m = -(-m) := by ring
      _ = -((↑p : ℤ) * a) := by rw [ha]
      _ = (↑p : ℤ) * (-a) := by ring
  · intro hm n
    have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
    constructor
    · intro hdiv
      have hproduct : (↑p : ℤ) ∣ n * (n + m) := by
        exact dvd_mul_of_dvd_left hdiv _
      exact dvd_pow hproduct he_ne
    · intro hpower
      have hproduct : (↑p : ℤ) ∣ n * (n + m) := by
        exact Int.Prime.dvd_pow' hp hpower
      have hproduct_nat : p ∣ (n * (n + m)).natAbs :=
        (Int.natCast_dvd).mp hproduct
      rw [Int.natAbs_mul] at hproduct_nat
      rcases hp.dvd_mul.mp hproduct_nat with hdiv | htranslated
      · exact (Int.natCast_dvd).mpr hdiv
      · have htranslated_int : (↑p : ℤ) ∣ n + m :=
          (Int.natCast_dvd).mpr htranslated
        have hcanceled : (↑p : ℤ) ∣ (n + m) - m := by
          exact dvd_sub htranslated_int hm
        convert hcanceled using 1 <|> ring
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh.me__0286cdbe

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0286cdbe3cb2b06d

4.266 266. prime_dvd_iff_even_bounded_power_modEq_zero

The problem

For a prime number p , an integer n , and a natural number e with $1 \leq e$, prove that $p \mid n$ if and only if $n^{2e} \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_even_bounded_power_modEq_zero (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :
  (↑p : ℤ) ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD p]
```

Parent. Show that for any prime p and any integer n , we have $p \mid n$ if and only if $n^2 \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem numbertheory_prmdvsneqnsqmodpeq0 (n : ℤ) (p : ℕ) (h₀ : Nat.Prime p) :
  ↑p ∣ n ↔ n ^ 2 ≡ 0 [ZMOD p] := by
```

What it contributes. Preserves the prime-modulus and integer-divisibility setting and reuses its square-divisibility bridge on n^e , while adding the bounded exponent parameter and a power-divisibility step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the exponent and requires reducing divisibility of $n^{(2e)}$ to a square, then using primality to descend from $p \mid n^e$ to $p \mid n$. This is not recoverable by simply recalling or applying the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the square criterion to every positive even exponent. Its reverse direction requires introducing n^e , applying the prime square-divisibility lemma, and then lifting divisibility through a power—reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the square criterion to a symbolic positive exponent. Its converse requires rewriting the even power as a square of n^e , applying the prime square-divisibility lemma, and then using prime divisibility of a power to recover $p \mid n$; this is not mere recall of the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2084 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_even_bounded_power_modEq_zero.extracted_1_1 (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :  
  ↑p ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For an integer n and natural numbers p and e , if p is prime and $1 \leq e$, then p divides n if and only if n raised to the power $2e$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem prime_dvd_iff_even_bounded_power_modEq_zero (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :  
  (↑p : ℤ) ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD p] := by  
  rw [Int.modEq_zero_iff_dvd]  
  constructor  
  · intro h  
    exact dvd_pow h (Nat.ne_of_gt (Nat.mul_pos (by norm_num) (Nat.zero_lt_of_lt he₁)))  
  · intro h  
    apply Int.natCast_dvd.mpr  
    have hs : (↑p : ℤ) ∣ (n ^ e) ^ 2 := by  
      simpa [pow_mul, Nat.mul_comm] using h  
    have hpabs : p ∣ (n ^ e).natAbs :=  
      Int.Prime.dvd_natAbs_of_coe_dvd_sq h₀ (n ^ e) hs  
    apply h₀.dvd_of_dvd_pow  
    simpa [Int.natAbs_pow] using hpabs
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me__dfe2a291

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 960dc5475db79eb2

4.267 267. seven_not_dvd_five_pow_twenty_one

The problem

Show that 7 does not divide 5^{21} by determining its remainder modulo 7.

Lean statement

```
theorem seven_not_dvd_five_pow_twenty_one : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 21
```

Parent. Show that 7 does not divide 5^{15} .

Parent in Lean

```
theorem seven_not_dvd_five_pow_fifteen : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 15
```

What it contributes. Supplied the divisibility-to-modular-contradiction checkpoint and theorem-proof style; the child changes the exponent and computes the direct residue \$6\$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the exponent and independently computes the residue $(5^{21} \bmod 7) = 6$, then derives a contradiction from divisibility implying residue 0. The parent's square-based argument for exponent 15 does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the new exponent case directly by converting divisibility to a zero residue and computing $(5 : \mathbb{Z})^{21} \equiv 6 \pmod{7}$, rather than reusing the parent's square criterion and exponent-doubling argument. It satisfies the requested fresh modular contradiction and cannot be solved by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different modular argument: it derives the exponent-21 residue as 6 directly and contradicts divisibility via the zero-residue criterion. The parent's squaring argument and residue-1 conclusion do not supply this proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2425 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem seven_not_dvd_five_pow_twenty_one.extracted_1_1 : ¬ 7 ∣ 5 ^ 21
```

Read back by a model that saw only the Lean. 7 does not divide 5 raised to the 21st power.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem seven_not_dvd_five_pow_twenty_one : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 21 := by
  intro hdiv
  have hzero : (5 : ℤ) ^ 21 ≡ 0 [ZMOD 7] := by
    exact (Int.modEq_zero_iff_dvd).2 hdiv
  have hresidue : (5 : ℤ) ^ 21 ≡ 6 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have contradiction_mod : (0 : ℤ) ≡ 6 [ZMOD 7] := by
    exact hzero.symm.trans hresidue
  norm_num [Int.ModEq] at contradiction_mod
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.me__787dad62

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 787dad628ffb8836

4.268 268. dvd_add_one_iff_exp_eq_three

The problem

Let x be an integer congruent to 5 modulo 7. For $0 \leq k \leq 3$, show that 7 divides $x^k + 1$ exactly when $k = 3$.

Lean statement

```
theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3
```

Parent. Let x be an integer congruent to 5 modulo 7. Show that 7 does not divide x^{15} .

Parent in Lean

```
theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15
```

What it contributes. Supplied the invariant $x \equiv 5$ modulo 7 and the modular-power transport method.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes nondivisibility of a fixed fifteenth power into a bounded exact characterization for all exponents up to 3. It requires transporting a variable power and performing a finite exponent analysis, so the parent's proof cannot be recalled to solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a valid bounded characterization: it transports the residue for arbitrary k and separately checks $k=0,1,2,3$, with divisibility occurring only at $k=3$. The parent's fixed-exponent contradiction proof does not supply this case analysis or characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes nondivisibility of a fixed fifteenth power into a bounded exact characterization for all exponents up to 3. It requires transporting a variable power and checking each bounded exponent, so recalling the parent proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2439 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dvd_add_one_iff_exp_eq_three.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) :
  7 ∣ x ^ k + 1 ↔ k = 3
```

Read back by a model that saw only the Lean. For an integer x satisfying $x \equiv 5$ modulo 7 and a natural number k with $k \leq 3$, 7 divides $x^k + 1$ if and only if $k = 3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3 := by
  have htransport : (7 : ℤ) ∣ x ^ k + 1 ↔ (7 : ℤ) ∣ (5 : ℤ) ^ k + 1 := by
    have hp : x ^ k + 1 ≡ (5 : ℤ) ^ k + 1 [ZMOD 7] := by
      exact (h.pow k).add_right 1
    constructor
    · intro hd
      have hz : x ^ k + 1 ≡ 0 [ZMOD 7] := by
        rw [Int.modEq_zero_iff_dvd]
        exact hd
      have hz' : (5 : ℤ) ^ k + 1 ≡ 0 [ZMOD 7] := hp.symm.trans hz
      rw [Int.modEq_zero_iff_dvd] at hz'
      exact hz'
    · intro hd
      have hz : (5 : ℤ) ^ k + 1 ≡ 0 [ZMOD 7] := by
        rw [Int.modEq_zero_iff_dvd]
        exact hd
      have hz' : x ^ k + 1 ≡ 0 [ZMOD 7] := hp.trans hz
      rw [Int.modEq_zero_iff_dvd] at hz'
      exact hz'
  have hbase : (7 : ℤ) ∣ (5 : ℤ) ^ k + 1 ↔ k = 3 := by
    interval_cases k <|> norm_num
  exact htransport.trans hbase
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh.me__814266dc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 814266dc73c2bfad

4.269 269. square_translation_modulus

The problem

For positive integers n and all integers a, k , prove that translating a by a multiple of n preserves its square modulo n : $(a + nk)^2 \equiv a^2 \pmod{n}$.

Lean statement

```
theorem square_translation_modulus (a k n : ℤ) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n] := by
```

Parent. For all integers a and k , prove that translating a by a multiple of 3 preserves its square modulo 3: $(a + 3k)^2 \equiv a^2 \pmod{3}$.

Parent in Lean

```
theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
```

What it contributes. Generalizes the parent's square-expansion and modular-translation checkpoint from fixed modulus 3 to an arbitrary positive integer modulus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the modulus from the fixed value 3 to an arbitrary integer n , and its proof requires explicit divisibility reasoning rather than reusing the parent's modulo-3 simplification. The claim is valid even without a positivity assumption on n .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the modulus from the fixed value 3 to an arbitrary integer n , so recalling the parent does not solve it. Its proof must establish divisibility for a symbolic modulus rather than reuse the parent's concrete modular simplification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed modulus 3 to an arbitrary integer modulus and proves the result via divisibility and a congruence calculation, rather than recalling the parent's simp-based modulo-3 proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2687 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_translation_modulus.extracted_1_1 (a k n : ℤ) (hn : 0 < n) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n]
```

Read back by a model that saw only the Lean. For integers a , k , and n with $n > 0$, $(a + nk)^2$ is congruent to a^2 modulo n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_translation_modulus (a k n : ℤ) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n] := by
  have h_expand : (a + n * k) ^ 2 = a ^ 2 + n * (2 * a * k + n * k ^ 2) := by
    ring
  have h_multiple : n * (2 * a * k + n * k ^ 2) ≡ 0 [ZMOD n] := by
    apply Int.modEq_zero_iff_dvd.mpr
    refine ⟨2 * a * k + n * k ^ 2, ?_⟩
    ring
  calc
    (a + n * k) ^ 2 = a ^ 2 + n * (2 * a * k + n * k ^ 2) := h_expand
    _ ≡ a ^ 2 + 0 [ZMOD n] := h_multiple.add_left (a ^ 2)
    _ = a ^ 2 := by ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__me__7c947b95

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6948b265b39ea149

4.270 270. square_mod_three_translation

The problem

For all integers a and k , prove that translating a by a multiple of 3 preserves its square modulo 3: $(a + 3k)^2 \equiv a^2 \pmod{3}$.

Lean statement

```
theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
```

Parent. Show that the square of any integer is congruent to 0 or 1 modulo 3.

Parent in Lean

```
theorem numbertheory_sqmod3in01d (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ∨ a ^ 2 ≡ 1 [ZMOD 3] := by
```

What it contributes. Uses the parent's modulus-3 square context, while replacing its residue classification with a new congruence-expansion checkpoint for arbitrary translations by $3k$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different translation-invariance congruence: expanding $(a+3k)^2$ and showing the difference is divisible by 3. The parent only classifies square residues modulo 3, so recalling its case split does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different claim: translating by a multiple of 3 preserves the square modulo 3. Its algebraic expansion and divisibility argument do not reuse the parent's residue-class case split, so recalling the parent would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a distinct translation-invariance congruence, requiring polynomial expansion and showing the added multiple-of-3 terms vanish modulo 3. The parent's residue case split only classifies square residues and does not directly supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2680 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_mod_three_translation.extracted_1_1 (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3]
```


Read back by a model that saw only the Lean. *For all integers a and k , $(a + 3k)^2$ is congruent to a^2 modulo 3.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
  have h_expand : (a + 3 * k) ^ 2 = a ^ 2 + 3 * (2 * a * k + 3 * k ^ 2) := by
    ring
  rw [h_expand]
  simp [Int.ModEq, Int.add_emod, Int.mul_emod]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__me__46dc4c00

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 46dc4c00eaa36df3

4.271 271. finite_field_prod_units_root_characterization

The problem

Let K be a field with finitely many nonzero elements. Prove that the product of all nonzero elements of K is either 1, or else is not 1 and is the unique nonidentity element z satisfying $z^2 = 1$.

Lean statement

```
theorem finite_field_prod_units_root_characterization {K : Type*} [Field K] [Fintype K*] :
  (∏ x : K*, x) = 1 ∨
  ((∏ x : K*, x) ≠ 1 ∧ ∃ z : K*, z ^ 2 = 1 → z = 1 ∨ z = ∏ x : K*, x) := by
```

Parent. Let K be a finite field. Prove that the product of the nonzero elements of K is -1 .

Parent in Lean

```
theorem Artin_exercise_13_6_10 {K : Type*} [Field K] [Fintype K*] :
  (∏ x : K*, x) = -1 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine characterization of all square roots of unity in the unit group, requiring factorization in the field and a characteristic-two case split. The parent's product identity alone does not supply this root analysis.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine root-characterization obligation: it factors z^2-1 in the field and handles the resulting cases, beyond the parent's product identity. Although it reuses the parent lemma for the product, the new universal statement is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child retains the product identity but additionally proves a complete characterization of square roots of unity in the unit group via factorization and the domain property. That root-classification argument is not supplied by the parent's direct product lemma, so recalling the parent does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1972 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_field_prod_units_root_characterization.extracted_1_1.{u_1} {K : Type u_1} [inst : Field K]
  [inst_1 : Fintype K*] :  $\prod x, x = 1 \vee \prod x, x \neq 1 \wedge \forall (z : K^*), z^2 = 1 \rightarrow z = 1 \vee z = \prod x, x$ 
```

Read back by a model that saw only the Lean. *Let K be a field such that its group of units K^x is finite. Then either the product of all units of K equals 1, or that product is not equal to 1 and every unit z satisfying $z^2 = 1$ is either 1 or equal to the product of all units.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd

theorem finite_field_prod_units_root_characterization {K : Type*} [Field K] [Fintype K*] :
  ( $\prod x : K^*, x$ ) = 1  $\vee$ 
  (( $\prod x : K^*, x$ )  $\neq$  1  $\wedge \forall z : K^*, z^2 = 1 \rightarrow z = 1 \vee z = \prod x : K^*, x$ ) := by
  classical
  have hprod : ( $\prod x : K^*, x$ ) = (-1 : K*) :=
    FiniteField.prod_univ_units_id_eq_neg_one (K := K)
  have hroot :  $\forall z : K^*, z^2 = 1 \rightarrow z = 1 \vee z = (-1 : K^*)$  := by
    intro z hz
    have hzK : (z : K) ^ 2 = 1 := by
      simp using congrArg (fun u : K* => (u : K)) hz
    have hfac : ((z : K) - 1) * ((z : K) + 1) = 0 := by
      calc
        ((z : K) - 1) * ((z : K) + 1) = (z : K) ^ 2 - 1 := by ring
        _ = 0 := by rw [hzK]; ring
    rcases mul_eq_zero.mp hfac with hzero | hzero
    · left
      apply Units.ext
      have hval : (z : K) = 1 := by linear_combination hzero
      simp using hval
    · right
      apply Units.ext
      have hval : (z : K) = -1 := by linear_combination hzero
      simp using hval
  rw [hprod]
  by_cases heq : (-1 : K*) = 1
  · exact Or.inl heq
  · right
    constructor
    · exact heq
    · intro z hz
      exact hroot z hz
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_13_6_10__mh__96bd728d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7f6c646d04f96f55

4.272 272. even_finite_group_has_fixed_point_free_left_involution

The problem

Prove that every finite group of even order contains an element x of order 2 such that left multiplication by x is a fixed-point-free involution.

Lean statement

```
theorem even_finite_group_has_fixed_point_free_left_involution {G : Type*} [Group G] [Fintype G]
  (hG : Even (Fintype.card G)) :
  ∃ x : G, orderOf x = 2 ∧ ∃ f : G → G,
    f = (fun y => x * y) ∧ Function.Involutive f ∧ ∀ y : G, f y ≠ y
```

Parent. Prove that a group of even order contains an element of order 2.

Parent in Lean

```
theorem Artin_exercise_2_11_3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses the parent's order-2 witness but adds genuinely nontrivial mathematics: deriving $x*x = 1$ and $x \neq 1$, then proving left multiplication by x is involutive and has no fixed points via cancellation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves the order-2 existence but adds a genuinely new construction: left multiplication by that element must be proved involutive and fixed-point free. These obligations require deriving $x^2 = 1$ and $x \neq 1$, reasoning not supplied by the parent's Cauchy argument alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive structure: it must construct the left-translation involution and prove both involutivity and absence of fixed points from order two. The parent only supplies the order-two element, so recalling its proof does not discharge the new obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1904 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem even_finite_group_has_fixed_point_free_left_involution.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
  [inst.1 : Fintype G] (hG : Even (card G)) :
  ∃ x, orderOf x = 2 ∧ ∃ f, (f = fun y => x * y) ∧ Involutive f ∧ ∀ (y : G), f y ≠ y
```

Read back by a model that saw only the Lean. *For every finite group G of even cardinality, there exists an element x of G whose order is 2 and a function $f: G \rightarrow G$ such that f is the left-multiplication map $f(y) = x \cdot y$, f is involutive ($f(f(y)) = y$ for every $y \in G$), and $f(y) \neq y$ for every $y \in G$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem even_finite_group_has_fixed_point_free_left_involution {G : Type*} [Group G] [Fintype G]
  (hG : Even (Fintype.card G)) :
  ∃ x : G, orderOf x = 2 ∧ ∃ f : G → G,
    f = (fun y => x * y) ∧ Function.Involutive f ∧ ∀ y : G, f y ≠ y := by
  obtain ⟨k, hk⟩ := hG
  have h_two_dvd : 2 ∣ Fintype.card G := by
    exact ⟨k, by simpa [two_mul] using hk⟩
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card (G := G) (p := 2) h_two_dvd
  have hsq : x * x = 1 := by
    have hp := pow_orderOf_eq_one x
    rw [hx] at hp
    simpa [pow_two] using hp
  have hxne : x ≠ 1 := by
    intro h
    subst x
    simpa using hx
  refine ⟨x, hx, (fun y => x * y), rfl, ?_, ?_⟩
  · intro y
    change x * (x * y) = y
    rw [← mul_assoc, hsq, one_mul]
  · intro y h
    have h' : x * y = 1 * y := by
      simpa using h
    exact hxne (mul_right_cancel h')
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_2_11_3__mh__40815eed

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d2fbc07787022f25

4.273 273. exists_subgroup_card_two_of_card_dvd_two

The problem

Show that every finite group whose order is divisible by 2 contains a subgroup with exactly 2 elements.

Lean statement

```
theorem exists_subgroup_card_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ H : Subgroup G, Nat.card H = 2
```

Parent. Prove that a finite group whose number of elements is divisible by 2 contains an element of order 2.

Parent in Lean

```
theorem finite_group_order_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ x : G, orderOf x = 2
```

What it contributes. Supplies the divisible-by-two finite-group argument yielding an element of order two; that witness is then consumed to construct the subgroup target.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves genuinely new mathematics: it constructs the subgroup of powers generated by an element of order two and establishes that its cardinality is two. The parent only provides existence of the order-two element and does not supply this subgroup construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuine subgroup construction: it turns the order-two witness into the subgroup of integer powers and proves its cardinality via the cardinality theorem for cyclic subgroups. The parent's direct existence proof supplies the witness but does not supply this structural argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from finding an element of order two to constructing a subgroup of cardinality two. It must build the cyclic subgroup generated by the witness and prove its cardinality, which is substantive reasoning absent from the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2308 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_subgroup_card_two_of_card_dvd_two.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  [inst_1 : Fintype G] (hG : 2 ∣ card G) : ∃ H, Nat.card ↑H = 2
```

Read back by a model that saw only the Lean. *For a finite group G , if 2 divides the cardinality of G , then there exists a subgroup H of G whose cardinality is 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem exists_subgroup_card_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ H : Subgroup G, Nat.card H = 2 := by
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card (G := G) (p := 2) hG
  let H : Subgroup G := Subgroup.zpowers x
  have hcard : Nat.card H = orderOf x := by
    dsimp [H]
    rw [Nat.card_eq_fintype_card, Fintype.card_zpowers]
  have hH : Nat.card H = 2 := by
    calc
      Nat.card H = orderOf x := hcard
      _ = 2 := hx
  exact ⟨H, hH⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Artin_exercise_2_11_3__ms.mh__7b9f6edc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7b9f6edc6d4eb13b

4.274 274. subgroup_contains_adjacent_power_words

The problem

Let a and b be elements of a group and let n be a nonnegative integer. A subgroup contains both a and b exactly when it contains both bab^n and bab^{n+1} .

Lean statement

```
theorem subgroup_contains_adjacent_power_words {G : Type*} [Group G]
  (a b : G) (n : ℕ) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H)
```

Parent. Let a and b be elements of a group. Show that a subgroup contains both a and b exactly when it contains both bab^2 and bab^3 .

Parent in Lean

```
theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed powers 2 and 3 to arbitrary adjacent exponents n and $n+1$. Its reverse direction requires symbolic power identities and a new algebraic normalization argument, which the parent's fixed-exponent proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the two exponents and requires proving the general cancellation identity for adjacent powers, which is not supplied by the parent's fixed-exponent simplifications.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the two adjacent exponents, requiring a general power-cancellation identity for arbitrary n rather than the parent's fixed exponents 2 and 3. The added 'pow_succ' rewrite and algebraic normalization are substantive proof obligations, not decoration or recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2302 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem subgroup_contains_adjacent_power_words.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (a b : G) (n : ℕ)
  (H : Subgroup G), a ∈ H ∧ b ∈ H ↔ b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H
```

Read back by a model that saw only the Lean. *For every group G , every $a, b \in G$, every natural number n , and every subgroup H of G , a belongs to H and b belongs to H if and only if both $b * a * b^n$ and $b * a * b^{n+1}$ belong to H .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem subgroup_contains_adjacent_power_words {G : Type*} [Group G]
  (a b : G) (n : ℕ) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H) := by
  intro H
  constructor
  · rintro ⟨ha, hb⟩
    constructor
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb n)
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb (n + 1))
  · rintro ⟨h₁, h₂⟩
    have hb_prod :
      (b * a * b ^ n)⁻¹ * (b * a * b ^ (n + 1)) ∈ H :=
      H.mul_mem (H.inv_mem h₁) h₂
    have hb_eq :
      (b * a * b ^ n)⁻¹ * (b * a * b ^ (n + 1)) = b := by
      rw [pow_succ]
      group
    have hb : b ∈ H := by
      exact hb_eq ▸ hb_prod
    have ha_prod :
      (b⁻¹ * (b * a * b ^ n)) * (b ^ n)⁻¹ ∈ H :=
      H.mul_mem
      (H.mul_mem (H.inv_mem hb) h₁)
      (H.inv_mem (H.pow_mem hb n))
    have ha_eq :
      (b⁻¹ * (b * a * b ^ n)) * (b ^ n)⁻¹ = a := by
      group
    have ha : a ∈ H := by
      exact ha_eq ▸ ha_prod
    exact ⟨ha, hb⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__ms.mh__108154db

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 108154db84973d4c

4.275 275. unit_power_characterization_in_subgroup

The problem

Let g be a unit of exact order p . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a unit u such that $u^m = 1$ exactly when p divides m .

Lean statement

```
theorem unit_power_characterization_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ ∀ m : ℕ, u ^ m = 1 ↔ p ∣ m
```

Parent. Let p be the exact order of a unit g . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a cyclic subgroup of cardinality p .

Parent in Lean

```
theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)*, K ≤ H ∧ Nat.card K = p
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence of a subgroup of cardinality p to a power-period characterization of an element in H . Although it reuses the parent's derivation that $g \in H$, it must additionally prove the new equivalence using the order-divisibility theorem; the parent's subgroup-cardinality argument does not supply this.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from constructing a subgroup of cardinality p to characterizing the exact powers of an element. After the shared extraction of $g \in H$, it requires the new order-divisibility argument via `orderOf_dvd_iff_pow_eq_one`; the parent's subgroup-cardinality proof does not supply this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential conclusion from finding a subgroup of cardinality p to characterizing all powers of a specific unit by divisibility by p . After reusing the extraction of $g \in H$, it requires the new `orderOf_dvd_iff_pow_eq_one` argument, which the parent's subgroup-cardinality proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2323 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unit_power_characterization_in_subgroup.extracted_1_1 (n p k : ℕ) (g b : (ZMod n)*) (hg : orderOf g = p)
  (H : Subgroup (ZMod n)*) (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ u ∈ H, ∀ (m : ℕ), u ^ m = 1 ↔ p ∣ m
```

Read back by a model that saw only the Lean. For natural numbers n , p , and k , units g and b of $Z\text{Mod } n$, and a subgroup H of the units of $Z\text{Mod } n$, suppose that the order of g is p , that $b \cdot g \cdot b^k$ belongs to H , and that $b \cdot g \cdot b^{k+1}$ belongs to H . Then there exists an element u in H such that, for every natural number m , $u^m = 1$ if and only if p divides m .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem unit_power_characterization_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ ∀ m : ℕ, u ^ m = 1 ↔ p ∣ m := by
  have hbprod :
    (b * g * b ^ k)⁻¹ * (b * g * b ^ (k + 1)) ∈ H :=
    H.mul_mem (H.inv_mem h₁) h₂
  have hb : b ∈ H := by
    convert hbprod using 1 <|> group
  have hbinv : b⁻¹ ∈ H := H.inv_mem hb
  have hbk : (b⁻¹) ^ k ∈ H := H.pow_mem hbinv k
  have hleft : b⁻¹ * (b * g * b ^ k) ∈ H := H.mul_mem hbinv h₁
  have hall : (b⁻¹ * (b * g * b ^ k)) * (b⁻¹) ^ k ∈ H :=
    H.mul_mem hleft hbk
  have hgH : g ∈ H := by
    convert hall using 1 <|> group
  have hpower : ∀ m : ℕ, g ^ m = 1 ↔ p ∣ m := by
    intro m
    rw [← orderOf_dvd_iff_pow_eq_one, hg]
  exact ⟨g, hgH, hpower⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__x__Herstein_exercise_2_4_36__mh.xe.mh.mh__7056edf3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7056edf3a6009cab

4.276 276. cyclic_subgroup_of_prime_order_in_subgroup

The problem

Let p be the exact order of a unit g . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a cyclic subgroup of cardinality p .

Lean statement

```
theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)*, K ≤ H ∧ Nat.card K = p
```

Parent. Let p be prime, and let g be a unit of exact order p . Show that any subgroup containing bgb^2 and bgb^3 also contains a unit of exact order p .

Parent in Lean

```
theorem prime_order_unit_forced_into_subgroup
  (n p : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ 2 ∈ H) (h₂ : b * g * b ^ 3 ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ orderOf u = p
```

What it contributes. Supplied the transformed-word quotient checkpoint that derives $b \in H$ and the subsequent cancellation argument placing g in H ; the child generalizes exponents 2,3 to $k,k+1$ and changes the conclusion to a cyclic subgroup.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponents from fixed values to an arbitrary k and additionally constructs the cyclic subgroup generated by g , proving its containment and cardinality. The parent's argument only derives $g \in H$ and does not supply these subgroup-cardinality steps.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The exponent is genuinely generalized from fixed powers to arbitrary k , and the conclusion changes from producing an element of order p to constructing the entire cyclic subgroup with cardinality p . The proof therefore requires new subgroup construction and cardinality reasoning beyond the parent's membership argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The variable exponent k genuinely generalizes the parent's cancellation argument, and the conclusion additionally requires constructing the cyclic subgroup generated by g and proving its cardinality is p . This is not recoverable by recalling the parent's existential witness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2306 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B restricts the ambient setting to the abelian unit group $(\mathbb{Z}\text{Mod } n)^x$, whereas Statement A is stated for unspecified units and subgroups.; Statement A requires the contained subgroup of cardinality p to be cyclic, while Statement B only requires a subgroup K of cardinality p and does not state cyclicity.

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_of_prime_order_in_subgroup.extracted_1_1 (n p k : ℕ) (g b : (ZMod n)^x) (hg : orderOf g = p)
  (H : Subgroup (ZMod n)^x) (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) : ∃ K ≤ H, Nat.card K = p
```

Read back by a model that saw only the Lean. For natural numbers n , p , and k , and units g and b of $\mathbb{Z}\text{Mod } n$, suppose that the order of g is p . If H is a subgroup of the unit group $(\mathbb{Z}\text{Mod } n)^x$, and both $b g b^k$ and $b g b^{k+1}$ belong to H , then there exists a subgroup K of H whose cardinality is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)^x)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)^x)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)^x, K ≤ H ∧ Nat.card K = p := by
  have hbprod :
    (b * g * b ^ k)^-1 * (b * g * b ^ (k + 1)) ∈ H :=
    H.mul_mem (H.inv_mem h1) h2
  have hb : b ∈ H := by
    convert hbprod using 1 <|> group
  have hbinv : b^-1 ∈ H := H.inv_mem hb
  have hbk : (b^-1) ^ k ∈ H := H.pow_mem hbinv k
  have hleft : b^-1 * (b * g * b ^ k) ∈ H := H.mul_mem hbinv h1
  have hall : (b^-1 * (b * g * b ^ k)) * (b^-1) ^ k ∈ H :=
    H.mul_mem hleft hbk
  have hgH : g ∈ H := by
    convert hall using 1 <|> group
  let K : Subgroup (ZMod n)^x := Subgroup.zpowers g
  have hK : K ≤ H := by
    dsimp [K]
    exact Subgroup.zpowers_le.mpr hgH
  have hcard : Nat.card K = p := by
    dsimp [K]
    rw [Nat.card_zpowers, hg]
  exact ⟨K, hK, hcard⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__x__Herstein_exercise_2_4_36__mh.xe.mh__2a21cbb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2a21cbb7e5acc33a

4.277 277. additive_inverse_characterization

The problem

In a vector space, show that $v + w = 0$ if and only if $w = -v$.

Lean statement

```
theorem additive_inverse_characterization {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] : ∀ v w : V, v + w = 0 ↔ w = -v
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from double negation to an iff characterizing additive inverses. Its proof requires cancellation and additive rearrangement in both directions, which the parent's 'simp' proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a double-negation identity into a two-way characterization of additive inverses. Its proof requires cancellation, rearrangement via 'abel', and both implication directions, so recalling the parent does not solve the new obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a direct involution identity into a two-way characterization of additive inverses. Proving the forward implication requires cancellation and additive rearrangement via 'abel', so the parent's 'simp' proof does not supply the main argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2776 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem additive_inverse_characterization.extracted_1_1 : ∀ {V : Type u_2} [inst : AddCommGroup V] (v w : V),
  v + w = 0 ↔ w = -v
```

Read back by a model that saw only the Lean. For every type V equipped with an additive commutative group structure

and all elements v, w of V , $v + w = 0$ if and only if $w = -v$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem additive_inverse_characterization {F V : Type*} [AddCommGroup V] [Field F]
[Module F V] : ∀ v w : V, v + w = 0 ↔ w = -v := by
  intro v w
  have hdouble : -(-v) = v := by
    simp
  have hcancel : -v + v = 0 := by
    calc
      -v + v = -v + -(-v) := by rw [hdouble]
      _ = 0 := by simp
  constructor
  · intro h
    calc
      w = w + 0 := by simp
      _ = w + (-v + v) := by rw [hcancel]
      _ = (v + w) + (-v) := by abel
      _ = 0 + (-v) := by rw [h]
      _ = -v := by simp
  · intro h
    rw [h]
    simpa [add_comm] using hcancel
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__mh__c3a4c52a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropept, Quot.sound, fingerprint c3a4c52a33e2ccfd

4.278 278. double_neg_smul

The problem

Let F be a field and V an F -vector space. Prove that $(-a)(-v) = av$ for every scalar $a \in F$ and vector $v \in V$.

Lean statement

```
theorem double_neg_smul {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (a : F) (v : V) : (-a) • (-v) = a • v := by
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the visible double-negation checkpoint, generalized from one vector to every vector in the module.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely extends double-negation to simultaneous negation of a scalar and vector. Proving it requires module-specific identities ('smul_neg' and 'neg_smul') in addition to the parent's 'neg_neg', so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes double negation to the interaction of negation with scalar multiplication. Its proof requires the new module identities 'smul_neg' and 'neg_smul', so recalling the parent's 'neg_neg' proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes double negation to scalar multiplication and requires two new module identities, 'smul_neg' and 'neg_smul', before applying vector double negation. The parent's 'simp' proof does not supply this bilinearity reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem double_neg_smul.extracted_1_1.{u_2, u_1} {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] (a : F) (v : V) : -a • -v = a • v
```

Read back by a model that saw only the Lean. *Let F be a field and V an additive commutative group equipped with an F -module structure. For any scalar $a \in F$ and vector $v \in V$, the scalar multiplication of $-v$ by $-a$ equals the scalar multiplication of v by a : $(-a) \cdot (-v) = a \cdot v$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem double_neg_smul {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (a : F) (v : V) : (-a) • (-v) = a • v := by
  have hneg : ∀ x : V, -(-x) = x := by
    intro x
    exact neg_neg x
  have hsmul_neg : (-a) • (-v) = -((-a) • v) := by
    rw [smul_neg]
  have hneg_smul : (-a) • v = -(a • v) := by
    rw [neg_smul]
  calc
    (-a) • (-v) = -((-a) • v) := hsmul_neg
    _ = -(-(a • v)) := by rw [hneg_smul]
    _ = a • v := hneg (a • v)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__mh__c9459ce0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropt, Quot.sound, fingerprint c9459ce043e9c50c

4.279 279. fixed_point_intersection_is_greatest

The problem

Let $(u_i)_{i \in I}$ be a finite nonempty family of subspaces of V , and let $f : V \rightarrow V$ be linear and preserve every u_i . Prove that the subspace of vectors fixed by f inside their intersection is the greatest subspace contained in every u_i on which f acts as the identity.

Lean statement

```
theorem fixed_point_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ (∀ i : ι, x ∈ u i) ∧ f x = x) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x = x) →
      W ≤ U
```

Parent. Let $(u_i)_{i \in I}$ be a finite nonempty family of subspaces of V , and let $f : V \rightarrow V$ be linear and preserve every u_i . Prove that their intersection is the greatest f -invariant subspace contained in every u_i .

Parent in Lean

```
theorem finite_nonempty_invariant_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ ∀ i : ι, x ∈ u i) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x ∈ W) → W ≤ U
```

What it contributes. Supplied the finite intersection construction, its membership characterization, and the invariance checkpoint used internally to prove the fixed-point submodule is invariant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target from the invariant intersection to its fixed-point submodule, requiring construction with the kernel of $f - \text{id}$ and a new greatest-subspace argument under pointwise fixedness. The parent's intersection-and-invariance proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the characterized submodule by intersecting the invariant intersection with the fixed-point kernel of $f - \text{id}$. Proving the kernel equivalence and showing this smaller submodule is invariant requires reasoning absent from the parent, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the fixed-point submodule by intersecting the invariant intersection with the kernel of $f - \text{id}$. Proving its invariance and maximality requires fixed-point reasoning absent from the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2245 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fixed_point_intersection_is_greatest.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2} {ι : Type u_3}
[inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] [inst_3 : Nonempty ι] (u : ι → Submodule F V)
(f : V →ι[F] V) (hf : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
∃ U,
  (∀ (x : V), x ∈ U ↔ (∀ (i : ι), x ∈ u i) ∧ f x = x) ∧
  (∀ x ∈ U, f x ∈ U) ∧ ∀ (W : Submodule F V), (∀ (i : ι), W ≤ u i) → (∀ x ∈ W, f x = x) → W ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, let V be an additive commutative group and an F -module, and let ι be a nonempty type. Given a family $u : \iota \rightarrow \text{Submodule } F V$, a linear map $f : V \rightarrow_{\iota}[F] V$, and the hypothesis that for every $i : \iota$ and every $x \in u i$, one has $f x \in u i$, there exists a submodule U of V such that, for every $x \in V$, $x \in U$ if and only if x belongs to $u i$ for every $i : \iota$ and $f x = x$; moreover, for every $x \in U$, $f x \in U$; and for every submodule W of V , if $W \leq u i$ for every $i : \iota$ and $f x = x$ for every $x \in W$, then $W \leq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem fixed_point_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →ι[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ (∀ i : ι, x ∈ u i) ∧ f x = x) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x = x) →
      W ≤ U := by
  let I : Submodule F V := ⋂ i, u i
  let U : Submodule F V := I ∩ LinearMap.ker (f - (LinearMap.id : V →ι[F] V))
  have hI_mem : ∀ x : V, x ∈ I ↔ ∀ i : ι, x ∈ u i := by
    intro x
    change x ∈ (⋂ i, u i) ↔ ∀ i : ι, x ∈ u i
    exact Submodule.mem_iInf u
  have hI_inv : ∀ x : V, x ∈ I → f x ∈ I := by
    intro x hx
    apply Submodule.mem_iInf u |>.mpr
```

```

intro i
apply hf i x
exact (hI_mem x).mp hx i
have hker :  $\forall x : V, x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V)) \leftrightarrow f x = x$  := by
  intro x
  constructor
  · intro hx
    change (f - (LinearMap.id :  $V \rightarrow_{\mathbb{I}}[F] V$ )) x = 0 at hx
    change f x - x = 0 at hx
    exact sub_eq_zero.mp hx
  · intro hx
    change (f - (LinearMap.id :  $V \rightarrow_{\mathbb{I}}[F] V$ )) x = 0
    change f x - x = 0
    exact sub_eq_zero.mpr hx
have hU_mem :  $\forall x : V, x \in U \leftrightarrow (\forall i : \mathbb{I}, x \in u i) \wedge f x = x$  := by
  intro x
  constructor
  · intro hx
    change  $x \in I \wedge x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V))$  at hx
    exact ⟨(hI_mem x).mp hx.1, (hker x).mp hx.2⟩
  · intro hx
    change  $x \in I \wedge x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V))$ 
    exact ⟨(hI_mem x).mpr hx.1, (hker x).mpr hx.2⟩
have hU_inv :  $\forall x : V, x \in U \rightarrow f x \in U$  := by
  intro x hx
  have hx' := (hU_mem x).mp hx
  apply (hU_mem (f x)).mpr
  constructor
  · exact (hI_mem (f x)).mp (hI_inv x ((hI_mem x).mpr hx'.1))
  · simpa [hx'.2]
have hgreatest :  $\forall W : \text{Submodule } F V,$ 
  ( $\forall i : \mathbb{I}, W \leq u i$ )  $\rightarrow$ 
  ( $\forall x : V, x \in W \rightarrow f x = x$ )  $\rightarrow$ 
   $W \leq U$  := by
  intro W hW hfixed x hx
  apply (hU_mem x).mpr
  constructor
  · intro i
    exact hW i hx
  · exact hfixed x hx
exact ⟨U, hU_mem, hU_inv, hgreatest⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.mh__3405d747

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3405d747515afd56

4.280 280. intersection_is_unique_greatest_lower_bound

The problem

Prove that the intersection of any family of subspaces is the unique subspace that is contained in every member and contains every subspace with that property.

Lean statement

```
theorem intersection_is_unique_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule
  F V) :
  ∃! U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
      ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Parent. Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Parent in Lean

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
      ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence to a genuine uniqueness statement. Although the parent supplies the candidate intersection submodule, the child requires a new extensionality argument proving any other submodule with the same carrier is equal to it; this is not contained in the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a greatest lower bound into existence and uniqueness. It must additionally prove that any other candidate has the same carrier and hence is equal as a submodule, which is a genuine obligation absent from the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a greatest lower bound into existence and uniqueness, requiring a new extensionality argument to show any qualifying submodule has the same carrier. The parent's construction proves existence and the GLB property but does not supply this uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2940 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_unique_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃! U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. *Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, and let u be a family of F -submodules of V indexed by ι . Then there exists a unique F -submodule U of V such that the intersection of the underlying sets of the submodules $u(i)$, over all i , is the underlying set of U , and, for every F -submodule W of V , W is contained in U if and only if W is contained in $u(i)$ for every i .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_unique_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule
  F V) :
  ∃! U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
  classical
  let U : Submodule F V := ⋂ i, u i
  have hCarrier : (⋂ i : ι, (u i).carrier) = (U : Set V) := by
    dsimp [U]
    exact (Submodule.coe_iInf u).symm
  have hGLB : ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
    dsimp [U]
    intro W
    exact le_iInf_iff
  refine ⟨U, ⟨hCarrier, hGLB⟩, ?_⟩
  intro U' hU'
  have hset : (U : Set V) = (U' : Set V) := by
    rw [← hCarrier, hU'.1]
  apply Submodule.ext
  intro x
  exact (Set.ext_iff.mp hset x).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.mh__83e0bdc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 83e0bdc6510a2780

4.281 281. submodule_intersection_is_greatest

The problem

Prove that for any family of submodules of V , there exists a submodule U whose underlying set is the intersection of the family, and such that a submodule W is contained in every member of the family if and only if W is contained in U .

Lean statement

```
theorem submodule_intersection_is_greatest {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, (∀ i, W ≤ u i) ↔ W ≤ U
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the existential intersection statement into a greatest-submodule characterization, requiring proof of the full universal property via both order directions. The parent's extensional membership proof does not supply this lattice-theoretic uniqueness/greatest argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from mere representability of the set intersection to a greatest-submodule characterization. Its proof requires the additional lattice universal-property arguments 'le_iInf' and 'iInf_le', which are not supplied by the parent's extensionality proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes mere existence of an intersection submodule into a greatest-submodule characterization, requiring the infimum universal property via le_iInf and iInf_le. The parent's carrier equality proof does not supply this order-theoretic equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1931 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem submodule_intersection_is_greatest.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃ U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), (∀ (i : ι), W ≤ u i) ↔ W ≤ U
```

Read back by a model that saw only the Lean. For any family u assigning to each element i of an index type ι a submodule $u(i)$ of the F -module V , there exists a submodule U of V such that the intersection of the underlying sets of all $u(i)$ equals the underlying set of U , and for every submodule W of V , W is contained in every $u(i)$ if and only if W is contained in U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem submodule_intersection_is_greatest {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, (∀ i, W ≤ u i) ↔ W ≤ U := by
  classical
  let U : Submodule F V := ⋂ i, u i
  refine ⟨U, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro W
    constructor
    · intro hW
      exact le_iInf hW
    · intro hW i
      exact le_trans hW (iInf_le u i)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__mh__7a088215

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 597069d000ab8471

4.282 282. genEigenspace_invariant_of_commute

The problem

Every generalized eigenspace of S associated with a scalar $c \in F$ and an order $k \in \mathbb{N} \cup \{\infty\}$ is invariant under T whenever S and T commute.

Lean statement

```
theorem genEigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, ∀ k : ℕ∞, Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k)
```

Parent. Every eigenspace of S is invariant under T when S and T commute.

Parent in Lean

```
theorem eigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, Set.MapsTo T (Module.End.eigenspace S c) (Module.End.eigenspace S c)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes invariant eigenspaces to generalized eigenspaces for every ordinal-valued exponent k , which is a stronger mathematical claim than the parent. Its proof uses a dedicated generalized-eigenspace commutation theorem rather than reusing the parent's ordinary eigenspace argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes invariance from the ordinary eigenspace to every generalized eigenspace and arbitrary ordinal bound k . The parent's eigenvector-level calculation does not establish this broader claim; the child requires a distinct generalized-eigenspace commutation theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens invariance from ordinary eigenspaces to generalized eigenspaces for every exponent k , requiring preservation of kernels of powers of $S - c$. The parent's pointwise eigenvector calculation does not supply this power-commutation argument, and the child uses a distinct generalized-eigenspace lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem genEigenspace_invariant_of_commute.extracted_1_1 : ∀ {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] {S T : End F V},
  S * T = T * S → ∀ (c : F) (k : ℕ), Set.MapsTo ↗T ↑((S.genEigenspace c) k) ↑((S.genEigenspace c) k)
```

Read back by a model that saw only the Lean. *For any field F and any additive commutative group V equipped with an F -module structure, and for F -linear endomorphisms S and T of V , if S and T commute ($S \circ T = T \circ S$), then for every scalar $c \in F$ and every $k \in \mathbb{N}_\infty$, the endomorphism T maps the generalized eigenspace of S corresponding to c at level k into itself.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem genEigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, ∀ k : ℕ, Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k) := by
  intro c k
  have h_commute : Commute S T := h_comm
  have h_preserved : Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k) :=
    Module.End.mapsTo_genEigenspace_of_comm h_commute c k
  exact h_preserved
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_5_20__mh__80232ce9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint ec92e5e9d1b2f4a8

4.283 283. normalClosure_eq_intersection_normal_containers

The problem

Let T be a subset of a group. Prove that the normal closure of T is the intersection of all normal subgroups containing T .

Lean statement

```
theorem normalClosure_eq_intersection_normal_containers {G : Type*} [Group G] (T : Set G) :
  Subgroup.normalClosure T =
    ⋂ (N : Subgroup G) (N.Normal) (T ⊆ N), N
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKNormal : K.Normal) :
  T ⊆ K ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T) →
      Subgroup.normalClosure S ≤ K
```

What it contributes. Used the parent's normal-closure universal property to establish containment in every normal subgroup containing T .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child makes a substantive change of shape: it characterizes the normal closure as an intersection over all normal subgroups containing the set. Its proof requires lattice-theoretic intersection bounds and selecting the normal closure among the containers, which the parent's implication proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different characterization: normal closure equals the intersection of all normal subgroups containing the set. Its proof requires lattice infimum bounds and minimality, which the parent's quantified-containment equivalence does not supply directly.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an implication characterization for one normal subgroup to an equality identifying the normal closure with the intersection of all normal subgroups containing the set. Proving both lattice inequalities requires explicit infimum reasoning and is not obtained by recalling the parent's quantified subset argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2978 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_eq_intersection_normal_containers.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (T : Set G) : normalClosure T =  $\bigcap N$ ,  $\bigcap (\_ : N.Normal)$ ,  $\bigcap (\_ : T \subseteq \uparrow N)$ , N
```

Read back by a model that saw only the Lean. *For any group G and any subset T of G , the normal closure of T is equal to the intersection of all normal subgroups N of G that contain T .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_eq_intersection_normal_containers {G : Type*} [Group G] (T : Set G) :
  Subgroup.normalClosure T =
     $\bigcap (N : \text{Subgroup } G) (\_ : N.Normal) (\_ : T \subseteq (N : \text{Set } G))$ , N := by
  let I : Subgroup G :=  $\bigcap (N : \text{Subgroup } G) (\_ : N.Normal) (\_ : T \subseteq (N : \text{Set } G))$ , N
  have h_intersection : I =  $\bigcap (N : \text{Subgroup } G) (\_ : N.Normal) (\_ : T \subseteq (N : \text{Set } G))$ , N := by
    rfl
  have h_left : Subgroup.normalClosure T  $\leq$  I := by
    rw [h_intersection]
    refine le_iInf (fun N => ?_)
    refine le_iInf (fun hN => ?_)
    refine le_iInf (fun hT => ?_)
    letI : N.Normal := hN
    exact Subgroup.normalClosure_le_normal hT
  have h_right : I  $\leq$  Subgroup.normalClosure T := by
    rw [h_intersection]
    refine iInf_le_of_le (Subgroup.normalClosure T) ?_
    refine iInf_le_of_le (by infer_instance) ?_
    exact iInf_le_of_le Subgroup.subset_normalClosure le_rfl
  exact le_antisymm h_left h_right
```

Operator. mutation / mutation_hard **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.mh__101f67b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 101f67b57e008ece

4.284 284. normalClosure_le_iff_subset

The problem

Let S be a subset of a group and let M be a normal subgroup. Prove that the normal closure of S is contained in M if and only if S is contained in M .

Lean statement

```
theorem normalClosure_le_iff_subset {G : Type*} [Group G] (S : Set G)
  (M : Subgroup G) (hMnormal : M.Normal) :
  Subgroup.normalClosure S ≤ M ↔ S ⊆ (M : Set G)
```

Parent. Let S and T be subsets of a group, and let M be a normal subgroup containing the normal closure of T . If S is contained in the normal closure of T , prove that the normal closure of S is contained in M .

Parent in Lean

```
theorem normalClosure_le_of_subset_normalClosure {G : Type*} [Group G] (S T : Set G)
  (M : Subgroup G) (hMnormal : M.Normal)
  (hTM : (Subgroup.normalClosure T : Set G) ⊆ M)
  (hST : S ⊆ (Subgroup.normalClosure T : Set G)) :
  Subgroup.normalClosure S ≤ M
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way containment result into an iff characterization. Its forward direction requires the new argument that generators lie in their normal closure, which the parent proof does not provide; the extra hypotheses involving T are also eliminated.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-way transport result into a full iff characterization of normal closure containment. Its forward direction requires the new inclusion of S into its normal closure, so recalling the parent's proof does not supply the argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way containment result into an iff characterization. The reverse direction is new: it requires using that the generating set lies inside its normal closure, which the parent proof does not establish.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2969 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_le_iff_subset.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S : Set G) (M : Subgroup G)
(hMnormal : M.Normal) : normalClosure S ≤ M ↔ S ⊆ ↑M
```

Read back by a model that saw only the Lean. For any group G , any subset S of G , and any normal subgroup M of G , the normal closure of S is contained in M if and only if S is contained in M .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_le_iff_subset {G : Type*} [Group G] (S : Set G)
(M : Subgroup G) (hMnormal : M.Normal) :
Subgroup.normalClosure S ≤ M ↔ S ⊆ (M : Set G) := by
letI : M.Normal := hMnormal
have hsubset : S ⊆ (Subgroup.normalClosure S : Set G) :=
Subgroup.subset_normalClosure
have hforward : Subgroup.normalClosure S ≤ M → S ⊆ (M : Set G) := by
intro hclosure x hx
exact hclosure (hsubset hx)
have hbackward : S ⊆ (M : Set G) → Subgroup.normalClosure S ≤ M := by
intro hSM
exact (Subgroup.normalClosure_subset_iff).mp hSM
constructor
· exact hforward
· exact hbackward
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.mh__46206500

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46206500cd88548d

4.285 285. normal_subgroup_le_finset_inf_iff_le_normalCore_inf

The problem

Let G be a group and let Σ be a finite family of subgroups. For every normal subgroup N , prove that N is contained in every member of Σ if and only if N is contained in the normal core of their meet.

Lean statement

```
theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  ∀ N : Subgroup G, N.Normal →
    ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf (fun H => H)).normalCore)
```

Parent. Let G be a group and let S be a finite family of subgroups, possibly empty. Prove that the normal core of the meet of the subgroups equals the meet of their normal cores.

Parent in Lean

```
theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore)
```

What it contributes. Used the parent's finite-meet and normal-core maximality checkpoints, changing the equality target into a quantified iff.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the equality into a characterization theorem quantified over an arbitrary normal subgroup. Its proof uses normality to pass between containment in the finite meet and containment in its normal core, rather than reproducing the parent's finite-set induction or equality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the equality into an iff characterizing all normal subgroups contained in every member of the finite family. Its proof uses normality of an arbitrary subgroup and the universal property of the finite infimum, without recalling or applying the parent's finset induction or equality.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an equality of finite subgroup infima into an iff characterizing all normal subgroups contained in the family. Its proof requires the normal-subgroup-to-normal-core lemma and a two-direction containment argument, none of which is supplied by recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2330 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G]
  (S : Finset (Subgroup G)) (N : Subgroup G), N.Normal → ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf fun H => H).normalCore)
```

Read back by a model that saw only the Lean. *For every group G , every finite set S of subgroups of G , and every subgroup N of G , if N is normal in G , then N is contained in every subgroup H belonging to S if and only if N is contained in the normal core of the infimum of the subgroups in S .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  ∀ N : Subgroup G, N.Normal →
    ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf (fun H => H)).normalCore) := by
  classical
  intro N hN
  let I : N.Normal := hN
  constructor
  · intro hAll
    have hMeet : N ≤ S.inf (fun H => H) := by
      refine Finset.le_inf ?_
      intro H hH
      exact hAll H hH
    have hCore : N ≤ (S.inf (fun H => H)).normalCore := by
      exact Subgroup.normal_le_normalCore.mpr hMeet
    exact hCore
  · intro hCore H hH
    have hMeet : N ≤ S.inf (fun H => H) := by
      exact hCore.trans (Subgroup.normalCore_le _)
    exact hMeet.trans (Finset.inf_le hH)
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.mh.mh__74053082

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 740530822b717c0a

4.286 286. normalCore_finset_inf_eq_inf_normalCore

The problem

Let G be a group and let $\mathcal{S}$ be a finite family of subgroups, possibly empty. Prove that the normal core of the meet of the subgroups equals the meet of their normal cores.

Lean statement

```
theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore)
```

Parent. Let G be a group and let $(H_i)_{i \in I}$ be a nonempty indexed family of subgroups. Prove that the normal core of their intersection is the intersection of their normal cores.

Parent in Lean

```
theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

What it contributes. Supplied the monotonicity, normality of intersections, and normal-core maximality checkpoints; these were adapted to a finite meet with an explicit empty-family case.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the result to finite subgroup meets, including the empty case. Its induction and empty-meet handling require reasoning absent from the parent's nonempty indexed-intersection proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to finite subgroup meets, including the empty meet. Its induction and separate empty-case reasoning are not supplied by the parent's nonempty indexed-inf proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the result to finite subgroup meets that may be empty. Its proof requires an induction over the finset, a separate empty case, and new normality reasoning for the inserted finite infimum; the parent's indexed-infimum argument does not supply this.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2325 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_finset_inf_eq_inf_normalCore.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
  (S : Finset (Subgroup G)) : (S.inf fun H => H).normalCore = S.inf fun H => H.normalCore
```

Read back by a model that saw only the Lean. *For any group G and any finite set S of subgroups of G , the normal core of the infimum of the subgroups in S is equal to the infimum of their normal cores.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore) := by
  classical
  induction S using Finset.induction_on with
  | empty =>
    simp
  | @insert H S hS ih =>
    rw [Finset.inf_insert, Finset.inf_insert]
    apply le_antisymm
    · refine le_inf ?_ ?_
      · exact Subgroup.normalCore_mono inf_le_left
      · exact le_trans (Subgroup.normalCore_mono inf_le_right) ih.le
    · have hS_normal : (S.inf (fun K => K.normalCore)).Normal := by
        rw [Finset.inf_eq_iInf]
        exact Subgroup.normal_iInf_normal (fun K =>
          Subgroup.normal_iInf_normal (fun hK : K ∈ S => K.normalCore_normal))
      let I : (H.normalCore n S.inf (fun K => K.normalCore)).Normal :=
        Subgroup.normal_inf_normal _ _
      apply Subgroup.normal_le_normalCore.mpr
      refine le_inf ?_ ?_
      · exact inf_le_left.trans (Subgroup.normalCore_le H)
      · refine le_trans inf_le_right ?_
        refine Finset.le_inf ?_
        intro K hK
        exact le_trans (Finset.inf_le hK) (Subgroup.normalCore_le K)
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.mh__6cd94798

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6cd947984fdfebd

4.287 287. normal_iInf_is_greatest

The problem

Show that the intersection of any indexed family of normal subgroups is normal and is the greatest subgroup contained in every member of the family, even when the family is empty.

Lean statement

```
theorem normal_iInf_is_greatest {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : (⋂ i, H i).Normal ∧
  ↪ (∀ i : I, (⋂ j, H j) ≤ H i) ∧ ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ (⋂ i, H i)
```

Parent. Prove that the intersection of any indexed family of normal subgroups is closed under conjugation, even when the index set is empty.

Parent in Lean

```
theorem indexed_inf_conjugation_closed {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : ∀ n, n ∈ (⋂
  ↪ i, H i) → ∀ g : G, g * n * g-1 ∈ (⋂ i, H i)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the conclusion from conjugation-closure to a full characterization of the infimum: normality, containment in each family member, and its greatest-subgroup property. The latter two obligations require lattice reasoning ('iInf_le' and 'le_iInf') absent from the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a substantive greatest-lower-bound characterization, requiring the new lattice arguments 'iInf_le' and 'le_iInf' in addition to normality. The parent's conjugation-closure proof does not supply these universal containment properties.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine greatest-lower-bound characterization of the indexed infimum: it must prove both containment into every $H i$ and that every subgroup below all $H i$ lies below the infimum. The parent's conjugation argument supplies only normality and cannot establish these lattice obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2963 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_iInf_is_greatest.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) :
  (⋂ i, H i).Normal ∧ (∀ (i : I), ⋂ j, H j ≤ H i) ∧ ∀ (K : Subgroup G), (∀ (i : I), K ≤ H i) → K ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a type, and let H assign to each element i of I a normal subgroup $H(i)$ of G . Then the infimum $\bigcap_i H(i)$ is normal; for every $i \in I$, $\bigcap_j H(j)$ is contained in $H(i)$; and for every subgroup K of G , if K is contained in $H(i)$ for every $i \in I$, then K is contained in $\bigcap_i H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_iInf_is_greatest {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : (⋂ i, H i).Normal ∧
  (∀ i : I, (⋂ j, H j) ≤ H i) ∧ ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ ⋂ i, H i := by
  have hNormal : (⋂ i, H i).Normal := Subgroup.normal_iInf_normal hH
  have hContained : ∀ i : I, (⋂ j, H j) ≤ H i := by
    intro i
    exact iInf_le H i
  have hGreatest : ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ ⋂ i, H i := by
    intro K hK
    exact le_iInf hK
  exact ⟨hNormal, hContained, hGreatest⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3.1_22b__me.me.ms.mh__c7c9901b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c7c9901bbb7e6cba

4.288 288. normalCore_iInf_finset

The problem

Let H_i be a finite nonempty family of subgroups of a group G . Prove that the normal core of their intersection equals the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_finset {G 1 : Type*} [Group G] (s : Finset 1) (H : 1 → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore
```

Parent. Show that the normal core of a subgroup H is the supremum of all normal subgroups of G contained in H .

Parent in Lean

```
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore = ⋃ (N : Subgroup G) (N : Normal N) (N : N ≤ H), N
```

What it contributes. Supplied the normal-core facts that each core is normal and contained in its subgroup, together with the universal property `normal_le_normalCore` used for both inclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different lattice identity: normal core commutes with an indexed infimum, requiring normality and universal-property reasoning in the opposite direction from the parent's supremum characterization. It is not recoverable by recalling or directly applying the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine family-level lattice identity: normality of an infimum of normal cores and both universal-property inequalities are required. The parent's single-subgroup supremum characterization does not supply this finite-infimum argument by recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different lattice identity: normal core distributes over a finite indexed intersection. Its proof requires establishing normality of the intersection and both lattice inequalities, whereas the parent only characterizes one normal core as a supremum and does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2156 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_finset.extracted_1_1.{u_2, u_1} {G : Type u_1} {ι : Type u_2} [inst : Group G] (s : Finset ι)
  (hs : s.Nonempty) (H : ι → Subgroup G) : ⋂ i ∈ s, (H i).normalCore = (⋂ i ∈ s, H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be a type, let s be a nonempty finite subset of ι , and let H assign to each $i \in \iota$ a subgroup $H(i)$ of G . Then the infimum, over all i in s , of the normal cores of $H(i)$ equals the normal core of the infimum, over all i in s , of $H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_finset {G ι : Type*} [Group G] (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore := by
  have hleft_normal : Normal (⋂ i ∈ s, (H i).normalCore) := by
    constructor
    intro n hn g
    rw [Subgroup.mem_iInf]
    intro i
    rw [Subgroup.mem_iInf]
    intro hi
    apply (H i).normalCore_normal.conj_mem n
    rw [Subgroup.mem_iInf] at hn
    exact (Subgroup.mem_iInf.mp (hn i)) hi
  have hright_normal : Normal (⋂ i ∈ s, H i).normalCore := by
    exact Subgroup.normalCore_normal _
  have hleft_le_right_arg : (⋂ i ∈ s, (H i).normalCore) ≤ (⋂ i ∈ s, H i) := by
    refine le_iInf (fun i => ?_)
    refine le_iInf (fun hi => ?_)
    exact le_trans (iInf_le_of_le i (iInf_le_of_le hi le_refl)) (H i).normalCore_le
  have hright_le_left : (⋂ i ∈ s, H i).normalCore ≤ (⋂ i ∈ s, (H i).normalCore) := by
    refine le_iInf (fun i => ?_)
    refine le_iInf (fun hi => ?_)
    have harg_le_hi : (⋂ i ∈ s, H i) ≤ H i := by
      exact iInf_le_of_le i (iInf_le_of_le hi le_refl)
    letI : Normal (⋂ i ∈ s, H i).normalCore := hright_normal
    exact (Subgroup.normal_le_normalCore).mpr ((Subgroup.normalCore_le _).trans harg_le_hi)
  have hleft_le_right : (⋂ i ∈ s, (H i).normalCore) ≤ (⋂ i ∈ s, H i).normalCore := by
    letI : Normal (⋂ i ∈ s, (H i).normalCore) := hleft_normal
    exact (Subgroup.normal_le_normalCore).mpr hleft_le_right_arg
  exact le_antisymm hleft_le_right hright_le_left
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh.mh__e633a98d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 5fa322b68d8c112a

4.289 289. existsUnique_normal_lower_bound

The problem

Let S be a collection of normal subgroups of a group G . Prove that their intersection is the unique greatest normal subgroup contained in every member of S .

Lean statement

```
theorem existsUnique_normal_lower_bound {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, N.Normal) : ∃! H : Subgroup G,
  ↪ H.Normal ∧ (∀ N ∈ S, H ≤ N) ∧ (∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ H)
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  ↪ G).Normal
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from proving normality to proving a uniqueness and universal characterization of the greatest normal lower bound. Its proof requires new order-theoretic arguments using both ‘sInf_le’ and ‘le_sInf’, which the parent proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes normality of the infimum into an exists-unique characterization of the greatest normal lower bound. Beyond the parent’s normality argument, it requires proving the infimum’s lower-bound property and uniqueness via the lattice universal property.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion into an existence-and-uniqueness characterization of the greatest normal lower bound. Beyond the parent’s normality proof, it requires proving the lower-bound property and antisymmetric uniqueness via lattice infimum lemmas.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2826 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem existsUnique_normal_lower_bound.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] (S : Set (Subgroup G))
  (hS : ∀ N ∈ S, N.Normal) :
```

$$\exists! H, H.\text{Normal} \wedge (\forall N \in S, H \leq N) \wedge \forall (K : \text{Subgroup } G), K.\text{Normal} \rightarrow (\forall N \in S, K \leq N) \rightarrow K \leq H$$

Read back by a model that saw only the Lean. For a group G and a set S of subgroups of G such that every subgroup N in S is normal in G , there exists a unique subgroup H of G such that H is normal in G , H is contained in every subgroup N in S , and for every normal subgroup K of G that is contained in every subgroup N in S , K is contained in H .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem existsUnique_normal_lower_bound {G : Type*} [Group G] (S : Set (Subgroup G))
  (hS : ∀ N ∈ S, N.Normal) :
  ∃! H : Subgroup G,
    H.Normal ∧ (∀ N ∈ S, H ≤ N) ∧
    (∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ H) := by
  have hNormal : (sInf S : Subgroup G).Normal := by
    rw [sInf_eq_iInf]
    apply Subgroup.normal_iInf_normal
    intro N
    apply Subgroup.normal_iInf_normal
    intro hN
    exact hS N hN
  have hLower : ∀ N ∈ S, (sInf S : Subgroup G) ≤ N := by
    intro N hN
    exact sInf_le hN
  have hGreatest : ∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ sInf S := by
    intro K hK hKL
    exact le_sInf hKL
  refine ⟨sInf S, ?_, ?_⟩
  · exact ⟨hNormal, hLower, hGreatest⟩
  · intro K hK
    apply le_antisymm
    · exact hGreatest K hK.1 hK.2.1
    · exact hK.2.2 (sInf S) hNormal hLower
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__7fdfabde

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7fdfabde60505d2a

4.290 290. normalClosure_normal_and_minimal

The problem

For a subset s of a group, prove that its normal closure is normal and is contained in every normal subgroup containing s .

Lean statement

```
theorem normalClosure_normal_and_minimal {G : Type*} [Group G] (s : Set G) :
  (Subgroup.normalClosure s).Normal ∧
  ∀ N : Subgroup G, N.Normal → s ⊆ N → Subgroup.normalClosure s ≤ N
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  G).Normal
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from normality of an infimum to a conjunction asserting both normality and the universal minimality property of the normal closure. Proving the containment for every normal subgroup containing the generating set requires a new infimum-order argument not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from normality of an infimum to a normality-and-minimality characterization of the normal closure. Its second conjunct requires proving the universal lower-bound property, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving normality of an infimum to establishing the universal property of the normal closure: both normality and minimality among normal subgroups containing the generators. The parent's normality argument does not supply the new minimality proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2815 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_normal_and_minimal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  (normalClosure s).Normal  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$   $s \subseteq N \rightarrow$  normalClosure s  $\leq$  N
```

Read back by a model that saw only the Lean. *Let G be a group and let s be a subset of G . Then the normal closure of s in G is a normal subgroup of G , and for every normal subgroup N of G such that s is contained in N , the normal closure of s is contained in N .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_normal_and_minimal {G : Type*} [Group G] (s : Set G) :  
  (Subgroup.normalClosure s).Normal  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$   $s \subseteq N \rightarrow$  Subgroup.normalClosure s  $\leq$  N := by  
  rw [Subgroup.normalClosure_eq_iInf]  
  constructor  
  · apply Subgroup.normal_iInf_normal  
    intro N  
    apply Subgroup.normal_iInf_normal  
    intro hN  
    apply Subgroup.normal_iInf_normal  
    intro hs  
    exact hN  
  · intro N hN hs  
    exact iInf_le_of_le N (iInf_le_of_le hN (iInf_le_of_le hs le_refl))
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__aada0d88

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aada0d8800a170db

4.291 291. normalCore_iInf_fintype

The problem

For a finite nonempty family of subgroups of a group, the normal core of their intersection equals the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_fintype {G : Type*} [Group G] {I : Type*} [Fintype I] [Nonempty I]
  (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  ⊆ G).Normal
```

What it contributes. Supplies the normality-of-an-infimum checkpoint, used for the finite meet of the family's normal cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new lattice identity: normalCore distributes over an indexed infimum. Its proof requires both inequalities via the normal-core universal property and establishes normality of the infimum, none of which follows by recalling the parent's normality result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive normal-core lattice identity, requiring two lattice inequalities and normality of the infimum of normal cores; the parent only establishes normality of an arbitrary set infimum. The finite and nonempty assumptions are redundant, but the mathematical conclusion is genuinely different and cannot be obtained by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different lattice identity: normal core commutes with a finite indexed infimum. Its proof uses the universal property of normalCore and the fact that an infimum of normal subgroups is normal, which the parent's normality theorem alone does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2949 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_fintype.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {I : Type u_2}
  [inst_1 : Nonempty I] (H : I → Subgroup G) : (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, and let H be a family of subgroups of G indexed by I . Then the normal core of the infimum of the subgroups $H(i)$ is equal to the infimum, over all i in I , of the normal cores of $H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_fintype {G : Type*} [Group G] {I : Type*} [Fintype I] [Nonempty I]
  (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore := by
  have hFiniteMeetNormal : (⋂ i, (H i).normalCore).Normal := by
    apply Subgroup.normal_iInf_normal
    intro i
    exact (H i).normalCore_normal
  apply le_antisymm
  · refine le_iInf fun i => ?_
    apply Subgroup.normal_le_normalCore.mpr
    exact (Subgroup.normalCore_le (⋂ i, H i)).trans (iInf_le H i)
  · apply Subgroup.normal_le_normalCore.mpr
    refine le_iInf fun i => ?_
    exact (iInf_le (fun j => (H j).normalCore) i).trans (Subgroup.normalCore_le (H i))
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__b696cc8e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint b696cc8eef7d0af0

4.292 292. normalCore_eq_iSup_child

The problem

Show that the normal core of a subgroup H is the supremum of all normal subgroups of G contained in H .

Lean statement

```
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore = ⋃ (N : Subgroup G) (N : Normal N) (N : N ≤ H), N
```

Parent. For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Parent in Lean

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

What it contributes. Supplies the normality and containment checkpoints for $H.normalCore$, which establish the lower bound in the supremum equality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the characterization into an equality with an indexed supremum, requiring new lattice reasoning in both directions and an explicit supremum construction. The parent's normality, containment, and maximality facts are checkpoints, but its proof alone does not supply the equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the characterization from a universal maximality conjunction into an explicit supremum equality. Its proof requires new complete-lattice reasoning in both directions, including constructing the core as a supremum member and bounding every indexed subgroup by the core.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion into an $iSup$ characterization of the normal core. Proving the reverse inclusion requires new supremum-order reasoning, while the forward inclusion uses the parent's normality and containment facts rather than merely replaying its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2151 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_eq_iSup_child.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore = ⋃ N, ⋃ (_ : N.Normal), ⋃ (_ : N ≤ H), N
```

Read back by a model that saw only the Lean. For any group G and subgroup H of G , the normal core of H is equal to the supremum of all subgroups N of G such that N is normal in G and N is contained in H .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore = ⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N := by  
  have hcore_normal : Normal H.normalCore := by  
    exact H.normalCore_normal  
  have hcore_contained : H.normalCore ≤ H := by  
    exact H.normalCore_le  
  have hcore_le_sup : H.normalCore ≤ ⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N := by  
    exact le_iSup_of_le H.normalCore  
      (le_iSup_of_le hcore_normal  
        (le_iSup_of_le hcore_contained le_refl))  
  have hsup_le_core : (⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N) ≤ H.normalCore := by  
    refine iSup_le ?_  
    intro N  
    refine iSup_le ?_  
    intro hN  
    refine iSup_le ?_  
    intro hNH  
    letI : Normal N := hN  
    exact Subgroup.normal_le_normalCore.mpr hNH  
  exact le_antisymm hcore_le_sup hsup_le_core
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__cc302730

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cc30273078e0f798

4.293 293. normal_core_mono_and_contains

The problem

Let $H \leq K$ be subgroups of a group G . Prove that the normal core of H is contained in the normal core of K , and that every normal subgroup contained in H is contained in the normal core of K .

Lean statement

```
theorem normal_core_mono_and_contains {G : Type*} [Group G] (H K : Subgroup G) (hHK : H ≤ K) :
  H.normalCore ≤ K.normalCore ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore
```

Parent. For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Parent in Lean

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds the substantive monotonicity theorem $H.\text{normalCore} \leq K.\text{normalCore}$, which requires a new argument absent from the parent proof. The containment conclusion then combines the parent's normal-core characterization with this new monotonicity result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely new monotonicity claim $H.\text{normalCore} \leq K.\text{normalCore}$, which is not supplied by the parent's characterization proof. The containment claim then uses this new relation, so recalling the parent does not close the whole theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely new monotonicity law $H.\text{normalCore} \leq K.\text{normalCore}$ from $H \leq K$, requiring the separate `normalCore_mono` argument. The containment conjunct then transports normal subgroups through this new inclusion, so the parent's characterization proof alone does not close the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2127 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_core_mono_and_contains.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
  (hHK : H ≤ K) : H.normalCore ≤ K.normalCore ∧ ∀ (N : Subgroup G), N.Normal → N ≤ H → N ≤ K.normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, and let H and K be subgroups of G . If H is contained in K , then the normal core of H is contained in the normal core of K , and every normal subgroup N of G that is contained in H is contained in the normal core of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_core_mono_and_contains {G : Type*} [Group G] (H K : Subgroup G) (hHK : H ≤ K) :
  H.normalCore ≤ K.normalCore ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore := by
  have hmono : H.normalCore ≤ K.normalCore := by
    exact Subgroup.normalCore_mono hHK
  have hcontains : ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore := by
    intro N hN hNH
    letI : Normal N := hN
    have hNcore : N ≤ H.normalCore := by
      exact Subgroup.normal_le_normalCore.mpr hNH
    exact hNcore.trans hmono
  exact ⟨hmono, hcontains⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__d953fd9f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint d953fd9fcb379eb3

4.294 294. normalCore_closure_properties

The problem

Show that taking the normal core of a subgroup is normal, monotone under inclusion, and idempotent.

Lean statement

```
theorem normalCore_closure_properties {G : Type*} [Group G] :
  ∀ H K : Subgroup G, H ≤ K →
    H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧
    H.normalCore.normalCore = H.normalCore
```

Parent. For a subgroup H of a group G , the normal core of H is normal. Moreover, every normal subgroup N is contained in the normal core exactly when it is contained in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup_reversed {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H.normalCore ↔ N ≤ H)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves genuinely different closure properties: monotonicity of the normal core and idempotence of applying the normal core, in addition to normality. These claims are not supplied by the parent's greatest-normal-subgroup equivalence or its closing argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a substantive semantic change: it requires monotonicity of normal cores and idempotence of the normal-core operation in addition to normality. The parent's greatest-normal-subgroup equivalence does not directly supply these claims, and the child proof uses distinct library results.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to monotonicity and idempotence of the normal core, neither of which follows from the parent's greatest-subgroup equivalence proof. Its hypothesis is substantive, and the proof requires distinct normal-core lemmas.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2344 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_closure_properties.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (H K : Subgroup G),  
  H ≤ K → H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧ H.normalCore.normalCore = H.normalCore
```

Read back by a model that saw only the Lean. *For every group G and subgroups H and K of G , if H is contained in K , then the normal core of H is a normal subgroup of G , the normal core of H is contained in the normal core of K , and the normal core of the normal core of H equals the normal core of H .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_closure_properties {G : Type*} [Group G] :  
  ∀ H K : Subgroup G, H ≤ K →  
    H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧  
    H.normalCore.normalCore = H.normalCore := by  
  intro H K hHK  
  have hnormal : H.normalCore.Normal := Subgroup.normalCore_normal H  
  have hmono : H.normalCore ≤ K.normalCore := Subgroup.normalCore_mono hHK  
  have hidem : H.normalCore.normalCore = H.normalCore :=  
    Subgroup.normalCore_idempotent H  
  exact ⟨hnormal, hmono, hidem⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms.ms.mh__e58692b3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint e58692b36402bfd3

4.295 295. existsUnique_least_normalClosure

The problem

For a subset S of a group, show that there exists a unique least normal subgroup containing S .

Lean statement

```
theorem existsUnique_least_normalClosure {G : Type*} [Group G] (s : Set G) :
  ∃! N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

Parent. For a subset S of a group, show that its normal closure is the least normal subgroup containing S .

Parent in Lean

```
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

What it contributes. Supplies the normal-closure witness, normality, subset containment, and leastness checkpoints; the child adds mutual comparison and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence of a least normal subgroup to existence and uniqueness. Its proof adds substantive mutual comparison and antisymmetry reasoning to establish uniqueness, which the parent does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential leastness claim into an existence-and-uniqueness characterization. Its uniqueness proof requires a new mutual-comparison argument using both universal properties and antisymmetry, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence of a least normal subgroup to uniqueness. Its proof adds a genuine mutual-inclusion argument using each candidate's universal leastness, which the parent's proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2955 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem existsUnique_least_normalClosure.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  ∃! N, N.Normal ∧ s ⊆ ↑N ∧ ∀ (M : Subgroup G), M.Normal → s ⊆ ↑M → N ≤ M
```

Read back by a model that saw only the Lean. *There exists a unique normal subgroup N of G such that s is contained in N and, for every normal subgroup M of G containing s , N is contained in M .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem existsUnique_least_normalClosure {G : Type*} [Group G] (s : Set G) :  
  ∃! N : Subgroup G, N.Normal ∧ s ⊆ N ∧  
    ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
  let N : Subgroup G := Subgroup.normalClosure s  
  have hN : N.Normal := by  
    dsimp [N]  
    exact Subgroup.normalClosure_normal  
  have hsN : s ⊆ N := by  
    dsimp [N]  
    exact Subgroup.subset_normalClosure  
  have hleast : ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
    intro M hM hsM  
    letI : M.Normal := hM  
    dsimp [N]  
    exact Subgroup.normalClosure_le_normal hsM  
  have huniq : ∀ M : Subgroup G,  
    (M.Normal ∧ s ⊆ M ∧ ∀ K : Subgroup G, K.Normal → s ⊆ K → M ≤ K) → M = N := by  
    intro M hM  
    have hMN : M ≤ N := hM.2.2 N hN hsN  
    have hNM : N ≤ M := hleast M hM.1 hM.2.1  
    exact le_antisymm hMN hNM  
  refine ⟨N, ⟨hN, hsN, hleast⟩, ?_⟩  
  intro M hM  
  exact huniq M hM
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh.mh__8e79d468

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 8e79d4687ae16da7

4.296 296. normalClosure_is_least_normal

The problem

For a subset S of a group, show that its normal closure is the least normal subgroup containing S .

Lean statement

```
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the central arbitrary-intersection normality checkpoint, applied to the three nested index families in the normal-closure construction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from normality of an indexed intersection to an existence-and-leastness characterization of the normal closure. Proving minimality for every normal subgroup containing the set requires mathematics not supplied by the parent's normality lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes normality of an arbitrary intersection into the substantive universal-property characterization of the normal closure. Its proof requires constructing the closure, proving containment, and applying the leastness theorem, so recalling the parent's one-lemma proof does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving normality of an indexed intersection to constructing and characterizing the least normal subgroup containing an arbitrary set. It requires existence, containment, and the minimality argument via 'normalClosure_le_normal', which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2939 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_is_least_normal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  ∃ N, N.Normal ∧ s ⊆ ↑N ∧ ∀ (M : Subgroup G), M.Normal → s ⊆ ↑M → N ≤ M
```

Read back by a model that saw only the Lean. For a group G and a subset s of G , there exists a normal subgroup N of G such that s is contained in N , and for every normal subgroup M of G containing s , N is a subgroup of M .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :  
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧  
    ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
  let N : Subgroup G := Subgroup.normalClosure s  
  have hN : N.Normal := by  
    dsimp [N]  
    rw [Subgroup.normalClosure_eq_iInf]  
    apply Subgroup.normal_iInf_normal  
    intro H  
    apply Subgroup.normal_iInf_normal  
    intro hH  
    apply Subgroup.normal_iInf_normal  
    intro hs  
    exact hH  
  have hsN : s ⊆ N := by  
    dsimp [N]  
    exact Subgroup.subset_normalClosure  
  refine ⟨N, hN, hsN, ?_⟩  
  intro M hM hsM  
  letI : M.Normal := hM  
  dsimp [N]  
  exact Subgroup.normalClosure_le_normal hsM
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh__16e3ee99

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16e3ee99450f2958

4.297 297. Dummit_Foote_exercise_3_1_22b_mutation

The problem

Prove that the intersection of an arbitrary nonempty collection of normal subgroups is the greatest normal subgroup contained in every member of the collection.

Lean statement

```
theorem Dummit_Foote_exercise_3_1_22b_mutation {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  IsGreatest {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} (iInf H)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3_1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from mere normality to a greatest-element characterization, requiring a new universal-order argument that every subgroup contained in all $H(i)$ lies below $i\text{Inf } H$. Although it reuses the parent's normality fact, the added obligation is substantive and not supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality to a greatest-element characterization. Besides establishing normality via the parent, it must prove membership and the universal order bound using the lattice properties of infima.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality to a greatest-element characterization of the intersection. The parent's normality lemma supplies only one component; proving maximality requires a separate lattice argument using ' le_iInf ', so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2810 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Dummit_Foote_exercise_3_1_22b_mutation.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  [inst_1 : Nonempty I] (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) :
  IsGreatest {K | K.Normal ∧ ∀ (i : I), K ≤ H i} (iInf H)
```

Read back by a model that saw only the Lean. Let G be a group, let I be a nonempty type, and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Then the infimum $\bigcap_{i \in I} H(i)$ is the greatest normal subgroup K of G such that $K \leq H(i)$ for every $i \in I$; that is, it itself is normal and contained in every $H(i)$, and every normal subgroup contained in every $H(i)$ is contained in $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem Dummit_Foote_exercise_3_1_22b_mutation {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  IsGreatest {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} (iInf H) := by
  have hnormal : Normal (iInf H) :=
    Subgroup.normal_iInf_normal (fun i => hH i)
  have hlower : ∀ i : I, iInf H ≤ H i := by
    intro i
    exact iInf_le H i
  have hmem : iInf H ∈ {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} := by
    exact ⟨hnormal, hlower⟩
  have hbound : ∀ {K : Subgroup G},
    K ∈ {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} → K ≤ iInf H := by
    intro K hK
    exact le_iInf (fun i => hK.2 i)
  exact ⟨hmem, hbound⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh__f53b8227

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f53b8227f02957dc

4.298 298. quotient_is_commutative_of_commutator_le

The problem

Let A be a group and let B be a normal subgroup containing the commutator subgroup of A . Prove that A/B is abelian.

Lean statement

```
theorem quotient_is_commutative_of_commutator_le {A : Type*} [Group A] (B : Subgroup A) [B.Normal] (h : _root_.commutator A ≤ B) :
  ∀ a b : A / B, a * b = b * a
```

Parent. Let A be an abelian group and let B be a subgroup of A . Prove that A/B is abelian.

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.3a {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ a b : A / B, a * b = b * a := by
```

What it contributes. Preserves the parent's quotient-commutativity checkpoint while weakening the ambient hypothesis from an abelian group to an arbitrary group with a commutator-containment condition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from an ambient commutative group to an arbitrary group whose normal subgroup contains the commutator subgroup. Proving quotient commutativity now requires the commutator characterization and cannot be obtained by directly applying the parent's 'mul_comm' argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the ambient commutative-group assumption with a general group and requires proving quotient commutativity from the substantive hypothesis that the commutator subgroup lies in the normal subgroup. The parent's direct application of multiplication commutativity cannot supply this quotient-group argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from an inherently commutative ambient group to an arbitrary group, requiring the nontrivial commutator-subgroup criterion and normality hypothesis to establish quotient commutativity. The parent's direct 'mul_comm' proof cannot supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2934 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_is_commutative_of_commutator_le.extracted_1_1 :  $\forall \{A : \text{Type } u_1\} [\text{inst} : \text{Group } A] (B : \text{Subgroup } A)$   
[inst_1 : B.Normal],  $\_root_.\text{commutator } A \leq B \rightarrow \forall (a \ b : A / B), a * b = b * a$ 
```

Read back by a model that saw only the Lean. *For every group A and every subgroup B of A that is normal, if the commutator subgroup of A is contained in B , then every pair of elements a and b of the quotient group A / B commutes: $a * b = b * a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_is_commutative_of_commutator_le {A : Type*} [Group A] (B : Subgroup A) [B.Normal] (h :  $\_root_.\text{commutator } A \leq B$ ) :
   $\forall a \ b : A / B, a * b = b * a$  := by
  have hc : IsMulCommutative (A / B) :=
    (Subgroup.Normal.quotient_commutative_iff_commutator_le).2 h
  intro a b
  exact hc.is_comm.comm a b
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__mh__68f603c7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 68f603c74754d74f

4.299 299. finite_abelian_group_prime_power_subgroup_quotient

The problem

Let G be a finite abelian group and let p^k divide its order, where p is prime. Show that G has a subgroup of order p^k whose quotient has order $|G|/p^k$.

Lean statement

```
theorem finite_abelian_group_prime_power_subgroup_quotient {G : Type*} [CommGroup G] [Fintype G] {p k : Nat} (hp : Nat.Prime p) (h : p ^ k ∣ Fintype.card G) :
  ∃ (H : Subgroup G) (hH : Fintype H), Nat.card H = p ^ k ∧ Nat.card (G / H) = Fintype.card G / (p ^ k)
```

Parent. Use Cauchy's Theorem and induction to show that a finite abelian group has a subgroup of order n for each positive divisor n of its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_3_4_4 {G : Type*} [CommGroup G] [Fintype G] {n : ℕ}
  (hn : n ∣ (card G)) :
  ∃ (H : Subgroup G) (H_fin : Fintype H), @card H H_fin = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive quotient-cardinality conclusion, requiring normality of the subgroup and the finite quotient cardinality formula, which the parent's existence-and-order result does not supply. Although Sylow's theorem supplies the subgroup order in both proofs, the new quotient argument is genuine proof-obligation expansion rather than recall.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from merely producing a subgroup of order n to producing a prime-power subgroup together with an exact quotient-cardinality formula. Its proof requires normality, quotient finiteness, the subgroup-quotient cardinality identity, and division arithmetic beyond the parent's construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive quotient-cardinality theorem, requiring normality, quotient finiteness, and a cardinality product/division argument beyond the parent's subgroup-existence conclusion. Although Sylow supplies the subgroup order, the new quotient equality is not a forced wrapper or trivial conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2215 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_abelian_group_prime_power_subgroup_quotient.extracted_1_1.{u_1} {G : Type u_1} [inst : CommGroup G]
  [inst_1 : Fintype G] {p k : ℕ} (hp : Nat.Prime p) (h : p ^ k ∣ card G) :
  ∃ H x, Nat.card H = p ^ k ∧ Nat.card (G / H) = card G / p ^ k
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let p and k be natural numbers, assume that p is prime and that p^k divides the cardinality of G . Then there exist a subgroup H of G and an element x of G such that the cardinality of H is p^k and the cardinality of the quotient group G/H is $\text{card}(G)$ divided by p^k .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem finite_abelian_group_prime_power_subgroup_quotient {G : Type*} [CommGroup G] [Fintype G] {p k : Nat} (hp : Nat.Prime p) (h : p ^
  k ∣ Fintype.card G) :
  ∃ (H : Subgroup G) (h : Fintype H), Nat.card H = p ^ k ∧ Nat.card (G / H) = Fintype.card G / (p ^ k) := by
  classical
  letI : Fact (Nat.Prime p) := ⟨hp⟩
  have h' : p ^ k ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using h
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) p h'
  letI : Fintype H := Fintype.ofFinite H
  have hcard : Nat.card H = p ^ k := hH
  have hcardF : Fintype.card H = p ^ k := by
    simpa [Nat.card_eq_fintype_card] using hcard
  letI : H.Normal := H.normal_of_comm
  letI : Fintype (G / H) := Quotient.fintype (QuotientGroup.leftRel H)
  have hmul : Fintype.card (G / H) * Fintype.card H = Fintype.card G := by
    simpa using (Subgroup.card_eq_card_quotient_mul_card_subgroup H).symm
  have hquot : Nat.card (G / H) = Fintype.card G / (p ^ k) := by
    have hpos : 0 < p ^ k := pow_pos hp.pos k
    have hmul' : Fintype.card (G / H) * (p ^ k) = Fintype.card G := by
      simpa [hcardF] using hmul
    calc
      Nat.card (G / H) = Fintype.card (G / H) := Nat.card_eq_fintype_card (α := G / H)
      _ = Fintype.card (G / H) * (p ^ k) / (p ^ k) := (Nat.mul_div_cancel _ hpos).symm
      _ = Fintype.card G / (p ^ k) := by rw [hmul']
  exact ⟨H, inferInstance, hcard, hquot⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_4_4__mh__174fe9de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 174fe9de19a2f61f

4.300 300. unique_normal_sylow_of_card_eq_mul

The problem

Let p be a prime with $25 \leq p \leq 100$. If a finite group has order $24p$, prove that it has a unique normal Sylow p -subgroup.

Lean statement

```
theorem unique_normal_sylow_of_card_eq_mul {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p | Nat.card G ∧ P.Normal ∧ ∀ Q : Sylow p G, Q = P
```

Parent. Prove that a group of order 312 has a normal Sylow p -subgroup for some prime p dividing its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_14 {G : Type*} [Group G] [Fintype G]
  (hG : card G = 312) :
  ∃ (p : ℕ) (P : Sylow p G), p.Prime ∧ (p | card G) ∧ P.Normal := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed prime and group order, and changes existence of a normal Sylow subgroup into a uniqueness conclusion. Its proof must establish the variable-prime bound argument forcing the Sylow count to equal one, which the parent's fixed-instance proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the specific prime and group order into arbitrary prime p with $25 \leq p \leq 100$, and changes existence of a normal Sylow subgroup into uniqueness of that subgroup. The bound is used essentially to force the Sylow count n dividing 24 and satisfying $n \equiv 1 \pmod p$ to equal 1, yielding a new uniqueness obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the prime and group order, requiring the new bound-based argument that the Sylow count satisfies $n < p$, and it strengthens normality to uniqueness of the Sylow subgroup. The parent's fixed arithmetic for $p = 13$ and $|G| = 312$ cannot be recalled directly to solve this statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2239 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_normal_sylow_of_card_eq_mul.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (p : ℕ)
  (hp : Nat.Prime p) (hp_bound : 25 ≤ p ∧ p ≤ 100) (hG : Nat.card G = 24 * p) :
  ∃ P, p ∣ Nat.card G ∧ (↑P).Normal ∧ ∀ (Q : Sylow p G), Q = P
```

Read back by a model that saw only the Lean. For a group G and a natural number p , if p is prime, $25 \leq p \leq 100$, and the cardinality of G is $24p$, then there exists a Sylow p -subgroup P of G such that p divides the cardinality of G , the subgroup P is normal in G , and every Sylow p -subgroup Q of G equals P .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem unique_normal_sylow_of_card_eq_mul {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p ∣ Nat.card G ∧ P.Normal ∧ ∀ Q : Sylow p G, Q = P := by
  classical
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨P⟩ := (inferInstance : Nonempty (Sylow p G))
  set n : ℕ := Nat.card (Sylow p G) with hn
  have hmod : n ≡ 1 [MOD p] := by
    simpa [n] using (card_sylow_modEq_one p G)
  have hdiv_index : n ∣ P.index := by
    simpa [n] using (Sylow.card_dvd_index (p := p) (G := G) P)
  have hindex_dvd : P.index ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
  have hdiv_card : n ∣ Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
  have hdiv_mul : n ∣ p * 24 := by
    rw [hG] at hdiv_card
    simpa [Nat.mul_comm] using hdiv_card
  have hnotdiv : ¬p ∣ n := by
    simpa [n] using (not_dvd_card_sylow p G)
  have hcoprime : Nat.Coprime n p := by
    simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
  have hdiv24 : n ∣ 24 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
  have hpos : 0 < n := by
    simpa [n] using (Nat.card_pos : 0 < Nat.card (Sylow p G))
  have hn_le : n ≤ 24 := Nat.le_of_dvd (by decide) hdiv24
  have hn_lt_p : n < p := by omega
  have hmod_eq : n % p = 1 % p := by
    simpa [Nat.ModEq] using hmod
  have hn_eq_one : n = 1 := by
    have hn_mod : n % p = n := Nat.mod_eq_of_lt hn_lt_p
    have hmod : 1 % p = 1 := Nat.mod_eq_of_lt (by omega)
    omega
  have hsub : Subsingleton (Sylow p G) := by
    exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
  have hnormal : P.Normal := by
    letI : Subsingleton (Sylow p G) := hsub
    exact Sylow.normal_of_subsingleton P
  have hunique : ∀ Q : Sylow p G, Q = P := by
    intro Q
    exact Subsingleton.elim Q P
  refine ⟨P, ?_, hnormal, hunique⟩
  rw [hG]
  exact ⟨24, by simp [Nat.mul_comm]⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_14__mh__15ef8ed7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 15ef8ed7b86c3e91

4.301 301. bounded_normal_sylow

The problem

Let G be a finite group and let p be a prime with $25 \leq p \leq 100$. If $|G| = 24p$, prove that G has a normal Sylow p -subgroup and that p divides $|G|$.

Lean statement

```
theorem bounded_normal_sylow {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p ∣ Nat.card G ∧ P.Normal := by
```

Parent. Prove that a group of order 312 has a normal Sylow p -subgroup for some prime p dividing its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_14 {G : Type*} [Group G] [Fintype G]
  (hG : card G = 312) :
  ∃ (p : ℕ) (P : Sylow p G), p.Prime ∧ (p ∣ card G) ∧ P.Normal := by
```

What it contributes. Reused the finite-group Sylow existence, Sylow-count congruence, count divisibility through the subgroup index, and coprime cancellation infrastructure while replacing 312 and 13 by the bounded parameter family $24p$ and p .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the Sylow argument over primes p with $25 \leq p \leq 100$. It derives that the Sylow count divides 24 while being congruent to 1 modulo p , then uses the bound $p > 24$ to force uniqueness and normality; this reasoning is not supplied by the fixed-order parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the Sylow-counting argument: it derives $n \mid 24$, uses $p > 24$ to force $n < p$, and combines this with $n \equiv 1 \pmod p$ to prove $n = 1$. The fixed-order parent does not supply this symbolic bounded-family reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes 13 as an arbitrary prime p greater than 24 and replaces fixed arithmetic case-work with a symbolic divisibility and congruence argument showing $n=1$. A memorized proof for order 312 and $p=13$ does not directly establish this family.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1918 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_normal_sylow.extracted.1.1.{u_1} {G : Type u_1} [inst : Group G] (p : ℕ) (hp : Nat.Prime p)
  (hp_bound : 25 ≤ p ∧ p ≤ 100) (hG : Nat.card G = 24 * p) : ∃ P, p | Nat.card G ∧ (↑P).Normal
```

Read back by a model that saw only the Lean. *For a group G , a natural number p that is prime and satisfies $25 \leq p \leq 100$, if the cardinality of G is $24p$, then there exists a Sylow p -subgroup P of G such that p divides the cardinality of G and the subgroup underlying P is normal in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem bounded_normal_sylow {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p | Nat.card G ∧ P.Normal := by
  classical
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨P⟩ := (inferInstance : Nonempty (Sylow p G))
  set n : ℕ := Nat.card (Sylow p G) with hn
  have hmod : n ≡ 1 [MOD p] := by
    simpa [n] using (card_sylow_modEq_one p G)
  have hdiv_index : n | P.index := by
    simpa [n] using (Sylow.card_dvd_index (p := p) (G := G) P)
  have hindex_dvd : P.index | Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
  have hdiv_card : n | Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
  have hdiv_mul : n | p * 24 := by
    rw [hG] at hdiv_card
    simpa [Nat.mul_comm] using hdiv_card
  have hnotdiv : ¬p | n := by
    simpa [n] using (not_dvd_card_sylow p G)
  have hcoprime : Nat.Coprime n p := by
    simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
  have hdiv24 : n | 24 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
  have hpos : 0 < n := by
    simpa [n] using (Nat.card_pos : 0 < Nat.card (Sylow p G))
  have hn_le : n ≤ 24 := Nat.le_of_dvd (by decide) hdiv24
  have hn_lt_p : n < p := by omega
  have hmod_eq : n % p = 1 % p := by
    simpa [Nat.ModEq] using hmod
  have hn_eq_one : n = 1 := by
    have hn_mod : n % p = n := Nat.mod_eq_of_lt hn_lt_p
    have hone_mod : 1 % p = 1 := Nat.mod_eq_of_lt (by omega)
    omega
  have hsub : Subsingleton (Sylow p G) := by
    exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
  have hnormal : P.Normal := by
    letI : Subsingleton (Sylow p G) := hsub
    exact Sylow.normal_of_subsingleton P
  refine ⟨P, ?_, hnormal⟩
  rw [hG]
  exact ⟨24, by simp [Nat.mul_comm]⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_14__mh__fd07aa1a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d05b91cd454267e7

4.302 302. exists_element_order_eleven

The problem

Let G be a finite group of order 462. Prove that G contains an element of order 11.

Lean statement

```
theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Preserves the finite group and cardinality-462 setting while redirecting the cardinality factorization toward an element-order witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from nonsimplicity to existence of an element of order 11 and proves it via a prime-order divisibility theorem, rather than reusing the parent's Sylow-counting argument. The factorization of 462 supplies the required divisibility checkpoint, and no parent conclusion is wrapped or recalled.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from nonsimplicity to existence of an element of order 11 and proves it via prime-order divisibility, using the factorization of 462. The parent's Sylow-subgroup uniqueness and contradiction argument cannot be recalled to solve this statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from non-simplicity to explicitly constructing an element of order 11, using divisibility of the group order and a prime-order existence theorem. This is not obtained by simply recalling the parent's contradiction argument or its normal-subgroup conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2768 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_element_order_eleven.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
(hG : card G = 462) : ∃ x, orderOf x = 11
```

Read back by a model that saw only the Lean. *For any finite group G with exactly 462 elements, there exists an element x in G whose order is 11.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
(hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11 := by
  have h_factor : Fintype.card G = 11 * 42 := by
    calc
      Fintype.card G = 462 := hG
      _ = 11 * 42 := by norm_num
  have h_prime11 : Nat.Prime 11 := by
    norm_num
  letI : Fact (Nat.Prime 11) := ⟨h_prime11⟩
  have h_dvd : 11 ∣ Fintype.card G := by
    rw [h_factor]
    norm_num
  have h_witness : ∃ x : G, orderOf x = 11 :=
    exists_prime_orderOf_dvd_card 11 h_dvd
  exact h_witness
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__mh__fe35117f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fe35117fe069b9e1

4.303 303. rational_sum_sq_mul

The problem

Prove that if an integer n is the sum of the squares of two rational numbers and an integer m is also the sum of the squares of two rational numbers, then nm is the sum of the squares of two rational numbers.

Lean statement

```
theorem rational_sum_sq_mul {n m : ℤ} {r s u v : ℚ}
  (hn : r ^ 2 + s ^ 2 = n) (hm : u ^ 2 + v ^ 2 = m) :
  ∃ x y : ℚ, x ^ 2 + y ^ 2 = (n * m : ℚ)
```

Parent. Prove that if an integer is the sum of two rational squares, then it is the sum of two integer squares.

Parent in Lean

```
theorem Dummit_Foote_exercise_8_3_4 {n : ℤ} {r s : ℚ}
  (h : r^2 + s^2 = n) :
  ∃ a b : ℤ, a^2 + b^2 = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different closure theorem: two rational sum-of-two-squares representations combine via the Brahmagupta identity to represent the product. The parent's denominator-clearing and integer-witness argument does not supply this multiplicative construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child establishes the symbolic multiplicative closure of rational sums of two squares, using the Brahmagupta–Fibonacci identity and two independent hypotheses. This reasoning is absent from the parent's denominator-clearing argument and cannot be recovered by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely new Brahmagupta–Fibonacci two-square multiplication identity, combining both hypotheses through a nontrivial ring calculation. The parent only supplies separate representability results and does not provide this multiplicative construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1886 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_sum_sq_mul.extracted_1_1 {n m : ℤ} {r s u v : ℚ} (hn : r ^ 2 + s ^ 2 = ↑n) (hm : u ^ 2 + v ^ 2 = ↑m) :  
  ∃ x y, x ^ 2 + y ^ 2 = ↑n * ↑m
```

Read back by a model that saw only the Lean. For integers n and m and rational numbers r, s, u , and v , if $r^2 + s^2 = n$ and $u^2 + v^2 = m$ (viewing n and m as rational numbers), then there exist rational numbers x and y such that $x^2 + y^2 = nm$ (viewing n and m as rational numbers).

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem rational_sum_sq_mul {n m : ℤ} {r s u v : ℚ}  
  (hn : r ^ 2 + s ^ 2 = n) (hm : u ^ 2 + v ^ 2 = m) :  
  ∃ x y : ℚ, x ^ 2 + y ^ 2 = (n * m : ℚ) := by  
  refine ⟨r * u - s * v, r * v + s * u, ?_⟩  
  calc  
    (r * u - s * v) ^ 2 + (r * v + s * u) ^ 2 =  
      (r ^ 2 + s ^ 2) * (u ^ 2 + v ^ 2) := by ring  
    _ = (n : ℚ) * (m : ℚ) := by rw [hn, hm]  
    _ = (n * m : ℚ) := by norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_8_3_4__mh__18a1117f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 23fe6774c7a20af8

4.304 304. unit_cyclic_action_factorization

The problem

Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$ and a is a unit modulo m , show that exponentiation by a defines a multiplicative cyclic action of the n residue classes.

Lean statement

```
theorem unit_cyclic_action_factorization {a n m : ℕ} (hn : 0 < n)
  (hunit : IsUnit (a : ZMod m))
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → (ZMod m)×,
    (∀ k : ℕ, f ⟨k % n, Nat.mod_lt _ hn⟩ = hunit.unit ^ k) ∧
    (∀ r s : Fin n,
      f r * f s = f ⟨(r : ℕ) + (s : ℕ) % n, Nat.mod_lt _ hn⟩)
```

Parent. Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$, show that exponentiation by a depends only on the residue of the exponent modulo n , and hence defines a function on the n residue classes.

Parent in Lean

```
theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f ⟨k % n, Nat.mod_lt _ hn⟩ = (a : ZMod m) ^ k)
```

What it contributes. Used the division-algorithm reduction of exponents modulo n and the visible period equation; upgraded the codomain from residues to units and added multiplicative factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires genuinely new mathematics: lifting the residue-class element to a unit, transferring the period relation to the unit group, and proving the cyclic multiplicative law. The parent only establishes residue invariance and existence of a function into $\text{ZMod } m$, so recalling its proof does not supply the unit-valued group-action argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely lifts the residue-indexed action from $\text{ZMod } m$ to its unit group and adds a cyclic multiplication law. The parent's reduction argument does not supply the unit lift, the induced period in $(\text{ZMod } m)^\times$, or the group-action identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child upgrades the codomain from $\text{ZMod } m$ to its unit group and adds a genuine cyclic-action law for combining residue indices. Its proof requires transporting the period relation through `Units.ext` and proving multiplicative compatibility, which the parent's residue-reduction argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2319 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unit_cyclic_action_factorization.extracted_1_1 {a n m : ℕ} (hn : 0 < n) (hunit : IsUnit ↑a)
(hperiod : ↑a ^ n = 1) :
  ∃ f, (∀ (k : ℕ), f (k % n, --) = hunit.unit ^ k) ∧ ∀ (r s : Fin n), f r * f s = f ((↑r + ↑s) % n, --)
```

Read back by a model that saw only the Lean. For natural numbers a , n , and m , if n is positive, the natural number a (viewed in the relevant monoid) is a unit, and the associated unit hunit.unit satisfies $\text{hunit.unit}^n = 1$, then there exists a function f from $\text{Fin } n$ to the monoid of units such that, for every natural number k , f at the residue class $k \bmod n$ equals hunit.unit^k , and for all $r, s : \text{Fin } n$, $f(r) \cdot f(s)$ equals f at the residue class $(r+s) \bmod n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem unit_cyclic_action_factorization {a n m : ℕ} (hn : 0 < n)
(hunit : IsUnit (a : ZMod m))
(hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → (ZMod m)×,
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = hunit.unit ^ k) ∧
    (∀ r s : Fin n,
      f r * f s = f ((r : ℕ) + (s : ℕ)) % n, Nat.mod_lt _ hn)) := by
  have hperiodu : hunit.unit ^ n = 1 := by
    apply Units.ext
    simpa [hunit.unit_spec] using hperiod
  have hreduce : ∀ k : ℕ,
    hunit.unit ^ k = hunit.unit ^ (k % n) := by
    intro k
    have hrepr : k = k % n + n * (k / n) := by
      exact (Nat.mod_add_div k n).symm
    calc
      hunit.unit ^ k = hunit.unit ^ (k % n + n * (k / n)) := by
        exact congrArg (fun t : ℕ => hunit.unit ^ t) hrepr
      _ = hunit.unit ^ (k % n) := by
        rw [pow_add, pow_mul, hperiodu]
      simp
  refine (fun r : Fin n => hunit.unit ^ (r : ℕ), ?_, ?_)
  · intro k
    exact (hreduce k).symm
  · intro r s
    change hunit.unit ^ (r : ℕ) * hunit.unit ^ (s : ℕ) =
      hunit.unit ^ ((r : ℕ) + (s : ℕ)) % n
    calc
      hunit.unit ^ (r : ℕ) * hunit.unit ^ (s : ℕ) =
        hunit.unit ^ ((r : ℕ) + (s : ℕ)) := by rw [← pow_add]
      _ = hunit.unit ^ ((r : ℕ) + (s : ℕ)) % n := hreduce _
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me.mh.mh__769445da

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 769445da3e5fcf13

4.305 305. residue_power_action

The problem

Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$, show that exponentiation by a depends only on the residue of the exponent modulo n , and hence defines a function on the n residue classes.

Lean statement

```
theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = (a : ZMod m) ^ k)
```

Parent. Let a , n , and m be natural numbers. If a^n is congruent to 1 modulo m , show that a^k is congruent to 1 modulo m for every exponent k divisible by n .

Parent in Lean

```
theorem power_period_of_modulus {a n m : ℕ}
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∀ k : ℕ, n ∣ k → ((a : ZMod m) ^ k = 1)
```

What it contributes. Generalizes the parent's divisible-exponent power argument: its period equation and power decomposition become the reduction lemma that supports a residue-indexed function.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes divisibility to a residue-indexed cyclic action: it proves powers depend only on the exponent modulo the period and constructs the corresponding function on 'Fin n'. This requires new remainder decomposition and periodicity reasoning beyond the parent's single divisible-exponent case.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from powers at multiples of n to a well-defined residue-indexed action, requiring division-with-remainder decomposition and reduction of arbitrary exponents. The parent's divisibility argument alone does not establish this remainder-based periodicity or construct the function.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes periodicity from exponents divisible by n to a well-defined action on all residue classes modulo n . Its proof requires division-algorithm decomposition, reduction to the remainder, and construction of a subtype-indexed function, which the parent's divisibility argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2313 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B replaces the hypothesis $a^n \equiv 1 \pmod m$ with the stronger exact equality $a^n = 1$ and likewise asserts exact equality $a^x = a^y$ rather than congruence modulo m ; it also omits the modulus m from the conclusion.

Goal round-trip: what Lean elaborated

```
theorem residue_power_action.extracted_1_1 {a n m : ℕ} (hn : 0 < n) (hperiod : ↑a ^ n = 1) :
  ∃ f, (∀ (x y : ℕ), x % n = y % n → ↑a ^ x = ↑a ^ y) ∧ ∀ (k : ℕ), f (k % n, --) = ↑a ^ k
```

Read back by a model that saw only the Lean. For natural numbers a , n , and m , if n is positive and the n th power of a (interpreted in the relevant ambient type) equals 1, then there exists a function f on natural numbers less than n such that, for all natural numbers x and y , if x and y have the same remainder modulo n , then $a^x = a^y$, and for every natural number k , f applied to the residue $k \bmod n$ equals a^k .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = (a : ZMod m) ^ k) := by
  have hreduce : ∀ k : ℕ,
    (a : ZMod m) ^ k = (a : ZMod m) ^ (k % n) := by
    intro k
    have hrepr : k = k % n + n * (k / n) := by
      exact (Nat.mod_add_div k n).symm
    calc
      (a : ZMod m) ^ k = (a : ZMod m) ^ (k % n + n * (k / n)) := by
        exact congrArg (fun t : ℕ => (a : ZMod m) ^ t) hrepr
      _ = (a : ZMod m) ^ (k % n) := by
        rw [pow_add, pow_mul, hperiod]
        simp
  refine (fun r : Fin n => (a : ZMod m) ^ (r : ℕ), ?_, ?_)
  · intro x y hxy
    calc
      (a : ZMod m) ^ x = (a : ZMod m) ^ (x % n) := hreduce x
      _ = (a : ZMod m) ^ (y % n) := by rw [hxy]
      _ = (a : ZMod m) ^ y := (hreduce y).symm
  · intro k
    exact (hreduce k).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me.mh__0604a773

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 0604a773fdfdc2ec

4.306 306. odd_multiple_period_mod_add_one

The problem

Let $a > 1$ and $n > 0$ be integers. Show that every exponent of the form $n(2q + 1)$ gives $a^{n(2q+1)} \equiv -1 \pmod{a^n + 1}$ for every nonnegative integer q .

Lean statement

```
theorem odd_multiple_period_mod_add_one {a n : ℕ} :
  ∀ q : ℕ, ((a : ℤMod (a ^ n + 1)) ^ (n * (2 * q + 1))) = -1
```

Parent. Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Parent in Lean

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ℤMod (a ^ n - 1)) ^ k = 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the modulus from $a^n - 1$ to $a^n + 1$ and establishes a sign-changing result: the base power is -1 , so only odd multiples yield the stated value. The parent's argument for a base power equal to 1 does not supply this new odd-exponent reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the modulus and target: it derives $a^n = -1$ modulo $a^n + 1$ and must additionally reason that every odd power of -1 remains -1 . This is not recoverable by recalling the parent's period-to-1 argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the modulus from $a^n - 1$ to $a^n + 1$ and the conclusion from powers divisible by n equaling 1 to odd multiples of n equaling -1 . Its proof requires establishing $a^n = -1$ and handling odd powers of -1 , which the parent's argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2288 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_multiple_period_mod_add_one.extracted_1.1 : ∀ {a n : ℕ},
  1 < a → 0 < n → ∀ (q : ℕ), ↑a ^ (n * (2 * q + 1)) = -1
```

Read back by a model that saw only the Lean. For all natural numbers a and n , if $1 < a$ and $0 < n$, then for every natural number q , the $(n(2q+1))$ -st power of the cast of a equals -1 .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem odd_multiple_period_mod_add_one {a n : ℕ} :
  ∀ q : ℕ, ((a : ZMod (a ^ n + 1)) ^ (n * (2 * q + 1))) = -1 := by
  let m : ℕ := a ^ n + 1
  change ∀ q : ℕ, ((a : ZMod m) ^ (n * (2 * q + 1))) = -1
  have hzero : ((a ^ n + 1 : ℕ) : ZMod m) = 0 := by
    simp only [m] using (ZMod.natCast_self (a ^ n + 1))
  have hsum : ((a ^ n : ℕ) : ZMod m) + 1 = 0 := by
    simp only [Nat.cast_add, Nat.cast_one] using hzero
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = -1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = (((a ^ n : ℕ) : ZMod m) + 1) - 1 := by ring
      _ = 0 - 1 := by rw [hsum]
      _ = -1 := by simp
  have hbase : ((a : ZMod m) ^ n) = -1 := by
    simp [Nat.cast_pow] using hbase_cast
  intro q
  have hodd : ((-1 : ZMod m) ^ (2 * q + 1)) = -1 := by
    calc
      (-1 : ZMod m) ^ (2 * q + 1) = ((-1 : ZMod m) ^ 2) ^ q * (-1 : ZMod m) ^ 1 := by
        rw [pow_add, pow_mul]
      _ = -1 := by simp only [pow_one]; simp
  calc
    (a : ZMod m) ^ (n * (2 * q + 1)) = ((a : ZMod m) ^ n) ^ (2 * q + 1) := by
      rw [pow_mul]
    _ = (-1 : ZMod m) ^ (2 * q + 1) := by rw [hbase]
    _ = -1 := hodd
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.mh__946d4e11

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint b2e35bfc0f1aaf9c

4.307 307. coprime_base_pow_sub_one

The problem

For natural numbers a and n , if $a > 1$ and $n > 0$, show that a and $a^n - 1$ are coprime.

Lean statement

```
theorem coprime_base_pow_sub_one {a n : ℕ} (ha : a > 1) (hn : 0 < n) : Nat.Coprime a (a ^ n - 1)
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4_36 {a n : ℕ} (h : a > 1) :  
  n ∣ (a ^ n - 1).totient := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a totient divisibility statement to proving coprimality of a with $a^n - 1$. Its proof uses a successor-coprimality argument and divisibility transfer, reasoning not supplied by the parent's cyclic-group totient proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility of the totient into a coprimality claim and proves it via consecutive-number coprimality plus divisibility transfer. This requires genuinely different arithmetic reasoning and cannot be obtained by recalling the parent's cyclic-group argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1879 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coprime_base_pow_sub_one.extracted_1_1 {a n : ℕ} (ha : a > 1) (hn : 0 < n) : a.Coprime (a ^ n - 1)
```

Read back by a model that saw only the Lean. For natural numbers a and n , if $a > 1$ and $n > 0$, then a is coprime to $a^n - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem coprime_base_pow_sub_one {a n : ℕ} (ha : a > 1) (hn : 0 < n) : Nat.Coprime a (a ^ n - 1) := by
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have ha_pow : a ∣ a ^ n := dvd_pow_self a (Nat.ne_of_gt hn)
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hsum : (a ^ n - 1) + 1 = a ^ n := Nat.sub_add_cancel hle
  have hcop_pow : Nat.Coprime (a ^ n - 1) (a ^ n) := by
    have hcop_succ : Nat.Coprime (a ^ n - 1) ((a ^ n - 1) + 1) := by
      rw [Nat.coprime_self_add_right]
      exact Nat.coprime_one_right _
    rw [← hsum]
    exact hcop_succ
  have hcop_a : Nat.Coprime (a ^ n - 1) a := Nat.Coprime.of_dvd_right ha_pow hcop_pow
  exact hcop_a.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh__41f3065a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 029c75d57914be73

4.308 308. prime_order_unit_mod_pow_sub_one

The problem

Let $a > 1$, $n > 1$, and let p be a prime divisor of n . If n divides the Euler totient of $a^n - 1$, prove that the group of units modulo $a^n - 1$ contains an element of exact order p .

Lean statement

```
theorem prime_order_unit_mod_pow_sub_one {a n p : ℕ}
  (ha : a > 1) (hn : n > 1) (hp : p.Prime) (hpn : p | n)
  (hφ : n | (a ^ n - 1).totient) :
  ∃ u : (ZMod (a ^ n - 1))^*, orderOf u = p
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where φ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :
  n | (a ^ n - 1).totient := by
```

What it contributes. Supplies the totient-divisibility checkpoint $n \mid \varphi(a^n - 1)$, consumed via transitivity with $p \mid n$ to obtain the exact-order witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes divisibility of the totient into the substantive group-theoretic conclusion that every prime divisor p of n occurs as the exact order of a unit modulo $a^n - 1$. Its proof requires applying the unit-group cardinality theorem and extracting an element of prime order, which the parent's proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine group-theoretic existence claim: from the totient divisibility it derives that every prime divisor of n divides the unit-group cardinality, then obtains an element of exact order p . This requires reasoning about units and prime orders that is absent from the parent's proof and is not the retained power-period theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility of the totient into the substantive group-theoretic conclusion that every prime divisor p of n occurs as the exact order of a unit modulo $a^n - 1$. Its proof uses prime divisibility of the unit-group cardinality and a prime-order existence theorem, which the parent's argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2276 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_order_unit_mod_pow_sub_one.extracted_1_1 {a n p : ℕ} (ha : a > 1) (hn : n > 1) (hp : Nat.Prime p)
(hpn : p ∣ n) (hφ : n ∣ (a ^ n - 1).totient) : ∃ u, orderOf u = p
```

Read back by a model that saw only the Lean. For natural numbers a , n , and p , if $a > 1$, $n > 1$, p is prime, p divides n , and n divides the Euler totient of $a^n - 1$, then there exists an element u whose order is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem prime_order_unit_mod_pow_sub_one {a n p : ℕ}
(ha : a > 1) (hn : n > 1) (hp : p.Prime) (hpn : p ∣ n)
(hφ : n ∣ (a ^ n - 1).totient) :
  ∃ u : (ZMod (a ^ n - 1))^*, orderOf u = p := by
  have hnpos : 0 < n := by omega
  have hpow : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha
  have hmpos : 0 < a ^ n - 1 := Nat.sub_pos_of_lt hpow
  letI : NeZero (a ^ n - 1) := ⟨Nat.ne_of_gt hmpos⟩
  have hpcard : p ∣ Fintype.card (ZMod (a ^ n - 1))^* := by
    rw [ZMod.card_units_eq_totient]
    exact Nat.dvd_trans hpn hφ
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨u, hu⟩ := exists_prime_orderOf_dvd_card p hpcard
  exact ⟨u, hu⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh__b37ad707

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b37ad70767ec3371

4.309 309. two_distinct_order_eleven_totient

The problem

Let $a > 1$ be an integer, and suppose that every positive integer n divides $\varphi(a^n - 1)$. If a finite group has order 462, prove that it contains two distinct elements x and y of order 11, each satisfying $11 \mid \varphi(a^{\text{ord}(\cdot)} - 1)$.

Lean statement

```
theorem two_distinct_order_eleven_totient
  {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y : G,
    orderOf x = 11 ∧ orderOf y = 11 ∧ x ≠ y ∧
    11 ∣ (a ^ orderOf x - 1).totient ∧
    11 ∣ (a ^ orderOf y - 1).totient
```

Parent. Let $a > 1$ be an integer, and suppose that $n \mid \varphi(a^n - 1)$ for every positive integer n . If a finite group has order 462, show that it contains an element x of order 11 such that $11 \mid \varphi(a^{\text{ord}(x)} - 1)$.

Parent in Lean

```
theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 ∣ (a ^ orderOf x - 1).totient
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens existence of one order-11 element to two distinct such elements. Proving $x \neq x^{-1}$ requires a new group-order argument using order 11, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence to two distinct elements of order 11. It requires the new argument that an order-11 element cannot equal its inverse, then separately transports the totient divisibility to the inverse; this is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine distinctness obligation: it must prove an element of order 11 cannot equal its inverse, using the resulting order divisibility contradiction. This is new mathematics beyond the parent's existence proof, and the Lean proof is complete.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem two_distinct_order_eleven_totient.extracted_1.1.{u_1} {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ (n : ℕ), a > 1 → n | (a ^ n - 1).totient) {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y,
    orderOf x = 11 ∧
    orderOf y = 11 ∧ x ≠ y ∧ 11 | (a ^ orderOf x - 1).totient ∧ 11 | (a ^ orderOf y - 1).totient
```

Read back by a model that saw only the Lean. *Let a be a natural number with $a > 1$, and suppose that for every natural number n , if $a > 1$ then n divides the Euler totient of $a^n - 1$. Let G be a finite group with exactly 462 elements. Then there exist distinct elements x and y of G such that both x and y have order 11, and 11 divides the Euler totient of $a^{(\text{orderOf } x)} - 1$ as well as the Euler totient of $a^{(\text{orderOf } y)} - 1$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem two_distinct_order_eleven_totient
  {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n | (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y : G,
    orderOf x = 11 ∧ orderOf y = 11 ∧ x ≠ y ∧
    11 | (a ^ orderOf x - 1).totient ∧
    11 | (a ^ orderOf y - 1).totient := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : 11 | Fintype.card G := by
    rw [hG]
    norm_num
  obtain (x, hx) := exists_prime_orderOf_dvd_card 11 hcard
  have htot : 11 | (a ^ 11 - 1).totient := hHerstein 11 ha
  have hy : orderOf x⁻¹ = 11 := by
    simpa using hx
  have htot_inv : 11 | (a ^ orderOf x⁻¹ - 1).totient := by
    simpa [hy] using htot
  have hne : x ≠ x⁻¹ := by
    intro heq
    have hmul := congrArg (fun z : G => z * x) heq
    have hpow : x ^ 2 = 1 := by
      simpa [pow_two] using hmul
    have hd : orderOf x | 2 := orderOf_dvd_of_pow_eq_one hpow
    rw [hx] at hd
    norm_num at hd
  have htot_x : 11 | (a ^ orderOf x - 1).totient := by
    simpa [hx] using htot
  exact ⟨x, x⁻¹, hx, hy, hne, htot_x, htot_inv⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__x__Dummit_Foote_exercise_4_5_23__mh.xh.mh__9c519466

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9c5194665bf17a96

4.310 310. mem_zpowers_conjugate_iff

The problem

Assume every subgroup of a group G is normal. Show that an element x belongs to the cyclic subgroup generated by a if and only if its conjugate by b does too.

Lean statement

```
theorem mem_zpowers_conjugate_iff {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) :
  x ∈ Subgroup.zpowers a ↔ b * x * b⁻¹ ∈ Subgroup.zpowers a
```

Parent. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-way claim about conjugated powers into a bidirectional membership characterization for arbitrary x . Its reverse implication requires a new conjugation-by-inverse argument, which is not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from conjugates of specific powers a^n to an iff characterization for every element of the subgroup. Its reverse direction requires applying normality to the inverse conjugation and simplifying, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from conjugates of powers of a to an equivalence for every element x , requiring the new reverse-conjugation argument using b^{-1} and group simplification. The parent proof only establishes forward membership for a^n and does not supply this converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2316 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mem_zpowers_conjugate_iff.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) : x ∈ Subgroup.zpowers a ↔ b * x * b-1 ∈ Subgroup.zpowers a
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. For any elements a, b , and x of G , x belongs to the subgroup of G consisting of the integer powers of a if and only if bxb^{-1} belongs to that same subgroup.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem mem_zpowers_conjugate_iff {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) :
  x ∈ Subgroup.zpowers a ↔ b * x * b-1 ∈ Subgroup.zpowers a := by
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  constructor
  · intro hx
    exact hnormal.conj_mem x hx b
  · intro hx
    have hreverse : b-1 * (b * x * b-1) * (b-1)-1 ∈ Subgroup.zpowers a :=
      hnormal.conj_mem (b * x * b-1) hx b-1
    simpa [mul_assoc] using hreverse
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.mh__50d566d4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 50d566d4c0588323

4.311 311. conjugation_bijective_on_zpowers

The problem

Show that conjugation by b restricts to a bijection of the cyclic subgroup generated by a .

Lean statement

```
theorem conjugation_bijective_on_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F : Subgroup.zpowers a → Subgroup.zpowers a,
  Function.Bijective F ∧
  ∀ x : Subgroup.zpowers a, (F x : G) = b * (x : G) * b⁻¹
```

Parent. In a group where every subgroup is normal, conjugating any integer power of an element by another element produces a power of the original element. Express this by exhibiting an integer exponent for each conjugate.

Parent in Lean

```
theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b⁻¹ = a ^ k
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely stronger structural result: conjugation restricts to a bijection on the subgroup of powers, requiring explicit inverse construction and injectivity/surjectivity arguments. The parent's existence-of-an-exponent result supplies at most closure, not the bijection proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence of an exponent for each conjugated power to a bijective self-map of the entire subgroup. Proving bijectivity requires constructing and verifying an explicit inverse, a genuinely new obligation absent from the parent's existential membership argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of an exponent for each conjugate power into a bijective self-map of the entire subgroup of integer powers. Proving bijectivity requires constructing and verifying the inverse conjugation map, which is not supplied by the parent's membership-and-extraction argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2314 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B adds the hypothesis that every subgroup of G is normal, while Statement A does not state this restriction; without it, conjugation by b need not preserve the cyclic subgroup generated by a .

Goal round-trip: what Lean elaborated

```
theorem conjugation_bijective_on_zpowers.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F, Function.Bijective F ∧ ∀ (x : ↑(Subgroup.zpowers a)), ↑(F x) = b * ↑x * b-1
```

Read back by a model that saw only the Lean. For a group G such that every subgroup of G is normal, and for any elements $a, b \in G$, there exists a bijective function F from the subgroup of G consisting of the powers of a to itself such that, for every element x of this subgroup, the underlying element of G represented by $F(x)$ equals $b \cdot x \cdot b^{-1}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugation_bijective_on_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F : Subgroup.zpowers a → Subgroup.zpowers a,
    Function.Bijective F ∧
      ∀ x : Subgroup.zpowers a, (F x : G) = b * (x : G) * b-1 := by
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  let F : Subgroup.zpowers a → Subgroup.zpowers a := fun x =>
    (b * (x : G) * b-1, by
      exact hnormal.conj_mem (x : G) x.property b)
  let R : Subgroup.zpowers a → Subgroup.zpowers a := fun x =>
    (b-1 * (x : G) * b, by
      exact hnormal.conj_mem' (x : G) x.property b)
  have hleft : ∀ x : Subgroup.zpowers a, R (F x) = x := by
    intro x
    apply Subtype.ext
    change b-1 * (b * (x : G) * b-1) * b = (x : G)
    group
  have hright : ∀ y : Subgroup.zpowers a, F (R y) = y := by
    intro y
    apply Subtype.ext
    change b * (b-1 * (y : G) * b) * b-1 = (y : G)
    group
  refine ⟨F, ?_, ?_⟩
  · constructor
    · intro x y hxy
      have h := congrArg R hxy
      rw [hleft x, hleft y] at h
      exact h
    · intro y
      exact ⟨R y, hright y⟩
  · intro x
    rfl
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.ms.mh__6af04906

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 6af049069f0272af

4.312 312. unique_commutation_exponent

The problem

Let G be a group in which every subgroup is normal. Suppose that a has infinite order. Prove that for every $b \in G$, there is a unique integer j such that $ba = a^j b$.

Lean statement

```
theorem unique_commutation_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ, b * a = a ^ j * b
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

What it contributes. Supplies the cyclic-subgroup normality and conjugation argument establishing the existence of an exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness theorem: the infinite-order hypothesis is used to show that any two exponents satisfying the commutation equation are equal. The parent only supplies existence and does not provide this cancellation-plus-infinite-order argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness conclusion: after obtaining existence via the parent's conjugation argument, it uses the infinite-order hypothesis on a to prove any two valid exponents are equal. This additional cancellation and integer-exponent reasoning is not supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens existence to uniqueness by using the infinite-order hypothesis to derive $a^j = a^k$ and then $j = k$ via $a^{j-k} = 1$. The parent's cyclic-subgroup normality argument supplies only existence and does not determine the exponent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2197 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_commutation_exponent.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a : G) (hinf : ∀ (n : ℤ), a ^ n = 1 → n = 0) (b : G) :
  ∃! j, b * a = a ^ j * b
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. Let a and b be elements of G , and suppose that for every integer n , if $a^n = 1$ then $n = 0$. Then there exists a unique integer j such that $b \cdot a = a^j \cdot b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem unique_commutation_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ, b * a = a ^ j * b := by
  classical
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : b * a * b⁻¹ ∈ Subgroup.zpowers a := by
    simpa using hHnormal.conj_mem a ha b
  obtain (j, hj) := Subgroup.mem_zpowers_iff.mp (by
    simpa using hconj)
  have hex : b * a = a ^ j * b := by
    calc
      b * a = (b * a) * (b⁻¹ * b) := by simp
      _ = (b * a * b⁻¹) * b := by simp [mul_assoc]
      _ = a ^ j * b := by simp [hj]
  refine (j, hex, ?_)
  intro k hk
  have hpow : a ^ j = a ^ k := by
    exact mul_right_cancel (hex.symm.trans hk)
  have hrel : a ^ (j - k) = 1 := by
    rw [zpow_sub, hpow]
    simp
  have hzero : j - k = 0 := hinf (j - k) hrel
  exact (sub_eq_zero.mp hzero).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh__85f601c1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 85f601c1761229c5

4.313 313. Herstein_exercise_2_5_23_mutation

The problem

Let G be a group in which every subgroup is normal. Show that for any $a, b \in G$, there is an integer j such that $ba^n = a^{jn}b$ for every integer n .

Lean statement

```
theorem Herstein_exercise_2_5_23_mutation {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j : ℤ, ∀ n : ℤ, b * a ^ n = a ^ (j * n) * b
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^jb$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ) , b*a = a^j * b := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the single relation to a uniform identity for every integer power n . Proving this requires showing conjugation preserves integer powers and combining exponents, which is new reasoning beyond the parent's base-case rearrangement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the single conjugation identity to a uniform statement for every integer power. Proving it requires recognizing conjugation as a group homomorphism and applying preservation of integer powers, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the conclusion from one conjugation identity to a uniform identity for every integer power. Proving it requires recognizing conjugation as a group automorphism and applying that map to arbitrary integer powers, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2817 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Herstein_exercise_2_5_23_mutation.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) : ∃ j, ∀ (n : ℤ), b * a ^ n = a ^ (j * n) * b
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. Then for any elements $a, b \in G$, there exists an integer j such that, for every integer n , $b a^n = a^{jn} b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem Herstein_exercise_2_5_23_mutation {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j : ℤ, ∀ n : ℤ, b * a ^ n = a ^ (j * n) * b := by
  classical
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : b * a * b⁻¹ ∈ Subgroup.zpowers a := by
    simpa using hnormal.conj_mem a ha b
  obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp (by
    simpa using hconj)
  refine ⟨j, ?_⟩
  intro n
  have hpow : b * a ^ n * b⁻¹ = (b * a * b⁻¹) ^ n := by
    change (MulAut.conj b) (a ^ n) = ((MulAut.conj b) a) ^ n
    exact map_zpow (MulAut.conj b) a n
  calc
    b * a ^ n = (b * a ^ n * b⁻¹) * b := by simp [mul_assoc]
    _ = (b * a * b⁻¹) ^ n * b := by rw [hpow]
    _ = (a ^ j) ^ n * b := by rw [hj]
    _ = a ^ (j * n) * b := by simp [zpow_mul]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh__ba56f709

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint ba56f7097045d343

4.314 314. anticommutator_commutates_with_int_polynomial

The problem

Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with integer coefficients.

Lean statement

```
theorem anticommutator_commutates_with_int_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℤ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with natural-number coefficients.

Parent in Lean

```
theorem anticommutator_commutates_with_nat_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℕ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

What it contributes. Supplied the square-zero calculation, power-commutation induction, natural-coefficient transport, and support-sum evaluation pattern; the child extends the coefficient argument through signed integer decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes polynomial coefficients from $\mathbb{N}$ to $\mathbb{Z}$. Its proof requires a new signed-integer decomposition and additive-inverse argument to establish centrality of integer casts, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends centrality from natural coefficients to integer coefficients, requiring a new signed-integer decomposition and additive-inverse compatibility argument. This is substantive reasoning absent from the parent and is not the listed repeat.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the result from natural coefficients to integer coefficients. Its proof requires new reasoning for signed coefficients via decomposition into a difference of naturals and additive inverses, which the parent does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2894 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutes_with_int_polynomial.extracted_1.1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R)
  (h : a ^ 2 = 0) (p : ℤ[X]) :
  ((a * x + x * a) * p.sum fun n c => ↑c * a ^ n) = (p.sum fun n c => ↑c * a ^ n) * (a * x + x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, let a and x be elements of R such that $a^2 = 0$, and let p be a polynomial in $\mathbb{Z}[X]$. Then $(a x + x a)$ commutes with the element of R given by the finite sum over all degrees n of the coefficient of X^n in p , coerced from $\mathbb{Z}$ to R , multiplied by a^n ; that is, $(a x + x a) \cdot \sum_n (p.\text{coeff } X^n \text{ coerced to } R) a^n = (\sum_n (p.\text{coeff } X^n \text{ coerced to } R) a^n) \cdot (a x + x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutes_with_int_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℤ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
    p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a) := by
  let c : R := a * x + x * a
  have h_mul_sq : a * a = 0 := by
    simpa [pow_two] using h
  have h_base : a * c = c * a := by
    dsimp [c]
    calc
      a * (a * x + x * a) = (a * a) * x + (a * x) * a := by
        rw [mul_add, ← mul_assoc, ← mul_assoc]
      _ = (a * x) * a + x * (a * a) := by simp [h_mul_sq]
      _ = (a * x + x * a) * a := by rw [add_mul, ← mul_assoc]
  have h_power : ∀ k : ℕ, c * a ^ k = a ^ k * c := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      calc
        c * a ^ (Nat.succ k) = c * (a ^ k * a) := by rw [pow_succ]
        _ = (c * a ^ k) * a := by rw [mul_assoc]
        _ = (a ^ k * c) * a := by rw [ih]
        _ = a ^ k * (c * a) := by rw [mul_assoc]
        _ = a ^ k * (a * c) := by rw [h_base]
        _ = (a ^ k * a) * c := by rw [← mul_assoc]
        _ = a ^ (Nat.succ k) * c := by rw [pow_succ]
  have h_nat_coeff : ∀ n : ℕ, c * (n : R) = (n : R) * c := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      simp [Nat.cast_succ, add_mul, mul_add, ih]
  have h_signed : ∀ z : ℤ, ∃ m n : ℕ, z = (m : ℤ) - n := by
    intro z
    cases z with
    | ofNat n => exact ⟨n, 0, by simp⟩
    | negSucc n => exact ⟨0, n + 1, by omega⟩
  have h_int_coeff : ∀ z : ℤ, c * (z : R) = (z : R) * c := by
    intro z
    rcases h_signed z with ⟨m, n, rfl⟩
    calc
      c * ((m : ℤ) - n : ℤ : R) = c * ((m : R) - (n : R)) := by norm_num
      _ = c * (m : R) - c * (n : R) := by rw [mul_sub]
      _ = (m : R) * c - (n : R) * c := by rw [h_nat_coeff m, h_nat_coeff n]
      _ = ((m : R) - (n : R)) * c := by rw [sub_mul]
      _ = ((m : ℤ) - n : ℤ : R) * c := by norm_num
  have h_coeff : ∀ n : ℕ, c * ((p.coeff n : R) * a ^ n) =
    ((p.coeff n : R) * a ^ n) * c := by
    intro n
    calc
      c * ((p.coeff n : R) * a ^ n) = (c * (p.coeff n : R)) * a ^ n := by rw [mul_assoc]
```

```

_ = ((p.coeff n : R) * c) * a ^ n := by rw [h_int_coeff]
_ = (p.coeff n : R) * (c * a ^ n) := by rw [mul_assoc]
_ = (p.coeff n : R) * (a ^ n * c) := by rw [h_power n]
_ = (p.coeff n : R) * a ^ n * c := by rw [mul_assoc]
have h_eval : c * p.sum (fun n k => (k : R) * a ^ n) =
  p.support.sum (fun n => (p.coeff n : R) * a ^ n * c) := by
  change c * p.support.sum (fun n => (p.coeff n : R) * a ^ n) = _
  rw [Finset.mul_sum]
  apply Finset.sum_congr rfl
  intro n hn
  exact h_coeff n
calc
(a * x + x * a) * p.sum (fun n k => (k : R) * a ^ n) =
  c * p.sum (fun n k => (k : R) * a ^ n) := by rfl
_ = p.support.sum (fun n => (p.coeff n : R) * a ^ n * c) := h_eval
_ = p.sum (fun n k => (k : R) * a ^ n) * c := by
  change _ = p.support.sum (fun n => (p.coeff n : R) * a ^ n) * c
  rw [Finset.sum_mul]
_ = p.sum (fun n k => (k : R) * a ^ n) * (a * x + x * a) := by rfl

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh.mh__9b2b60d0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 9b2b60d0dc7fa9e9

4.315 315. anticommutator_commutates_with_ordered_product_of_power_sums

The problem

Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every ordered product whose factors are finite sums of natural powers of a .

Lean statement

```
theorem anticommutator_commutates_with_ordered_product_of_power_sums {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι κ : Type*} (t : List
  <- κ) (s : κ → Finset ι) (f : κ → ι → ℕ) :
  (a * x + x * a) * List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) =
    List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) * (a * x + x * a)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every finite sum of natural powers of a .

Parent in Lean

```
theorem anticommutator_commutates_with_finite_power_sum {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι : Type*} (s : Finset ι) (f : ι →
  <- ℕ) :
  (a * x + x * a) * s.sum (fun i => a ^ f i) = s.sum (fun i => a ^ f i) * (a * x + x * a)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes commutation from one finite power sum to an ordered product of arbitrarily many such sums. Its proof requires a new induction over the list of factors after establishing each factor commutes with the anticommutator, so the parent's proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single finite sum into an ordered product of independently indexed finite power sums. Proving commutation with the product requires a new induction over the list and a factor-commutation argument beyond the parent's sum induction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely expands the claim from one finite sum to an ordered product of many such sums. Its proof requires a new induction showing that a commuting element commutes with the entire product, beyond the parent's finite-sum argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2907 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutates_with_ordered_product_of_power_sums.extracted_1_1.{u_3, u_2, u_1} {R : Type u_1}
  [inst : Ring R] (a x : R) (h : a ^ 2 = 0) {ι : Type u_2} {κ : Type u_3} (t : List κ) (s : κ → Finset ι)
  (f : κ → ι → ℕ) :
  (a * x + x * a) * (List.map (fun k => ∑ i ∈ s k, a ^ f k i) t).prod =
  (List.map (fun k => ∑ i ∈ s k, a ^ f k i) t).prod * (a * x + x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, let a and x be elements of R satisfying $a^2 = 0$, let ι and κ be types, let t be a list of elements of κ , let s assign to each $k : \kappa$ a finite subset of ι , and let f assign to each $k : \kappa$ and $i : \iota$ a natural number. Then $(a x + x a)$ multiplied by the ordered product, in the order specified by t , of the finite sums $\sum_{i \in s(k)} a^{f(k,i)}$ equals that ordered product multiplied by $(a x + x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutates_with_ordered_product_of_power_sums {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι κ : Type*} (t : List
  κ) (s : κ → Finset ι) (f : κ → ι → ℕ) :
  (a * x + x * a) * List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) =
  List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) * (a * x + x * a) := by
  let c : R := a * x + x * a
  let g : κ → R := fun k => (s k).sum (fun i => a ^ f k i)
  have h_mul_sq : a * a = 0 := by
    simp [pow_two] using h
  have h_base : a * c = c * a := by
    dsimp [c]
    calc
      a * (a * x + x * a) = (a * a) * x + (a * x) * a := by
        rw [mul_add, ← mul_assoc, ← mul_assoc]
      _ = (a * x) * a + x * (a * a) := by
        simp [h_mul_sq]
      _ = (a * x + x * a) * a := by
        rw [add_mul, ← mul_assoc]
  have h_power : ∀ k : ℕ, c * a ^ k = a ^ k * c := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      calc
        c * a ^ (Nat.succ k) = c * (a ^ k * a) := by rw [pow_succ]
        _ = (c * a ^ k) * a := by rw [mul_assoc]
        _ = (a ^ k * c) * a := by rw [ih]
        _ = a ^ k * (c * a) := by rw [mul_assoc]
        _ = a ^ k * (a * c) := by rw [h_base]
        _ = (a ^ k * a) * c := by rw [← mul_assoc]
        _ = a ^ (Nat.succ k) * c := by rw [pow_succ]
  have h_sum : ∀ (k : κ) (u : Finset ι),
    c * u.sum (fun i => a ^ f k i) = u.sum (fun i => a ^ f k i) * c := by
    intro k u
    calc
      c * u.sum (fun i => a ^ f k i) = u.sum (fun i => c * a ^ f k i) := by
        rw [Finset.mul_sum]
      _ = u.sum (fun i => a ^ f k i * c) := by
        apply Finset.sum_congr rfl
        intro i hi
        exact h_power (f k i)
      _ = u.sum (fun i => a ^ f k i) * c := by
        rw [Finset.sum_mul]
  have h_factor : ∀ k : κ, c * g k = g k * c := by
    intro k
    dsimp [g]
    exact h_sum k (s k)
  have h_prod : ∀ l : List κ,
    c * List.prod (l.map g) = List.prod (l.map g) * c := by
```

```

intro l
induction l with
| nil => simp
| cons k l ih =>
  change c * (g k * List.prod (l.map g)) = (g k * List.prod (l.map g)) * c
  calc
    c * (g k * List.prod (l.map g)) = (c * g k) * List.prod (l.map g) := by rw [mul_assoc]
    _ = (g k * c) * List.prod (l.map g) := by rw [h_factor k]
    _ = g k * (c * List.prod (l.map g)) := by rw [mul_assoc]
    _ = g k * (List.prod (l.map g) * c) := by rw [ih]
    _ = (g k * List.prod (l.map g)) * c := by rw [← mul_assoc]
  simp [c, g] using h_prod t

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh__95b28844

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 95b2884480061167

4.316 316. nilpotent_symmetrized_square

The problem

If $a^2 = 0$ in a ring R , show that $(ax + xa)^2 = (ax)(ax) + (ax)(xa) + (xa)(xa)$.

Lean statement

```
theorem nilpotent_symmetrized_square {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a)
```

Parent. If $a^2 = 0$ in R , show that $ax + xa$ commutes with a .

Parent in Lean

```
theorem Herstein_exercise_4_2_6 {R : Type*} [Ring R] (a x : R)
  (h : a ^ 2 = 0) : a * (a * x + x * a) = (a * x + x * a) * a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from commutation of the symmetrized term with a to an explicit square expansion, requiring a new noncommutative expansion and proof that the cross term $(x*a)*(a*x)$ vanishes from $a^2 = 0$. The parent's proof does not supply this identity directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new noncommutative expansion and proves the distinct cross term $(x*a)*(a*x)$ vanishes from $a^2 = 0$. The parent only identifies two products involving $a*x + x*a$ and does not supply this square expansion or vanishing-cross-term argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from commutation of the symmetrized product to a square expansion and must additionally prove the distinct cross term $(x*a)*(a*x) = 0$ from $a^2 = 0$. This reasoning is not supplied by the parent's reduction to $a*x*a$, and the proof is complete.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1982 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nilpotent_symmetrized_square.extracted_1.1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = a * x * (a * x) + a * x * (x * a) + x * a * (x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, and let a and x be elements of R . If $a^2 = 0$, then $(a x + x a)^2 = a x (a x) + a x (x a) + x a (x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem nilpotent_symmetrized_square {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a) := by
  have hsq : a * a = 0 := by
    simpa [pow_two] using h
  have hcross : (x * a) * (a * x) = 0 := by
    calc
      (x * a) * (a * x) = x * (a * a) * x := by simp [mul_assoc]
      _ = 0 := by simp [hsq]
  have hexpand :
    (a * x + x * a) ^ 2 =
      (a * x) * (a * x) + (a * x) * (x * a) +
      (x * a) * (a * x) + (x * a) * (x * a) := by
    noncomm_ring
  calc
    (a * x + x * a) ^ 2 =
      (a * x) * (a * x) + (a * x) * (x * a) +
      (x * a) * (a * x) + (x * a) * (x * a) := hexpand
    _ = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a) := by
      rw [hcross]
      simp
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh__1651a8b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 981aaf31b2c9ae13

4.317 317. gaussian_square_dvd_iff_even_parts

The problem

A Gaussian integer is divisible by $(1+i)^2$ exactly when both its real and imaginary parts are even.

Lean statement

```
theorem gaussian_square_dvd_iff_even_parts (b : GaussianInt) : ((1, 1) : GaussianInt) ^ 2 ∣ b ↔ Even b.re ∧ Even b.im
```

Parent. For every Gaussian integer a , show that the quotient of $2a$ by $(1+i)^2$ exists uniquely and equals $a(-i)$.

Parent in Lean

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes an existential uniqueness result into a divisibility characterization for arbitrary Gaussian integers. Its proof requires decomposing Gaussian components and proving both parity directions, which is not supplied by the parent's factor-and-cancellation argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a scaled factorization with uniqueness to a divisibility characterization in terms of both Gaussian components being even. Its proof requires a bidirectional coordinate argument and does not reuse the parent's cancellation-based uniqueness reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence and uniqueness of a scaled factor to a divisibility characterization in terms of the real and imaginary parts. Its proof requires decomposing Gaussian integers and constructing an explicit quotient from parity witnesses, reasoning not supplied by the parent's cancellation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2784 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_square_dvd_iff_even_parts.extracted_1_1 (b : GaussianInt) : { re := 1, im := 1 } ^ 2 ∣ b ↔ Even b.re ∧ Even b.im
```

Read back by a model that saw only the Lean. For every Gaussian integer b , the square of the Gaussian integer with real part 1 and imaginary part 1 divides b if and only if the real part of b is even and the imaginary part of b is even.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_square_dvd_iff_even_parts (b : GaussianInt) :  $\langle 1, 1 \rangle : \text{GaussianInt}$  ^ 2 | b  $\leftrightarrow$  Even b.re  $\wedge$  Even b.im := by
  have hfactor :  $\langle 1, 1 \rangle : \text{GaussianInt}$  ^ 2 = (2 : GaussianInt) *  $\langle 0, 1 \rangle$  := by
    ext <|> norm_num [pow_two]
  constructor
  · intro h
    rcases h with  $\langle q, hq \rangle$ 
    rcases q with  $\langle x, y \rangle$ 
    constructor
    · refine  $\langle -y, ?_ \rangle$ 
      rw [hq, hfactor]
      norm_num
      ring
    · refine  $\langle x, ?_ \rangle$ 
      rw [hq, hfactor]
      norm_num
      ring
  · rintro  $\langle \langle r, hr \rangle, \langle s, hs \rangle \rangle$ 
    refine  $\langle \langle s, -r \rangle, ?_ \rangle$ 
    rw [hfactor]
    ext <|> norm_num [hr, hs] <|> ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh.mh__ed2761e4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ed2761e4731b3929

4.318 318. gaussian_factorization_unique_scaled

The problem

For every Gaussian integer a , show that the quotient of $2a$ by $(1 + i)^2$ exists uniquely and equals $a(-i)$.

Lean statement

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

Parent. Show that the quotient in the factorization of 2 by $(1 + i)^2$ exists, is unique, and equals $-i$.

Parent in Lean

```
theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) :  
  ↪ GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
```

What it contributes. Supplied the factorization identity $(1+i)^2(-i)=2$, which is scaled by the arbitrary Gaussian multiplier.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The arbitrary Gaussian integer parameter changes the uniqueness target from a fixed quotient to the law $q = a \cdot (0-i)$. The parent's coordinate-wise omega argument no longer supplies this result; the child requires scaling, commutativity, and cancellation by the nonzero Gaussian factor.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The arbitrary Gaussian integer parameter changes the uniqueness argument: coordinate-wise reasoning for the fixed quotient is replaced by proving the Gaussian factor is nonzero and cancelling it. Although the fixed factorization is reused, the parent proof does not establish this scaled law.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The arbitrary Gaussian integer parameter requires proving the scaled witness equation by distributivity and establishing cancellation from the nonzero Gaussian factor. This is substantive new reasoning beyond the parent's coordinate-wise uniqueness proof, so recalling the parent does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2762 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_factorization_unique_scaled.extracted_1_1 (a : GaussianInt) :  
  ∃ q,  
    { re := 1, im := 1 } ^ 2 * q = 2 * a ∧  
    ∀ (q₁ : GaussianInt), { re := 1, im := 1 } ^ 2 * q₁ = 2 * a → q₁ = a * { re := 0, im := -1 }
```

Read back by a model that saw only the Lean. For every Gaussian integer a , there exists a Gaussian integer q such that $(1 + i)^2 q = 2a$, and for every Gaussian integer q_1 , if $(1 + i)^2 q_1 = 2a$, then $q_1 = a(0 - i)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Real  
open scoped BigOperators  
  
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩ := by  
  have hfactor : ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by  
    ext <|> simp [pow_two]  
  have hne : ((1, 1) : GaussianInt) ^ 2 ≠ 0 := by  
    intro hz  
    have hzero : (0 : GaussianInt) = 2 := by  
      calc  
        (0 : GaussianInt) = ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ := by rw [hz]; simp  
        _ = 2 := hfactor  
    norm_num at hzero  
  have hscaled : ((1, 1) : GaussianInt) ^ 2 * (a * ⟨0, -1⟩) = 2 * a := by  
    calc  
      ((1, 1) : GaussianInt) ^ 2 * (a * ⟨0, -1⟩) = (((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩) * a := by ring  
      _ = 2 * a := by rw [hfactor]  
  refine ⟨a * ⟨0, -1⟩, hscaled, ?_⟩  
  intro q₁ hq  
  apply mul_left_cancel₀ hne  
  exact hq.trans hscaled.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh__7d67637c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7d67637cd579935f

4.319 319. gaussianInt_sq_one_add_i_dvd_imp_two_dvd

The problem

Show that every Gaussian integer divisible by $(1 + i)^2$ is divisible by 2.

Lean statement

```
theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd : ∀ z : GaussianInt, ((1, 1) : GaussianInt) ^ 2 | z → (2 : GaussianInt) | z
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem IrelandRosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 | 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine universal divisibility implication: every multiple of $(1+i)^2$ is shown to be a multiple of 2. This requires factoring $(1+i)^2$ as $2 \cdot i$ and composing arbitrary witnesses, beyond the parent's single witness for divisibility by 2.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely universal divisibility-transfer theorem: from $(1+i)^2 \mid z$ it factors $(1+i)^2$ as $2i$ and constructs a quotient witnessing $2 \mid z$. The parent only establishes divisibility for the single element 2 and supplies no argument for arbitrary z .

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed divisibility witness to an arbitrary Gaussian integer and proves that divisibility by $(1+i)^2$ implies divisibility by 2 via an explicit factorization and distributive rearrangement. The parent's single numerical witness does not supply this universal implication.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2000 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd.extracted_1_1 : ∀ (z : GaussianInt),
  { re := 1, im := 1 } ^ 2 | z → 2 | z
```

Read back by a model that saw only the Lean. For every Gaussian integer z , if $(1 + i)^2$ divides z , then 2 divides z .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Real
open scoped BigOperators

theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd :  $\forall z : \text{GaussianInt}, (\langle 1, 1 \rangle : \text{GaussianInt})^2 \mid z \rightarrow (2 : \text{GaussianInt}) \mid z$  := by
  have hfactor :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 = (2 : \text{GaussianInt}) * \langle 0, 1 \rangle$  := by
    ext <|> simp [pow_two]
  intro z hz
  rcases hz with ⟨q, hq⟩
  refine ⟨⟨0, 1⟩ : GaussianInt⟩ * q, ?_
  calc
    z =  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := hq
    _ =  $(2 : \text{GaussianInt}) * \langle 0, 1 \rangle * q$  := by rw [hfactor]
    _ =  $(2 : \text{GaussianInt}) * (\langle 0, 1 \rangle * q)$  := by ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__mh__13b3d12d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 131d0641f6190be8

4.320 320. gaussianInt_sq_div_two_unique

The problem

Show that there is a unique Gaussian integer q such that $(1 + i)^2 q = 2$.

Lean statement

```
theorem gaussianInt_sq_div_two_unique : ∃! q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem IrelandRosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 | 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes divisibility into an existence-and-uniqueness statement. Beyond reusing the parent's quotient for existence, it must prove the squared Gaussian integer is nonzero and use cancellation to establish uniqueness.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility into an existence-and-uniqueness claim. Although the parent proof supplies the witness, the child additionally proves the squared Gaussian integer is nonzero and uses cancellation to establish uniqueness, which the parent does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a quotient into a genuine uniqueness theorem. Proving uniqueness requires establishing that $(1+i)^2$ is nonzero and applying cancellation, which is reasoning absent from the parent's coordinate computation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2010 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussianInt_sq_div_two_unique.extracted_1_1 : ∃! q, { re := 1, im := 1 } ^ 2 * q = 2
```

Read back by a model that saw only the Lean. *There exists a unique Gaussian integer q such that $(1 + i)^2 q = 2$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Real
open scoped BigOperators

theorem gaussianInt_sq_div_two_unique :  $\exists! q : \text{GaussianInt}, (\langle 1, 1 \rangle : \text{GaussianInt})^2 * q = 2$  := by
  let q : GaussianInt :=  $\langle 0, -1 \rangle$ 
  have h_parent :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q = 2$  := by
    dsimp [q]
    ext <=> simp [pow_two]
  have h_nonzero :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 \neq (0 : \text{GaussianInt})$  := by
    intro h
    norm_num [pow_two] at h
    have hr := congrArg (fun x : GaussianInt => x.re) h
    norm_num at hr
  refine ⟨q, h_parent, ?_⟩
  intro y hy
  have h_eq :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * y = (\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := by
    calc
       $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * y = 2$  := hy
      _ =  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := h_parent.symm
  exact mul_left_cancel₀ h_nonzero h_eq
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__mh__d5878a1f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8b5ca32c4cbd8a0d

4.321 321. bounded_family_union_eq_union_interiors

The problem

Let X be a topological space, let $n \in \mathbb{N}$, and let $(A_i)_{i < n}$ be a family of subsets of X . Suppose that for every $i < n$ and every $x \in A_i$, there is an open set U with $x \in U \subseteq A_i$. Prove that $\bigcup_{i < n} \text{int}(A_i) = \bigcup_{i < n} A_i$.

Lean statement

```
theorem bounded_family_union_eq_union_interiors (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  (⋃ i : Fin n, interior (A i)) = ⋃ i : Fin n, A i
```

Parent. A bounded indexed family of locally open sets has an open union.

Parent in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  IsOpen (⋃ i : Fin n, A i)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from proving openness of the union to characterizing it as a union of interiors. Its reverse inclusion requires applying the local neighborhood hypothesis to each member and using interior maximality, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving openness to characterizing the union as the union of interiors. Its reverse inclusion requires the new interior-maximality argument, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to an equality characterizing the union via interiors. Its reverse inclusion requires proving each locally open neighborhood lies inside interior (using `interior_maximal`), reasoning not supplied by the parent's union-openness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1970 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_family_union_eq_union_interiors.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X) (h : ∀ (i : Fin n), ∀ x ∈ A i, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  U i, interior (A i) = U i, A i
```

Read back by a model that saw only the Lean. *Let X be a topological space, let n be a natural number, and let A be a family of subsets of X indexed by the finite type $\text{Fin } n$. Suppose that for every index $i \in \text{Fin } n$ and every point $x \in A i$, there exists a set U such that $x \in U$, U is open in X , and $U \subseteq A i$. Then the union over all $i \in \text{Fin } n$ of the interiors of $A i$ is equal to the union over all $i \in \text{Fin } n$ of $A i$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_family_union_eq_union_interiors (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  (U i : Fin n, interior (A i)) = U i : Fin n, A i := by
  ext x
  constructor
  · intro hx
    rcases Set.mem_iUnion.mp hx with ⟨i, hxi⟩
    exact Set.mem_iUnion.mpr ⟨i, interior_subset hxi⟩
  · intro hx
    rcases Set.mem_iUnion.mp hx with ⟨i, hxi⟩
    rcases h i x hxi with ⟨U, hxU, hUopen, hUsub⟩
    have hUint : U ⊆ interior (A i) := interior_maximal hUsub hUopen
    have hxinterior : x ∈ interior (A i) := hUint hxU
    exact Set.mem_iUnion.mpr ⟨i, hxinterior⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__me.mh__0e47166e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d44f23eb7bec6433

4.322 322. local_open_neighborhoods_eq_sUnion

The problem

Let X be a topological space and let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is the union of all open subsets of X contained in A .

Lean statement

```
theorem local_open_neighborhoods_eq_sUnion (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  A = ⋃₀ {U : Set X | IsOpen U ∧ U ⊆ A}
```

Parent. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from openness to an exact characterization of A as the union of all open subsets contained in A . Its proof requires proving both inclusions and does not merely replay the parent's openness criterion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to an equality characterizing A as the union of all open subsets contained in A . Its proof requires a new two-sided set-inclusion argument using $sUnion$ membership, rather than applying the parent's openness criterion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the openness conclusion into an extensional characterization of A as the union of all open subsets contained in it. Its proof requires new $sUnion$ membership and subset arguments in both directions, which are not supplied by the parent's criterion proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1974 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem local_open_neighborhoods_eq_sUnion.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (A : Set X)
(h1 : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) : A = ⋃₀ {U | IsOpen U ∧ U ⊆ A}
```

Read back by a model that saw only the Lean. *Let X be a topological space and let A be a subset of X . Assume that for every $x \in A$, there exists a set U such that $x \in U$, U is open, and $U \subseteq A$. Then A equals the union of all open subsets U of X satisfying $U \subseteq A$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem local_open_neighborhoods_eq_sUnion (X : Type*) [TopologicalSpace X] (A : Set X)
(h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
A = ⋃₀ {U : Set X | IsOpen U ∧ U ⊆ A} := by
  apply Set.Subset.antisymm
  · intro x hx
    rcases h1 x hx with ⟨U, hxU, hU, hUA⟩
    exact Set.mem_sUnion.2 ⟨U, ⟨hU, hUA⟩, hxU⟩
  · intro x hx
    rcases Set.mem_sUnion.1 hx with ⟨U, hU, hxU⟩
    exact hU.2 hxU
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__mh__4cf2fc25

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 198b5089cea48d8c

4.323 323. Munkres_exercise_13_1_mutation

The problem

Let X be a topological space and $A \subseteq X$. Suppose that every point of A lies in an open subset of A . Show that A is the union of a family of open subsets, each containing the point of A indexing it.

Lean statement

```
theorem Munkres_exercise_13_1_mutation (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U : A → Set X,
    (∀ x : A, IsOpen (U x) ∧ (x : X) ∈ U x ∧ U x ⊆ A) ∧
    (U x : A, U x) = A
```

Parent. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine constructive covering theorem: it must choose an open neighborhood for every subtype point and prove that their indexed union is exactly A . The parent's openness criterion does not supply the choice function or the two-sided union equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the openness criterion by constructing a point-indexed family of open neighborhoods and proving their union is exactly A . This requires choice over the subtype and a separate set-extensionality argument for both inclusions, which the parent's criterion application does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine structure expansion: it must choose a neighborhood for every subtype point and prove that their union is exactly A . The parent only establishes openness via the local criterion and does not supply the choice-function or union-equality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2875 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Munkres_exercise_13_1_mutation.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U, (∀ (x : ↑A), IsOpen (U x) ∧ ↑x ∈ U x ∧ U x ⊆ A) ∧ U x, U x = A
```

Read back by a model that saw only the Lean. Let X be a topological space and let A be a subset of X . Suppose that for every $x \in A$, there exists an open set V such that $x \in V$ and $V \subseteq A$. Then there exists a function U assigning to each point x of the subtype A an open subset $U(x)$ of X such that $x \in U(x)$, $U(x) \subseteq A$, and the union of all the sets $U(x)$, as x ranges over A , is exactly A .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem Munkres_exercise_13_1_mutation (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U : A → Set X,
    (∀ x : A, IsOpen (U x) ∧ (x : X) ∈ U x ∧ U x ⊆ A) ∧
    (U x : A, U x) = A := by
  classical
  choose U hmem hopen hsub using fun x : A => h1 x.1 x.2
  refine ⟨U, ?_, ?_⟩
  · intro x
    exact ⟨hopen x, hmem x, hsub x⟩
  · ext y
    constructor
    · intro hy
      rcases Set.mem_iUnion.mp hy with ⟨x, hx⟩
      exact hsub x hx
    · intro hy
      exact Set.mem_iUnion.mpr ⟨⟨y, hy⟩, hmem ⟨y, hy⟩⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__mh__bde9eb06

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-
gerprint bde9eb067942729d

4.324 324. iInf_topology_eq_of_surjective

The problem

For any family of topologies on a space and any surjective reindexing map, the universal topology below every topology in the family equals the universal topology below every reindexed topology.

Lean statement

```
theorem iInf_topology_eq_of_surjective (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :
  (⊓ i, T i) = ⊓ j, T (f j)
```

Parent. Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X , and let $f : J \rightarrow I$. Show that every topology on X contained in each $\mathcal{T}_i$ is contained in the largest topology contained in all the $\mathcal{T}_i$, and that this largest topology is contained in the largest topology contained in all the $\mathcal{T}_{f(j)}$.

Parent in Lean

```
theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⊓ i, T i) ∧
  (⊓ i, T i) ≤ ⊓ j, T (f j) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the comparison to an equality under surjectivity, requiring a genuinely new reverse inequality by selecting a preimage for each index. The parent supplies only the forward comparison and does not provide this surjectivity-based argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the comparison to equality under surjectivity and proves a genuinely new reverse inequality by selecting a preimage for each index via hf . The parent's proof supplies only the forward inequality and cannot establish this direction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's one-sided comparison to equality under surjectivity, requiring a genuinely new reverse inequality and an index-selection argument via hf . This cannot be obtained by recalling the parent proof, which supplies only the forward direction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1874 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem iInf_topology_eq_of_surjective.extracted_1_1.{u_3, u_2, u_1} (X : Type u_1) (I : Type u_2) (J : Type u_3)
  (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :  $\prod i, T\ i = \prod j, T\ (f\ j)$ 
```

Read back by a model that saw only the Lean. *Let X be a type, let I and J be types, let T assign a topological space on X to each element of I , and let $f : J \rightarrow I$ be surjective. Then the infimum of the topological spaces $T\ i$ over all $i \in I$ equals the infimum of the topological spaces $T\ (f\ j)$ over all $j \in J$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem iInf_topology_eq_of_surjective (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :
  ( $\prod i, T\ i$ ) =  $\prod j, T\ (f\ j)$  := by
  have h_parent_direction : ( $\prod i, T\ i$ ) ≤  $\prod j, T\ (f\ j)$  := by
    refine le_iInf ?_
    intro j
    exact iInf_le T (f j)
  have h_selected_lower :  $\forall j : J, (\prod j, T\ (f\ j)) \leq T\ (f\ j)$  := by
    intro j
    exact iInf_le (fun j : J => T (f j)) j
  have h_reverse_direction : ( $\prod j, T\ (f\ j)$ ) ≤  $\prod i, T\ i$  := by
    refine le_iInf ?_
    intro i
    obtain ⟨j, rfl⟩ := hf i
    exact h_selected_lower j
  exact le_antisymm h_parent_direction h_reverse_direction
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__me.mh__630e48d1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9f20fa4ac1aba7ea

4.325 325. smallest_topology_containing_family

The problem

Let $\{\mathcal{T}_\alpha\}$ be a family of topologies on X . Show that there is a unique smallest topology on X containing every $\mathcal{T}_\alpha$.

Lean statement

```
theorem smallest_topology_containing_family (X I : Type*) (τ : I → TopologicalSpace X) :
  ∃! τ' : TopologicalSpace X, (∀ i, τ i ≤ τ') ∧
  ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → τ' ≤ σ
```

Parent. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from an intersection/ greatest lower topology to the least upper topology containing the family. Its proof uses the complete-lattice supremum and a new antisymmetry uniqueness argument, none of which is supplied by the parent's intersection construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the construction from the topology of sets common to every family member (an infimum) to the smallest topology containing them (a supremum). Its proof requires lattice supremum and uniqueness reasoning absent from the parent's intersection-based argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from the greatest topology contained in every member of a family to the least topology containing every member. Its proof requires the complete-lattice supremum universal property, which is absent from the parent's elementwise intersection/topology-axiom argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2226 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem smallest_topology_containing_family.extracted_1_1.{u_2, u_1} (X : Type u_1) (I : Type u_2)
  (τ : I → TopologicalSpace X) :
  ∃! τ', (∀ (i : I), τ i ≤ τ') ∧ ∀ (σ : TopologicalSpace X), (∀ (i : I), τ i ≤ σ) → τ' ≤ σ
```

Read back by a model that saw only the Lean. *For every type X , every type I , and every family of topological spaces τ indexed by I on X , there exists a unique topological space structure τ' on X such that, for every $i \in I$, $\tau i \leq \tau'$, and for every topological space σ on X , if $\tau i \leq \sigma$ for every $i \in I$, then $\tau' \leq \sigma$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem smallest_topology_containing_family (X I : Type*) (τ : I → TopologicalSpace X) :
  ∃! τ' : TopologicalSpace X, (∀ i, τ i ≤ τ') ∧
    ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → τ' ≤ σ := by
  have h_upper : ∀ i, τ i ≤ ⋃ j, τ j := by
    intro i
    exact le_iSup (fun j => τ j) i
  have h_least : ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → (⋃ i, τ i) ≤ σ := by
    intro σ hσ
    exact iSup_le hσ
  refine ⟨⋃ i, τ i, ⟨h_upper, h_least⟩, ?_⟩
  intro σ hσ
  apply le_antisymm
  · exact hσ.2 _ h_upper
  · exact h_least σ hσ.1
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__mh__c587ebb9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c587ebb98b02f88d

4.326 326. isOpen_prod_iff_of_nonempty

The problem

Let U and V be nonempty subsets of topological spaces X and Y . Show that $U \times V$ is open in $X \times Y$ if and only if U and V are both open.

Lean statement

```
theorem isOpen_prod_iff_of_nonempty {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V
```

Parent. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from quotient-map properties of projections to an iff characterization of openness for nonempty product sets. Its reverse implication is standard, but the forward direction requires new image computations and uses nonemptiness essentially, so the parent's proof does not supply the child's argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from quotient-map properties of projections to an equivalence characterizing openness of a product set. Its reverse direction and image computations using nonempty fibers require mathematics not supplied by the parent's quotient-map proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from quotient-map existence to a characterization of openness of product sets. Its proof requires constructing projection images, using nonemptiness to identify those images with U and V , and proving both directions; the parent's quotient-map proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpen_prod_iff_of_nonempty.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V
```

Read back by a model that saw only the Lean. *For topological spaces X and Y , and subsets U of X and V of Y , if U and V are both nonempty, then the product set $U \times V$ is open in the product topology if and only if U is open in X and V is open in Y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem isOpen_prod_iff_of_nonempty {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
(U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V := by
  constructor
  · intro hprod
    have hopen1 : IsOpenMap (Prod.fst : X × Y → X) := isOpenMap_fst
    have hopen2 : IsOpenMap (Prod.snd : X × Y → Y) := isOpenMap_snd
    have hUimg : IsOpen (Prod.fst '' (U ×s V)) := hopen1 _ hprod
    have hVimg : IsOpen (Prod.snd '' (U ×s V)) := hopen2 _ hprod
    have hfst_image : Prod.fst '' (U ×s V) = U := by
      ext x
      constructor
      · rintro ⟨p, hp, rfl⟩
        exact hp.1
      · intro hx
        obtain ⟨y, hy⟩ := hV
        exact ⟨(x, y), ⟨hx, hy⟩, rfl⟩
    have hsnd_image : Prod.snd '' (U ×s V) = V := by
      ext y
      constructor
      · rintro ⟨p, hp, rfl⟩
        exact hp.2
      · intro hy
        obtain ⟨x, hx⟩ := hU
        exact ⟨(x, y), ⟨hx, hy⟩, rfl⟩
    have hUopen : IsOpen U := by
      simpa [hfst_image] using hUimg
    have hVopen : IsOpen V := by
      simpa [hsnd_image] using hVimg
    exact ⟨hUopen, hVopen⟩
  · rintro ⟨hUopen, hVopen⟩
    exact hUopen.prod hVopen
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me.mh__041f8e45

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 041f8e451da480db

4.327 327. continuous_iff_continuous_comp_fst

The problem

Let X , Y , and Z be topological spaces with Y nonempty. A map $g : X \rightarrow Z$ is continuous if and only if $(x, y) \mapsto g(x)$ is continuous on $X \times Y$.

Lean statement

```
theorem continuous_iff_continuous_comp_fst {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [Nonempty Y]
  (g : X → Z) :
  Continuous g ↔ Continuous (fun p : X × Y => g p.1)
```

Parent. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from establishing quotient maps to an equivalence of continuity, requiring a new converse argument via the coinducing/quotient-map property of the first projection. The parent's construction of quotient maps does not itself supply this continuity characterization.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from establishing quotient maps to an equivalence of continuity under composition with the first projection. Its reverse direction requires a coinducing/quotient-map argument and continuity characterization not supplied by the parent's proof, so recalling the parent does not substantially solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from proving quotient-map properties to a continuity reflection equivalence. Its reverse direction requires using the quotient/coinducing property of the first projection to recover continuity of g , which is not supplied by merely recalling the parent's conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2829 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_iff_continuous_comp_fst.extracted_1.1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] [inst_3 : Nonempty Y]
  (g : X → Z) : Continuous g ↔ Continuous fun p => g p.1
```

Read back by a model that saw only the Lean. *Let X , Y , and Z be topological spaces, with Y nonempty, and let $g : X \rightarrow Z$. Then g is continuous if and only if the function from $X \times Y$ to Z defined by $(x, y) \mapsto g(x)$ is continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_iff_continuous_comp_fst {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [Nonempty Y]
  (g : X → Z) :
  Continuous g ↔ Continuous (fun p : X × Y => g p.1) := by
  constructor
  · intro hg
    simpa [Function.comp] using hg.comp (continuous_fst : Continuous (Prod.fst : X × Y → X))
  · intro hg
    have hq : Topology.IsQuotientMap (Prod.fst : X × Y → X) := by
      exact isQuotientMap_fst
    have hcoind : Topology.IsCoinducing (Prod.fst : X × Y → X) := by
      exact (Topology.isQuotientMap_iff (Prod.fst : X × Y → X)).mp hq |>.1
    have hiff : ∀ s : Set X, IsOpen ((Prod.fst : X × Y → X) ⁻¹' s) ↔ IsOpen s := by
      exact Topology.isCoinducing_iff.mp hcoind
    rw [continuous_def]
    intro s hs
    apply (hiff (g ⁻¹' s)).mp
    have hp : IsOpen ((fun p : X × Y => g p.1) ⁻¹' s) :=
      (continuous_def.mp hg) s hs
    simpa using hp
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me.mh__eb4b00a0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint eb4b00a0491487ec

4.328 328. isOpenMap_prodMap_general

The problem

Let $f : X \rightarrow Y$ and $g : Z \rightarrow W$ be open maps between topological spaces. Show that the map $X \times Z \rightarrow Y \times W$ given by $(x, z) \mapsto (f(x), g(z))$ is open.

Lean statement

```
theorem isOpenMap_prodMap_general {X Y Z W : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [TopologicalSpace W]
  (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) : IsOpenMap (Prod.map f g)
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from the two coordinate projections to arbitrary open maps and proves openness of their product map, requiring the library composition theorem ‘hf.prodMap hg’. The parent’s projection-specific conjunction proof does not supply this generalized argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the special-case projection result with a genuinely general theorem: arbitrary open maps f and g induce an open product map. Its proof requires the library product-map preservation lemma and cannot be obtained from the parent’s projection rewrites alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the two canonical projections to arbitrary open maps and requires the product-map openness lemma, which the parent’s direct projection proof does not supply. Its proof obligation is not derivable by merely recalling or applying the parent’s conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1969 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpenMap_prodMap_general.extracted_1_1.{u_4, u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  {W : Type u_4} [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z]
  [inst_3 : TopologicalSpace W] (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) :
  IsOpenMap (Prod.map f g)
```

Read back by a model that saw only the Lean. *Let X, Y, Z , and W be topological spaces. If $f : X \rightarrow Y$ and $g : Z \rightarrow W$ are open maps, then the product map $\text{Prod.map } f g : X \times Z \rightarrow Y \times W$, defined by $(x, z) \mapsto (f(x), g(z))$, is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem isOpenMap_prodMap_general {X Y Z W : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [TopologicalSpace W]
  (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) : IsOpenMap (Prod.map f g) := by
  have hprod : IsOpenMap (Prod.map f g) := hf.prodMap hg
  exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__198d2479

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7c4726cf882d08b5

4.329 329. joint_projection_isOpenMap

The problem

Let X and Y be topological spaces, and let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be the coordinate projections. Show that the map $X \times Y \rightarrow X \times Y$ sending (x, y) to $(\pi_1(x, y), \pi_2(x, y))$ is an open map.

Lean statement

```
theorem joint_projection_isOpenMap {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π₁ p, π₂ p))
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target from openness of two projections to openness of their jointly paired map, proving that map equals the identity and invoking identity-map openness. The parent's two projection-openness lemmas do not supply this identity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from two separate open-map facts to openness of the joint map, requiring the new observation that pairing the two projections is the identity and then applying `IsOpenMap.id`. The parent's constructor-based projection proof does not supply this identity characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conjunction of separate projection claims into openness of the joint map. Its proof requires establishing that the paired projections equal the identity and invoking openness of the identity, reasoning absent from the parent's separate-library-lemma proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1945 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem joint_projection_isOpenMap.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (π1 : X × Y → X) (π2 : X × Y → Y) (h1 : π1 = Prod.fst) (h2 : π2 = Prod.snd) :
IsOpenMap fun p => (π1 p, π2 p)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, and let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then the function from $X \times Y$ to $X \times Y$ that sends each p to $(\pi_1(p), \pi_2(p))$ is an open map, meaning it maps every open subset of $X \times Y$ to an open subset of $X \times Y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem joint_projection_isOpenMap {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
(π1 : X × Y → X)
(π2 : X × Y → Y)
(h1 : π1 = Prod.fst)
(h2 : π2 = Prod.snd) :
IsOpenMap (fun p : X × Y => (π1 p, π2 p)) := by
have hjoint : (fun p : X × Y => (π1 p, π2 p)) = id := by
  funext p
  simp [h1, h2]
rw [hjoint]
exact IsOpenMap.id
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__226822b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 20f72843a49667ce

4.330 330. coordinate_images_and_local_rectangles

The problem

Let X and Y be topological spaces, and let the maps from $X \times Y$ to X and Y be the coordinate projections. Show that for every open subset U of $X \times Y$, both coordinate images of U are open, and every point of U lies in an open rectangle contained in U .

Lean statement

```
theorem coordinate_images_and_local_rectangles {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  ∀ U : Set (X × Y), IsOpen U →
    IsOpen (π₁ '' U) ∧ IsOpen (π₂ '' U) ∧
      ∀ a b, (a, b) ∈ U →
        ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds genuinely new mathematics: it characterizes every open subset locally by product rectangles, in addition to proving openness of both projection images. The parent's projection proof alone cannot supply this local rectangle obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the obligation by characterizing open product neighborhoods at every point, in addition to proving openness of both coordinate images. The local-rectangle existence requires reasoning absent from the parent's projection-openness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the obligation by adding the local product-neighborhood characterization: every point of an open set must lie in an open rectangle contained in that set. Although the projection-open-map arguments reuse the parent's lemmas, the new rectangle existence requires independent topological reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2316 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coordinate_images_and_local_rectangles.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (π₁ : X × Y → X) (π₂ : X × Y → Y),
  π₁ = Prod.fst →
  π₂ = Prod.snd →
  ∀ (U : Set (X × Y)),
  IsOpen U →
  IsOpen (π₁ '' U) ∧
  IsOpen (π₂ '' U) ∧
  ∀ (a : X) (b : Y), (a, b) ∈ U → ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U
```

Read back by a model that saw only the Lean. For any topological spaces X and Y , and any functions $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$, if π_1 is the first projection and π_2 is the second projection, then for every open set $U \subseteq X \times Y$, both images $\pi_1(U) \subseteq X$ and $\pi_2(U) \subseteq Y$ are open. Moreover, for every $a \in X$ and $b \in Y$ with $(a, b) \in U$, there exist open sets $u \subseteq X$ and $v \subseteq Y$ such that $a \in u$, $b \in v$, and $u \times v \subseteq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem coordinate_images_and_local_rectangles {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  ∀ U : Set (X × Y), IsOpen U →
  IsOpen (π₁ '' U) ∧ IsOpen (π₂ '' U) ∧
  ∀ a b, (a, b) ∈ U →
  ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U := by
  have hfst : IsOpenMap π₁ := by
    simpa [h₁] using (isOpenMap_fst : IsOpenMap (Prod.fst : X × Y → X))
  have hsnd : IsOpenMap π₂ := by
    simpa [h₂] using (isOpenMap_snd : IsOpenMap (Prod.snd : X × Y → Y))
  intro U hU
  have hlocal : ∀ a b, (a, b) ∈ U →
    ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U := by
    intro a b hab
    exact isOpen_prod_iff.mp hU a b hab
  exact ⟨hfst U hU, hsnd U hU, hlocal⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__51a02d05

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 51a02d05f8b603f8

4.331 331. combined_projection_is_open_bijective

The problem

Suppose two maps from $X \times Y$ are the coordinate projections. Show that the combined map $(x, y) \mapsto (\pi_1(x, y), \pi_2(x, y))$ is an open bijection of $X \times Y$ onto itself.

Lean statement

```
theorem combined_projection_is_open_bijective {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π₁ p, π₂ p)) ∧
  Function.Bijective (fun p : X × Y => (π₁ p, π₂ p))
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from openness of each coordinate projection to openness and bijectivity of the combined map. Its proof must identify the combined map with the identity and establish bijectivity via product extensionality, none of which is supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from separate openness of the coordinate projections to openness and bijectivity of their combined map. Proving that this map is the identity and establishing bijectivity requires reasoning not supplied by the parent's two projection lemmas.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to the openness and bijectivity of the combined projection map. Proving that this map is the identity and hence bijective requires reasoning absent from the parent's separate openness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem combined_projection_is_open_bijective.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (π1 : X × Y → X) (π2 : X × Y → Y) (h1 : π1 = Prod.fst)
  (h2 : π2 = Prod.snd) : (IsOpenMap fun p => (π1 p, π2 p)) ∧ Function.Bijective fun p => (π1 p, π2 p)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then the map sending each $p \in X \times Y$ to $(\pi_1(p), \pi_2(p))$ is an open map and is bijective.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem combined_projection_is_open_bijective {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π1 : X × Y → X)
  (π2 : X × Y → Y)
  (h1 : π1 = Prod.fst)
  (h2 : π2 = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π1 p, π2 p)) ∧
  Function.Bijective (fun p : X × Y => (π1 p, π2 p)) := by
  have hmap : (fun p : X × Y => (π1 p, π2 p)) = id := by
    funext p
    rw [h1, h2]
    rfl
  have hopen : IsOpenMap (fun p : X × Y => (π1 p, π2 p)) := by
    rw [hmap]
    exact IsOpenMap.id
  have hbij : Function.Bijective (fun p : X × Y => (π1 p, π2 p)) := by
    constructor
    · intro a b hab
      apply Prod.ext
      · have h := congrArg Prod.fst hab
        simpa [h1] using h
      · have h := congrArg Prod.snd hab
        simpa [h2] using h
    · intro p
      refine ⟨p, ?_⟩
      simp [h1, h2]
  exact ⟨hopen, hbij⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__9ac34659

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 9ac346591f88514b

4.332 332. simultaneous_comparison_closed_and_closure_contains

The problem

Let Y be a linearly ordered topological space, let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a finite nonempty set, and let $h : X \rightarrow X$ be continuous. If $f_i(x) \leq g_i(h(x))$ on a dense set D , show that the set of points satisfying all these inequalities is closed and contains the closure of D .

Lean statement

```
theorem simultaneous_comparison_closed_and_closure_contains {ι X Y : Type*} [Fintype ι] [Nonempty ι]
[TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
(hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X)
(hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} ∧
closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)}
```

Parent. Let Y be a linearly ordered topological space, let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a finite nonempty set, and let $h : X \rightarrow X$ be continuous. If $f_i(x) \leq g_i(h(x))$ for every x in a dense subset of X and every i , show that this inequality holds for all $x \in X$ and every i .

Parent in Lean

```
theorem dense_coupled_simultaneous_comparison {ι X Y : Type*} [Fintype ι] [Nonempty ι]
[TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
(hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X) (hD : Dense D)
(hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
∀ x, ∀ i, f i x ≤ g i (h x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the proof obligation from pointwise global comparison to proving that the simultaneous feasible set is closed and contains the closure of D . This requires reasoning about arbitrary intersections of closed sets via `isClosed_iInter`, which the parent's pointwise density argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a closedness-and-closure characterization, requiring construction of the finite intersection of coordinatewise inequality sets and proving its closedness. It also removes the density hypothesis, so the result is not recoverable by simply recalling the parent's pointwise extension proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a pointwise global inequality to a conjunction of two structural claims: closedness of the simultaneous feasible set and containment of the closure of D . Proving closedness requires identifying the set as an indexed intersection and applying a new lattice/topological argument, so the parent's proof is not sufficient by recall alone.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2973 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem simultaneous_comparison_closed_and_closure_contains.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2}
{Y : Type u_3} [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y]
[inst_3 : LinearOrder Y] [inst_4 : OrderTopology Y] (f g : ι → X → Y) (h : X → X) (hf : ∀ (i : ι), Continuous (f i))
(hg : ∀ (i : ι), Continuous (g i)) (hh : Continuous h) (D : Set X) (hD : Dense D)
(hineq : ∀ x ∈ D, ∀ (i : ι), f i x ≤ g i (h x)) :
IsClosed {x | ∀ (i : ι), f i x ≤ g i (h x)} ∧ closure D ⊆ {x | ∀ (i : ι), f i x ≤ g i (h x)}
```

Read back by a model that saw only the Lean. Let ι , X , and Y be types, with ι nonempty; let X and Y be topological spaces, and let Y be a linearly ordered topological space whose topology is the order topology. Let f and g be functions from ι to functions $X \rightarrow Y$, let $h : X \rightarrow X$, and let $D \subseteq X$. Assume that for every $i \in \iota$, $f i$ is continuous; for every $i \in \iota$, $g i$ is continuous; h is continuous; D is dense in X ; and for every $x \in D$ and every $i \in \iota$, $f i x \leq g i (h x)$. Then the set of all $x \in X$ such that, for every $i \in \iota$, $f i x \leq g i (h x)$ is closed, and the closure of D is contained in this set.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem simultaneous_comparison_closed_and_closure_contains {ι X Y : Type*} [Fintype ι] [Nonempty ι]
[TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
(hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X)
(hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} ∧
closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
have hclosed_i : ∀ i, IsClosed {x : X | f i x ≤ g i (h x)} := by
  intro i
  exact isClosed_le (hf i) ((hg i).comp hh)
have hset : {x : X | ∀ i, f i x ≤ g i (h x)} =
  ⋂ i, {x : X | f i x ≤ g i (h x)} := by
  ext x
  simp
have hclosed : IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} := by
  rw [hset]
  exact isClosed_iInter hclosed_i
have hsubset : D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
  intro x hx i
  exact hineq x hx i
have hclosure : closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
  exact closure_minimal hsubset hclosed
exact ⟨hclosed, hclosure⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.me.mh__1a717344

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7d7aed89fd94f6f3

4.333 333. finite_bounded_infeasible_locus_open

The problem

Let X be a topological space and Y a linearly ordered topological space. If finitely many lower and upper bounds, together with a comparison function, are continuous, show that the set of points violating at least one bound is open.

Lean statement

```
theorem finite_bounded_infeasible_locus_open {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x}
```

Parent. Let X be a topological space and Y a linearly ordered topological space. If finitely many lower and upper bounds, together with a comparison function, are continuous, show that the set of points satisfying every lower bound and every upper bound is closed.

Parent in Lean

```
theorem finite_bounded_feasible_locus_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsClosed {x | ∀ i, l i x ≤ h x ∧ h x ≤ u i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the obligation from a closed universal intersection of weak inequalities to an open existential union of strict violations. The parent's 'isClosed_le'/intersection argument cannot be directly applied; the child requires the distinct open-set lemmas and union construction, even though the overall topological pattern is analogous.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from closedness of a finite intersection of non-strict feasibility constraints to openness of a finite union of strict infeasibility constraints. Its proof requires strict-order openness lemmas, unions, and existential reformulation, so the parent's proof does not supply it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a closed finite intersection of non-strict feasibility constraints to an open finite union of strict infeasibility constraints. It requires dual topological reasoning with strict inequalities, unions, and existential indexing; the parent's closed-set proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2947 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_bounded_infeasible_locus_open.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2} {Y : Type u_3}
  [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (l u : ι → X → Y) (h : X → Y) (hl : ∀ (i : ι), Continuous (l i))
  (hu : ∀ (i : ι), Continuous (u i)) (hh : Continuous h) : IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x}
```

Read back by a model that saw only the Lean. *Let ι be a nonempty type, let X and Y be topological spaces, and suppose Y is linearly ordered with its order topology. Let $l, u : \iota \rightarrow X \rightarrow Y$ and $h : X \rightarrow Y$. Assume that for every $i \in \iota$, the functions $x \mapsto l(i, x)$ and $x \mapsto u(i, x)$ are continuous, and that h is continuous. Then the set of all $x \in X$ for which there exists $i \in \iota$ such that $h(x) < l(i, x)$ or $u(i, x) < h(x)$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_bounded_infeasible_locus_open {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x} := by
  have hlocal : ∀ i : ι, IsOpen {x : X | h x < l i x ∨ u i x < h x} := by
    intro i
    exact (isOpen_lt hh (hl i)).union (isOpen_lt (hu i) hh)
  have hopen : IsOpen (U i : ι, {x : X | h x < l i x ∨ u i x < h x}) :=
    isOpen_iUnion hlocal
  have hset : (U i : ι, {x : X | h x < l i x ∨ u i x < h x}) =
    {x | ∃ i, h x < l i x ∨ u i x < h x} := by
    ext x
    simp
  rw [← hset]
  exact hopen
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.me.mh__74c69977

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 74c69977e7d45292

4.334 334. closed_restricted_eq

The problem

Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) = g(x)\}$ is closed in X .

Lean statement

```
theorem closed_restricted_eq {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x = g x}
```

Parent. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

What it contributes. Reuses the parent's continuous comparison closedness checkpoint and closed-domain intersection, while replacing the single inequality locus with a two-direction equality characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the coupling from inequality to equality and requires proving equality as the intersection of two closed order loci, followed by antisymmetry. The parent's one-sided 'isClosed_le' argument cannot be recalled to close this goal directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality locus to an equality locus, which requires proving equality as the intersection of both inequality loci and applying antisymmetry. The parent proof supplies only one closed inequality and cannot be recalled unchanged.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality locus to an equality locus, requiring a second closed inequality set and a genuine antisymmetry argument. The parent proof only establishes one inequality and cannot be recalled to close the new goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2801 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_restricted_eq.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) : IsClosed {x | x ∈ s ∧ f x = g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is its given topology. For a closed set $s \subseteq X$ and continuous functions $f, g : X \rightarrow Y$, the set $\{x \in X \mid x \in s \text{ and } f(x) = g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem closed_restricted_eq {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x = g x} := by
have hle : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hge : IsClosed {x | g x ≤ f x} := by
  exact isClosed_le hg hf
have hcmp : IsClosed ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) := by
  exact hle.inter hge
have heq : {x | f x = g x} = {x | f x ≤ g x} ∩ {x | g x ≤ f x} := by
  ext x
  constructor
  · intro hx
    exact ⟨le_of_eq hx, le_of_eq hx.symm⟩
  · intro hx
    exact le_antisymm hx.1 hx.2
have heqclosed : IsClosed {x | f x = g x} := by
  rw [heq]
  exact hcmp
have hfinal : IsClosed (s ∩ {x | f x = g x}) := by
  exact hs.inter heqclosed
have hset : {x | x ∈ s ∧ f x = g x} = s ∩ {x | f x = g x} := by
  ext x
  rfl
rw [hset]
exact hfinal
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.mh__08974092

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 08974092f919d370

4.335 335. finite_joint_equality_isClosed

The problem

For a finite family of continuous functions from a topological space X into a linearly ordered topological space Y , the set of points where every corresponding pair of functions is equal is closed.

Lean statement

```
theorem finite_joint_equality_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x = g i x}
```

Parent. Let X be a topological space and Y a linearly ordered topological space. Let $n \in \mathbb{N}$, and for each $i = 0, \dots, n-1$, let $f_i, g_i : X \rightarrow Y$ be continuous. Show that the set $\{x \in X \mid f_i(x) \leq g_i(x) \text{ for every } i = 0, \dots, n-1\}$ is closed in X .

Parent in Lean

```
theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x}
```

What it contributes. Used the parent's finite-intersection and continuity-based closedness checkpoint for the forward comparison; the reverse comparison and antisymmetry provide the mutation-specific equality argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a simultaneous inequality locus to a simultaneous equality locus. Its proof requires proving both inequality directions for every index, intersecting the resulting closed sets, and recovering equality by antisymmetry; the parent supplies only one direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality locus requires proving and intersecting two independent finite joint inequality loci, using both $f \leq g$ and $g \leq f$ before applying antisymmetry. The parent's proof supplies only the forward inequality locus and cannot establish closedness of the equality set by itself.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality locus requires proving both inequality directions for every index and combining them via antisymmetry; the parent only establishes one-sided comparisons. The child therefore demands genuinely new reasoning beyond the parent's intersection-of-closed-sets argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1956 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_joint_equality_isClosed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {n : ℕ}
  {f g : Fin n → X → Y} (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  IsClosed {x | ∀ (i : Fin n), f i x = g i x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. Let n be a natural number, and let f and g be families of functions from X to Y indexed by $\text{Fin } n$. Assume that for every i in $\text{Fin } n$, $f i$ is continuous and $g i$ is continuous. Then the set of all x in X such that $f i x = g i x$ for every i in $\text{Fin } n$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_joint_equality_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x = g i x} := by
  have hle : ∀ i : Fin n, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hge : ∀ i : Fin n, IsClosed {x | g i x ≤ f i x} := by
    intro i
    exact isClosed_le (hg i) (hf i)
  have hforward : IsClosed (⋂ i : Fin n, {x | f i x ≤ g i x}) := by
    exact isClosed_iInter hle
  have hreverse : IsClosed (⋂ i : Fin n, {x | g i x ≤ f i x}) := by
    exact isClosed_iInter hge
  have hjoint : IsClosed ((⋂ i : Fin n, {x | f i x ≤ g i x}) ⋂
    (⋂ i : Fin n, {x | g i x ≤ f i x})) := by
    exact hforward.inter hreverse
  have hrepr : {x | ∀ i, f i x = g i x} =
    (⋂ i : Fin n, {x | f i x ≤ g i x}) ⋂
    (⋂ i : Fin n, {x | g i x ≤ f i x}) := by
    ext x
    simp only [Set.mem_iInter, Set.mem_setOf_eq, Set.mem_inter_iff]
  constructor
  · intro hx
    constructor
    · intro i
      exact le_of_eq (hx i)
    · intro i
      exact le_of_eq (hx i).symm
  · intro hx i
    exact le_antisymm (hx.1 i) (hx.2 i)
  rw [hrepr]
  exact hjoint
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.mh__8fcf5cd1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 73ec102f73c680c2

4.336 336. dense_eq_everywhere

The problem

Let X be a topological space and let Y be a linearly ordered set with the order topology. If continuous functions $f, g : X \rightarrow Y$ agree on a dense subset of X , show that they agree everywhere.

Lean statement

```
theorem dense_eq_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (heq : ∀ x ∈ s, f x = g x) :
  ∀ x, f x = g x
```

Parent. Let X be a topological space and let Y be a linearly ordered set with the order topology. If continuous functions $f, g : X \rightarrow Y$ satisfy $f(x) \leq g(x)$ on a dense subset of X , show that $f(x) \leq g(x)$ for every $x \in X$.

Parent in Lean

```
theorem dense_le_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (hle : ∀ x ∈ s, f x ≤ g x) :
  ∀ x, f x ≤ g x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-sided inequality into an equality, requiring two separate closure arguments and anti-symmetry; the parent proof supplies only one direction. This is a genuine change in proof obligation rather than decoration or recall.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a one-sided inequality to equality, requiring two separate density-closure arguments: one for $f \leq g$ and a new reversed one for $g \leq f$. The parent's proof supplies only the first direction, so recalling it does not complete the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an inequality to equality, requiring two separate density-closure arguments: one for $f \leq g$ and a new reversed argument for $g \leq f$, followed by antisymmetry. The parent's one-sided proof cannot close the child's goal by recall alone.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2257 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dense_eq_everywhere.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} {s : Set X},
  Continuous f → Continuous g → Dense s → (∀ x ∈ s, f x = g x) → ∀ (x : X), f x = g x
```

Read back by a model that saw only the Lean. *For any topological spaces X and Y , where Y is linearly ordered and equipped with its order topology, any functions $f, g : X \rightarrow Y$, and any subset s of X , if f and g are continuous, s is dense in X , and $f(x) = g(x)$ for every $x \in s$, then $f(x) = g(x)$ for every $x \in X$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem dense_eq_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (heq : ∀ x ∈ s, f x = g x) :
  ∀ x, f x = g x := by
  have hclosed_le : IsClosed {x : X | f x ≤ g x} := isClosed_le hf hg
  have hsubset_le : s ⊆ {x : X | f x ≤ g x} := by
    intro x hx
    exact le_of_eq (heq x hx)
  have hclosure_le : closure s ⊆ {x : X | f x ≤ g x} := by
    exact closure_minimal hsubset_le hclosed_le
  have hle : ∀ x, f x ≤ g x := by
    intro x
    exact hclosure_le (hs x)
  have hclosed_ge : IsClosed {x : X | g x ≤ f x} := isClosed_le hg hf
  have hsubset_ge : s ⊆ {x : X | g x ≤ f x} := by
    intro x hx
    exact le_of_eq (heq x hx).symm
  have hclosure_ge : closure s ⊆ {x : X | g x ≤ f x} := by
    exact closure_minimal hsubset_ge hclosed_ge
  have hge : ∀ x, g x ≤ f x := by
    intro x
    exact hclosure_ge (hs x)
  intro x
  exact le_antisymm (hle x) (hge x)
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh.mh__180e4a76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 180e4a765ca2cfc b

4.337 337. closed_order_region_on_closed_set

The problem

Let S be a closed subset of a topological space X , and let $f, g : X \rightarrow Y$ be continuous on S , where Y has the order topology. Show that the points of S where $f(x) \leq g(x)$ form a closed subset of X .

Lean statement

```
theorem closed_order_region_on_closed_set {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x ∈ s | f x ≤ g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from a globally closed order region for continuous functions to the intersection of a closed set with an order region for functions only continuous on that set. The parent's direct 'isClosed_le' proof does not apply; the new argument must use the closedness of S together with restricted continuity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from globally continuous functions on all of X to functions continuous only on a closed subset s , proving the order region within s is ambiently closed. The parent's direct 'isClosed_le' argument does not establish this restricted-set result; the new closed-subset reasoning is substantive.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the obligation from global continuity on all of X to continuity only on a closed subset s , while proving the restricted order region is closed. Its proof requires the specialized closed-set/ContinuousOn lemma and cannot be obtained by recalling the parent's 'isClosed_le' proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2132 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_order_region_on_closed_set.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y]
  {s : Set X} {f g : X → Y} (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. Let s be a closed subset of X , and let $f, g : X \rightarrow Y$ be continuous on s . Then the subset $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_order_region_on_closed_set {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x ∈ s | f x ≤ g x} := by
  have hrestricted : IsClosed {x ∈ s | f x ≤ g x} := hs.isClosed_le hf hg
  exact hrestricted
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__7798f78c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7798f78c1903dfd4

4.338 338. continuous_eq_set_closed

The problem

Let Y be a linearly ordered topological space with its order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) = g(x)\}$ is closed in X .

Lean statement

```
theorem continuous_eq_set_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a sublevel set to an equalizer and requires representing equality as the intersection of two closed inequality sets. Although it reuses the parent lemma twice, the parent's single application does not prove the new set-identity and intersection argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness of an order region into closedness of an equalizer, requiring two order-region closures, an intersection argument, and conversion via antisymmetry. The parent's single 'isClosed_le' application does not supply this proof structure.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided order region into an equalizer, requiring both inequalities, intersection closure, and extensional conversion to equality. This reasoning is not supplied by the parent's direct application of 'isClosed_le'.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2800 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_eq_set_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsClosed {x | f x = g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is its given topology. If f and g are continuous functions from X to Y , then the set of points x in X such that $f(x) = g(x)$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_eq_set_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsClosed {x | f x = g x} := by
have hle : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hge : IsClosed {x | g x ≤ f x} := by
  exact isClosed_le hg hf
have heq : {x | f x = g x} = {x | f x ≤ g x} ∩ {x | g x ≤ f x} := by
  ext x
  simp only [mem_setOf_eq, mem_inter_iff]
  exact le_antisymm_iff
rw [heq]
exact hle.inter hge
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__7cd0cba0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7cd0cba08b23dcb1

4.339 339. closed_le_and_open_lt

The problem

Let Y be an ordered set in the order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that $x \mid f(x) \leq g(x)$ is closed and $x \mid f(x) < g(x)$ is open in X .

Lean statement

```
theorem closed_le_and_open_lt {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child adds the genuinely separate obligation that the strict-order locus is open, requiring ‘isOpen_lt’ in addition to the parent’s closedness argument. The conjunction is not a forced or trivial wrapper because its second claim is an independent topological theorem.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely new obligation that the strict-inequality set is open, requiring ‘isOpen_lt’; the parent proves only closedness of the non-strict inequality set. The conjunction is therefore substantive rather than a restatement or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely distinct obligation that the strict-inequality set is open, requiring the separate theorem ‘isOpen_lt’; the parent’s ‘isClosed_le’ argument proves only the first conjunct. This is more than a trivial conjunction wrapper because the added conjunct has substantive topological content and its own proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2261 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_le_and_open_lt.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y also equipped with a linear order whose topology is the order topology. For functions $f, g : X \rightarrow Y$, if f and g are continuous, then the set $\{x \in X \mid f(x) \leq g(x)\}$ is closed in X , and the set $\{x \in X \mid f(x) < g(x)\}$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_le_and_open_lt {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x} := by
have hClosed : IsClosed {x | f x ≤ g x} := by
  simpa using isClosed_le hf hg
have hOpen : IsOpen {x | f x < g x} := by
  simpa using isOpen_lt hf hg
exact ⟨hClosed, hOpen⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__b6262118

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b6262118944a5e2d

4.340 340. equality_and_bound_closed

The problem

Let Y be an ordered set in the order topology, and let $f, g, h : X \rightarrow Y$ be continuous. Show that the set of points where $f = g$ and $g \leq h$ is closed in X .

Lean statement

```
theorem equality_and_bound_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g h : X → Y}
  (hf : Continuous f) (hg : Continuous g) (hh : Continuous h) :
  IsClosed {x | f x = g x ∧ g x ≤ h x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from one sublevel set to the closed intersection of an equality locus and a second inequality locus. Proving equality as the intersection of both opposite inequalities, then combining three closed sets, requires reasoning absent from the parent's direct one-lemma proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from one comparison set to a conjunction combining equality and an independent bound. Proving it requires decomposing equality into two inequalities and taking multiple closed-set intersections, which is not supplied by the parent's single application of `isClosed_le`.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the proof obligation from one comparison set to a conjunction combining equality and an upper bound. Proving equality requires two opposite closed inequalities, then intersecting with the third comparison; the parent's single application of `'isClosed_le'` does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2804 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equality_and_bound_closed.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g h : X → Y} (hf : Continuous f)
(hg : Continuous g) (hh : Continuous h) : IsClosed {x | f x = g x ∧ g x ≤ h x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. For functions $f, g, h : X \rightarrow Y$, if f, g , and h are continuous, then the subset of X consisting of all points x such that $f(x) = g(x)$ and $g(x) \leq h(x)$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem equality_and_bound_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g h : X → Y}
(hf : Continuous f) (hg : Continuous g) (hh : Continuous h) :
IsClosed {x | f x = g x ∧ g x ≤ h x} := by
have hfg : IsClosed {x | f x ≤ g x} := isClosed_le hf hg
have hgf : IsClosed {x | g x ≤ f x} := isClosed_le hg hf
have hgh : IsClosed {x | g x ≤ h x} := isClosed_le hg hh
have hpair : IsClosed ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) := hfg.inter hgf
have hcombined : IsClosed (({x | f x ≤ g x} ∩ {x | g x ≤ f x}) ∩ {x | g x ≤ h x}) := hpair.inter hgh
have htarget : {x | f x = g x ∧ g x ≤ h x} = ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) ∩ {x | g x ≤ h x} := by
  ext x
  simp only [Set.mem_setOf_eq, Set.mem_inter_iff]
  constructor
  · rintro ⟨he, hle⟩
    exact ⟨⟨le_of_eq he, le_of_eq he.symm⟩, hle⟩
  · rintro ⟨⟨h₁, h₂⟩, hle⟩
    exact ⟨le_antisymm h₁ h₂, hle⟩
rw [htarget]
exact hcombined
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__b6fcc7fc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b6fcc7fc87d61208

4.341 341. quotient_image_connectedComponent

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. Show that p maps the connected component of each point onto the connected component of its image.

Lean statement

```
theorem quotient_image_connectedComponent {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) (x : X) :
  p '' connectedComponent x = connectedComponent (p x)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. If $C \subseteq Y$ is closed and connected, then $p^{-1}(C)$ is connected.

Parent in Lean

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' C)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from connectedness of a preimage to an equality of connected components. Its proof requires a component-preimage theorem and separately derives surjectivity from nonempty connected fibers, neither of which is supplied by the parent's argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from connectedness of a preimage of a closed connected set to an equality of images of connected components. Its proof requires component-specific reasoning and surjectivity derived from nonempty connected fibers, none of which is supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the result from connectedness of a preimage of a closed connected set to an equality characterizing the image of a connected component. Proving this requires the new connected-component preimage theorem and a surjectivity argument from nonempty fibers, neither of which is supplied by the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2209 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_image_connectedComponent.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ (y : Y), IsConnected (p ⁻¹ {y})) (x : X) : p '' connectedComponent x = connectedComponent (p x)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and suppose that for every $y \in Y$ the fiber $p^{-1}(\{y\})$ is connected. Then, for every $x \in X$, the image under p of the connected component of x is exactly the connected component of $p(x)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem quotient_image_connectedComponent {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹ ({y} : Set Y))) (x : X) :
  p '' connectedComponent x = connectedComponent (p x) := by
  have hcoind : IsCoinducing p := hq.isCoinducing
  have hpre : p ⁻¹ '' connectedComponent (p x) = connectedComponent x :=
    hcoind.preimage_connectedComponent hX x
  have hsurj : Function.Surjective p := by
    intro y
    rcases (hX y).nonempty with ⟨z, hz⟩
    exact ⟨z, by simpa using hz⟩
  have himage : p '' (p ⁻¹ '' connectedComponent (p x)) = connectedComponent (p x) :=
    image_preimage_eq _ hsurj
  rw [hpre] at himage
  exact himage
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me.mh__d1579e3d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1579e3dca610c95

4.342 342. continuous_strict_gap_side

The problem

Let $p : X \rightarrow Y$ be a map and suppose the lifted subspace $p^{-1}(C)$ is connected. If a continuous function $f : p^{-1}(C) \rightarrow \mathbb{R}$ satisfies $f(x) < a$ or $b < f(x)$ for every x , where $a < b$, prove that all values of f lie strictly on the same side of the gap.

Lean statement

```
theorem continuous_strict_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x < a ∨ b < f x) :
  (∀ x, f x < a) ∨ (∀ x, b < f x)
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The strict-gap conclusion is not obtainable by simply applying the parent: the proof must rule out values on both sides using the intermediate value property and a midpoint argument. This is a genuinely stronger, different proof obligation with no redundant hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The strict inequalities require a genuinely different intermediate-value argument: assuming values occur on both sides forces the continuous image through the forbidden midpoint. The parent's closed-set/clopen proof does not directly supply this strict-gap reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion and hypotheses to strict separation, requiring an intermediate-value argument to rule out simultaneously occurring lower and upper values. The parent's clopen-set proof for non-strict inequalities does not supply this proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2859 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_strict_gap_side.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  (p : X → Y) (C : Set Y) (hD : ConnectedSpace { x // p x ∈ C }) (f : { x // p x ∈ C } → ℝ) (hf : Continuous f)
  (a b : ℝ) (hab : a < b) (hd : ∀ (x : { x // p x ∈ C }), f x < a ∨ b < f x) :
  (∀ (x : { x // p x ∈ C }), f x < a) ∨ ∀ (x : { x // p x ∈ C }), b < f x
```

Read back by a model that saw only the Lean. *Let X and Y be types, with X equipped with a topological space structure. Let $p : X \rightarrow Y$ and $C \subseteq Y$, and suppose that the subtype $\{x \in X \mid p(x) \in C\}$ is connected. Let f be a continuous real-valued function on this subtype. For real numbers a and b with $a < b$, assume that for every x in the subtype, either $f(x) < a$ or $b < f(x)$. Then either $f(x) < a$ for every x in the subtype, or $b < f(x)$ for every x in the subtype.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_strict_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace { x : X // p x ∈ C })
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x < a ∨ b < f x) :
  (∀ x, f x < a) ∨ (∀ x, b < f x) := by
  have hpre : IsPreconnected (Set.univ : Set { x : X // p x ∈ C }) :=
    (connectedSpace_iff_univ.mp hD).isPreconnected
  by_cases hlow : ∃ x, f x < a
  · by_cases hupp : ∃ y, b < f y
    · rcases hlow with ⟨x, hx⟩
      rcases hupp with ⟨y, hy⟩
      let c : ℝ := (a + b) / 2
      have hxc : f x ≤ c := by
        dsimp [c]
        linarith
      have hcy : c ≤ f y := by
        dsimp [c]
        linarith
      have hinterval :
        Set.Icc (f x) (f y) ⊆
          f '' (Set.univ : Set { x : X // p x ∈ C }) :=
        hpre.intermediate_value (Set.mem_univ x) (Set.mem_univ y) hf.continuousOn
      have hc : c ∈ f '' (Set.univ : Set { x : X // p x ∈ C }) :=
        hinterval ⟨hxc, hcy⟩
      rcases hc with ⟨z, hz, hzc⟩
      rcases hd z with hza | hzb
      · exfalso
        dsimp [c] at hzc
        linarith
      · exfalso
        dsimp [c] at hzc
        linarith
    · left
```

```

    intro x
    rcases hd x with hxa | hxb
    · exact hxa
    · exact False.elim (hupp ⟨x, hxb⟩)
  · right
    intro x
    rcases hd x with hxa | hxb
    · exact False.elim (hlow ⟨x, hxa⟩)
    · exact hxb

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__1550c1e9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1550c1e9931c183b

4.343 343. continuous_gap_image_side

The problem

Let S be a connected subset of the lifted space $p^{-1}(C)$. If a continuous function on S takes every value at most a or at least b , where $a < b$, prove that its image lies entirely on one side of the gap (a, b) .

Lean statement

```
theorem continuous_gap_image_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (S : Set {x : X // p x ∈ C})
  (hS : IsConnected S)
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : ContinuousOn f S)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  (f '' S ⊆ Set.Iic a) ∨ (f '' S ⊆ Set.Ici b)
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the obligation from a global dichotomy on the whole connected subtype to a dichotomy for the image of an arbitrary connected subset. Its proof must handle `ContinuousOn`, relative connectedness, and image witnesses, none of which are supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a global pointwise dichotomy to a dichotomy for the image of an arbitrary connected subset, requiring a new intermediate-value argument with `ContinuousOn`. The parent's clopen-set proof does not supply this image-level reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the obligation to a dichotomy for the image of an arbitrary connected subset under a function continuous only on that subset. Its proof requires an intermediate-value argument to rule out crossing the forbidden gap, which is not supplied by the parent's clopen-subset proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2854 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_gap_image_side.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  (p : X → Y) (C : Set Y) (S : Set { x // p x ∈ C }) (hS : IsConnected S) (f : { x // p x ∈ C } → ℝ)
  (hf : ContinuousOn f S) (a b : ℝ) (hab : a < b) (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  f '' S ⊆ Set.Iic a ∨ f '' S ⊆ Set.Ici b
```

Read back by a model that saw only the Lean. *Let X be a topological space, let Y be a type, let $p : X \rightarrow Y$, and let $C \subseteq Y$. Consider the subtype $\{x \in X \mid p(x) \in C\}$, let S be a connected subset of this subtype, and let $f : \{x \in X \mid p(x) \in C\} \rightarrow \mathbb{R}$ be continuous on S . If $a, b \in \mathbb{R}$ satisfy $a < b$ and every $x \in S$ satisfies $f(x) \leq a$ or $b \leq f(x)$, then either $f(S) \subseteq (-\infty, a]$ or $f(S) \subseteq [b, \infty)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_gap_image_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (S : Set { x : X // p x ∈ C })
  (hS : IsConnected S)
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : ContinuousOn f S)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  (f '' S ⊆ Set.Iic a) ∨ (f '' S ⊆ Set.Ici b) := by
  let m : ℝ := (a + b) / 2
  have hma : a < m := by
    dsimp [m]
    linarith
  have hmb : m < b := by
    dsimp [m]
    linarith
  by_cases hlow : ∃ x ∈ S, f x ≤ a
  · rcases hlow with ⟨x, hxS, hxa⟩
    left
    intro z hz
    rcases hz with ⟨y, hyS, rfl⟩
    rcases hd y hyS with hya | hyb
    · exact hya
    · exfalso
      have hxy : f x ≤ f y := by
        linarith
      have hmid : m ∈ Set.Icc (f x) (f y) := by
        constructor <|> linarith
      have hmemb : m ∈ f '' S :=
        (hS.isPreconnected.intermediate_value hxS hyS hf) hmid
      rcases hmemb with ⟨w, hwS, hw⟩
      rcases hd w hwS with hwa | hwb
      · linarith
      · linarith
  · right
    intro z hz
    rcases hz with ⟨y, hyS, rfl⟩
```

```
rcases hd y hyS with hya | hyb
• exact False.elim (hlow ⟨y, hyS, hya⟩)
• exact hyb
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__909d624c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 909d624c0275592a

4.344 344. continuous_gap_intermediate_value

The problem

Let $p : X \rightarrow Y$ be a map whose lifted subspace $p^{-1}(C)$ is connected. If a continuous function on this subspace takes a value at most a and another at least b , where $a < b$, prove that it takes some value in the interval $[a, b]$.

Lean statement

```
theorem continuous_gap_intermediate_value
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hlo : ∃ x, f x ≤ a)
  (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine intermediate-value theorem: endpoint witnesses on opposite sides force a value in $[a, b]$. Its proof uses connectedness of the domain and continuity via a range lemma, reasoning not supplied by the parent's clopen-set dichotomy argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a global one-sided dichotomy to an intermediate-value existence claim. Its proof requires the connected-space intermediate value theorem using both low and high witnesses, which the parent's clopen-set argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's gap dichotomy with a genuine intermediate-value existence claim. Its proof uses continuity and connectedness to establish that the image contains the endpoint value, reasoning not supplied by the parent's side-classification argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2848 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_gap_intermediate_value.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] (p : X → Y) (C : Set Y) (hD : ConnectedSpace { x // p x ∈ C }) (f : { x // p x ∈ C } → ℝ)
  (hf : Continuous f) (a b : ℝ) (hab : a < b) (hlo : ∃ x, f x ≤ a) (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b
```

Read back by a model that saw only the Lean. *Let X and Y be types, let X have a topological-space structure, let $p : X \rightarrow Y$, and let C be a subset of Y . Assume that the subtype of elements x of X satisfying $p(x) \in C$ is connected. Let f be a continuous real-valued function on this subtype. For real numbers a and b with $a < b$, suppose there exists an x in the subtype such that $f(x) \leq a$, and there exists an x in the subtype such that $b \leq f(x)$. Then there exists an x in the subtype such that $a \leq f(x)$ and $f(x) \leq b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_gap_intermediate_value
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace { x : X // p x ∈ C })
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hlo : ∃ x, f x ≤ a)
  (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b := by
  letI : ConnectedSpace {x : X // p x ∈ C} := hD
  have hupper : ∃ x, a ≤ f x := by
    rcases hhi with ⟨x, hx⟩
    exact ⟨x, le_trans (le_of_lt hab) hx⟩
  have hvalue : a ∈ Set.range f := by
    exact mem_range_of_exists_le_of_exists_ge hf hlo hupper
  rcases hvalue with ⟨x, hx⟩
  refine ⟨x, ?, ?⟩
  · exact hx.symm ▸ le_rfl
  · exact hx.symm ▸ le_of_lt hab
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__c65fa6ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c65fa6ef4ca9de06

4.345 345. quotientMap_image_connectedComponent_eq_univ

The problem

Let $p : X \rightarrow Y$ be a quotient map with connected fibers, and suppose Y is connected. Show that the image under p of the connected component containing any $x \in X$ is all of Y .

Lean statement

```
theorem quotientMap_image_connectedComponent_eq_univ {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) (x : X) :
  p '' connectedComponent x = Set.univ
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

What it contributes. Preserves the parent's quotient-map, connected-base, and connected-fiber setup; the child consumes the connected-component checkpoint to prove a stronger image-level conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from global connectedness of X to the substantive characterization that every connected component maps onto all of Y . Its proof requires the quotient-map connected-component image theorem and connectedness of Y , neither of which is supplied by recalling the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens connectedness of X to the pointwise characterization $p '' \text{connectedComponent } x = \text{Set.univ}$. Its proof requires the quotient-map image-of-component theorem and connected-component collapse in Y , reasoning not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from global connectedness of X to a component-image characterization, requiring the distinct mathematical facts that quotient maps preserve connected components and that every component of connected Y is universal. Memorizing the parent's open-cover proof would not substantially solve this equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2181 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotientMap_image_connectedComponent_eq_univ.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p) (hY : ConnectedSpace Y)
  (hX : ∀ (y : Y), IsConnected (p ⁻¹ {y})) (x : X) : p '' connectedComponent x = univ
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, assume that Y is connected and that for every $y \in Y$ the fiber $p^{-1}(\{y\})$ is connected. Then, for every $x \in X$, the image under p of the connected component of x is all of Y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem quotientMap_image_connectedComponent_eq_univ {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹ {y})) (x : X) :
  p '' connectedComponent x = Set.univ := by
  have himage : p '' connectedComponent x = connectedComponent (p x) := by
    exact hq.image_connectedComponent hX x
  have hcomponent : connectedComponent (p x) = (Set.univ : Set Y) := by
    exact PreconnectedSpace.connectedComponent_eq_univ (p x)
  exact himage.trans hcomponent
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__mh__4b02c9a8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b02c9a8e68032af

4.346 346. connected_preimage_of_closed_connected

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. Show that the inverse image of every closed connected subset of Y is connected.

Lean statement

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  ∀ t : Set Y, IsClosed t → IsConnected t → IsConnected (p ⁻¹' t)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the global connectedness hypothesis on Y and proves a parametrized preservation theorem for every closed connected subset t , rather than merely deriving connectedness of X from the special case $t = Y$. Its proof uses the stronger coinducing preimage lemma, so recalling the parent's proof does not solve the new obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from global connectedness of X to connectedness of the preimage of every closed connected subset of Y . Its proof uses the quotient map's coinducing property and a closed-set preimage theorem, which the parent's open-cover argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes connectedness from the whole space to the preimage of every closed connected subset, requiring the quotient-map preimage theorem rather than the parent's global cover argument. The parent's hypotheses on the codomain are removed, and the conclusion is genuinely set-level.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2815 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_of_closed_connected.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y),
  IsQuotientMap p →
  (∀ (y : Y), IsConnected (p-1 {y})) → ∀ (t : Set Y), IsClosed t → IsConnected t → IsConnected (p-1 t)
```

Read back by a model that saw only the Lean. *For any topological spaces X and Y and any function $p : X \rightarrow Y$, if p is a quotient map and the fiber $p^{-1}(\{y\})$ is connected for every $y \in Y$, then for every closed and connected subset t of Y , the preimage $p^{-1}(t)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace Topology

theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p-1 ({y} : Set Y))) :
  ∀ t : Set Y, IsClosed t → IsConnected t → IsConnected (p-1 t) := by
  intro t ht htc
  have hco : IsCoinducing p := hq.isCoinducing
  have hpre : IsConnected (p-1 t) :=
    hco.isConnected_preimage_of_isClosed hX ht htc
  exact hpre
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__mh__ddc293ff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ddc293ff8c2706b4

4.347 347. connected_bounded_trace_prefix

The problem

Let (A_n) be connected subspaces and let f , g , and h be uniformly continuous. If, for every $n < N$, A_{n+1} intersects $A_0 \cup \dots \cup A_n$, show that $\bigcup_{n=0}^N \{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Lean statement

```
theorem connected_bounded_trace_prefix
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (N : ℕ)
  (hA : ∀ n, IsConnected (A n))
  (hOverlap : ∀ n, n < N → (A (n + 1) ∩ ⋃ k ≤ n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n ≤ N, (fun x => (g (f x), h (g (f x)))) '' A n)
```

Parent. Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f , g , and h are uniformly continuous, show that the union of the traced sets $\{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Parent in Lean

```
theorem connected_adjacent_trace_iUnion
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n, (fun x => (g (f x), h (g (f x)))) '' A n)
```

What it contributes. Retains the parent's connected family, three-stage uniformly continuous trace, and overlap-witness transport, while replacing the terminal predecessor-union conclusion with induction over a bounded prefix.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the infinite-union theorem into a finite-prefix characterization and uses induction to maintain connectedness while adjoining each successive image. Its bounded overlap hypothesis and finite-union equality require reasoning absent from the parent's global reachability argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's infinite predecessor-connectedness argument with a bounded induction over finite prefixes, using a new union decomposition and a stepwise overlap invariant. This is substantive reasoning that the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the infinite predecessor-union theorem into a finite-prefix result under only bounded overlap hypotheses. Its induction must establish connectedness of each finite union and prove the prefix decomposition, reasoning not supplied by the parent's infinite reachability argument.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2190 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_bounded_trace_prefix.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {γ : Type u_2} {δ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace γ] [inst_2 : UniformSpace δ] {A : ℕ → Set α} {f : α → γ} {g : γ → γ}
  {h : γ → δ} (N : ℕ) (hA : ∀ (n : ℕ), IsConnected (A n))
  (hOverlap : ∀ n < N, (A (n + 1) ∩ U k ≤ n, A k).Nonempty) (hf : UniformContinuous f)
  (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (U n, U (λ n, n ≤ N), (fun x => (g (f x), h (g (f x)))) '' A n)
```

Read back by a model that saw only the Lean. Let α , γ , and δ be uniformly structured spaces, let A assign to each natural number n a subset $A(n)$ of α , and let $f : \alpha \rightarrow \gamma$, $g : \gamma \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. For a natural number N , suppose that every $A(n)$ is connected, and that for every $n < N$, the set $A(n+1)$ intersects the union of all $A(k)$ with $k \leq n$. Then the union, over all $n \leq N$, of the image of $A(n)$ under the map $x \mapsto (g(f(x)), h(g(f(x))))$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_bounded_trace_prefix
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (N : ℕ)
  (hA : ∀ n, IsConnected (A n))
  (hOverlap : ∀ n, n < N → (A (n + 1) ∩ U k ≤ n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (U n ≤ N, (fun x => (g (f x), h (g (f x)))) '' A n) := by
  classical
  let T : α → γ × δ := fun x => (g (f x), h (g (f x)))
  have hT : Continuous T := by
    dsimp [T]
    exact ((hg.comp hf).prodMk (hh.comp (hg.comp hf))).continuous
  let B : ℕ → Set (γ × δ) := fun n => T '' A n
  have hB : ∀ n, IsConnected (B n) := by
    intro n
    exact (hA n).image T hT.continuousOn
  let U : ℕ → Set (γ × δ) := fun m => U n ≤ m, B n
  induction N with
  | zero =>
    change IsConnected (U 0)
    simpa [U] using hB 0
  | succ N ih =>
    change IsConnected (U (N + 1))
    have hprev : IsConnected (U N) := by
      apply ih
      intro n hn
      exact hOverlap n (by omega)
    have hw : (U N ∩ B (N + 1)).Nonempty := by
      obtain ⟨x, hxnext, hxprev⟩ := hOverlap N (by omega)
      refine ⟨T x, ?, ⟨x, hxnext, rfl⟩⟩
      rcases Set.mem_iUnion.mp hxprev with ⟨k, hxprev⟩
      rcases Set.mem_iUnion.mp hxprev with ⟨hk, hxk⟩
```



```

    exact Set.mem_iUnion.mpr ⟨k, Set.mem_iUnion.mpr ⟨hk, ⟨x, hxk, rfl⟩⟩⟩
have hu : IsConnected (U N ∪ B (N + 1)) := hprev.union hw (hB (N + 1))
have heq : U (N + 1) = U N ∪ B (N + 1) := by
  ext y
  constructor
  · intro hy
    rcases Set.mem_iUnion.mp hy with ⟨n, hy⟩
    rcases Set.mem_iUnion.mp hy with ⟨hn, hyn⟩
    by_cases hnm : n ≤ N
    · exact Or.inl (Set.mem_iUnion.mpr ⟨n, Set.mem_iUnion.mpr ⟨hnm, hyn⟩⟩)
    · have hne : n = N + 1 := by omega
      exact Or.inr (hne ▸ hyn)
  · intro hy
    rcases hy with hy | hy
    · rcases Set.mem_iUnion.mp hy with ⟨n, hy⟩
      rcases Set.mem_iUnion.mp hy with ⟨hn, hyn⟩
      exact Set.mem_iUnion.mpr ⟨n, Set.mem_iUnion.mpr ⟨le_trans hn (Nat.le_succ N), hyn⟩⟩
    · exact Set.mem_iUnion.mpr ⟨N + 1, Set.mem_iUnion.mpr ⟨le_refl _, hy⟩⟩
rw [heq]
exact hu

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.mh__5fcff258

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5fcff258a72c4753

4.348 348. connected_preimage_expanded_chain

The problem

Let $p : X \rightarrow Y$ be a quotient map. Suppose $\{A_n\}$ is a chain of connected subspaces with consecutive overlaps, and let B be connected and intersect every A_n . If $B \cup \bigcup_n A_n$ is closed and every fiber of p is connected, show that $p^{-1}(B \cup \bigcup_n A_n)$ is connected.

Lean statement

```
theorem connected_preimage_expanded_chain {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y} (B : Set Y)
  (hA : ∀ n, IsConnected (A n))

  (hB : IsConnected B)
  (hBA : ∀ n, (B ∩ A n).Nonempty)
  (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (B ∪ ⋃ n, A n))
```

Parent. Let $p : X \rightarrow Y$ be a quotient map, and let $\{A_n\}$ be connected subspaces of Y with $A_n \cap A_{n+1} \neq \emptyset$ for every n . If their union is closed and every fiber of p is connected, show that $p^{-1}(\bigcup_n A_n)$ is connected.

Parent in Lean

```
theorem connected_preimage_adjacent_union {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y}
  (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  (hclosed : IsClosed (⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (⋃ n, A n))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's adjacent-overlap hypothesis and instead proves connectedness through a new common hub B, constructing the family $C_n = B \cup A_n$ and identifying its union with the target. The parent's direct chain argument is unavailable without hAn, so the proof obligation genuinely changes.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely expands the connected chain by adjoining an arbitrary connected set B and proving each $B \cup A_n$ is connected via intersection hypotheses. It also requires a new set-indexed union equality and connectivity argument, so the parent's proof does not suffice by recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces adjacent overlap between successive sets with a genuinely different construction: a connected set B intersects every A_n , making each $B \cup A_n$ connected and providing the chain overlaps. The parent's adjacent-chain argument cannot supply this expanded-union proof, and the child also removes hAn.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_expanded_chain.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y}
  (B : Set Y) (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), (A n ∩ A (n + 1)).Nonempty) (hB : IsConnected B)
  (hBA : ∀ (n : ℕ), (B ∩ A n).Nonempty) (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ (y : Y), IsConnected (p ⁻¹' {y})) : IsConnected (p ⁻¹' (B ∪ ⋃ n, A n))
```

Read back by a model that saw only the Lean. Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and let $A : \mathbb{N} \rightarrow \text{Set } Y$ be a sequence of subsets of Y . Let $B \subseteq Y$. Assume that each $A\ n$ is connected; for every $n \in \mathbb{N}$, the intersection $A\ n \cap A\ (n + 1)$ is nonempty; B is connected; for every $n \in \mathbb{N}$, the intersection $B \cap A\ n$ is nonempty; $B \cup \bigcup_{n \in \mathbb{N}} A\ n$ is closed in Y ; and for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage $p^{-1}(B \cup \bigcup_{n \in \mathbb{N}} A\ n)$ is connected in X .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_preimage_expanded_chain {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y} (B : Set Y)
  (hA : ∀ n, IsConnected (A n))

  (hB : IsConnected B)
  (hBA : ∀ n, (B ∩ A n).Nonempty)
  (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' {y} : Set Y)) :
  IsConnected (p ⁻¹' (B ∪ ⋃ n, A n)) := by
  let C : ℕ → Set Y := fun n => B ∪ A n
  have hC : ∀ n, IsConnected (C n) := by
    intro n
    exact hB.union (hBA n) (hA n)
  have hBne : B.Nonempty := by
    rcases hBA 0 with ⟨y, hyB, hyA⟩
    exact ⟨y, hyB⟩
  have hAdj : ∀ n, (C n ∩ C (Nat.succ n)).Nonempty := by
    intro n
    rcases hBne with ⟨y, hy⟩
    exact ⟨y, ⟨Or.inl hy, Or.inl hy⟩⟩
  have hbase : IsConnected (⋃ n, C n) := by
    apply IsConnected.iUnion_of_chain hC
    intro n
    exact hAdj n
  have hEq : (⋃ n, C n) = B ∪ ⋃ n, A n := by
    ext y
    constructor
    · intro hy
      rcases Set.mem_iUnion.mp hy with ⟨n, hn⟩
      rcases hn with hn | hn
      · exact Or.inl hn
      · exact Or.inr (Set.mem_iUnion.mpr ⟨n, hn⟩)
    · intro hy
      rcases hy with hy | hy
      · exact Set.mem_iUnion.mpr ⟨0, Or.inl hy⟩
```

```

    · rcases Set.mem_iUnion.mp hy with ⟨n, hn⟩
      exact Set.mem_iUnion.mpr ⟨n, Or.inr hn⟩
have hclosedC : IsClosed (U n, C n) := by
  rw [hEq]
  exact hclosed
have hpre : IsConnected (p ⁻¹' (U n, C n)) := by
  exact hq.isCoinducing.isConnected_preimage_of_isClosed hfib hclosedC hbase
rw [← hEq]
exact hpre

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.mh__467e4adc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2f4814a992e9ee7

4.349 349. connected_iUnion_of_generated_overlap

The problem

Let $\{A_i\}_{i \in I}$ be connected subspaces of a topological space. Suppose a relation on I links any two related indices through a nonempty intersection, and any two indices are connected by a finite chain of such relations. Show that $\bigcup_{i \in I} A_i$ is connected.

Lean statement

```
theorem connected_iUnion_of_generated_overlap {X : Type*} [TopologicalSpace X] {ι : Type*} [Nonempty ι]
  {A : ι → Set X} {R : ι → ι → Prop}
  (hA : ∀ i, IsConnected (A i))
  (hR : ∀ i j, R i j → (A i ∩ A j).Nonempty)
  (hChain : ∀ i j, Relation.ReflTransGen R i j) :
  IsConnected (⋃ i, A i)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the natural-number chain to an arbitrary nonempty index type and an arbitrary relation whose reflexive-transitive closure connects every pair. Its proof uses a general connected-union theorem and requires transporting reachability through intersection overlap, which the parent's successor-chain argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the consecutive-natural-index argument to an arbitrary nonempty index type and an abstract relation, requiring an induction that transfers relational reachability into overlap reachability. The parent's successor-chain construction cannot prove this theorem directly, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes consecutive-index overlap to an arbitrary nonempty index type and an arbitrary generating relation. Its proof must transport relation reachability into overlap reachability via induction, which the parent's successor-chain argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2839 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B assumes the index type is nonempty, while Statement A does not explicitly require the index set to be nonempty.

Goal round-trip: what Lean elaborated

```
theorem connected_iUnion_of_generated_overlap.extracted_1_1.{u_2, u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {ι : Type u_2} [inst_1 : Nonempty ι] {A : ι → Set X} {R : ι → ι → Prop} (hA : ∀ (i : ι), IsConnected (A i))
  (hR : ∀ (i j : ι), R i j → (A i ∩ A j).Nonempty) (hChain : ∀ (i j : ι), Relation.ReflTransGen R i j) :
  IsConnected (U i, A i)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let ι be a nonempty type, let A assign to each $i \in \iota$ a subset $A(i)$ of X , and let R be a binary relation on ι . Assume that each $A(i)$ is connected; whenever $R(i,j)$ holds, the intersection $A(i) \cap A(j)$ is nonempty; and for every $i,j \in \iota$, j is reachable from i by the reflexive transitive closure of R . Then the union $\bigcup \{i \in \iota \mid A(i)\}$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem connected_iUnion_of_generated_overlap {X : Type*} [TopologicalSpace X] {ι : Type*} [Nonempty ι]
  {A : ι → Set X} {R : ι → ι → Prop}
  (hA : ∀ i, IsConnected (A i))
  (hR : ∀ i j, R i j → (A i ∩ A j).Nonempty)
  (hChain : ∀ i j, Relation.ReflTransGen R i j) :
  IsConnected (U i, A i) := by
  have hOverlap : ∀ i j, Relation.ReflTransGen (fun i j => (A i ∩ A j).Nonempty) i j := by
    intro i j
    induction hChain i j with
    | refl => exact Relation.ReflTransGen.refl
    | tail hxy hyz ih => exact ih.tail (hR _ _ hyz)
  exact IsConnected.iUnion_of_reflTransGen hA hOverlap
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__mh__5db0f6a7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5db0f6a772de6f26

4.350 350. connected_iUnion_closure

The problem

Let $\{A_n\}$ be a sequence of connected subsets of a topological space X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup_n \overline{A_n}$ is connected.

Lean statement

```
theorem connected_iUnion_closure {X : Type*} [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n)) (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) : IsConnected (⋃ n, closure (A n))
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*} [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n)) (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) : IsConnected (⋃ n, A n) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely changes the sets being unioned to their closures, requiring preservation of connectedness under closure and transport of adjacent intersection witnesses into the closures. The parent's union-connectedness proof does not supply these closure-specific steps.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to a union of closures and requires new reasoning that closures preserve connectedness and preserve adjacent intersection witnesses via `subset_closure`. Although it reuses the chain argument, the parent proof does not supply these closure-specific obligations.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the union members to their closures, requiring separate reasoning that connectedness is preserved by closure and that the original overlap witnesses remain in both closures. This is not supplied by the parent's chain argument or obtainable by simply recalling its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_iUnion_closure.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) : IsConnected (⋃ n, closure (A n))
```

Read back by a model that saw only the Lean. *Let X be a topological space and let $(A_n)_{n \in \mathbb{N}}$ be a sequence of subsets of X . Assume that each A_n is connected and that $A_n \cap A_{n+1}$ is nonempty for every $n \in \mathbb{N}$. Then the union $\bigcup_{n \in \mathbb{N}} \overline{A_n}$ of their closures is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Set
open Filter Set TopologicalSpace
open scoped Topology

theorem connected_iUnion_closure {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, closure (A n)) := by
  have hConnected : ∀ n, IsConnected (closure (A n)) := by
    intro n
    exact (hA n).closure
  have hAdjacent : ∀ n, (closure (A n) ∩ closure (A (n + 1))).Nonempty := by
    intro n
    rcases (nonempty_iff_ne_empty.mpr (hAn n)) with ⟨x, hx₁, hx₂⟩
    exact ⟨x, subset_closure hx₁, subset_closure hx₂⟩
  have hChain : ∀ n, (closure (A n) ∩ closure (A (Nat.succ n))).Nonempty := by
    intro n
    simpa [Nat.succ_eq_add_one] using hAdjacent n
  exact IsConnected.iUnion_of_chain hConnected hChain
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__mh__65acdd42

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8c85080e91e6cb12

4.351 351. continuous_fixed_point_on_interval

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous, and let $a \leq b$. If $a \leq f(a)$ and $f(b) \leq b$, show that there is some $x \in [a, b]$ such that $f(x) = x$.

Lean statement

```
theorem continuous_fixed_point_on_interval (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ)
  (hab : a ≤ b) (ha : a ≤ f a) (hb : f b ≤ b) :
  ∃ x ∈ Set.Icc a b, f x = x
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-point theorem from self-maps of $[0,1]$ to arbitrary continuous real functions on any interval, with endpoint inequalities replacing the codomain hypothesis. The parent's endpoint argument for a self-map does not directly solve this broader claim.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed-point result from self-maps of $[0,1]$ to arbitrary continuous real functions on $[a,b]$ satisfying endpoint inequalities. Its proof uses a new interval fixed-point theorem and does not reduce to recalling the parent's subtype-valued argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the fixed-point result from continuous self-maps of $[0,1]$ to arbitrary continuous real functions on $[a,b]$ satisfying endpoint inequalities. Its proof uses a general interval fixed-point theorem and requires reasoning absent from the parent's subtype-specific endpoint argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2344 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_fixed_point_on_interval.extracted_1_1 (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ) (hab : a ≤ b)
(ha : a ≤ f a) (hb : f b ≤ b) : ∃ x ∈ Icc a b, f x = x
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be continuous, and let $a, b \in \mathbb{R}$ satisfy $a \leq b$, $a \leq f(a)$, and $f(b) \leq b$. Then there exists $x \in [a, b]$ such that $f(x)=x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_fixed_point_on_interval (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ)
(hab : a ≤ b) (ha : a ≤ f a) (hb : f b ≤ b) :
  ∃ x ∈ Set.Icc a b, f x = x := by
  have hcont : ContinuousOn f (Set.Icc a b) := hf.continuousOn
  obtain ⟨x, hx, hfixed⟩ :=
    exists_mem_Icc_isFixedPt hcont hab ha hb
  have hxeq : f x = x := hfixed.eq
  exact ⟨x, hx, hxeq⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__mh__6885d5d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6885d5d803bd17ef

4.352 352. unique_fixed_point_interval

The problem

Let $f : [0, 1] \rightarrow [0, 1]$ be continuous, and suppose that for distinct $x, y \in [0, 1]$, the distance between $f(x)$ and $f(y)$ is strictly smaller than the distance between x and y . Prove that f has a unique fixed point.

Lean statement

```
theorem unique_fixed_point_interval (f : {x : ℝ // x ∈ Set.Icc 0 1} → {x : ℝ // x ∈ Set.Icc 0 1}) (hf : Continuous f) (hcontract : ∀ x y,
  x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Supplies the continuous auxiliary function $g(x)=f(x)-x$, endpoint sign bounds, and intermediate-value existence argument; the child adds a separate uniqueness contradiction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness argument: strict distance shrinking contradicts two distinct fixed points, while continuity still supplies existence. This is substantive reasoning absent from the parent and is not a forced wrapper or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence to uniqueness by adding a strict distance-shrinking hypothesis and proving that any two fixed points must coincide. The parent's intermediate-value argument supplies existence, but cannot supply the new uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens fixed-point existence to uniqueness by adding a strict distance-shrinking hypothesis and proving distinct fixed points impossible. The parent's interval/continuity argument supplies existence but does not supply this uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2878 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_fixed_point_interval.extracted_1_1 (f : { x // x ∈ Icc 0 1 } → { x // x ∈ Icc 0 1 }) (hf : Continuous f)
(hcontract : ∀ (x y : { x // x ∈ Icc 0 1 }), x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x
```

Read back by a model that saw only the Lean. Let f be a function from the real interval $[0,1]$ to itself. Assume that f is continuous and that for any distinct x,y in $[0,1]$, the distance between $f(x)$ and $f(y)$ is strictly less than the distance between x and y . Then there exists exactly one x in $[0,1]$ such that $f(x)=x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem unique_fixed_point_interval (f : {x : ℝ // x ∈ Set.Icc 0 1} → {x : ℝ // x ∈ Set.Icc 0 1}) (hf : Continuous f) (hcontract : ∀ x y,
  x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x := by
  classical
  let a : {x : ℝ // x ∈ Set.Icc 0 1} := ⟨0, by exact ⟨le_rfl, zero_le_one⟩⟩
  let b : {x : ℝ // x ∈ Set.Icc 0 1} := ⟨1, by exact ⟨zero_le_one, le_rfl⟩⟩
  let g : {x : ℝ // x ∈ Set.Icc 0 1} → ℝ := fun x => (f x : ℝ) - (x : ℝ)
  have hg_cont : Continuous g := by
    have h_toReal : Continuous fun x : {x : ℝ // x ∈ Set.Icc 0 1} => (f x : ℝ) := continuous_subtype_val.comp hf
    exact h_toReal.sub continuous_subtype_val
  have hga_nonneg : 0 ≤ g a := by
    have hfa_mem : ((f a : ℝ) ∈ Set.Icc 0 1) := (f a).property
    have hfa_lower : 0 ≤ (f a : ℝ) := (Set.mem_Icc.mp hfa_mem).1
    simp [g, a] using hfa_lower
  have hgb_nonpos : g b ≤ 0 := by
    have hfb_mem : ((f b : ℝ) ∈ Set.Icc 0 1) := (f b).property
    have hfb_upper : (f b : ℝ) ≤ 1 := (Set.mem_Icc.mp hfb_mem).2
    have h : (f b : ℝ) - 1 ≤ 0 := sub_nonpos.mpr hfb_upper
    simp [g, b] using h
  have hZeroInRange : (0 : ℝ) ∈ Set.range g := mem_range_of_exists_le_of_exists_ge hg_cont ⟨b, hgb_nonpos⟩ ⟨a, hga_nonneg⟩
  rcases hZeroInRange with ⟨x, hx⟩
  have hRealEq : (f x : ℝ) = (x : ℝ) := by
    have h : (f x : ℝ) - (x : ℝ) = 0 := by simp [g] using hx
    exact sub_eq_zero.mpr h
  have hFixed : f x = x := by
    apply Subtype.ext
    simp using hRealEq
  refine ⟨x, hFixed, ?_⟩
  intro y hy
  by_contra hne
  have hstrict := hcontract x y (Ne.symm hne)
  rw [hFixed, hy] at hstrict
  exact (lt_irrefl _ hstrict)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__mh__9559776e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9559776e2b164cd0

4.353 353. compact_pair_observable_localization

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and the set of pairs $(f(x), g(x))$ for $x \in \overline{W}$ compact.

Lean statement

```
theorem compact_pair_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact ((fun x => (f x, g x)) '' closure W)
```

Parent. Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W})$ compact.

Parent in Lean

```
theorem compact_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact (f '' closure W)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from compactness of a scalar image to compactness of a paired image and requires proving continuity of the product map $x \mapsto (f x, g x)$. This genuinely adds a new proof obligation not supplied by the parent's localization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the compact-image obligation from a scalar continuous image to the image of a pair-valued map. Proving continuity of the product map and applying compactness to it requires reasoning absent from the parent's proof, and the new hypothesis is essential.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the compact-image conclusion from a real-valued image to a pair-valued image. Proving compactness now requires constructing continuity of the paired map from both hypotheses, which the parent's proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2357 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_pair_observable_localization.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
[inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W,
    IsOpen V ∧
      K ⊆ V ∧
        closure V ⊆ W ∧
          IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧ IsCompact ((fun x => (f x, g x)) '' closure W)
```

Read back by a model that saw only the Lean. *For every topological space X that is Hausdorff and locally compact, every compact subset K of X , every open subset U of X with $K \subseteq U$, and every pair of continuous functions $f, g : X \rightarrow \mathbb{R}$, there exist subsets $V, W \subseteq X$ such that V and W are open, $K \subseteq V$, the closure of V is contained in W , the closure of W is compact, the closure of W is contained in U , and the image of the closure of W under the map $x \mapsto (f(x), g(x))$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem compact_pair_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
(K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact ((fun x => (f x, g x)) '' closure W) := by
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hU hKU
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧ closure W ⊆ U ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hU hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  have hPairCont : Continuous (fun x => (f x, g x)) := by
    exact continuous_prodMk.mpr ⟨hf, hg⟩
  have hPairImageCompact : IsCompact ((fun x => (f x, g x)) '' closure W) := by
    exact hWcompact.image hPairCont
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsub, hPairImageCompact⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me.mh.mh__1f2f481b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 1f2f481bb6f693db

4.354 354. compact_observable_range_buffer

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. If $f(K) \subset (a, b)$, show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W}) \subset (a, b)$.

Lean statement

```
theorem compact_observable_range_buffer {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Set.Ioo a b) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    f '' closure W ⊆ Set.Ioo a b
```

Parent. Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W})$ compact.

Parent in Lean

```
theorem compact_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact (f '' closure W)
```

What it contributes. Supplied the two-stage nested open localization with compact outer closure; the mutation replaces the parent's decorative compact-image conclusion by localization inside the continuity-preimage range buffer.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires genuinely new reasoning: it intersects U with the continuous preimage of the prescribed interval and uses the range-buffer hypothesis to localize K there. The parent's nested-neighborhood and compact-image argument does not establish this stronger range containment.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range-buffer argument: it intersects U with the preimage of an interval and uses continuity to ensure the entire compact outer closure maps into that interval. This reasoning is absent from the parent's compact-image conclusion and cannot be recovered by recalling the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range-buffer specialization: it must intersect the domain with the preimage of the prescribed interval and propagate that constraint through nested compact neighborhoods. The parent's compact-image argument does not supply this preimage and range-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2348 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_observable_range_buffer.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
[inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K)
(hU : IsOpen U) (hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Ioo a b) :
  ∃ V W,
    IsOpen V ∧
      K ⊆ V ∧ closure V ⊆ W ∧ IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧ f '' closure W ⊆ Ioo a b
```

Read back by a model that saw only the Lean. *Let X be a topological space that is Hausdorff and locally compact. For sets $K, U \subseteq X$, a function $f : X \rightarrow \mathbb{R}$, and real numbers a, b , suppose K is compact, U is open, $K \subseteq U$, f is continuous, and $f(K) \subseteq (a, b)$. Then there exist sets $V, W \subseteq X$ such that V is open, $K \subseteq V$, the closure of V is contained in W , W is open, the closure of W is compact, the closure of W is contained in U , and $f(\text{closure } W) \subseteq (a, b)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem compact_observable_range_buffer {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
(K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Set.Ioo a b) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
      f '' closure W ⊆ Set.Ioo a b := by
  have hDomainOpen : IsOpen (U ∩ f ⁻¹' Set.Ioo a b) := by
    exact hU.inter (isOpen_Ioo.preimage hf)
  have hKDomain : K ⊆ U ∩ f ⁻¹' Set.Ioo a b := by
    intro x hx
    exact ⟨hKU hx, hbuf ⟨x, hx, rfl⟩⟩
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧
    closure V ⊆ U ∩ f ⁻¹' Set.Ioo a b ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hDomainOpen hKDomain
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧
    closure W ⊆ U ∩ f ⁻¹' Set.Ioo a b ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hDomainOpen hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  have hWsubU : closure W ⊆ U := by
    intro x hx
    exact (hWsub hx).1
  have hRange : f '' closure W ⊆ Set.Ioo a b := by
    intro y hy
    rcases hy with ⟨x, hx, rfl⟩
    exact (hWsub hx).2
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsubU, hRange⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me.mh.mh__1f30fe9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1f30fe9a994e1537

4.355 355. compact_open_refinement

The problem

Let K be a compact subset of an open set U in a locally compact regular Hausdorff space. Show that there is an open set V such that $K \subset V$, $\bar{V} \subset U$, and $\bar{V}$ is compact.

Lean statement

```
theorem compact_open_refinement {X : Type*}
  [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X] [RegularSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V)
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces refinement around a single point with a simultaneous refinement of an arbitrary compact set K , requiring a compactness-based finite-cover argument and additional regularity structure. Its proof does not reuse the parent's pointwise separation construction and instead establishes a genuinely stronger compact-set theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the pointwise local-compactness conclusion into a compact-set refinement theorem: an arbitrary compact K must be enlarged to an open V whose compact closure remains inside U . This requires a distinct compact-set refinement argument and cannot be obtained by recalling the parent's point-neighborhood construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from a single point to an arbitrary compact set and requires an open neighborhood whose closure remains compact and lies inside the target. This is not recoverable by recalling the parent's pointwise separation-and-compactness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_open_refinement.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] [inst_1 : T2Space X]
[inst_2 : LocallyCompactSpace X] [inst_3 : RegularSpace X] (K U : Set X) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) : ∃ V, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V)
```

Read back by a model that saw only the Lean. *Let X be a topological space that is Hausdorff, locally compact, and regular. For sets $K, U ⊆ X$, suppose K is compact, U is open, and $K ⊆ U$. Then there exists an open set $V ⊆ X$ such that $K ⊆ V$, the closure of V is contained in U , and the closure of V is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem compact_open_refinement {X : Type*}
[TopologicalSpace X] [T2Space X] [LocallyCompactSpace X] [RegularSpace X]
(K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
have hRefinement := exists_open_between_and_isCompact_closure hK hU hKU
rcases hRefinement with ⟨V, hVopen, hKV, hclosureU, hcompact⟩
have hConclusion :
  IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
  exact ⟨hVopen, hKV, hclosureU, hcompact⟩
exact ⟨V, hConclusion⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__23d513d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 23d513d89cd7c9b1

4.356 356. countable_basis_compact_closure_subset

The problem

Let X be a Hausdorff locally compact space, and let $(B_n)_{n \in \mathbb{N}}$ be a countable open neighborhood basis at x . Show that for every open neighborhood U of x , some B_n has compact closure contained in U .

Lean statement

```
theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℳ X, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Preserves the Hausdorff/local-compact neighborhood argument while replacing the unrestricted witness by a member of the supplied countable local basis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new basis-refinement argument: after obtaining an open neighborhood with compact closure, one must select an indexed basis element inside it and transfer compactness via closure monotonicity. The parent's direct construction of an open set does not supply this index-selection step, and the local compactness hypothesis replaces the parent's explicit compact-neighborho

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a countable neighborhood basis and must select an indexed basis element whose closure remains compact and lies in U . This requires a new basis-refinement argument and does not follow by recalling the parent's arbitrary-neighborhood construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely substantive countable-basis specialization: it must select an indexed basis element contained in an open neighborhood whose compact closure lies in U . The parent's separation-and-finite-subcover argument does not provide this indexed selection, and the child instead uses local compactness plus basis refinement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2892 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem countable_basis_compact_closure_subset.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  [inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (x : X) (B : ℕ → Set X) (hBopen : ∀ (n : ℕ), IsOpen (B n))
  (hxB : ∀ (n : ℕ), x ∈ B n) (hBbasis : ∀ s ∈ ℳ x, ∃ n, B n ⊆ s) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

Read back by a model that saw only the Lean. *Let X be a locally compact Hausdorff topological space, let x be a point of X , and let $B : \mathbb{N} \rightarrow \text{Set } X$ be a sequence of subsets of X such that every $B n$ is open, x belongs to every $B n$, and for every neighborhood s of x there exists n with $B n \subseteq s$. Let U be an open subset of X containing x . Then there exists $n \in \mathbb{N}$ such that $B n$ is open, $x \in B n$, the closure of $B n$ is compact, and the closure of $B n$ is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℳ x, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U := by
  have hcompact_x : IsCompact ({x} : Set X) := isCompact_singleton
  have hsingletonU : ({x} : Set X) ⊆ U := by
    rintro y rfl
    exact hxU
  obtain ⟨W, hWopen, hsingletonW, hclWsub, hclWcompact⟩ :=
    exists_open_between_and_isCompact_closure hcompact_x hU hsingletonU
  have hxW : x ∈ W := hsingletonW (mem_singleton x)
  have hWnhds : W ∈ ℳ x := hWopen.mem_nhds hxW
  obtain ⟨n, hn⟩ := hBbasis W hWnhds
  have hclsub : closure (B n) ⊆ closure W := closure_mono hn
  have hclU : closure (B n) ⊆ U := hclsub.trans hclWsub
  have hclcompact : IsCompact (closure (B n)) :=
    hclWcompact.of_isClosed_subset isClosed_closure hclsub
  exact ⟨n, hBopen n, hxB n, hclcompact, hclU⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__cb92bce9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cb92bce9f8dccfb1

4.357 357. prescribed_pair_exact_support

The problem

Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates with $x_i \neq 0$ and $x_j \neq 0$. Prove that there is a unit vector orthogonal to x whose support is exactly $\{i, j\}$, with both prescribed coordinates nonzero.

Lean statement

```
theorem prescribed_pair_exact_support
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hi j : i ≠ j)
  (hxi : x i ≠ 0)
  (hxj : x j ≠ 0) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) ∧
    y i ≠ 0 ∧
    y j ≠ 0
```

Parent. Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates. Prove that there is a unit vector supported only on i and j and orthogonal to x .

Parent in Lean

```
theorem prescribed_pair_unit_orthogonal
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hi j : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

What it contributes. Reuse the prescribed-pair witness and normalization construction; strengthen the proof with explicit nonvanishing hypotheses and establish exact support plus nonzero values at both selected coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes to nonzero prescribed coordinates and proves a new exact-support characterization, including both coordinate nonvanishing conditions. The parent supplies normalization and orthogonality, but not the support equality or its required coordinate arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to nonzero prescribed coordinates and proves the genuinely stronger exact-support characterization, including both coordinates being nonzero. This requires new support and nonvanishing reasoning beyond the parent's merely contained-support construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to the nonzero-coordinate case and strengthens the conclusion to exact support, requiring proofs that both prescribed coordinates remain nonzero and all others vanish. The parent's construction only establishes support containment and allows a zero coordinate, so recalling it does not discharge the new support-equality obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2375 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_exact_support.extracted_1_1 (n : ℕ) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n) (hij : i ≠ j)
(hxi : x.ofLp i ≠ 0) (hxj : x.ofLp j ≠ 0) :
  ∃ y, ‖y‖ = 1 ∧ ⟨x, y⟩_ℝ = 0 ∧ {k | y.ofLp k ≠ 0} = {i, j} ∧ y.ofLp i ≠ 0 ∧ y.ofLp j ≠ 0
```

Read back by a model that saw only the Lean. *For every natural number n , every vector x in the Euclidean space $\mathbb{R}^n$ indexed by $\text{Fin } n$, and every pair of distinct indices $i, j \in \text{Fin } n$ such that the i -th and j -th coordinates of x are both nonzero, there exists a vector $y \in \mathbb{R}^n$ with Euclidean norm 1, orthogonal to x , whose nonzero coordinates are exactly those indexed by i and j , and whose i -th and j -th coordinates are both nonzero.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_exact_support
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j)
  (hxi : x i ≠ 0)
  (hxj : x j ≠ 0) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) ∧
    y i ≠ 0 ∧
    y j ≠ 0 := by
  classical
  let z := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
  have hji : j ≠ i := Ne.symm hij
  have hz_i : z i = x j := by
    simp [z, EuclideanSpace.single_apply, hij, hji]
  have hz_j : z j = -x i := by
    simp [z, EuclideanSpace.single_apply, hij, hji]
  have hz : z ≠ 0 := by
    intro hzero
    have hzero_j := congrArg (fun v => v j) hzero
```

```

have : x i = 0 := by
  simpa [hz_j] using hzero_j
exact hxi this
have hnormz : ‖z‖ ≠ 0 := norm.ne_zero_iff.mpr hz
let y := (‖z‖ : ℝ)⁻¹ • z
have hnorm : ‖y‖ = 1 := by
  simpa [y] using (norm.smul_inv_norm hz)
have horthz : ⟨x, z⟩_R = 0 := by
  simp [z, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]
have horth : ⟨x, y⟩_R = 0 := by
  change ⟨x, (‖z‖ : ℝ)⁻¹ • z⟩_R = 0
  rw [real_inner_smul_right, horthz, mul_zero]
have hy_i : y i ≠ 0 := by
  simp [y, hz_i, hnormz, hxj]
have hy_j : y j ≠ 0 := by
  simp [y, hz_j, hnormz, hxi]
have hsupport : {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) := by
  ext k
  constructor
  · intro hk
    change y k ≠ 0 at hk
    by_cases hki : k = i
    · simp [hki]
    · by_cases hkj : k = j
      · simp [hkj]
      · have hyzero : y k = 0 := by
          simp [y, z, EuclideanSpace.single_apply, hki, hkj]
          exact False.elim (hk hyzero)
  · intro hk
    simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at hk
    rcases hk with rfl | rfl
    · exact hy_i
    · exact hy_j
exact ⟨y, hnorm, horth, hsupport, hy_i, hy_j⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me.me.mh__9c4a022c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9c4a022cb361de77

4.358 358. irrational_quotient_injective

The problem

For an irrational real number x , division by nonzero rational numbers is injective: if $x/y = x/z$, then $y = z$.

Lean statement

```
theorem irrational_quotient_injective
  (x : ℝ) (hx : Irrational x) :
  Function.Injective (fun y : {q : ℚ // q ≠ 0} => x / (y.1 : ℝ))
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG
model gpt-5.6-luna · saw parents, child
The child changes the conclusion from irrationality preservation to injectivity over nonzero rational divisors. Its proof requires cross-multiplication and cancellation using irrationality, which is not supplied by the parent’s equivalence argument.

Re-judged, first pass STRONG
model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results
The child changes an irrationality equivalence into an injectivity statement and requires a new cancellation argument: cross-multiplication, zero-product reasoning, and use of irrationality to show the denominator difference vanishes. The parent’s rational-scaling equivalence does not directly supply this proof.

Re-judged, second pass STRONG
model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results
The child changes the equivalence into an injectivity theorem for division by nonzero rationals. Its proof requires cross-multiplication and cancellation using the irrational numerator’s nonzeroness, which is not supplied by the parent’s irrationality-transport argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A quantifies over nonzero rational numbers, while Statement B quantifies over natural numbers, omitting noninteger and negative rationals and including 0, where division is undefined.

Goal round-trip: what Lean elaborated

```
theorem irrational_quotient_injective.extracted_1_1 (x : ℝ) (hx : Irrational x) :  
  Function.Injective fun y => x / ↑↑y
```

Read back by a model that saw only the Lean. For every irrational real number x , the function that maps each natural number y to x divided by y , viewed as a real number, is injective; that is, whenever x divided by y equals x divided by z , we have $y = z$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_quotient_injective  
  (x : ℝ) (hx : Irrational x) :  
    Function.Injective (fun y : {q : ℚ // q ≠ 0} => x / (y.1 : ℝ)) := by  
  intro y z hEq  
  apply Subtype.ext  
  have hy : (y.1 : ℝ) ≠ 0 := by  
    exact_mod_cast y.property  
  have hz : (z.1 : ℝ) ≠ 0 := by  
    exact_mod_cast z.property  
  have hcross : x * (z.1 : ℝ) = x * (y.1 : ℝ) :=  
    (div_eq_div_iff hy hz).mp hEq  
  have hprod : x * ((z.1 : ℝ) - (y.1 : ℝ)) = 0 := by  
    nlinarith [hcross]  
  have hdiff : (z.1 : ℝ) - (y.1 : ℝ) = 0 :=  
    (mul_eq_zero.mp hprod).resolve_left hx.ne_zero  
  have hcast : (z.1 : ℝ) = (y.1 : ℝ) :=  
    sub_eq_zero.mp hdiff  
  exact_mod_cast hcast.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.mh__10b2735a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 10b2735aeff611d2

4.359 359. irrational_mul_ratCast_iff_of_irrational

The problem

Let x be irrational and r rational. Prove that rx is irrational if and only if $r \neq 0$.

Lean statement

```
theorem irrational_mul_ratCast_iff_of_irrational (x : ℝ) (r : ℚ) (hx : Irrational x) : Irrational (x * r) ↔ r ≠ 0
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way preservation result into an iff and requires a new converse argument: if the rational multiplier is zero, the product is rational, contradicting its irrationality. The parent's 'mul_ratCast' application supplies only the forward direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the implication into an iff and adds the substantive converse that irrationality of the product forces the rational multiplier to be nonzero, proved by contradiction using irrationality of zero. Although one direction reuses the parent lemma, the new converse requires reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the implication into an iff and adds the converse argument that a zero rational multiplier makes the product rational, which is not supplied by the parent's forward lemma. The proof therefore requires genuinely new reasoning beyond recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2140 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_mul_ratCast_iff_of_irrational.extracted_1_1 (x : ℝ) (r : ℚ) (hx : Irrational x) :
  Irrational (x * ↑r) ↔ r ≠ 0
```

Read back by a model that saw only the Lean. For a real number x that is irrational and a rational number r , the product of x and the real-number cast of r is irrational if and only if r is nonzero.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_mul_ratCast_iff_of_irrational (x : ℝ) (r : ℚ) (hx : Irrational x) : Irrational (x * r) ↔ r ≠ 0 := by
  constructor
  · intro hprod hr
    subst r
    have hzero : Irrational (0 : ℝ) := by
      simpa using hprod
    norm_num at hzero
  · intro hr
    exact hx.mul_ratCast hr
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__4decaa2e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 4decaa2ed2957406

4.360 360. irrational_add_rational

The problem

If x is irrational and r is rational, prove that $x + r$ is irrational.

Lean statement

```
theorem irrational_add_rational
(x : ℝ)
(r : ℚ)
: Irrational x → Irrational (x + r) := by
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes nonzero rational scaling into rational translation, requiring a different irrationality-preservation theorem; the parent’s ‘mul_ratCast’ argument cannot prove the additive conclusion. The proof is complete and uses no redundant hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes nonzero rational scaling to rational translation, requiring a different irrationality theorem and eliminating the parent’s nonzero hypothesis. The parent’s proof and closing lemma do not solve this additive claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes rational scaling to rational translation, so the parent’s nonzero-multiplier argument and ‘mul_ratCast’ lemma no longer apply. Proving irrationality is preserved under addition requires a distinct mathematical lemma and proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2746 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_add_rational.extracted_1_1 (x : ℝ) (r : ℚ) : Irrational x → Irrational (x + ↑r)
```

Read back by a model that saw only the Lean. *For every real number x and rational number r , if x is irrational, then x plus the real number represented by r is irrational.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_add_rational
(x : ℝ)
(r : ℚ)
: Irrational x → Irrational (x + r) := by
  intro hx
  exact hx.add_ratCast r
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__69129ea3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 69129ea3cd3a3493

4.361 361. irrational_inv_of_irrational

The problem

If x is a nonzero irrational real number, prove that its reciprocal x^{-1} is irrational.

Lean statement

```
theorem irrational_inv_of_irrational {x : ℝ} (hx : Irrational x) : Irrational x⁻¹
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

What it contributes. Transferred the parent’s irrationality-by-contradiction structure, while replacing rational multiplication with reciprocal rationality and explicitly using the nonzero hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different specialization: inversion preserves irrationality, relying on inverse-specific reasoning and the fact that irrational reals are nonzero. The parent’s rational-scaling lemma does not directly supply this conclusion.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes multiplication by a nonzero rational into inversion, relying on a distinct irrational-inverse theorem. The parent’s ‘mul_ratCast’ argument cannot establish irrationality of the reciprocal.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different preservation result: inversion of an irrational real remains irrational. The parent’s rational-scaling lemma and proof do not supply this claim, and the irrationality hypothesis itself supplies the needed nonzero fact.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2933 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_inv_of_irrational.extracted_1_1 {x : ℝ} (hx : Irrational x) (hx0 : x ≠ 0) : Irrational x-1
```

Read back by a model that saw only the Lean. *For a real number x , if x is irrational and $x \neq 0$, then its reciprocal $1/x$ is irrational.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_inv_of_irrational {x : ℝ} (hx : Irrational x) : Irrational x-1 := by
  exact hx.inv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__9f344909

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 74e5ab2986fb0495

4.362 362. affine_irrational

The problem

If x is irrational, a and b are rational, and $a \neq 0$, prove that $ax + b$ is irrational.

Lean statement

```
theorem affine_irrational (x : ℝ) (a b : ℚ) (ha : a ≠ 0) : Irrational x → Irrational ((a : ℝ) * x + b)
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1.1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

What it contributes. Preserves the parent’s real irrational input and nonzero rational multiplier domain while extending the transformation with a rational translation; the parent’s contradiction idea is realized through explicit subtraction and division.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a rational translation and proves irrationality by recovering x through the nonzero affine coefficient, then deriving a rational representation of x for contradiction. This reasoning is not supplied by the parent’s multiplicative lemma and is distinct from the already-kept scaling and inversion results.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a rational translation to the nonzero rational scaling. Its proof must derive a rational preimage under the affine map, using subtraction and division, which the parent’s multiplication-only lemma does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves irrationality under a nondegenerate rational affine transformation, requiring both translation handling and inversion of the affine map; the parent’s multiplicative scaling lemma alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2940 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_irrational.extracted_1_1 (x : ℝ) (a b : ℚ) (ha : a ≠ 0) :  
  Irrational x → Irrational (↑a * x + ↑b)
```

Read back by a model that saw only the Lean. *For a real number x and rational numbers a and b with $a \neq 0$, if x is irrational, then $a x + b$ is irrational.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem affine_irrational (x : ℝ) (a b : ℚ) (ha : a ≠ 0) : Irrational x → Irrational ((a : ℝ) * x + b) := by  
  intro hx hbad  
  obtain ⟨q, hq⟩ := hbad  
  have haR : (a : ℝ) ≠ 0 := by  
    exact_mod_cast ha  
  have hsub : (q : ℝ) - b = (a : ℝ) * x := by  
    linarith [hq]  
  have hxa : x = ((q : ℝ) - b) / a := by  
    apply (eq_div_iff haR).2  
    calc  
      x * (a : ℝ) = (a : ℝ) * x := by ring  
      _ = (q : ℝ) - b := hsub.symm  
  apply hx  
  refine ⟨(q - b) / a, ?_⟩  
  rw [hxa]  
  push_cast  
  rfl
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__a86cb245

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a86cb2458ef9e167

4.363 363. irrational_mul_ratCast_characterization

The problem

For a real number x and a rational number y , prove that xy is irrational if and only if $y \neq 0$ and x is irrational.

Lean statement

```
theorem irrational_mul_ratCast_characterization (x : ℝ) (y : ℚ) : Irrational (x * y) ↔ y ≠ 0 ∧ Irrational x
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG
model gpt-5.6-luna · saw parents, child

The child proves a genuine two-way characterization: from $\text{Irrational } (x * y)$ it must derive both $y \neq 0$ and $\text{Irrational } x$, using cancellation and contradiction. The parent only supplies the forward scaling implication, so memorizing its proof does not solve the new direction.

Re-judged, first pass STRONG
model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse obligation: from irrationality of $x*y$ it derives both $y \neq 0$ and $\text{Irrational } x$, yielding an iff characterization. This is not supplied by the parent's one-way scaling proof, despite reusing that proof for the forward direction.

Re-judged, second pass STRONG
model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way preservation result into a full characterization. Its reverse direction requires proving both that the rational multiplier is nonzero and that irrationality of x follows by cancellation, obligations absent from the parent's forward-only proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2001 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_mul_ratCast_characterization.extracted_1_1 (x : ℝ) (y : ℚ) :  
  Irrational (x * ↑y) ↔ y ≠ 0 ∧ Irrational x
```

Read back by a model that saw only the Lean. *For a real number x and a rational number y , the product of x and the real-number cast of y is irrational if and only if y is nonzero and x is irrational.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_mul_ratCast_characterization (x : ℝ) (y : ℚ) :  
  Irrational (x * y) ↔ y ≠ 0 ∧ Irrational x := by  
  constructor  
  · intro hxy  
    have hx : Irrational x := hxy.of_mul_ratCast  
    have hy : y ≠ 0 := by  
      intro hy0  
      subst hy0  
      simpa using hxy  
    exact ⟨hy, hx⟩  
  · intro h  
    have hscaled : Irrational (x * y) := h.2.mul_ratCast h.1  
    exact hscaled
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__ccdca90e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 79d3091bb0fb36b1

4.364 364. countable_trace_basis_countable_union

The problem

Let K be a compact metric space, A a subset of K , and B a countable topological basis for K . Show that the traces of the sets in B form a countable basis for A , and that every open subset of A is a countable union of such traces.

Lean statement

```
theorem countable_trace_basis_countable_union {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  ∀ V : Set A, IsOpen V →
    ∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B,
      s.Countable ∧ V = ⋃ U ∈ s, U
```

Parent. Let K be a compact metric space, A a closed subset of K , and B a countable topological basis for K . Show that the traces of the sets in B on A form a countable topological basis for A .

Parent in Lean

```
theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive countable-union representation for every open subset of the subtype. Proving it requires deriving second countability and applying the basis union theorem, which is not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive open-set decomposition claim: every open subset of the subspace is a countable union of traced basis elements. Proving it requires deriving second countability and applying a countable-basis union theorem, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive theorem: every open subset of the subspace is a countable union of members of the traced basis. Proving this requires deriving second countability and applying a countable-basis union lemma, which is not supplied by the parent's countability-and-basis proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2367 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem countable_trace_basis_countable_union.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} {B : Set (Set K)} (hBcount : B.Countable) (hBbasis : IsTopologicalBasis B) :
((fun U => Subtype.val ^-1' U) '' B).Countable ∧
IsTopologicalBasis ((fun U => Subtype.val ^-1' U) '' B) ∧
∀ (V : Set ↑A), IsOpen V → ∃ s ⊆ (fun U => Subtype.val ^-1' U) '' B, s.Countable ∧ V = ⋃ U ∈ s, U
```

Read back by a model that saw only the Lean. Let K be a compact metric space, let A be a subset of K , and let B be a countable topological basis for K . Then the collection of traces of the sets in B on the subtype of points of A , namely $\{x : K \mid x \in A\}$ with each trace given by $\{x \in A \mid x \in U\}$ for $U \in B$, is countable and is a topological basis for the subtype. Moreover, every open set V in this subtype is the union of some countable subcollection s of these traces: there exists a countable set s contained in the collection of traces such that $V = \bigcup \{U \in s\} U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem countable_trace_basis_countable_union {K : Type*} [MetricSpace K] [CompactSpace K]
{A : Set K} {B : Set (Set K)}
(hBcount : Set.Countable B)
(hBbasis : TopologicalSpace.IsTopologicalBasis B) :
Set.Countable
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) ∧
TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) ∧
∀ V : Set A, IsOpen V →
∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B,
s.Countable ∧ V = ⋃ U ∈ s, U := by
let I : SecondCountableTopology K := hBbasis.secondCountableTopology hBcount
have hcount : Set.Countable
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) :=
hBcount.image (fun U : Set K => (Subtype.val : A → K) ^-1' U)
have hbasis : TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) :=
hBbasis.induced (Subtype.val : A → K)
have hunion : ∀ V : Set A, IsOpen V →
∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B,
s.Countable ∧ V = ⋃ U ∈ s, U := by
intro V hV
exact hbasis.exists_countable_biUnion_of_isOpen hV
exact ⟨hcount, hbasis, hunion⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.me.mh__75316feb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 75316febde0967fe

4.365 365. arbitrary_subspace_countable_trace_basis

The problem

Let K be a compact metric space and let A be any subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Lean statement

```
theorem arbitrary_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] (A : Set K) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Preserves the compact metric ambient space and trace-basis conclusion, while replacing closed-subspace inheritance with the direct ambient countable-basis construction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes closedness and changes the construction: it uses a countable ambient basis of K and proves its traces form a basis on arbitrary A via the induced-topology theorem. The parent's compact-subtype basis and ambient lifting argument does not directly supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the closedness hypothesis and proves the stronger result for every subspace by using a countable ambient basis and the induced-topology basis theorem. This requires a genuinely different argument from the parent's compactness-based basis construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely removes the closedness hypothesis and proves the result by tracing a countable ambient basis of the compact metric space, rather than using compactness and a basis of the closed subspace. This requires a materially different argument and is not recoverable by recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2353 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem arbitrary_subspace_countable_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] (A : Set K) :
  ∃ B, B.Countable ∧ (∀ U ∈ B, IsOpen U) ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *For every subset A of a compact metric space K , there exists a countable family B of open subsets of K such that the family of their traces on the subtype A , namely $\{x \in A \mid x \in U\}$ for $U \in B$, is a topological basis for A with its subspace topology.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem arbitrary_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] (A : Set K) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  have hCount : (countableBasis K).Countable := countable_countableBasis K
  have hOpen : ∀ U ∈ countableBasis K, IsOpen U := by
    intro U hU
    exact isOpen_of_mem_countableBasis hU
  have hTrace : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) (countableBasis K)) := by
    simpa only using
      (TopologicalSpace.IsTopologicalBasis.induced (Subtype.val : A → K)
        (isBasis_countableBasis K))
  exact ⟨countableBasis K, hCount, hOpen, hTrace⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh__1c9af0d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1c9af0d8112d4ffa

4.366 366. closed_subspace_countable_separating_trace_basis

The problem

Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Lean statement

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Supplied the compact-subtype countable basis, ambient open representatives, countability argument, and trace-basis construction; the mutation adds point separation via metric balls.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The added point-separation conclusion is not a trivial conjunction: it requires a new metric argument using positive distance, open balls, and basis refinement to construct a separating trace. The parent proof only constructs the countable trace basis and does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine countable separation property for distinct points, requiring a new metric-ball and basis-refinement argument absent from the parent proof. The existing basis construction alone does not establish this claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine point-separation theorem: it uses positive distance, an open metric ball, and basis refinement to find a basis element containing one point but not the other. The parent's countable trace-basis construction alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2745 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_separating_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U
```

Read back by a model that saw only the Lean. *Let K be a metrizable compact topological space, and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that every U in B is open in K , the collection of subsets of A given by the preimages of the members of B under the inclusion map $A \rightarrow K$ is a topological basis for A with its subspace topology, and for any two distinct points x and y of A , there is some U in B such that x belongs to the preimage of U in A while y does not.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) := by
  classical
  letI : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  obtain ⟨C, hCcount, hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧
    (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
    x ∈ (Subtype.val : A → K) ⁻¹' U ∧
    y ∉ (Subtype.val : A → K) ⁻¹' U := by
    intro x y hxy
    have hxypos : 0 < dist x y := dist_pos.mpr hxy
```

```

have hxball : x ∈ Metric.ball x (dist x y) := by
  rw [Metric.mem_ball]
  simpa using hxypos
have hyball : y ∉ Metric.ball x (dist x y) := by
  intro hy
  have hy' : dist x y < dist x y := by
    simpa [Metric.mem_ball, dist_comm] using hy
  exact (lt_irrefl _ hy')
have hopen : IsOpen (Metric.ball x (dist x y)) := Metric.isOpen_ball
obtain ⟨c, hc, hxc, hsub⟩ := hCbasis.exists_subset_of_mem_open hxball hopen
have hg : g c = f c hc := by
  simp [g, dif_pos hc]
refine ⟨f c hc, ⟨c, hc, hg⟩, ?_, ?_⟩
· rw [hftrace c hc]
  exact hxc
· intro hy
  have hyc : y ∈ c := by
    rw [← hftrace c hc]
    exact hy
  exact hyball (hsub hyc)
refine ⟨B, hBcount, hBopen, ?_, hSeparate⟩
rw [htrace]
exact hCbasis

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh__f713c059

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f713c059eca8929b

4.367 367. proper_pseudoMetricSpace_has_countable_basis

The problem

Prove that every proper pseudo-metric space has a countable base.

Lean statement

```
theorem proper_pseudoMetricSpace_has_countable_basis {K : Type*} [PseudoMetricSpace K] [ProperSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from compact metric spaces to proper pseudometric spaces. Its proof requires the distinct theorem that proper spaces are second-countable, rather than merely reusing compactness-based inference.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the result from compact metric spaces to proper pseudometric spaces, so the parent's compactness-based instance argument does not apply. Its proof requires invoking the distinct theorem that proper spaces are second-countable and constructing the canonical countable basis.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes compact metric spaces to proper pseudo-metric spaces and must invoke the distinct theorem that proper spaces are second-countable. The parent's compactness-based instance argument does not supply this result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2950 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem proper_pseudoMetricSpace_has_countable_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : PseudoMetricSpace K]
  [inst_1 : ProperSpace K] : ∃ B, B.Countable ∧ IsTopologicalBasis B
```

Read back by a model that saw only the Lean. For every proper pseudometric space K , there exists a countable collection

B of subsets of K such that B is a topological basis for K .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem proper_pseudoMetricSpace_has_countable_basis {K : Type*} [PseudoMetricSpace K] [ProperSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
  let I : SecondCountableTopology K := secondCountable_of_proper
  have hCount : (TopologicalSpace.countableBasis K).Countable :=
    TopologicalSpace.countable_countableBasis K
  have hBasis : IsTopologicalBasis (TopologicalSpace.countableBasis K) :=
    TopologicalSpace.isBasis_countableBasis K
  exact ⟨TopologicalSpace.countableBasis K, hCount, hBasis⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__mh__8949e07b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8949e07b76d99ef5

4.368 368. compact_metric_countable_dense_basis

The problem

Prove that every compact metric space has a countable dense subset and a countable base whose every nonempty member meets that dense subset.

Lean statement

```
theorem compact_metric_countable_dense_basis {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ D : Set K, D.Countable ∧ Dense D ∧
  ∃ B : Set (Set K), B.Countable ∧ IsTopologicalBasis B ∧
  ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive countable dense subset and proves that every nonempty basis element meets it, requiring separability and a density–openness argument absent from the parent. The parent’s basis result is reused only as one component of a genuinely expanded conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new existence requirement: a countable dense subset and a proof that it meets every nonempty basis element. This requires separability and the dense-set/open-set intersection argument, which are absent from the parent’s basis-extraction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new existence-and-incidence claim: it must construct a countable dense set and prove every nonempty basis element intersects it. The parent only supplies countable-basis existence, so its proof does not discharge the new density argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2926 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_metric_countable_dense_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] :
  ∃ D, D.Countable ∧ Dense D ∧ ∃ B, B.Countable ∧ IsTopologicalBasis B ∧ ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty
```

Read back by a model that saw only the Lean. *For every compact metric space K , there exists a countable dense subset D of K and a countable collection B of subsets of K that is a topological basis, such that every nonempty U in B has nonempty intersection with D .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_metric_countable_dense_basis {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ D : Set K, D.Countable ∧ Dense D ∧
    ∃ B : Set (Set K), B.Countable ∧ IsTopologicalBasis B ∧
      ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  letI : SecondCountableTopology K := hSecond
  have hSep : SeparableSpace K := inferInstance
  letI : SeparableSpace K := hSep
  obtain ⟨D, hDcount, hDdense⟩ := TopologicalSpace.exists_countable_dense K
  obtain ⟨B, hBcount, -, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hIncidence : ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
    intro U hU hUne
    have hMeet : (U ∩ D).Nonempty :=
      hDdense.inter_open_nonempty U (hBbasis.isOpen hU) hUne
    simpa [Set.inter_comm] using hMeet
  exact ⟨D, hDcount, hDdense, B, hBcount, hBbasis, hIncidence⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__mh__b74d259e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b74d259e7d109c53

4.369 369. three_disjoint_perfect_subsets

The problem

Let A be an uncountable closed subset of a separable metric space. Prove that A contains three pairwise disjoint nonempty perfect subsets.

Lean statement

```
theorem three_disjoint_perfect_subsets (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ F : Fin 3 → Set X,
    (∀ i, F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧
    Pairwise (fun i j => Disjoint (F i) (F j))
```

Parent. Let A be an uncountable closed subset of a separable metric space. Prove that A contains an uncountable perfect subset.

Parent in Lean

```
theorem uncountable_perfect_subset (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, P ⊆ A ∧ Perfect P ∧ ¬ Set.Countable P
```

What it contributes. Supplied the initial nonempty perfect subset of the closed uncountable set A ; repeated perfect-set splitting produces the $\text{Fin } 3$ family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child substantially strengthens the parent from one uncountable perfect subset to three nonempty, pairwise disjoint perfect subsets. Its proof requires iterative small-diameter splitting and new disjointness arguments, so recalling the parent does not supply the result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent from one uncountable perfect subset to three pairwise disjoint nonempty perfect subsets contained in the closed set. Its proof requires iterative perfect-set splitting and a new finite-index disjointness argument, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing three pairwise disjoint nonempty perfect subsets via repeated perfect-set splitting, containment propagation, and finite-index pairwise reasoning. The parent only supplies existence of one uncountable perfect subset and does not provide this disjoint-family construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2800 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem three_disjoint_perfect_subsets.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) (hunc : ¬A.Countable) :
  ∃ F, (∀ (i : Fin 3), F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧ Pairwise fun i j => Disjoint (F i) (F j)
```

Read back by a model that saw only the Lean. *For every separable metric space X and every closed, uncountable subset A of X , there exists a family of three subsets $F(0)$, $F(1)$, and $F(2)$ of X such that each $F(i)$ is contained in A , is perfect, and is nonempty, and any two subsets $F(i)$ and $F(j)$ with $i \neq j$ are disjoint.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem three_disjoint_perfect_subsets (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ F : Fin 3 → Set X,
    (∀ i, F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧
    Pairwise (fun i j => Disjoint (F i) (F j)) := by
  classical
  obtain ⟨P, hPperf, hPne, hPsub⟩ :=
    exists_perfect_nonempty_of_isClosed_of_not_countable (C := A) hA hunc
  obtain ⟨P0, P1, h0, h1, hd01⟩ :=
    Perfect.small_diam_splitting hPperf hPne (ε := 1) (by norm_num)
  obtain ⟨P10, P11, h10, h11, hd1011⟩ :=
    Perfect.small_diam_splitting h1.1 h1.2.1 (ε := 1) (by norm_num)
  have h0sub : P0 ⊆ A := h0.2.2.1.trans hPsub
  have h10sub : P10 ⊆ A := (h10.2.2.1.trans h1.2.2.1).trans hPsub
  have h11sub : P11 ⊆ A := (h11.2.2.1.trans h1.2.2.1).trans hPsub
  have hd010 : Disjoint P0 P10 := by
    refine Set.disjoint_left.2 ?_
    intro x hx0 hx10
    exact Set.disjoint_left.1 hd01 hx0 (h10.2.2.1 hx10)
  have hd011 : Disjoint P0 P11 := by
    refine Set.disjoint_left.2 ?_
    intro x hx0 hx11
    exact Set.disjoint_left.1 hd01 hx0 (h11.2.2.1 hx11)
  have hd100 : Disjoint P10 P0 := hd010.symm
  have hd110 : Disjoint P11 P0 := hd011.symm
  have hd110 : Disjoint P11 P0 := hd011.symm
  let F : Fin 3 → Set X := fun i =>
    if i = 0 then P0 else if i = 1 then P10 else P11
  have hFpair : Pairwise (fun i j => Disjoint (F i) (F j)) := by
    intro i j hij
    fin_cases i <|> fin_cases j <|> simp_all [F]
  refine ⟨F, ?_, hFpair⟩
  intro i
  fin_cases i <|> simp_all [F]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__00c8e1e0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 00c8e1e07a25fec4

4.370 370. Rudin_exercise_2_28_mutation

The problem

Prove that every closed set in a separable metric space is either at most countable or contains a nonempty perfect subset.

Lean statement

```
theorem Rudin_exercise_2_28_mutation (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  A.Countable ∨ ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the decomposition conclusion into a genuine countability-versus-perfect-subset dichotomy. Beyond invoking the parent theorem, its proof requires a nontrivial case split and derives that the countable component covers A when the perfect component is empty.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a genuine dichotomy: either the closed set is countable or it contains a nonempty perfect subset. Proving this requires a new nonemptiness case split and deriving countability when the perfect component is empty, which the parent's direct repackaging proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a substantive dichotomy: either the entire closed set is countable or it contains a nonempty perfect subset. Proving this requires a nontrivial case split on the perfect component and subset/countability reasoning not present in the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2785 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Rudin_exercise_2_28_mutation.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) : A.Countable ∨ ∃ P, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Read back by a model that saw only the Lean. *Let X be a separable metric space, and let A be a closed subset of X . Then either A is countable, or there exists a nonempty perfect subset P of X such that P is contained in A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem Rudin_exercise_2_28_mutation (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  A.Countable ∨ ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A := by
  classical
  obtain ⟨C, P, hC, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  by_cases hPne : P.Nonempty
  · right
    refine ⟨P, hP, hPne, ?_⟩
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  · left
    have hPsub : P ⊆ C := by
      intro x hx
      exact False.elim (hPne ⟨x, hx⟩)
    have hAsub : A ⊆ C := by
      intro x hx
      rw [hdecomp] at hx
      rcases hx with hx | hx
      · exact hx
      · exact hPsub hx
    exact hC.mono hAsub
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__0657fa6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0657fa6f251f1874

4.371 371. exists_nonempty_perfect_subset_of_isClosed_of_not_countable

The problem

Let A be an uncountable closed subset of a separable metric space. Prove that A contains a nonempty perfect subset.

Lean statement

```
theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Uses the parent decomposition checkpoint: a closed set is the union of a countable set and a perfect set; the child additionally consumes uncountability to force the perfect component nonempty.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child derives a nonempty perfect component from the decomposition only after a new contradiction argument: if the perfect part were empty, the closed set would be countable, contradicting the hypothesis. This nonemptiness reasoning is absent from the parent's proof and is not a forced wrapper or trivial conjunction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into the substantive claim that an uncountable closed set contains a nonempty perfect subset. It requires a new contradiction argument showing the perfect component cannot be empty, plus an inclusion proof; recalling the parent's decomposition proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into an existence theorem for a nonempty perfect subset and must use uncountability to prove the perfect component is nonempty. This contradiction and subset argument are substantive reasoning absent from the parent's mere repackaging proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2794 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable.extracted_1.1.{u_1} (X : Type u_1)
  [inst : MetricSpace X] [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) (hunc : ¬A.Countable) :
  ∃ P, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Read back by a model that saw only the Lean. *For a separable metric space X and a closed, uncountable subset A of X , there exists a nonempty perfect subset P of X such that P is contained in A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A := by
  classical
  obtain ⟨C, P, hC, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  have hPsub : P ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  have hPne : P.Nonempty := by
    by_contra hPne
    have hPEmpty : P = ∅ := by
      ext x
      constructor
      · intro hx
        exact (hPne ⟨x, hx⟩).elim
      · intro hx
        exact False.elim (by simpa using hx)
    have hAcount : A.Countable := by
      rw [hdecomp, hPEmpty]
      simpa using hC
    exact hunc hAcount
  exact ⟨P, hP, hPne, hPsub⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__87abe271

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 87abe271b87ea292

4.372 372. closed_set_countable_or_contains_perfect

The problem

Prove the Cantor–Bendixson alternative: every closed subset of a separable metric space is either at most countable or contains a nonempty perfect subset.

Lean statement

```
theorem closed_set_countable_or_contains_perfect (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  Set.Countable A ∨ ∃ P : Set X, P.Nonempty ∧ Perfect P ∧ P ⊆ A := by
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the decomposition conclusion into a countable-or-nonempty-perfect-subset dichotomy. Beyond obtaining the Cantor–Bendixson decomposition, it requires a new nonemptiness case split and a separate countability argument for the empty perfect component.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a genuine countability-versus-perfect-subset dichotomy. Its proof requires a nonemptiness case split, subset extraction, and deriving countability when the perfect component is empty; this reasoning is not supplied by the parent's direct repackaging proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition existence claim into a countability-versus-perfect-subset dichotomy. It requires a new case split on whether the perfect component is nonempty, plus deriving countability of the closed set when that component is empty; the parent's repackaging proof does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2195 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B says A is countable, which may mean countably infinite, whereas Statement A allows A to be finite or countably infinite (at most countable).

Goal round-trip: what Lean elaborated

```
theorem closed_set_countable_or_contains_perfect.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) : A.Countable ∨ ∃ P, P.Nonempty ∧ Perfect P ∧ P ⊆ A
```

Read back by a model that saw only the Lean. *Let X be a separable metric space, let A be a closed subset of X . Then either A is countable, or there exists a nonempty perfect subset P of X such that P is contained in A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_set_countable_or_contains_perfect (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  Set.Countable A ∨ ∃ P : Set X, P.Nonempty ∧ Perfect P ∧ P ⊆ A := by
  classical
  obtain ⟨V, D, hV, hD, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  by_cases hDne : D.Nonempty
  · right
    have hsubset : D ⊆ A := by
      intro x hx
      rw [hdecomp]
      exact Or.inr hx
    exact ⟨D, hDne, hD, hsubset⟩
  · left
    have hDempty : D = ∅ := Set.not_nonempty_iff_eq_empty.mp hDne
    have hAcount : Set.Countable (V ∪ D) := by
      simpa [hDempty] using hV
    simpa [hdecomp] using hAcount
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__b4fbdc16

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b4fbdc163b4ce6c9

4.373 373. closed_set_disjoint_perfect_countable

The problem

Prove that every closed set in a separable metric space is the disjoint union of a perfect set and a countable set, with both pieces contained in the original set.

Lean statement

```
theorem closed_set_disjoint_perfect_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ Set.Countable C ∧ P ⊆ A ∧ C ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the decomposition by requiring the perfect and countable pieces to be disjoint and both contained in A. Its proof must construct a new remainder via set difference and establish disjointness, inclusions, and preservation of the decomposition; this reasoning is not supplied by the parent.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the decomposition by requiring the perfect and countable pieces to be disjoint, while preserving coverage and adding subset obligations. Its proof must construct a modified countable component via set difference and establish the new decomposition and disjointness, which the parent's direct theorem application does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the decomposition to a disjoint perfect/countable partition and explicitly proves both components are subsets of A. This requires constructing the remainder as a set difference and establishing disjointness, which the parent's direct decomposition proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2247 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_set_disjoint_perfect_countable.extracted_1.1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst.1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ C.Countable ∧ P ⊆ A ∧ C ⊆ A
```

Read back by a model that saw only the Lean. *For any separable metric space X , any subset A of X that is closed can be written as $A = P \cup C$ for some subsets P and C of X such that P and C are disjoint, P is perfect, C is countable, $P \subseteq A$, and $C \subseteq A$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_set_disjoint_perfect_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ Set.Countable C ∧ P ⊆ A ∧ C ⊆ A := by
  classical
  obtain ⟨V, D, hVcount, hDperfect, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  have hDsub : D ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  have hVsub : V ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inl hx
  let C : Set X := V \ D
  have hnewdecomp : A = D ∪ C := by
    ext x
    constructor
    · intro hx
      rw [hdecomp] at hx
      rcases hx with hxV | hxD
      · by_cases hxD' : x ∈ D
        · exact Or.inl hxD'
        · exact Or.inr ⟨hxV, hxD'⟩
      · exact Or.inl hxD
    · intro hx
      rw [hdecomp]
      rcases hx with hxD | hxC
      · exact Or.inr hxD
      · exact Or.inl hxC.1
  have hdisj : Disjoint D C := by
    refine Set.disjoint_left.2 ?_
    intro x hxD hxC
    exact hxC.2 hxD
  have hCcount : Set.Countable C := by
    exact hVcount.mono (by intro x hx; exact hx.1)
  have hCsub : C ⊆ A := by
    intro x hx
    exact hVsub hx.1
  exact ⟨D, C, hnewdecomp, hdisj, hDperfect, hCcount, hDsub, hCsub⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__c14d2d0c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c14d2d0c752f1883

4.374 374. closed_compact_perfect_countable_with_dense_perfect_part

The problem

Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that the subspace P has a countable dense subset.

Lean statement

```
theorem closed_compact_perfect_countable_with_dense_perfect_part
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
    ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s
```

Parent. Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that P has a countable basis obtained by restricting a countable basis of the ambient space.

Parent in Lean

```
theorem closed_compact_perfect_countable_with_restricted_basis
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, ∃ B : Set (Set K), ∃ R : Set (Set {x // x ∈ P}),
    A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
    B.Countable ∧ TopologicalSpace.IsTopologicalBasis B ∧
    R = Set.preimage (Subtype.val : {x // x ∈ P} → K) '' B ∧
    R.Countable ∧ TopologicalSpace.IsTopologicalBasis R
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from constructing a global basis and its restricted image to proving that the perfect subtype has a countable dense subset. Its proof requires compactness of the perfect part and separability of the subtype, which the parent's basis-construction proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the proof obligation from constructing a restricted countable topological basis to proving existence of a countable dense subset of the perfect part. This requires the new separability argument and 'exists_countable_dense', which the parent's basis-construction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from constructing a countable topological basis and its restricted image to constructing a countable dense subset of the perfect part. Its proof requires a new compact-subtype separability and dense-set argument that the parent's basis construction does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2264 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B requires a countable dense subset of K , whereas Statement A requires a countable dense subset of the subspace P ; B omits the latter requirement.

Goal round-trip: what Lean elaborated

```
theorem closed_compact_perfect_countable_with_dense_perfect_part.extracted_1_1.{u_1} {K : Type u_1}
  [inst : MetricSpace K] [inst_1 : CompactSpace K] (A : Set K) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧ ∃ s, s.Countable ∧ Dense s
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, and let A be a closed subset of K . Then there exist subsets P and C of K such that $A = P \cup C$, P is perfect, C is countable, and there exists a countable dense subset s of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem closed_compact_perfect_countable_with_dense_perfect_part
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
  ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  let I : SecondCountableTopology K := hSecond
  obtain ⟨C, P, hCcount, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := K) (C := A) hA
  have hCompactP : IsCompact P := hP.closed.isCompact
  have hCompactSubtype : CompactSpace {x // x ∈ P} :=
    isCompact_iff_compactSpace.mp hCompactP
  let I := hCompactSubtype
  have hSeparable : TopologicalSpace.SeparableSpace {x // x ∈ P} := inferInstance
  obtain ⟨s, hs, hd⟩ :=
    TopologicalSpace.exists_countable_dense {x // x ∈ P}
  have hDecomp : A = P ∪ C := hdecomp.trans (Set.union_comm C P)
  have hDensePart : ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s :=
    ⟨s, hs, hd⟩
  exact ⟨P, C, hDecomp, hP, hCcount, hDensePart⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Rudin_exercise_2_25__xe.mh__6cee1840

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 6cee184021acdc6

4.375 375. uniformContinuousOn_typed_bounded_chain

The problem

Let $(X_k)_{k \geq 0}$ be a sequence of uniform spaces and let $f_k : X_k \rightarrow X_{k+1}$. If the partial composites $F_k : X_0 \rightarrow X_k$ begin with the identity and satisfy $F_{k+1} = f_k \circ F_k$, and each f_k is uniformly continuous on $F_k(S)$, prove that F_n is uniformly continuous on S .

Lean statement

```
theorem uniformContinuousOn_typed_bounded_chain
  {X : ℕ → Type*} [∀ k, UniformSpace (X k)]
  {s : Set (X 0)}
  (f : ∀ k : ℕ, X k → X (k + 1))
  (F : ∀ k : ℕ, X 0 → X k)
  (n : ℕ)
  (hF0 : F 0 = id)
  (hFsucc : ∀ k : ℕ, F (k + 1) = f k ∘ F k)
  (hcont : ∀ k : ℕ, UniformContinuousOn (f k) (F k '' s)) :
  UniformContinuousOn (F n) s
```

Parent. Let $(f_k)_{k \geq 0}$ be a sequence of self-maps of a uniform space and let S be a subset of that space. If each f_k is uniformly continuous on the image of S after the preceding k stages, prove that the composite of the first n stages is uniformly continuous on S .

Parent in Lean

```
theorem uniformContinuousOn_bounded_chain
  {α : Type*} [UniformSpace α]
  {s : Set α} (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ k, UniformContinuousOn (f k)
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) '' s)) :
  UniformContinuousOn
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s
```

What it contributes. Supplied the finite-stage induction and local composition pattern; this version generalizes it to heterogeneous spaces $X k$ with explicit typed image transport.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from self-maps on one uniform space to a dependent typed chain with different spaces and explicitly propagated composites. The parent's definitional recursion and simplification no longer apply; the proof must use the supplied recurrence and typed composition across spaces.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law to a dependent sequence of distinct uniform spaces and arbitrary recursively specified maps, so the parent's concrete self-map recursion cannot be recalled verbatim. The proof must use the typed recurrence for F and propagate image domains through the heterogeneous chain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law from self-maps on one uniform space to a dependent chain of distinct uniform spaces. The parent's proof does not directly apply because composition and image domains must be type-propagated through the indexed family.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2915 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_typed_bounded_chain.extracted_1_1.{u_1} {X : ℕ → Type u_1}
  [inst : (k : ℕ) → UniformSpace (X k)] {s : Set (X 0)} (f : (k : ℕ) → X k → X (k + 1)) (F : (k : ℕ) → X 0 → X k)
  (n : ℕ) (hF0 : F 0 = id) (hFsucc : ∀ (k : ℕ), F (k + 1) = f k ∘ F k)
  (hcont : ∀ (k : ℕ), UniformContinuousOn (f k) (F k '' s)) : UniformContinuousOn (F n) s
```

Read back by a model that saw only the Lean. Let $(X_k)_{k \in \mathbb{N}}$ be a family of types, each equipped with a uniform space, and let s be a subset of X_0 . Suppose $f_k : X_k \rightarrow X_{k+1}$ and $F_k : X_0 \rightarrow X_k$ are families of functions, and let $n \in \mathbb{N}$. If F_0 is the identity function on X_0 , if for every $k \in \mathbb{N}$ we have $F_{k+1} = f_k \circ F_k$, and if for every $k \in \mathbb{N}$ the function f_k is uniformly continuous on the image $F_k(s)$, then F_n is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_typed_bounded_chain
  {X : ℕ → Type*} [∀ k, UniformSpace (X k)]
  {s : Set (X 0)}
  (f : ∀ k : ℕ, X k → X (k + 1))
  (F : ∀ k : ℕ, X 0 → X k)
  (n : ℕ)
  (hF0 : F 0 = id)
  (hFsucc : ∀ k : ℕ, F (k + 1) = f k ∘ F k)
  (hcont : ∀ k : ℕ, UniformContinuousOn (f k) (F k '' s)) :
  UniformContinuousOn (F n) s := by
  induction n with
  | zero =>
    rw [hF0]
    exact uniformContinuous_id.uniformContinuousOn
  | succ n ih =>
    have hmap : Set.MapsTo (F n) s (F n '' s) := by
      intro x hx
      exact (x, hx, rfl)
    have hstep : UniformContinuousOn (f n ∘ F n) s := by
      exact (hcont n).comp ih hmap
    rw [hFsucc n]
    exact hstep
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me.mh.me.mh__7c97e722

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7c97e722db1740e3

4.376 376. graph_cauchy_net

The problem

Let (u_i) be a Cauchy net indexed by a preordered set, and let f and g be uniformly continuous. Show that the net $(u_i, g(f(u_i)))$ is Cauchy.

Lean statement

```
theorem graph_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq (fun i => (u i, g (f (u i))))
```

Parent. Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Parent in Lean

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target to Cauchy convergence of a graph-valued sequence in a product space. Its proof requires constructing the uniformly continuous map $x \mapsto (x, g(f x))$ and transporting Cauchy-ness through it, which is additional reasoning beyond the parent's composition argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from preservation of Cauchy-ness under composition to Cauchy-ness of a paired graph sequence. It requires the new product-uniformity argument via 'uniformContinuous_id.prodMk', which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely stronger joint Cauchy claim for the graph-valued sequence, requiring construction of a uniform-continuous map into a product space. The parent only establishes Cauchy preservation under composition and does not supply this product-valued obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2189 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem graph_cauchy_net.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3} {ι : Type u_4}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : Preorder ι] {f : α → β}
  {g : β → γ} {u : ι → α} (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq fun i => (u i, g (f (u i)))
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform space structures, and let ι be a preorder. Let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ be functions, and let $u : \iota \rightarrow \alpha$ be a Cauchy sequence. If f and g are uniformly continuous, then the sequence $i \mapsto (u(i), g(f(u(i))))$ in the product space $\alpha \times \gamma$ is Cauchy.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem graph_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq (fun i => (u i, g (f (u i)))) := by
  have hgf : UniformContinuous (g ∘ f) := hg.comp hf
  have hgraph : UniformContinuous (fun x => (x, g (f x))) :=
    uniformContinuous_id.prodMk hgf
  have htransport : CauchySeq ((fun x => (x, g (f x))) ∘ u) :=
    hgraph.comp_cauchySeq hu
  simpa [Function.comp_def] using htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.mh__2c31f62c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 2c31f62c94f7d586

4.377 377. uniformContinuous_comp_of_on_range

The problem

If f is uniformly continuous and g is uniformly continuous on the range of f , then $g \circ f$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Preserves the composition checkpoint while weakening the outer-function hypothesis from global uniform continuity to uniform continuity on the range of the inner function.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens the outer function's hypothesis from global uniform continuity to uniform continuity on the range of f . Its proof must establish that f maps into this range and combine that restricted continuity with hf , so the parent's direct composition proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the outer-function hypothesis from global uniform continuity to uniform continuity on the range of f . Its proof must use the on-range composition lemma and establish range membership, which the parent's direct ' $hg.comp hf$ ' argument cannot supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the outer hypothesis from global uniform continuity to uniform continuity on the range of f . Proving the composition requires restricting f to its uniform-continuous-on-univ form and explicitly establishing that $f x$ lies in $\text{Set.range } f$, so the parent's direct composition proof does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2007 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_comp_of_on_range.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst_1 : UniformSpace α] [inst_2 : UniformSpace β] [inst_3 : UniformSpace γ] {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuousOn g (Set.range f)) : UniformContinuous (g ∘ f)
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform spaces. Let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$. If f is uniformly continuous and g is uniformly continuous on the range of f , then the composition $g \circ f : \alpha \rightarrow \gamma$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f) := by
  apply uniformContinuousOn_univ.mp
  refine hg.comp hf.uniformContinuousOn ?_
  intro x hx
  exact ⟨x, rfl⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh__bb8290d4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1565e00b8634bb60

4.378 378. composite_cauchy_net

The problem

Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Lean statement

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Supplies the intermediate checkpoint that the composite $g \circ f$ is uniformly continuous.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires an additional mathematical step: transporting a Cauchy net through the uniformly continuous composite using ‘comp_cauchySeq’. The parent’s composition proof alone does not establish Cauchy preservation, so recalling it is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from uniform continuity of a composite to preservation of a Cauchy sequence under that composite. It requires both composing uniform-continuity proofs and applying the separate Cauchy-transport lemma, which the parent’s proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires the additional mathematical step that a uniformly continuous map sends a Cauchy sequence to a Cauchy sequence, after composing the maps. Recalling the parent’s composition proof supplies only the intermediate uniform-continuity fact and does not solve the new Cauchy conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2183 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem composite_cauchy_net.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  {ι : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α} (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Read back by a model that saw only the Lean. Let α , β , and γ be types equipped with uniform spaces, and let ι be a type equipped with a preorder. For functions $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $u : \iota \rightarrow \alpha$, assume that f and g are uniformly continuous and that u is a Cauchy sequence (net). Then the net obtained by applying g after f to u , namely $\iota \mapsto g(f(u(i)))$, is a Cauchy sequence (net) in γ .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u) := by
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  have htransport : CauchySeq ((g ∘ f) ∘ u) := hcomp.comp_cauchySeq hu
  exact htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh__fcde3616

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint fcde3616b61ab92a

4.379 379. uniformContinuousOn_composed_output_pair

The problem

Let uniformly continuous maps be composed, and replace the first map by a map agreeing with it on a set. Prove that the ordered pair of the intermediate and final outputs is uniformly continuous on that set.

Lean statement

```
theorem uniformContinuousOn_composed_output_pair
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s
```

Parent. Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Parent in Lean

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

What it contributes. Reuses the parent's local-agreement transfer and successive range-based composition checkpoints, with the final obligation strengthened to uniform continuity of an output pair.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the codomain and conclusion to uniform continuity of a paired intermediate/final output. The parent's composition proof does not establish this; the child requires constructing a product-valued uniformly continuous map and composing it locally.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composed output to a paired output and requires proving uniform continuity of the product map, using a new product-composition argument absent from the parent proof. This is a genuine change of claim rather than decoration or recall.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composed output to a paired output and requires proving uniform continuity of both components via a product construction. The parent's composition argument alone does not establish this pair-valued conclusion, so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2178 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_composed_output_pair.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) (s : Set α) (m : α → β) (hm : ∀ x ∈ s, m x = f x) :
UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Let s be a subset of α , and let $m : \alpha \rightarrow \beta$ be a function such that for every $x \in s$, $m(x) = f(x)$. Then the function $x \mapsto (g(m(x)), h(g(m(x))))$ from α to $\gamma \times \delta$ is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_composed_output_pair
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
(s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s := by
have hmOn : UniformContinuousOn m s :=
  hf.uniformContinuousOn.congr (fun x hx => (hm x hx).symm)
have hhg : UniformContinuous (h ∘ g) := hh.comp hg
have hgh : UniformContinuous (fun y : β => (g y, (h ∘ g) y)) :=
  hg.prodMk hhg
have hcomp : UniformContinuousOn ((fun y : β => (g y, (h ∘ g) y)) ∘ m) s :=
  hgh.comp_uniformContinuousOn hmOn
simpa [Function.comp_def] using hcomp
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me.mh__51bf287b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 51bf287bbfed901e

4.380 380. uniformContinuous_pair_with_three_stage

The problem

If three maps between uniform spaces are uniformly continuous, and another uniformly continuous map has the same domain and codomain as their three-stage composition, then the paired map formed from these two outputs is uniformly continuous.

Lean statement

```
theorem uniformContinuous_pair_with_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (hq : UniformContinuous q) :
  UniformContinuous (fun x => (q x, h (g (f x))))
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The conclusion changes from continuity of a single three-stage composition to continuity of a paired map with an independent uniformly continuous component. The proof must construct continuity into a product via ‘prodMk’, an obligation absent from the parent’s composition-only proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from continuity of a single composite to continuity of a product-valued map, requiring the separate product-continuity construction via ‘prodMk’ and an additional branch ‘q’; the parent’s composition proof alone does not establish this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from uniform continuity of a single three-stage composition to uniform continuity of a product-valued map. Proving it requires the additional product-continuity construction for the independent map q, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2160 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_pair_with_three_stage.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ} (hf : UniformContinuous f)
(hg : UniformContinuous g) (hh : UniformContinuous h) (hq : UniformContinuous q) :
UniformContinuous fun x => (q x, h (g (f x)))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be types equipped with uniform spaces. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, $h : \gamma \rightarrow \delta$, and $q : \alpha \rightarrow \delta$. Assume that f, g, h , and q are uniformly continuous. Then the function from α to $\delta \times \delta$ defined by $x \mapsto (q(x), h(g(f(x))))$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_pair_with_three_stage
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
(hq : UniformContinuous q) :
UniformContinuous (fun x => (q x, h (g (f x)))) := by
exact hq.prodMk (hh.comp (hg.comp hf))
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.mh__a2494e74

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a2494e744469d309

4.381 381. uniformContinuous_paired_composite

The problem

If $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ are uniformly continuous, then the map $x \mapsto (f(x), g(f(x)))$ into $\beta \times \gamma$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_paired_composite
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (fun x => (f x, g (f x)))
```

Parent. A uniformly continuous map followed by another uniformly continuous map is uniformly continuous, even when the intermediate map is named separately and agrees pointwise with the first.

Parent in Lean

```
theorem uniformContinuous_named_intermediate
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g)
  (m : α → β) (hm : ∀ x, m x = f x) :
  UniformContinuous (fun x => g (m x))
```

What it contributes. Supplied the composition checkpoint `hcomp : UniformContinuous (g ∘ f)`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the codomain and conclusion to a product-valued map, requiring the product uniform-continuity construction ‘`hf.prodMk hcomp`’ in addition to composition. The parent proves only continuity of the composite and cannot directly supply this paired-map result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a single composite to continuity of a product-valued map, requiring the product construction and continuity of both components. The parent’s proof of the composite alone does not supply the paired-map argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a scalar composite to uniform continuity of a product-valued map. Although it reuses the composition argument, it additionally requires constructing continuity of the paired map via the product lemma, which the parent’s proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2147 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_paired_composite.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) : UniformContinuous fun x => (f x, g (f x))
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform space structures. If $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ are uniformly continuous, then the map from α to $\beta \times \gamma$ defined by $x \mapsto (f(x), g(f(x)))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_paired_composite
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (fun x => (f x, g (f x))) := by
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  simpa [Function.comp_apply] using hf.prodMk hcomp
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.mh__ab5b4a93

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ab5b4a93f6986564

4.382 382. finite_existential_pointwise_order_closed

The problem

Let X be a metric space and let a finite family of pairs of continuous real-valued functions be given on X . Prove that the set of points where at least one first function is at most its paired second function is closed.

Lean statement

```
theorem finite_existential_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∃ i ∈ s, f i x ≤ g i x} := by
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the proof obligation from a finite intersection (universal condition) to a finite union (existential condition), requiring union closure and a different induction structure. It also correctly removes the parent's nonemptiness hypothesis because the empty union is closed.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the finite intersection of closed inequality loci into a finite union, requiring closure under finite unions and an existential disjunction rather than the parent's intersection argument. A memorized proof of the parent does not directly establish this differently shaped conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a finite intersection of closed order sets into a finite union, requiring a different induction normalization and the closed-union argument. The parent's intersection proof does not directly supply this obligation, and the child has no redundant hypotheses.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2855 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_existential_pointwise_order_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∃ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space, let ι be a type, and let s be a finite set of indices in ι . Let f and g be functions assigning to each $i \in \iota$ real-valued functions on α . Assume that for every $i \in \iota$, the functions $f(i)$ and $g(i)$ are continuous. Then the set of points $x \in \alpha$ for which there exists an index $i \in s$ such that $f(i)(x) \leq g(i)(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_existential_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∃ i ∈ s, f i x ≤ g i x} := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp
  | @insert i s hi ih =>
    have h_i : IsClosed {x : α | f i x ≤ g i x} := by
      exact isClosed_le (hf i) (hg i)
    have h_s : IsClosed {x : α | ∃ j ∈ s, f j x ≤ g j x} := by
      exact ih
    have h_union : IsClosed ({x : α | f i x ≤ g i x} ∪ {x : α | ∃ j ∈ s, f j x ≤ g j x}) := by
      exact h_i.union h_s
    simpa [Finset.mem_insert, or_assoc] using h_union
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__429658dc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 429658dc689b33fb

4.383 383. finite_common_strict_order_closure_subset

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Show that the closure of the set where every first function is strictly less than its paired second function is contained in the set where every first function is at most its paired second function.

Lean statement

```
theorem finite_common_strict_order_closure_subset
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  closure {x : α | ∀ i ∈ s, f i x < g i x} ⊆
  {x : α | ∀ i ∈ s, f i x ≤ g i x}
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from closedness of a finite intersection of non-strict inequalities to a closure-containment theorem for strict inequalities. Its proof requires the new topological fact that the closure of a strict real inequality is contained in the corresponding non-strict inequality, which the parent's finite-intersection induction does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the problem from proving a finite intersection of weak-order sets is closed to proving that the closure of a strict-order feasible set lies inside the weak-order feasible set. Its proof uses closure monotonicity and the strict-to-nonstrict closure lemma, with no reliance on the parent's induction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the result from closedness of a finite intersection of weak inequalities to a closure-containment theorem for strict inequalities. Its proof requires monotonicity of closure and the strict-to-weak closure lemma, so the parent's finite induction and intersection argument do not supply it.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2887 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_strict_order_closure_subset.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  closure {x | ∀ i ∈ s, f i x < g i x} ⊆ {x | ∀ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α and ι be types, with α equipped with a metric, and let s be a nonempty finite set of indices in ι . Let f and g be functions assigning to each index i functions from α to $\mathbb{R}$. Assume that for every i , both f_i and g_i are continuous. Then every point in the closure of the set of points $x \in \alpha$ such that $f_i(x) < g_i(x)$ for every $i \in s$ also satisfies $f_i(x) \leq g_i(x)$ for every $i \in s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_strict_order_closure_subset
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  closure {x : α | ∀ i ∈ s, f i x < g i x} ⊆
    {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
  intro x hx i hi
  have hsub : {y : α | ∀ j ∈ s, f j y < g j y} ⊆
    {y : α | f i y < g i y} := by
    intro y hy
    exact hy i hi
  have hxi : x ∈ closure {y : α | f i y < g i y} :=
    closure_mono hsub hx
  exact (closure_lt_subset_le (hf i) (hg i)) hxi
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__a5ad1af2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 08ddd17158cbaa74

4.384 384. continuous_finite_minimum_of_differences

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the pointwise minimum of the differences between each second function and its paired first function is continuous.

Lean statement

```
theorem continuous_finite_minimum_of_differences
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (hs : s.Nonempty)
  (f g : 1 → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  Continuous (fun x => s.inf' hs (fun i => g i x - f i x))
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (hs : s.Nonempty)
  (f g : 1 → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from closedness of a common pointwise-order set to continuity of a finite pointwise minimum of differences. Its proof requires continuity of subtraction and a finite-infimum continuity argument, neither supplied by the parent's closed-set induction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a finite intersection of order constraints to continuity of a finite minimum of difference functions. Its proof requires continuity under subtraction and the finite infimum continuity theorem, which the parent's induction proof does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a closed-set claim into continuity of a finite infimum of difference functions. Its proof requires subtraction continuity and a finite-infimum continuity theorem, which the parent's induction-and-intersection argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_finite_minimum_of_differences.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  Continuous fun x => s.inf' hs fun i => g i x - f i x
```

Read back by a model that saw only the Lean. *Let α be a metric space and ι be a type. For a nonempty finite set s of indices in ι , let f and g assign to each index i continuous real-valued functions $f(i)$, $g(i)$ on α . Then the function sending $x \in \alpha$ to the minimum, over $i \in s$, of $g(i)(x) - f(i)(x)$ is continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_finite_minimum_of_differences
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  Continuous (fun x => s.inf' hs (fun i => g i x - f i x)) := by
  have hdiff : ∀ i, Continuous (fun x => g i x - f i x) := by
    intro i
    exact (hg i).sub (hf i)
  have haggregate : Continuous (fun x => s.inf' hs (fun i => g i x - f i x)) := by
    exact Continuous.finset_inf'_apply hs (fun i hi => hdiff i)
  exact haggregate
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__b39ad3be

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b39ad3be276287ba

4.385 385. finite_sum_order_closed

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where the sum of the first functions is at most the sum of the paired second functions is closed.

Lean statement

```
theorem finite_sum_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x}
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

What it contributes. Preserves the metric-space, finite-index-family, and continuous-function setting while changing the final obligation from an intersection of pointwise inequalities to an inequality between finite aggregates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a finite conjunction of pointwise inequalities with a single inequality between finite sums. Its proof requires establishing continuity of indexed finite sums, which is not supplied by the parent's induction-and-intersection argument, and the nonemptiness hypothesis is genuinely unnecessary.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the mathematics from a finite intersection of pointwise inequalities to a single inequality between finite sums. Its proof requires establishing continuity of the aggregate sums and applying closedness of an order relation, so the parent's induction argument does not supply it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the pointwise finite conjunction into a closed sublevel set for finite sums, requiring continuity of finite sums and an aggregate order argument; the parent's induction-and-intersection proof does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2852 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_sum_order_closed.extracted_1.1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∑ i ∈ s, f i x ≤ ∑ i ∈ s, g i x}
```

Read back by a model that saw only the Lean. Let α and ι be types, with α equipped with a metric-space structure. Let s be a nonempty finite set of elements of ι , and let f and g be functions assigning to each $i : \iota$ continuous real-valued functions on α . Then the set of all $x \in \alpha$ such that the finite sum $\sum_{i \in s} f(i)(x)$ is less than or equal to $\sum_{i \in s} g(i)(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem finite_sum_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x} := by
  classical
  have hfs : Continuous (fun x : α => ∑ i ∈ s, f i x) := by
    exact continuous_finset_sum s (fun i hi => hf i)
  have hgs : Continuous (fun x : α => ∑ i ∈ s, g i x) := by
    exact continuous_finset_sum s (fun i hi => hg i)
  have h_order : IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x} := by
    exact isClosed_le hfs hgs
  exact h_order
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__bda8ceec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 239e307b69e11bb5

4.386 386. continuous_equalizer_closed_hausdorff

The problem

Let X be a topological space and Y a Hausdorff space. If $f, g : X \rightarrow Y$ are continuous, prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Lean statement

```
theorem continuous_equalizer_closed_hausdorff
  {α β : Type} [TopologicalSpace α] [TopologicalSpace β] [T2Space β]
  (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

Parent. Let f and g be continuous real-valued functions on a metric space X . Prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Parent in Lean

```
theorem continuous_equalizer_closed
  {α : Type} [MetricSpace α]
  (f g : α → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the result from metric-space real-valued functions to continuous maps into an arbitrary Hausdorff topological space. Its proof relies on the general closed-equality theorem, which is not supplied by the parent's real-valued difference/preimage argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from metric spaces with real-valued functions to arbitrary topological domains and Hausdorff codomains. The parent's subtraction-and-zero-set argument no longer applies; closure must instead follow from the Hausdorff diagonal/equalizer theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from real-valued functions on metric spaces to functions into any Hausdorff topological space. The parent's subtraction-and-zero-preimage argument no longer applies; the child requires the distinct Hausdorff equalizer-closedness theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2866 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_equalizer_closed_hausdorff.extracted_1_1 {α β : Type} [inst : TopologicalSpace α]
  [inst_1 : TopologicalSpace β] [inst_2 : T2Space β] (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = g x}
```

Read back by a model that saw only the Lean. Let α and β be topological spaces, with β Hausdorff. If $f, g : \alpha \rightarrow \beta$ are continuous, then the set $\{x \in \alpha \mid f(x) = g(x)\}$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_equalizer_closed_hausdorff
  {α β : Type} [TopologicalSpace α] [TopologicalSpace β] [T2Space β]
  (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
  have hclosed : IsClosed {x : α | f x = g x} := by
    exact isClosed_eq hf hg
  exact hclosed
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.mh__7049d394

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7049d394becbadf3

4.387 387. continuous_translated_interval_band_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of pairs (x, t) satisfying $a \leq f(x) - t \leq b$ is closed in $X \times \mathbb{R}$.

Lean statement

```
theorem continuous_translated_interval_band_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} := by
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Parent in Lean

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

What it contributes. Used the parent's closed-interval and continuous-preimage checkpoints, with the new product-coordinate difference map supplying the mutation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the domain to a product and proves continuity of the coupled map $(x, t) \mapsto f(x) - t$, requiring product-space projection and subtraction reasoning absent from the parent. It is a genuine translated-band generalization, not a decorative restatement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving continuity of the coupled map $(x, t) \mapsto f(x) - t$ on a product space, then taking its closed interval preimage. The parent only handles the preimage of an interval under f , so its proof does not supply the essential product-continuity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves closure of a genuinely new subset of the product space: it must establish continuity of the coupled map $(x, t) \mapsto f(x) - t$ using projections and subtraction before applying the closed-interval preimage argument. The parent's one-variable continuity proof does not supply this coupling step.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2858 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_translated_interval_band_closed.extracted.1.1 {α : Type} [inst : MetricSpace α] (f : α → ℝ)
(h : Continuous f) (a b : ℝ) : IsClosed {p | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b}
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric, let $f : \alpha \rightarrow \mathbb{R}$ be continuous, and let $a, b \in \mathbb{R}$. Then the set of pairs $(p_1, p_2) \in \alpha \times \mathbb{R}$ such that $a \leq f(p_1) - p_2$ and $f(p_1) - p_2 \leq b$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_translated_interval_band_closed
{α : Type} [MetricSpace α]
(f : α → ℝ) (h : Continuous f) (a b : ℝ) :
IsClosed {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} := by
have h_difference_continuous : Continuous (fun p : α × ℝ => f p.1 - p.2) := by
  exact (h.comp continuous_fst).sub continuous_snd
have h_interval_closed : IsClosed (Set.Icc a b) := by
  exact isClosed_Icc
have h_band_closed : IsClosed ((fun p : α × ℝ => f p.1 - p.2) ⁻¹' Set.Icc a b) := by
  exact h_interval_closed.preimage h_difference_continuous
have h_set_eq :
  {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} =
    (fun p : α × ℝ => f p.1 - p.2) ⁻¹' Set.Icc a b := by
  ext p
  simp [Set.mem_Icc]
rw [h_set_eq]
exact h_band_closed
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__08cbac12

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 08cbac121aaa9276

4.388 388. compact_level_set

The problem

Let K be a compact subset of a metric space X , and let $f : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points in K where $f(x) = c$ is compact for every $c \in \mathbb{R}$.

Lean statement

```
theorem compact_level_set
  {α : Type} [MetricSpace α]
  (K : Set α) (hK : IsCompact K)
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = K ∩ f ⁻¹' ({c} : Set ℝ)) : IsCompact z
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $c \in \mathbb{R}$. Prove that the level set $\{x \in X \mid f(x) = c\}$ is closed.

Parent in Lean

```
theorem continuous_level_set_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = f ⁻¹' ({c} : Set ℝ)) : IsClosed z := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens closedness of a level set to compactness of its intersection with an arbitrary compact set. Its proof still reuses the level-set argument, but additionally requires proving the intersection is closed and invoking compactness inheritance, which the parent does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from closedness to compactness and adds the substantive argument that the level set is a closed subset of the compact set K . The parent proof only establishes closedness of the level set and does not supply the compact-subset reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness of a level set into compactness of its intersection with a compact set. The parent's continuity argument supplies only closedness; the new proof additionally requires showing the intersection is a closed subset of K and invoking compactness inheritance.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_level_set.extracted_1_1 {α : Type} [inst : MetricSpace α] (K : Set α) (hK : IsCompact K) (f : α → ℝ)
(h : Continuous f) (c : ℝ) (z : Set α) (g : z = K ∩ f-1 {c}) : IsCompact z
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric space structure, let K be a compact subset of α , and let $f : \alpha \rightarrow \mathbb{R}$ be continuous. For any real number c and subset z of α , if $z = K \cap \{x \in \alpha \mid f(x) = c\}$, then z is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_level_set
  {α : Type} [MetricSpace α]
  (K : Set α) (hK : IsCompact K)
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = K ∩ f-1 ({c} : Set ℝ)) : IsCompact z := by
  have h_shift : Continuous (fun x => f x - c) := h.sub continuous_const
  have h_zero_closed : IsClosed ({0} : Set ℝ) := by
    simp
  have h_shift_preimage :
    IsClosed ((fun x => f x - c)-1 ({0} : Set ℝ)) :=
    IsClosed.preimage h_shift h_zero_closed
  have h_level_eq :
    (fun x => f x - c)-1 ({0} : Set ℝ) = f-1 ({c} : Set ℝ) := by
    ext x
    simp [sub_eq_zero]
  have h_preimage_closed : IsClosed (f-1 ({c} : Set ℝ)) := by
    rw [← h_level_eq]
    exact h_shift_preimage
  have h_target_closed : IsClosed (K ∩ f-1 ({c} : Set ℝ)) := by
    exact hK.isClosed.inter h_preimage_closed
  rw [g]
  apply hK.of_isClosed_subset h_target_closed
  intro x hx
  exact hx.1
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__62b4c13e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 62b4c13e914dfe94

4.389 389. finite_pointwise_order_locus_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Lean statement

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every i is closed.

Parent in Lean

```
theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
```

What it contributes. Retained the metric-space setting, finite indexing, continuity-of-differences argument, and finite intersection checkpoint; replaced equality with membership in the closed nonpositive ray.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces equality loci with pointwise inequality loci. The parent's zero-set argument no longer applies; the proof must use the closed ray $(-\infty, 0]$ and establish the inequality via subtraction, then intersect the finite family.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality loci into pointwise inequality loci, requiring a new characterization as preimages of the closed ray $(-\infty, 0]$ and an arithmetic equivalence proved by linear reasoning. The parent's zero-set argument does not supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces equality loci with order loci, requiring the distinct fact that the closed ray $(-\infty, 0]$ is closed and a nontrivial equivalence between $f_i x \leq g_i x$ and $f_i x - g_i x \leq 0$. The parent's zero-set argument cannot be recalled to prove this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2829 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_order_locus_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (z : Set α)
  (hz : z = {x | ∀ (i : Fin n), f i x ≤ g i x}) : IsClosed z
```

Read back by a model that saw only the Lean. *Let α be a metric space, let n be a natural number, and let f and g be functions from $\text{Fin } n$ to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \text{Fin } n$, the function $x \mapsto f(i)(x)$ is continuous, and for every $i \in \text{Fin } n$, the function $x \mapsto g(i)(x)$ is continuous. If $z \subseteq \alpha$ satisfies $z = \{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f(i)(x) \leq g(i)(x)\}$, then z is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | f i x ≤ g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hpre : IsClosed (q⁻¹' Set.Iic (0 : ℝ)) :=
      IsClosed.preimage hq isClosed_Iic
    have heq : {x : α | f i x ≤ g i x} = q⁻¹' Set.Iic (0 : ℝ) := by
      ext x
      change (f i x ≤ g i x) ↔ q x ≤ 0
      constructor <=> intro h <=> dsimp [q] <=> linarith
    rw [heq]
    exact hpre
  have hcne : IsClosed (⋂ i : Fin n, {x : α | f i x ≤ g i x}) :=
    isClosed_iInter hclosed
  have hz' : z = ⋂ i : Fin n, {x : α | f i x ≤ g i x} := by
    rw [hz]
    ext x
    simp
    rw [hz']
    exact hcne
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__69f231de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 69f231defa9ab9cd

4.390 390. finite_common_equalizer_closed

The problem

Let X be a metric space, let s be a finite index set, and let (f_i) and (g_i) be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every $i \in s$ is closed.

Lean statement

```
theorem finite_common_equalizer_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f g : 1 → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = g i x} := by
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces fixed-level sets with equalizer sets for two independently varying continuous families. Proving each equalizer is closed requires a distinct closure argument via 'isClosed_eq'; the parent's singleton-preimage proof does not directly supply this step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a common level-set problem into a finite equalizer problem for two independently continuous function families. Its proof requires the distinct closedness theorem for equality sets, so recalling the parent's singleton-preimage argument does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes a common level set to a finite intersection of equalizers between two independently varying continuous function families. The parent's singleton-preimage argument does not prove this; the child requires the distinct closed-equalizer lemma for each index.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2364 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_equalizer_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ i ∈ s, f i x = g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space and let ι be a type. For a finite set s of elements of ι , let f and g assign to each $i \in \iota$ real-valued functions on α . Assume that for every $i \in \iota$, the functions $f(i)$ and $g(i)$ are continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) = g(i)(x)$ for every $i \in s$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_equalizer_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = g i x} := by
  have h_equal : ∀ i, IsClosed {x : α | f i x = g i x} := by
    intro i
    exact isClosed_eq (hf i) (hg i)
  have h_common : IsClosed (⋂ i ∈ s, {x : α | f i x = g i x}) := by
    exact isClosed_biInter (fun i hi => h_equal i)
  have h_repr :
    {x : α | ∀ i ∈ s, f i x = g i x} =
    ⋂ i ∈ s, {x : α | f i x = g i x} := by
    ext x
    simp
  rw [h_repr]
  exact h_common
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__a08deb7d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a08deb7d15400b3e

4.391 391. finite_common_zero_locus_sum_squares

The problem

Let X be a metric space and let $(f_i)_{i \in s}$ be finitely many continuous real-valued functions on X . Prove that their common zero locus is closed and is exactly the zero set of $\sum_{i \in s} f_i(x)^2$.

Lean statement

```
theorem finite_common_zero_locus_sum_squares
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = 0} ∧
  {x : α | ∀ i ∈ s, f i x = 0} =
  {x : α | ∑ i ∈ s, (f i x)^2 = 0}
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine algebraic characterization: it proves that the common zero locus equals the zero set of a finite sum of squares, requiring nonnegativity and a sum-of-squares zero argument beyond the parent's intersection-of-closed-preimages proof. The appended equality is substantive rather than a trivial conjunction or restatement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive characterization: the common zero locus equals the zero set of a finite sum of squares. Proving the reverse inclusion requires nontrivial sum-of-squares reasoning unavailable from the parent's closedness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive characterization: the common zero locus equals the zero set of a finite sum of squares. Proving the converse requires a nontrivial sum-of-squares argument absent from the parent's closure proof, while the continuity hypotheses are still used for the closure conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2352 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_zero_locus_sum_squares.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (h : ∀ (i : ι), Continuous (f i)) :
  IsClosed {x | ∀ i ∈ s, f i x = 0} ∧ {x | ∀ i ∈ s, f i x = 0} = {x | ∑ i ∈ s, f i x ^ 2 = 0}
```

Read back by a model that saw only the Lean. Let α and ι be types, let α be equipped with a metric-space structure, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a real-valued function on α . Assume that for every $i \in \iota$, the function $f(i)$ is continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) = 0$ for every $i \in s$ is closed, and this set is exactly the set of all $x \in \alpha$ such that the finite sum $\sum_{i \in s} (f(i)(x))^2$ equals 0.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_zero_locus_sum_squares
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = 0} ∧
  {x : α | ∀ i ∈ s, f i x = 0} =
  {x : α | ∑ i ∈ s, (f i x)^2 = 0} := by
  have h_level : ∀ i, IsClosed (f i ⁻¹' ({0} : Set ℝ)) := by
    intro i
    exact IsClosed.preimage (h i) (by simp)
  have h_common :
    IsClosed (⋂ i ∈ s, f i ⁻¹' ({0} : Set ℝ)) := by
    exact isClosed_biInter (fun i hi => h_level i)
  have h_common_eq :
    {x : α | ∀ i ∈ s, f i x = 0} = ⋂ i ∈ s, f i ⁻¹' ({0} : Set ℝ) := by
    ext x
    simp
  have h_closed : IsClosed {x : α | ∀ i ∈ s, f i x = 0} := by
    rw [h_common_eq]
    exact h_common
  have h_sum_cont : Continuous (fun x : α => ∑ i ∈ s, (f i x)^2) := by
    fun_prop
  constructor
  · exact h_closed
  · ext x
    constructor
    · intro hx
      simp only [Set.mem_setOf_eq] at hx ⊢
      apply Finset.sum_eq_zero
      intro i hi
      rw [hx i hi]
      simp
    · intro hx
      simp only [Set.mem_setOf_eq] at hx ⊢
      have hs : ∑ i ∈ s, f i x * f i x = 0 := by
        simpa [pow_two] using hx
      exact (Finset.sum_mul_self_eq_zero_iff s (fun i => f i x)).mp hs
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__a6c49faf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a6c49fafba296734

4.392 392. constrained_common_zero_locus_closed

The problem

Let X be a metric space, let $(f_i)_{i \in I}$ be a family of continuous real-valued functions, and let $g : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points where every f_i vanishes and g is nonnegative is closed.

Lean statement

```
theorem constrained_common_zero_locus_closed
  {α ι : Type} [MetricSpace α]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i))
  (g : α → ℝ) (hg : Continuous g) :
  IsClosed {x : α | (∀ i, f i x = 0) ∧ 0 ≤ g x}
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the finite-index assumption, requiring closure under an arbitrary intersection, and adds a continuous nonnegativity constraint requiring a separate preimage/intersection argument. The parent's finite sum-of-squares proof does not supply either obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite-index assumption and adds a genuinely relevant closed inequality constraint. Its proof must establish arbitrary intersection closure and preimage closure for the constraint, so the parent's finite sum-of-squares argument cannot be recalled to solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Removing the finite-index hypothesis breaks the parent's finite sum-of-squares argument, and the child must instead prove closure via arbitrary intersections. It also genuinely adds a continuous inequality constraint and proves the resulting intersection is closed.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2146 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem constrained_common_zero_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (f : ι → α → ℝ)
  (h : ∀ (i : ι), Continuous (f i)) (g : α → ℝ) (hg : Continuous g) :
  IsClosed {x | (∀ (i : ι), f i x = 0) ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. Let α and ι be types, let α be equipped with a metric-space structure, and let $f : \iota \rightarrow \alpha \rightarrow \mathbb{R}$ be such that for every $i \in \iota$, the function $x \mapsto f(i, x)$ is continuous. Let $g : \alpha \rightarrow \mathbb{R}$ be continuous. Then the set of all $x \in \alpha$ such that $f(i, x) = 0$ for every $i \in \iota$ and $0 \leq g(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem constrained_common_zero_locus_closed
  {α ι : Type} [MetricSpace α]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i))
  (g : α → ℝ) (hg : Continuous g) :
  IsClosed {x : α | (∀ i, f i x = 0) ∧ 0 ≤ g x} := by
  have hzero : IsClosed {x : α | ∀ i, f i x = 0} := by
    have hi : IsClosed (⋂ i, (f i)⁻¹' ({0} : Set ℝ)) := by
      exact isClosed_iInter (fun i => (isClosed_singleton : IsClosed ({0} : Set ℝ)).preimage (h i))
    have heq : {x : α | ∀ i, f i x = 0} = ⋂ i, (f i)⁻¹' ({0} : Set ℝ) := by
      ext x
      simp
    rw [heq]
    exact hi
  have hconstraint : IsClosed {x : α | 0 ≤ g x} := by
    have hi : IsClosed (Set.Ici (0 : ℝ)) := isClosed_Ici
    have hp := hi.preimage hg
    simpa only [Set.mem_preimage, Set.mem_Ici, Set.mem_setOf_eq] using hp
  have hinter : IsClosed ({x : α | ∀ i, f i x = 0} ∩ {x : α | 0 ≤ g x}) :=
    hzero.inter hconstraint
  convert hinter using 1
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.mh__3ba4bcc3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 3ba4bcc389bdec88

4.393 393. zero_constrained_positivity_compact

The problem

Let X be a compact metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points where f vanishes and g is nonnegative is compact.

Lean statement

```
theorem zero_constrained_positivity_compact
  {X : Type*} [MetricSpace X] [CompactSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x} := by
```

Parent. Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Parent in Lean

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from closedness to compactness and requires the new argument that a closed subset of a compact space is compact. Although it reuses the parent's closedness proof, that proof alone does not establish the child's compactness claim.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness to compactness and adds a genuine CompactSpace hypothesis. Its proof must establish the constrained set is closed and then derive compactness as a closed subset of the compact universe, which the parent's argument does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to compactness and uses the new compactness hypothesis together with a closed-subset argument. This requires reasoning not supplied by the parent's closedness proof and is not a recall or decorative extension.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2155 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem zero_constrained_positivity_compact.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X]
  [inst.1 : CompactSpace X] (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. *Let X be a compact metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous functions. Then the set of all $x \in X$ such that $f(x) = 0$ and $0 \leq g(x)$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem zero_constrained_positivity_compact
  {X : Type*} [MetricSpace X] [CompactSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_constrained_closed :
    IsClosed ({x ∈ {x | f x = 0} | (fun _ : X => 0) x ≤ g x}) := by
    exact h_zero_closed.isClosed_le continuous_const.continuousOn hg.continuousOn
  have h_compact_univ : IsCompact (Set.univ : Set X) := by
    exact isCompact_univ
  have h_subset : {x | f x = 0 ∧ 0 ≤ g x} ⊆ (Set.univ : Set X) := by
    intro x hx
    simp
  exact h_compact_univ.of_isClosed_subset h_constrained_closed h_subset
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe.mh__324f6568

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 324f656808cc9a8b

4.394 394. tendsto_increment_of_tendsto_deriv

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be differentiable on $(0, \infty)$, and suppose f' tends to L at $+\infty$. Then for every $h > 0$, the increment $f(x+h) - f(x)$ tends to hL as x tends to $+\infty$.

Lean statement

```
theorem tendsto_increment_of_tendsto_deriv
  {f : ℝ → ℝ} {h L : ℝ}
  (hh : 0 < h)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Filter.Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℳ (h * L))
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℳ 0) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both the increment length h and derivative limit L , proving the new limit $h * L$ via a scaled mean-value-theorem estimate. The parent only handles $h = 1$ and $L = 0$, so its proof does not supply this argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment length and the derivative limit, requiring a centered derivative estimate and scaling by h to obtain the limit $h * L$. The parent only handles unit increments and limit zero, so its proof does not directly supply this obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment length and derivative limit, requiring a scaled error estimate and algebra showing the increment approaches $h * L$. The parent only handles the fixed case $h = 1$ and $L = 0$, so its proof does not supply the generalized conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1949 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem tendsto_increment_of_tendsto_deriv.extracted_1_1 {f : ℝ → ℝ} {h L : ℝ} (hh : 0 < h)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ L)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ (h * L))
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $h, L \in \mathbb{R}$. Assume $h > 0$, f is differentiable on the open interval $(0, \infty)$, and the derivative $f'(x)$ tends to L as x tends to $+\infty$. Then $f(x+h) - f(x)$ tends to hL as x tends to $+\infty$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem tendsto_increment_of_tendsto_deriv
{f : ℝ → ℝ} {h L : ℝ}
(hh : 0 < h)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0))
(hf : Filter.Tendsto (deriv f) Filter.atTop (ℳ L)) :
Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℳ (h * L)) := by
  refine Metric.tendsto_atTop.2 ?_
  intro ε he
  have hderiv_small : ∀f y : ℝ in Filter.atTop, |deriv f y - L| < ε / h := by
    have hb := hf.eventually (Metric.ball_mem_nhds L (div_pos he hh))
    refine hb.mono ?_
    intro y hy
    simpa [Metric.mem_ball, Real.dist_eq] using hy
  obtain (N, hN) := Filter.eventually_atTop.1 hderiv_small
  refine (max N 1, ?_)
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left N 1) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hcc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hoo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont : ContinuousOn f (Set.Icc x (x + h)) := hfd.continuousOn.mono hcc
  have hdiff : DifferentiableOn ℝ f (Set.Ioo x (x + h)) := hfd.mono hoo
  have hxlt : x < x + h := lt_add_of_pos_right x hx
  obtain (c, hc, hcderiv) := exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
  (hab := hxlt) (hfc := hcont) (hfd := hdiff)
  have hinc : f (x + h) - f x = h * deriv f c := by
    have hden : (x + h) - x = h := by ring
    have heq : deriv f c = (f (x + h) - f x) / h := by
      simpa [hden] using hcderiv
    calc
      f (x + h) - f x = ((f (x + h) - f x) / h) * h := by field_simp
      _ = deriv f c * h := by rw [heq]
      _ = h * deriv f c := by ring
  have hcge : N ≤ c := le_trans hxN (le_of_lt hc.1)
  have hbound : |deriv f c - L| < ε / h := hN c hcge
  have hscaled : h * |deriv f c - L| < h * (ε / h) :=
    mul_lt_mul_of_pos_left hbound hh
  have hquot : h * (ε / h) = ε := by field_simp
  rw [hquot] at hscaled
  rw [Real.dist_eq, hinc, ← mul_sub, abs_mul, abs_of_pos hh]
  exact hscaled
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__0c421778

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 484916ad7b404fef

4.395 395. bounded_shift_difference_tendsto

The problem

Suppose f is differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. For every $B > 0$ and every fixed shift h with $0 < h \leq B$, prove that $f(x+h) - f(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Lean statement

```
theorem bounded_shift_difference_tendsto
  {f : ℝ → ℝ} {B : ℝ}
  (hB : 0 < B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ h : ℝ, h ∈ Set.Ioc 0 B →
    Tendsto (fun x => f (x + h) - f x) atTop (ℕ 0)
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℕ 0) := by
```

What it contributes. Generalizes the parent's mean-value-theorem argument from unit shift to a parameter h in the bounded interval $(0,B]$, with the derivative estimate scaled by ε/B .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the unit shift to an arbitrary positive bounded shift and introduces new scaling reasoning via ε/B and $h \leq B$. Its proof must control the mean-value slope over an interval of symbolic length h , so recalling the parent's unit-interval argument is insufficient.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the unit shift to an arbitrary positive bounded shift and must scale the derivative bound by ε/B before applying the mean value theorem. The parent's fixed-unit argument does not supply this parameterized estimate.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the increment by an arbitrary positive bounded shift and must scale the derivative tolerance by ε/B , then use $h \leq B$ to recover the ε -bound. The parent's unit-increment argument does not supply this uniform-in- h estimate.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2137 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_difference_tendsto.extracted_1_1 : ∀ {f : ℝ → ℝ} {B : ℝ},
  0 < B →
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℳ 0) → ∀ h ∈ Set.Ioc 0 B, Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0)
```

Read back by a model that saw only the Lean. For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ and every real number B , if $B > 0$, f is differentiable on $(0, \infty)$, and the derivative f' tends to 0 as x tends to $+\infty$, then for every $h \in (0, B]$, the function $x \mapsto f(x+h) - f(x)$ tends to 0 as x tends to $+\infty$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_difference_tendsto
  {f : ℝ → ℝ} {B : ℝ}
  (hB : 0 < B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  ∀ h : ℝ, h ∈ Set.Ioc 0 B →
    Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0) := by
  intro h hh
  have hhpos : 0 < h := hh.1
  refine Metric.tendsto_atTop.2 ?_
  intro ε εpos
  have hεB : 0 < ε / B := div_pos εpos hB
  have hderiv_small : ∀f y : ℝ in atTop, |deriv f y| < ε / B := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hεB)
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N 1, ?_⟩
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left _ _) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right _ _) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hsubset_cc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + h)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + h)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + h := lt_add_of_pos_right x hhpos
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
  have hxden : (x + h) - x = h := by ring
  have hc_deriv' : deriv f c = (f (x + h) - f x) / h := by
    simp [hxden] using hc_deriv
  have hdiff_eq : deriv f c * h = f (x + h) - f x := by
    rw [hc_deriv']
```

```

field_simp [ne_of_gt hhpos]
have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
have hbound : |deriv f c| < ε / B := hN c hc_ge
have hprod : |deriv f c| * h < ε := by
  calc
    |deriv f c| * h < (ε / B) * h := mul_lt_mul_of_pos_right hbound hhpos
    _ ≤ (ε / B) * B := mul_le_mul_of_nonneg_left hh.2 (le_of_lt hεB)
    _ = ε := by field_simp [ne_of_gt hB]
have hgoal : |f (x + h) - f x| < ε := by
  rw [← hdiff_eq, abs_mul]
  simpa [abs_of_pos hhpos] using hprod
  simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__3b64044d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3b64044d16efdd86

4.396 396. bounded_shift_difference

The problem

Let f be differentiable on the positive reals and suppose $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. For every $B \geq 0$, prove that $f(x+h) - f(x) \rightarrow 0$ uniformly for $0 \leq h \leq B$.

Lean statement

```
theorem bounded_shift_difference
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℕ 0)) :
  ∀ B : ℝ, 0 ≤ B →
    ∀ ε : ℝ, 0 < ε →
      ∃ N : ℝ, ∀ x : ℝ, N ≤ x → ∀ h ∈ Set.Icc 0 B,
        dist (f (x + h) - f x) 0 < ε
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℕ 0) := by
```

What it contributes. Reuses the parent's derivative-at-infinity estimate, positive-domain restriction, and mean-value-theorem argument, while replacing the fixed unit interval by a quantified bounded interval and explicitly handling $h=0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the increment by an arbitrary bounded shift and proves uniform convergence over $h \in [0, B]$. The parent's fixed-unit mean-value argument does not supply the epsilon scaling, interval control, or separate zero-shift case needed here.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely uniform bounded-shift estimate, handling every $h \in [0, B]$ with a derivative bound scaled by $1+B$ and a separate zero-shift case. This requires new reasoning beyond the parent's fixed unit increment and is not merely the kept fixed-parameter theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely upgrades a fixed unit increment to a uniform estimate over every shift h in a bounded interval, requiring a new zero-shift case and scaling the derivative bound by $1+B$. This uniform bounded-shift argument is not supplied by the parent's single-interval proof or by the fixed-parameter limit statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2217 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_difference.extracted_1_1 : ∀ {f : ℝ → ℝ},
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℕ 0) →
  ∀ (B : ℝ),
    0 ≤ B → ∀ (ε : ℝ), 0 < ε → ∃ N, ∀ (x : ℝ), N ≤ x → ∀ h ∈ Set.Icc 0 B, dist (f (x + h) - f x) 0 < ε
```

Read back by a model that saw only the Lean. For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ that is differentiable on the open interval $(0, \infty)$ and whose derivative tends to 0 as x tends to $+\infty$, for every real number $B \geq 0$ and every $\varepsilon > 0$, there exists a real number N such that, for every real $x \geq N$ and every $h \in [0, B]$, the distance between $f(x + h) - f(x)$ and 0 is less than ε .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_difference
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℕ 0)) :
  ∀ B : ℝ, 0 ≤ B →
  ∀ ε : ℝ, 0 < ε →
    ∃ N : ℝ, ∀ x : ℝ, N ≤ x → ∀ h ∈ Set.Icc 0 B,
      dist (f (x + h) - f x) 0 < ε := by
  intro B hB ε hε
  have hden : 0 < 1 + B := by linarith
  have hsmall : 0 < ε / (1 + B) := div_pos hε hden
  have hderiv_small : ∀f y : ℝ in Filter.atTop, |deriv f y| < ε / (1 + B) := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hsmall)
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N (1 + B), ?_⟩
  intro x hx h hmem
  have hxN : N ≤ x := le_trans (le_max_left N (1 + B)) hx
  have hx1B : 1 + B ≤ x := le_trans (le_max_right N (1 + B)) hx
  have hxpos : 0 < x := by linarith
  rcases eq_or_ne h 0 with rfl | hne
  · simp using hε
  have hpos : 0 < h := lt_of_le_of_ne hmem.1 (Ne.symm hne)
  have hsubset_cc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + h)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + h)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + h := lt_add_of_pos_right x hpos
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
  (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
```



```

have hdiff_eq : deriv f c = (f (x + h) - f x) / h := by
  simpa [sub_eq_add_neg] using hc_deriv
have hinc : f (x + h) - f x = h * deriv f c := by
  rw [hdiff_eq]
  field_simp
have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
have hbound : |deriv f c| < ε / (1 + B) := hN c hc_ge
have hprod : h * |deriv f c| ≤ B * (ε / (1 + B)) := by
  exact mul_le_mul hmem.2 (le_of_lt hbound) (abs_nonneg _) hB
have hBlt : B < 1 + B := by linarith
have hstrict : B * (ε / (1 + B)) < ε := by
  calc
    B * (ε / (1 + B)) < (1 + B) * (ε / (1 + B)) :=
      mul_lt_mul_of_pos_right hBlt hsmall
    _ = ε := by field_simp
have habs : |f (x + h) - f x| = h * |deriv f c| := by
  rw [hinc, abs_mul, abs_of_pos hpos]
have hgoal : |f (x + h) - f x| < ε := by
  calc
    |f (x + h) - f x| = h * |deriv f c| := habs
    _ ≤ B * (ε / (1 + B)) := hprod
    _ < ε := hstrict
simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__46380a86

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 46380a86008e6a6f

4.397 397. Rudin_exercise_5_5_unit_oscillation

The problem

Suppose f is differentiable for every positive input and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Show that for every $\varepsilon > 0$, there is N such that whenever $x, y \geq N$ and $|x - y| \leq 1$, one has $|f(x) - f(y)| < \varepsilon$.

Lean statement

```
theorem Rudin_exercise_5_5_unit_oscillation
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x y : ℝ, N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε
```

Parent. Suppose a response curve is differentiable for every positive input and its derivative tends to zero as the input grows without bound. Show that for every tolerance $\varepsilon > 0$, the change in response between inputs one unit apart is eventually smaller than ε .

Parent in Lean

```
theorem Rudin_exercise_5_5_eventual
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x : ℝ, N ≤ x → |f (x + 1) - f x| < ε
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the fixed unit-step estimate to a uniform bound for every pair of sufficiently large points with distance at most 1. The parent's proof of the specific increment does not supply this two-variable oscillation claim; the child needs a new ordering and arbitrary-interval mean-value argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the conclusion from a fixed unit increment to uniform oscillation over every pair of sufficiently large points within distance 1. Its proof must handle arbitrary interval orientation and variable interval lengths, with a new mean-value estimate using the distance bound; the parent's fixed-step proof does not directly supply this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed unit increment into a uniform oscillation bound for every pair of sufficiently large points at distance at most one. Its proof must handle arbitrary interval lengths, order both endpoints, and combine the mean value theorem with the distance bound, which the parent's fixed-step argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2830 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Rudin_exercise_5_5_unit_oscillation.extracted_1_1 : ∀ {f : ℝ → ℝ},
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℕ 0) →
  ∀ (ε : ℝ), 0 < ε → ∃ N, ∀ (x y : ℝ), N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is differentiable on $(0, \infty)$ and its derivative tends to 0 as x tends to $+\infty$, then for every $\varepsilon > 0$ there exists a real number N such that for all real numbers x and y satisfying $N \leq x$, $N \leq y$, and $|x - y| \leq 1$, we have $|f(x) - f(y)| < \varepsilon$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem Rudin_exercise_5_5_unit_oscillation
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x y : ℝ, N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε := by
  intro ε hε
  have hsmall : ∀ z : ℝ in atTop, |deriv f z| < ε := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hε)
    filter_upwards [hball] with z hz
    simpa [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hz
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hsmall
  refine ⟨max N 1, ?_⟩
  intro x y hx hy hxy
  have hforward : ∀ a b : ℝ, max N 1 ≤ a → max N 1 ≤ b → a ≤ b → |b - a| ≤ 1 → |f b - f a| < ε := by
    intro a b ha hb hab hd
    by_cases heq : a = b
    · simpa [heq] using hε
    have hlt : a < b := lt_of_le_of_ne hab heq
    have ha1 : (1 : ℝ) ≤ a := le_of_max_le_right ha
    have hapos : 0 < a := lt_of_lt_of_le zero_lt_one ha1
    have hcc : Set.Icc a b ⊆ Set.Ioi (0 : ℝ) := by
      intro z hz
      exact lt_of_lt_of_le hapos hz.1
    have hoo : Set.Ioo a b ⊆ Set.Ioi (0 : ℝ) := by
      intro z hz
      exact lt_trans hapos hz.1
    have hcont : ContinuousOn f (Set.Icc a b) := hfd.continuousOn.mono hcc
    have hdiff : DifferentiableOn ℝ f (Set.Ioo a b) := hfd.mono hoo
    obtain ⟨c, hc, hcs⟩ := exists_deriv_eq_slope (f := f) (a := a) (b := b)
    (hab := hlt) (hfc := hcont) (hfd := hdiff)
    have hcN : N ≤ c := le_trans (le_trans (le_max_left N 1) ha) (le_of_lt hc.1)
    have hcbound : |deriv f c| < ε := hN c hcN
    have hdiff_eq : f b - f a = deriv f c * (b - a) := by
      rw [hcs]
      field_simp
    have habs : |f b - f a| = |deriv f c| * |b - a| := by
      rw [hdiff_eq, abs_mul]
    have hdp : 0 < |b - a| := abs_pos.mpr (sub_ne_zero.mpr (ne_of_gt hlt))
    have hmul : |deriv f c| * |b - a| < ε * |b - a| :=
      mul_lt_mul_of_pos_right hcbound hdp
    have hdist : |b - a| ≤ 1 := hd
    rw [habs]
    nlinarith
  rcases le_total x y with hxy' | hxy'
```

```
• have h := hforward x y hx hy hxy' (by simpa [abs_sub_comm] using hxy)
  simpa [abs_sub_comm] using h
• exact hforward y x hy hx hxy' hxy
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__ms.mh__9761fa5f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 9761fa5f46749496

4.398 398. weighted_summable_majorant_complex_partial_sums

The problem

Let (a_n) be a sequence of complex numbers dominated in norm by a summable real sequence (g_n) . Show that for any complex weights (c_n) with $|c_n| \leq 1$, the weighted partial sums $\sum_{k < n} c_k a_k$ converge.

Lean statement

```
theorem weighted_summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (c : ℕ → ℂ)
  (hc : ∀ n : ℕ, ‖c n‖ ≤ 1)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℕ y)
```

Parent. Let (a_n) be a sequence of complex numbers. If there is a summable nonnegative majorant (g_n) with $\|a_n\| \leq g_n$ for every n , prove that the partial sums of $\sum a_n$ converge.

Parent in Lean

```
theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℕ y)
```

What it contributes. Preserves the parent's majorant-to-summability and partial-sum convergence checkpoints, while applying them to the new weighted sequence $c_n a_n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the summands by arbitrary complex weights and requires new reasoning to establish that the weighted terms remain dominated by the summable majorant. The parent's convergence argument alone does not provide this weighted domination step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the summands by an arbitrary bounded complex coefficient sequence and requires proving a new norm domination before establishing summability. The parent's direct majorant argument does not supply this weighted domination step.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes each summand by an arbitrary complex weight sequence and must prove the new norm domination using multiplicativity and the bound on c . This reasoning is not supplied by the parent proof, which can only directly establish summability of a .

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2153 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_summable_majorant_complex_partial_sums.extracted_1_1 (a : ℕ → ℂ) (g : ℕ → ℝ) (hmajorant : Summable g)
(hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (c : ℕ → ℂ) (hc : ∀ (n : ℕ), ‖c n‖ ≤ 1) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, c i * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a sequence $a : \mathbb{N} \rightarrow \mathbb{C}$, a sequence $g : \mathbb{N} \rightarrow \mathbb{R}$ with summable series, and a sequence $c : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $\|a(n)\| \leq g(n)$ and $\|c(n)\| \leq 1$ for every n , if $s : \mathbb{N} \rightarrow \mathbb{C}$ is defined by $s(n) = \sum_{\{i < n\}} c(i) a(i)$, then there exists $y \in \mathbb{C}$ such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem weighted_summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
(hmajorant : Summable g)
(hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
(c : ℕ → ℂ)
(hc : ∀ n : ℕ, ‖c n‖ ≤ 1)
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)) :
∃ y : ℂ, Tendsto s atTop (ℳ y) := by
have hweighted : ∀ n : ℕ, ‖c n * a n‖ ≤ g n := by
intro n
calc
‖c n * a n‖ = ‖c n‖ * ‖a n‖ := norm_mul _ _
_ ≤ 1 * ‖a n‖ := mul_le_mul_of_nonneg_right (hc n) (norm_nonneg _)
_ = ‖a n‖ := one_mul _
_ ≤ g n := hdom n
have hsum : Summable (fun n : ℕ => c n * a n) := by
exact Summable.of_norm_bounded hmajorant hweighted
have hpartial :
Tendsto (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)
atTop (ℳ (∑' n, c n * a n)) :=
hsum.hasSum.tendsto_sum_nat
refine (∑' n, c n * a n, ?_)
simp [h] using hpartial
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me.mh__4ce18d21

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4ce18d21a38cae68

4.399 399. dominated_phase_series_converges_bounded

The problem

Let z be a complex number with $|z| \leq 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Lean statement

```
theorem dominated_phase_series_converges_bounded (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n)
  ↪ (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  ↪ : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retained the parent's phased-coefficient and summable-majorant framework, while replacing the unit-modulus step with an inductive bound for powers under $\|z\| \leq 1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely weakens $\|z\| = 1$ to $\|z\| \leq 1$. Its proof must establish the new bound $\|z^{(n+1)}\| \leq 1$ by induction before applying domination, which is reasoning absent from the parent proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weakened hypothesis $\|z\| \leq 1$ requires a new induction proving $\|z^{(n+1)}\| \leq 1$ before establishing domination; the parent's direct norm-equality simplification no longer applies. The convergence argument therefore depends on genuinely new mathematics, and the statement accurately matches the Lean theorem.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the phase assumption from $\|z\| = 1$ to $\|z\| \leq 1$ and supplies a new inductive argument establishing $\|z\|^k \leq 1$. This produces the domination needed for summability, so the parent's proof cannot simply be recalled.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2197 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_converges_bounded.extracted_1_1 (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ)
  (s : ℕ → ℂ) (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm at most 1, functions $a : \mathbb{N} \rightarrow \mathbb{C}$ and $g : \mathbb{N} \rightarrow \mathbb{R}$ satisfying $\|a(n)\| \leq g(n)$ for every natural number n , and such that the series $\sum n, g(n)$ is summable, suppose that $s : \mathbb{N} \rightarrow \mathbb{C}$ is defined by $s(n) = \sum_{i=0}^{n-1} z^{i+1} a(i)$. Then there exists a complex number y such that the sequence $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem dominated_phase_series_converges_bounded (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hpow : ∀ k : ℕ, ‖z‖ ^ k ≤ 1 := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      rw [pow_succ]
      calc
        ‖z‖ ^ k * ‖z‖ ≤ 1 * ‖z‖ :=
          mul_le_mul_of_nonneg_right ih (norm_nonneg z)
        _ ≤ 1 := by simpa using hz
  have hpower : ∀ n : ℕ, ‖z‖ ^ (n + 1) ≤ 1 := by
    intro n
    rw [norm_pow]
    exact hpow (n + 1)
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    calc
      ‖f n‖ = ‖z‖ ^ (n + 1) * ‖a n‖ := by simp [f, norm_mul]
      _ ≤ 1 * ‖a n‖ := mul_le_mul_of_nonneg_right (hpower n) (norm_nonneg (a n))
      _ = ‖a n‖ := by simp
      _ ≤ g n := hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hsum : HasSum f (∑' k, f k) := hf.hasSum
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    simpa using hsum.tendsto_sum_nat
  refine (∑' k, f k, ?_)
  simpa [h, f] using hconv
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.mh__7d0d3155

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 7d0d315577046b18

4.400 400. dominated_phase_series_has_unique_limit

The problem

Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , then the partial sums of $\sum z^{n+1} a_n$ converge to a unique limit.

Lean statement

```
theorem dominated_phase_series_has_unique_limit (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n)
  (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃! y, Tendsto s atTop (ℕ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℕ y)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of a limit into existence and uniqueness of the limit. Its uniqueness branch requires the separate Hausdorff-limit argument via ‘tendsto_nhds_unique’, which the parent’s proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into uniqueness of the limit, requiring a genuinely new Hausdorff-space uniqueness argument via ‘tendsto_nhds_unique’; the witness is not forced by a wrapper hypothesis. The parent’s convergence construction supplies existence but does not prove the added uniqueness obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence to a genuine uniqueness statement. After establishing convergence, it must additionally use Hausdorff-limit uniqueness to prove that every possible limit equals the constructed one; this is not supplied by the parent’s proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2178 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_has_unique_limit.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) :
```

$\exists! y, \text{Tendsto } s \text{ atTop } (\mathcal{U} \ y)$

Read back by a model that saw only the Lean. For a complex number z with norm 1, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$, $g : \mathbb{N} \rightarrow \mathbb{R}$, and $s : \mathbb{N} \rightarrow \mathbb{C}$, suppose that for every natural number n , $\|a \ n\| \leq g \ n$, that the real series $\sum_n g \ n$ is summable, and that for every n , $s \ n = \sum_{i < n} \{i < n\} z^{i+1} a \ i$. Then there exists a unique complex number y such that the sequence s converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem dominated_phase_series_has_unique_limit (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃! y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    simp [f, norm_mul, norm_pow, hz] using hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  have hsconv : Tendsto s atTop (ℳ (∑' k, f k)) := by
    simp [h, f] using hconv
  refine ⟨∑' k, f k, hsconv, ?_⟩
  intro y hy
  exact (tendsto_nhds_unique hsconv hy).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.mh__97dfff79

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 97dfff79790543fe

4.401 401. Shakarchi_exercise_1_19b_unique

The problem

Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle, and that its limit is unique.

Lean statement

```
theorem Shakarchi_exercise_1_19b_unique (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃! y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of a limit into existence and uniqueness of the limit. Although the convergence argument is inherited, the added obligation requires a separate Hausdorff-topology uniqueness argument, so recalling the parent alone does not complete the proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into existence and uniqueness, requiring the new Hausdorff-limit argument via ‘tendsto_nhds_unique’; the parent’s convergence proof alone cannot discharge that obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into existence and uniqueness of the limit. The parent’s summability argument supplies existence, but the new uniqueness obligation requires a separate Hausdorff-limit argument and is not a forced-witness wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propect, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2901 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Shakarchi_exercise_1_19b_unique.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (i + 1) ^ 2) : ∃! y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm 1, and a sequence s of complex numbers defined by $s(n) =$ the sum over all natural numbers $i < n$ of z^{i+1} divided by $(i+1)^2$, there exists a unique complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem Shakarchi_exercise_1_19b_unique (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃! y, Tendsto s atTop (ℕ y) := by
  classical
  set f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ 2 with hfdef
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    funext n
    have hzpow : ‖z ^ (n + 1)‖ = 1 := by
      simp [hz, Complex.norm_pow z (n + 1)]
    have hdenpow :
      ‖(n + 1 : ℂ) ^ 2‖ = (n + 1 : ℝ) ^ 2 := by
      have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
        simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
      simp [pow_two, hden]
    calc
      ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ 2‖ := by simp [f]
      _ = 1 / ‖(n + 1 : ℂ) ^ 2‖ := by simp [hzpow]
      _ = (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by simp [hdenpow]
  have h_pSeries :
    Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
  have h_base :
    Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
    (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
  simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    simp [h_norm] using h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine (∑' n, f n, ?_, ?_)
  · have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
    simp [h, f] using hy.tendsto_sum_nat
  · intro y hy
    have hlim : Tendsto s atTop (ℕ (∑' n, f n)) := by
      have hsum : HasSum f (∑' n, f n) := hfSummable.hasSum
      simp [h, f] using hsum.tendsto_sum_nat
    exact (tendsto_nhds_unique hlim hy).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh__02a0eb22

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 02a0eb2227165634

4.402 402. Shakarchi_exercise_1_19b_mutation

The problem

Prove that for every complex number z on the unit circle and every integer $p > 1$, the power series $\sum z^n/n^p$ converges.

Lean statement

```
theorem Shakarchi_exercise_1_19b_mutation (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1) ^ p)) :
  ∃ y, Tendsto s atTop (ℕ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The exponent 2 is genuinely parameterized to an arbitrary natural number $p > 1$. The proof must establish p -series summability using the new hypothesis hp , which is not supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the denominator exponent from fixed 2 to arbitrary $p > 1$, requiring a new p -series summability argument and carrying that parameter through the norm calculation. The parent proof does not establish this family, so recalling it is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the denominator exponent from 2 to an arbitrary natural p with $1 < p$. Its proof requires the generalized p -series summability theorem and uses the new hypothesis hp , so recalling the parent's fixed-exponent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2930 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Shakarchi_exercise_1_19b_mutation.extracted_1_1 (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (i + 1) ^ p) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. Let z be a complex number, let p be a natural number with $1 < p$, and suppose that the norm of z is 1. Let s be a sequence of complex numbers satisfying, for every natural number n , $s(n) = \sum_{i=0}^{n-1} z^{i+1} / (i+1)^p$. Then there exists a complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem Shakarchi_exercise_1_19b_mutation (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  classical
  set f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ p with hfdef
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p := by
  funext n
  have hzpow : ‖z ^ (n + 1)‖ = 1 := by
    simp [hz]
  have hdenpow :
    ‖(n + 1 : ℂ) ^ p‖ = (n + 1 : ℝ) ^ p := by
  have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
    simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
    simp [hden]
  calc
    ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ p‖ := by simp [f]
    _ = 1 / ‖(n + 1 : ℂ) ^ p‖ := by rw [hzpow]
    _ = (1 : ℝ) / (n + 1 : ℝ) ^ p := by rw [hdenpow]
  have h_pSeries :
    Summable fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p := by
  have h_base :
    Summable fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p :=
    (Real.summable_one_div_nat_pow (p := p)).mpr hp
  simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable fun n : ℕ => ‖f n‖ := by
    rw [h_norm]
    exact h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine ⟨∑' n, f n, ?_⟩
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  have hconv : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℳ (∑' n, f n)) :=
    hy.tendsto_sum_nat
  simp [h, f] using hconv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh__0fb1e4c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0fb1e4c08ebaa52d

4.403 403. aime_1983_p9_mutation

The problem

Let $x \in \mathbb{R}$, let $S = (0, \pi)$, and define $f(t) = \frac{9t^2 \sin^2 t + 4}{t \sin t}$. For every y of the form $f(z)$ with $z \in S$, we have $12 \leq y$, and $y = 12$ if and only if there exists $z \in S$ such that $z \sin z = \frac{2}{3}$ and $y = f(z)$.

Lean statement

```
theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  ∀ y ∈ Set.image f S,
    12 ≤ y ∧
    (y = 12 ↔ ∃ z ∈ S, z * Real.sin z = 2 / 3 ∧ y = f z) := by
```

Parent. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 12.

Parent in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an IsLeast assertion to a pointwise characterization of exactly when an image value equals 12. Besides reproving the lower bound, it must derive the equality condition $t = z \cdot \sin z = 2/3$ and prove both directions of the equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum claim into a full equality characterization: it must prove that equality forces $z \cdot \sin z = 2/3$, then establish the converse for any witness. This equality-case reasoning is not supplied by the parent's IsLeast proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum claim into an equality characterization for every image value. Proving that equality forces $z \cdot \sin z = 2/3$ requires a new algebraic equality-case argument beyond the parent's lower-bound and witness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1833 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1983_p9_mutation.extracted_1.1 : ∀ (S : Set ℝ) (f : ℝ → ℝ),
  (f = fun x => (9 * (x ^ 2 * sin x ^ 2) + 4) / (x * sin x)) →
    S = Set.Ioo 0 π → ∀ y ∈ f '' S, 12 ≤ y ∧ (y = 12 ↔ ∃ z ∈ S, z * sin z = 2 / 3 ∧ y = f z)
```

Read back by a model that saw only the Lean. For every set S of real numbers and every function f from the real numbers to the real numbers, if $f(x) = (9(x^2 \sin^2 x) + 4)/(x \sin x)$ for all real x , and S is the open interval $(0, \pi)$, then for every real number y in the image $f(S)$, we have $12 \leq y$, and $y = 12$ if and only if there exists z in S such that $z \sin z = 2/3$ and $y = f(z)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  ∀ y ∈ Set.image f S,
    12 ≤ y ∧
      (y = 12 ↔ ∃ z ∈ S, z * Real.sin z = 2 / 3 ∧ y = f z) := by
  rw [h₀, h₁]
  intro y hy
  rcases hy with ⟨z, hz, rfl⟩
  have hvalue : ∀ q : ℝ, q ∈ Set.Ioo 0 Real.pi → q * Real.sin q = 2 / 3 →
    (9 * (q ^ 2 * Real.sin q ^ 2) + 4) / (q * Real.sin q) = 12 := by
    intro q hq hqval
    have hsq : q ^ 2 * Real.sin q ^ 2 = (q * Real.sin q) ^ 2 := by
      ring
    rw [hsq, hqval]
    norm_num
  have hzpos : 0 < z * Real.sin z := by
    exact mul_pos hz.1 (Real.sin_pos_of_pos_of_lt_pi hz.1 hz.2)
  let t : ℝ := z * Real.sin z
  have hsq : z ^ 2 * Real.sin z ^ 2 = t ^ 2 := by
    dsimp [t]
    ring
  have hbound : 12 ≤ (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) := by
    apply (le_div_iff hzpos).2
    rw [hsq]
    nlinarith [sq_nonneg (3 * t - 2)]
  constructor
  · exact hbound
  · constructor
    · intro heq
      have hmul := (div_eq_iff (ne_of_gt hzpos)).mp heq
      rw [hsq] at hmul
      have ht : t = 2 / 3 := by
        dsimp [t] at hmul
        nlinarith [sq_nonneg (3 * t - 2)]
      refine ⟨z, hz, ?, rfl⟩
      simp [t] using ht
    · rintro ⟨w, hw, hwt, hyw⟩
```



```
calc
  (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) =
    (9 * (w ^ 2 * Real.sin w ^ 2) + 4) / (w * Real.sin w) := hyw
  _ = 12 := hvalue w hw hwt
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__mh__1afd2f1c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8f8979c2b397776c

4.404 404. aime_1983_p9_mutation

The problem

For $0 < x < \pi$, prove that $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ has minimum value 12, that this value is attained, and that equality holds exactly when $x \sin x = \frac{2}{3}$.

Lean statement

```
theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 ∧
    (∃ z, z ∈ S ∧ f z = 12) ∧
    (∀ z, z ∈ S → (f z = 12 ↔ z * Real.sin z = 2 / 3))
```

Parent. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 012.

Parent in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added equivalence is a genuine characterization: proving that any point attaining the minimum must satisfy $z \backslash \sin z = 2/3$ requires a new reverse algebraic argument, not supplied by the parent's existence and lower-bound proof. The conjunction also retains the parent result, but the appended claim is mathematically substantive rather than decorative.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine characterization of every point attaining the minimum: proving $f(z)=12$ iff $z \backslash \sin z = 2/3$ requires an equality-case algebra argument not supplied by the parent's least-value proof. The existential conjunct is redundant, but the universal iff changes the proof obligation substantively.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2368 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1983_p9_mutation.extracted_1_1 (S : Set ℝ) (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * sin x ^ 2) + 4) / (x * sin x)) (h₁ : S = Set.Ioo 0 π) :
  IsLeast (f '' S) 12 ∧ (∃ z ∈ S, f z = 12) ∧ ∀ z ∈ S, f z = 12 ↔ z * sin z = 2 / 3
```

Read back by a model that saw only the Lean. Let S be a subset of the real numbers and let f be a real-valued function on the real numbers. Suppose that, for every real number x , $f(x) = (9(x^2 \sin^2 x) + 4)/(x \sin x)$, and that S is the open interval $(0, \pi)$. Then 12 is the least element of the image $f(S)$; there exists some z in S such that $f(z) = 12$; and for every z in S , $f(z) = 12$ if and only if $z \sin z = 2/3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 ∧
    (∃ z, z ∈ S ∧ f z = 12) ∧
    (∀ z, z ∈ S → (f z = 12 ↔ z * Real.sin z = 2 / 3)) := by
  subst f
  subst S
  let g : ℝ → ℝ := fun t => t * Real.sin t
  have hg : Continuous g := continuous_id.mul Real.continuous_sin
  have hpi : 0 ≤ Real.pi / 2 := by positivity
  have htarget : (2 / 3 : ℝ) ∈ Set.Icc (g 0) (g (Real.pi / 2)) := by
    constructor
    · norm_num [g]
    · simp [g, Real.sin_pi_div_two]
      nlinarith [Real.pi_gt_three]
  obtain ⟨z, hz, hzval⟩ :=
    intermediate_value_Icc hpi hg.continuousOn htarget
  have hzval' : z * Real.sin z = 2 / 3 := by
    simp [g] using hzval
  have hz0 : 0 < z := by
    have hzle : 0 ≤ z := hz.1
    have hzne : z ≠ 0 := by
      intro h
      subst z
      norm_num at hzval'
      exact lt_of_le_of_ne hzle (Ne.symm hzne)
  have hzpi : z < Real.pi := by
    nlinarith [hz.2, Real.pi_pos]
  have hleast : IsLeast (Set.image (fun x : ℝ => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x)) (Set.Ioo 0 Real.pi)) 12 := by
    constructor
    · refine ⟨z, ⟨hz0, hzpi⟩, ?_⟩
      change (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) = 12
      have hsq : z ^ 2 * Real.sin z ^ 2 = (z * Real.sin z) ^ 2 := by ring
      rw [hsq, hzval']
      norm_num
    · rintro y ⟨w, hw, rfl⟩
      have hwpos : 0 < w * Real.sin w := by
        exact mul_pos hw.1 (Real.sin_pos_of_pos_of_lt_pi hw.1 hw.2)
```

```

have hsq : w ^ 2 * Real.sin w ^ 2 = (w * Real.sin w) ^ 2 := by ring
apply (le_div_iff hwpos).2
rw [hsq]
nlinarith [sq_nonneg (3 * (w * Real.sin w) - 2)]
refine (hleast, ?_, ?_)
· rcases hleast.1 with ⟨z, hz, hzval⟩
  exact ⟨z, hz, hzval⟩
· intro z hz
  have hsin : 0 < Real.sin z := by
    exact Real.sin_pos_of_pos_of_lt_pi hz.1 hz.2
  have hypos : 0 < z * Real.sin z := by
    exact mul_pos hz.1 hsin
  have hsq : z ^ 2 * Real.sin z ^ 2 = (z * Real.sin z) ^ 2 := by ring
  constructor
  · intro heq
    have heq' : (9 * (z * Real.sin z) ^ 2 + 4) / (z * Real.sin z) = 12 := by
      simp only [hsq] using heq
    have hmul : 9 * (z * Real.sin z) ^ 2 + 4 = 12 * (z * Real.sin z) := by
      exact (div_eq_iff (ne_of_gt hypos)).mp heq'
    have hzero : (3 * (z * Real.sin z) - 2) ^ 2 = 0 := by
      nlinarith [hmul]
    nlinarith [hzero]
  · intro heq
    calc
      (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) =
        (9 * (z * Real.sin z) ^ 2 + 4) / (z * Real.sin z) := by rw [hsq]
      _ = 12 := by rw [heq]; norm_num

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__mh__6f3029bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6f3029bc9d9f901a

4.405 405. aime_1984_p15_squared_tuple

The problem

Suppose real numbers x, y, z, w satisfy the four weighted-square equations for $n = 2, 4, 6, 8$. Determine the exact values of x^2, y^2, z^2, w^2 .

Lean statement

```
theorem aime_1984_p15_squared_tuple (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hy : y ^ 2 = 10395 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hz : z ^ 2 = 9009 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hw : w ^ 2 = 6435 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  exact ⟨hx, hy, hz, hw⟩
```

Parent. Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Parent in Lean

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from determining only x^2 to determining the complete tuple of squared variables. The parent's proof supplies only the x^2 component; proving the three additional exact values requires extending the linear-system reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining only x^2 to determining the complete tuple (x^2, y^2, z^2, w^2) . The parent's proof supplies only the first component, so the additional three values require genuinely new deductions from the coupled equations.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining only x^2 to determining the complete tuple (x^2, y^2, z^2, w^2) . Proving the additional coordinates requires extracting new information from the coupled linear system, beyond the parent's single-coordinate result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2722 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_squared_tuple.extracted_1_1 (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_squared_tuple (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hy : y ^ 2 = 10395 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hz : z ^ 2 = 9009 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hw : w ^ 2 = 6435 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  exact ⟨hx, hy, hz, hw⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms.mh__5434b06d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5434b06d384cdcad

4.406 406. aime_1984_p15_x_two_roots

The problem

Suppose real numbers x, y, z, w satisfy the four weighted-square equations above. Show that $x = 105/32$ or $x = -105/32$.

Lean statement

```
theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x = 105 / 32 ∨ x = -(105 / 32) := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
    nlinarith [hx_sq]
  rcases mul_eq_zero.mp hfactor with hpos | hneg
  · left
    nlinarith
  · right
    nlinarith
```

Parent. Suppose x, y, z, w are real numbers satisfying the four weighted-square equations above, and $x \geq 0$. Determine x .

Parent in Lean

```
theorem aime_1984_p15_x (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
  (hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

What it contributes. Reuses the parent’s weighted-square hypotheses and linear-elimination checkpoint, while removing the sign hypothesis and adding an explicit square-factorization step for the two-root conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the sign hypothesis and replaces the single signed-value conclusion with a genuine two-root classification. After deriving the squared value as in the parent, it must factor the quadratic and perform a zero-product case split, which the parent’s final nonnegativity argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the nonnegativity hypothesis and correctly classifies both possible real roots from the derived squared value. Its factorization and case split require reasoning not supplied by the parent’s sign-based conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the sign hypothesis and changes the conclusion to a complete two-root classification. The parent's nonnegativity-based final step cannot establish the negative branch, so recalling the parent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2701 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_two_roots.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x = 105 / 32 ∨ x = -(105 / 32)
```

Read back by a model that saw only the Lean. For real numbers $x, y, z,$ and w , suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x = 105/32$ or $x = -(105/32)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x = 105 / 32 ∨ x = -(105 / 32) := by
norm_num at h₀ h₁ h₂ h₃
have hx_sq : x ^ 2 = 11025 / 1024 := by
  linarith [h₀, h₁, h₂, h₃]
have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
  nlinarith [hx_sq]
rcases mul_eq_zero.mp hfactor with hpos | hneg
· left
  nlinarith
· right
  nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh__9b2d27e0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9b2d27e03931a264

4.407 407. unique_affine_coefficients

The problem

For any complex numbers a , b , and c , show that there exists a unique pair of complex numbers (u, v) such that, for every complex number d , $(a - d)(a - c)(a - b) = ud + v$.

Lean statement

```
theorem unique_affine_coefficients (a b c : ℂ) :
  ∃! p : ℂ × ℂ, ∀ d : ℂ,
    (a - d) * (a - c) * (a - b) = p.1 * d + p.2
```

Parent. Show that for any complex numbers a, b, c, d , $(a-d)(a-c)(a-b) = -(((a^2-(b+c)a)+cb)d)+(a^2-(b+c)a+cb)a$.

Parent in Lean

```
theorem algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta (a b c d : ℂ) :
  (a - d) * (a - c) * (a - b) =
  -((a ^ 2 - (b + c) * a + c * b) * d) + (a ^ 2 - (b + c) * a + c * b) * a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the goal from an algebraic identity to a uniqueness statement for affine coefficients. Although the parent suggests the witness and helps with existence, the child additionally proves coefficient uniqueness by evaluating at $d = 0$ and $d = 1$, which the parent's ring proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an algebraic identity into a genuine existence-and-uniqueness characterization of the affine coefficients. Its proof must establish uniqueness by evaluating at two distinct values and solving the resulting equations, which the parent's ring proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine uniqueness theorem for affine coefficients. Its proof requires extracting the coefficients by evaluating at $d = 0$ and $d = 1$ and proving product equality, reasoning not supplied by the parent's direct ring normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_affine_coefficients.extracted_1_1 (a b c : ℂ) :  
  ∃! p, ∀ (d : ℂ), (a - d) * (a - c) * (a - b) = p.1 * d + p.2
```

Read back by a model that saw only the Lean. For any complex numbers a , b , and c , there exists a unique ordered pair p of complex numbers such that, for every complex number d , $(a - d)(a - c)(a - b) = p_1 d + p_2$, where p_1 and p_2 are the first and second components of p .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_affine_coefficients (a b c : ℂ) :  
  ∃! p : ℂ × ℂ, ∀ d : ℂ,  
    (a - d) * (a - c) * (a - b) = p.1 * d + p.2 := by  
  refine ⟨(-(a - c) * (a - b)), a * (a - c) * (a - b)⟩, ?_, ?_  
  · intro d  
    ring  
  · intro p hp  
    have h0 := hp 0  
    have h1 := hp 1  
    apply Prod.ext  
    · change p.1 = -(a - c) * (a - b)  
      linear_combination h0 - h1  
    · change p.2 = a * (a - c) * (a - b)  
      linear_combination -h0
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__mh__9b14d385

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8a5c6f9045c9c663

4.408 408. reciprocal_equality_characterization

The problem

For positive real numbers x , y , and z satisfying $x + y + z = 1$, prove that $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ if and only if $x = y = z = 1/3$.

Lean statement

```
theorem reciprocal_equality_characterization (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z) (hsum : x + y + z = 1) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔
  x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3
```

Parent. Show that for any three positive real numbers x , y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxypzleqsum2onxpy (x y z : ℝ) (he : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Supplies the reciprocal-sum lower-bound structure and its denominator-clearing bound checkpoint; the child adds a new equality-gap rigidity argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an equality characterization, requiring extraction of equality from a sum-of-squares identity and then using $x+y+z=1$ to derive $x=y=z=1/3$. The parent's proof establishes nonnegativity but does not supply the zero-case analysis or uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an equality characterization: it must extract vanishing of each squared-difference term and then derive $x=y=z=1/3$. The parent only proves nonnegativity and supplies no equality-case reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an inequality into a genuine equality characterization. Its forward direction requires proving that equality forces every nonnegative squared-difference term to vanish, then deriving $x=y=z=1/3$; the parent only establishes nonnegativity and supplies no equality-case reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1844 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equality_characterization.extracted_1_1 (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z)
(hsum : x + y + z = 1) :
9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3
```

Read back by a model that saw only the Lean. For real numbers x, y , and z satisfying $x > 0, y > 0, z > 0$, and $x + y + z = 1$, the equality $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ holds if and only if $x = 1/3, y = 1/3$, and $z = 1/3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem reciprocal_equality_characterization (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z) (hsum : x + y + z = 1) :
9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔
x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3 := by
rcases hpos with ⟨hx, hy, hz⟩
have hxy : 0 < x + y := by linarith
have hyz : 0 < y + z := by linarith
have hzx : 0 < z + x := by linarith
have hxyz : 0 < x + y + z := by linarith
constructor
· intro heq
have hpoly : 9 * (x + y) * (y + z) * (z + x) =
2 * (x + y + z) * ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) := by
field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] at heq
nlinarith [heq]
have hid : 2 * (x + y + z) * ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -
9 * (x + y) * (y + z) * (z + x) =
(x + y) * ((y + z) - (z + x)) ^ 2 +
(y + z) * ((z + x) - (x + y)) ^ 2 +
(z + x) * ((x + y) - (y + z)) ^ 2 := by ring
have hzero : (x + y) * ((y + z) - (z + x)) ^ 2 +
(y + z) * ((z + x) - (x + y)) ^ 2 +
(z + x) * ((x + y) - (y + z)) ^ 2 = 0 := by
rw [← hid]
nlinarith [hpoly]
have ht1 : 0 ≤ (x + y) * ((y + z) - (z + x)) ^ 2 := by positivity
have ht2 : 0 ≤ (y + z) * ((z + x) - (x + y)) ^ 2 := by positivity
have ht3 : 0 ≤ (z + x) * ((x + y) - (y + z)) ^ 2 := by positivity
have h1 : (x + y) * ((y + z) - (z + x)) ^ 2 = 0 := by
nlinarith [hzero, ht2, ht3]
have h2 : (y + z) * ((z + x) - (x + y)) ^ 2 = 0 := by
nlinarith [hzero, ht1, ht3]
have h3 : (z + x) * ((x + y) - (y + z)) ^ 2 = 0 := by
nlinarith [hzero, ht1, ht2]
have hs1 : ((y + z) - (z + x)) ^ 2 = 0 := by
rcases mul_eq_zero.mp h1 with h | h
· exfalso; linarith
· exact h
have hs2 : ((z + x) - (x + y)) ^ 2 = 0 := by
rcases mul_eq_zero.mp h2 with h | h
· exfalso; linarith
· exact h
have hs3 : ((x + y) - (y + z)) ^ 2 = 0 := by
rcases mul_eq_zero.mp h3 with h | h
· exfalso; linarith
· exact h
have hxy' : x = y := by nlinarith [hs3]
have hyz' : y = z := by nlinarith [hs1]
have hxval : x = 1 / 3 := by nlinarith [hsum, hxy', hyz']
exact ⟨hxval, by nlinarith [hxy'], by nlinarith [hyz']⟩
· rintro ⟨rfl, rfl, rfl⟩
norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxypyzleqsum2onxpy__mh__4daebd7c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 67c46be14930e086

4.409 409. reciprocal_inequality_eq_iff

The problem

For positive real numbers x , y , and z , equality holds in the reciprocal inequality exactly when $x = y = z$.

Lean statement

```
theorem reciprocal_inequality_eq_iff (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z
```

Parent. Show that for any three positive real numbers x , y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Reuses the parent reciprocal expression, positivity setup, common-denominator normalization, and numerator comparison, while replacing the inequality conclusion with a full equality-locus characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an exact equality characterization, requiring the converse direction to extract vanishing squared differences and prove $x = y = z$. The parent's nonnegative-polynomial argument only establishes the inequality and does not supply this equality-locus reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an exact equality characterization. Its reverse direction requires deriving vanishing square terms from a sum-of-squares decomposition, reasoning not supplied by the parent's one-sided inequality proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an exact equality characterization, requiring the new direction that equality forces each squared-difference term to vanish and hence $x=y=z$. The parent only proves the inequality and does not supply this equality-locus argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2569 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_inequality_eq_iff.extracted_1_1 (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z
```

Read back by a model that saw only the Lean. For real numbers x, y , and z satisfying $0 < x$, $0 < y$, and $0 < z$, the equality $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ holds if and only if $x = y$ and $y = z$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem reciprocal_inequality_eq_iff (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z := by
  rcases h₀ with ⟨hx, hy, hz⟩
  have hxy : 0 < x + y := by linarith
  have hyz : 0 < y + z := by linarith
  have hzx : 0 < z + x := by linarith
  have hxyz : 0 < x + y + z := by linarith
  have hden : 0 < (x + y + z) * (x + y) * (y + z) * (z + x) := by
    positivity
  have hleft :
    9 / (x + y + z) =
      (9 * (x + y) * (y + z) * (z + x)) /
        ((x + y + z) * (x + y) * (y + z) * (z + x)) := by
    field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] <|> ring
  have hright :
    2 / (x + y) + 2 / (y + z) + 2 / (z + x) =
      (2 * (x + y + z) *
        ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y))) /
        ((x + y + z) * (x + y) * (y + z) * (z + x)) := by
    field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] <|> ring
  constructor
  · intro heq
    rw [hleft, hright] at heq
    have hcross := (div_eq_div_iff (ne_of_gt hden) (ne_of_gt hden)).mp heq
    have hprod :
      (2 * (x + y + z) *
        ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -
        9 * (x + y) * (y + z) * (z + x)) *
        ((x + y + z) * (x + y) * (y + z) * (z + x)) = 0 := by
      nlinarith [hcross]
    have hzero :
      2 * (x + y + z) *
        ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -
        9 * (x + y) * (y + z) * (z + x) = 0 := by
      rcases mul_eq_zero.mp hprod with h | h
      · exact h
      · exact False.elim (ne_of_gt hden h)
    have hdecomp :
      2 * (x + y + z) *
        ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -
        9 * (x + y) * (y + z) * (z + x) =
        (x + y) * (x - y) ^ 2 + (y + z) * (y - z) ^ 2 +
        (z + x) * (z - x) ^ 2 := by
      ring
    have ht1 : 0 ≤ (x + y) * (x - y) ^ 2 :=
      mul_nonneg (le_of_lt hxy) (sq_nonneg _)
    have ht2 : 0 ≤ (y + z) * (y - z) ^ 2 :=
      mul_nonneg (le_of_lt hyz) (sq_nonneg _)
    have ht3 : 0 ≤ (z + x) * (z - x) ^ 2 :=
      mul_nonneg (le_of_lt hzx) (sq_nonneg _)
    have hs1 : (x - y) ^ 2 = 0 := by
      nlinarith [hzero, hdecomp]
    have hs2 : (y - z) ^ 2 = 0 := by
      nlinarith [hzero, hdecomp]
    have hxy' : x = y := by nlinarith [hs1]
```

```

have hyz' : y = z := by nlinarith [hs2]
exact ⟨hxy', hyz'⟩
· rintro ⟨hxy', hyz'⟩
  subst y
  subst z
  field_simp [ne_of_gt hx] <|> ring

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__mh__bcee330d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint bcee330ddcbc6305

4.410 410. reciprocal_amgm_eq_iff

The problem

Let x be a positive real number. Show that $2 - \sqrt{2} = 2 - x - \frac{1}{2x}$ if and only if $x = \frac{\sqrt{2}}{2}$.

Lean statement

```
theorem reciprocal_amgm_eq_iff :
  ∀ x : ℝ, x > 0 → (2 - Real.sqrt 2 = 2 - x - 1 / (2 * x) ↔ x = Real.sqrt 2 / 2)
```

Parent. Let x be a positive real number. Show that $2 - \sqrt{2} \geq 2 - x - \frac{1}{2x}$.

Parent in Lean

```
theorem algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x :
  ∀ x > 0, 2 - Real.sqrt 2 ≥ 2 - x - 1 / (2 * x) := by
```

What it contributes. Reuses the parent's positive-real reciprocal expression and square-root normalization, while replacing the inequality target with an equality-case characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an equality-case characterization, requiring a new factorization and zero-square argument to derive $x = \sqrt{2}/2$, plus a converse verification. The parent's proof establishes only the bound and does not supply this uniqueness reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child characterizes the equality case, proving that equality forces the square term $(2x - \sqrt{2})^2$ to vanish and hence $x = \sqrt{2}/2$, then verifies the converse. The parent only proves the reciprocal AM-GM inequality and supplies no equality analysis.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the equality case, requiring a square-factorization argument to show equality forces $2x = \sqrt{2}$ and a separate verification of the converse. The parent's inequality proof does not supply this uniqueness/equality reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1732 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_amgm_eq_iff.extracted_1_1 : ∀ x > 0, 2 - √2 = 2 - x - 1 / (2 * x) ↔ x = √2 / 2
```

Read back by a model that saw only the Lean. *For every real number $x > 0$, $2 - \sqrt{2} = 2 - x - 1/(2x)$ if and only if $x = \sqrt{2}/2$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_amgm_eq_iff :
  ∀ x : ℝ, x > 0 → (2 - Real.sqrt 2 = 2 - x - 1 / (2 * x) ↔ x = Real.sqrt 2 / 2) := by
  intro x hx
  have hs2 : (Real.sqrt 2) ^ 2 = (2 : ℝ) := by
    norm_num
  have hx0 : x ≠ 0 := ne_of_gt hx
  constructor
  · intro heq
    have hsum : x + 1 / (2 * x) = Real.sqrt 2 := by
      linarith
    have hfactor :
      (2 * x - Real.sqrt 2) ^ 2 =
        4 * x * (x + 1 / (2 * x) - Real.sqrt 2) := by
      field_simp [hx0]
      nlinarith [hs2]
    have hzero : (2 * x - Real.sqrt 2) ^ 2 = 0 := by
      rw [hfactor, hsum]
      ring
      nlinarith [hzero]
    · intro hxval
      rw [hxval]
      have hspos : 0 < Real.sqrt 2 := by positivity
      have hs0 : Real.sqrt 2 ≠ 0 := ne_of_gt hspos
      field_simp [hs0]
      nlinarith [hs2]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x__mh__7650d2bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2c19cbaa9f79a4a6

4.411 411. bounded_subtractive_prime_balance_unique_complement

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that every integer root of $z^2 - (p + q - 2)z + t + 1$ has a unique complementary integer root whose sum with it is $p + q - 2$ and whose product with it is $t + 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_unique_complement :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 →
      ∃! w : ℤ,
        w^2 - ((p : ℤ) + (q : ℤ) - 2) * w + (t : ℤ) + 1 = 0 ∧
        z + w = (p : ℤ) + (q : ℤ) - 2 ∧
        z * w = (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a genuine existence-and-uniqueness theorem for the complementary root. Its witness construction, symmetry argument, product derivation, and uniqueness from the sum are new obligations not supplied by the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from characterizing all roots to proving existence and uniqueness of a complementary root, requiring a symmetry argument and a separate uniqueness proof from the sum relation. The parent's root-factorization proof does not directly supply this construction or uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2463 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_unique_complement.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t →
  ∀ (z : ℤ),
    z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1 = 0 →
    ∃! w, w ^ 2 - (↑p + ↑q - 2) * w + ↑t + 1 = 0 ∧ z + w = ↑p + ↑q - 2 ∧ z * w = ↑t + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime numbers, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if the natural-number subtraction $p \cdot q - (p + q)$ equals t , then for every integer z satisfying $z^2 - (p + q - 2)z + t + 1 = 0$, there exists a unique integer w such that $w^2 - (p + q - 2)w + t + 1 = 0$, $z + w = p + q - 2$, and $z \cdot w = t + 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_unique_complement :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 →
      ∃! w : ℤ,
        w^2 - ((p : ℤ) + (q : ℤ) - 2) * w + (t : ℤ) + 1 = 0 ∧
        z + w = (p : ℤ) + (q : ℤ) - 2 ∧
        z * w = (t : ℤ) + 1 := by
  intro t p q hdom hbal z hz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q) : ℕ) : ℤ) = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hconstant : (t : ℤ) + 1 = ((p : ℤ) - 1) * ((q : ℤ) - 1) :=
    hfactor.symm
  refine ⟨(p : ℤ) + (q : ℤ) - 2 - z, ?_, ?_⟩
  · constructor
    · have hsymm :
        ((p : ℤ) + (q : ℤ) - 2 - z)^2 -
          ((p : ℤ) + (q : ℤ) - 2) * ((p : ℤ) + (q : ℤ) - 2 - z) +
          (t : ℤ) + 1 =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          ring
        rw [hsymm]
        exact hz
    constructor
    · ring
    · nlinarith [hz, hconstant]
  · intro y hy
    rcases hy with ⟨hyroot, hysum, hyprod⟩
    linarith [hysum]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__5bc81901

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5bc8190112540d25

4.412 412. bounded_subtractive_prime_balance_positive_right_of_larger_root

The problem

Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 > 0$ for every integer $z > q - 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

What it contributes. Reused the parent's bounded-subtraction cast and factorization checkpoint, then consumed it in a new quantified strict-sign argument using the added ordering $p < q$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a strict positivity claim and requires using the new ordering hypothesis to prove both factors are positive to the right of the larger root. The parent's zero-product argument does not supply this sign reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an ordered pair and proves strict positivity beyond the larger root, requiring a new sign argument for both factored terms. The parent only characterizes the roots and does not supply this order-and-inequality reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2449 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
  p * q - (p + q) = t → ∀ (z : ℤ), ↑q - 1 < z → 0 < z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, and $p < q$, and if $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for every integer z satisfying $q - 1 < z$, one has $0 < z^2 - (p + q - 2)z + t + 1$, where p , q , and t are viewed as integers in the inequality and expression.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
  intro t p q hdom hbal z hz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, hpq⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hpz : 0 < z - ((p : ℤ) - 1) := by
    have hp_int : (p : ℤ) < q := by
      exact_mod_cast hpq
    linarith
  have hqz : 0 < z - ((q : ℤ) - 1) := by
    linarith
  have hprod : 0 < (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by
    exact mul_pos hpz hqz
  calc
    0 < (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := hprod
    _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
      calc
        (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
        _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          rw [hfactor]
          ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__8f23d403

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8f23d4038c1f35ce

4.413 413. bounded_subtractive_balance_quadratic_nonnegative_outside

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that $z^2 - (p + q - 2)z + t + 1$ is nonnegative whenever z lies outside the interval with endpoints $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_balance_quadratic_nonnegative_outside :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (z ≤ (p : ℤ) - 1 ∧ z ≤ (q : ℤ) - 1) ∨
      ((p : ℤ) - 1 ≤ z ∧ (q : ℤ) - 1 ≤ z) →
      0 ≤ z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an exact root characterization to a nonnegativity result outside the interval between the two roots. Its proof requires new order and sign reasoning for the factored quadratic, which the parent's zero-product argument does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root-characterization equivalence into a sign claim outside both roots. Its proof requires new order and product-sign reasoning that the parent's root extraction argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from characterizing the two integer roots to proving nonnegativity on both exterior regions. Although it reuses the parent's factorization identity, it requires new sign reasoning for the two factors and is not recoverable by recalling the parent's iff proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2443 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_balance_quadratic_nonnegative_outside.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t →
  ∀ (z : ℤ), z ≤ tp - 1 ∧ z ≤ tq - 1 ∨ tp - 1 ≤ z ∧ tq - 1 ≤ z → 0 ≤ z^2 - (tp + tq - 2) * z + tt + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if the natural-number subtraction $p \cdot q - (p + q)$ equals t , then for every integer z satisfying either ($z \leq p - 1$ and $z \leq q - 1$) or ($p - 1 \leq z$ and $q - 1 \leq z$), one has $0 \leq z^2 - (p + q - 2)z + t + 1$, where the natural numbers p , q , and t are regarded as integers in the latter expression and inequalities.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_balance_quadratic_nonnegative_outside :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (z ≤ (p : ℤ) - 1 ∧ z ≤ (q : ℤ) - 1) ∨
      ((p : ℤ) - 1 ≤ z ∧ (q : ℤ) - 1 ≤ z) →
      0 ≤ z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
  intro t p q hdom hbal z hout
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hproduct :
    (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
    z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
    calc
      (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
        z^2 - ((p : ℤ) + (q : ℤ) - 2) * z +
          ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
      _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
        rw [hfactor]
        ring
  rcases hout with hout | hout
  · have hleft : z - ((p : ℤ) - 1) ≤ 0 := by linarith
    have hright : z - ((q : ℤ) - 1) ≤ 0 := by linarith
    have hnonneg : 0 ≤ (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) :=
      mul_nonneg_of_nonpos_of_nonpos hleft hright
    rw [hproduct] at hnonneg
    exact hnonneg
  · have hleft : 0 ≤ z - ((p : ℤ) - 1) := by linarith
    have hright : 0 ≤ z - ((q : ℤ) - 1) := by linarith
    have hnonneg : 0 ≤ (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) :=
```

```
mul_nonneg hleft hright  
rw [hproduct] at hnonneg  
exact hnonneg
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__9296a78a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 9296a78a4f4fca68

4.414 414. bounded_subtractive_prime_balance_integer_roots

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Parent. For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

What it contributes. Reuses the bounded distinct-prime domain and the cast-based treatment of truncated natural subtraction, while changing the conclusion to an exhaustive integer-root classification with an explicit factorization checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an existential arithmetic equivalence to an exact characterization of all integer roots, requiring polynomial factorization and the integer zero-product argument. The parent supplies only the coefficient identity, not the root analysis, so recalling its proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the equivalence of two arithmetic expressions into an integer-root characterization, requiring a new factorization argument and zero-product case split. Although it reuses the parent's algebraic identity, the parent proof does not supply the polynomial-root reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new root-characterization claim: after deriving the shifted-factor identity, it must factor the integer quadratic and use the integral-domain zero-product property to prove both directions. The parent only establishes an existential reformulation and does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2428 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_integer_roots.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t → ∀ (z : ℤ), z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1 = 0 ↔ z = ↑p - 1 ∨ z = ↑q - 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if $p \cdot q - (p + q) = t$, then for every integer z , $z^2 - (p + q - 2)z + t + 1 = 0$ if and only if $z = p - 1$ or $z = q - 1$, where the natural numbers p , q , and t are regarded as integers in the equation.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1 := by
  intro t p q hdom hbal z
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  constructor
  · intro hz
    have hprod : (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) = 0 := by
      calc
        (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
        _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          rw [hfactor]
          ring
        _ = 0 := hz
    rcases mul_eq_zero.mp hprod with hroot | hroot
    · left
      linarith
    · right
      linarith
  · intro hroot
    rcases hroot with hroot | hroot
    · subst z
      nlinarith [hfactor]
    · subst z
      nlinarith [hfactor]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh__1520daff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1520daff89b22cbd

4.415 415. bounded_prime_product_balance_value_classification

The problem

For every nonnegative integer t , characterize when two distinct primes between 4 and 18 satisfy $pq = t + (p + q)$. Show that the possible values of t are exactly 23, 39, 47, 59, 63, 71, 95, 119, 159, and 191.

Lean statement

```
theorem bounded_prime_product_balance_value_classification :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
      t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191
```

Parent. For every nonnegative integer t , characterize when t satisfies the additive balance $pq = t + (p + q)$ for two distinct primes between 4 and 18. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an algebraic equivalence into an exact finite classification of all possible values of t . Its proof requires enumerating the bounded primes, checking all pairings, and constructing witnesses, none of which is supplied by the parent's normalization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential balance equation into an exact classification of all possible values of t , requiring bounded prime enumeration and a ten-case witness construction. The parent's algebraic equivalence proof does not supply this finite classification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from an algebraic equivalence to a complete finite classification of all admissible values of t . Its proof requires enumerating the bounded primes, analyzing all distinct pairs, and proving the converse witnesses; the parent's factoring argument does not supply this classification.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2447 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_balance_value_classification.extracted_1_1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = t + (p + q)) ↔
  t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $p \cdot q = t + (p + q)$, if and only if t is one of 23, 39, 47, 59, 63, 71, 95, 119, 159, or 191.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_balance_value_classification :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
      t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191 := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
    have hp_cases : p = 5 ∨ p = 7 ∨ p = 11 ∨ p = 13 ∨ p = 17 := by
      interval_cases p <|> norm_num at hp <|> omega
    have hq_cases : q = 5 ∨ q = 7 ∨ q = 11 ∨ q = 13 ∨ q = 17 := by
      interval_cases q <|> norm_num at hq <|> omega
    rcases hp_cases with rfl | rfl | rfl | rfl | rfl <|>
    rcases hq_cases with rfl | rfl | rfl | rfl | rfl <|>
    omega
  · intro ht
    rcases ht with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num⟩
    · exact ⟨5, 11, by norm_num⟩
    · exact ⟨5, 13, by norm_num⟩
    · exact ⟨7, 11, by norm_num⟩
    · exact ⟨5, 17, by norm_num⟩
    · exact ⟨7, 13, by norm_num⟩
    · exact ⟨7, 17, by norm_num⟩
    · exact ⟨11, 13, by norm_num⟩
    · exact ⟨11, 17, by norm_num⟩
    · exact ⟨13, 17, by norm_num⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.ms.mh__9e1c3aa2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9e1c3aa207fabbf1

4.416 416. prime_pair_quadratic_sign_characterization

The problem

Let $p < q$ be natural numbers. For an integer x , show that $x^2 - (p + q)x + pq > 0$ exactly when $x < p$ or $x > q$.

Lean statement

```
theorem prime_pair_quadratic_sign_characterization (p q : ℕ) (x : ℤ)
  (hpq : p < q) :
  (0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)) ↔
  x < (p : ℤ) ∨ (q : ℤ) < x
```

Parent. Let $p < q$ be primes between 4 and 18 satisfying $pq - p - q = 119$. Show that $x^2 - (p + q)x + pq > 0$ for every integer $x > q$.

Parent in Lean

```
theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)
  (hpq : p < q)
  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-sided positivity result into a full sign characterization, requiring a new forward argument to exclude the interval between the roots and a reverse argument for the left-of-root case. The parent's right-of-root positivity proof alone cannot establish this biconditional.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-sided positivity result into a full sign characterization: it proves positivity both below p and above q , and proves nonpositivity throughout the interval. This requires a new converse and an interval sign argument beyond the parent's outside- q factor positivity proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-sided positivity result into a full sign characterization, proving both exterior cases and nonpositivity on the interval between the roots. This requires new case analysis and contradiction reasoning beyond the parent's factorization-and-positive-product argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2520 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_sign_characterization.extracted_1_1 (p q : ℕ) (x : ℤ) (hpq : p < q) :
  0 < x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q ↔ x < ↑p ∨ ↑q < x
```

Read back by a model that saw only the Lean. For natural numbers p and q and an integer x , assuming $p < q$, the inequality $0 < x^2 - (p + q)x + pq$ holds if and only if $x < p$ or $q < x$, where p and q are regarded as integers in the inequality.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_sign_characterization (p q : ℕ) (x : ℤ)
  (hpq : p < q) :
  (0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)) ↔
  x < (p : ℤ) ∨ (q : ℤ) < x := by
  have hpq' : (p : ℤ) ≤ (q : ℤ) := by
    omega
  have hfac :
    x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =
    (x - (p : ℤ)) * (x - (q : ℤ)) := by
    ring
  rw [hfac]
  constructor
  · intro hpos
    by_cases hxp : x < (p : ℤ)
    · exact Or.inl hxp
    · right
      by_contra hqx
      have hpx : (p : ℤ) ≤ x := by
        omega
      have hxq : x ≤ (q : ℤ) := by
        omega
      have hleft : 0 ≤ x - (p : ℤ) := by
        omega
      have hright : x - (q : ℤ) ≤ 0 := by
        omega
      have hnonpos : (x - (p : ℤ)) * (x - (q : ℤ)) ≤ 0 := by
        exact mul_nonpos_of_nonneg_of_nonpos hleft hright
      exact (not_lt_of_ge hnonpos) hpos
  · intro h
    rcases h with hxp | hqx
    · have hleft : x - (p : ℤ) < 0 := by
        omega
      have hright : x - (q : ℤ) < 0 := by
        omega
      exact mul_pos_of_neg_of_neg hleft hright
    · have hleft : 0 < x - (p : ℤ) := by
        omega
      have hright : 0 < x - (q : ℤ) := by
        omega
      exact mul_pos hleft hright
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh.mh__c31d4118

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c31d4118491553ff

4.417 417. shifted_prime_product_mod_four

The problem

For every natural number n , if distinct primes p and q between 4 and 18 satisfy $pq - (p + q) = n$, then n is congruent to 3 modulo 4.

Lean statement

```
theorem shifted_prime_product_mod_four (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 4 = 3
```

Parent. Let n be a natural number. If two distinct prime numbers between 4 and 18 satisfy $pq - (p + q) = n$, prove that n is odd.

Parent in Lean

```
theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1
```

What it contributes. Reused the bounded distinct-prime hypothesis, shifted factorization checkpoint, and elimination of the even-prime branches; strengthened the final residue conclusion from modulo 2 to modulo 4.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the parity conclusion to a modulo-four obstruction by proving both shifted factors $p-1$ and $q-1$ are even, hence $n+1$ is divisible by 4. This extra divisibility reasoning is not supplied by the parent's modulo-two proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parity conclusion to a modulo-four obstruction by proving both shifted prime factors $p-1$ and $q-1$ are even, hence $n+1$ is divisible by 4. This requires new factorization arithmetic beyond the parent's proof of mere evenness.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens parity to the modulo-four obstruction $n+1 = 4ab$, requiring both shifted odd-prime factors $p-1$ and $q-1$ to be explicitly shown even. The parent only establishes $n+1$ is even and does not supply this stronger factorization reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2053 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_prime_product_mod_four.extracted_1_1 (n : ℕ) :  
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) →  
    n % 4 = 3
```

Read back by a model that saw only the Lean. For a natural number n , if there exist distinct natural numbers p and q such that p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and the natural-number subtraction $p \cdot q - (p + q)$ equals n , then the remainder when n is divided by 4 is 3.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem shifted_prime_product_mod_four (n : ℕ) :  
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 4 = 3 := by  
  rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩  
  have hsum : p + q ≤ p * q := by  
    nlinarith  
  have hmul : p * q = n + (p + q) := by  
    omega  
  have hp_sub : p - 1 + 1 = p := by  
    omega  
  have hq_sub : q - 1 + 1 = q := by  
    omega  
  have hfac : (p - 1) * (q - 1) = n + 1 := by  
    nlinarith [hmul, hp_sub, hq_sub]  
  obtain ⟨a, ha⟩ | ⟨a, ha⟩ := Nat.even_or_odd p  
  · have hp2 : p = 2 := (Nat.Prime.even_iff hp).mp ⟨a, ha⟩  
    omega  
  · obtain ⟨b, hb⟩ | ⟨b, hb⟩ := Nat.even_or_odd q  
    · have hq2 : q = 2 := (Nat.Prime.even_iff hq).mp ⟨b, hb⟩  
      omega  
    · have hp_shift : p - 1 = 2 * a := by  
        omega  
      have hq_shift : q - 1 = 2 * b := by  
        omega  
      have hn1 : n + 1 = 4 * (a * b) := by  
        nlinarith [hfac, hp_shift, hq_shift]  
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__1522f78c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1529e38793ea1e9

4.418 418. prime_pair_quadratic_positive_outside

The problem

Let $p < q$ be primes between 4 and 18 satisfying $pq - p - q = 119$. Show that $x^2 - (p+q)x + pq > 0$ for every integer $x > q$.

Lean statement

```
theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)
  (hpq : p < q)
  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)
```

Parent. Let $p < q$ be distinct primes between 4 and 18 satisfying $pq - p - q = 119$. Show that the integer roots of $x^2 - (p+q)x + pq = 0$ are exactly p and q .

Parent in Lean

```
theorem prime_pair_quadratic_roots (p q : ℕ)
  (x : ℤ) :
  (x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) = 0) ↔
  x = (p : ℤ) ∨ x = (q : ℤ)
```

What it contributes. Uses the parent's quadratic factorization checkpoint and the ordered relation $p < q$, while changing the conclusion to a strict outside-root sign claim.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an equality characterization into a strict-positivity claim outside the larger root. Its proof requires ordered-root reasoning and positivity of both factors via 'mul_pos', which the parent's zero-product argument does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root-equation conclusion into strict positivity beyond the larger root. It uses the factorization from the parent but additionally reasons from the ordered hypotheses to show both factors are positive, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from characterizing roots to proving strict positivity beyond the larger root. Its proof requires sign reasoning for both factors, which the parent's zero-product argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2482 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_positive_outside.extracted_1_1 (p q : ℕ) (x : ℤ) (hp : Nat.Prime p) (hq : Nat.Prime q)
  (hp4 : 4 < p) (hq18 : q < 18) (hpq : p < q) (hrel : ↑p * ↑q - ↑p - ↑q = 119) (hx : ↑q < x) :
  0 < x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q
```

Read back by a model that saw only the Lean. For natural numbers p and q and an integer x , if p and q are prime, $4 < p$, $q < 18$, $p < q$, $p \cdot q - p - q = 119$, and $q < x$, then $0 < x^2 - (p + q)x + p \cdot q$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)

  (hpq : p < q)

  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) := by
have hfac :
  x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =
  (x - (p : ℤ)) * (x - (q : ℤ)) := by
  ring
have hxp : 0 < x - (p : ℤ) := by
  omega
have hxq : 0 < x - (q : ℤ) := by
  omega
have hprod : 0 < (x - (p : ℤ)) * (x - (q : ℤ)) := by
  exact mul_pos hxp hxq
rw [hfac]
exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__af2d4b34

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9a55ebe023da8640

4.419 419. shifted_quadratic_equal_values

The problem

Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that two integers give the same value of $x^2 - (p + q)x + (t + p + q)$ exactly when they are equal or their sum is $p + q$.

Lean statement

```
theorem shifted_quadratic_equal_values :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    ∀ x y : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
        y ^ 2 - ((p : ℤ) + q) * y + ((t : ℤ) + p + q) ↔
      x = y ∨ x + y = (p : ℤ) + q := by
```

Parent. Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Parent in Lean

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from characterizing the polynomial's roots to characterizing all pairs of integers with equal polynomial values. Although factorization from the parent remains useful, the new difference-of-squares argument and two-variable case split are not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from identifying the quadratic's roots to characterizing all pairs of integers with equal quadratic values. Its proof requires a new difference-of-squares argument, deriving $(x-y)(x+y-(p+q))=0$, which is not supplied by the parent's root proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a distinct two-variable equal-value characterization. Its proof requires deriving and solving the factored difference equation, which is not supplied by the parent's root-solving argument.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2538 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_quadratic_equal_values.extracted_1_1 : ∀ (p q t : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p →
  p ≤ 18 →
  4 ≤ q →
  q ≤ 18 →
  p ≠ q →
  t ≤ 200 →
  p * q - (p + q) = t →
  ∀ (x y : ℤ),
    x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = y ^ 2 - (↑p + ↑q) * y + (↑t + ↑p + ↑q) ↔
    x = y ∨ x + y = ↑p + ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p , q , and t , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $t \leq 200$, and $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for all integers x and y , $x^2 - (p + q)x + (t + p + q) = y^2 - (p + q)y + (t + p + q)$ if and only if $x = y$ or $x + y = p + q$, where the natural numbers p , q , and t are regarded as integers in the quadratic expressions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_quadratic_equal_values :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    ∀ x y : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
        y ^ 2 - ((p : ℤ) + q) * y + ((t : ℤ) + p + q) ↔
        x = y ∨ x + y = (p : ℤ) + q := by
  intro p q t hp hq hp4 hp18 hq4 hq18 hne htbound hrel
  have hsum_le : p + q ≤ p * q := by
    interval_cases p <|> interval_cases q <|> norm_num
  have hprod : p * q = t + (p + q) :=
    (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q : ℕ) : ℤ) := by
      exact_mod_cast hprod
    simpa [Nat.cast_add, add_assoc] using hprodZ
  have hfactor : ∀ z : ℤ,
    z ^ 2 - ((p : ℤ) + q) * z + ((t : ℤ) + p + q) =
      (z - p) * (z - q) := by
    intro z
    nlinarith [hrelZ]
  intro x y
  constructor
  · intro hxy
    have hzero : (x - y) * (x + y - ((p : ℤ) + q)) = 0 := by
      nlinarith [hxy, hfactor x, hfactor y]
```

```

rcases mul_eq_zero.mp hzero with hzero | hzero
· left
  omega
· right
  omega
· intro hxy
  rcases hxy with rfl | hxy
· rfl
· have hy : y = (p : ℤ) + q - x := by
  omega
  rw [hy]
  ring

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__f398235d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f398235dde5761c8

4.420 420. prime_pair_quadratic_nonnegative_iff

The problem

Let $p \leq q$ be natural numbers and let x be an integer. Show that $x^2 - (p + q)x + pq \geq 0$ exactly when $x \leq p$ or $q \leq x$.

Lean statement

```
theorem prime_pair_quadratic_nonnegative_iff (p q : ℕ) (x : ℤ) (hpq : (p : ℤ) ≤ (q : ℤ)) :
  0 ≤ x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ↔
  x ≤ (p : ℤ) ∨ (q : ℤ) ≤ x
```

Parent. For natural numbers p, q and an integer x , show that x is neither p nor q exactly when $x^2 - (p + q)x + pq$ is nonzero.

Parent in Lean

```
theorem prime_pair_quadratic_nonzero_iff (p q : ℕ) (x : ℤ) :
  (¬ (x = (p : ℤ) ∨ x = (q : ℤ))) ↔
  x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ≠ 0
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a zero-characterization into a sign characterization over the ordered roots. Its proof requires new interval and product-sign reasoning; the parent's root/nonzero argument does not supply this.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from nonvanishing to a full sign characterization, requiring order reasoning and a case split between the roots. The parent's factorization is useful but its root-equivalence argument does not supply the new sign proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the zero-root characterization into a sign characterization: proving nonnegativity requires ordering the roots and reasoning about the sign of both factors, including exclusion of the interval between them. The parent's factorization alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2532 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_nonnegative_iff.extracted_1_1 (p q : ℕ) (x : ℤ) (hpq : ↑p ≤ ↑q) :  
  0 ≤ x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q ↔ x ≤ ↑p ∨ ↑q ≤ x
```

Read back by a model that saw only the Lean. For natural numbers p and q , an integer x , and the hypothesis that $p \leq q$, the inequality $0 \leq x^2 - (p + q)x + pq$ holds if and only if $x \leq p$ or $q \leq x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_nonnegative_iff (p q : ℕ) (x : ℤ) (hpq : (p : ℤ) ≤ (q : ℤ)) :  
  0 ≤ x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ↔  
  x ≤ (p : ℤ) ∨ (q : ℤ) ≤ x := by  
  have hfac :  
    x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =  
      (x - (p : ℤ)) * (x - (q : ℤ)) := by  
    ring  
  constructor  
  · intro hnon  
    by_cases hleft : x ≤ (p : ℤ)  
    · exact Or.inl hleft  
    · right  
      by_contra hright  
      have hxp : (p : ℤ) < x := by omega  
      have hxq : x < (q : ℤ) := by omega  
      have hprodneg : (x - (p : ℤ)) * (x - (q : ℤ)) < 0 := by  
        exact mul_neg_of_pos_of_neg (sub_pos.mpr hxp) (sub_neg.mpr hxq)  
      have hpolyneg : x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) < 0 := by  
        rw [hfac]  
        exact hprodneg  
      linarith  
  · intro hout  
    rcases hout with hleft | hright  
    · have h1 : x - (p : ℤ) ≤ 0 := by linarith  
      have h2 : x - (q : ℤ) ≤ 0 := by linarith  
      rw [hfac]  
      exact mul_nonneg_of_nonpos_of_nonpos h1 h2  
    · have h1 : 0 ≤ x - (p : ℤ) := by linarith  
      have h2 : 0 ≤ x - (q : ℤ) := by linarith  
      rw [hfac]  
      exact mul_nonneg h1 h2
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.ms.mh__94f43e84

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 94f43e8478d5e09c

4.421 421. prime_pair_119_characterization

The problem

For any natural numbers p and q that are distinct primes between 4 and 18, $pq - (p + q) = 119$ if and only if $(p, q) = (11, 13)$ or $(p, q) = (13, 11)$.

Lean statement

```
theorem prime_pair_119_characterization :
  ∀ p q : ℕ,
    Nat.Prime p → Nat.Prime q → 4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    (p * q - (p + q) = 119 ↔ (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11))
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the nested existence claim into a full iff characterization, requiring both verification of the two ordered prime pairs and exclusion of every other bounded pair. The parent proof only derives a contradiction from the 22 case by enumerating p and does not establish any characterization of the 119 case.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a 119-witness into a full iff characterization, requiring exhaustive reasoning about both p and q and identifying the only ordered pairs. The parent only rules out several target values and supplies no uniqueness or classification argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential fact into a biconditional characterization for every bounded prime pair, requiring exhaustive reasoning over both p and q and proving both directions. The parent's contradiction proof only handles the surrounding conjunction and cannot supply this universal uniqueness characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1726 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_119_characterization.extracted_1.1 : ∀ (p q : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q → (p * q - (p + q) = 119 ↔ p = 11 ∧ q = 13 ∨ p = 13 ∧ q = 11)
```

Read back by a model that saw only the Lean. *For all natural numbers p and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, then $p \cdot q - (p + q) = 119$ if and only if $(p = 11 \text{ and } q = 13) \text{ or } (p = 13 \text{ and } q = 11)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_pair_119_characterization :
  ∀ p q : ℕ,
    Nat.Prime p → Nat.Prime q → 4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    (p * q - (p + q) = 119 ↔ (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11)) := by
  intro p q hp hq hp4 hp18 hq4 hq18 hne
  interval_cases p <|> interval_cases q <|> norm_num at *
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__02d6b54d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 51e450964c9682a5

4.422 422. amc12_2000_p6_mutation_range

The problem

For two distinct primes between 4 and 18, the possible values of $pq - (p + q)$ are exactly 23, 39, 47, 59, 63, 71, 95, 119, 159, and 191.

Lean statement

```
theorem amc12_2000_p6_mutation_range :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = n) ↔
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? **(A)** 22 **(B)** 60 **(C)** 119 **(D)** 180 **(E)** 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes five isolated existence/nonexistence checks into a complete characterization of every attainable value. Its forward direction must exhaustively classify all distinct prime pairs and its reverse direction supplies ten witnesses, so the parent proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces five selected existence/nonexistence checks with a complete characterization of every attainable value, requiring exhaustive forward enumeration and explicit witnesses for the reverse direction. Its proof does not reuse the parent's contradiction argument and establishes a genuinely stronger range statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from refuting a specific conjunction of equations to characterizing the complete range of $p*q-(p+q)$ for distinct primes in $[4,18]$. Its proof must perform exhaustive enumeration and provide witnesses for ten values, which the parent's single contradiction proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2524 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_mutation_range.extracted_1_1 : ∀ (n : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) ↔
  n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191
```

Read back by a model that saw only the Lean. *For every natural number n , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q minus the sum $p + q$ equals n , if and only if n is one of 23, 39, 47, 59, 63, 71, 95, 119, 159, or 191.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_mutation_range :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = n) ↔
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191 := by
  intro n
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
    have hforward :
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨
      n = 119 ∨ n = 159 ∨ n = 191 := by
      interval_cases p <|> interval_cases q <|>
      simp_all (config := {decide := true})
    exact hforward
  · intro hn
    rcases hn with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨13, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__1995d807

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1995d8075402f51d

4.423 423. amc12_2000_p6_mutation

The problem

Two different primes between 4 and 18 are chosen. If their product minus their sum is 119, then the primes are 11 and 13, in either order.

Lean statement

```
theorem amc12_2000_p6_mutation :
  ∀ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119 →
    (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11)
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of one witness into a complete characterization of every admissible pair, requiring exclusion of all other bounded prime cases; the parent's witness proof does not supply that uniqueness argument. Its finite case analysis is a genuinely new proof obligation, not a recall of the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a witness for 119 into a complete uniqueness characterization of both ordered prime pairs. Its proof must enumerate and eliminate all bounded pairs, whereas the parent can close using the unrelated 22 contradiction and does not establish the pair classification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential claim into a uniqueness characterization of all bounded prime pairs satisfying the equation. Proving both possible orderings requires exhaustive reasoning about both variables, which the parent's contradiction proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2512 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_mutation.extracted_1_1 : ∀ (p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = 119 →
  p = 11 ∧ q = 13 ∨ p = 13 ∧ q = 11
```

Read back by a model that saw only the Lean. *For all natural numbers p and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, and $p \cdot q - (p + q) = 119$, then either $p = 11$ and $q = 13$, or $p = 13$ and $q = 11$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_mutation :
  ∀ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119 →
    (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11) := by
  intro p q h
  rcases h with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
  have hp_cases : 4 ≤ p ∧ p ≤ 18 := ⟨hp4, hp18⟩
  have hq_cases : 4 ≤ q ∧ q ≤ 18 := ⟨hq4, hq18⟩
  interval_cases p <|> interval_cases q <|> norm_num at *
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__502f3339

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 502f3339c29bdd78

4.424 424. shifted_prime_product_mod_two

The problem

Let n be a natural number. If two distinct prime numbers between 4 and 18 satisfy $pq - (p + q) = n$, prove that n is odd.

Lean statement

```
theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Uses the parent's bounded distinct-prime shifted product equation and its prime-modulus obstruction surface, but changes the endpoint to a parameterized parity congruence derived through the shifted factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's fixed numerical contradiction into a parity theorem for every representable n . Its proof requires factoring the shifted product and using the oddness of primes greater than 2, reasoning absent from the parent's bounded interval_cases and norm_num proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new universal parity obstruction for arbitrary n by factoring $p*q - (p+q) + 1$ and using that distinct primes at least 4 are odd. The parent's finite interval enumeration of five fixed target values supplies none of this modular reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new universal parity obstruction for every represented n , using oddness of distinct bounded primes and a shifted factorization; the parent only performs a finite contradiction for the fixed value 22. Its argument is not recoverable by recalling the parent's witness-based interval case split.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2048 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_prime_product_mod_two.extracted_1_1 (n : ℕ) :
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) →
    n % 2 = 1
```

Read back by a model that saw only the Lean. *For every natural number n , if there exist distinct prime natural numbers p and q such that $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \cdot q - (p + q) = n$, then n leaves remainder 1 upon division by 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1 := by
  rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
  have hsum : p + q ≤ p * q := by
    nlinarith
  have hmul : p * q = n + (p + q) := by
    omega
  have hp_sub : p - 1 + 1 = p := by
    omega
  have hq_sub : q - 1 + 1 = q := by
    omega
  have hfac : (p - 1) * (q - 1) = n + 1 := by
    nlinarith [hmul, hp_sub, hq_sub]
  obtain ⟨a, ha⟩ | ⟨a, ha⟩ := Nat.even_or_odd p
  · have hp2 : p = 2 := (Nat.Prime.even_iff hp).mp ⟨a, ha⟩
    omega
  · obtain ⟨b, hb⟩ | ⟨b, hb⟩ := Nat.even_or_odd q
    · have hq2 : q = 2 := (Nat.Prime.even_iff hq).mp ⟨b, hb⟩
      omega
    · have hn1 : ∃ k : ℕ, n + 1 = 2 * k := by
        refine ⟨2 * a * b, ?_⟩
        nlinarith [hfac, hp_sub, hq_sub, ha, hb]
      rcases hn1 with ⟨k, hk⟩
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__92f062d2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint bd34dfb531ac1cab

4.425 425. parameterized_shifted_roots

The problem

Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Lean statement

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Retains the parent's bounded distinct-prime domain and product-minus-sum relation, while replacing the fixed attainable value with the parameter t and proving the resulting factorization and root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the attainable value with t and proves a root characterization for the resulting shifted quadratic. Its factorization and integer-root argument replace the parent's finite case enumeration, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-value existence test with a parameterized algebraic identity and a root characterization for the shifted quadratic. Its proof requires factorization and zero-product reasoning, which are not supplied by the parent's bounded interval enumeration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the attainable value and proves a new algebraic factorization and root characterization, requiring polynomial reasoning absent from the parent's finite contradiction proof. The parent's proof cannot be recalled to establish either conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2514 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_shifted_roots.extracted_1_1 : ∀ (p q t : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p →
  p ≤ 18 →
  4 ≤ q →
  q ≤ 18 →
  p ≠ q →
  t ≤ 200 →
  p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧
  ∀ (x : ℤ), x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = 0 ↔ x = ↑p ∨ x = ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p, q , and t , if p is prime, q is prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $t \leq 200$, and $pq - (p + q) = t$, then $(p - 1)(q - 1) = t + 1$ and, for every integer x , $x^2 - (p + q)x + (t + p + q) = 0$ if and only if $x = p$ or $x = q$, where the coefficients p, q , and t in the integer equation are regarded as integers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
  Nat.Prime p → Nat.Prime q →
  4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
  t ≤ 200 → p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧
  ∀ x : ℤ,
  x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
  x = p ∨ x = q := by
  intro p q t hp hq hp4 hp18 hq4 hq18 hne htbound hrel
  have hsum_le : p + q ≤ p * q := by
    interval_cases p <|> interval_cases q <|> norm_num
  have hprod : p * q = t + (p + q) :=
    (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hfac : (p - 1) * (q - 1) = t + 1 := by
    interval_cases p <|> interval_cases q <|> norm_num at hrel <|> omega
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q : ℕ) : ℤ) := by
      exact_mod_cast hprod
    simp [Nat.cast_add, add_assoc] using hprodZ
  have hpoly : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
    (x - p) * (x - q) := by
    intro x
    nlinarith [hrelZ, hfac]
  have hroots : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
    x = p ∨ x = q := by
```

```

intro x
constructor
· intro hx
  have hzero : (x - p) * (x - q) = 0 := by
    rw [← hpoly x]
    exact hx
  rcases mul_eq_zero.mp hzero with hzero | hzero
  · left
    omega
  · right
    omega
· intro hx
  rcases hx with rfl | rfl
  · simp [hpoly]
  · simp [hpoly]
exact ⟨hfacs, hroots⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__f43b4d36

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint f43b4d361c08e0f6

4.426 426. characterize_bounded_threshold

The problem

For the sequence beginning with 4, 7, where each later term is the units digit of the sum of the preceding two, characterize the thresholds t for which 2 is the least index with $S_n > t$ and $n \leq 3$. Also show that the first three terms have sum 12.

Lean statement

```
theorem characterize_bounded_threshold
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (t : ℕ) :
  (IsLeast {n : ℕ | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12
```

Parent. Let a sequence begin with 4, 7, and let each later term be the units digit of the sum of the two preceding terms. If S_n is the sum of the first n terms, show that 2 is the least n such that $S_n > 10$ and $n \leq 3$.

Parent in Lean

```
theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sₙ 2
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the threshold and proves a symbolic least-index characterization, requiring new lower-bound and upper-bound reasoning for arbitrary t ; it also derives the additional value $S(3)=12$. The parent's fixed-threshold IsLeast proof and hypothesis h_4 cannot be recalled to solve this child.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child parameterizes the threshold and proves a two-way characterization of exactly when 2 is the least qualifying index, requiring exclusion of competing indices and handling both threshold directions. It also consumes the recurrence to establish the additional value $S(3)=12$, so recalling the parent's fixed-threshold leastness proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the threshold and proves a full characterization of when 2 is the least qualifying index, while also deriving the new value $S 3 = 12$. This requires reasoning beyond the parent's fixed-threshold membership proof.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2753 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem characterize_bounded_threshold.extracted_1_1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ k ∈ Finset.range n, u k) (t : ℕ) :
(IsLeast {n | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers such that $u(0)=4$, $u(1)=7$, for every natural number $n \geq 2$ we have $u(n)=(u(n-2)+u(n-1)) \bmod 10$, and for every natural number n , $S(n)=\sum_{k \in \{0,1,\dots,n-1\}} u(k)$. Then, for every natural number t , 2 is the least natural number n satisfying $S(n) > t$ and $n \leq 3$ if and only if $4 \leq t$ and $t < 11$; moreover, $S(3)=12$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem characterize_bounded_threshold
(u S : ℕ → ℕ)
(h₀ : u 0 = 4)
(h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
(h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
(t : ℕ) :
(IsLeast {n : ℕ | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12 := by
have hu2 : u 2 = 1 := by
  have h := h₂ 2 (by omega)
  norm_num [h₀, h₁] at h ⊢
  exact h
have hS0 : S 0 = 0 := by
  rw [h₃]
  simp
have hS1 : S 1 = 4 := by
  rw [h₃]
  simp [h₀]
have hS2 : S 2 = 11 := by
  rw [h₃]
  norm_num [Finset.sum_range_succ, h₀, h₁]
have hS3 : S 3 = 12 := by
  rw [h₃]
  norm_num [Finset.sum_range_succ, h₀, h₁, hu2]
constructor
· constructor
  · intro h
    have hmem := h.1
    change S 2 > t ∧ 2 ≤ 3 at hmem
    have ht_upper : t < 11 := by
      rw [hS2] at hmem
      omega
    have ht_lower : 4 ≤ t := by
      by_contra hlt
      have ht : t < 4 := by omega
      have h1mem : 1 ∈ {n : ℕ | S n > t ∧ n ≤ 3} := by
        change S 1 > t ∧ 1 ≤ 3
        rw [hS1]
```

```

      omega
      have h1e := h.2 h1mem
      omega
    exact ⟨ht_lower, ht_upper⟩
  · rintro ⟨ht_lower, ht_upper⟩
  have hmem :  $2 \in \{n : \mathbb{N} \mid S\ n > t \wedge n \leq 3\}$  := by
    change  $S\ 2 > t \wedge 2 \leq 3$ 
    rw [hS2]
    omega
  refine ⟨hmem, ?_⟩
  intro n hn
  change  $S\ n > t \wedge n \leq 3$  at hn
  rcases hn with ⟨hgt, hn3⟩
  have hn0 :  $n \neq 0$  := by
    intro hn0
    subst n
    rw [hS0] at hgt
    omega
  have hn1 :  $n \neq 1$  := by
    intro hn1
    subst n
    rw [hS1] at hgt
    omega
  omega
  · exact hS3

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2002_p21__me.mh__b19ac235

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint b19ac235ddcfcfed

4.427 427. sequence_periodic_twelve

The problem

Show that the sequence is periodic with period 12: for every nonnegative integer n , the $(n+12)$ -th term equals the n -th term.

Lean statement

```
theorem sequence_periodic_twelve
  (u : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) :
  ∀ n, u (n + 12) = u n := by
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10000}) :
  IsLeast Sₙ 1999 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from locating a threshold of partial sums to proving a universal period-12 law for the recurrence. Its proof requires strong induction and recurrence alignment, which the parent's finite threshold computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a threshold minimum for cumulative sums to a global period-12 theorem for the recurrence. Its strong induction and recurrence-shift argument require mathematics absent from the parent's finite-sum threshold proof, so recalling the parent does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the threshold-set minimum into a global periodicity theorem. Its strong induction and recurrence-shifting argument require genuinely new reasoning and do not reuse the parent's cumulative-sum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B universally quantifies over sequences and adds initial-value and recurrence hypotheses, whereas Statement A refers only to an unspecified particular sequence and states periodicity.

Goal round-trip: what Lean elaborated

```
theorem sequence_periodic_twelve.extracted_1_1 :  $\forall (u : \mathbb{N} \rightarrow \mathbb{N}),$   

 $u\ 0 = 4 \rightarrow u\ 1 = 7 \rightarrow (\forall n \geq 2, u\ n = (u\ (n - 2) + u\ (n - 1)) \% 10) \rightarrow \forall (n : \mathbb{N}), u\ (n + 12) = u\ n$ 
```

Read back by a model that saw only the Lean. *For every sequence u of natural numbers, if $u(0)=4$, $u(1)=7$, and for every natural number $n \geq 2$, $u(n)=(u(n-2)+u(n-1)) \bmod 10$, then for every natural number n , $u(n+12)=u(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem sequence_periodic_twelve
  (u :  $\mathbb{N} \rightarrow \mathbb{N}$ )
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ :  $\forall n \geq 2, u\ n = (u\ (n - 2) + u\ (n - 1)) \% 10$ ) :
 $\forall n, u\ (n + 12) = u\ n$  := by
  intro n
  refine Nat.strong_induction_on n ?_
  intro n ih
  cases n with
  | zero =>
    norm_num [h₂, h₀, h₁]
  | succ n =>
    cases n with
    | zero =>
      norm_num [h₂, h₀, h₁]
    | succ n =>
      have hp :  $u\ (n + 12) = u\ n$  := ih n (by omega)
      have hq :  $u\ (n + 1 + 12) = u\ (n + 1)$  := ih (n + 1) (by omega)
      have hleft := h₂ (n + 14) (by omega)
      have hright := h₂ (n + 2) (by omega)
      calc
         $u\ (n + 2 + 12) = (u\ (n + 12) + u\ (n + 13)) \% 10$  := by
          convert hleft using 1 <|> omega
        _ =  $(u\ n + u\ (n + 1)) \% 10$  := by
          rw [hp]
          exact congrArg (fun x => (u n + x) \% 10) (by
            simpa [Nat.add_assoc, Nat.add_comm, Nat.add_left_comm] using hq)
        _ =  $u\ (n + 2)$  := by
          exact hright.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh__ab69afd6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ab69afd68794e092

4.428 428. least_threshold_from_period_block

The problem

Let a sequence satisfy the same units-digit recurrence, and suppose it has period p with one-period sum b . If every partial block sum before $kp + r$ is at most T , while the sum at $kp + r$ exceeds T , prove that $kp + r$ is the least index whose partial sum exceeds T .

Lean statement

```
theorem least_threshold_from_period_block
  (u S : ℕ → ℕ)

  (p b T k r : ℕ)
  (hp : 0 < p)
  (hperiod : ∀ n, u (n + p) = u n)
  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
  IsLeast {n | S n > T} (k * p + r) := by
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sn : Set ℕ)
  (h₄ : Sn = {n | S n > 10000}) :
  IsLeast Sn 1999 := by
```

What it contributes. Preserves the units-digit recurrence, initial values, partial-sum definition, and least-threshold conclusion shape while replacing the fixed threshold computation with a symbolic period-and-block-sum criterion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the threshold and crossing index and replaces the parent's concrete recurrence computation with a general period/block decomposition using quotient-remainder reasoning. Its proof must establish that every earlier index has the required block representation and bound, so recalling the parent's finite sum calculation does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete recurrence to arbitrary periodic sequences and symbolic block data, then uses quotient-remainder decomposition to prove minimality. This requires reasoning absent from the parent's finite numerical computation; the unused period hypothesis is redundant but does not collapse the new theorem to recall.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's finite recurrence computation with a symbolic quotient-and-remainder argument using an arbitrary period, block sum, and threshold-crossing bound. Its proof requires new decomposition reasoning and does not reuse the parent's explicit calculation or closing lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2723 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem least_threshold_from_period_block.extracted_1.1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ i ∈ Finset.range n, u i) (p b T k r : ℕ)
(hp : 0 < p) (hperiod : ∀ (n : ℕ), u (n + p) = u n)
(hblock : ∀ (j s : ℕ), S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i) (hr : r < p)
(hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
(hbound : ∀ (j s : ℕ), j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
IsLeast {n | S n > T} (k * p + r)
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers such that $u(0)=4$, $u(1)=7$, and for every natural number $n \geq 2$, $u(n)=(u(n-2)+u(n-1)) \bmod 10$. Suppose that for every natural number n , $S(n)=\sum_{i \in \text{Finset.range } n} u(i)$. Let p, b, T, k, r be natural numbers with $p > 0$, such that $u(n+p)=u(n)$ for every natural number n . Suppose also that for all natural numbers j and s , $S(jp+s)=jb+\sum_{i \in \text{Finset.range } s} u(i)$, that $r < p$, and that $kb+\sum_{i \in \text{Finset.range } r} u(i) > T$. Finally, assume that for all natural numbers j and s , if $jp+s < kp+r$, then $jb+\sum_{i \in \text{Finset.range } s} u(i) \leq T$. Then $kp+r$ is the least natural number n such that $S(n) > T$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem least_threshold_from_period_block
(u S : ℕ → ℕ)

(p b T k r : ℕ)
(hp : 0 < p)
(hperiod : ∀ n, u (n + p) = u n)
(hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

(hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
(hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
IsLeast {n | S n > T} (k * p + r) := by
have hperiod_checkpoint : ∀ n, u (n + p) = u n := hperiod
constructor
· change S (k * p + r) > T
  rw [hblock]
  exact hcross
· intro n hn
```

```

change S n > T at hn
by_contra hnot
have hnlt : n < k * p + r := Nat.lt_of_not_ge hnot
let j := n / p
let s := n % p
have hslt : s < p := Nat.mod_lt n hp
have hdecomp : j * p + s = n := by
  dsimp [j, s]
  simpa [Nat.mul_comm] using Nat.div_add_mod n p
have harg : j * p + s < k * p + r := by
  simpa [hdecomp] using hnlt
have hsum : j * b +  $\sum i \in \text{Finset.range } s, u i \leq T$  := hbound j s harg
have hSn : S n = j * b +  $\sum i \in \text{Finset.range } s, u i$  := by
  rw [← hdecomp]
  exact hblock j s
have hle : S n ≤ T := by
  rw [hSn]
  exact hsum
omega

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh__c0433309

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 808f7f3adc74ba2b

4.429 429. sequence_characterization

The problem

Let $p, q \in \mathbb{R}$ and let a be a real sequence indexed by the natural numbers. Suppose that $a_{n+2} - a_{n+1} = a_{n+1} - a_n$ for every natural number n , and that $a_1 = p$, $a_2 = 9$, $a_3 = 3p - q$, and $a_4 = 3p + q$. Prove that $a_n = 4n + 1$ for every natural number n .

Lean statement

```
theorem sequence_characterization (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9)
  ↔ (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : ∀ n : ℕ, a n = 4 * (n : ℝ) + 1
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the 2010th term of this sequence? Show that it is 8041.

Parent in Lean

```
theorem amc12a_2010_p10 (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : a 2010 = 8041 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child strengthens a single-value conclusion to a full characterization of the sequence, requiring the new boundary case $a_0 = 1$ and an induction over every natural index. The parent's formula covers positive indices, but it does not supply the argument at index 0.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single distant-value claim into a complete characterization for every natural index, including the newly required base case $a_0 = 1$. Its proof must derive the initial value and establish the recurrence globally, which the parent's proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single-index evaluation into a full characterization for every natural index, including the newly required base case $a_0 = 1$. Its proof must use the recurrence at index 0 and establish the step formula by induction, obligations absent from the parent's fixed-index argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2038 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sequence_characterization.extracted_1_1 : ∀ (p q : ℝ) (a : ℕ → ℝ),  
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →  
    a 1 = p → a 2 = 9 → a 3 = 3 * p - q → a 4 = 3 * p + q → ∀ (n : ℕ), a n = 4 * ↑n + 1
```

Read back by a model that saw only the Lean. For all real numbers p and q and every sequence a of real numbers indexed by the natural numbers, if for every natural number n , $a(n + 2) - a(n + 1) = a(n + 1) - a(n)$, and $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$, then for every natural number n , $a(n) = 4n + 1$, where n is viewed as a real number.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem sequence_characterization (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9)  
  ↪ (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : ∀ n : ℕ, a n = 4 * (n : ℝ) + 1 := by  
  have hA := h₀ 1  
  have hB := h₀ 2  
  norm_num at hA hB  
  rw [h₃, h₂, h₁] at hA  
  rw [h₄, h₃, h₂] at hB  
  have hp : p = 5 := by  
    linarith  
  have hzero := h₀ 0  
  norm_num at hzero  
  rw [h₂, h₁, hp] at hzero  
  have ha0 : a 0 = 1 := by  
    linarith  
  have hstep : ∀ n : ℕ, a (n + 1) - a n = 4 := by  
    intro n  
    induction n with  
    | zero =>  
      norm_num [h₁, hp, ha0]  
    | succ n ih =>  
      have hrec : a (n + 2) - a (n + 1) = a (n + 1) - a n := h₀ n  
      linarith  
  intro n  
  induction n with  
  | zero =>  
    norm_num [ha0]  
  | succ n ih =>  
    have hs := hstep n  
    have ha : a (n + 1) = a n + 4 := by  
      linarith  
    rw [ha, ih]  
    norm_num [Nat.cast_succ]  
  ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2010_p10__mh__43a400bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0927852decccfbe

4.430 430. bounded_power_sum_product

The problem

For every natural number n , show that there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $\prod_{k=0}^{n-1} (2^{e_k} + 3^{e_k}) = 3^{e_n} - 2^{e_n}$.

Lean statement

```
theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128}-2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k : ℕ) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Supplied the power-sum finite-product domain and the terminal difference-of-powers conclusion shape; its fixed seven-factor identity is generalized to arbitrary n .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed seven-factor identity to every finite index and explicitly proves the exponent recurrence. Its induction derives the telescoping step over $\mathbb{Z}$, whereas the parent is only a concrete ‘norm_num’ computation, so memorizing the parent does not supply the child’s proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor numerical identity with a universally bounded-index product law and explicitly requires constructing an exponent sequence satisfying a recurrence. Its inductive telescoping proof is genuinely new and is not recoverable by recalling the parent’s finite norm_num computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor numerical identity with a universal finite-product law and explicitly proves the exponent recurrence. Its induction and telescoping argument require mathematical reasoning absent from the parent’s one-shot normalization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1797 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_sum_product.extracted_1_1 : ∀ (n : ℕ),
  ∃ e,
    e 0 = 1 ∧ (∀ (k : ℕ), e (k + 1) = 2 * e k) ∧ ∏ k ∈ Finset.range n, (2 ^ e k + 3 ^ e k) = 3 ^ e n - 2 ^ e n
```

Read back by a model that saw only the Lean. *For every natural number n , there exists a sequence e of natural numbers such that $e(0) = 1$, for every natural number k we have $e(k + 1) = 2 \cdot e(k)$, and the product over all k in $\{0, 1, \dots, n - 1\}$ of $(2^{e(k)} + 3^{e(k)})$ equals $3^{e(n)} - 2^{e(n)}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n := by
  intro n
  let e : ℕ → ℕ := fun k => 2 ^ k
  refine ⟨e, ?_, ?_, ?_⟩
  · simp [e]
  · intro k
    simp [e, pow_succ, Nat.mul_comm]
  · have htel : ∀ m : ℕ,
      (∏ k ∈ Finset.range m,
        ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
        (3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m) := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      rw [Finset.prod_range_succ, ih]
      have hexp : 2 ^ (Nat.succ m) = 2 * 2 ^ m := by
        simp [pow_succ, Nat.mul_comm]
      have h3 : (3 : ℤ) ^ (2 ^ (Nat.succ m)) =
        ((3 : ℤ) ^ (2 ^ m)) ^ 2 := by
        rw [hexp, Nat.mul_comm, pow_mul]
      have h2 : (2 : ℤ) ^ (2 ^ (Nat.succ m)) =
        ((2 : ℤ) ^ (2 ^ m)) ^ 2 := by
        rw [hexp, Nat.mul_comm, pow_mul]
      rw [h3, h2]
      ring
    simp [e] using htel n
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh__48b45c78

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0e021d853daa103f

4.431 431. generalized_amc12b_2020_p22

The problem

For every real number $a \geq 1$, the set of values $(2^t - a)t / 4^t$, as t ranges over $\mathbb{R}$, has greatest element $1/(4a)$.

Lean statement

```
theorem generalized_amc12b_2020_p22
  (a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t : ℝ, x = (2 ^ t - a * t) * t / 4 ^ t}) :
  IsGreatest S (1 / (4 * a))
```

Parent. What is the maximum value of $\frac{(2^t - 3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12 : ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the coefficient and maximum, requiring the new hypothesis $a \geq 1$ to establish that $1/(2a)$ lies in the intermediate-value interval and to weight the square nonnegativity argument by a . The parent's fixed coefficient proof cannot supply the generalized theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient from 3 to arbitrary $a \geq 1$, requiring a new target value $1/(2a)$ in the intermediate-value argument and an a -dependent square-completion bound. The parent's fixed target and algebra do not directly supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient and maximum, requiring the new hypothesis $a \geq 1$ to establish positivity and a parameter-dependent square-completion identity. The parent's fixed coefficient 3 argument does not supply this symbolic reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2093 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_amc12b_2020_p22.extracted_1_1 (a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
(hS : S = {x | ∃ t, x = (2 ^ t - a * t) * t / 4 ^ t}) : IsGreatest S (1 / (4 * a))
```

Read back by a model that saw only the Lean. For every real number a with $1 \leq a$, if S is the set of all real numbers of the form $((2^t - a)t)/4^t$ as t ranges over the real numbers, then $1/(4a)$ is the greatest element of S .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem generalized_amc12b_2020_p22
(a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
(hS : S = {x : ℝ | ∃ t : ℝ, x = (2 ^ t - a * t) * t / 4 ^ t}) :
IsGreatest S (1 / (4 * a)) := by
have ha0 : 0 < a := by linarith
let g : ℝ → ℝ := fun t => t / 2 ^ t
have hg : Continuous g := by
dsimp [g]
exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
(fun x => by positivity)
have hi := intermediate_value_Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
have hleft : g 0 = 0 := by norm_num [g]
have hright : g 1 = (1 / 2 : ℝ) := by norm_num [g]
have hm : (1 / (2 * a) : ℝ) ∈ Set.Icc (g 0) (g 1) := by
rw [hleft, hright]
constructor
· positivity
· apply (div_le_iff (by positivity : (0 : ℝ) < 2 * a)).2
  nlinarith
obtain ⟨t, ht, hgt⟩ := hi hm
rw [hS]
constructor
· refine ⟨t, ?_⟩
have hden : (4 : ℝ) ^ t = (2 ^ t) ^ 2 := by
calc
(4 : ℝ) ^ t = (2 ^ 2) ^ t := by norm_num
_ = 2 ^ (2 * t) :=
(Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 t).symm
_ = 2 ^ t * 2 ^ t := by
rw [show (2 : ℝ) * t = t + t by ring, Real.rpow_add (by norm_num)]
_ = (2 ^ t) ^ 2 := by ring
rw [hden]
have hp : 0 < (2 : ℝ) ^ t := by positivity
have hrel : t / 2 ^ t = 1 / (2 * a) := by
simp [g] using hgt
have hid :
(2 ^ t - a * t) * t / (2 ^ t) ^ 2 =
t / 2 ^ t - a * (t / 2 ^ t) ^ 2 := by
field_simp [ne_of_gt hp] <|> ring
rw [hid, hrel]
field_simp [ne_of_gt ha0] <|> ring
· rintro x ⟨s, rfl⟩
have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
calc
(4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
_ = 2 ^ (2 * s) :=
(Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
_ = 2 ^ s * 2 ^ s := by
rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
_ = (2 ^ s) ^ 2 := by ring
rw [hden]
have hp : 0 < (2 : ℝ) ^ s := by positivity
have hid :
(2 ^ s - a * s) * s / (2 ^ s) ^ 2 =
s / 2 ^ s - a * (s / 2 ^ s) ^ 2 := by
field_simp [ne_of_gt hp] <|> ring
```

```

rw [hid]
have hsquare : 0 ≤ (s / 2 ^ s - 1 / (2 * a)) ^ 2 := sq_nonneg _
have hnonneg :
  0 ≤ a * (s / 2 ^ s - 1 / (2 * a)) ^ 2 :=
  mul_nonneg (le_of_lt ha0) hsquare
have hident :
  s / 2 ^ s - a * (s / 2 ^ s) ^ 2 =
  1 / (4 * a) - a * (s / 2 ^ s - 1 / (2 * a)) ^ 2 := by
  field_simp [ne_of_gt hp, ne_of_gt ha0] <|> ring
rw [hident]
linarith

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__c39c930b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c39c930bbcd712e1

4.432 432. exact_maximizer_characterization

The problem

Prove that for every real number s , $\frac{(2^s-3s)s}{4^s} \leq \frac{1}{12}$, with equality if and only if $\frac{s}{2^s} = \frac{1}{6}$; moreover, the value $\frac{1}{12}$ is attained for some real number t .

Lean statement

```
theorem exact_maximizer_characterization
  (S : Set ℝ)
  :
  (∀ s : ℝ, (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ s : ℝ,
    (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔
    s / 2 ^ s = 1 / 6) ∧
  (∃ t : ℝ, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12)
```

Parent. What is the maximum value of $\frac{(2^t-3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS: S = {x:ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-element claim to a full equality characterization, requiring proof that equality occurs exactly when $s / 2^s = 1/6$, in addition to the bound and attainment. This characterization is not supplied by merely recalling the parent's IsGreatest proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-element claim to a full maximizer characterization, including the nontrivial iff identifying equality exactly with $s/2^s = 1/6$. The parent's inequality proof does not establish this equality condition, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving an IsGreatest set witness to a full analytic characterization: global upper bound, exact equality condition, and attainment. The equality condition requires new reasoning that the parent proof does not supply, namely showing equality in the square bound forces $s / 2^s = 1/6$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2070 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exact_maximizer_characterization.extracted_1_1 :
  (∀ (s : ℝ), (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ (s : ℝ), (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔ s / 2 ^ s = 1 / 6) ∧
  ∃ t, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12
```

Read back by a model that saw only the Lean. For every real number s , $((2^s - 3s)s)/4^s \leq 1/12$; for every real number s , $((2^s - 3s)s)/4^s = 1/12$ if and only if $s/2^s = 1/6$; and there exists a real number t such that $((2^t - 3t)t)/4^t = 1/12$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem exact_maximizer_characterization
  (S : Set ℝ)
  :
  (∀ s : ℝ, (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ s : ℝ,
    (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔
    s / 2 ^ s = 1 / 6) ∧
  (∃ t : ℝ, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12) := by
  let g : ℝ → ℝ := fun t => t / 2 ^ t
  have hg : Continuous g := by
    dsimp [g]
    exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
    (fun x => by positivity)
  have hi := intermediate_value_Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
  have hm : (1 / 6 : ℝ) ∈ Set.Icc (g 0) (g 1) := by
    norm_num [g]
  obtain (t, ht, hgt) := hi hm
  have hattain : (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12 := by
    have hden : (4 : ℝ) ^ t = (2 ^ t) ^ 2 := by
      calc
        (4 : ℝ) ^ t = (2 ^ 2) ^ t := by norm_num
        _ = 2 ^ (2 * t) :=
          (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 t).symm
        _ = 2 ^ t * 2 ^ t := by
          rw [show (2 : ℝ) * t = t + t by ring, Real.rpow_add (by norm_num)]
        _ = (2 ^ t) ^ 2 := by ring
    rw [hden]
  have hp : 0 < (2 : ℝ) ^ t := by positivity
  dsimp [g] at hgt
  field_simp [ne_of_gt hp] at hgt
  nlinarith
constructor
· intro s
  have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
    calc
      (4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
      _ = 2 ^ (2 * s) :=
        (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
      _ = 2 ^ s * 2 ^ s := by
        rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
      _ = (2 ^ s) ^ 2 := by ring
  rw [hden]
  have hp : 0 < (2 : ℝ) ^ s := by positivity
  have hsquare : 0 ≤ (s / 2 ^ s - 1 / 6) ^ 2 := sq_nonneg _
  have hid :
    (2 ^ s - 3 * s) * s / (2 ^ s) ^ 2 =
```

```

      s / 2 ^ s - 3 * (s / 2 ^ s) ^ 2 := by
    field_simp [ne_of_gt hp] <|> ring
  rw [hid]
  nlinarith
· constructor
· intro s
  have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
    calc
      (4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
      _ = 2 ^ (2 * s) :=
        (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
      _ = 2 ^ s * 2 ^ s := by
        rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
      _ = (2 ^ s) ^ 2 := by ring
  rw [hden]
  have hp : 0 < (2 : ℝ) ^ s := by positivity
  have hsquare : 0 ≤ (s / 2 ^ s - 1 / 6) ^ 2 := sq_nonneg _
  have hid :
    (2 ^ s - 3 * s) * s / (2 ^ s) ^ 2 =
      s / 2 ^ s - 3 * (s / 2 ^ s) ^ 2 := by
    field_simp [ne_of_gt hp] <|> ring
  rw [hid]
  constructor
  · intro heq
    nlinarith
  · intro hq
    nlinarith
· exact ⟨t, hattain⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__dfb91952

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fb4e4a95f2f0f3fb

4.433 433. amc12b_2020_p22_mutation

The problem

Show that every real value between 0 and $\frac{(2^t-3t)t}{4^t}$ for some $t \in [0, 1]$.

Lean statement

```
theorem amc12b_2020_p22_mutation
  : ∀ y : ℝ, y ∈ Set.Icc 0 (1 / 12 : ℝ) →
    ∃ t ∈ Set.Icc 0 1,
      y = ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t := by
```

Parent. What is the maximum value of $\frac{(2^t-3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS: S = {x:ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the maximum-value claim into a substantially different range characterization: every value in $[0, 1/12]$ is attained on $[0, 1]$. Its proof requires a second continuity and intermediate-value argument, which the parent's upper-bound proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-value claim into a range-surjectivity statement: every y in $[0, 1/12]$ must be attained by an exponent t in $[0, 1]$. It requires a new continuity argument for the original expression and a second intermediate-value construction, beyond the parent's single extremum proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving a greatest element to proving that every value in the full interval $[0, 1/12]$ is attained. Its proof requires a new continuity argument for the original expression and a second intermediate-value application, which the parent's maximum proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2373 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12b_2020_p22_mutation.extracted_1_1 : ∀ y ∈ Set.Icc 0 (1 / 12),  
  ∃ t ∈ Set.Icc 0 1, y = (2 ^ t - 3 * t) * t / 4 ^ t
```

Read back by a model that saw only the Lean. For every real number y in the closed interval $[0, 1/12]$, there exists a real number t in the closed interval $[0, 1]$ such that $y = ((2^t - 3t)t) / 4^t$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem amc12b_2020_p22_mutation
  : ∀ y : ℝ, y ∈ Set.Icc 0 (1 / 12 : ℝ) →
    ∃ t ∈ Set.Icc 0 1,
      y = ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t := by
  let g : ℝ → ℝ := fun t => t / 2 ^ t
  let f : ℝ → ℝ := fun t => ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t
  have hg : Continuous g := by
    dsimp [g]
    exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
    (fun x => by positivity)
  have hf : Continuous f := by
    have h2 : Continuous (fun t : ℝ => (2 : ℝ) ^ t) :=
      Real.continuous_const_rpow (by norm_num)
    have h4 : Continuous (fun t : ℝ => (4 : ℝ) ^ t) :=
      Real.continuous_const_rpow (by norm_num)
    dsimp [f]
    exact ((h2.sub (continuous_const.mul continuous_id)).mul continuous_id).div h4
    (fun x => by positivity)
  have hi := intermediate_value_Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
  have hm : (1 / 6 : ℝ) ∈ Set.Icc (g 0) (g 1) := by
    norm_num [g]
  obtain ⟨t₀, ht₀, hgt₀⟩ := hi hm
  have hformula : ∀ u : ℝ, u / 2 ^ u = 1 / 6 → f u = 1 / 12 := by
    intro u hu
    have hden : (4 : ℝ) ^ u = (2 ^ u) ^ 2 := by
      calc
        (4 : ℝ) ^ u = (2 ^ 2) ^ u := by norm_num
        _ = 2 ^ (2 * u) :=
          (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 u).symm
        _ = 2 ^ u * 2 ^ u := by
          rw [show (2 : ℝ) * u = u + u by ring, Real.rpow_add (by norm_num)]
        _ = (2 ^ u) ^ 2 := by ring
    dsimp [f]
    rw [hden]
    have hp : 0 < (2 : ℝ) ^ u := by positivity
    field_simp [ne_of_gt hp] at hu ⊢
    nlinarith
  have hgt₀' : t₀ / 2 ^ t₀ = 1 / 6 := by
    simpa [g] using hgt₀
  have hft₀ : f t₀ = 1 / 12 := hformula t₀ hgt₀'
  intro y hy
  have hf0 : f 0 = 0 := by
    norm_num [f]
  have hi' := intermediate_value_Icc ht₀.1 hf.continuousOn
  have hy' : y ∈ Set.Icc (f 0) (f t₀) := by
    rw [hf0, hft₀]
    exact hy
  obtain ⟨t, ht, hyt⟩ := hi' hy'
  have ht01 : t ∈ Set.Icc 0 1 := by
    exact ⟨ht.1, le_trans ht.2 ht₀.2⟩
  exact ⟨t, ht01, hyt.symm⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__eba9b6e6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint eba9b6e66e48af7e

4.434 434. unique_lattice_coordinate_period

The problem

Let f have periods p and q , and suppose no nontrivial integer combination of p and q is zero. Show that every displacement represented as $mp + nq$ has unique integer coordinates (m, n) and is a period of f .

Lean statement

```
theorem unique_lattice_coordinate_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x)
  (hind : ∀ a b : ℤ, (a : ℝ) * p + (b : ℝ) * q = 0 → a = 0 ∧ b = 0) :
  ∀ d : ℝ, (∃ m n : ℤ, d = (m : ℝ) * p + (n : ℝ) * q) →
    ∃! z : ℤ × ℤ,
      d = (z.1 : ℝ) * p + (z.2 : ℝ) * q ∧
      ∀ x : ℝ, f (x + d) = f x
```

Parent. Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Parent in Lean

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

What it contributes. Supplies the integer-linear-combination periodicity checkpoint, reproduced as the named have hcomb and consumed to prove periodicity of the represented displacement.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new uniqueness characterization of integer coordinates, proved using the integer-independence hypothesis; the parent only establishes periodicity of integer combinations. Although the periodicity argument is reused, the uniqueness proof requires reasoning unavailable from the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness characterization of lattice coordinates, requiring the integer-independence hypothesis to show two representations coincide. The parent only establishes periodicity for integer combinations and does not supply this uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness theorem for lattice coordinates, requiring the integer-independence hypothesis and a difference-of-coordinates argument. Although it reuses the parent's periodicity result for existence, the parent does not supply the essential uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2502 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_lattice_coordinate_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p q : ℝ),
  (∀ (x : ℝ), f (x + p) = f x) →
  (∀ (x : ℝ), f (x + q) = f x) →
  (∀ (a b : ℤ), ↑a * p + ↑b * q = 0 → a = 0 ∧ b = 0) →
  ∀ (d : ℝ), (∃ m n, d = ↑m * p + ↑n * q) → ∃! z, d = ↑z.1 * p + ↑z.2 * q ∧ ∀ (x : ℝ), f (x + d) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers and every real numbers p and q , if p and q are periods of f —that is, $f(x + p) = f(x)$ and $f(x + q) = f(x)$ for every real x —and if the only integers a and b satisfying $a \cdot p + b \cdot q = 0$ are $a = 0$ and $b = 0$, then for every real number d that can be written as $d = m \cdot p + n \cdot q$ for some integers m and n , there exists a unique pair $z = (z_1, z_2)$ of integers such that $d = z_1 \cdot p + z_2 \cdot q$ and d is a period of f , meaning $f(x + d) = f(x)$ for every real x .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem unique_lattice_coordinate_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x)
  (hind : ∀ a b : ℤ, (a : ℝ) * p + (b : ℝ) * q = 0 → a = 0 ∧ b = 0) :
  ∀ d : ℝ, (∃ m n : ℤ, d = (m : ℝ) * p + (n : ℝ) * q) →
    ∃! z : ℤ × ℤ,
      d = (z.1 : ℝ) * p + (z.2 : ℝ) * q ∧
      ∀ x : ℝ, f (x + d) = f x := by
  have hcomb : ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x := by
    have hp_periodic : Function.Periodic f p := hp
    have hq_periodic : Function.Periodic f q := hq
    intro m n x
    have hpm : Function.Periodic f ((m : ℝ) * p) := hp_periodic.int_mul m
    have hqn : Function.Periodic f ((n : ℝ) * q) := hq_periodic.int_mul n
    calc
      f (x + (m : ℝ) * p + (n : ℝ) * q) =
        f ((x + (n : ℝ) * q) + (m : ℝ) * p) := by
          congr 1
          ring
      _ = f (x + (n : ℝ) * q) := hpm (x + (n : ℝ) * q)
      _ = f x := hqn x
  intro d hd
  obtain ⟨m, n, hdn⟩ := hd
  have hperiod : ∀ x : ℝ, f (x + d) = f x := by
    intro x
    rw [hdn]
    simpa [add_assoc] using hcomb m n x
  refine ⟨(m, n), ⟨hdn, hperiod⟩, ?_⟩
  intro z hz
  have hrel : (z.1 : ℝ) * p + (z.2 : ℝ) * q =
    (m : ℝ) * p + (n : ℝ) * q := by
    linarith [hz.1, hdn]
  have hdiff : ((z.1 - m : ℤ) : ℝ) * p +
    ((z.2 - n : ℤ) : ℝ) * q = 0 := by
    push_cast
    linarith [hrel]
```

```
have hcoords := hind (z.1 - m) (z.2 - n) hdiff
apply Prod.ext
· exact sub_eq_zero.mp hcoords.1
· exact sub_eq_zero.mp hcoords.2
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh.mh__1d992577

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1d9925779b572639

4.435 435. finite_integer_combination_period

The problem

Let f be a real-valued function, and let each member of a finite indexed family be a period of f . Show that every integer linear combination of these periods is also a period of f .

Lean statement

```
theorem finite_integer_combination_period (f : ℝ → ℝ) {ι : Type} [Fintype ι]
  (p : ι → ℝ) (hperiod : ∀ i : ι, ∀ x : ℝ, f (x + p i) = f x) :
  ∀ c : ι → ℤ, ∀ x : ℝ,
    f (x + ∑ i : ι, (c i : ℝ) * p i) = f x
```

Parent. Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Parent in Lean

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

What it contributes. Generalizes the parent's integer-multiple and period-composition argument from two named generators to an arbitrary finite indexed family and finite sum.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from two fixed periods to an arbitrary finite indexed family. Its proof requires finite-set induction and aggregation of periods, which the parent's two-generator argument does not directly supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two fixed generators to an arbitrary finite indexed family and requires induction over the finite sum, combining each generator's integer multiple as a period. The parent's two-generator calculation does not supply this finite-family induction or its sum bookkeeping.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two fixed generators to an arbitrary finite indexed family. Its proof requires finite-set induction and combining periods across the family, which the parent's two-generator argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2500 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_integer_combination_period.extracted_1_1 : ∀ (f : ℝ → ℝ) {ι : Type} [inst : Fintype ι] (p : ι → ℝ),
  (∀ (i : ι) (x : ℝ), f (x + p i) = f x) → ∀ (c : ι → ℤ) (x : ℝ), f (x + ∑ i, ↑(c i) * p i) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers, every finite type ι , and every family of real numbers p indexed by ι , if for every i in ι and every real number x we have $f(x + p(i)) = f(x)$, then for every family of integers c indexed by ι and every real number x , we have $f(x + \sum_{i \in \iota} (c(i) \cdot p(i))) = f(x)$, where each integer $c(i)$ is regarded as a real number.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem finite_integer_combination_period (f : ℝ → ℝ) {ι : Type} [Fintype ι]
  (p : ι → ℝ) (hperiod : ∀ i : ι, ∀ x : ℝ, f (x + p i) = f x) :
  ∀ c : ι → ℤ, ∀ x : ℝ,
    f (x + ∑ i : ι, (c i : ℝ) * p i) = f x := by
  classical
  intro c x
  have hsum : ∀ s : Finset ι,
    Function.Periodic f (∑ i ∈ s, (c i : ℝ) * p i) := by
    intro s
    induction s using Finset.induction_on with
    | empty =>
      intro y
      simp
    | @insert a s ha ih =>
      have hca : Function.Periodic f ((c a : ℝ) * p a) := by
        have hpa : Function.Periodic f (p a) := hperiod a
        exact hpa.int_mul (c a)
      have hadd := ih.add_period hca
      simp [Finset.sum_insert, ha, add_assoc, add_comm, add_left_comm] using hadd
  exact hsum Finset.univ x
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh.mh__5e6484a3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5e6484a32a7b7d00

4.436 436. integer_combination_period

The problem

Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Lean statement

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

Parent. Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Parent in Lean

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

What it contributes. Supplied the natural-multiple induction checkpoint for a single period; the child extends it to negative shifts and composes two independent period translations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one natural nonnegative iterate to arbitrary integer combinations of two independent periods. Its proof requires integer-period closure and composition of two translations, which the parent's induction argument does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires combining two independent periods and extending periodicity to arbitrary positive and negative integer multiples, using reasoning absent from the parent's natural-number induction. It is therefore not solvable by recalling the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from one positive natural iterate to arbitrary integer combinations of two independent periods. It requires handling integer multiples, including negative coefficients, and combining the two periodicities, none of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2497 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem integer_combination_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p q : ℝ),
  (∀ (x : ℝ), f (x + p) = f x) →
  (∀ (x : ℝ), f (x + q) = f x) → ∀ (m n : ℤ) (x : ℝ), f (x + ↑m * p + ↑n * q) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers and every real numbers p and q , if $f(x + p) = f(x)$ for every real x and $f(x + q) = f(x)$ for every real x , then for every integers m and n and every real x , $f(x + m p + n q) = f(x)$, where the integers m and n are regarded as real numbers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x := by
  have hp_periodic : Function.Periodic f p := hp
  have hq_periodic : Function.Periodic f q := hq
  intro m n x
  have hpm : Function.Periodic f ((m : ℝ) * p) := hp_periodic.int_mul m
  have hqn : Function.Periodic f ((n : ℝ) * q) := hq_periodic.int_mul n
  calc
    f (x + (m : ℝ) * p + (n : ℝ) * q) =
      f ((x + (n : ℝ) * q) + (m : ℝ) * p) := by
        congr 1
        ring
    _ = f (x + (n : ℝ) * q) := hpm (x + (n : ℝ) * q)
    _ = f x := hqn x
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh__4b068dd2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b068dd24ba4c73c

4.437 437. periodic_integer_subgroup_action

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x + p) = f(x)$ for every real x . Show that every translation belonging to the additive subgroup generated by p is a period of f , including negative integer multiples.

Lean statement

```
theorem periodic_integer_subgroup_action (f : ℝ → ℝ) (p : ℝ) (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ d : ℝ, d ∈ AddSubgroup.zmultiples p → Function.Periodic f d := by
```

Parent. For a real-valued function and a real shift p , the law $f(x + p) = f(x)$ for every real x is equivalent to saying that every natural multiple of p is a period of f .

Parent in Lean

```
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :
  (∀ x : ℝ, f (x + p) = f x) ↔
  ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by
```

What it contributes. Supplied the base-period conversion from the pointwise equation and the natural-multiple induction checkpoint; the child extends this to signed multiples and subgroup membership.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely strengthens natural multiples to all signed integer multiples represented via additive-subgroup membership. Its proof requires new reasoning for negative periods and extracting an integer representation from subgroup membership, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires extending period invariance from natural multiples to all signed integer multiples, then converting subgroup membership into such a representation. The parent's proof supplies only natural multiplication and does not establish negative-period invariance or the subgroup representation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens natural-multiple periodicity to every signed integer multiple represented by membership in `AddSubgroup.zmultiples p`. Its proof must handle negative integers and extract an integer representation from subgroup membership, neither of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2501 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem periodic_integer_subgroup_action.extracted_1_1 : ∀ (f : ℝ → ℝ) (p : ℝ),  
  (∀ (x : ℝ), f (x + p) = f x) → ∀ d ∈ AddSubgroup.zmultiples p, Function.Periodic f d
```

Read back by a model that saw only the Lean. For every function $f : \mathbb{R} \rightarrow \mathbb{R}$ and every real number p , if for every real number x we have $f(x + p) = f(x)$, then for every d belonging to the additive subgroup of $\mathbb{R}$ consisting of the integer multiples of p , d is a period of f ; that is, for every real number x , $f(x + d) = f(x)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem periodic_integer_subgroup_action (f : ℝ → ℝ) (p : ℝ) (hperiod : ∀ x : ℝ, f (x + p) = f x) :  
  ∀ d : ℝ, d ∈ AddSubgroup.zmultiples p → Function.Periodic f d := by  
  intro d hd  
  have hp : Function.Periodic f p := by  
    intro x  
    exact hperiod x  
  have hnég : Function.Periodic f (-p) := hp.neg  
  have hsigned : ∀ n : ℤ, Function.Periodic f (n • p) := by  
    intro n  
    cases n with  
    | ofNat n =>  
      simpa using hp.nsmul n  
    | negSucc n =>  
      have hh := hnég.nsmul (n + 1)  
      convert hh using 1 <|> simp [negSucc_zsmul] <|> ring  
  have hrepr : ∃ n : ℤ, n • p = d := (AddSubgroup.mem_zmultiples_iff).mp hd  
  rcases hrepr with ⟨n, hn⟩  
  rw [← hn]  
  exact hsigned n
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.ms.mh__76336c3e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 76336c3ebf68c293

4.438 438. odd_iff_three_dvd_pow_add_one

The problem

For every natural base b congruent to 2 modulo 3 and every natural exponent k , k is odd if and only if 3 divides $b^k + 1$.

Lean statement

```
theorem odd_iff_three_dvd_pow_add_one (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reused the residue-derived divisor $3 \mid b+1$ and the odd-power divisibility checkpoint, while adding an inductive even-power residue obstruction for the converse.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: from $3 \mid b^k + 1$ it must prove k is odd, requiring a new modular-periodicity argument for even powers. The parent only establishes divisibility for known odd exponents, so recalling its proof does not solve the child.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way odd-exponent divisibility result into an iff for arbitrary exponents. Its reverse direction requires a new even/odd case split and modular induction showing even powers are congruent to 1 modulo 3, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent from one guaranteed odd-exponent divisibility instance to an iff characterizing all exponents. Its reverse direction requires a new modular argument ruling out even exponents, which the parent's odd-power factorization proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2064 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_iff_three_dvd_pow_add_one.extracted.1.1 (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1
```

Read back by a model that saw only the Lean. For natural numbers b and k , if b has remainder 2 upon division by 3, then k is odd if and only if 3 divides $b^k + 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_iff_three_dvd_pow_add_one (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1 := by
  have hbase : 3 ∣ b + 1 := by
    have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
    rw [hb] at hdecomp
    use b / 3 + 1
    omega
  have hb2 : b ^ 2 % 3 = 1 := by
    rw [Nat.pow_two, Nat.mul_mod, hb]
  have h_even_pow : ∀ n : ℕ, b ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      have hexp : 2 * Nat.succ n = 2 * n + 2 := by omega
      calc
        b ^ (2 * Nat.succ n) % 3 = (b ^ (2 * n) * b ^ 2) % 3 := by rw [hexp, pow_add]
        _ = ((b ^ (2 * n) % 3) * (b ^ 2 % 3)) % 3 := by rw [Nat.mul_mod]
        _ = 1 := by rw [ih, hb2]
  constructor
  · intro hk
    have hpow : b + 1 ∣ b ^ k + 1 ^ k := by
      exact Odd.nat_add_dvd_pow_add_pow b 1 hk
    have hpow2 : b + 1 ∣ b ^ k + 1 := by simpa using hpow
    exact dvd_trans hbase hpow2
  · intro hdiv
    rcases even_or_odd k with he | ho
    · ex falso
      obtain ⟨n, hn⟩ := he
      have hkform : k = 2 * n := by omega
      have hzero : (b ^ k + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hdiv
      have hbad : (b ^ (2 * n) + 1) % 3 = 2 := by
        rw [Nat.add_mod, h_even_pow]
      rw [hkform] at hzero
      omega
    · exact ho
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.mh__c1bdceed

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c1bdceed49f09898

4.439 439. fourth_periodic_power_divisibility_iff_base_congruent_four

The problem

For natural numbers b and n , prove that $5 \mid b^{4n+1} + 1$ if and only if b leaves remainder 4 upon division by 5.

Lean statement

```
theorem fourth_periodic_power_divisibility_iff_base_congruent_four (b n : ℕ) : 5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4
```

Parent. For natural numbers b and n , prove that $3 \mid b^{2n+1} + 1$ if and only if b leaves remainder 2 upon division by 3.

Parent in Lean

```
theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

What it contributes. Preserves the parent's residue-class divisibility strategy and the reverse-direction factorization through $b+1$, while changing the modulus and exponent periodicity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the modulus and the periodic exponent, requiring a five-way residue split and new periodicity arguments for residues 2 and 3 modulo 5. The parent's modulo-3 proof does not supply these cases, so this is substantive parameter-shift reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the modulus and periodic exponent, requiring new residue classes modulo 5 and inductive proofs that residues 2 and 3 retain their values under exponent shifts by 4. This reasoning is not supplied by the parent's modulo-3 argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Changing the modulus from 3 to 5 and the exponent pattern from odd to $4n+1$ requires new residue-cycle reasoning, including induction for powers of 2 and 3 modulo 5. The parent's mod-3 case split and divisibility argument do not supply this proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2479 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fourth_periodic_power_divisibility_iff_base_congruent_four.extracted_1_1 (b n : ℕ) :  
  5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4
```

Read back by a model that saw only the Lean. *For natural numbers b and n , 5 divides b raised to the power $4n + 1$ plus 1 if and only if the remainder when b is divided by 5 is 4.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem fourth_periodic_power_divisibility_iff_base_congruent_four (b n : ℕ) : 5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4 := by
  have hepos : 0 < 4 * n + 1 := by omega
  have hpow2 : ∀ k : ℕ, (2 : ℕ) ^ (4 * k + 1) % 5 = 2 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 4 * (Nat.succ k) + 1 = (4 * k + 1) + 4 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have hpow3 : ∀ k : ℕ, (3 : ℕ) ^ (4 * k + 1) % 5 = 3 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 4 * (Nat.succ k) + 1 = (4 * k + 1) + 4 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  constructor
  · intro hd
    have hzero : (b ^ (4 * n + 1) + 1) % 5 = 0 :=
      Nat.dvd_iff_mod_eq_zero.mp hd
    have hlt : b % 5 < 5 := Nat.mod_lt b (by norm_num)
    have hcases : b % 5 = 0 ∨ b % 5 = 1 ∨ b % 5 = 2 ∨ b % 5 = 3 ∨ b % 5 = 4 := by omega
    rcases hcases with hres | hres | hres | hres | hres
    · have hbmod : Nat.ModEq 5 b 0 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm0 : b ^ (4 * n + 1) % 5 = 0 := by
        simpa [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm0] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 1 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm1 : b ^ (4 * n + 1) % 5 = 1 := by
        simpa [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm1] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 2 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm2 : b ^ (4 * n + 1) % 5 = 2 := by
        calc
          b ^ (4 * n + 1) % 5 = (2 : ℕ) ^ (4 * n + 1) % 5 := by
            simpa [Nat.ModEq] using hp
          _ = 2 := hpow2 n
      rw [Nat.add_mod, hpm2] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 3 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm3 : b ^ (4 * n + 1) % 5 = 3 := by
        calc
          b ^ (4 * n + 1) % 5 = (3 : ℕ) ^ (4 * n + 1) % 5 := by
            simpa [Nat.ModEq] using hp
          _ = 3 := hpow3 n
      rw [Nat.add_mod, hpm3] at hzero
      norm_num at hzero
    · exact hres
  · intro hres
```

```

have hbase : 5 ∣ b + 1 := by
  refine ⟨b / 5 + 1, ?_⟩
  have hdecomp := Nat.mod_add_div b 5
  omega
have hodd : Odd (4 * n + 1) := by
  exact ⟨2 * n, by omega⟩
have hpow : b + 1 ∣ b ^ (4 * n + 1) + 1 ^ (4 * n + 1) := by
  exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
have hpow' : b + 1 ∣ b ^ (4 * n + 1) + 1 := by simpa using hpow
exact dvd_trans hbase hpow'

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.mh__e6074f46

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e6074f46c14b4bc1

4.440 440. factorial_polynomial_bound

The problem

Prove that for every natural number n with $4 \leq n$, we have $n^2 + n \leq n!$.

Lean statement

```
theorem factorial_polynomial_bound (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n ! := by
```

Parent. Show that for any integer $n \geq 4$, we have $n^2 \leq n!$.

Parent in Lean

```
theorem induction_ineq_nsqlefactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the conclusion to $n^2 + n \leq n!$, which is not derivable from the parent's bound. Its proof requires a new factorial-growth argument via $(n-2)!$ and does not reuse the parent's closing lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bound from n^2 to $n^2 + n$ and requires the new estimate $n+1 \leq (n-1)!$, proved through a separate factorial recurrence and arithmetic argument. The parent's proof only establishes $n \leq (n-1)!$ and cannot supply this stronger obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bound by proving the additional linear term, requiring a new factorial-growth argument involving $(n-2)!$ and nonlinear arithmetic. The parent's proof only establishes $n^2 \leq n!$ and cannot supply this stronger obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2060 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem factorial_polynomial_bound.extracted_1_1 (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n !
```

Read back by a model that saw only the Lean. For every natural number n satisfying $4 \leq n$, the sum $n^2 + n$ is at most $n!$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem factorial_polynomial_bound (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n ! := by
  have hpred : n - 2 ≤ (n - 2)! := by
    exact Nat.self_le_factorial (n - 2)
  have hmul : (n - 1) * (n - 2) ≤ (n - 1) * (n - 2)! := by
    exact Nat.mul_le_mul_left (n - 1) hpred
  have hsub1 : n - 1 = n - 2 + 1 := by
    omega
  have hsub2 : n = n - 2 + 2 := by
    omega
  have hgrowth : n + 1 ≤ (n - 1)! := by
    have hrec : (n - 1) * (n - 2)! = (n - 1)! := by
      exact Nat.mul_factorial_pred (by omega)
    rw [← hrec]
    nlinarith [hmul]
  have hfact : n * (n - 1)! = n ! := by
    exact Nat.mul_factorial_pred (by omega)
  calc
    n ^ 2 + n = n * (n + 1) := by ring
    _ ≤ n * (n - 1)! := Nat.mul_le_mul_left n hgrowth
    _ = n ! := hfact
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_ineq_nsqlefactn__mh__aa61f93c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aa61f93c981b6696

4.441 441. sq_le_factorial_iff

The problem

For which natural numbers n does $n^2 \leq n!$ hold? Show that it holds exactly when $n = 0$, $n = 1$, or $n \geq 4$.

Lean statement

```
theorem sq_le_factorial_iff (n : ℕ) : n ^ 2 ≤ Nat.factorial n ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n
```

Parent. Show that for any integer $n \geq 4$, we have $n^2 \leq n!$.

Parent in Lean

```
theorem induction_ineq_nsqlefactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
```

What it contributes. Retained the parent's factorial-growth multiplication checkpoint as the induction step, while extending it to an all-natural-number classification with explicit exceptional cases.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child gives a complete iff characterization, requiring separate exceptional-case analysis for $n=0,1,2,3$ and an induction proving the inequality for all $n \geq 4$. This is substantive new reasoning beyond the parent's one-direction threshold argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a complete iff characterization, requiring explicit exclusion of $n=2,3$ and a separate induction proving the bound for every $n \geq 4$. This substantially exceeds the parent's one-way threshold argument and cannot be solved by recalling it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a complete iff characterization, including explicit exceptional cases $n = 0$ and $n = 1$ and exclusion of $n = 2,3$. Proving the forward cases and the all- $n \geq 4$ direction requires case analysis and induction beyond the parent's one-way threshold argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2714 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sq_le_factorial_iff.extracted_1_1 (n : ℕ) : n ^ 2 ≤ n ! ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n
```

Read back by a model that saw only the Lean. For every natural number n , n squared is less than or equal to n factorial if and only if n equals 0, n equals 1, or n is at least 4.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sq_le_factorial_iff (n : ℕ) : n ^ 2 ≤ Nat.factorial n ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n := by
  have htail : ∀ k : ℕ, 4 ≤ k → k ^ 2 ≤ Nat.factorial k := by
    intro k
    induction k with
    | zero =>
      intro hk
      omega
    | succ k ih =>
      intro hk
      by_cases hprev : 4 ≤ k
      · have hprevineq : k ^ 2 ≤ Nat.factorial k := ih hprev
        have hstep : (Nat.succ k) ^ 2 ≤ Nat.succ k * k ^ 2 := by
          nlinarith
        have hmul : Nat.succ k * k ^ 2 ≤ Nat.succ k * Nat.factorial k :=
          Nat.mul_le_mul_left (Nat.succ k) hprevineq
        calc
          (Nat.succ k) ^ 2 ≤ Nat.succ k * k ^ 2 := hstep
          _ ≤ Nat.succ k * Nat.factorial k := hmul
          _ = Nat.factorial (Nat.succ k) := by simp [Nat.factorial_succ]
      · have hk3 : k = 3 := by omega
        subst k
        norm_num
  constructor
  · intro h
    by_cases hs : n < 4
    · have hcases : n = 0 ∨ n = 1 ∨ n = 2 ∨ n = 3 := by omega
      rcases hcases with rfl | rfl | rfl | rfl
      · left
        rfl
      · exact Or.inr (Or.inl rfl)
      · norm_num at h
      · norm_num at h
    · exact Or.inr (Or.inr (by omega))
  · intro h
    rcases h with rfl | rfl | hn
    · norm_num
    · norm_num
    · exact htail n hn
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_ineq_nsqulefactn__mh__c95b5f83

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c95b5f83df9dc725

4.442 442. quadratic_strict_sublevel_radius

The problem

For every positive radius and every real number x , the strict quadratic sublevel condition $x^2 - 5x - 4 < r^2 - 41/4$ holds exactly when x lies in the centered open interval $(5/2 - r, 5/2 + r)$.

Lean statement

```
theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) : x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

Parent. For every nonnegative radius at most 10, the quadratic sublevel set is exactly the centered interval determined by that radius.

Parent in Lean

```
theorem quadratic_sublevel_radius (r x : ℝ) (hr₀ : 0 ≤ r) (hr₁₀ : r ≤ 10) : x ^ 2 - 5 * x - 4 ≤ r ^ 2 - 41 / 4 ↔ x ∈ Set.Icc (5 / 2 - r) (5 / 2 + r)
```

What it contributes. Reused the parent's centered quadratic normalization and factor-product interval argument, strengthened from non-strict closed-radius bounds to strict positivity and open endpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes both the inequality and the interval from non-strict to strict, requiring boundary exclusion and strictly positive factor products. Its reverse direction also needs strict positivity from hr , so the parent's nonnegative-product argument does not directly supply the proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the non-strict sublevel characterization into a strict one, requiring endpoint exclusion and strictly positive factor products; the parent's nonnegative-product argument does not supply this. The positive-radius hypothesis is used to establish strict inequalities and the open interval conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the inequality and interval from closed to strict, requiring new boundary-exclusion reasoning and strict positivity arguments from hr . The parent's non-strict sublevel proof does not by itself establish this open-interval characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_strict_sublevel_radius.extracted_1_1 (r x : ℝ) (hr : 0 < r) :  
  x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

Read back by a model that saw only the Lean. For real numbers r and x , if $0 < r$, then $x^2 - 5x - 4 < r^2 - 41/4$ if and only if x lies in the open interval $(5/2 - r, 5/2 + r)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) :  
  x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔  
  x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r) := by  
  have hcompleted : x ^ 2 - 5 * x - 4 = (x - 5 / 2) ^ 2 - 41 / 4 := by  
    ring  
  have hfactor :  
    (x - 5 / 2 + r) * (x - 5 / 2 - r) =  
    (x - 5 / 2) ^ 2 - r ^ 2 := by  
    ring  
  constructor  
  · intro h  
    constructor  
    · by_contra hx  
      have hx' : x ≤ 5 / 2 - r := le_of_not_gt hx  
      have h₁ : x - 5 / 2 + r ≤ 0 := by linarith  
      have h₂ : x - 5 / 2 - r ≤ 0 := by linarith  
      have hp : 0 ≤ (x - 5 / 2 + r) * (x - 5 / 2 - r) :=  
        mul_nonneg_of_nonpos_of_nonpos h₁ h₂  
      linarith [hcompleted, hfactor]  
    · by_contra hx  
      have hx' : 5 / 2 + r ≤ x := le_of_not_gt hx  
      have h₁ : 0 ≤ x - 5 / 2 - r := by linarith  
      have h₂ : 0 ≤ x - 5 / 2 + r := by linarith  
      have hp : 0 ≤ (x - 5 / 2 - r) * (x - 5 / 2 + r) :=  
        mul_nonneg h₁ h₂  
      linarith [hcompleted, hfactor]  
  · rintro ⟨hleft, hright⟩  
    have h₁ : 0 < r - (x - 5 / 2) := by linarith  
    have h₂ : 0 < r + (x - 5 / 2) := by linarith  
    have hp : 0 < (r - (x - 5 / 2)) * (r + (x - 5 / 2)) :=  
      mul_pos h₁ h₂  
    have hfactor' :  
      (r - (x - 5 / 2)) * (r + (x - 5 / 2)) =  
      r ^ 2 - (x - 5 / 2) ^ 2 := by  
      ring  
    linarith [hcompleted, hfactor']
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_101__me.mh__f403dcac

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 67bb649ddbcaf76c

4.443 443. quadratic_interval_characterization

The problem

For real numbers a , b , and x with $a \leq b$, show that $(x - a)(x - b) \leq 0$ if and only if $x \in [a, b]$.

Lean statement

```
theorem quadratic_interval_characterization (a b x : ℝ) (hab : a ≤ b) : ((x - a) * (x - b) ≤ 0) ↔ x ∈ Set.Icc a b
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed quadratic bound to arbitrary endpoints and proves an iff characterization of the entire interval. It additionally requires the converse implication from interval membership to the product inequality, which the parent proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the specific quadratic bound to an iff characterization for arbitrary endpoints and requires proving both directions, including deriving interval membership from the product inequality. The parent's one-way contradiction proof does not supply the reverse implication or the symbolic endpoint reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely general equivalence characterizing an interval by the sign of a quadratic, requiring both-direction reasoning for arbitrary endpoints and a converse product-sign argument. The parent only establishes one specific inequality-to-interval implication and cannot supply this universal characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1775 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_interval_characterization.extracted_1_1 (a b x : ℝ) (hab : a ≤ b) :  
  (x - a) * (x - b) ≤ 0 ↔ x ∈ Set.Icc a b
```

Read back by a model that saw only the Lean. For real numbers a , b , and x , if $a \leq b$, then $(x - a)(x - b) \leq 0$ if and only if x lies in the closed interval $[a, b]$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_interval_characterization (a b x : ℝ) (hab : a ≤ b) :
  ((x - a) * (x - b) ≤ 0) ↔ x ∈ Set.Icc a b := by
  constructor
  · intro h
    constructor
    · by_contra hxa
      have hxa' : x < a := lt_of_not_ge hxa
      have hxb' : x < b := by linarith
      have hprod : 0 < (x - a) * (x - b) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      linarith
    · by_contra hxb
      have hxb' : b < x := lt_of_not_ge hxb
      have hxa' : a < x := by linarith
      have hprod : 0 < (x - a) * (x - b) :=
        mul_pos (by linarith) (by linarith)
      linarith
  · intro hx
    rcases hx with ⟨hxa, hxb⟩
    have hleft : 0 ≤ x - a := sub_nonneg.mpr hxa
    have hright : x - b ≤ 0 := sub_nonpos.mpr hxb
    have hprod : (x - a) * (x - b) ≤ 0 :=
      mul_nonpos_of_nonneg_of_nonpos hleft hright
    exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__mh__4030f0e5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e678c3b76bd1bd90

4.444 444. interval_from_factored_quadratic

The problem

Let $a \leq b$. Show that every real number x satisfying $(x - a)(x - b) \leq 0$ lies in the interval $[a, b]$.

Lean statement

```
theorem interval_from_factored_quadratic (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) : x ∈ Set.Icc a b
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the two fixed endpoints and requires proving the symbolic factorization into $(x - a)(x - b)$, then using the arbitrary ordering $a \leq b$. The parent's fixed-constant sign argument cannot be recalled directly to establish this general theorem.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed quadratic to arbitrary ordered roots a and b , requiring symbolic factorization and a parameterized sign argument. The parent's proof for the specific endpoints -2 and 7 does not directly supply this reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed interval and quadratic to arbitrary endpoints $a \leq b$. It requires establishing the symbolic factorization and using endpoint ordering, reasoning not supplied by the parent's fixed-constant proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2467 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem interval_from_factored_quadratic.extracted_1_1 (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) :  
  x ∈ Set.Icc a b
```

Read back by a model that saw only the Lean. For real numbers a , b , and x , if $a \leq b$ and $x^2 - (a + b)x + ab \leq 0$, then x belongs to the closed interval $[a, b]$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem interval_from_factored_quadratic (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) : x ∈ Set.Icc a b := by
  have hfactor : x ^ 2 - (a + b) * x + a * b = (x - a) * (x - b) := by
    ring
  constructor
  · by_contra hx
    have hx' : x < a := lt_of_not_ge hx
    have hleft : x - a < 0 := by linarith
    have hright : x - b < 0 := by linarith
    have hprod : 0 < (x - a) * (x - b) :=
      mul_pos_of_neg_of_neg hleft hright
    nlinarith [h, hfactor]
  · by_contra hx
    have hx' : b < x := lt_of_not_ge hx
    have hleft : 0 < x - a := by linarith
    have hright : 0 < x - b := by linarith
    have hprod : 0 < (x - a) * (x - b) :=
      mul_pos hleft hright
    nlinarith [h, hfactor]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__mh__cd27eeb2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cd27eeb20e339bda

4.445 445. shifted_square_value_eq_twenty_nine_iff

The problem

For the shifted-square quadratic $f(x) = x^2 - 6x + 13$, prove that for every real number x satisfying $x^2 - 5x - 4 \leq 10$, $f(x) = 29$ if and only if $x = -2$.

Lean statement

```
theorem shifted_square_value_eq_twenty_nine_iff
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  ∀ x : ℝ, x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2)
```

Parent. The shifted-square quadratic attains its maximum value 29 on the domain defined by $x^2 - 5x - 4 \leq 10$.

Parent in Lean

```
theorem shifted_square_isGreatest_on_sublevel
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsGreatest (f '' {x : ℝ | x ^ 2 - 5 * x - 4 ≤ 10}) 29 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a greatest-value statement into an equality characterization: it must factor equality at 29, analyze both roots, and use the feasible interval to exclude $x = 8$. The parent's upper-bound argument alone does not supply this iff proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-value claim into a uniqueness characterization. Proving equality forces a factorization and a zero-product case split, with the interval hypothesis needed to exclude the extraneous root $x = 8$; this reasoning is not supplied by the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-value claim to a uniqueness characterization. Proving equality forces $(x + 2) * (x - 8) = 0$ and then uses the domain bound to exclude $x = 8$, reasoning not supplied by the parent's upper-bound proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1782 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_square_value_eq_twenty_nine_iff.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 6 * x + 13) → ∀ (x : ℝ), x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2)
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers, if $f(x) = x^2 - 6x + 13$ for every real number x , then for every real number x satisfying $x^2 - 5x - 4 \leq 10$, $f(x) = 29$ if and only if $x = -2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem shifted_square_value_eq_twenty_nine_iff
  (f : ℝ → ℝ)
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :
  ∀ x : ℝ, x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2) := by
  have hformula : ∀ z : ℝ, f z = z ^ 2 - 6 * z + 13 := by
    intro z
    rw [h₀]
  have hcert : ∀ z : ℝ, z ^ 2 - 5 * z - 4 ≤ 10 → z ∈ Set.Icc (-2) 7 := by
    intro z hz
    constructor
    · by_contra h
      have hz' : z < -2 := lt_of_not_ge h
      have hp : 0 < (z + 2) * (z - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra h
      have hz' : 7 < z := lt_of_not_ge h
      have hp : 0 < (z + 2) * (z - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  intro x hx
  constructor
  · intro hvalue
    have hinterval := hcert x hx
    have hzero : (x + 2) * (x - 8) = 0 := by
      rw [hformula x] at hvalue
      nlinarith
    rcases mul_eq_zero.mp hzero with hleft | hright
    · linarith
    · linarith [hinterval.2]
  · intro hx2
    subst x
    rw [hformula (-2)]
    norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_101__x__mathd_algebra_410__xe.mh__9ef0003d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 418cae2b9cdfd644

4.446 446. circle_unique_y_coordinate

The problem

For which real values of x does the equation $x^2 + 8x + y^2 - 6y = 0$ determine a unique real value of y ? Show that these values are $x = -9$ and $x = 1$.

Lean statement

```
theorem circle_unique_y_coordinate (x : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

Parent. Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Parent in Lean

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

What it contributes. Reuses the parent circle's completed-square structure and endpoint factorization, but changes the target to an exact uniqueness characterization via reflection symmetry.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new uniqueness argument: it reflects any solution across $y=3$, uses uniqueness to force $y=3$, and then characterizes the boundary cases. The parent only establishes existence over the full interval and does not supply this reflection-and-uniqueness reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a circle point into an exact uniqueness characterization. Its proof requires a reflection argument showing uniqueness forces $y = 3$, followed by endpoint factorization, which is not supplied by the parent's projection proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a point on the circle into a uniqueness characterization. Its reflection argument shows that uniqueness forces $y=3$, followed by a zero-product endpoint case split, reasoning not supplied by the parent's projection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2634 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_unique_y_coordinate.extracted_1_1 (x : ℝ) :  
  (∃! y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

Read back by a model that saw only the Lean. For every real number x , there exists exactly one real number y such that $x^2 + 8x + y^2 - 6y = 0$ if and only if $x = -9$ or $x = 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_unique_y_coordinate (x : ℝ) :  
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1 := by  
  constructor  
  · rintro ⟨y, hy, hyuniq⟩  
    have hy_reflected : x ^ 2 + 8 * x + (6 - y) ^ 2 - 6 * (6 - y) = 0 := by  
      nlinarith [hy]  
    have heq : 6 - y = y := hyuniq (6 - y) hy_reflected  
    have hy3 : y = 3 := by  
      linarith  
    have hcircle : (x + 4) ^ 2 = 25 := by  
      nlinarith [hy, hy3]  
    have hprod : (x + 9) * (x - 1) = 0 := by  
      nlinarith [hcircle]  
    rcases mul_eq_zero.mp hprod with hleft | hright  
    · left  
      linarith  
    · right  
      linarith  
  · intro hx  
    rcases hx with rfl | rfl  
    · refine ⟨3, by norm_num, ?_⟩  
      intro y hy  
      nlinarith [hy]  
    · refine ⟨-9, by norm_num, ?_⟩  
      intro y hy  
      nlinarith [hy]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh__958b7837

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 958b783736d1ea6e

4.447 447. quadratic_sublevel_characterization

The problem

Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. Show that on $[a, b]$, the points where $k(x - v)^2 + c \leq c + k\delta^2$ are exactly those satisfying $|x - v| \leq \delta$.

Lean statement

```
theorem quadratic_sublevel_characterization
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  :
  ∀ x ∈ Set.Icc a b,
    k * (x - v)^2 + c ≤ c + k * δ^2 ↔ |x - v| ≤ δ
```

Parent. Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. For $f(x) = k(x - v)^2 + c$ on $[a, b]$, prove that v is the unique minimizer and that every point at distance at least δ from v has value at least $c + k\delta^2$.

Parent in Lean

```
theorem scaled_translated_square_gap
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => k * (x - v)^2 + c) (Set.Icc a b) v ∧
    (fun x : ℝ => k * (x - v)^2 + c) v = c ∧
    (∀ x ∈ Set.Icc a b, δ ≤ |x - v| →
      c + k * δ^2 ≤ k * (x - v)^2 + c) ∧
    (∀ x ∈ Set.Icc a b, k * (x - v)^2 + c = c → x = v)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from one-sided minimum/gap facts to an exact sublevel-set characterization, requiring a converse argument using absolute-value squares and positivity of k . The parent's minimization and equality-uniqueness proof does not supply this equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided lower-gap statement into an iff characterization of the entire quadratic sublevel set. Its proof requires establishing the reverse inequality and relating squared absolute values, reasoning not supplied by the parent's minimum and uniqueness arguments.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided lower-bound claim into an iff characterization of the entire quadratic sublevel set. Its converse requires absolute-value squaring and a nonnegative-factor product argument that the parent's minimum and gap proof does not supply.

Check

statement_type_check

By

lean

Result

type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_sublevel_characterization.extracted_1_1 : ∀ (a b v c k δ : ℝ),
  0 < k → 0 ≤ δ → a ≤ v → v ≤ b → ∀ x ∈ Set.Icc a b, k * (x - v) ^ 2 + c ≤ c + k * δ ^ 2 ↔ |x - v| ≤ δ
```

Read back by a model that saw only the Lean. For all real numbers a, b, v, c, k , and δ , if $k > 0$, $\delta \geq 0$, $a \leq v$, and $v \leq b$, then for every real number x in the closed interval $[a, b]$, the inequality $k(x-v)^2 + c \leq c + k\delta^2$ holds if and only if $|x-v| \leq \delta$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_sublevel_characterization
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  :
  ∀ x ∈ Set.Icc a b,
    k * (x - v) ^ 2 + c ≤ c + k * δ ^ 2 ↔ |x - v| ≤ δ := by
  intro x hx
  have habs_sq : |x - v| ^ 2 = (x - v) ^ 2 := by
    by_cases h : 0 ≤ x - v
    · rw [abs_of_nonneg h]
    · have h' : x - v ≤ 0 := le_of_not_ge h
      rw [abs_of_nonpos h']
      ring
  constructor
  · intro hvalue
    have hsq : (x - v) ^ 2 ≤ δ ^ 2 := by
      nlinarith
    have habs_nonneg : 0 ≤ |x - v| := abs_nonneg (x - v)
    nlinarith [habs_sq]
  · intro hdist
    have hplus : 0 ≤ δ + |x - v| := by
      nlinarith [abs_nonneg (x - v)]
    have hprod : 0 ≤ (δ - |x - v|) * (δ + |x - v|) := by
      exact mul_nonneg (sub_nonneg.mpr hdist) hplus
    have habs_sq_le : |x - v| ^ 2 ≤ δ ^ 2 := by
      nlinarith [hprod]
    have hsq : (x - v) ^ 2 ≤ δ ^ 2 := by
      nlinarith [habs_sq, habs_sq_le]
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.me.mh__5c254eae

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 53fcdf72f7fb3b88

4.448 448. equal_values_scaled_translated_quadratic

The problem

Let $a \leq v \leq b$ and $k > 0$. If $x, y \in [a, b]$ satisfy $k(x - v)^2 + c = k(y - v)^2 + c$, prove that either $x = y$ or $x + y = 2v$.

Lean statement

```
theorem equal_values_scaled_translated_quadratic
  (a b v c k x y : ℝ)
  (hk : 0 < k)

  (heq : k * (x - v)^2 + c = k * (y - v)^2 + c) :
  x = y ∨ x + y = 2 * v
```

Parent. Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. For $f(x) = k(x - v)^2 + c$ on $[a, b]$, prove that v is the unique minimizer and that every point at distance at least δ from v has value at least $c + k\delta^2$.

Parent in Lean

```
theorem scaled_translated_square_gap
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => k * (x - v)^2 + c) (Set.Icc a b) v ∧
  (fun x : ℝ => k * (x - v)^2 + c) v = c ∧
  (∀ x ∈ Set.Icc a b, δ ≤ |x - v| →
    c + k * δ^2 ≤ k * (x - v)^2 + c) ∧
  (∀ x ∈ Set.Icc a b, k * (x - v)^2 + c = c → x = v)
```

What it contributes. Retains the parent's positive scaled translated quadratic and vertex interval assumptions, while changing the conclusion to a two-point level-set characterization proved through factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the parent's minimum-at-the-vertex claim into a global characterization of all equal quadratic values. Its proof requires factoring a difference of squares and using positivity of the scale, reasoning not supplied by the parent's minimum argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from minimum and equality-at-the-vertex facts to a global characterization of equal quadratic values, requiring difference-of-squares factorization and a new equality case split. The parent's proof does not supply this reflection argument, and the child is not a repeat of the retained midpoint or sublevel results.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from minimum and uniqueness at the vertex to a global characterization of equal quadratic values. Its proof requires factoring the difference of squares and using positivity of k to derive equality or reflection, which the parent's minimum argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2467 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equal_values_scaled_translated_quadratic.extracted_1_1 (a b v c k x y : ℝ) (hk : 0 < k) (hav : a ≤ v)
(hvb : v ≤ b) (hx : x ∈ Set.Icc a b) (hy : y ∈ Set.Icc a b) (heq : k * (x - v) ^ 2 + c = k * (y - v) ^ 2 + c) :
x = y ∨ x + y = 2 * v
```

Read back by a model that saw only the Lean. For real numbers a, b, v, c, k, x , and y , suppose that $k > 0$, $a \leq v$, $v \leq b$, x and y both lie in the closed interval $[a, b]$, and $k(x - v)^2 + c = k(y - v)^2 + c$. Then either $x = y$ or $x + y = 2v$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem equal_values_scaled_translated_quadratic
(a b v c k x y : ℝ)
(hk : 0 < k)

(hk : k * (x - v)^2 + c = k * (y - v)^2 + c) :
x = y ∨ x + y = 2 * v := by
have hdiff : k * ((x - v)^2 - (y - v)^2) = 0 := by
  nlinarith [hk]
have hfactor : k * ((x - y) * (x + y - 2 * v)) = 0 := by
  calc
    k * ((x - y) * (x + y - 2 * v)) = k * ((x - v)^2 - (y - v)^2) := by ring
    _ = 0 := hdiff
have hprod : (x - y) * (x + y - 2 * v) = 0 := by
  exact (mul_eq_zero.mp hfactor).resolve_left (ne_of_gt hk)
rcases mul_eq_zero.mp hprod with hxy | href
· left
  linarith
· right
  linarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.me.mh__efc5a618

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e84ca08787f4be0a

4.449 449. centered_quadratic_strict_growth_characterization

The problem

For any real numbers a and d , if $f(x) = a(x - 7)^2 + d$, then for every real number x , $f(x) > f(7)$ exactly when $a > 0$ and $x \neq 7$.

Lean statement

```
theorem centered_quadratic_strict_growth_characterization
  (a d : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x > f 7 ↔ 0 < a ∧ x ≠ 7
```

Parent. For real numbers a and d with $a \neq 0$, let $f(x) = a(x - 7)^2 + d$. Then $f(x) = f(7)$ holds exactly when $x = 7$.

Parent in Lean

```
theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child strengthens the semantic characterization from equality at the vertex to a strict-inequality classification that also determines the sign of the coefficient, while dropping the nonzero hypothesis. Its proof requires new positivity and square-nonzero reasoning that the parent's equality proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality characterization into a strict-inequality characterization, proving both the necessary positivity of a and the exclusion of the vertex. This requires sign reasoning and does not follow from the parent's nonzero-coefficient argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality at the vertex into a strict-inequality classification and removes the nonzero-coefficient hypothesis. Proving the result requires a new sign analysis showing positivity exactly when $a > 0$ and $x \neq 7$, which the parent's proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2058 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem centered_quadratic_strict_growth_characterization.extracted_1_1 : ∀ (a d : ℝ) (f : ℝ → ℝ),
  (f = fun x => a * (x - 7) ^ 2 + d) → ∀ (x : ℝ), f x > f 7 ↔ 0 < a ∧ x ≠ 7
```

Read back by a model that saw only the Lean. For every real numbers a and d and every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is the function defined by $f(x) = a(x - 7)^2 + d$ for every real x , then for every real x , $f(x) > f(7)$ if and only if $a > 0$ and $x \neq 7$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem centered_quadratic_strict_growth_characterization
  (a d : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x > f 7 ↔ 0 < a ∧ x ≠ 7 := by
  have hfactor : ∀ x : ℝ, f x - f 7 = a * (x - 7)^2 := by
    intro x
    rw [hf]
    ring
  intro x
  constructor
  · intro h
    have hprodpos : 0 < a * (x - 7)^2 := by
      nlinarith [hfactor x]
    have hsqnonneg : 0 ≤ (x - 7)^2 := sq_nonneg (x - 7)
    have hapos : 0 < a := by
      by_contra hnot
      have hanonpos : a ≤ 0 := le_of_not_gt hnot
      have hprodnonpos : a * (x - 7)^2 ≤ 0 :=
        mul_nonpos_of_nonpos_of_nonneg hanonpos hsqnonneg
      linarith
    have hxne : x ≠ 7 := by
      intro hx
      subst x
      exact (lt_irrefl (f 7)) h
    exact ⟨hapos, hxne⟩
  · rintro ⟨hapos, hxne⟩
    have hne : x - 7 ≠ 0 := sub_ne_zero.mpr hxne
    have hsqpos : 0 < (x - 7)^2 := by
      have hmul : 0 < (x - 7) * (x - 7) := mul_self_pos.mpr hne
      simp [pow_two] using hmul
    have hprodpos : 0 < a * (x - 7)^2 := mul_pos hapos hsqpos
    nlinarith [hfactor x]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.mh.mh__865eb777

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4e83bc9182afdcf7

4.450 450. centered_quadratic_equality_characterization

The problem

For real numbers a and d with $a \neq 0$, let $f(x) = a(x - 7)^2 + d$. Then $f(x) = f(7)$ holds exactly when $x = 7$.

Lean statement

```
theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7
```

Parent. For $f(x) = x^2 - 14x + 3$, the equality $f(x) = f(7)$ holds exactly when $x = 7$.

Parent in Lean

```
theorem quadratic_equality_characterization
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  ∀ x, f x = f 7 ↔ x = 7
```

What it contributes. Retains the parent's vertex-equality characterization and completed-square checkpoint, while replacing the fixed leading coefficient with an explicit nonzero parameter.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the leading coefficient and requires using the new nonzero hypothesis to infer that $a \cdot (x-7)^2 = 0$ forces $(x-7)^2 = 0$. The parent's fixed-coefficient square identity and proof do not supply this cancellation argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading coefficient: proving equality at the vertex forces $a \cdot (x-7)^2 = 0$, and the new hypothesis $a \neq 0$ is essential to derive $x = 7$. This reasoning is not supplied by the parent's fixed-coefficient square identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading coefficient and uses the new hypothesis $a \neq 0$ to infer from $a \cdot (x-7)^2 = 0$ that $x = 7$. The parent's fixed square identity and proof do not supply this cancellation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2042 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem centered_quadratic_equality_characterization.extracted_1_1 : ∀ (a d : ℝ),
  a ≠ 0 → ∀ (f : ℝ → ℝ), (f = fun x => a * (x - 7) ^ 2 + d) → ∀ (x : ℝ), f x = f 7 ↔ x = 7
```

Read back by a model that saw only the Lean. *For every real number a and real number d , if a is nonzero, then for every function f from $\mathbb{R}$ to $\mathbb{R}$ such that $f(x) = a(x - 7)^2 + d$ for every real x , and for every real number x , $f(x) = f(7)$ if and only if $x = 7$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7 := by
  have hfactor : ∀ x : ℝ, f x - f 7 = a * (x - 7)^2 := by
    intro x
    rw [hf]
    ring
  intro x
  constructor
  · intro h
    have hprod : a * (x - 7)^2 = 0 := by
      nlinarith [hfactor x]
    have hsq : (x - 7)^2 = 0 := by
      rcases mul_eq_zero.mp hprod with ha0 | hs
      · exact (ha ha0).elim
      · exact hs
    have hxzero : x - 7 = 0 := by
      nlinarith [hsq]
    linarith
  · intro hx
    have hsame : f x = f 7 := by
      rw [hx]
    exact hsame
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.mh__099ead40

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 099ead40ddfbbcf7

4.451 451. quadratic_unique_minimizer

The problem

What value of x will give the minimum value for $f(x) = x^2 - 14x + 3$? Show that x is a minimizer if and only if $x = 7$.

Lean statement

```
theorem quadratic_unique_minimizer
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  ∀ x, IsMinOn f Set.univ x ↔ x = 7
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes a minimum-value claim into a uniqueness characterization: it must prove any minimizer equals 7, using both inequalities and strict quadratic equality. Although it reproves the parent’s minimum as a sublemma, the parent proof alone does not establish the new uniqueness direction.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum-value claim into a genuine uniqueness characterization. Proving the reverse direction requires showing any minimizer has equal value to the known minimizer and then deriving $x = 7$ via polynomial reasoning, which the parent’s proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum-value claim into a genuine uniqueness characterization. Beyond reproving the parent’s minimum, it must compare an arbitrary minimizer with the known one and derive equality, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_unique_minimizer.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 14 * x + 3) → ∀ (x : ℝ), IsMinOn f Set.univ x ↔ x = 7
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is equal to the function defined by $f(x) = x^2 - 14x + 3$ for all real x , then for every real number x , x is a global minimizer of f on $\mathbb{R}$ if and only if $x = 7$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_unique_minimizer  
  (f : ℝ → ℝ)  
  (hf : f = fun x => x^2 - 14 * x + 3) :  
  ∀ x, IsMinOn f Set.univ x ↔ x = 7 := by  
  have hmin : IsMinOn f Set.univ 7 := by  
    rw [hf, isMinOn_univ_iff]  
    intro x  
    nlinarith [sq_nonneg (x - 7)]  
  intro x  
  constructor  
  · intro hx  
    have hupper : f x ≤ f 7 := by  
      rw [isMinOn_univ_iff] at hx  
      exact hx 7  
    have hlower : f 7 ≤ f x := by  
      rw [isMinOn_univ_iff] at hmin  
      exact hmin x  
    rw [hf] at hupper hlower  
    nlinarith  
  · intro hx  
    subst x  
    exact hmin
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh__00ae8c07

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint df28ca854e198b0a

4.452 452. translated_square_quadratic_interval

The problem

Let $a \leq v \leq b$. For the quadratic $f(x) = (x - v)^2 + c$ on $[a, b]$, prove that v is its unique minimizer and that the minimum value is c .

Lean statement

```
theorem translated_square_quadratic_interval
  (a b v c : ℝ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => (x - v)^2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v)^2 + c = c → x = v) ∧
  (fun x : ℝ => (x - v)^2 + c) v = c
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. Generalizes the parent's square-completion argument from a fixed global quadratic to a translated quadratic on an interval, while adding uniqueness and the explicit minimum value.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the vertex and constant, restricts the domain to an interval, and adds a uniqueness conclusion. Its proof requires establishing interval membership and showing equality of the square forces $x = v$, reasoning not supplied by the parent's minimum-only argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and changes the conclusion to a bounded-interval minimum characterization with uniqueness and an explicit minimum value. The parent's fixed global minimization argument does not provide the symbolic translated-square reasoning or the uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent to arbitrary translated squares on a closed interval and adds a unique-minimizer characterization plus an explicit minimum value. Proving uniqueness and interval membership requires reasoning not supplied by the parent's global-minimum argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2400 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_quadratic_interval.extracted_1_1 (a b v c : ℝ) (hav : a ≤ v) (hvb : v ≤ b) :
  IsMinOn (fun x => (x - v) ^ 2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v) ^ 2 + c = c → x = v) ∧ (fun x => (x - v) ^ 2 + c) v = c
```

Read back by a model that saw only the Lean. For real numbers a, b, v , and c , if $a \leq v$ and $v \leq b$, then v is a point where the function $x \mapsto (x - v)^2 + c$ attains its minimum on the closed interval $[a, b]$; moreover, for every $x \in [a, b]$, if $(x - v)^2 + c = c$, then $x = v$; and the function's value at v is c .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_quadratic_interval
  (a b v c : ℝ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => (x - v)^2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v)^2 + c = c → x = v) ∧
  (fun x : ℝ => (x - v)^2 + c) v = c := by
  have hv_mem : v ∈ Set.Icc a b := ⟨hav, hvb⟩
  have hcompletion (x : ℝ) : (x - v)^2 + c = c + (x - v)^2 := by
    ring
  constructor
  · rw [isMinOn_iff]
    intro x hx
    rw [hcompletion v, hcompletion x]
    nlinarith [sq_nonneg (x - v)]
  · constructor
    · intro x hx heq
      have hsq : (x - v)^2 = 0 := by
        nlinarith [heq]
      have hmul : (x - v) * (x - v) = 0 := by
        simpa [pow_two] using hsq
      have hzero : x - v = 0 := by
        rcases mul_eq_zero.mp hmul with h | h
        · exact h
        · exact h
      linarith
    · dsimp
      ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh__e0068f38

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e0068f387c7839a9

4.453 453. weighted_square_remainder_parameterized

The problem

If x minimizes $t^2 - 2at + 3$ over the real numbers and x, y, z, w satisfy the four weighted-square equations with weights determined by 2, 4, 6, 8 and 1, 3, 5, 7, show that $y^2 + z^2 + w^2 = 36 - a^2$.

Lean statement

```
theorem weighted_square_remainder_parameterized (a x y z w : ℝ)
  (hmin : IsMinOn (fun t : ℝ => t ^ 2 - 2 * a * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2
```

Parent. Show that no real numbers x, y, z, w satisfy the four weighted-square equations when x is a minimizer of $t^2 - 14t + 3$.

Parent in Lean

```
theorem weighted_square_infeasible_at_minimizer (x y z w : ℝ)
  (hmin : IsMinOn (fun t : ℝ => t ^ 2 - 14 * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  False
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the minimizer: it must derive the symbolic identity $x = a$ and retain a^2 through the final algebra, whereas the parent only substitutes the fixed value $x = 7$ to derive a contradiction. The conclusion is a nontrivial remainder identity rather than the parent's infeasibility claim.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The constant minimizer 7 is genuinely parameterized to an arbitrary minimizer a , and the contradictory endpoint in the parent is replaced by the symbolic identity $y^2 + z^2 + w^2 = 36 - a^2$. The parent's final infeasibility argument no longer applies; the proof must preserve a^2 symbolically.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2683 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_square_remainder_parameterized.extracted_1_1 (a x y z w : ℝ)
  (hmin : IsMinOn (fun t => t ^ 2 - 2 * a * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2
```

Read back by a model that saw only the Lean. Let a, x, y, z , and w be real numbers. Suppose that x minimizes the function $t \mapsto t^2 - 2at + 3$ over all real numbers, so that for every real t , $x^2 - 2ax + 3 \leq t^2 - 2at + 3$. Suppose also that $x^2/(2^2 - 1) + y^2/(2^2 - 3^2) + z^2/(2^2 - 5^2) + w^2/(2^2 - 7^2) = 1$, $x^2/(4^2 - 1) + y^2/(4^2 - 3^2) + z^2/(4^2 - 5^2) + w^2/(4^2 - 7^2) = 1$, $x^2/(6^2 - 1) + y^2/(6^2 - 3^2) + z^2/(6^2 - 5^2) + w^2/(6^2 - 7^2) = 1$, and $x^2/(8^2 - 1) + y^2/(8^2 - 3^2) + z^2/(8^2 - 5^2) + w^2/(8^2 - 7^2) = 1$. Then $y^2 + z^2 + w^2 = 36 - a^2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem weighted_square_remainder_parameterized (a x y z w : ℝ)
  (hmin : IsMinOn (fun t : ℝ => t ^ 2 - 2 * a * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2 := by
  rw [isMinOn_univ_iff] at hmin
  have hmin' : ∀ t : ℝ, x ^ 2 - 2 * a * x + 3 ≤ t ^ 2 - 2 * a * t + 3 := by
    exact hmin
  have hxa : x = a := by
    have ha' := hmin' a
    nlinarith [sq_nonneg (x - a)]
  norm_num at h₀ h₁ h₂ h₃
  have hnorm : x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
    linarith
  have hxsq : x ^ 2 = a ^ 2 := by
    rw [hxa]
  nlinarith [hnorm, hxsq]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__x__aime_1984_p15__xe.mh__dc319d07

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint dc319d07409b0ac1

4.454 454. parameterized_nested_real_root_normalization

The problem

For real numbers a and b satisfying $0 \leq b$ and $a^2 = 64b^3$, show that $(16(a^2)^{1/3})^{1/3} = 4b^{1/3}$.

Lean statement

```
theorem parameterized_nested_real_root_normalization (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3)
```

Parent. If $a = 8$, what is the value of $(16\sqrt[3]{a^2})^{\frac{1}{3}}$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_114 (a : ℝ) (h₀ : a = 8) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
```

What it contributes. Preserves the parent nested real-radical structure and its cube-root normalization checkpoint, while replacing the fixed value hypothesis with the parameterized relation $a^2 = 64b^3$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the radicand and proves two new real-power identities using the variable b and its nonnegativity. Its proof depends on h , hb , and $rpow$ normalization, whereas the parent only substitutes the fixed value $a = 8$.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the radicand and requires symbolic real-power reasoning: it uses $b \geq 0$ to normalize $(b^3)^{1/3}$, applies multiplicativity of real powers, and proves the outer parameterized normalization. The parent's fixed-value `norm_num` argument does not supply these steps.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the root condition: it must derive the inner cube-root normalization from $64 * b^3$ and then normalize the outer real power symbolically in b . The parent only handles the fixed case $a = 8$ and its constant computation cannot supply these parameterized $rpow$ identities.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1710 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_nested_real_root_normalization.extracted_1_1 (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :  
  (16 * (a ^ 2) ^ (1 / 3)) ^ (1 / 3) = 4 * b ^ (1 / 3)
```

Read back by a model that saw only the Lean. For real numbers a and b , if b is nonnegative and $a^2 = 64b^3$, then $(16 \cdot (a^2)^{1/3})^{1/3} = 4 \cdot b^{1/3}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_nested_real_root_normalization (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :  
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3) := by  
  have hcube : 0 ≤ b ^ 3 := by positivity  
  have hprod : 0 ≤ 64 * b ^ 3 := by positivity  
  have hroot : (b ^ 3) ^ ((1 : ℝ) / 3) = b := by  
    convert Real.pow_rpow_inv_natCast hb (by norm_num : (3 : ℕ) ≠ 0) using 1 <|> norm_num  
  have hinner : (64 * b ^ 3) ^ ((1 : ℝ) / 3) = 4 * b := by  
    rw [Real.mul_rpow (by norm_num) hcube]  
    rw [show (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 by  
      convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num]  
    rw [hroot]  
  have houter : (64 * b) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3) := by  
    rw [Real.mul_rpow (by norm_num) hb]  
    rw [show (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 by  
      convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num]  
    rw [h, hinner]  
  have hscale : 16 * (4 * b) = 64 * b := by ring  
  rw [hscale, houter]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_114__mh__af90cf1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b614b03f4056ea03

4.455 455. radical_difference_eq_zero_iff_zero_or_one

The problem

For a natural number b , characterize when $\sqrt{b^6} - \sqrt[3]{b^6} = 0$.

Lean statement

```
theorem radical_difference_eq_zero_iff_zero_or_one (b : ℕ) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = 0 ↔ b = 0 ∨ b = 1
```

Parent. For a natural number $b \geq 2$, show that $\sqrt{b^6} - \sqrt[3]{b^6} > 0$.

Parent in Lean

```
theorem positive_radical_difference (b : ℕ) (hb : 2 ≤ b) :
  0 < Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes positivity under the hypothesis $b \geq 2$ into a full zero-set characterization over all naturals. It must analyze the factored expression's zero cases, including excluding $b = 0$ and deriving $b = 1$, which the parent's positivity proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes positivity into an exact zero-set characterization, including the new $b=0$ case and a reverse argument that factors the expression and analyzes zero products to force $b=1$. The parent's radical simplifications are reusable, but its positivity argument does not supply this equality-and-uniqueness reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a positivity result under the hypothesis $b \geq 2$ into a full zero-locus characterization for all natural numbers. Its proof requires handling the zero case, factoring the resulting polynomial, and proving both directions, none of which is supplied by the parent's positivity argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2549 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem radical_difference_eq_zero_iff_zero_or_one.extracted_1_1 (b : ℕ) :
  √(↑b ^ 6) - (↑b ^ 6) ^ (1 / 3) = 0 ↔ b = 0 ∨ b = 1
```

Read back by a model that saw only the Lean. For every natural number b , the square root of the sixth power of b minus the one-third power of the sixth power of b equals zero if and only if b equals 0 or b equals 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem radical_difference_eq_zero_iff_zero_or_one (b : ℕ) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = 0 ↔ b = 0 ∨ b = 1 := by
  by_cases hb0 : b = 0
  · subst b
    norm_num
  · have hbpos_nat : 0 < b := Nat.pos_of_ne_zero hb0
    have hbpos : 0 < (b : ℝ) := by
      exact_mod_cast hbpos_nat
    have hsqrt : Real.sqrt ((b : ℝ) ^ 6) = (b : ℝ) ^ 3 := by
      calc
        Real.sqrt ((b : ℝ) ^ 6) = Real.sqrt (((b : ℝ) ^ 3) ^ 2) := by
          congr 1
        ring
        _ = |(b : ℝ) ^ 3| := by rw [Real.sqrt_sq_eq_abs]
        _ = (b : ℝ) ^ 3 := by rw [abs_of_nonneg] <|> positivity
    have hcuberooroot : ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = (b : ℝ) ^ 2 := by
      have hroot := Real.pow_rpow_inv_natCast (x := (b : ℝ) ^ 2) (n := 3)
      (by positivity) (by norm_num)
      convert hroot using 1 <|> norm_num <|> ring
    constructor
    · intro h
      have hpoly : (b : ℝ) ^ 3 - (b : ℝ) ^ 2 = 0 := by
        rw [hsqrt, hcuberooroot] at h
        exact h
      have hfactor : (b : ℝ) ^ 3 - (b : ℝ) ^ 2 =
        (b : ℝ) ^ 2 * ((b : ℝ) - 1) := by ring
      have hprod : (b : ℝ) ^ 2 * ((b : ℝ) - 1) = 0 := by
        rw [← hfactor]
        exact hpoly
      have hsub : (b : ℝ) - 1 = 0 := by
        rcases mul_eq_zero.mp hprod with hsq | hsub
        · exact False.elim ((ne_of_gt (sq_pos_of_pos hbpos)) hsq)
        · exact hsub
      right
      have hreal : (b : ℝ) = 1 := by linarith
      exact_mod_cast hreal
    · intro h
      rcases h with hzero | hone
      · exact False.elim (hb0 hzero)
      · subst b
        norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_208__me.mh.mh__f2108e35

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f2108e35eb4bd027

4.456 456. sqrt_eq_iff_square

The problem

For nonnegative real numbers x and y , show that $\sqrt{x} = y$ if and only if $x = y^2$.

Lean statement

```
theorem sqrt_eq_iff_square : ∀ x y : ℝ, 0 ≤ x → 0 ≤ y → (Real.sqrt x = y ↔ x = y ^ 2) := by
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's single numerical identity to a full characterization of square roots for arbitrary nonnegative real numbers. Proving the biconditional requires reasoning absent from the parent's closed numerical normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single numerical evaluation with a general iff characterization of square roots for arbitrary nonnegative real numbers. Its proof obligation is not supplied by the parent's norm_num computation and does not overlap the retained bounded radical identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's single numerical radical computation into a two-variable iff characterization of square roots under nonnegativity hypotheses. Its proof relies on a general theorem rather than recalling the parent's norm_num calculation, and it introduces a genuinely different proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2496 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sqrt_eq_iff_square.extracted_1_1 : ∀ (x y : ℝ), 0 ≤ x → 0 ≤ y → (√x = y ↔ x = y ^ 2)
```

Read back by a model that saw only the Lean. For all real numbers x and y , if x is nonnegative and y is nonnegative, then the square root of x equals y if and only if x equals y squared.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sqrt_eq_iff_square :  $\forall x y : \mathbb{R}, 0 \leq x \rightarrow 0 \leq y \rightarrow (\text{Real.sqrt } x = y \leftrightarrow x = y^2)$  := by
  intro x y hx hy
  have hcharacterization :  $\text{Real.sqrt } x = y \leftrightarrow x = y^2$  :=
    Real.sqrt_eq_iff_eq_sq hx hy
  exact hcharacterization
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__mh__10933a7a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 10933a7ac7747316

4.457 457. sqrt_sq_of_nonneg

The problem

For a nonnegative real number x , show that $\sqrt{x^2} = x$.

Lean statement

```
theorem sqrt_sq_of_nonneg (x : ℝ) (hx : 0 ≤ x) : Real.sqrt (x ^ 2) = x := by
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a closed numerical identity with a symbolic square-root identity requiring a nonnegativity hypothesis and an absolute-value argument. The parent's numerical normalization proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the numeric radical computation to an arbitrary nonnegative real and requires reasoning through the absolute-value identity, which the parent's normalization proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a fixed numerical computation with a universally quantified square-root identity, requiring symbolic reasoning about absolute values and nonnegativity. The parent's 'norm_num' proof supplies none of this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2481 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sqrt_sq_of_nonneg.extracted_1_1 (x : ℝ) (hx : 0 ≤ x) : √(x ^ 2) = x
```

Read back by a model that saw only the Lean. For a real number x satisfying $0 \leq x$, the square root of x squared equals x .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sqrt_sq_of_nonneg (x : ℝ) (hx : 0 ≤ x) : Real.sqrt (x ^ 2) = x := by
  have h_abs : |x| = x := abs_of_nonneg hx
  calc
    Real.sqrt (x ^ 2) = |x| := by simpa using Real.sqrt_sq_eq_abs x
    _ = x := h_abs
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__mh__fafacabf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fafacabf7dc7f991

4.458 458. mathd_algebra_214_range_characterization

The problem

For real numbers a, b, c and a function $f : \mathbb{R} \rightarrow \mathbb{R}$ of the form $f(x) = ax^2 + bx + c$ for every real x , attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, a real number h is attained by f if and only if $h \geq 3$.

Lean statement

```
theorem mathd_algebra_214_range_characterization
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ h : ℝ, (∃ x : ℝ, f x = h) ↔ 3 ≤ h
```

Parent. For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Parent in Lean

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a pointwise fiber equivalence into a full range characterization, requiring both exclusion of values below the extremum and an explicit existence argument for every $h \geq 3$. The shared derivation of the quadratic's vertex parameters is groundwork, but the existential/surjectivity obligation is not supplied by merely recalling the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fiber equation into a full range characterization, requiring a new existence argument for every $h \geq 3$ (via solving a square-root condition) in addition to deriving the quadratic's orientation. This is substantive reasoning not supplied by the parent's pointwise equivalence proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fiber identity into a full characterization of the range, requiring both the lower-bound argument and a new existence construction for every $h \geq 3$. The parent's pointwise equivalence does not itself supply that proof obligation or its witness construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2092 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_range_characterization.extracted_1.1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
    IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (h : ℝ), (∃ x, f x = h) ↔ 3 ≤ h
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c and every function f from the real numbers to the real numbers, if $f(x) = ax^2 + bx + c$ for every real x , if f has an extremum at $x = 2$ over all real numbers—meaning either $f(x) \leq f(2)$ for every real x or $f(2) \leq f(x)$ for every real x —if $f(2) = 3$, and if $f(4) = 4$, then for every real number h , there exists a real number x such that $f(x) = h$ if and only if $3 \leq h$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_range_characterization
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ h : ℝ, (∃ x : ℝ, f x = h) ↔ 3 ≤ h := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra hn
        have ha' : a = 0 := by nlinarith
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let z : ℝ := 2 - (4 * a + b) / (2 * a)
      have hz : f 2 ≤ f z := by simpa using hmin (Set.mem_univ z)
      rw [h₀ z, h₂] at hz
      have hpoly :
        a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
          -(4 * a + b) ^ 2 / (4 * a) := by
        dsimp [z]
        field_simp [ha_ne]
        ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hz, e2, hpoly]
      have hprod : (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
        field_simp [ha_ne]
      have hd : (4 * a + b) ^ 2 = 0 := by
```

```

      have hp := mul_nonneg hdiff (le_of_lt (by nlinarith : 0 < 4 * a))
      nlinarith [hprod]
    nlinarith
  · have h1 : f 1 ≤ f 2 := by simpa using hmax (Set.mem_univ (1 : ℝ))
    have h3' : f 3 ≤ f 2 := by simpa using hmax (Set.mem_univ (3 : ℝ))
    rw [h0 1, h2] at h1
    rw [h0 3, h2] at h3'
    have e2 := h0 2
    have e4 := h0 4
    rw [h2] at e2
    rw [h3] at e4
    have ha : a < 0 := by
      by_contra hn
      have ha' : a = 0 := by nlinarith
      nlinarith [h1, h3', e2, e4]
    have ha_ne : a ≠ 0 := ne_of_lt ha
    let z : ℝ := 2 - (4 * a + b) / (2 * a)
    have hz : f z ≤ f 2 := by simpa using hmax (Set.mem_univ z)
    rw [h0 z, h2] at hz
    have hpoly :
      a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
      -(4 * a + b) ^ 2 / (4 * a) := by
      dsimp [z]
      field_simp [ha_ne]
      ring
    have hdiff : -(4 * a + b) ^ 2 / (4 * a) ≤ 0 := by
      nlinarith [hz, e2, hpoly]
    have hprod : (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
      field_simp [ha_ne]
    have hd : (4 * a + b) ^ 2 = 0 := by
      have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith : 4 * a < 0))
      nlinarith [hprod]
    nlinarith
  have ha : a = (1 : ℝ) / 4 := by
    have e2 := h0 2
    have e4 := h0 4
    rw [h2] at e2
    rw [h3] at e4
    nlinarith [e2, e4, hb]
  have hc : c = 4 := by
    have e2 := h0 2
    rw [h2] at e2
    nlinarith [e2, ha, hb]
  have hform : ∀ x : ℝ, f x = (x - 2) ^ 2 / 4 + 3 := by
    intro x
    rw [h0 x, hb, ha, hc]
    ring
  intro h
  constructor
  · rintro ⟨x, hx⟩
    rw [hform x] at hx
    nlinarith [sq_nonneg (x - 2)]
  · intro hh
    let x : ℝ := 2 + Real.sqrt (4 * (h - 3))
    have hs : 0 ≤ 4 * (h - 3) := by nlinarith
    have hsqrt : (Real.sqrt (4 * (h - 3))) ^ 2 = 4 * (h - 3) := by
      exact Real.sq_sqrt hs
    refine ⟨x, ?_⟩
    rw [hform x]
    dsimp [x]
    nlinarith

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me.mh__79cf573c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 79cf573c44f2d2bf

4.459 459. mathd_algebra_214_strictMono_right

The problem

For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, whenever $2 \leq x < y$, we have $f(x) < f(y)$.

Lean statement

```
theorem mathd_algebra_214_strictMono_right
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x y : ℝ, 2 ≤ x < y → f x < f y
```

Parent. For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Parent in Lean

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

What it contributes. Reused the parent's extremum-based coefficient recovery and completed-square representation, while changing the final obligation to strict order on the right branch and adding the squared-distance checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fiber equation into strict monotonicity on the right of the vertex. After deriving the quadratic's centered form and positive leading coefficient, it must prove strict growth from $x < y$ and $x, y \geq 2$, which is reasoning absent from the parent's level-set equivalence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to strict monotonicity on the right of the vertex, requiring a new comparison-factorization argument after deriving the vertex and positive leading coefficient. The parent's fiber characterization does not supply this monotonicity proof, and the child is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new argument showing strict growth from the vertex on the right, using the derived sign of the quadratic coefficient and factoring the difference between $f y$ and $f x$. The parent's fiber identity does not provide this monotonicity proof, so recalling the parent would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2090 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_strictMono_right.extracted_1_1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
    IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (x y : ℝ), 2 ≤ x → x < y → f x < f y
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c , and every function f from $\mathbb{R}$ to $\mathbb{R}$, if for every real number x , $f(x) = ax^2 + bx + c$, if 2 is an extremum of f on $\mathbb{R}$, if $f(2) = 3$, and if $f(4) = 4$, then for all real numbers x and y , whenever $2 \leq x$ and $x < y$, we have $f(x) < f(y)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_strictMono_right
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x y : ℝ, 2 ≤ x → x < y → f x < f y := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra ha'
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let z : ℝ := 2 - (4 * a + b) / (2 * a)
      have hz : f 2 ≤ f z := by simpa using hmin (Set.mem_univ z)
      rw [h₀ z, h₂] at hz
      have hpoly :
        a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
          -(4 * a + b) ^ 2 / (4 * a) := by
          dsimp [z]
          field_simp [ha_ne]
          ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hz, e2, hpoly]
      have hprod :
        (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
          field_simp [ha_ne]
      have hsquare : (4 * a + b) ^ 2 = 0 := by
        have hp := mul_nonneg hdiff (le_of_lt (by nlinarith : 0 < 4 * a))
```

```

    nlinarith [hprod]
  nlinarith
  · have h1 : f 1 ≤ f 2 := by simp using hmax (Set.mem_univ (1 : ℝ))
    have h3' : f 3 ≤ f 2 := by simp using hmax (Set.mem_univ (3 : ℝ))
    rw [h₀ 1, h₂] at h1
    rw [h₀ 3, h₂] at h3'
    have e2 := h₀ 2
    have e4 := h₀ 4
    rw [h₂] at e2
    rw [h₃] at e4
    have ha : a < 0 := by
      by_contra ha'
      nlinarith [h1, h3', e2, e4]
    have ha_ne : a ≠ 0 := ne_of_lt ha
    let z : ℝ := 2 - (4 * a + b) / (2 * a)
    have hz : f z ≤ f 2 := by simp using hmax (Set.mem_univ z)
    rw [h₀ z, h₂] at hz
    have hpoly :
      a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
        -(4 * a + b) ^ 2 / (4 * a) := by
      dsimp [z]
      field_simp [ha_ne]
      ring
    have hdiff : -(4 * a + b) ^ 2 / (4 * a) ≤ 0 := by
      nlinarith [hz, e2, hpoly]
    have hprod :
      -(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 := by
      field_simp [ha_ne]
    have hsquare : (4 * a + b) ^ 2 = 0 := by
      have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith : 4 * a < 0))
      nlinarith [hprod]
    nlinarith
  have ha : a = (1 : ℝ) / 4 := by
    have e2 := h₀ 2
    have e4 := h₀ 4
    rw [h₂] at e2
    rw [h₃] at e4
    nlinarith [e2, e4, hb]
  have hc : c = 4 := by
    have e2 := h₀ 2
    rw [h₂] at e2
    nlinarith [e2, ha, hb]
  have hform : ∀ t : ℝ, f t = (t - 2) ^ 2 / 4 + 3 := by
    intro t
    rw [h₀ t, hb, ha, hc]
    ring
  intro x y hx hxy
  have hsum : 0 < y + x - 4 := by
    nlinarith
  have hprod : 0 < (y - x) * (y + x - 4) := by
    exact mul_pos (by linarith) hsum
  have hfactor :
    (y - 2) ^ 2 - (x - 2) ^ 2 = (y - x) * (y + x - 4) := by
    ring
  have hdist : (x - 2) ^ 2 < (y - 2) ^ 2 := by
    nlinarith [hprod, hfactor]
  rw [hform x, hform y]
  nlinarith [hdist]

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me.mh__a39ca298

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fin-
gerprint a39ca298a16f4f50

4.460 460. mathd_algebra_234_mutation

The problem

A sequence of rational numbers satisfies a constant-ratio recurrence and begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$. Show that its n th term is $\frac{27}{125} \left(\frac{5}{3}\right)^n$ for every nonnegative integer n .

Lean statement

```
theorem mathd_algebra_234_mutation
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n, a n = (27 / 125) * (5 / 3) ^ n
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{27}{125} \left(\frac{5}{3}\right)^6$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 27 / 9 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed-index evaluation into a universal closed-form characterization for every n , requiring induction and recurrence transport beyond the parent’s finite three-step calculation. The proof is not a recall of the parent’s final computation, even though it independently derives the same ratio.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed-index evaluation into a universal closed-form theorem. It requires deriving the ratio and proving the formula by induction for every natural index, which the parent’s finite unfolding argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent’s single-index conclusion to a closed-form identity for every natural index. Its proof requires induction and a recurrence step, which the parent’s finite propagation argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_234_mutation.extracted_1_1 : ∀ (a : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) →
    a 0 = 27 / 125 → a 1 = 9 / 25 → a 2 = 3 / 5 → ∀ (n : ℕ), a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. For every sequence of rational numbers a indexed by the natural numbers and every rational number d , if $a(n+1) = a(n) \cdot d$ for every natural number n , $a(0) = 27/125$, $a(1) = 9/25$, and $a(2) = 3/5$, then for every natural number n , $a(n) = (27/125) \cdot (5/3)^n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_234_mutation
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n, a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  intro n
  induction n with
  | zero =>
    norm_num [h₁]
  | succ n ih =>
    calc
      a (Nat.succ n) = a (n + 1) := by rfl
      _ = a n * d := h₀ n
      _ = ((27 / 125) * (5 / 3) ^ n) * (5 / 3) := by rw [ih, hd]
      _ = (27 / 125) * (5 / 3) ^ (Nat.succ n) := by rw [pow_succ]; ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__mh__ff222c9f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9b5e4a16bef34ba4

4.461 461. reciprocal_solution_map_injective

The problem

Let $a \neq 0$. Show that the map sending each real number $b \neq 1$ to the unique nonzero solution of $a + \frac{1}{x} = \frac{b}{x}$ is injective.

Lean statement

```
theorem reciprocal_solution_map_injective (a : ℝ) (ha : a ≠ 0) :
  ∃ f : {b : ℝ // b ≠ 1} → ℝ,
    (∀ b, f b = (b.1 - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b.1 / f b) ∧
    (∀ b x, x ≠ 0 ∧ a + 1 / x = b.1 / x → x = f b) ∧
    Function.Injective f
```

Parent. Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Parent in Lean

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

What it contributes. Reuses the parent's reciprocal-equation solution formula, nonzero-witness argument, and denominator-clearing uniqueness step, while promoting them to a domain-level injective map.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new global claim: the solution assignment over all right-hand sides is injective. Proving injectivity requires comparing two solution values and recovering equality of their right-hand sides, which is not supplied by the parent's pointwise uniqueness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child promotes the scalar formula to a function on all right-hand sides other than 1 and additionally proves that this solution map is injective. Injectivity requires a new argument comparing two outputs and using $a \neq 0$, which is not supplied by the parent's per-instance uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child promotes the scalar solution formula to a function over all right-hand sides with $b \neq 1$ and additionally proves that this function is injective. Injectivity requires a new cancellation argument across two parameters, which is not supplied by the parent's pointwise uniqueness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2469 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_solution_map_injective.extracted_1_1 (a : ℝ) (ha : a ≠ 0) :
  ∃ f,
    (∀ (b : { b // b ≠ 1 }), f b = (↑b - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = ↑b / f b) ∧
    (∀ (b : { b // b ≠ 1 }) (x : ℝ), x ≠ 0 ∧ a + 1 / x = ↑b / x → x = f b) ∧ Function.Injective f
```

Read back by a model that saw only the Lean. *For every real number a with $a \neq 0$, there exists an injective function f from the set of real numbers b satisfying $b \neq 1$ to $\mathbb{R}$ such that, for every such b , $f(b) = (b - 1)/a$, $f(b) \neq 0$, and $a + 1/f(b) = b/f(b)$; moreover, for every such b and every real number x , if $x \neq 0$ and $a + 1/x = b/x$, then $x = f(b)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_solution_map_injective (a : ℝ) (ha : a ≠ 0) :
  ∃ f : {b : ℝ // b ≠ 1} → ℝ,
    (∀ b, f b = (b.1 - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b.1 / f b) ∧
    (∀ b x, x ≠ 0 ∧ a + 1 / x = b.1 / x → x = f b) ∧
    Function.Injective f := by
  let f : {b : ℝ // b ≠ 1} → ℝ := fun b => (b.1 - 1) / a
  have hformula : ∀ b, f b = (b.1 - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b.1 / f b := by
    intro b
    have hnum : b.1 - 1 ≠ 0 := sub_ne_zero.mpr b.2
    have hnonzero : (b.1 - 1) / a ≠ 0 := div_ne_zero hnum ha
    refine ⟨rfl, hnonzero, ?_⟩
    dsimp [f]
    field_simp [hnonzero, ha]
    ring
  have hunique : ∀ b x, x ≠ 0 ∧ a + 1 / x = b.1 / x → x = f b := by
    intro b x hx
    rcases hx with ⟨hx0, hxeq⟩
    have hxeq' := hxeq
    field_simp [hx0] at hxeq'
    have htarget : x = (b.1 - 1) / a := by
      field_simp [ha]
      nlinarith [hxeq']
    simpa [f] using htarget
  have hinjective : Function.Injective f := by
    intro b c hbc
    apply Subtype.ext
    have hquot : (b.1 - 1) / a = (c.1 - 1) / a := by
      simpa [f] using hbc
    have hquot' := hquot
    field_simp [ha] at hquot'
    nlinarith [hquot']
  exact ⟨f, hformula, hunique, hinjective⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me.mh__78561f8e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 78561f8ebf2706a6

4.462 462. reciprocal_equation_full_characterization

The problem

For real numbers a and b , the equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $a \neq 0$ and $b \neq 1$. In that case, the solution is $\frac{b-1}{a}$.

Lean statement

```
theorem reciprocal_equation_full_characterization (a b : ℝ) :
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Parent. Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Parent in Lean

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the hypothesis $a \neq 0$ and must prove it from unique existence, using a genuinely new contradiction via the distinct solutions x and $-x$ when $a=0$. It also establishes the full necessary-and-sufficient characterization, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the parent's hypothesis that $a \neq 0$ and proves it as a necessary condition from uniqueness, using a genuinely new x versus $-x$ argument. It also characterizes existence and uniqueness exactly by $a \neq 0$ and $b \neq 1$, so the parent's proof cannot simply be recalled.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the parent's fixed hypothesis $a \neq 0$ and proves it is necessary for existence and uniqueness, using a new $-x$ counterargument when $a = 0$. This full parameter characterization requires reasoning not supplied by the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2484 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equation_full_characterization.extracted_1_1 (a b : ℝ) :
  ((∃! x, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x, x ≠ 0 ∧ a + 1 / x = b / x) → ∀ (x : ℝ), x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Read back by a model that saw only the Lean. For all real numbers a and b , the following two assertions both hold: (1) there exists exactly one real number x such that x is nonzero and $a + 1/x = b/x$ if and only if $a \neq 0$ and $b \neq 1$; and (2) if there exists exactly one real number x such that x is nonzero and $a + 1/x = b/x$, then every real number x satisfying $x \neq 0$ and $a + 1/x = b/x$ is equal to $(b - 1)/a$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_equation_full_characterization (a b : ℝ) :
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
  have hchar :
    (∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1 := by
    constructor
    · rintro ⟨x, ⟨hx0, hxeq⟩, huniq⟩
      have hxeq' := hxeq
      field_simp [hx0] at hxeq'
      have ha : a ≠ 0 := by
        intro ha0
        have hb1 : b = 1 := by
          have ht := hxeq'
          rw [ha0] at ht
          nlinarith
        have hnexx : -x ≠ 0 := neg_ne_zero.mpr hx0
        have hnegEq : a + 1 / (-x) = b / (-x) := by
          simp [ha0, hb1]
        have heqneg := huniq (-x) ⟨hnexx, hnegEq⟩
        have hxzero : x = 0 := by
          nlinarith [heqneg]
        exact hx0 hxzero
      have hb : b ≠ 1 := by
        intro hb1
        have ht := hxeq'
        rw [hb1] at ht
        have hax : a * x = 0 := by
          nlinarith [ht]
        exact ha ((mul_eq_zero.mpr hax).resolve_right hx0)
      exact ⟨ha, hb⟩
    · rintro ⟨ha, hb⟩
      have hnum : b - 1 ≠ 0 := sub_ne_zero.mpr hb
      have hx0 : (b - 1) / a ≠ 0 := div_ne_zero hnum ha
      have heq : a + 1 / ((b - 1) / a) = b / ((b - 1) / a) := by
        field_simp [hx0, ha]
      ring
      refine ⟨(b - 1) / a, ⟨hx0, heq⟩, ?_⟩
      intro y hy
      rcases hy with ⟨hy0, hyeq⟩
      have hyeq' := hyeq
      field_simp [hy0] at hyeq'
      field_simp [ha]
      nlinarith [hyeq']
  have hformula :
    (∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a := by
```



```

intro hex x hx
have hab := (hchar.mp hex).1
rcases hx with ⟨hx0, hxeq⟩
have hxeq' := hxeq
field_simp [hx0] at hxeq'
field_simp [hab]
nlinarith [hxeq']
exact ⟨hchar, hformula⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me.mh__91c74945

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 91c74945784f58fd

4.463 463. affine_reciprocal_solution

The problem

For real numbers a, b, c , and x with $c \neq 0$, if $c + \frac{a}{x} = \frac{b}{x}$, show that $x = \frac{b-a}{c}$.

Lean statement

```
theorem affine_reciprocal_solution (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) : x = (b - a) / c
```

Parent. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal equation: it proves the symbolic solution $x = (b-a)/c$ for arbitrary real a, b, c with $c \neq 0$. The proof must establish $x \neq 0$ from the generalized hypotheses and perform symbolic field algebra, so memorizing the parent's numerical result is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation: arbitrary coefficients a, b , and nonzero c must be handled, including deriving $x \neq 0$ and solving symbolically. The parent's fixed constants and final linear arithmetic do not supply this generalized argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation: it must derive $x \neq 0$ from symbolic $c \neq 0$ and then solve for the general expression $(b-a)/c$. The parent's numerical algebra does not supply this generalized argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1831 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_reciprocal_solution.extracted_1_1 (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) :
  x = (b - a) / c
```

Read back by a model that saw only the Lean. Let a, b, c , and x be real numbers. Assume that c is nonzero and that $c + a/x = b/x$. Then $x = (b - a)/c$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem affine_reciprocal_solution (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) : x = (b - a) / c := by
  have hx : x ≠ 0 := by
    intro hx
    subst x
    have hc0 : c = 0 := by
      simp using h
    exact hc hc0
  field_simp [hx] at h
  have hmul : c * x = b - a := by
    linarith [h]
  calc
    x = (c * x) / c := by
      field_simp [hc]
    _ = (b - a) / c := by rw [hmul]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__mh__9a5ed105

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6d0f241ac952258b

4.464 464. quadratic_discriminant_range

The problem

Show that the feasible values of c form exactly the interval $(-\infty, \frac{25}{4a}]$ when $a > 0$.

Lean statement

```
theorem quadratic_discriminant_range (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 * a))
```

Parent. For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from identifying a greatest element to characterizing the entire set as an interval. Its reverse inclusion proves a new implication from the upper bound back to the discriminant inequality, which the parent's proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an extremum claim into an exact set characterization. Besides proving the parent's upper-bound direction, it must prove the converse that every c below the bound satisfies the discriminant inequality, which is a new proof obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-element claim into an exact characterization of the entire feasible range, requiring both inclusions and a reverse implication. The parent's proof supplies only the upper-bound direction and cannot be recalled to complete the set equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2603 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_discriminant_range.extracted_1_1 (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 * a))
```

Read back by a model that saw only the Lean. For a real number a with $0 < a$, and a set S of real numbers satisfying $S = \{c \in \mathbb{R} \mid 0 \leq 25 - 4ac\}$, we have $S = \{c \in \mathbb{R} \mid c \leq 25/(4a)\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_discriminant_range (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 *
  a)) := by
  rw [hS]
  ext c
  simp only [Set.mem_setOf_eq, Set.mem_Iic]
  have hpos : 0 < 4 * a := by
    nlinarith
  constructor
  · intro hc
    apply (le_div_iff₀ hpos).2
    nlinarith [hc]
  · intro hc
    have hmul : c * (4 * a) ≤ 25 := (le_div_iff₀ hpos).1 hc
    nlinarith [hmul]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms.mh__a2cdcf5f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2cdcf5f69651759

4.465 465. quadratic_solution_range

The problem

Describe exactly which real numbers c make the equation $2x^2 + 5x + c = 0$ have a real solution.

Lean statement

```
theorem quadratic_solution_range :
  ∀ c : ℝ, c ∈ {d : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ (25 / 8 : ℝ) := by
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x:ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the greatest-element conclusion into a full characterization of the quadratic’s solution range. Its reverse implication requires constructing a real root via a square root, reasoning not supplied by the parent’s upper-bound-and-attainment proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a greatest element to characterizing the entire solution range. Its reverse direction requires constructing a real root via a square root, reasoning not supplied by the parent’s maximum proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying the greatest element to characterizing the entire solution set via an iff. Its reverse direction requires constructing a real root using a square root, which is genuinely new reasoning beyond the parent’s upper-bound and witness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2596 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_solution_range.extracted_1_1 : ∀ (c : ℝ),  
  c ∈ {d | ∃ x, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ 25 / 8
```

Read back by a model that saw only the Lean. *For every real number c , there exists a real number x satisfying $2x^2 + 5x + c = 0$ if and only if $c \leq 25/8$.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_solution_range :  
  ∀ c : ℝ, c ∈ {d : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ (25 / 8 : ℝ) := by  
  intro c  
  change (∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0) ↔ c ≤ (25 / 8 : ℝ)  
  constructor  
  · rintro ⟨x, hx⟩  
    have hsq : 0 ≤ (4 * x + 5) ^ 2 := sq_nonneg (4 * x + 5)  
    nlinarith  
  · intro hc  
    have hd : 0 ≤ 25 - 8 * c := by  
      nlinarith  
    let s : ℝ := Real.sqrt (25 - 8 * c)  
    have hs : s ^ 2 = 25 - 8 * c := by  
      dsimp [s]  
      exact Real.sq_sqrt hd  
    refine ⟨(-5 + s) / 4, ?_⟩  
    nlinarith [hs]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__mh__22408ea6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 22408ea608393a55

4.466 466. quadratic_root_bound_and_equality

The problem

Show that if $2x^2 + 5x + c = 0$ has a real solution, then $c \leq \frac{25}{8}$, with equality exactly when $x = -\frac{5}{4}$.

Lean statement

```
theorem quadratic_root_bound_and_equality :
  ∀ (x c : ℝ), 2 * x ^ 2 + 5 * x + c = 0 →
    c ≤ (25 / 8 : ℝ) ∧ (c = (25 / 8 : ℝ) ↔ x = -5 / 4)
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x:ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-element statement about a set to a universal root characterization, adding the nontrivial equality case $c = 25/8$ iff $x = -5/4$. The parent's bound argument helps but does not supply this equality characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a set maximum to a root-wise bound plus an equality characterization. Proving equality forces $x = -5/4$ and proving the converse requires substituting that root, which is new reasoning beyond the parent's existence-and-bound argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from establishing a set maximum to characterizing equality for every quadratic root. Proving that attaining the bound forces $x = -5/4$ is additional reasoning beyond the parent's existence-and-upper-bound argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2612 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_root_bound_and_equality.extracted_1_1 : ∀ (x c : ℝ),  
  2 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / 8 ∧ (c = 25 / 8 ↔ x = -5 / 4)
```

Read back by a model that saw only the Lean. For every real numbers x and c , if $2x^2 + 5x + c = 0$, then $c \leq 25/8$ and ($c = 25/8$ if and only if $x = -5/4$).

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_root_bound_and_equality :  
  ∀ (x c : ℝ), 2 * x ^ 2 + 5 * x + c = 0 →  
    c ≤ (25 / 8 : ℝ) ∧ (c = (25 / 8 : ℝ) ↔ x = -5 / 4) := by  
  intro x c hx  
  have hsq : 0 ≤ (4 * x + 5) ^ 2 := sq_nonneg (4 * x + 5)  
  have hbound : c ≤ (25 / 8 : ℝ) := by  
    nlinarith [hsq]  
  constructor  
  · exact hbound  
  · constructor  
    · intro hc  
      nlinarith [hsq]  
    · intro hxval  
      subst x  
      norm_num at hx  
      linarith
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__mh__3871bd82

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 3871bd8251a48c27

4.467 467. generalized_abs_interval

The problem

For real numbers a, c and $r > 0$, solve $(1/r)|c + 2a| < 1$ for a in interval notation.

Lean statement

```
theorem generalized_abs_interval (a c r : ℝ) (hr : 0 < r) :  
  (1 / r * abs (c + 2 * a) < 1) ↔  
    a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2)
```

Parent. Solve for a : $\frac{1}{5}|9 + 2a| < 1$. Express your answer in interval notation. Show that it is $(-7, -2)$.

Parent in Lean

```
theorem mathd_algebra_327  
  (a : ℝ) :  
  (1 / 5 * abs (9 + 2 * a) < 1) ↔ a ∈ Set.Ioo (-7:ℝ) (-2) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the center and radius and requires using the essential positivity hypothesis to transport an inequality through division. The parent’s fixed-constant linear arithmetic proof does not directly supply this generalized argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the center and radius: proving the equivalence requires using the hypothesis $0 < r$ to multiply or divide inequalities safely. The parent’s fixed-constant ‘nlinarith’ argument does not directly supply this variable-denominator reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the center term and radius, requiring positivity-aware division reasoning and preserving symbolic dependence on c and r . The parent’s fixed-radius linear arithmetic proof does not directly supply this generalized obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2622 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_abs_interval.extracted_1_1 (a c r : ℝ) (hr : 0 < r) :  
  1 / r * |c + 2 * a| < 1 ↔ a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2)
```

Read back by a model that saw only the Lean. For real numbers a , c , and r with $r > 0$, the inequality $(1/r) \cdot |c + 2a| < 1$ holds if and only if a lies in the open interval $((-r - c)/2, (r - c)/2)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem generalized_abs_interval (a c r : ℝ) (hr : 0 < r) :  
  (1 / r * abs (c + 2 * a) < 1) ↔  
    a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2) := by  
  constructor  
  · intro h  
    have hdiv : abs (c + 2 * a) / r < 1 := by  
      simpa [div_eq_mul_inv, mul_comm] using h  
    have habs : abs (c + 2 * a) < r := by  
      have h' := (div_lt_iff₀ hr).mp hdiv  
      nlinarith  
    rw [abs_lt] at habs  
    exact ⟨by nlinarith [habs.1], by nlinarith [habs.2]⟩  
  · intro h  
    have hbounds : -r < c + 2 * a ∧ c + 2 * a < r := by  
      exact ⟨by nlinarith [h.1], by nlinarith [h.2]⟩  
    have habs : abs (c + 2 * a) < r := by  
      exact (abs_lt).2 hbounds  
    have hdiv : abs (c + 2 * a) / r < 1 := by  
      apply (div_lt_iff₀ hr).2  
      nlinarith  
    simpa [div_eq_mul_inv, mul_comm] using hdiv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_327__mh__12b4c4db

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 12b4c4dbb57a5ecc

4.468 468. bounded_quadratic_cutoff_classification

The problem

For every natural cutoff $c \leq 17$, the finite set of positive natural numbers whose quadratic expression $x^2 + 4x + 4$ is below c is empty when $c \leq 9$, is $\{1\}$ when $c \leq 16$, and is $\{1, 2\}$ otherwise.

Lean statement

```
theorem bounded_quadratic_cutoff_classification (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :  
  S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2}
```

Parent. For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Parent in Lean

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :  
  S.card = if c ≤ 9 then 0 else 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bound is genuinely strengthened from 10 to 17, requiring a new cutoff analysis: proving that $x=2$ enters at $c=17$ while all $x \geq 3$ remain excluded. The added $\{1,2\}$ classification and extra case split impose mathematics absent from the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child raises the bound from 10 to 17 and must identify the new admissible case $x = 2$ at $c = 17$, requiring a genuinely new cutoff and case analysis beyond the parent's 0/1 cardinality argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the bound to 17 and classifies the entire finset, requiring a new case and proof that 2 is included while larger values are excluded. The parent only handles the $c \leq 10$ regime and cannot supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1816 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_cutoff_classification.extracted_1_1 (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ)  
  (h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
```

```
S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2}
```

Read back by a model that saw only the Lean. For a natural number c with $c \leq 17$, and a finite set S of natural numbers such that, for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < c$, we have $S = \emptyset$ if $c \leq 9$, $S = \{1\}$ if $c \leq 16$ but $c > 9$, and $S = \{1, 2\}$ if $c > 16$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_cutoff_classification (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2} := by
  by_cases h9 : c ≤ 9
  · have hS : S = ∅ := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hsquare : 1 ≤ x ^ 2 := by
          simpa [pow_two] using Nat.mul_le_mul hx hx
        omega
      · intro hx
        simp at hx
  rw [hS]
  simp [h9]
  · by_cases h16 : c ≤ 16
    · have hS : S = {1} := by
        ext x
        rw [h₀]
        constructor
        · rintro ⟨hx, hineq⟩
          have hxle : x ≤ 1 := by
            by_contra h
            have hx2 : 2 ≤ x := by omega
            have hsquare : 4 ≤ x ^ 2 := by
              simpa [pow_two] using Nat.mul_le_mul hx2 hx2
            omega
          have hxone : x = 1 := by omega
          simpa [hxone]
        · intro hx
          have hxone : x = 1 := by simpa using hx
          subst x
          omega
      rw [hS]
      simp [h9, h16]
    · have hc17 : c = 17 := by omega
      have hS : S = {1, 2} := by
        ext x
        rw [h₀]
        constructor
        · rintro ⟨hx, hineq⟩
          have hxle : x ≤ 2 := by
            by_contra h
            have hx3 : 3 ≤ x := by omega
            have hsquare : 9 ≤ x ^ 2 := by
              simpa [pow_two] using Nat.mul_le_mul hx3 hx3
            omega
          have hxcases : x = 1 ∨ x = 2 := by omega
          rcases hxcases with rfl | rfl <|> simp
        · intro hx
          simp at hx
          rcases hx with rfl | rfl <|> norm_num [hc17]
      rw [hS]
      simp [h9, h16]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_405__me.mh__cb2846cc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3104249434c0c327

4.469 469. quadratic_minimum_general

The problem

For real numbers a and b , the minimum value of $y = (x - a)^2 + b$ is b .

Lean statement

```
theorem quadratic_minimum_general
  (a b : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - a) ^ 2 + b) :
  IsLeast (Set.range f) b
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both the vertex and minimum value, requiring a symbolic witness $x = a$ and a general nonnegativity argument. The parent's fixed constants 3 and 4 do not directly supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the minimizer and minimum, requiring a symbolic witness and algebraic verification rather than the parent's fixed numerical calculation. Its lower-bound argument must also remain valid for arbitrary real parameters, so this is not recall or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the vertex location and minimum value, so the parent's fixed numerical witness and normalization no longer suffice; symbolic algebra is required to establish the new minimum.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2682 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_minimum_general.extracted_1_1 (a b : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - a) ^ 2 + b) :  
  IsLeast (Set.range f) b
```

Read back by a model that saw only the Lean. For real numbers a and b and a function f from the real numbers to the real numbers, suppose that $f(x) = (x - a)^2 + b$ for every real number x . Then b is the least element of the range of f : b belongs to the range of f , and for every real number y in the range of f , $b \leq y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_minimum_general  
  (a b : ℝ)  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - a) ^ 2 + b) :  
  IsLeast (Set.range f) b := by  
  subst f  
  constructor  
  · refine ⟨a, ?_⟩  
    ring  
  · rintro y ⟨x, rfl⟩  
    nlinarith [sq_nonneg (x - a)]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__mh__9adf79c2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9adf79c2a7ecc20b

4.470 470. translated_square_range

The problem

For any real number c , the range of $f(x) = (x - c)^2 + 4$ is exactly the set of real numbers at least 4.

Lean statement

```
theorem translated_square_range
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  Set.range f = Set.Ici 4
```

Parent. For any real number c , the least value of $(x - c)^2 + 4$ over all real x is 4.

Parent in Lean

```
theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from attaining a lower bound to characterizing the entire range, requiring a surjectivity argument using square roots for every value at least 4. The parent's witness-and-lower-bound proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a minimum value to characterizing the entire range, requiring a new surjectivity argument using a square root. The parent's witness-and-lower-bound proof does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving a minimum to characterizing the entire range. Its reverse inclusion requires constructing a preimage using a square root, which is not supplied by the parent's minimum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2720 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_range.extracted_1_1 (c : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - c) ^ 2 + 4) :  
  Set.range f = Set.Ici 4
```

Read back by a model that saw only the Lean. For a real number c and a function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = (x - c)^2 + 4$ for every real number x , then the range of f is the set of all real numbers greater than or equal to 4.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_range  
  (c : ℝ)  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - c) ^ 2 + 4) :  
  Set.range f = Set.Ici 4 := by  
  subst f  
  ext y  
  constructor  
  · rintro ⟨x, rfl⟩  
    change 4 ≤ (x - c) ^ 2 + 4  
    nlinarith [sq_nonneg (x - c)]  
  · intro hy  
    change 4 ≤ y at hy  
    have hnonneg : 0 ≤ y - 4 := by linarith  
    have hsqrt : (Real.sqrt (y - 4)) ^ 2 = y - 4 := by  
      simp using Real.sq_sqrt hnonneg  
    refine ⟨c + Real.sqrt (y - 4), ?_⟩  
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.mh__07e632bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 07e632bf03d13fb5

4.471 471. bounded_translated_square_unique_min

The problem

Let $a > 0$ and let $l \leq v \leq u$. For $f(x) = a(x - v)^2 + c$, show that the minimum of f on $[l, u]$ is $f(v)$ and that it is attained uniquely at $x = v$.

Lean statement

```
theorem bounded_translated_square_unique_min
  (f : ℝ → ℝ) (a c l u v : ℝ)
  (ha : 0 < a) (hlv : l ≤ v) (hvu : v ≤ u)
  (hf : f = fun x ↦ a * (x - v) ^ 2 + c) :
  IsLeast {y : ℝ | ∃ x : ℝ, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧
  ∃! x : ℝ, l ≤ x ∧ x ≤ u ∧ f x = f v
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Uses the parent's completed-square nonnegativity argument for the lower-bound component, generalized to arbitrary center, coefficient, level, and an interval containing the vertex.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the quadratic with symbolic coefficient, center, offset, and interval, and adds a uniqueness characterization. Its proof requires positivity-based equality reasoning to show the sole minimizer is v , which the parent's fixed global-minimum proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds an essential bounded-interval generalization and proves uniqueness of the minimizer using the positive coefficient and equality of squares. The parent only establishes a global minimum for one fixed quadratic and supplies neither parameterization nor uniqueness reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and interval, then proves a new uniqueness characterization of the minimizer. The parent's global minimum proof does not supply the interval-bound hypotheses or the argument that equality forces $x = v$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2686 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_translated_square_unique_min.extracted_1_1 (f : ℝ → ℝ) (a c l u v : ℝ) (ha : 0 < a) (hlv : l ≤ v)
(hvu : v ≤ u) (hf : f = fun x => a * (x - v) ^ 2 + c) :
  IsLeast {y | ∃ x, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧ ∃! x, l ≤ x ∧ x ≤ u ∧ f x = f v
```

Read back by a model that saw only the Lean. Let f be a real-valued function on the real numbers, and let a, c, l, u , and v be real numbers. Assume that $a > 0$, $l \leq v$, $v \leq u$, and for every real x , $f(x) = a(x - v)^2 + c$. Then $f(v)$ is the least element of the set of all real numbers y for which there exists an x with $l \leq x \leq u$ and $y = f(x)$. Moreover, there exists a unique real number x such that $l \leq x \leq u$ and $f(x) = f(v)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_translated_square_unique_min
  (f : ℝ → ℝ) (a c l u v : ℝ)
  (ha : 0 < a) (hlv : l ≤ v) (hvu : v ≤ u)
  (hf : f = fun x => a * (x - v) ^ 2 + c) :
  IsLeast {y : ℝ | ∃ x : ℝ, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧
  ∃! x : ℝ, l ≤ x ∧ x ≤ u ∧ f x = f v := by
  subst f
  constructor
  · constructor
    · exact ⟨v, hlv, hvu, rfl⟩
    · rintro y ⟨x, hxL, hxU, rfl⟩
      have hbound : 0 ≤ (x - v) ^ 2 := sq_nonneg (x - v)
      nlinarith
  · refine ⟨v, ⟨hlv, hvu, rfl⟩, ?_⟩
    intro y hy
    have hvalue : a * (y - v) ^ 2 + c = a * (v - v) ^ 2 + c := by
      simpa using hy.2.2
    have hzero : (y - v) ^ 2 = 0 := by
      nlinarith
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.mh__60cdc9ff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 60cdc9ff998060d5

4.472 472. real_rpow_exponent_characterization

The problem

For a real base $b > 1$ and real exponents x and z , prove that $b^x = b^z$ if and only if $x = z$.

Lean statement

```
theorem real_rpow_exponent_characterization (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z
```

Parent. For a real base greater than 1, any two exponents in the interval $[0,10]$ that produce the same positive value are equal.

Parent in Lean

```
theorem bounded_power_exponent_unique (b y x z : ℝ) (hb : 1 < b) (hy : 0 < y) (hx : 0 ≤ x ∧ x ≤ 10) (hz : 0 ≤ z ∧ z ≤ 10) (hxp : b ^ x = y) (hzy : b ^ z = y) : x = z
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the boundedness and shared-value hypotheses and proves a global injectivity characterization of real exponentiation, including the reverse implication. This is a genuine generalization, and the parent's two logarithm-normalization arguments do not directly supply the child's iff proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the boundedness, positivity, and shared-value hypotheses and proves global injectivity of real exponentiation for every base $b > 1$. This is a genuine strengthening requiring a different injectivity lemma, not supplied by the parent's bounded logarithm argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bounded uniqueness result to global injectivity of real exponentiation, removing the shared-value and exponent-bound hypotheses. Its proof uses the dedicated injectivity theorem and establishes positivity and non-unit conditions, so the parent's logarithm-based argument does not supply it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem real_rpow_exponent_characterization.extracted_1_1 (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z
```

Read back by a model that saw only the Lean. For real numbers b , x , and z , if b is greater than 1, then the real power of b with exponent x equals the real power of b with exponent z if and only if x equals z .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem real_rpow_exponent_characterization (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z := by
  have hbpos : 0 < b := by
    linarith
  have hbne : b ≠ 1 := by
    linarith
  exact Real.rpow_right_inj hbpos hbne
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_484__me.me.mh__86bdc8a3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint d42c1ea548d432a5

4.473 473. positive_integer_is_gcd_of_40_iff_dvd

The problem

For every positive integer y , the greatest common divisor of 40 and y equals y exactly when y divides 40.

Lean statement

```
theorem positive_integer_is_gcd_of_40_iff_dvd
  (y : ℤ)
  (hy : 0 < y) :
  ((Int.gcd 40 y : ℤ) = y ↔ y ∣ 40)
```

Parent. There is no positive integer y whose greatest common divisor with 40 is greater than y and whose least common multiple with 40 equals the product of y and that greatest common divisor.

Parent in Lean

```
theorem gcd_strictly_exceeds_candidate_impossible
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) > y)
  :
  False
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine gcd characterization, requiring divisibility in both directions and the greatest-common-divisor universal property. The parent only derives the upper bound $\text{gcd}(40, y) \leq y$ and does not supply the converse equality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an impossibility bound into a biconditional characterization: proving the reverse direction requires the gcd greatest-divisor theorem and the new hypothesis that y divides 40. The parent's proof of $\text{gcd}(40, y) \leq y$ does not establish this characterization or its converse.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a genuine characterization: equality of the gcd with positive y is equivalent to y dividing 40. Its reverse direction requires the gcd greatestness theorem and divisibility reasoning absent from the parent's contradiction proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2550 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_integer_is_gcd_of_40_iff_dvd.extracted_1_1 (y : ℤ) (hy : 0 < y) : ↑(Int.gcd 40 y) = y ↔ y ∣ 40
```

Read back by a model that saw only the Lean. *For an integer y with $0 < y$, the greatest common divisor of 40 and y , regarded as an integer, equals y if and only if y divides 40.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem positive_integer_is_gcd_of_40_iff_dvd
  (y : ℤ)
  (hy : 0 < y) :
  ((Int.gcd 40 y : ℤ) = y ↔ y ∣ 40) := by
  constructor
  · intro h_eq
    have h_gcd_dvd : (Int.gcd 40 y : ℤ) ∣ 40 :=
      Int.gcd_dvd_left 40 y
    rw [h_eq] at h_gcd_dvd
    exact h_gcd_dvd
  · intro h_div
    have h_nonneg : 0 ≤ y := by
      omega
    have h_greatest : y = (Int.gcd 40 y : ℤ) := by
      apply Int.gcd_greatest h_nonneg h_div (dvd_refl y)
    intro e _ h_e_y
    exact h_e_y
    exact h_greatest.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me.mh__d2494ad1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint d2494ad1427013e1

4.474 474. bounded_anchor_gcd_lcm_characterization

The problem

For an integer anchor a with $1 \leq a \leq 16$, consider positive integers b whose greatest common divisor and least common multiple with a are both a . Prove that this set has least element 8 exactly when $a = 8$.

Lean statement

```
theorem bounded_anchor_gcd_lcm_characterization
  (a : ℕ)
  :
  IsLeast {b : ℕ | 0 < b ∧ Nat.gcd b a = a ∧ Nat.lcm b a = a} 8 ↔ a = 8
```

Parent. The greatest common divisor of two integers is $(x + 3)$ and their least common multiple is $x(x + 3)$, where x is a positive integer. If one of the integers is 40, what is the smallest possible value of the other one? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_126
  (x a : ℤ)
  (h₀ : 0 < x)
  (h₁ : x = 40 ∨ a = 40)
  (Sₐ Sₓ : Set ℤ)
  (hSₐ : Sₐ = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)})
  (hSₓ : Sₓ = {y | 0 < y ∧ Int.gcd a y = y + 3 ∧ Int.lcm a y = y * (y + 3)})
  (h₂ : Nonempty Sₐ)
  (h₃ : Nonempty Sₓ) :
  IsLeast Sₐ 8 ∨ IsLeast Sₓ 8 := by
```

What it contributes. Generalizes the parent's integer gcd/lcm divisibility argument to a bounded natural-number anchor and replaces the fixed least-set disjunction with an anchor characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine parameterized characterization: the compatible set has least element 8 exactly when the anchor equals 8. Its divisibility and antisymmetry argument is unrelated to the parent's fixed-anchor disjunction and gcd-lcm contradiction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the anchor and proves a new iff characterization: the compatible set has least element 8 exactly when the anchor is 8. Its divisibility and antisymmetry argument is not supplied by the parent's fixed-40, disjunctive contradiction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-anchor disjunction and shifted gcd/lcm equations with a genuine characterization over arbitrary anchors: the compatible set has least element 8 exactly when the anchor is 8. Its proof uses divisibility and antisymmetry to identify the anchor, reasoning not supplied by the parent's contradiction-based argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2493 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_anchor_gcd_lcm_characterization.extracted_1_1 (a : ℕ) (ha₁ : 1 ≤ a) (ha₁₆ : a ≤ 16) :
  IsLeast {b | 0 < b ∧ b.gcd a = a ∧ b.lcm a = a} 8 ↔ a = 8
```

Read back by a model that saw only the Lean. *For a natural number a satisfying $1 \leq a \leq 16$, 8 is the least natural number b such that $0 < b$, $\gcd(b,a) = a$, and $\text{lcm}(b,a) = a$, if and only if $a = 8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_anchor_gcd_lcm_characterization
  (a : ℕ)
  :
  IsLeast {b : ℕ | 0 < b ∧ Nat.gcd b a = a ∧ Nat.lcm b a = a} 8 ↔ a = 8 := by
  constructor
  · intro h
    have h8mem := h.1
    have hg : Nat.gcd 8 a = a := h8mem.2.1
    have hl : Nat.lcm 8 a = a := h8mem.2.2
    have hdiv₁ : a | 8 := by
      have hd := Nat.gcd_dvd_left 8 a
      rw [hg] at hd
      exact hd
    have hdiv₂ : 8 | a := by
      have hd := Nat.dvd_lcm_left 8 a
      rw [hl] at hd
      exact hd
    exact Nat.dvd_antisymm hdiv₁ hdiv₂
  · intro ha
    subst a
    refine ⟨?, ?⟩
    · norm_num
    · intro b hb
      have hd : Nat.gcd b 8 | b := Nat.gcd_dvd_left b 8
      have hg : Nat.gcd b 8 = 8 := hb.2.1
      rw [hg] at hd
      rcases hd with ⟨k, hk⟩
      have hbpos : 0 < b := hb.1
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh__34f6bf04

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e87564ebb5c22cb4

4.475 475. odd_powers_ending_nine_sum

The problem

For natural numbers a, b, m, n , show that $(10a + 9)^{2m+1} + (10b + 9)^{2n+1}$ has remainder 8 upon division by 10.

Lean statement

```
theorem odd_powers_ending_nine_sum (a b m n : ℕ) : ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8 := by
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed numerical computation: it proves every odd power of any number ending in 9 is congruent to 9 modulo 10, using an inductive periodicity argument before deriving the sum as 8. The parent's one-line norm_num proof supplies none of this symbolic reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both fixed bases to arbitrary numbers ending in 9 and allows independent odd exponents. Its proof requires establishing a modular-periodicity invariant by induction, which the parent's one-shot norm_num proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both bases from the fixed pair 19 and 99 to arbitrary numbers ending in 9 and requires an induction showing every odd power remains congruent to 9 modulo 10. The parent's one-step norm_num proof supplies none of this universal reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1871 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_powers_ending_nine_sum.extracted_1_1 (a b m n : ℕ) :
  ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8
```

Read back by a model that saw only the Lean. For all natural numbers a, b, m , and n , the remainder when $(10a +$

$9)^{(2m+1)} + (10b+9)^{(2n+1)}$ is divided by 10 is 8.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_powers_ending_nine_sum (a b m n : ℕ) :
  ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8 := by
  have hsq_a : (10 * a + 9) ^ 2 % 10 = 1 := by
    norm_num [pow_two, Nat.mul_mod, Nat.add_mod]
  have hsq_b : (10 * b + 9) ^ 2 % 10 = 1 := by
    norm_num [pow_two, Nat.mul_mod, Nat.add_mod]
  have hpow_a : ∀ k : ℕ, (10 * a + 9) ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero =>
      norm_num [Nat.add_mod, Nat.mul_mod]
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega]
      rw [pow_add, Nat.mul_mod, ih, hsq_a]
  have hpow_b : ∀ k : ℕ, (10 * b + 9) ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero =>
      norm_num [Nat.add_mod, Nat.mul_mod]
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega]
      rw [pow_add, Nat.mul_mod, ih, hsq_b]
  calc
    ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 =
      (((10 * a + 9) ^ (2 * m + 1) % 10 + (10 * b + 9) ^ (2 * n + 1) % 10) % 10) := by
        rw [Nat.add_mod]
    _ = 8 := by
      rw [hpow_a m, hpow_b n]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__mh__d40adbf9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7f775a4c9d8532cc

4.476 476. numbertheory_202_parity_characterization

The problem

For which natural numbers e and f is $19^e + 99^f$ congruent to 8 modulo 10? Show that this occurs exactly when both e and f are odd.

Lean statement

```
theorem numbertheory_202_parity_characterization (e f : ℕ) : (19 ^ e + 99 ^ f ≡ 8 [MOD 10]) ↔ (e % 2 = 1 ∧ f % 2 = 1) := by
```

Parent. Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Parent in Lean

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

What it contributes. Retains the parent bases \$19\$, \$99\$, and modulus \$10\$, while replacing the fixed computation with the reusable parity classification underlying its congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponents and changes the conclusion into an exact characterization: the congruence holds precisely when both exponents are odd. Proving both directions requires parity-based residue reasoning that the parent's direct numerical normalization does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent from one fixed exponent pair to an exact characterization of all exponent parities. Its proof requires modular periodicity and a four-case parity analysis, which the parent's single 'norm_num' computation does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single fixed-exponent congruence with an exact characterization for arbitrary natural exponents. Proving both directions requires parity decomposition of the powers modulo 10, which is not supplied by the parent's one-line normalization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2766 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem numbertheory_202_parity_characterization.extracted_1_1 (e f : ℕ) :  
  19 ^ e + 99 ^ f ≡ 8 [MOD 10] ↔ e % 2 = 1 ∧ f % 2 = 1
```

Read back by a model that saw only the Lean. For natural numbers e and f , 19 raised to the power e plus 99 raised to the power f is congruent to 8 modulo 10 if and only if the remainder when e is divided by 2 is 1 and the remainder when f is divided by 2 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem numbertheory_202_parity_characterization (e f : ℕ) :  
  (19 ^ e + 99 ^ f ≡ 8 [MOD 10]) ↔ (e % 2 = 1 ∧ f % 2 = 1) := by
  have residue19 : ∀ n : ℕ, 19 ^ n % 10 = (if n % 2 = 0 then 1 else 9) := by
    intro n
    have hsq : 19 ^ 2 ≡ 1 [MOD 10] := by norm_num [Nat.ModEq]
    have heven : ∀ k : ℕ, 19 ^ (2 * k) % 10 = 1 := by
      intro k
      have h := hsq.pow k
      simpa [Nat.ModEq, pow_mul] using h
    have hodd : ∀ k : ℕ, 19 ^ (2 * k + 1) % 10 = 9 := by
      intro k
      have h := (hsq.pow k).mul (show 19 ≡ 9 [MOD 10] by norm_num [Nat.ModEq])
      simpa [Nat.ModEq, pow_mul, pow_succ] using h
    rcases Nat.mod_two_eq_zero_or_one n with hn | hn
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using heven k
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k + 1 := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using hodd k
  have residue99 : ∀ n : ℕ, 99 ^ n % 10 = (if n % 2 = 0 then 1 else 9) := by
    intro n
    have hsq : 99 ^ 2 ≡ 1 [MOD 10] := by norm_num [Nat.ModEq]
    have heven : ∀ k : ℕ, 99 ^ (2 * k) % 10 = 1 := by
      intro k
      have h := hsq.pow k
      simpa [Nat.ModEq, pow_mul] using h
    have hodd : ∀ k : ℕ, 99 ^ (2 * k + 1) % 10 = 9 := by
      intro k
      have h := (hsq.pow k).mul (show 99 ≡ 9 [MOD 10] by norm_num [Nat.ModEq])
      simpa [Nat.ModEq, pow_mul, pow_succ] using h
    rcases Nat.mod_two_eq_zero_or_one n with hn | hn
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using heven k
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k + 1 := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using hodd k
  constructor
  · intro h
    by_cases he : e % 2 = 0 <|> by_cases hf : f % 2 = 0
    · have h19 := residue19 e
      have h99 := residue99 f
```

```

      simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · rintro ⟨he, hf⟩
    have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h19 h99
    omega

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms.mh__ad2e8e24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ad2e8e24376466dd

4.477 477. bounded_period_exponents_twelve

The problem

Among the first twelve positive exponents, 6 and 12 are exactly the exponents for which 5 raised to that exponent is congruent to 1 modulo 7.

Lean statement

```
theorem bounded_period_exponents_twelve (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 ∨ n = 12 := by
```

Parent. Let n be a positive integer with $n \leq 6$. Show that 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Parent in Lean

```
theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The bound is genuinely extended from 6 to 12, requiring new residue checks for exponents 7–11 and proving the additional period at 12. Although the case-split scaffold mirrors the parent, the child's conclusion requires establishing a new exponent class rather than merely wrapping or decorating the parent result.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the bounded classification from exponents 1–6 to 1–12, requiring new case analysis and establishing the additional period instance $5^{12} \equiv 1 \pmod{7}$. The parent's theorem cannot handle $n = 12$, so its proof does not supply the child's full obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child expands the bound to 12 and characterizes both period multiples, requiring new modular checks for exponents 7–12 and a separate proof of the 12th-period fact. The parent's bounded case analysis only reaches exponent 6 and cannot supply this obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1816 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_period_exponents_twelve.extracted_1.1 (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) :  
  5 ^ n ≡ 1 [ZMOD 7] ↔ n = 6 ∨ n = 12
```

Read back by a model that saw only the Lean. For a natural number n satisfying $0 < n$ and $n \leq 12$, 5^n is congruent to 1 modulo 7 if and only if $n = 6$ or $n = 12$.

Lean certificate

```

import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_period_exponents_twelve (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 ∨ n = 12 := by
  have hperiod6 : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hperiod12 : (5 : ℤ) ^ 12 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  constructor
  · intro h
    have hcases : n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4 ∨ n = 5 ∨ n = 6 ∨ n = 7 ∨ n = 8 ∨ n = 9 ∨ n = 10 ∨ n = 11 ∨ n = 12 := by
      omega
    rcases hcases with h1 | h2 | h3 | h4 | h5 | h6 | h7 | h8 | h9 | h10 | h11 | h12
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; exact Or.inl rfl
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; exact Or.inr rfl
  · intro h
    rcases h with h6 | h12
    · subst n
      simpa using hperiod6
    · subst n
      simpa using hperiod12

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me.mh__004a2b02

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint db49e39731cde1db

4.478 478. bounded_six_period_power_mod_seven

The problem

Let n be an integer congruent to 5 modulo 7, and let k be a natural number. Prove that n^{6k} is congruent to 1 modulo 7.

Lean statement

```
theorem bounded_six_period_power_mod_seven (n : Z) (k : N) (h : n ≡ 5 [ZMOD 7]) : n ^ (6 * k) ≡ 1 [ZMOD 7]
```

Parent. Prove that if an integer n is congruent to 5 modulo 7, then n^{30} is congruent to 1 modulo 7.

Parent in Lean

```
theorem parameterized_power_mod_seven (n : Z) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplied the modulo-7 residue-lifting strategy, generalized from the fixed exponent to the bounded parameterized exponent $6k$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent to every multiple of six, requiring the new periodicity argument that proves $5^6 \equiv 1$ and lifts it through an arbitrary k . The parent's direct computation of 5^{30} does not supply this parameterized reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent to an arbitrary six-period exponent and requires proving and iterating the modular period symbolically. The parent's numeric normalization at exponent 30 cannot supply this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 30 to an arbitrary six-period exponent and must establish the period $\backslash(5^6 \equiv 1 \pmod{7})$ before lifting it to $\backslash(6k)$. The parent's direct fixed-power normalization cannot close this parameterized goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2085 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_six_period_power_mod_seven.extracted_1_1 (n : ℤ) (k : ℕ) (h : n ≡ 5 [ZMOD 7]) :  
  n ^ (6 * k) ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. *For any integer n and natural number k , if n is congruent to 5 modulo 7, then n raised to the power $6k$ is congruent to 1 modulo 7.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_six_period_power_mod_seven (n : ℤ) (k : ℕ) (h : n ≡ 5 [ZMOD 7]) : n ^ (6 * k) ≡ 1 [ZMOD 7] := by  
  have hpow : n ^ (6 * k) ≡ (5 : ℤ) ^ (6 * k) [ZMOD 7] := h.pow (6 * k)  
  have hperiod : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by  
    norm_num [Int.ModEq]  
  have hperiod_pow : ((5 : ℤ) ^ 6) ^ k ≡ (1 : ℤ) ^ k [ZMOD 7] := hperiod.pow k  
  have hbase : (5 : ℤ) ^ (6 * k) ≡ 1 [ZMOD 7] := by  
    simpa [pow_mul] using hperiod_pow  
  have hresult : n ^ (6 * k) ≡ 1 [ZMOD 7] := hpow.trans hbase  
  exact hresult
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh.mh__91d988b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5b4924b55b4cd723

4.479 479. five_pow_six_mul_mod_seven

The problem

Show that for every natural number n , the remainder when 5^{6n} is divided by 7 is 1.

Lean statement

```
theorem five_pow_six_mul_mod_seven (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed exponent 30 to every exponent of the form $6 \cdot n$. Its proof requires establishing the new base congruence $5^6 \equiv 1 \text{ [ZMOD 7]}$ and lifting it through arbitrary powers, which the parent's direct norm_num proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent 30 to every exponent of the form $6n$. Its proof must establish the new base congruence modulo 7 and then use preservation of congruence under powers, which the parent's direct computation does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent 30 to every exponent of the form $6 \cdot n$, requiring the new modular-power argument via $5^6 \equiv 1$ and Int.ModEq.pow. The parent's one-off norm_num proof supplies no reasoning for this universal statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1780 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem five_pow_six_mul_mod_seven.extracted_1_1 (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number n , 5 raised to the power $6n$ is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem five_pow_six_mul_mod_seven (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7] := by
  have hbase : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hpow := Int.ModEq.pow n hbase
  simpa [pow_mul] using hpow
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__093b3f6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b34110a60f7d3d10

4.480 480. parameterized_power_mod_seven

The problem

Prove that if an integer n is congruent to 5 modulo 7, then n^{30} is congruent to 1 modulo 7.

Lean statement

```
theorem parameterized_power_mod_seven (n : ℤ) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplied the modulus-7 power checkpoint that 5^{30} is congruent to 1; the child transports it from a parameterized base.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base and uses the new modular hypothesis via congruence preservation under powers before invoking the fixed numerical computation. The parent's proof supplies only the constant case and cannot establish the parameterized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must use the hypothesis that n is congruent to 5 modulo 7 to transport the thirtieth-power congruence. The parent's numeral-only normalization proof supplies no reasoning for this variable case.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must transfer the congruence hypothesis through the thirtieth power before using the fixed-base computation. The parent's direct normalization proof supplies no way to handle an arbitrary integer congruent to 5 modulo 7.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2082 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_rountrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_power_mod_seven.extracted_1_1 (n : ℤ) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer n , if n is congruent to 5 modulo 7, then n raised to the

30th power is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_power_mod_seven (n : ℤ) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
  have hpow : n ^ 30 ≡ (5 : ℤ) ^ 30 [ZMOD 7] := h.pow 30
  have hbase : (5 : ℤ) ^ 30 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hpow.trans hbase
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__3745eb24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3745eb240b0fe738

4.481 481. every_int_seventh_power_mod_seven

The problem

Show that for every integer n , n^7 is congruent to n modulo 7.

Lean statement

```
theorem every_int_seventh_power_mod_seven (n : ℤ) : n ^ 7 ≡ n [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes a single numerical congruence to the polynomial identity $n^7 \equiv n$ for every integer, requiring a prime-modulus Fermat argument. The parent's norm_num computation for 5^{30} provides no proof of this universal statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single numerical congruence into the universal Fermat congruence $n^7 \equiv n$ for every integer n . Its proof requires invoking primality and a prime-power modular theorem, none of which is supplied by the parent's direct normalization proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from one concrete computation to Fermat's universal congruence for every integer, requiring a prime-exponent theorem and quantified reasoning absent from the parent's norm_num proof. It is not a restatement of the kept $5^{30} \equiv 1 \pmod{7}$ result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1789 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem every_int_seventh_power_mod_seven.extracted_1_1 (n : ℤ) : n ^ 7 ≡ n [ZMOD 7]
```

Read back by a model that saw only the Lean. For every integer n , n raised to the seventh power is congruent to n modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem every_int_seventh_power_mod_seven (n : ℤ) : n ^ 7 ≡ n [ZMOD 7] := by
  have hprime : Nat.Prime 7 := by
    norm_num
  have hfermat : ∀ m : ℤ, m ^ 7 ≡ m [ZMOD 7] := by
    intro m
    exact Int.ModEq.pow_prime_eq_self hprime m
  exact hfermat n
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__43b55310

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 294a39dfac5ba0c5

4.482 482. pow_fifteen_mod_seven_of_congruent_five

The problem

Show that every integer congruent to 5 modulo 7 has its 15th power congruent to -1 modulo 7.

Lean statement

```
theorem pow_fifteen_mod_seven_of_congruent_five : ∀ n : ℤ, n ≡ 5 [ZMOD 7] → n ^ 15 ≡ (-1 : ℤ) [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed residue to every integer congruent to 5 modulo 7 and changes the exponent to 15, requiring congruence transport plus the new computation $5^{15} \equiv -1$. The parent's computation of $5^{30} \equiv 1$ does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed numerical congruence into a universal congruence-class theorem: it must transfer $n \equiv 5$ through the fifteenth power and establish the new base result $5^{15} \equiv -1$. The parent's isolated $5^{30} \equiv 1$ result does not provide this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine universal congruence theorem: it must transport the hypothesis $n \equiv 5 [ZMOD 7]$ through the fifteenth power and separately establish $5^{15} \equiv -1$. The parent's isolated computation of 5^{30} provides neither this transport argument nor the new modular identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1807 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem pow_fifteen_mod_seven_of_congruent_five.extracted_1_1 : ∀ (n : ℤ),
  n ≡ 5 [ZMOD 7] → n ^ 15 ≡ -1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every integer n , if n is congruent to 5 modulo 7, then n raised to the 15th power is congruent to -1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem pow_fifteen_mod_seven_of_congruent_five :  $\forall n : \mathbb{Z}, n \equiv 5 \pmod{7} \rightarrow n^{15} \equiv (-1 : \mathbb{Z}) \pmod{7} :=$  by
  intro n hn
  have hbase :  $(5 : \mathbb{Z})^{15} \equiv (-1 : \mathbb{Z}) \pmod{7} :=$  by
    norm_num [Int.ModEq]
  have hpow :  $n^{15} \equiv (5 : \mathbb{Z})^{15} \pmod{7} :=$  by
    exact hn.pow 15
  exact hpow.trans hbase
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__79de960b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4a1e3944da3d3752

4.483 483. modular_power_of_coprime_seven

The problem

Let n be an integer relatively prime to 7. Show that n^6 leaves remainder 1 when divided by 7.

Lean statement

```
theorem modular_power_of_coprime_seven (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7]
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from one numerical congruence to Fermat's theorem for every integer coprime to 7, requiring a substantive coprimality-based modular argument absent from the parent's direct normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single computed residue with a universal Fermat-style theorem for every integer coprime to 7. Its proof requires invoking primality and a coprime modular-power theorem, reasoning absent from the parent's direct normalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed numerical congruence to every integer coprime to 7 and exponent 6. The parent's direct normalization proof provides no argument for this universal theorem; the child requires invoking Fermat's little theorem with the coprimality hypothesis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2409 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_of_coprime_seven.extracted_1_1 (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer n that is coprime to 7, the sixth power n^6 is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_of_coprime_seven (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7] := by
  have hprime : Nat.Prime 7 := by
    norm_num
  have hfermat : n ^ (7 - 1) ≡ 1 [ZMOD 7] :=
    Int.ModEq.pow_card_sub_one_eq_one hprime h
  simpa using hfermat
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__7a3c855f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 7a3c855f070a2583

4.484 484. fermat_modEq_general

The problem

Let p be a prime natural number and let n be an integer relatively prime to p . Prove that $n^{p-1} \equiv 1 \pmod{p}$.

Lean statement

```
theorem fermat_modEq_general (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n p) : n ^ (p - 1) ≡ 1 [ZMOD p]
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child makes both the modulus and base variable and requires proving the general Fermat congruence from primality and coprimality. The parent's concrete normalization proof supplies none of this reasoning, and the child is not a repeat of the retained parameterized mod-seven result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed congruence to Fermat's theorem for arbitrary prime moduli and coprime integers. Its proof requires a general modular-arithmetic lemma and hypotheses absent from the parent, so recalling the parent cannot solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine universal generalization from the fixed congruence to Fermat's theorem for arbitrary prime moduli and coprime integers. Its proof requires a modular-arithmetic theorem and the coprimality hypothesis, reasoning unavailable from the parent's numerical normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2089 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fermat_modEq_general.extracted_1_1 (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n ↑p) :
  n ^ (p - 1) ≡ 1 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For a natural number p and an integer n , if p is prime and n is coprime

to the integer p , then n raised to the natural-number exponent $p - 1$ is congruent to 1 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fermat_modEq_general (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n p) :
  n ^ (p - 1) ≡ 1 [ZMOD p] := by
  have hprime : Nat.Prime p := hp
  have hunit : IsCoprime n p := hcop
  have hfermat : n ^ (p - 1) ≡ 1 [ZMOD p] :=
    Int.ModEq.pow_card_sub_one_eq_one hprime hunit
  exact hfermat
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__8bb5ed7a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8bb5ed7a6cc95486

4.485 485. bounded_positional_multiplier_balance

The problem

For natural numbers $base$, $lead$, $unit$, and k with $1 < base$, $1 \leq lead < base$, $unit < base$, and $k \leq base$, the equation stating that k times the sum of the base- $base$ digits of the two-digit numeral $base \cdot lead + unit$ equals that numeral is equivalent to $(base - k) \cdot lead = (k - 1) \cdot unit$.

Lean statement

```
theorem bounded_positional_multiplier_balance
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  (hk : k ≤ base) :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  (base - k) * lead = (k - 1) * unit
```

Parent. For natural numbers $base$, $lead$, $unit$, and k with $1 < base$, $1 \leq lead < base$, and $unit < base$, prove that k times the sum of the base- $base$ digits of $base \cdot lead + unit$ equals $base \cdot lead + unit$ if and only if $k(lead + unit) = base \cdot lead + unit$.

Parent in Lean

```
theorem bounded_positional_multiplier_characterization
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  k * (lead + unit) = base * lead + unit
```

What it contributes. Supplied the bounded-base digit decomposition and digit-sum reduction; the child replaces the parent's raw sum equation with the coefficient-balance characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child retains the positional-digit decomposition but replaces the fixed-point equation with a nontrivial Nat-subtraction balance law. Establishing equivalence requires new case analysis for $k = 0$ and arithmetic reasoning using $k \leq base$, which the parent's proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a substantive balance characterization. Its proof requires a new case split on $k=0$ and arithmetic reasoning with truncated natural subtraction, which the parent's digit-decomposition proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a balance equation and requires a substantive natural-number algebra argument, including cases for $k = 0$ and $k \geq 1$. The parent's digit decomposition does not supply this equivalence by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2054 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_positional_multiplier_balance.extracted_1_1 (base lead unit k : ℕ) (hbase : 1 < base) (hlead : 1 ≤ lead)
(hlead_lt : lead < base) (hunit_lt : unit < base) (hk : k ≤ base) :
k * (base.digits (base * lead + unit)).sum = base * lead + unit ↔ (base - k) * lead = (k - 1) * unit
```

Read back by a model that saw only the Lean. For natural numbers $base$, $lead$, $unit$, and k , if $1 < base$, $1 \leq lead$, $lead < base$, $unit < base$, and $k \leq base$, then k multiplied by the sum of the base-base digits of $base \cdot lead + unit$ equals $base \cdot lead + unit$ if and only if $(base - k) \cdot lead = (k - 1) \cdot unit$, where the subtractions are natural-number subtractions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_positional_multiplier_balance
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  (hk : k ≤ base) :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  (base - k) * lead = (k - 1) * unit := by
  have hdigit : Nat.digits base (base * lead + unit) = unit :: Nat.digits base lead := by
    rw [Nat.add_comm (base * lead) unit]
    exact Nat.digits_add base hbase unit lead hunit_lt (Or.inr (Nat.ne_of_gt hlead))
  have hdecomp : (Nat.digits base (base * lead + unit)).sum = lead + unit := by
    rw [hdigit, Nat.digits_of_lt base lead (Nat.ne_of_gt hlead) hlead_lt]
    simp [Nat.add_comm]
  have hfixed_balance :
    k * (lead + unit) = base * lead + unit ↔
    (base - k) * lead = (k - 1) * unit := by
    by_cases hkzero : k = 0
    · subst k
      have hprod : 0 < base * lead := Nat.mul_pos (by omega) hlead
      constructor <=> intro h <=> simp at h <=> omega
    · have hkpos : 1 ≤ k := by omega
      have hklead : k * lead ≤ base * lead := Nat.mul_le_mul_right lead hk
      have hunit : unit ≤ k * unit := by
        calc
          unit = 1 * unit := by simp
          _ ≤ k * unit := Nat.mul_le_mul_right unit hkpos
      constructor
      · intro h
        simp only [Nat.sub_mul, Nat.one_mul]
        rw [Nat.mul_add] at h
        omega
      · intro h
        simp only [Nat.sub_mul, Nat.one_mul] at h
        rw [Nat.mul_add]
        omega
```

```
constructor
· intro h
  rw [hdecomp] at h
  exact hfixed_balance.mp h
· intro h
  rw [hdecomp]
  exact hfixed_balance.mpr h
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.me.mh__3832201b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint eaec694e7c7fb9d7

4.486 486. equal_nonzero_digits_fixed_point_impossible

The problem

Show that for every natural number k and every decimal digit a with $1 \leq a \leq 9$, the number $10a + a$ cannot equal k times the sum of its decimal digits.

Lean statement

```
theorem equal_nonzero_digits_fixed_point_impossible
  (k a : ℕ)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9) :
  ¬ (k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a)
```

Parent. For a bounded multiplier and a positive two-digit decimal pair, the multiplier fixed-point equation holds exactly when it holds after decomposing the number into its tens and units digits.

Parent in Lean

```
theorem bounded_decimal_multiplier_characterization
  (k a b : ℕ)
  (hk : k ≤ 10)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9)
  (hb₉ : b ≤ 9) :
  k * (Nat.digits 10 (10 * a + b)).sum = 10 * a + b ↔
  k * (a + b) = 10 * a + b
```

What it contributes. Uses the parent's bounded decimal digit-sum decomposition checkpoint, then specializes it to equal digits and consumes the resulting equation in an impossibility proof.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's bound on k and proves impossibility for every natural k , requiring a new arithmetic contradiction from $2*k = 11$ after specializing equal digits. The parent's bounded equivalence and finite digit computation do not supply this unbounded contradiction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to equal nonzero digits and derives the new contradiction $2*k = 11$ from the fixed-point equation, then uses arithmetic impossibility. The parent only establishes a digit-sum decomposition and an equivalence, so its proof does not supply the cancellation and contradiction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to equal digits and adds a genuinely new impossibility argument: it transforms the fixed-point equation into $2*k = 11$ and derives a contradiction. The parent's digit-sum decomposition only rewrites the equation and does not supply this parity/cancellation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2056 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equal_nonzero_digits_fixed_point_impossible.extracted_1_1 (k a : ℕ) (ha₁ : 1 ≤ a) (ha₉ : a ≤ 9) :
  ¬k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a
```

Read back by a model that saw only the Lean. *For all natural numbers k and a such that $1 \leq a$ and $a \leq 9$, it is not the case that k multiplied by the sum of the base-10 digits of $10a + a$ equals $10a + a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem equal_nonzero_digits_fixed_point_impossible
  (k a : ℕ)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9) :
  ¬ (k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a) := by
  intro h
  have hdecomp : (Nat.digits 10 (10 * a + a)).sum = a + a := by
    interval_cases a <|>
    norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def]
  have hfixed : k * (a + a) = 10 * a + a := by
    simpa [hdecomp] using h
  have hfactor : (2 * k) * a = 11 * a := by
    calc
      (2 * k) * a = k * (a + a) := by ring
      _ = 10 * a + a := hfixed
      _ = 11 * a := by ring
  have hbad : 2 * k = 11 := Nat.mul_right_cancel (by omega) hfactor
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.mh__68fc7013

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2e6c0da6e232809

4.487 487. characterize_digit_sum_multiplier

The problem

Let x be a positive two-digit number whose decimal digit sum, multiplied by a natural number k , equals x . Prove that $k \leq 10$, and that $k = 10$ exactly when x is divisible by 10.

Lean statement

```
theorem characterize_digit_sum_multiplier (k x : ℕ)
  (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) :
  k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0)
```

Parent. What positive two-digit natural number is exactly twice the sum of its decimal digits? Show that it is uniquely determined and equals 18.

Parent in Lean

```
theorem unique_positive_two_digit_digit_sum_solution :
  ∃! x : ℕ, 0 < x ∧ (Nat.digits 10 x).length = 2 ∧ 2 * (Nat.digits 10 x).sum = x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed multiplier 2 to symbolic k and proves a new characterization: k is bounded by 10 and equals 10 exactly when x is divisible by 10. The parent's unique-witness argument does not establish this variable-parameter iff claim, even though both proofs use finite digit enumeration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed multiplier 2 and proves a new characterization: every two-digit solution has $k \leq 10$, with equality exactly when the units digit is zero. The parent's uniqueness proof for $k = 2$ does not supply this symbolic bound or equivalence.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the multiplier and proves a full characterization: every two-digit solution has $k \leq 10$, with equality exactly when the number ends in zero. The parent's fixed-multiplier uniqueness proof cannot establish this symbolic classification, even though the finite digit-bound technique is reused.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2520 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem characterize_digit_sum_multiplier.extracted_1_1 (k x : ℕ) (hpos : 0 < x) (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) : k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0)
```

Read back by a model that saw only the Lean. For natural numbers k and x , if x is positive, the list of base-10 digits of x has length 2, and k multiplied by the sum of the base-10 digits of x equals x , then k is at most 10, and k equals 10 if and only if the remainder when x is divided by 10 is 0.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem characterize_digit_sum_multiplier (k x : ℕ)

  (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) :
  k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0) := by
  have hxlt : x < 100 := by
    have h := Nat.lt_base_pow_length_digits (m := x) (b := 10) (by norm_num)
    rw [hlen] at h
    norm_num at h
    exact h
  interval_cases x <=>
    norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] at hlen <=>
    norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] at heq <=>
    norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] <=>
    omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.mh__c0669468

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f6439907ff880f2c

4.488 488. common_cube_fourth_characterization

The problem

Show that n is both the cube of x and the fourth power of t exactly when there is a natural number c such that $n = c^{12}$, $x = c^4$, and $t = c^3$.

Lean statement

```
theorem common_cube_fourth_characterization
  (n x t : ℕ) :
  (x ^ 3 = n ∧ t ^ 4 = n) ↔ ∃ c : ℕ, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3
```

Parent. Among positive integers other than 1 that are at most 4096, an integer is both a perfect cube and a perfect fourth power if and only if it equals 4096.

Parent in Lean

```
theorem bounded_common_cube_fourth_characterization
  (n : ℕ)
  (hn : n ≤ 4096) :
  (0 < n ∧ n ≠ 1 ∧ ∃ x, x ^ 3 = n ∧ ∃ t, t ^ 4 = n) ↔ n = 4096
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's bounded finite enumeration with a universal characterization of shared cube and fourth-power values. Its proof requires the coprime-exponent power decomposition theorem and symbolic exponent algebra, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's bounded finite search with a genuinely symbolic characterization of all common cube/fourth powers, requiring coprime-exponent factorization and deriving the twelfth-power structure.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite bound and proves a genuinely symbolic characterization: common cube and fourth powers must arise from a shared twelfth power. Its proof requires coprime-exponent reasoning, which is absent from the parent's bounded case split.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2633 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem common_cube_fourth_characterization.extracted_1_1 (n x t : ℕ) :  
  x ^ 3 = n ∧ t ^ 4 = n ↔ ∃ c, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3
```

Read back by a model that saw only the Lean. For natural numbers n , x , and t , x cubed equals n and t to the fourth power equals n if and only if there exists a natural number c such that n equals c to the twelfth power, x equals c to the fourth power, and t equals c cubed.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem common_cube_fourth_characterization  
  (n x t : ℕ) :  
  (x ^ 3 = n ∧ t ^ 4 = n) ↔ ∃ c : ℕ, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3 := by  
  constructor  
  · rintro ⟨hx, ht⟩  
    have hxt : x ^ 3 = t ^ 4 := by  
      calc  
        x ^ 3 = n := hx  
        _ = t ^ 4 := ht.symm  
    obtain ⟨c, hxc, htc⟩ :=  
      Nat.exists_eq_pow_of_exponent_coprime_of_pow_eq_pow  
        (a := x) (b := t) (m := 3) (n := 4) (by norm_num) hxt  
    refine ⟨c, ?_, hxc, htc⟩  
    calc  
      n = x ^ 3 := hx.symm  
      _ = (c ^ 4) ^ 3 := by rw [hxc]  
      _ = c ^ 12 := by ring  
  · rintro ⟨c, hn, hxc, htc⟩  
    constructor  
    · calc  
      x ^ 3 = (c ^ 4) ^ 3 := by rw [hxc]  
      _ = c ^ 12 := by ring  
      _ = n := hn.symm  
    · calc  
      t ^ 4 = (c ^ 3) ^ 4 := by rw [htc]  
      _ = c ^ 12 := by ring  
      _ = n := hn.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_296__me.mh__23bfbf4a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 23bfbf4a6a684f04

4.489 489. inverse_seven_is_right_inverse

The problem

For every natural number m relatively prime to 7, if n is the multiplicative inverse of 7 modulo m , show that $7n = 1$ modulo m .

Lean statement

```
theorem inverse_seven_is_right_inverse
  (m : ℕ)
  (n : ZMod m)
  (hm : Nat.Coprime 7 m)
  (h : n = (7 : ZMod m)⁻¹) :
  (7 : ZMod m) * n = 1
```

Parent. Find the integer n such that $0 \leq n < 398$ and n is a multiplicative inverse to 7 modulo 398. Show that it is 57.

Parent in Lean

```
theorem mathd_numbertheory_33
  (n : ZMod 398)
  (h₁ : n = 7⁻¹) :
  n = 57 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from the fixed modulus 398 and computed inverse value 57 to every modulus coprime to 7, and changes the conclusion to a right-inverse identity. Its proof requires the general coprime-unit inverse lemma rather than recalling the parent’s concrete modular calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the fixed modulus 398 and computed inverse value 57 to arbitrary m coprime to 7, requiring the general ZMod inverse lemma. The parent’s finite computation cannot supply this universal right-inverse claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed modulus and proves a right-inverse identity for every coprime modulus, requiring the new coprimality-based lemma ‘ZMod.coe_mul_inv_eq_one’. The parent’s computation at modulus 398 does not supply this argument, and the child is not merely the already-kept characterization at 398.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1880 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem inverse_seven_is_right_inverse.extracted_1_1 (m : ℕ) (n : ZMod m) (hm : Coprime 7 m) (h : n = 7⁻¹) :  
  7 * n = 1
```

Read back by a model that saw only the Lean. For a natural number m and an element n of $ZMod\ m$, if 7 is coprime to m and n equals the multiplicative inverse of 7 in $ZMod\ m$, then $7n = 1$ in $ZMod\ m$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem inverse_seven_is_right_inverse  
  (m : ℕ)  
  (n : ZMod m)  
  (hm : Nat.Coprime 7 m)  
  (h : n = (7 : ZMod m)⁻¹) :  
  (7 : ZMod m) * n = 1 := by  
  have hinv : (7 : ZMod m) * (7 : ZMod m)⁻¹ = 1 :=  
    ZMod.coe_mul_inv_eq_one 7 hm  
  rw [h]  
  exact hinv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_33__mh__8d11f807

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Quot.sound, fingerprint 6ab90dfa0d6926bb

4.490 490. modular_inverse_unique_398_for_nine

The problem

Show that 177 is the unique residue modulo 398 whose product with 9 is congruent to 1.

Lean statement

```
theorem modular_inverse_unique_398_for_nine (x : ZMod 398) :  
  ((9 : ZMod 398) * x = 1) ↔ x = 177
```

Parent. Find the integer n such that $0 \leq n < 398$ and n is a multiplicative inverse to 7 modulo 398. Show that it is 57.

Parent in Lean

```
theorem mathd_numbertheory_33  
  (n : ZMod 398)  
  (h1 : n = 7-1) :  
  n = 57 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the modulus equation and the conclusion's shape: it proves a biconditional characterizing the unique solution to $9x = 1$ modulo 398. Its forward direction requires a general cancellation/uniqueness argument, which is not supplied by the parent's direct computation of 7^{-1} .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the multiplier and conclusion form, requiring a bidirectional uniqueness proof and cancellation argument for solutions to $9x=1$ modulo 398 . The parent's direct computation of $7^{-1}=57$ does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the concrete inverse computation into a biconditional uniqueness theorem for a different residue, requiring a general cancellation argument and a new modular calculation. Remembering the parent's fact about 7^{-1} would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2488 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_inverse_unique_398_for_nine.extracted_1_1 (x : ZMod 398) : 9 * x = 1 ↔ x = 177
```

Read back by a model that saw only the Lean. For an element x of the ring of integers modulo 398, $9x = 1$ if and only if $x = 177$ in $\mathbb{ZMod}\ 398$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem modular_inverse_unique_398_for_nine (x : ZMod 398) :
  ((9 : ZMod 398) * x = 1) ↔ x = 177 := by
  constructor
  · intro hx
    have h177 : (9 : ZMod 398) * 177 = 1 := by
      decide
    calc
      x = 1 * x := by simp
      _ = ((9 : ZMod 398) * 177) * x := by rw [h177]
      _ = 177 * ((9 : ZMod 398) * x) := by ring
      _ = 177 := by rw [hx]; simp
  · intro hx
    rw [hx]
    decide
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_33__mh__ae6d0d0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Quot.sound, fingerprint ae6d0d0e8b69e631

4.491 491. fibonacci_recurrence_unique

The problem

Let a be a sequence of natural numbers with $a(1) = 1$, $a(2) = 1$, and $a(n + 2) = a(n + 1) + a(n)$ for every natural number n . Prove that any sequence b of natural numbers satisfying $b(1) = 1$, $b(2) = 1$, and $b(n + 2) = b(n + 1) + b(n)$ for every natural number n must satisfy $b(n) = a(n)$ for every natural number n .

Lean statement

```
theorem fibonacci_recurrence_unique (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1) (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ b : ℕ → ℕ, b 1 = 1 → b 2 = 1 → (∀ n, b (n + 2) = b (n + 1) + b n) → ∀ n, b n = a n
```

Parent. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the fixed-index modular conclusion into a uniqueness theorem for the entire recurrence. Its proof must establish equality at every index by strong induction, including deriving equality of the otherwise unstated zeroth terms from both recurrences; the parent's Fibonacci-identification and mod-4 argument supplies none of this.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from computing one fixed Fibonacci residue to proving uniqueness of the entire recurrence sequence. Its proof must compare two arbitrary sequences and derive equality at every index, which the parent's proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed-index modular conclusion into a uniqueness theorem for the entire recurrence. Its proof must establish equality of two arbitrary sequences at every index, including deriving equality at index 0 and carrying out a fresh strong-induction argument, so the parent's proof does not directly supply it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2052 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fibonacci_recurrence_unique.extracted_1_1 :  $\forall (a : \mathbb{N} \rightarrow \mathbb{N}),$   
   $a\ 1 = 1 \rightarrow$   
     $a\ 2 = 1 \rightarrow$   
       $(\forall (n : \mathbb{N}), a\ (n + 2) = a\ (n + 1) + a\ n) \rightarrow$   
         $\forall (b : \mathbb{N} \rightarrow \mathbb{N}), b\ 1 = 1 \rightarrow b\ 2 = 1 \rightarrow (\forall (n : \mathbb{N}), b\ (n + 2) = b\ (n + 1) + b\ n) \rightarrow \forall (n : \mathbb{N}), b\ n = a\ n$ 
```

Read back by a model that saw only the Lean. *For every function a from the natural numbers to the natural numbers, if $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$, then for every function b from the natural numbers to the natural numbers, if $b(1) = 1$, $b(2) = 1$, and for every natural number n , $b(n + 2) = b(n + 1) + b(n)$, then for every natural number n , $b(n) = a(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fibonacci_recurrence_unique (a :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (h₀ :  $a\ 1 = 1$ ) (h₁ :  $a\ 2 = 1$ ) (h₂ :  $\forall n, a\ (n + 2) = a\ (n + 1) + a\ n$ ) :
   $\forall b : \mathbb{N} \rightarrow \mathbb{N}, b\ 1 = 1 \rightarrow b\ 2 = 1 \rightarrow (\forall n, b\ (n + 2) = b\ (n + 1) + b\ n) \rightarrow \forall n, b\ n = a\ n :=$  by
  intro b hb₀ hb₁ hb₂
  have ha₀ :  $a\ 0 = b\ 0 :=$  by
    have ha₂ := h₂ 0
    have hb₂' := hb₂ 0
    norm_num at ha₂ hb₂'
    omega
  intro n
  induction n using Nat.strong_induction_on with
  | h n ih =>
    by_cases hn₀ :  $n = 0$ 
    · subst n
      exact ha₀.symm
    by_cases hn₁ :  $n = 1$ 
    · subst n
      omega
    have hn₂ :  $2 \leq n :=$  by omega
    have ha_rec := h₂ (n - 2)
    have hb_rec := hb₂ (n - 2)
    have hi_prev := ih (n - 1) (by omega)
    have hi_prev2 := ih (n - 2) (by omega)
    rw [show  $n - 2 + 2 = n$  by omega, show  $n - 2 + 1 = n - 1$  by omega] at ha_rec hb_rec
    omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_483__mh__1af05244

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 1af05244000ddf3b

4.492 492. bounded_residue_representation_unique

The problem

For all natural numbers q , k , and r , if $r < 6$ and $6q + 5 = 6k + r$, then $r = 5$ and $k = q$.

Lean statement

```
theorem bounded_residue_representation_unique (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) : r = 5 ∧ k = q
```

Parent. For every natural number q at most 254, the number $6q + 5$ has remainder 5 upon division by 6.

Parent in Lean

```
theorem bounded_residue_law (q : ℕ) (hq : q ≤ 254) : (6 * q + 5) % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a fixed modular residue to uniqueness of both quotient and remainder in a representation, requiring derivation of $r = 5$ and $k = q$ from the equation and $r < 6$. The parent's residue calculation does not supply this uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a residue computation into a uniqueness theorem: from an arbitrary representation with $r < 6$ it must derive both $r = 5$ and $k = q$. The parent's fixed-residue proof supplies neither quotient uniqueness nor the use of the representation hypothesis, so recalling it does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed residue computation into a uniqueness theorem, requiring derivation of both the remainder and quotient from an arbitrary representation. The parent's decomposition proof does not establish these new equalities.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed hq
corpus_dedup	hash	unique against 1873 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_residue_representation_unique.extracted_1_1 (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) :  
  r = 5 ∧ k = q
```

Read back by a model that saw only the Lean. For natural numbers q , k , and r , if r is less than 6 and $6q + 5 = 6k + r$, then $r = 5$ and $k = q$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_residue_representation_unique (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) : r = 5 ∧ k = q := by
  have hdecomp : 6 * q + 5 = 5 + 6 * q := by
    omega
  have hrem : r = 5 := by
    omega
  have hquo : k = q := by
    omega
  exact ⟨hrem, hquo⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__me.mh__c475d546

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 6155848cddaca817

4.493 493. nat_mod_six_eq_five_iff

The problem

For every natural number n , show that n leaves remainder 5 upon division by 6 if and only if $n = 6k + 5$ for some natural number k .

Lean statement

```
theorem nat_mod_six_eq_five_iff (n : ℕ) : n % 6 = 5 ↔ ∃ k : ℕ, n = 6 * k + 5 := by
```

Parent. Determine the remainder of 1529 (mod 6). Show that it is 5.

Parent in Lean

```
theorem mathd_numbertheory_551 : 1529 % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child generalizes a single computed remainder to a full characterization for every natural number, requiring division/modulo decomposition and a converse arithmetic argument. The parent's 'norm_num' proof supplies none of this reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete remainder calculation to a characterization for every natural number. Its proof requires division-algorithm decomposition and a converse arithmetic argument, neither supplied by the parent's one-step norm_num proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single computed remainder with a universal characterization of all natural numbers congruent to 5 modulo 6. Its proof requires quotient-remainder decomposition and bidirectional existential reasoning, none of which is supplied by the parent's direct norm_num calculation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propect, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nat_mod_six_eq_five_iff.extracted_1_1 (n : ℕ) : n % 6 = 5 ↔ ∃ k, n = 6 * k + 5
```

Read back by a model that saw only the Lean. For every natural number n , n modulo 6 equals 5 if and only if there exists a natural number k such that $n = 6k + 5$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem nat_mod_six_eq_five_iff (n : ℕ) : n % 6 = 5 ↔ ∃ k : ℕ, n = 6 * k + 5 := by
  constructor
  · intro h
    have hdecomp : n = 6 * (n / 6) + n % 6 := by
      omega
    have hform : n = 6 * (n / 6) + 5 := by
      omega
    exact ⟨n / 6, hform⟩
  · rintro ⟨k, hk⟩
    subst n
    omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__mh__b54fa7c4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 33d3d25490397417

4.494 494. remainder_uniqueness

The problem

Show that if $1529 = 6q + r$ with q, r nonnegative integers and $r < 6$, then $q = 254$ and $r = 5$.

Lean statement

```
theorem remainder_uniqueness : ∀ q r : ℕ, 1529 = 6 * q + r → r < 6 → q = 254 ∧ r = 5 := by
```

Parent. Determine the remainder of 1529 (mod 6). Show that it is 5.

Parent in Lean

```
theorem mathd_numbertheory_551 : 1529 % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes a fixed remainder computation into a uniqueness theorem requiring arbitrary natural-number decomposition hypotheses and deriving both quotient and remainder. The parent's numerical fact and 'norm_num' proof do not supply this quantified arithmetic argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed remainder computation into a uniqueness theorem: from an arbitrary decomposition with a bounded remainder, it must derive both the quotient and remainder. This requires arithmetic reasoning about the hypotheses that the parent's direct normalization proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a concrete remainder computation into a uniqueness characterization of every valid quotient-remainder decomposition. Proving both the quotient and remainder from the hypotheses requires arithmetic reasoning not supplied by the parent's direct normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2432 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem remainder_uniqueness.extracted_1_1 : ∀ (q r : ℕ), 1529 = 6 * q + r → r < 6 → q = 254 ∧ r = 5
```

Read back by a model that saw only the Lean. For all natural numbers q and r , if $1529 = 6q + r$ and $r < 6$, then $q = 254$ and $r = 5$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem remainder_uniqueness :  $\forall q\ r : \mathbb{N}, 1529 = 6 * q + r \rightarrow r < 6 \rightarrow q = 254 \wedge r = 5$  := by
  intro q r hdecomp hrem
  have hq :  $q \leq 254$  := by
    omega
  have hresult :  $q = 254 \wedge r = 5$  := by
    omega
  exact hresult
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__mh__f928832a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f928832ab7633afd

4.495 495. power_difference_residue_bounded

The problem

Let t be a natural number and let $r \in \{0, 1, 2, 3, 4, 5\}$. If $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$, determine the residue of $A^{6t+r} - B^{6t+r}$ modulo 7.

Lean statement

```
theorem power_difference_residue_bounded (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + r) -
  B ^ (6 * t + r) ≡ (match r with | 0 => 0 | 1 => 3 | 2 => 4 | 3 => 2 | 4 => 6 | 5 => 5 | _ => 0) [ZMOD 7]
```

Parent. For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Parent in Lean

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡
  3 [ZMOD 7]
```

What it contributes. Reuses the modulus-7 exponent-period argument, transport of both base congruences, and subtraction of congruences, while extending the exponent to all bounded residue classes.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent from residue 1 to every bounded residue r and requires a six-way case split with distinct power residues modulo 7. The parent's fixed-residue proof does not supply these cases, especially the $r = 0$ treatment and the symbolic computation of $(1^r - 5^r)$ modulo 7.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from the parent's single residue 1 to all bounded residues 0 through 5, requiring a new residue case split and separate power computations. The parent's period argument alone does not supply these residue-specific conclusions.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from $6t+1$ to all bounded residues $6t+r$ and computes a different residue for each case. The parent's reduction to exponent 1 no longer applies; the proof must establish the period reduction for symbolic r and perform a six-way residue analysis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2527 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_bounded.extracted_1_1 (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7])
(hB : B ≡ 5 [ZMOD 7]) :
A ^ (6 * t + r) - B ^ (6 * t + r) ≡
  match r, hr with
  | 0, hr => 0
  | 1, hr => 3
  | 2, hr => 4
  | 3, hr => 2
  | 4, hr => 6
  | 5, hr => 5
  | x, hr => 0 [ZMOD
7]
```

Read back by a model that saw only the Lean. For natural numbers t and r and integers A and B , if $r \leq 5$, A is congruent to 1 modulo 7, and B is congruent to 5 modulo 7, then $A^{(6t+r)} - B^{(6t+r)}$ is congruent modulo 7 to 0 when $r = 0$, 3 when $r = 1$, 4 when $r = 2$, 2 when $r = 3$, 6 when $r = 4$, 5 when $r = 5$, and 0 in any other matching case.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_residue_bounded (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + r) -
  B ^ (6 * t + r) ≡ (match r with | 0 => 0 | 1 => 3 | 2 => 4 | 3 => 2 | 4 => 6 | 5 => 5 | _ => 0) [ZMOD 7] := by
  have main_pos : ∀ n : ℕ, 0 < n → A ^ (6 * t + n) - B ^ (6 * t + n) ≡ (1 : ℤ) ^ n - 5 ^ n [ZMOD 7] := by
    intro n hn
    have hperiodA : A ^ (6 * t + n) ≡ A ^ n [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + n) (y := n)
      · norm_num
      · use t
      · omega
      · omega
      · exact hn
    have hperiodB : B ^ (6 * t + n) ≡ B ^ n [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + n) (y := n)
      · norm_num
      · use t
      · omega
      · omega
      · exact hn
    have hAred : A ^ (6 * t + n) ≡ (1 : ℤ) ^ n [ZMOD 7] := by
      exact hperiodA.trans (hA.pow n)
    have hBred : B ^ (6 * t + n) ≡ (5 : ℤ) ^ n [ZMOD 7] := by
      exact hperiodB.trans (hB.pow n)
    exact hAred.sub hBred
  have hcases : r = 0 ∨ r = 1 ∨ r = 2 ∨ r = 3 ∨ r = 4 ∨ r = 5 := by
    omega
  rcases hcases with rfl | rfl | rfl | rfl | rfl | rfl
  · by_cases ht : t = 0
    · subst t
      norm_num [Int.ModEq]
    · have hperiodA : A ^ (6 * t) ≡ A ^ 6 [ZMOD 7] := by
        apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t) (y := 6)
        · norm_num
        · use t - 1
        · omega
        · omega
        · norm_num
      have hperiodB : B ^ (6 * t) ≡ B ^ 6 [ZMOD 7] := by
        apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t) (y := 6)
        · norm_num
        · use t - 1
        · omega
        · omega
        · norm_num
      have hAred : A ^ (6 * t) ≡ (1 : ℤ) ^ 6 [ZMOD 7] := by
        exact hperiodA.trans (hA.pow 6)
```

```

have hBred : B ^ (6 * t) ≡ (5 : Z) ^ 6 [ZMOD 7] := by
  exact hperiodB.trans (hB.pow 6)
calc
  A ^ (6 * t) - B ^ (6 * t) ≡ (1 : Z) ^ 6 - 5 ^ 6 [ZMOD 7] := hAred.sub hBred
  _ ≡ 0 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : Z) ^ 1 - 5 ^ 1 [ZMOD 7] := main_pos 1 (by omega)
  _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 2) - B ^ (6 * t + 2) ≡ (1 : Z) ^ 2 - 5 ^ 2 [ZMOD 7] := main_pos 2 (by omega)
  _ ≡ 4 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 3) - B ^ (6 * t + 3) ≡ (1 : Z) ^ 3 - 5 ^ 3 [ZMOD 7] := main_pos 3 (by omega)
  _ ≡ 2 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 4) - B ^ (6 * t + 4) ≡ (1 : Z) ^ 4 - 5 ^ 4 [ZMOD 7] := main_pos 4 (by omega)
  _ ≡ 6 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 5) - B ^ (6 * t + 5) ≡ (1 : Z) ^ 5 - 5 ^ 5 [ZMOD 7] := main_pos 5 (by omega)
  _ ≡ 5 [ZMOD 7] := by norm_num [Int.ModEq]

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.mh__39acdc36

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 39acdc36e99f12f5

4.496 496. power_difference_period_six

The problem

Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that increasing any natural exponent by 6 preserves the residue of $a^k - b^k$ modulo 7.

Lean statement

```
theorem power_difference_period_six (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7]
```

Parent. Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. For $1 \leq k \leq 13$, prove that $a^k - b^k \equiv 3 \pmod{7}$ if and only if $k \equiv 1 \pmod{6}$.

Parent in Lean

```
theorem bounded_variable_base_residue_iff (a b : ℤ) (k : ℕ) (hk : k ≤ 13) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (a ^ k - b ^ k ≡ 3 [ZMOD 7]) ↔ k % 6 = 1 := by
```

What it contributes. Uses the parent's modulus-7 base-residue setup and six-cycle mechanism, replacing its bounded residue classification with a reusable exponent-transport invariant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's bounded case analysis and proves a reusable exponent-period law for every k . Its proof requires modular power transport and the period-six identity, which the parent's residue enumeration does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new unbounded period-six transport law, requiring derivation that both bases have sixth power congruent to 1 and rewriting powers via exponent addition. The parent's bounded case split and residue characterization do not supply this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new unbounded period-six transport law for the power difference, using multiplicative periodicity of both bases. The parent only handles a bounded residue characterization by finite exponent cases and does not supply this reusable exponent-step argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2410 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_period_six.extracted_1_1 (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7]
```

Read back by a model that saw only the Lean. For any integers a and b and any natural number k , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then a raised to the power $k + 6$ minus b raised to the power $k + 6$ is congruent modulo 7 to a raised to the power k minus b raised to the power k .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_period_six (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7] := by  
  have ha1 : a ≡ (1 : ℤ) [ZMOD 7] := by  
    exact ha.trans (by norm_num [Int.ModEq])  
  have ha6 : a ^ 6 ≡ (1 : ℤ) [ZMOD 7] := by  
    exact (ha1.pow 6).trans (by norm_num [Int.ModEq])  
  have hb6 : b ^ 6 ≡ (1 : ℤ) [ZMOD 7] := by  
    exact (hb.pow 6).trans (by norm_num [Int.ModEq])  
  have hpa : a ^ (k + 6) ≡ a ^ k [ZMOD 7] := by  
    rw [pow_add]  
    calc  
      a ^ k * a ^ 6 ≡ a ^ k * 1 [ZMOD 7] := (Int.ModEq.refl (n := 7) (a ^ k)).mul ha6  
      _ = a ^ k := by ring  
  have hpb : b ^ (k + 6) ≡ b ^ k [ZMOD 7] := by  
    rw [pow_add]  
    calc  
      b ^ k * b ^ 6 ≡ b ^ k * 1 [ZMOD 7] := (Int.ModEq.refl (n := 7) (b ^ k)).mul hb6  
      _ = b ^ k := by ring  
  have hdiff : a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7] := hpa.sub hpb  
  exact hdiff
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.mh.mh.mh__bae561eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint bae561eb51a0712b

4.497 497. bounded_power_difference_mod_seven_iff

The problem

Let integers a and b be congruent to 29 and 5, respectively, modulo 7, and let n be a nonnegative integer with $n \leq 19$. Prove that $a^n - b^n$ leaves remainder 3 modulo 7 if and only if $n \equiv 1 \pmod{6}$.

Lean statement

```
theorem bounded_power_difference_mod_seven_iff (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn2 : n ≤ 19) : (a ^ n - b ^ n ≡ 3 [ZMOD 7]) ↔ n % 6 = 1 := by
```

Parent. Let integers a and b be congruent to 29 and 5, respectively, modulo 7. If $1 \leq n \leq 19$ and $n \equiv 1 \pmod{6}$, show that $a^n - b^n$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem bounded_power_difference_mod_seven (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn2 : n ≤ 19) (hres : n % 6 = 1) : a ^ n - b ^ n ≡ 3 [ZMOD 7] := by
```

What it contributes. Supplies the power-preservation and subtraction checkpoint and the bounded sufficiency direction; the child adds the reverse bounded cycle classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child strengthens the parent from a one-way implication to an iff characterization. Its reverse direction requires an exhaustive modulo-7 cycle analysis over the bounded exponent range, which the parent's proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way implication into a bounded iff characterization. Its reverse direction requires ruling out every other exponent in the bounded range, reasoning not supplied by the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: from the modular power-difference condition it must classify the bounded exponent and derive $n \bmod 6 = 1$. The parent only supplies the forward implication under that residue hypothesis, so recalling its proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2098 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_difference_mod_seven_iff.extracted_1_1 (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7])  
(hb : b ≡ 5 [ZMOD 7]) (hn₂ : n ≤ 19) : a ^ n - b ^ n ≡ 3 [ZMOD 7] ↔ n % 6 = 1
```

Read back by a model that saw only the Lean. For integers a and b and a natural number n , if a is congruent to 29 modulo 7, b is congruent to 5 modulo 7, and n is at most 19, then a raised to the n -th power minus b raised to the n -th power is congruent to 3 modulo 7 if and only if the remainder when n is divided by 6 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_power_difference_mod_seven_iff (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn₂ : n ≤ 19) : (a ^ n -  
↪ b ^ n ≡ 3 [ZMOD 7]) ↔ n % 6 = 1 := by
  have ha_pow : a ^ n ≡ (29 : ℤ) ^ n [ZMOD 7] := ha.pow n
  have hb_pow : b ^ n ≡ (5 : ℤ) ^ n [ZMOD 7] := hb.pow n
  have hdiff : a ^ n - b ^ n ≡ (29 : ℤ) ^ n - (5 : ℤ) ^ n [ZMOD 7] := ha_pow.sub hb_pow
  have hforward : n % 6 = 1 → a ^ n - b ^ n ≡ 3 [ZMOD 7] := by
    intro hres
    have hn_cases : n = 1 ∨ n = 7 ∨ n = 13 ∨ n = 19 := by
      omega
    rcases hn_cases with h | h | h | h
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
  constructor
  · intro htarget
    have hconst : (29 : ℤ) ^ n - (5 : ℤ) ^ n ≡ 3 [ZMOD 7] := hdiff.symm.trans htarget
    interval_cases n <|> norm_num [Int.ModEq] at hconst <|> norm_num
    · exact hforward
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.mh.mh__60d58253

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ce8bc703d9a577cd

4.498 498. numbertheory_728_generalized

The problem

Let a and b be integers with $a \equiv 1 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that $a^{13} - b^{13} \equiv 3 \pmod{7}$.

Lean statement

```
theorem numbertheory_728_generalized (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7]
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed integers to arbitrary integers specified only by their residue classes modulo 7. Its proof must transport congruences through exponentiation and subtraction before evaluating the resulting constant, which the parent's direct normalization does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed integers to arbitrary integers satisfying the corresponding congruences. Its proof must use congruence preservation under powers and subtraction before evaluating the constant residue, which the parent's direct norm_num proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed integers to arbitrary integers satisfying modular hypotheses. Its proof must transport congruence through powers and subtraction before evaluating the constant case, which the parent's direct norm_num proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1781 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem numbertheory_728_generalized.extracted_1.1 (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :
  a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers a and b , if a is congruent to 1 modulo 7 and b is congruent to 5 modulo 7, then a raised to the 13th power minus b raised to the 13th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem numbertheory_728_generalized (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
  have hpow_a : a ^ 13 ≡ (1 : ℤ) ^ 13 [ZMOD 7] := ha.pow 13
  have hpow_b : b ^ 13 ≡ (5 : ℤ) ^ 13 [ZMOD 7] := hb.pow 13
  have hdiff : a ^ 13 - b ^ 13 ≡ (1 : ℤ) ^ 13 - 5 ^ 13 [ZMOD 7] := hpow_a.sub hpow_b
  have hconst : (1 : ℤ) ^ 13 - 5 ^ 13 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdiff.trans hconst
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__b4bc93a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 694cb5f1a4d6738e

4.499 499. mod_seven_pow_sub_one

The problem

If an integer is congruent to 1 modulo 7, show that its thirteenth power minus 1 is congruent to 0 modulo 7.

Lean statement

```
theorem mod_seven_pow_sub_one (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's fixed numerical computation with a quantified modular-arithmetic theorem. Its proof must transport the congruence through exponentiation and subtraction, which the parent's direct 'norm_num' computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed numerical congruence to an arbitrary integer congruent to 1 modulo 7 and requires transporting the congruence through the 13th power and subtraction. The parent's direct computation by norm_num does not provide this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's fixed numerical congruence to an arbitrary integer congruent to 1 modulo 7, requiring propagation of the hypothesis through exponentiation and subtraction. The parent's direct normalization proof supplies none of this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2362 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mod_seven_pow_sub_one.extracted_1_1 (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer a , if a is congruent to 1 modulo 7, then a raised to the 13th power minus 1 is congruent to 0 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mod_seven_pow_sub_one (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7] := by
  have hpow : a ^ 13 ≡ (1 : ℤ) ^ 13 [ZMOD 7] := h.pow 13
  have hpow' : a ^ 13 ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hpow
  have hsub : a ^ 13 - 1 ≡ (1 : ℤ) - 1 [ZMOD 7] := hpow'.sub_right 1
  simpa using hsub
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__c4656633

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c465663375f7b042

4.500 500. modular_power_family

The problem

For every nonnegative integer k with $k \equiv 1 \pmod{6}$, show that $29^k - 5^k \equiv 3 \pmod{7}$.

Lean statement

```
theorem modular_power_family : ∀ k : ℕ, k % 6 = 1 → (29 : ℤ)^k - 5^k ≡ 3 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed exponent 13 to every natural exponent congruent to 1 modulo 6. Proving this requires periodic exponent reasoning modulo 7 for both bases, which the parent's direct norm_num computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent to all natural exponents congruent to 1 modulo 6. Its proof must establish exponent-periodicity separately for both bases and transport the difference, which is not supplied by the parent's one-step norm_num proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every natural k congruent to 1 modulo 6. Its proof requires periodicity of powers modulo 7 and arithmetic on the exponent, which the parent's direct normalization proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2412 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_family.extracted_1_1 : ∀ (k : ℕ), k % 6 = 1 → 29 ^ k - 5 ^ k ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number k , if the remainder when k is divided by 6 is 1, then 29 raised to the k -th power minus 5 raised to the k -th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_family :  $\forall k : \mathbb{N}, k \% 6 = 1 \rightarrow (29 : \mathbb{Z})^k - 5^k \equiv 3 \pmod{7}$  := by
  intro k hk
  have h29 :  $(29 : \mathbb{Z})^k \equiv 29^1 \pmod{7}$  := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := k) (y := 1) (n := 29)
    · norm_num
    · omega
    · omega
    · norm_num
  have h5 :  $(5 : \mathbb{Z})^k \equiv 5^1 \pmod{7}$  := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := k) (y := 1) (n := 5)
    · norm_num
    · omega
    · omega
    · norm_num
  have hdiff :  $(29 : \mathbb{Z})^k - 5^k \equiv 29^1 - 5^1 \pmod{7}$  := h29.sub h5
  calc
     $(29 : \mathbb{Z})^k - 5^k \equiv 29^1 - 5^1 \pmod{7}$  := hdiff
     $\equiv 3 \pmod{7}$  := by norm_num [Int.ModEq]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__c7a850d6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c7a850d614e0edd2

4.501 501. nat_digits_injective_general

The problem

For any base and natural numbers, two numbers with the same digit representation are equal, and equal numbers have the same digit representation.

Lean statement

```
theorem nat_digits_injective_general
  (b x y : ℕ) :
  Nat.digits b x = Nat.digits b y ↔ x = y := by
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed base-3, fixed-digit reconstruction to an injectivity equivalence for arbitrary bases and numbers. It also requires a new converse argument via `congrArg`, while the parent's final numerical evaluation cannot apply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed base-3 evaluation into a full injectivity theorem for digit representations and adds a reverse implication. The parent's numerical normalization proof does not establish equality of arbitrary numbers from equal digit lists.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from one fixed ternary digit expansion to injectivity of digit representations for arbitrary bases and numbers. Its proof requires a bidirectional equivalence and reconstructs both numbers via `'Nat.ofDigits'`, which the parent's concrete calculation does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2519 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nat_digits_injective_general.extracted_1_1 (b x y : ℕ) : b.digits x = b.digits y ↔ x = y
```

Read back by a model that saw only the Lean. *For all natural numbers b , x , and y , the sequence of digits of x in base b is equal to the sequence of digits of y in base b if and only if x equals y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem nat_digits_injective_general
  (b x y : ℕ) :
  Nat.digits b x = Nat.digits b y ↔ x = y := by
  constructor
  · intro h
    have hx : Nat.ofDigits b (Nat.digits b x) = x := Nat.ofDigits.digits b x
    have hy : Nat.ofDigits b (Nat.digits b y) = y := Nat.ofDigits.digits b y
    have hrepr : Nat.ofDigits b (Nat.digits b x) = Nat.ofDigits b (Nat.digits b y) :=
      congrArg (Nat.ofDigits b) h
    have hxy : x = y := by
      calc
        x = Nat.ofDigits b (Nat.digits b x) := hx.symm
        _ = Nat.ofDigits b (Nat.digits b y) := hrepr
        _ = y := hy
    exact hxy
  · intro h
    exact congrArg (Nat.digits b) h
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__8e90b5ce

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8e90b5cecf5f8291

4.502 502. digits_2221_value

The problem

For every natural number b and natural number x whose base- b digits, listed from least significant to most significant, are $[2, 2, 2, 1]$, show that $x = 2 + b(2 + b(2 + b \cdot 1))$.

Lean statement

```
theorem digits_2221_value (b x : ℕ) (h : Nat.digits b x = [2, 2, 2, 1]) : x = 2 + b * (2 + b * (2 + b * 1)) := by
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: it derives the positional-value formula for every base b from the digit representation, rather than evaluating only the fixed base 3 case. The symbolic expansion must remain valid independently of the parent's numerical computation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the base from 3 to an arbitrary natural b and replaces the concrete value with a symbolic positional-value formula. The parent's specific numerical normalization to 53 does not establish this parameterized identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: instead of computing the fixed-base value 53, it derives the symbolic value for every base b satisfying the digit hypothesis. The parent's numerical normalization no longer applies, so the conclusion is not recoverable by recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1702 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem digits_2221_value.extracted_1_1 (b x : ℕ) (h : b.digits x = [2, 2, 2, 1]) :  
  x = 2 + b * (2 + b * (2 + b * 1))
```

Read back by a model that saw only the Lean. For natural numbers b and x , if the digits of x in base b are $[2, 2, 2, 1]$, then $x = 2 + b * (2 + b * (2 + b * 1))$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem digits_2221_value (b x : ℕ) (h : Nat.digits b x = [2, 2, 2, 1]) :  
  x = 2 + b * (2 + b * (2 + b * 1)) := by  
  have hrecover : Nat.ofDigits b (Nat.digits b x) = x := Nat.ofDigits_digits b x  
  have hdigits : Nat.ofDigits b (Nat.digits b x) = Nat.ofDigits b [2, 2, 2, 1] := by  
    rw [h]  
  have hrepr : x = Nat.ofDigits b [2, 2, 2, 1] := by  
    calc  
      x = Nat.ofDigits b (Nat.digits b x) := hrecover.symm  
      _ = Nat.ofDigits b [2, 2, 2, 1] := hdigits  
  calc  
    x = Nat.ofDigits b [2, 2, 2, 1] := hrepr  
    _ = 2 + b * (2 + b * (2 + b * 1)) := by simp [Nat.ofDigits_cons]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__bcb2847

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 5053f27dbea21c3f

4.503 503. base_three_digits_value

The problem

If the base-three digits of a natural number are $[a, b, c, d]$, prove that the number is $a + 3b + 9c + 27d$.

Lean statement

```
theorem base_three_digits_value
  (x a b c d : ℕ)
  (h : Nat.digits 3 x = [a, b, c, d]) :
  x = a + 3 * b + 9 * c + 27 * d
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed digit tuple to symbolic digits and proves a positional-value identity, requiring symbolic simplification and algebra rather than merely evaluating the parent's constant expression. Its hypotheses are used and the proof is complete.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed digit sequence to symbolic digits and must derive and normalize a symbolic base-3 expansion, requiring algebraic reasoning beyond the parent's final numeric normalization. It is not a decoration or recall wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed digit list and must symbolically expand its base-3 value for arbitrary digits. The parent's numeral-specific 'norm_num' step no longer applies; the child requires a general 'ofDigits' expansion and algebraic normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2551 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem base_three_digits_value.extracted_1_1 (x a b c d : ℕ) (h : Nat.digits 3 x = [a, b, c, d]) :  
  x = a + 3 * b + 9 * c + 27 * d
```

Read back by a model that saw only the Lean. For natural numbers x , a , b , c , and d , if the base-3 digit expansion of x is the four-digit list $[a, b, c, d]$, then $x = a + 3b + 9c + 27d$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem base_three_digits_value  
  (x a b c d : ℕ)  
  (h : Nat.digits 3 x = [a, b, c, d]) :  
  x = a + 3 * b + 9 * c + 27 * d := by  
  have hrepr : x = Nat.ofDigits 3 (Nat.digits 3 x) :=  
    (Nat.ofDigits_digits 3 x).symm  
  have hdigits : Nat.ofDigits 3 (Nat.digits 3 x) = Nat.ofDigits 3 [a, b, c, d] := by  
    rw [h]  
  have hvalue : Nat.ofDigits 3 [a, b, c, d] = a + 3 * b + 9 * c + 27 * d := by  
    simp [Nat.ofDigits_cons] <|> ring  
  calc  
    x = Nat.ofDigits 3 (Nat.digits 3 x) := hrepr  
    _ = Nat.ofDigits 3 [a, b, c, d] := hdigits  
    _ = a + 3 * b + 9 * c + 27 * d := hvalue
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__e2008f66

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e2008f661757762c

4.504 504. mod_eleven_succ_mul

The problem

For every natural number n , what is the remainder when $11n + 1$ is divided by 11? Show that it is 1.

Lean statement

```
theorem mod_eleven_succ_mul (n : ℕ) : (11 * n + 1) % 11 = 1 := by
```

Parent. What is the remainder when 2003 is divided by 11? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_961 : 2003 % 11 = 1 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed computation to all natural n , requiring the prover to reason about the divisibility structure of $11*n+1$ rather than evaluate $2003 \% 11$. Its proof uses symbolic rearrangement and arithmetic reasoning, not the parent's numerical proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete computation to all natural n , so the parent's 'norm_num' proof cannot be reused. Proving the symbolic modular identity requires algebraic rearrangement and a general arithmetic argument via 'ring' and 'omega'.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete numeral computation to all $n : \mathbb{N}$, requiring modular arithmetic and normalization of $11 * n + 1$. The parent's norm_num proof supplies no reasoning for this parameterized claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1832 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mod_eleven_succ_mul.extracted_1_1 (n : ℕ) : (11 * n + 1) % 11 = 1
```

Read back by a model that saw only the Lean. For every natural number n , the remainder when $11n + 1$ is divided by 11 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mod_eleven_succ_mul (n : ℕ) : (11 * n + 1) % 11 = 1 := by
  have hdecomp : 11 * n + 1 = n * 11 + 1 := by
    ring
  rw [hdecomp]
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_961__mh__71d2132c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint ed6aa6664ef6cfd2

4.505 505. mathd_numbertheory_961_mutation

The problem

Show that adding any multiple of 11 to 2003 leaves remainder 1 when divided by 11.

Lean statement

theorem mathd_numbertheory_961_mutation (n : ℕ) : (2003 + 11 * n) % 11 = 1 := by

Parent. What is the remainder when 2003 is divided by 11? Show that it is 1.

Parent in Lean

theorem mathd_numbertheory_961 : 2003 % 11 = 1 := by

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed remainder to an arbitrary natural multiple of 11. Its proof must combine the computed decomposition of 2003 with symbolic modular arithmetic, which the parent's norm_num proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the dividend by an arbitrary natural number and proves the remainder remains 1, requiring symbolic modular arithmetic beyond the parent's closed computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed remainder to every natural parameter n. The parent's norm_num proof supplies only the base computation; the child additionally requires symbolic modular reasoning to show adding any multiple of 11 preserves the remainder.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2714 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

theorem mathd_numbertheory_961_mutation.extracted_1_1 (n : ℕ) : (2003 + 11 * n) % 11 = 1

Read back by a model that saw only the Lean. For every natural number n, the remainder when 2003 + 11n is divided by 11 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_961_mutation (n : ℕ) : (2003 + 11 * n) % 11 = 1 := by
  have h : 2003 = 11 * 182 + 1 := by
    norm_num
  rw [h]
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_961__mh__df101654

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint df101654ec090306

4.506 506. mathd_numbertheory_99_two_representatives

The problem

If $n < 94$ and $2n \equiv 15 \pmod{47}$, then n is either 31 or 78.

Lean statement

```
theorem mathd_numbertheory_99_two_representatives (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) : n = 31 ∨ n = 78
```

Parent. Among the natural numbers less than 47, the congruence $2n \equiv 15 \pmod{47}$ has the unique solution $n = 31$.

Parent in Lean

```
theorem mathd_numbertheory_99_bounded : ∃ n : ℕ, n = 31 ∧ n < 47 ∧ 2 * n ≡ 15 [MOD 47] ∧ ∀ m : ℕ, m < 47 → 2 * m ≡ 15 [MOD 47] → m = n
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The relaxed bound $n < 94$ requires deriving both possible representatives of the residue class, rather than using the parent's $m < 47$ step to identify the residue with the number itself. The proof therefore introduces a genuinely new bounded-case conclusion and reasoning obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded uniqueness claim into a two-representative classification over the larger range $n < 94$. The parent's final step identifying a residue with the number itself no longer applies, and the new representative 78 must be derived from the modular residue and the expanded bound.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the uniqueness claim into a two-representative characterization under the larger bound $n < 94$. The parent's final step relies on $m < 47$ to identify the residue with the number itself, whereas the child must perform a new bounded residue case split yielding 31 or 78.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2712 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_99_two_representatives.extracted_1_1 (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) :  
  n = 31 ∨ n = 78
```

Read back by a model that saw only the Lean. For a natural number n less than 94, if $2n$ is congruent to 15 modulo 47, then n equals 31 or n equals 78.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_99_two_representatives (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) : n = 31 ∨ n = 78 := by
  have hmod : (2 * n) % 47 = 15 % 47 := h
  have hcanonical : n % 47 = 31 := by
    norm_num at hmod ⊢
    omega
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_99__mh.mh__3a778cec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 3a778cec4799c418

4.507 507. doubling_cancel_mod_47

The problem

Show that doubling is cancellable modulo 47: if $2n \equiv 2r \pmod{47}$, then $n \equiv r \pmod{47}$.

Lean statement

```
theorem doubling_cancel_mod_47 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47] := by
```

Parent. Solve the congruence $2n \equiv 15 \pmod{47}$, as a residue modulo 47. (Give an answer between 0 and 46.) Show that it is 31.

Parent in Lean

```
theorem mathd_numbertheory_99 (n : ℕ) (h₀ : 2 * n ≡ 15 [MOD 47]) : n ≡ 31 [MOD 47] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent from solving one congruence to proving cancellation of the factor 2 modulo 47 for arbitrary n and r . Its proof requires a coprimality-based cancellation lemma, which the parent's direct arithmetic argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed residue equation to cancellation of a factor modulo 47 for arbitrary n and r . Its proof requires establishing coprimality and applying a cancellation theorem, which the parent's omega-based proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed congruence-solving task with a general cancellation theorem modulo 47, requiring a coprimality argument and a modular cancellation lemma. The parent's direct normalization and omega proof do not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2439 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem doubling_cancel_mod_47.extracted_1_1 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47]
```

Read back by a model that saw only the Lean. For natural numbers n and r , if $2n$ is congruent to $2r$ modulo 47, then n is congruent to r modulo 47.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem doubling_cancel_mod_47 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47] := by
  have hcop : Nat.gcd 47 2 = 1 := by
    norm_num
  have hcancel : n ≡ r [MOD 47] :=
    Nat.ModEq.cancel_left_of_coprime hcop h
  exact hcancel
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_99__mh__9367373a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 9367373a98ffd969

4.508 508. choose_translated_square_range_and_unique_min

The problem

For positive integers n and k , let $C = \binom{n}{k}$ and $D = \binom{n-1}{k} + \binom{n-1}{k-1}$. Show that every real y is attained by $(z - C)^2 + D$ exactly when $y \geq D$, and that its minimum D occurs only at $z = C$.

Lean statement

```
theorem choose_translated_square_range_and_unique_min (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  (∀ y : ℝ,
    ((∃ z : ℝ, (z - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) = y) ↔
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) ≤ y) ∧
    ((x - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) =
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
      x = (Nat.choose n k : ℝ)))
```

Parent. For positive integers n and k , show that $\lfloor \left(x - \binom{n}{k}\right)^2 + \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1} \rfloor$ if and only if $x = \binom{n}{k}$, for real x .

Parent in Lean

```
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  ((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
  x = (Nat.choose n k : ℝ)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine characterization of the entire range of the translated square, requiring a new existence proof via nonnegative differences and square roots; this is not supplied by the parent's pointwise minimum equivalence. It also retains the unique-minimizer claim, but the range component independently creates a harder proof obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range characterization: every value at least the Pascal-sum constant must be attained, requiring a square-root construction, while also proving the minimum's uniqueness. This obligation is not supplied by the parent's fixed-level equivalence proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a single equality characterization to a full range characterization of the translated square, requiring a new square-root existence argument, while also retaining the minimum characterization. The parent's proof does not supply the surjectivity/range construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2465 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_translated_square_range_and_unique_min.extracted_1_1 : ∀ (n k : ℕ),
  0 < n →
  0 < k →
  ∀ (x y : ℝ),
    ((∃ z, (z - ↑(n.choose k)) ^ 2 + ↑((n - 1).choose k + (n - 1).choose (k - 1)) = y) ↔
      ↑((n - 1).choose k + (n - 1).choose (k - 1)) ≤ y) ∧
    ((x - ↑(n.choose k)) ^ 2 + ↑((n - 1).choose k + (n - 1).choose (k - 1)) =
      ↑((n - 1).choose k + (n - 1).choose (k - 1)) ↔
      x = ↑(n.choose k))
```

Read back by a model that saw only the Lean. For all natural numbers n and k with $n > 0$ and $k > 0$, and for all real numbers x and y , both of the following hold: (1) there exists a real number z such that $(z - n.\text{choose } k)^2 + ((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) = y$ if and only if $((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) \leq y$; and (2) $(x - n.\text{choose } k)^2 + ((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) = ((n-1).\text{choose } k + (n-1).\text{choose } (k-1))$ if and only if $x = n.\text{choose } k$, where the natural-number binomial coefficients and sums are regarded as real numbers in the real-valued expressions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem choose_translated_square_range_and_unique_min (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  (∀ y : ℝ,
    ((∃ z : ℝ, (z - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) = y) ↔
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) ≤ y) ∧
    (((x - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) =
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
      x = (Nat.choose n k : ℝ))) := by
  have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
    simpa [Nat.add_comm] using Nat.choose_eq_choose_pred_add hn hk
  have hCD : (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by
    exact_mod_cast hpascal
  intro y
  constructor
  · constructor
    · rintro {z, hz}
      nlinarith [sq_nonneg (z - (Nat.choose n k : ℝ))]
    · intro hy
      have hs : 0 ≤ y - ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by
        linarith
      have hsqr := Real.sq_sqrt hs
      refine ((Nat.choose n k : ℝ) + Real.sqrt (y - ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)), ?_)
      nlinarith
  · constructor
    · intro h
      nlinarith [sq_nonneg (x - (Nat.choose n k : ℝ))]
    · intro hx
      rw [hx]
      simp
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me.me.mh__425f866a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 425f866a9309b486

4.509 509. complementary_pascal_sum_eq_n_iff

The problem

For integers $n \geq 2$ and $0 < k \leq n$, characterize exactly when $\binom{n-1}{k} + \binom{n-1}{k-1} = n$.

Lean statement

```
theorem complementary_pascal_sum_eq_n_iff (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = n ↔ k = 1 ∨ k = n - 1
```

Parent. For positive integers n and k with $k \leq n$, characterize exactly when $\binom{n-1}{k} + \binom{n-1}{k-1} = 1$.

Parent in Lean

```
theorem complementary_pascal_sum_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = 1 ↔ k = n
```

What it contributes. Reuses the parent's Pascal expansion and bounded positive-index setup, while replacing the unit-value classification with an inductive strict lower-bound argument for interior coefficients.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the target from a unit binomial value to the nontrivial classification of indices where $\binom{n}{k} = n$. Its proof requires a new interior-binomial lower bound and excludes the endpoint $k=n$, so the parent's choose-equals-one argument cannot solve it.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from the sum being 1 to being n , yielding a new two-endpoint classification. Its proof requires a substantive lower-bound argument for interior binomial coefficients via induction, which is not supplied by the parent's Pascal-collapse and unit-value reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transforms the Pascal sum into a binomial coefficient equal to n and proves a new classification: only the two endpoint indices $k = 1$ or $k = n - 1$ qualify. Its induction and interior lower-bound argument require mathematics absent from the parent's unit-value characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2655 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_pascal_sum_eq_n_iff.extracted_1_1 (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :  
(n - 1).choose k + (n - 1).choose (k - 1) = n ↔ k = 1 ∨ k = n - 1
```

Read back by a model that saw only the Lean. For natural numbers n and k satisfying $2 \leq n$, $0 < k$, and $k \leq n$, the equality $\binom{n-1}{k} + \binom{n-1}{k-1} = n$ holds if and only if $k = 1$ or $k = n - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem complementary_pascal_sum_eq_n_iff (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :  
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = n ↔ k = 1 ∨ k = n - 1 := by  
  have hpascal : Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := by  
    have hn' : n - 1 + 1 = n := by omega  
    have hk' : k - 1 + 1 = k := by omega  
    calc  
      Nat.choose n k = Nat.choose (n - 1 + 1) (k - 1 + 1) := by rw [hn', hk']  
      _ = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) (k - 1 + 1) := by  
        simpa [Nat.succ_eq_add_one] using Nat.choose_succ_succ (n - 1) (k - 1)  
      _ = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := by rw [hk']  
  have hcollapse : Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = Nat.choose n k := by  
    rw [Nat.add_comm, hpascal]  
  have hbig : ∀ m : ℕ, 2 ≤ m → ∀ j : ℕ, 2 ≤ j → j ≤ m - 2 → m < Nat.choose m j := by  
    intro m  
    induction m with  
    | zero => intro hm; omega  
    | succ m ih =>  
      intro hm j hj hjm  
      by_cases hm3 : m = 3  
      · subst m  
        have hj' : j = 2 := by omega  
        subst j  
        norm_num [Nat.choose]  
      · have hm2 : 2 ≤ m := by omega  
        have hjpos : 0 < j := by omega  
        rw [Nat.choose_succ_left m j hjpos]  
        by_cases hj2 : j = 2  
        · subst j  
          have hprev := ih hm2 2 (by omega) (by omega)  
          have hpos : 0 < Nat.choose m 2 := Nat.choose_pos (by omega)  
          rw [Nat.choose_one_right]  
          omega  
        · have hj3 : 3 ≤ j := by omega  
          have hprev := ih hm2 (j - 1) (by omega) (by omega)  
          have hpos : 0 < Nat.choose m j := Nat.choose_pos (by omega)  
          omega  
  constructor  
  · intro h  
    have hval : Nat.choose n k = n := hcollapse.symmetr.trans h  
    by_cases hk1 : k = 1  
    · exact Or.inl hk1  
    by_cases hkn : k = n  
    · subst k  
      have hval' : Nat.choose n n = n := hval  
      have : 1 = n := by simpa using hval'  
      omega  
    by_cases hkn1 : k = n - 1  
    · exact Or.inr hkn1  
    have hk2 : 2 ≤ k := by omega  
    have hkupper : k ≤ n - 2 := by omega  
    have hgt : n < Nat.choose n k := hbig n hn k hk2 hkupper  
    omega  
  · intro h  
    rcases h with rfl | rfl  
    · rw [hcollapse]
```

```

simp
· rw [hcollapse]
have hs : Nat.choose n (n - 1) = Nat.choose n 1 := by
  simpa using (Nat.choose_symm (n := n) (k := 1) (by omega))
rw [hs]
simp

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.me.me.mh__8477755b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8477755ba5deb995

4.510 510. hockey_stick_binomial

The problem

For natural numbers n and k , show that the sum $\sum_{i=0}^n \binom{i}{k}$ equals $\binom{n+1}{k+1}$.

Lean statement

```
theorem hockey_stick_binomial (n k : ℕ) :
  (∑ i ∈ Finset.range (n + 1), Nat.choose i k) = Nat.choose (n + 1) (k + 1)
```

Parent. For predecessor indices p and q , Pascal's recursion expresses the explicitly named value $r = \text{choose } (p + 1) (q + 1)$ as $\text{choose } p (q + 1) + \text{choose } p q$.

Parent in Lean

```
theorem pascal_predecessor_surface (p q r : ℕ) (hr : r = Nat.choose (p + 1) (q + 1)) : r = Nat.choose p (q + 1) + Nat.choose p q
```

What it contributes. Supplied the Pascal recursion checkpoint, adapted from the named-value predecessor identity into the induction step of the variable-index summation theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the local Pascal recurrence into a variable-index finite-sum identity. Its induction must decompose the sum and combine the induction hypothesis with Pascal recursion, which the parent's single-step equality does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes Pascal's predecessor identity to a variable-index hockey-stick sum. Its inductive proof requires a new summation decomposition and repeatedly consumes Pascal recursion, so recalling the parent does not supply the argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: proving the hockey-stick sum requires induction, finite-sum decomposition, and Pascal recursion at each step. The parent supplies only a single local Pascal identity and does not provide the induction or summation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2056 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem hockey_stick_binomial.extracted_1_1 (n k : ℕ) :  
  ∑ i ∈ Finset.range (n + 1), i.choose k = (n + 1).choose (k + 1)
```

Read back by a model that saw only the Lean. For all natural numbers n and k , the sum of the binomial coefficients $\binom{n}{i}$ for integers i from 0 through n equals $\binom{n+1}{k+1}$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem hockey_stick_binomial (n k : ℕ) :  
  (∑ i ∈ Finset.range (n + 1), Nat.choose i k) = Nat.choose (n + 1) (k + 1) := by  
  induction n generalizing k with  
  | zero =>  
    cases k with  
    | zero => simp  
    | succ k => simp [Nat.choose]  
  | succ n ih =>  
    have hsum :  
      (∑ i ∈ Finset.range (n + 1 + 1), Nat.choose i k) =  
        (∑ i ∈ Finset.range (n + 1), Nat.choose i k) + Nat.choose (n + 1) k := by  
      rw [Finset.sum_range_succ]  
    have hpascal :  
      Nat.choose (n + 1 + 1) (k + 1) =  
        Nat.choose (n + 1) (k + 1) + Nat.choose (n + 1) k := by  
      simpa [Nat.add_assoc, Nat.add_comm, Nat.add_left_comm] using  
        (Nat.choose_succ_succ (n + 1) k)  
    calc  
      (∑ i ∈ Finset.range (Nat.succ n + 1), Nat.choose i k) =  
        (∑ i ∈ Finset.range (n + 1 + 1), Nat.choose i k) := by  
          congr 2 <|> omega  
      _ = (∑ i ∈ Finset.range (n + 1), Nat.choose i k) + Nat.choose (n + 1) k := hsum  
      _ = Nat.choose (n + 1) (k + 1) + Nat.choose (n + 1) k := by rw [ih k]  
      _ = Nat.choose (n + 1 + 1) (k + 1) := hpascal.symm  
      _ = Nat.choose (Nat.succ n + 1) (k + 1) := by congr 2 <|> omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.mh__04540a98

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 04540a9858d9117f

4.511 511. translated_power_mul_quotient_iff_modEq_zero

The problem

Let p be prime, let n, a, k be integers, and let $e > 0$. Prove that there is a unique integer q satisfying $n^e a = pq$ if and only if $(n + kp)^e a \equiv 0 \pmod{p}$.

Lean statement

```
theorem translated_power_mul_quotient_iff_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)]
```

Parent. Let p be prime, let n, a, k be integers with $p \nmid a$, and let $e > 0$. Prove that there is a unique integer q satisfying $n^e a = pq$ if and only if $(n + kp)^e a \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem unique_power_mul_quotient_iff_translated_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬ ((↑p : ℤ) ∣ a)) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)]
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens the hypotheses by dropping both positivity of e and the nondivisibility condition on a . Its proof uses direct modular transport and uniqueness from $p \neq 0$, whereas the parent's argument depends essentially on those hypotheses to cancel a and extract $p \mid n$.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the theorem by removing both the positivity assumption on the exponent and the nondivisibility assumption on a . Its proof uses direct congruence transport and uniqueness from the nonzero modulus, avoiding the parent's cancellation argument entirely.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes both positivity of the exponent and the nondivisibility hypothesis, and its proof must replace the parent's cancellation argument with congruence transport plus direct uniqueness from the nonzero modulus. This is a genuine generalization that cannot be obtained by simply recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2679 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_power_mul_quotient_iff_modEq_zero.extracted_1_1 (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) : (∃! q, n ^ e * a = ↑p * q) ↔ (n + k * ↑p) ^ e * a ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For integers n , a , and k , natural numbers p and e , if p is prime and e is positive, then there exists a unique integer q such that $n^e a = p q$ if and only if $(n + k p)^e a$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_power_mul_quotient_iff_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] := by
  have hpne : (↑p : ℤ) ≠ 0 := by
    exact_mod_cast hp.ne_zero
  have hmod_zero (x : ℤ) :
    x ≡ 0 [ZMOD (↑p : ℤ)] ↔ (↑p : ℤ) ∣ x := by
    simpa using (Int.modEq_zero_iff_dvd (a := x) (n := (↑p : ℤ)))
  have hbase : n + k * (↑p : ℤ) ≡ n [ZMOD (↑p : ℤ)] := by
    apply (Int.modEq_iff_dvd).2
    refine ⟨-k, ?_⟩
    ring
  have htransport :
    (n + k * (↑p : ℤ)) ^ e * a ≡ n ^ e * a [ZMOD (↑p : ℤ)] :=
    (hbase.pow e).mul_right a
  constructor
  · rintro ⟨q, hq, _⟩
    have hdiv : (↑p : ℤ) ∣ n ^ e * a := ⟨q, hq⟩
    have hnzero : n ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] := (hmod_zero _).2 hdiv
    exact htransport.trans hnzero
  · intro htranslated
    have hnzero : n ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] :=
      htransport.symm.trans htranslated
    have hdiv : (↑p : ℤ) ∣ n ^ e * a := (hmod_zero _).1 hnzero
    rcases hdiv with ⟨q, hq⟩
    refine ⟨q, hq, ?_⟩
    intro y hy
    apply mul_left_cancel₀ hpne
    calc
      (↑p : ℤ) * y = n ^ e * a := hy.symm
      _ = (↑p : ℤ) * q := hq
```

Operator. mutation / mutation_hard **Lineage depth.** 4 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh.me.mh__7fa8c862

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ee4ea013f9536d7c

4.512 512. prime_dvd_iff_coupled_product_bounded_power_modEq_zero

The problem

For a prime number p , integers n, k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n(n + kp))^e \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p]
```

Parent. For a prime number p , integers n and k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n + kp)^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_translated_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n + k * (↑p : ℤ)) ^ e ≡ 0 [ZMOD p]
```

What it contributes. Supplied the prime-power divisibility and translated-factor cancellation pattern, generalized here to the coupled product $n*(n+k*p)$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is not a recall of the parent's translated-power argument: its reverse direction must factor prime divisibility of the coupled product, then handle either factor and cancel the translated term. This introduces genuinely new prime-factor reasoning while preserving essential hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The coupled product changes the reverse direction substantially: from divisibility of its power, the proof must use primality to split divisibility across the two factors, then handle both branches before cancellation. This is not recoverable by simply recalling the parent's translated-power argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The coupled product introduces a genuinely new zero-product argument: from prime divisibility of the product, the proof must split divisibility across its two factors and then eliminate the translated factor. The parent's direct translation-and-power argument does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2630 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero.extracted_1_1 (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) (he_bound : e ≤ B) : ↑p ∣ n ↔ (n * (n + k * ↑p)) ^ e ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For integers n and k and natural numbers p , e , and B , if p is prime, e is positive, and $e \leq B$, then the integer p divides n if and only if the e -th power of $n(n + kp)$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p] := by
  rw [Int.modEq_zero_iff_dvd]
  have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
  have hkp : (↑p : ℤ) ∣ k * (↑p : ℤ) := by
    exact dvd_mul_of_dvd_right (dvd_refl (↑p : ℤ)) k
  constructor
  · intro hdiv
    have hproduct : (↑p : ℤ) ∣ n * (n + k * (↑p : ℤ)) := by
      exact dvd_mul_of_dvd_left hdiv _
    exact dvd_pow hproduct he_ne
  · intro hpower
    have hproduct : (↑p : ℤ) ∣ n * (n + k * (↑p : ℤ)) := by
      exact Int.Prime.dvd_pow' hp hpower
    have hproduct_nat : p ∣ (n * (n + k * (↑p : ℤ))).natAbs :=
      (Int.natCast_dvd).mpr hproduct
    rw [Int.natAbs_mul] at hproduct_nat
    rcases hp.dvd_mul.mpr hproduct_nat with hdiv | htranslated
    · exact (Int.natCast_dvd).mpr hdiv
    · have htranslated_int : (↑p : ℤ) ∣ n + k * (↑p : ℤ) :=
      (Int.natCast_dvd).mpr htranslated
      have hcancel : (↑p : ℤ) ∣
        (n + k * (↑p : ℤ)) - k * (↑p : ℤ) := by
        exact dvd_sub htranslated_int hkp
      convert hcancel using 1 <|> ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh__0846e862

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 00c7dcccc0b0df4f

4.513 513. prime_dvd_iff_power_mul_factor_modEq_zero

The problem

Let p be prime, let n and a be integers with $p \nmid a$, and let $e > 0$. Show that $p \mid n$ if and only if $n^e a \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_power_mul_factor_modEq_zero
  (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬ ((↑p : ℤ) ∣ a)) :
  (↑p : ℤ) ∣ n ↔ n ^ e * a ≡ 0 [ZMOD p]
```

Parent. For a prime number p , an integer n , and a positive odd exponent e with $e \leq B$, show that p divides n if and only if $n^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_bounded_odd_power_modEq_zero
  (n : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ n ^ e ≡ 0 [ZMOD p]
```

What it contributes. Reused the parent's prime-modulus and positive-power framework, while adding a product-divisibility split and an explicit nondivisibility elimination checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The reverse implication requires genuinely new reasoning: extracting divisibility of n^e from divisibility of $n^e \cdot a$ using primality and the essential nondivisibility hypothesis on a , then descending from a power to n . The parent's proof supplies only the power-divisibility equivalence and cannot establish this product cancellation step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely requires using the nondivisibility of the auxiliary factor and Euclid's lemma to infer that p divides n^e from p dividing $n^e \cdot a$. This factor-cancellation argument is absent from the parent proof, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new reverse-direction argument: from divisibility of $n^e \cdot a$, it must split the prime divisor across the factors and use the nondivisibility hypothesis on a before extracting divisibility of n^e . This factor-cancellation reasoning is absent from the parent proof and the added hypothesis is essential.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2640 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_power_mul_factor_modEq_zero.extracted_1_1 (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
(hfactor : ¬(↑p ∣ a) : ↑p ∣ n ↔ n ^ e * a ≡ 0 [ZMOD ↑p])
```

Read back by a model that saw only the Lean. For integers n and a , and natural numbers p and e , if p is prime, e is positive, and p does not divide a , then p divides n if and only if n raised to the e -th power times a is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_power_mul_factor_modEq_zero
  (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬((↑p : ℤ) ∣ a)) :
  (↑p : ℤ) ∣ n ↔ n ^ e * a ≡ 0 [ZMOD p] := by
  rw [Int.modEq_zero_iff_dvd]
  have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
  constructor
  · intro hn
    have hpow : (↑p : ℤ) ∣ n ^ e := dvd_pow hn he_ne
    exact dvd_mul_of_dvd_left hpow a
  · intro h
    have hprod : p ∣ (n ^ e).natAbs * a.natAbs := by
      simpa [Int.natAbs_mul] using (Int.natCast_dvd.mp h)
    rcases hp.dvd_mul.mp hprod with hpow | ha
    · have hpow_int : (↑p : ℤ) ∣ n ^ e := by
        apply Int.natCast_dvd.mpr
        simpa [Int.natAbs_pow] using hpow
      exact Int.Prime.dvd_pow' hp hpow_int
    · exfalso
      apply hfactor
      exact Int.natCast_dvd.mpr ha
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh__3044da59

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3044da599b1e48ce

4.514 514. cubic_quotient_expansion

The problem

Let x be an integer congruent to 5 modulo 7. Show that there is an integer q such that $x = 7q + 5$ and

$$x^3 + 1 = 7(49q^3 + 105q^2 + 75q + 18).$$

Lean statement

```
theorem cubic_quotient_expansion (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ∃ q : ℤ, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18)
```

Parent. Let x be an integer congruent to 5 modulo 7. For $0 \leq k \leq 3$, show that 7 divides $x^k + 1$ exactly when $k = 3$.

Parent in Lean

```
theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to an explicit quotient representation together with a cubic polynomial identity. Proving the quotient decomposition and symbolic expansion requires reasoning absent from the parent's bounded exponent case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded divisibility equivalence into an existential quotient representation and an explicit cubic polynomial identity. Its proof requires extracting an integer quotient from the modular hypothesis and expanding the cubic, neither of which is supplied by the parent's finite exponent case split.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded divisibility characterization into an existential quotient decomposition plus an explicit cubic polynomial identity. Its proof requires extracting an integer quotient from the congruence and carrying out symbolic expansion, neither of which is supplied by the parent's finite residue check.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_quotient_expansion.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) :
  ∃ q, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18)
```

Read back by a model that saw only the Lean. For every integer x such that x is congruent to 5 modulo 7, there exists an integer q such that $x = 7q + 5$ and $x^3 + 1 = 7(49q^3 + 105q^2 + 75q + 18)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem cubic_quotient_expansion (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ∃ q : ℤ, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 *
  q + 18) := by
  have hdiv : (7 : ℤ) ∣ 5 - x := by
    rw [← Int.modEq_iff_dvd]
    exact h
  have hdecomp : ∃ q : ℤ, x = 7 * q + 5 := by
    rcases hdiv with ⟨q, hq⟩
    refine ⟨-q, ?_⟩
    nlinarith
  rcases hdecomp with ⟨q, hq⟩
  have hexpand : x ^ 3 + 1 = (7 * q + 5) ^ 3 + 1 := by
    rw [hq]
  have hpoly : (7 * q + 5) ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18) := by
    ring
  exact ⟨q, hq, hexpand.trans hpoly⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh.me.mh__c974a427

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c974a42724c65ef1

4.515 515. not_dvd_pow_fifteen_of_modEq_five

The problem

Let x be an integer congruent to 5 modulo 7. Show that 7 does not divide x^{15} .

Lean statement

```
theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15
```

Parent. Show that 7 does not divide 5^{15} .

Parent in Lean

```
theorem seven_not_dvd_five_pow_fifteen : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 15
```

What it contributes. Used the parent's divisibility-to-modular-obstruction strategy, generalized from the fixed base 5\$ to an arbitrary integer congruent to 5\$ modulo 7\$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base and requires transporting a modular congruence through the fifteenth power, then deriving a contradiction from the resulting residue. The parent's fixed-base square-criterion argument does not supply this hypothesis-dependent reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and requires transporting divisibility through the congruence hypothesis before evaluating the fixed residue. This is a different argument from the parent's square criterion and does not overlap with the retained exponent-21 theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must transport the congruence through the fifteenth power and derive a contradiction from the resulting residue. This reasoning is not supplied by the parent's fixed-base square argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2428 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem not_dvd_pow_fifteen_of_modEq_five.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ 7 ∣ x ^ 15
```

Read back by a model that saw only the Lean. For an integer x , if x is congruent to 5 modulo 7, then 7 does not divide

x raised to the 15th power.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15 := by
  intro hdiv
  have hzero : x ^ 15 ≡ 0 [ZMOD 7] := by
    rw [Int.modEq_zero_iff_dvd]
    exact hdiv
  have hpow : x ^ 15 ≡ (5 : ℤ) ^ 15 [ZMOD 7] := by
    exact h.pow 15
  have htransport : (0 : ℤ) ≡ (5 : ℤ) ^ 15 [ZMOD 7] := by
    exact hzero.symm.trans hpow
  norm_num [Int.ModEq] at htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh__c2f2c6a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c2f2c6a18458128a

4.516 516. square_mod_three_zero_iff_dvd

The problem

For an integer a , its square is congruent to 0 modulo 3 exactly when a is divisible by 3.

Lean statement

```
theorem square_mod_three_zero_iff_dvd (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Parent. Show that the square of any integer is congruent to 0 or 1 modulo 3.

Parent in Lean

```
theorem numbertheory_sqmod3in01d (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ∨ a ^ 2 ≡ 1 [ZMOD 3] := by
```

What it contributes. Uses the parent's modulo-3 remainder classification as the internal checkpoint, while changing the conclusion to an exact divisibility characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the disjunctive square-residue result into an exact divisibility characterization. Its reverse direction requires a divisibility-of-squares argument, and its forward direction must use the residue cases to exclude 1 and 2, which the parent does not establish.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's residue classification into an exact iff characterization, requiring both extraction of divisibility from the zero residue and a separate divisibility-of-squares converse. The parent's proof does not supply this equivalence.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a residue classification into an exact divisibility equivalence. Although the parent's three-residue split helps one direction, the converse requires proving that divisibility of the base yields divisibility of its square, so the parent proof does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2665 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_mod_three_zero_iff_dvd.extracted_1_1 (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Read back by a model that saw only the Lean. For every integer a , the square of a is congruent to 0 modulo 3 if and

only if 3 divides a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_mod_three_zero_iff_dvd (a : Z) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a := by
  constructor
  · intro hsq
    have hres : a % 3 = 0 ∨ a % 3 = 1 ∨ a % 3 = 2 := by
      omega
    rcases hres with hres | hres | hres
    · refine ⟨a / 3, ?_⟩
      omega
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsq
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsq
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsq
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsq
  · intro hdiv
    have hsqdiv : 3 ∣ a ^ 2 := by
      rcases hdiv with ⟨k, hk⟩
      refine ⟨3 * k ^ 2, ?_⟩
      rw [hk]
      ring
    exact Dvd.dvd.modEq_zero_int hsqdiv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__mh__f7f36d97

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint f7f36d9711eb1b4a

4.517 517. sum_succ_mul_choose_range

The problem

For every natural number n , show that $\sum_{k=0}^n (k+1) \binom{n}{k} = n \cdot 2^{n-1} + 2^n$.

Lean statement

```
theorem sum_succ_mul_choose_range (n : ℕ) :
  Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
    n * 2 ^ (n - 1) + 2 ^ n
```

Parent. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent in Lean

```
theorem numbertheory_summulnckeqnmul2pownm1 (n : ℕ) (h₀ : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the weighted sum by introducing a successor factor and requires a separate binomial-count identity, then combines both sums via distributivity. The parent's weighted-sum result helps but does not supply the new counting argument or the resulting shifted formula.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the summand from $k \cdot \text{choose}(n, k)$ to $(k+1) \cdot \text{choose}(n, k)$, requiring a separate binomial-sum identity in addition to the parent's weighted-sum result. Its proof therefore introduces genuinely new mathematics rather than merely reusing the parent's conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires an additional independent binomial identity, $\sum \text{choose } n \ k = 2^n$, and combines it with the parent's weighted-sum identity. Remembering the parent only supplies the weighted component and does not prove the shifted-sum formula.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed h_0
corpus_dedup	hash	unique against 1818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sum_succ_mul_choose_range.extracted_1_1 (n : ℕ) :
  ∑ k ∈ Finset.range (n + 1), (k + 1) * n.choose k = n * 2 ^ (n - 1) + 2 ^ n
```

Read back by a model that saw only the Lean. For every natural number n , the sum from $k = 0$ to n of $(k + 1)$ times the binomial coefficient " n choose k " equals n times 2^{n-1} plus 2^n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sum_succ_mul_choose_range (n : ℕ) :
  Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
    n * 2 ^ (n - 1) + 2 ^ n := by
  have h_weight :
    Finset.sum (Finset.range (n + 1)) (fun k => k * Nat.choose n k) =
      n * 2 ^ (n - 1) := by
    simp using Nat.sum_range_mul_choose n
  have h_count :
    Finset.sum (Finset.range (n + 1)) (fun k => Nat.choose n k) = 2 ^ n := by
    simp using Nat.sum_range_choose n
  calc
    Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
      Finset.sum (Finset.range (n + 1))
        (fun k => k * Nat.choose n k + Nat.choose n k) := by
        apply Finset.sum_congr rfl
        intro k hk
        simp [Nat.add_mul]
    _ = Finset.sum (Finset.range (n + 1)) (fun k => k * Nat.choose n k) +
      Finset.sum (Finset.range (n + 1)) (fun k => Nat.choose n k) := by
      rw [Finset.sum_add_distrib]
    _ = n * 2 ^ (n - 1) + 2 ^ n := by
      rw [h_weight, h_count]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sumkmulnckeqnmul2pownm1__mh__145c56eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d0598d47d18b5515

4.518 518. affine_weighted_binomial_sum

The problem

For positive integers n and natural numbers a, b , show that $\sum_{k=0}^n (ak+b) \binom{n}{k} = a \cdot 2^n + b \cdot 2^n$.

Lean statement

```
theorem affine_weighted_binomial_sum (n a b : ℕ) :
  (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
    a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1)
```

Parent. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent in Lean

```
theorem numbertheory_sumkmulnckeqnmul2pownm1 (n : ℕ) (h₀ : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the weighted sum from k to the affine weight $a \cdot k + b$, requiring a new unweighted binomial-sum identity and decomposition of the summand. Its proof cannot be obtained by simply recalling the parent's weighted identity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The affine weighting requires a new decomposition and an independent binomial-sum identity, in addition to the parent's weighted identity. The parameters a and b remain symbolic, so the result is not a decorative restatement or parent recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the summand and combines two distinct binomial identities: the weighted identity from the parent and a new unweighted sum identity. Proving the affine decomposition and the excluded-zero binomial sum requires mathematical obligations not supplied by merely recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2612 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_weighted_binomial_sum.extracted_1_1 (n a b : ℕ) (h₀ : 0 < n) :
  ∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose k = a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1)
```

Read back by a model that saw only the Lean. For natural numbers n , a , and b with $n > 0$, the sum over all natural numbers k satisfying $1 \leq k \leq n$ of $(a \cdot k + b)$ times the binomial coefficient $\binom{n}{k}$ equals $a \cdot (n \cdot 2^{n-1}) + b \cdot (2^n - 1)$, where subtraction and all operations are taken in the natural numbers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem affine_weighted_binomial_sum (n a b : ℕ) :
  (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
    a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1) := by
  have hset : Finset.range (n + 1) = insert 0 (Finset.Icc 1 n) := by
    ext x
    simp only [Finset.mem_range, Finset.mem_insert, Finset.mem_Icc]
  omega
  have hnot : 0 ∉ Finset.Icc 1 n := by
    simp
  have hweighted :
    (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
    calc
      (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) =
        ∑ k ∈ insert 0 (Finset.Icc 1 n), k * Nat.choose n k := by
          rw [Finset.sum_insert hnot]
        simp
      _ = ∑ k ∈ Finset.range (n + 1), k * Nat.choose n k := by
          rw [hset]
      _ = n * 2 ^ (n - 1) := Nat.sum_range_mul_choose n
  have hunweighted :
    (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) = 2 ^ n - 1 := by
    have htotal :
      (∑ k ∈ Finset.range (n + 1), Nat.choose n k) =
        1 + ∑ k ∈ Finset.Icc 1 n, Nat.choose n k := by
        calc
          (∑ k ∈ Finset.range (n + 1), Nat.choose n k) =
            ∑ k ∈ insert 0 (Finset.Icc 1 n), Nat.choose n k := by
              rw [hset]
            _ = 1 + ∑ k ∈ Finset.Icc 1 n, Nat.choose n k := by
              rw [Finset.sum_insert hnot]
            simp
    have hpow : 0 < 2 ^ n := by positivity
    rw [Nat.sum_range_choose n] at htotal
    omega
  have hdecomp :
    (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
      a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
      b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := by
    calc
      (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
        ∑ k ∈ Finset.Icc 1 n,
          (a * (k * Nat.choose n k) + b * Nat.choose n k) := by
          apply Finset.sum_congr rfl
          intro k hk
          ring
      _ = (∑ k ∈ Finset.Icc 1 n, a * (k * Nat.choose n k)) +
        (∑ k ∈ Finset.Icc 1 n, b * Nat.choose n k) := by
          rw [Finset.sum_add_distrib]
      _ = a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
        b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := by
          rw [Finset.mul_sum, Finset.mul_sum]
    calc
      (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
        a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
```

```

      b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := hdecomp
_ = a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1) := by
  rw [hweighted, hunweighted]

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sumkmulnckeqnmul2pownm1__mh__cc19245c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2ee3e7fdd68cca0c

4.519 519. Artin_exercise_6_8_1_containment

The problem

Let a and b be elements of a group. Show that a subgroup contains both a and b exactly when it contains both bab^2 and bab^3 .

Lean statement

```
theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H)
```

Parent. Prove that two elements a, b of a group generate the same subgroup as bab^2, bab^3 .

Parent in Lean

```
theorem Artin_exercise_6_8_1 {G : Type*} [Group G]
  (a b : G) : closure ({a, b} : Set G) = Subgroup.closure {b*a*b^2, b*a*b^3} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent to the parent: equality of the two subgroup closures holds exactly when every subgroup contains the original generators iff it contains the transformed generators. The universal subgroup-membership characterization substantially changes the surface while preserving the mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: two elements lie in the same generated subgroup exactly when every subgroup contains one generating pair iff it contains the Artin-transformed pair. It uses a genuine universal-subgroup characterization rather than a cosmetic restatement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent: equality of the two subgroup closures is characterized by every subgroup containing a and b exactly when it contains the two transformed generators. No hypotheses or scope are added or removed, and the universal-subgroup membership formulation is a substantive surface re-encoding.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2285 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Artin_exercise_6_8_1_containment.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (a b : G) (H : Subgroup G),  
  a ∈ H ∧ b ∈ H ↔ b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H
```

Read back by a model that saw only the Lean. For every group G , all elements a and b of G , and every subgroup H of G , a belongs to H and b belongs to H if and only if $b \cdot a \cdot b^2$ belongs to H and $b \cdot a \cdot b^3$ belongs to H .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H) := by
  intro H
  constructor
  · rintro ⟨ha, hb⟩
    constructor
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb 2)
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb 3)
  · rintro ⟨h₁, h₂⟩
    have hb_prod :
      (b * a * b ^ 2)⁻¹ * (b * a * b ^ 3) ∈ H :=
      H.mul_mem (H.inv_mem h₁) h₂
    have hb : b ∈ H := by
      simpa [pow_two, pow_succ, mul_assoc] using hb_prod
    have ha_prod :
      (b⁻¹ * (b * a * b ^ 2)) * (b⁻¹ * b⁻¹) ∈ H :=
      H.mul_mem (H.mul_mem (H.inv_mem hb) h₁)
      (H.mul_mem (H.inv_mem hb) (H.inv_mem hb))
    have ha : a ∈ H := by
      simpa [pow_two, mul_assoc] using ha_prod
    exact ⟨ha, hb⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__ms__132343b1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 132343b19aa72acf

4.520 520. normalCore_greatest_normal_subgroup_order

The problem

For a subgroup H of a group G , show that its normal core is normal and that, for every normal subgroup N , N is contained in H exactly when it is contained in the normal core of H .

Lean statement

```
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H ↔ N ≤ H.normalCore)
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent to the parent: taking $N = H.\text{normalCore}$ in the biconditional recovers $H.\text{normalCore} \leq H$, while the parent gives both directions for every normal N . The biconditional is a genuine order-theoretic re-encoding, not a rename or definitional decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: instantiating its biconditional at $N = H.\text{normalCore}$ recovers $H.\text{normalCore} \leq H$, while the parent supplies the reverse implication for every normal N . The universal order characterization is a genuine surface re-encoding of greatest normal-subgroup maximality.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: its biconditional gives both the maximality direction and $\$N \leq H.\text{normalCore} \rightarrow N \leq H\$$; instantiating it with $\$N = H.\text{normalCore}\$$ recovers $\$H.\text{normalCore} \leq H\$$. The order-theoretic reformulation is a substantive surface change requiring this equivalence to recognize.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2317 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_greatest_normal_subgroup_order.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  (N  $\leq$  H  $\leftrightarrow$  N  $\leq$  H.normalCore)
```

Read back by a model that saw only the Lean. *Let G be a group and H a subgroup of G . Then the normal core of H is a normal subgroup of G , and for every normal subgroup N of G , N is contained in H if and only if N is contained in the normal core of H .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$  (N  $\leq$  H  $\leftrightarrow$  N  $\leq$  H.normalCore) := by  
  refine (Subgroup.normalCore_normal H, ?_)  
  intro N hN  
  letI : N.Normal := hN  
  exact (Subgroup.normal_le_normalCore (H := H) (N := N)).symm
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms__d6f85b6a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint d6f85b6aaa5c648d

4.521 521. cyclic_subgroup_card_prime_units

The problem

Let $a > 1$, $n > 0$, and let p be prime. If p divides the number of invertible residue classes modulo $a^n - 1$, prove that these units contain a cyclic subgroup with p elements.

Lean statement

```
theorem cyclic_subgroup_card_prime_units {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hcard : p ∣ Nat.card ((ZMod (a ^ n - 1))^x)) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p
```

Parent. Let $a > 1$, $n > 0$, and let p be prime. If p divides the Euler totient of $a^n - 1$, prove that the group of units modulo $a^n - 1$ contains a cyclic subgroup of cardinality p .

Parent in Lean

```
theorem cyclic_subgroup_card_prime {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hφ : p ∣ (a ^ n - 1).totient) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child preserves exactly the same existence theorem: under the existing positivity assumptions, divisibility of the unit group's cardinality is equivalent to divisibility of the totient. Replacing the totient hypothesis with the explicit cardinality of the unit group is a genuine mathematical surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves all hypotheses and replaces divisibility by the totient with the equivalent divisibility of the unit group's cardinality; the assumptions ensure the required nonzero modulus. This is a genuine surface change requiring recognition of the cardinality–totient equivalence, not mere decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent under the preserved hypotheses: for the positive modulus ensured by $a > 1$ and $n > 0$, the cardinality of the unit group modulo $a^n - 1$ equals its Euler totient. Replacing totient divisibility with unit-group cardinality is a substantive surface re-encoding requiring the cardinality theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2312 statements
hypothesis_preservation	parser	changed: added #6# : #2# Nat.card((ZMod(#0#^#1#-1))^x)
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_card_prime_units.extracted_1_1 {a n p : ℕ} (ha : a > 1) (hn : n > 0) (hp : Nat.Prime p)
(hcard : p ∣ Nat.card (ZMod (a ^ n - 1))) : ∃ K, Nat.card K = p
```

Read back by a model that saw only the Lean. For natural numbers a , n , and p , if $a > 1$, $n > 0$, p is prime, and p divides the cardinality of the group of units of $Z\text{Mod}(a^n - 1)$, then there exists a subgroup K of that unit group whose cardinality is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cyclic_subgroup_card_prime_units {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hcard : p ∣ Nat.card ((ZMod (a ^ n - 1)))^x) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p := by
  have hpow : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha
  have hnpos : 0 < a ^ n - 1 := Nat.sub_pos_of_lt hpow
  letI : NeZero (a ^ n - 1) := ⟨Nat.ne_of_gt hnpos⟩
  have hpcard : p ∣ Fintype.card (ZMod (a ^ n - 1))^x := by
    simpa only [Nat.card_eq_fintype_card] using hcard
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨u, hu⟩ := exists_prime_orderOf_dvd_card p hpcard
  refine ⟨Subgroup.zpowers u, ?_⟩
  rw [Nat.card_zpowers, hu]
```

Operator. mutation / mutation_silent **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh.mh.ms__f86c15a2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f86c15a23843e442

4.522 522. conjugate_zpow_exists_exp

The problem

In a group where every subgroup is normal, conjugating any integer power of an element by another element produces a power of the original element. Express this by exhibiting an integer exponent for each conjugate.

Lean statement

```
theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b⁻¹ = a ^ k
```

Parent. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent to the parent because membership in ‘Subgroup.zpowers a’ is exactly the existence of an integer exponent k with the element equal to ‘ $a ^ k$ ’. It preserves all hypotheses and genuinely re-encodes membership as an explicit existential equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent via ‘ $x \in \text{Subgroup.zpowers } a \leftrightarrow \exists k : \mathbb{Z}, a ^ k = x$ ’; it preserves all hypotheses and quantifiers while replacing subgroup membership with an explicit exponent certificate. This is a substantive Lean-surface re-encoding, not decoration or an alias.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent because membership in ‘Subgroup.zpowers a’ is exactly the existence of an integer k with the element equal to ‘ $a ^ k$ ’. It preserves all hypotheses and changes the surface from subgroup membership to an explicit exponent certificate.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2291 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugate_zpow_exists_exp.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G],  
  (∀ (H : Subgroup G), H.Normal) → ∀ (a b : G) (n : ℤ), ∃ k, b * a ^ n * b-1 = a ^ k
```

Read back by a model that saw only the Lean. *For every group G in which every subgroup is normal, for all elements $a, b \in G$ and every integer n , there exists an integer k such that $b a^n b^{-1} = a^k$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k := by
  intro n
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have han : a ^ n ∈ Subgroup.zpowers a := Subgroup.zpow_mem_zpowers a n
  have hconj : b * a ^ n * b-1 ∈ Subgroup.zpowers a := by
    exact hHnormal.conj_mem (a ^ n) han b
  rw [Subgroup.mem_zpowers_iff] at hconj
  rcases hconj with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩

theorem conjugate_zpow_exists_exp_forward {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G)
  (h : ∀ n : ℤ, b * a ^ n * b-1 ∈ Subgroup.zpowers a) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k := by
  intro n
  have hn := h n
  rw [Subgroup.mem_zpowers_iff] at hn
  rcases hn with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩

theorem conjugate_zpow_exists_exp_backward {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G)
  (h : ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k) :
  ∀ n : ℤ, b * a ^ n * b-1 ∈ Subgroup.zpowers a := by
  intro n
  rw [Subgroup.mem_zpowers_iff]
  rcases h n with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.ms__76665334

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 766653349f82aaa0

4.523 523. aime_1984_p15_x_squared_value

The problem

Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Lean statement

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Parent. Suppose real numbers x, y, z, w satisfy the four weighted-square equations above. Show that $x = 105/32$ or $x = -105/32$.

Parent in Lean

```
theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x = 105 / 32 ∨ x = -(105 / 32) := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
    nlinarith [hx_sq]
  rcases mul_eq_zero.mp hfactor with hpos | hneg
  · left
    nlinarith
  · right
    nlinarith
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is exactly equivalent to the parent over $\mathbb{R}$: $x^2 = 11025/1024$ holds precisely when $x = 105/32$ or $x = -105/32$. Replacing the two-root disjunction with the squared-coordinate identity is a genuine surface re-encoding, not mere decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child derives exactly the squared-coordinate identity equivalent over $\mathbb{R}$ to the parent's two-root conclusion, under the same four hypotheses. Replacing the disjunction with the square equation is a genuine surface re-encoding, not a renaming or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The four weighted-square hypotheses force exactly $x^2 = 11025/1024$, which over $\mathbb{R}$ is equivalent to $x = 105/32 \vee x = -(105/32)$; no hypotheses or scope changed. The conclusion is a genuine algebraic re-encoding, not a renaming or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2705 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_squared_value.extracted_1_1 (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024
```

Read back by a model that saw only the Lean. *Let $x, y, z,$ and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Operator. mutation / mutation_silent **Lineage depth.** 2 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms__54ef2279

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 54ef22792ab0a8f3

4.524 524. bounded_prime_product_additive_balance_characterization

The problem

For every nonnegative integer t , characterize when t satisfies the additive balance $pq = t + (p + q)$ for two distinct primes between 4 and 18. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Lean statement

```
theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Parent. For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bounds imply $p + q \leq p * q$, so the natural-number subtraction equation $p * q - (p + q) = t$ is exactly equivalent to the additive balance $p * q = t + (p + q)$. The factorized characterization and all hypotheses are unchanged, while the surface requires reasoning about subtraction cancellation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounds imply $p + q \leq p * q$, so in $\mathbb{N}$ the additive balance $p * q = t + (p + q)$ is exactly equivalent to $p * q - (p + q) = t$, with the factorized characterization unchanged. Replacing truncated subtraction by an additive equation is a genuine mathematical re-encoding rather than decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the retained prime and bound hypotheses, $p + q \leq p * q$, so $p * q - (p + q) = t$ is exactly equivalent to $p * q = t + (p + q)$. The additive-balance formulation materially changes the surface while preserving the factorized characterization and theorem scope.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2407 statements

Check	By	Result
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_additive_balance_characterization.extracted_1_1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = t + (p + q)) ↔
  ∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ (p - 1) * (q - 1) = t + 1
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q equals t plus p plus q , if and only if there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $(p \text{ minus } 1)$ times $(q \text{ minus } 1)$ equals t plus 1.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1) := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
    have hbal_z : (p * q : ℤ) = (t : ℤ) + (p + q : ℤ) := by
      exact_mod_cast hbal
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
        Nat.cast_sub (by omega : 1 ≤ q)]
      nlinarith [hbal_z]
    have hfactor : (p - 1) * (q - 1) = t + 1 := by
      exact_mod_cast hfac_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      exact_mod_cast hfactor
    norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
      Nat.cast_sub (by omega : 1 ≤ q)] at hfac_z
    have hbal_z : (p * q : ℤ) = (t : ℤ) + (p + q : ℤ) := by
      nlinarith [hfac_z]
    have hbal : p * q = t + (p + q) := by
      exact_mod_cast hbal_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.ms__51827e7f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 51827e7f7966af1e

4.525 525. periodic_natural_multiple_equivalence

The problem

For a real-valued function and a real shift p , the law $f(x + p) = f(x)$ for every real x is equivalent to saying that every natural multiple of p is a period of f .

Lean statement

```
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :
  (∀ x : ℝ, f (x + p) = f x) ↔
  ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by
```

Parent. Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Parent in Lean

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent: the parent's period law implies that every natural multiple is a period, while the case $n = 1$ recovers the original law. Recasting the conclusion through `Function.Periodic` and an equivalence is a substantial Lean-surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent: the forward direction derives every natural multiple period, while the reverse direction recovers the original period from the case $n = 1$. Recasting the conclusion through `Function.Periodic` and an iff is a genuine surface re-encoding.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is logically equivalent to the parent: its forward direction is the natural-multiple period law, while the converse recovers the original period by taking $n = 1$. It substantially changes the Lean surface by expressing the conclusion through `Function.Periodic` and an iff.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2495 statements
hypothesis_preservation	parser	changed: dropped #2# : ∀#4# : ℝ, #0#(#4# + #1#) = #0##4#
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem periodic_natural_multiple_equivalence.extracted_1_1 (f : ℝ → ℝ) (p : ℝ) :  
  (∀ (x : ℝ), f (x + p) = f x) ↔ ∀ (n : ℕ), Function.Periodic f (↑n * p)
```

Read back by a model that saw only the Lean. For a function f from the real numbers to the real numbers and a real number p , the following are equivalent: for every real number x , $f(x + p) = f(x)$; and for every natural number n , the real number n multiplied by p is a period of f , meaning that for every real number x , $f(x + n p) = f(x)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :  
  (∀ x : ℝ, f (x + p) = f x) ↔  
    ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by  
  constructor  
  · intro h n  
    have hp : Function.Periodic f p := by  
      intro x  
      exact h x  
    exact hp.nat_mul n  
  · intro h x  
    simpa using (h 1 x)
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.ms__076225ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 076225efb862a51a

4.526 526. mathd_algebra_101_completed_square

The problem

For a real number x satisfying $x^2 - 5x - 4 \leq 10$, show that $x \in [-2, 7]$ if and only if $\left|x - \frac{5}{2}\right| \leq \frac{9}{2}$.

Lean statement

```
theorem mathd_algebra_101_completed_square (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 ↔ |x - (5 : ℝ) / 2| ≤ (9 : ℝ) / 2
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child preserves the hypothesis and restates the interval conclusion via the equivalent completed-square bound, explicitly proving both directions. The absolute-value formulation is a substantive surface re-encoding rather than a rename, unfolding, or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the same hypothesis, the completed-square inequality $|x - 5/2| \leq 9/2$ is exactly equivalent to $x \in [-2, 7]$, so the child preserves the parent's mathematics while changing the surface substantially.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves the same hypothesis and interval conclusion while expressing it equivalently as the completed-square bound $|x - 5/2| \leq 9/2$; the added biconditional is mathematically valid and does not change the theorem's scope. This is a genuine surface re-encoding, not a rename or definitional decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2429 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_101_completed_square.extracted_1_1 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :  
  x ∈ Set.Icc (-2) 7 ↔ |x - 5 / 2| ≤ 9 / 2
```

Read back by a model that saw only the Lean. For a real number x satisfying $x^2 - 5x - 4 \leq 10$, x lies in the closed interval $[-2, 7]$ if and only if the absolute value of $x - 5/2$ is at most $9/2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_101_completed_square (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 ↔ |x - (5 : ℝ) / 2| ≤ (9 : ℝ) / 2
↔ := by
  have h_interval : x ∈ Set.Icc (-2) 7 := by
    constructor
    · by_contra hx
      have hx' : x < -2 := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra hx
      have hx' : 7 < x := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  constructor
  · intro _
    have hx := h_interval
    rw [abs_le]
    constructor <|> linarith [hx.1, hx.2]
  · intro h_abs
    rw [abs_le] at h_abs
    exact ⟨by linarith [h_abs.1], by linarith [h_abs.2]⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__ms__34850c49

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 34850c495074d6bf

4.527 527. quadratic_discriminant_max

The problem

For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Lean statement

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

Parent. For a real number a with $0 < a \leq 10$, what is the largest number c such that $ax^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_real_root_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | ∃ x : ℝ, a * x ^ 2 + 5 * x + c = 0}) : IsGreatest S (25 / (4 * a))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The discriminant condition $0 \leq 25 - 4ac$ characterizes exactly those c for which the quadratic has a real root when $a > 0$, so the child proves the same greatest-element theorem over the same feasible set. Replacing the existential-root definition with the discriminant normal form is a substantive surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child's condition $0 \leq 25 - 4ac$ exactly characterizes the same feasible constants as the parent's existential real-root condition for $a > 0$, and it preserves the same greatest element. Recasting root feasibility via the discriminant is a substantive surface change requiring mathematical recognition, not mere renaming or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The discriminant condition $0 \leq 25 - 4ac$ is equivalent, under $a > 0$, to the existence of a real root of $ax^2 + 5x + c = 0$, so the same greatest-set theorem is preserved. Replacing the existential quadratic description with its discriminant normal form is a materially different surface.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2591 statements
hypothesis_preservation	parser	changed: dropped #3# : #2#={c ∃#4#:ℝ,#0#*#4#^2+5*#4#+c=0}
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_discriminant_max.extracted_1_1 (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | 0 ≤ 25 - 4 * a * c}) :  
  IsGreatest S (25 / (4 * a))
```

Read back by a model that saw only the Lean. *Let a be a real number with $0 < a$, and let S be a subset of the real numbers such that $S = \{c \in \mathbb{R} \mid 0 \leq 25 - 4ac\}$. Then $25/(4a)$ is the greatest element of S .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))  
  <:= by  
    rw [hS]  
    constructor  
    · change 0 ≤ 25 - 4 * a * (25 / (4 * a))  
      have hdisc : 0 ≤ 25 - 4 * a * (25 / (4 * a)) := by  
        field_simp [ha.ne'] <|> norm_num  
      exact hdisc  
    · intro c hc  
      change 0 ≤ 25 - 4 * a * c at hc  
      have hpos : 0 < 4 * a := by  
        nlinarith [ha]  
      apply (le_div_iff hpos).2  
      nlinarith [hc]
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms__e0abe93f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e0abe93f073ab7ec

4.528 528. mathd_algebra_313_affine

The problem

For complex numbers v , i , and z satisfying $v = i z$, $v = 1 + I$, and $z = 2 - I$, one has $5i = 1 + 3I$.

Lean statement

```
theorem mathd_algebra_313_affine (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I) (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 *
  ↪ Complex.I := by
```

Parent. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
  (h₂ : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent because 5 is nonzero in $\mathbb{C}$: multiplying the quotient equality by 5 yields $5 * i = 1 + 3 * \text{Complex.I}$, and conversely division by 5 recovers the parent conclusion. The denominator-cleared affine form is a genuine surface re-encoding, not mere decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent because multiplying the complex equality by the nonzero scalar 5 yields and recovers the denominator-cleared form. All hypotheses and the mathematical scope are preserved, while the conclusion is genuinely re-encoded.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2036 statements
hypothesis_preservation	parser	not run
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_313_affine.extracted_1_1 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
  (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 * Complex.I
```

Read back by a model that saw only the Lean. Let v , i , and z be complex numbers. Suppose that $v = i \cdot z$, $v = 1 + I$, and $z = 2 - I$, where I is the imaginary unit. Then $5 \cdot i = 1 + 3I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_313_affine (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I) (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 *
  Complex.I := by
  have hmain : i = 1 / 5 + 3 / 5 * Complex.I := by
    rw [h₁, h₂] at h₀
    have hr := congrArg Complex.re h₀
    have hm := congrArg Complex.im h₀
    apply Complex.ext
    · norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
      linarith
    · norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
      linarith
  calc
    5 * i = 5 * (1 / 5 + 3 / 5 * Complex.I) := by rw [hmain]
    _ = 1 + 3 * Complex.I := by ring
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_313__ms__6b23d2d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6b23d2d503df6c14

4.529 529. completed_square_minimum

The problem

What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Lean statement

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The completed-square hypothesis defines exactly the same function, since $(x-3)^2+4=x^2-6x+13$, and the `IsLeast` conclusion is unchanged. This is a genuine algebraic re-encoding that requires recognizing the completed-square identity, not mere renaming or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The completed-square form $(x-3)^2+4$ is definitionally equal to $x^2-6x+13$, so the hypothesis and `IsLeast` conclusion preserve exactly the parent theorem. The surface representation genuinely changes from expanded polynomial to completed square.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The completed-square expression $(x-3)^2+4$ is algebraically identical to $x^2-6x+13$, so the hypothesis and `IsLeast` conclusion describe exactly the same theorem. The surface is materially re-encoded through parameter shifting rather than merely renamed or decorated.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2676 statements
hypothesis_preservation	parser	changed: dropped #1# : #0# = fun #2# ↦ #2# ^ 2 - 6 * #2# + 13
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem completed_square_minimum.extracted_1_1 (f : ℝ → ℝ) (h : f = fun x => (x - 3) ^ 2 + 4) :  
  IsLeast (Set.range f) 4
```

Read back by a model that saw only the Lean. *Let f be a function from the real numbers to the real numbers, and suppose that for every real number x , $f(x) = (x - 3)^2 + 4$. Then 4 is the least element of the range of f .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem completed_square_minimum  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - 3) ^ 2 + 4) :  
  IsLeast (Set.range f) 4 := by  
  subst f  
  constructor  
  · use 3  
    norm_num  
  · rintro y ⟨x, rfl⟩  
    nlinarith [sq_nonneg (x - 3)]
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms__aaa74262

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aaa74262625fe908

4.530 530. mathd_numbertheory_202_congruence

The problem

Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Lean statement

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child asserts exactly the same fact: the sum has remainder 8 modulo 10, expressed as congruence rather than remainder equality. The modular-congruence surface is materially different and preserves all mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The modular congruence ' $19^{19} + 99^{99} \equiv 8 \text{ [MOD 10]}$ ' is equivalent to the parent remainder equation because 8 is already below 10. Congruence notation is a genuine mathematical re-encoding, not a rename or definitional decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child asserts exactly the same mathematical fact: the sum is congruent to 8 modulo 10, which is equivalent to having remainder 8 modulo 10. Replacing remainder equality with modular congruence is a genuine Lean-surface re-encoding.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2717 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_202_congruence.extracted_1_1 : 19 ^ 19 + 99 ^ 99 ≡ 8 [MOD 10]
```

Read back by a model that saw only the Lean. The sum of 19 raised to the 19th power and 99 raised to the 99th power is congruent to 8 modulo 10.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
  norm_num [Nat.ModEq]
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms__48ce67d7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 48ce67d79da3c99a

4.531 531. mathd_numbertheory_229_div

The problem

Show that the difference between 5^{30} and 1 is divisible by 7.

Lean statement

```
theorem mathd_numbertheory_229_div : 7 ∣ (5 ^ 30 - 1 : ℕ) := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent to the parent: integer congruence $5^{30} \equiv 1 \pmod{7}$ is exactly divisibility of the natural-number difference $7 \mid 5^{30} - 1$. The divisibility formulation is a substantive surface re-encoding, not mere decoration or renaming.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent to the parent: for these positive natural numbers, $7 \mid 5^{30} - 1$ exactly expresses $5^{30} \equiv 1$ modulo 7. Recasting modular congruence as divisibility is a substantive surface change that preserves the theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2411 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_229_div.extracted_1_1 : 7 ∣ 5 ^ 30 - 1
```

Read back by a model that saw only the Lean. 7 divides 5 raised to the 30th power minus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
```



```
open BigOperators Real Nat Topology Rat
```

```
theorem mathd_numbertheory_229_div : 7 ∣ (5 ^ 30 - 1 : ℕ) := by  
  have hpow : (5 ^ 30 : ℕ) = 931322574615478515625 := by  
    norm_num  
  rw [hpow]  
  norm_num
```

Operator. mutation/mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__ms__605bc864

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, fingerprint 605bc86482a76709

4.532 532. variable_base_residue_divisibility

The problem

Two sensors report integers a and b , with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that 7 divides $a^{13} - b^{13} - 3$.

Lean statement

```
theorem variable_base_residue_divisibility (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ 13 - b ^ 13 - 3) := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The divisibility conclusion is exactly equivalent to the parent congruence: $a^{13} - b^{13} \equiv 3 \pmod{7}$ iff 7 divides $a^{13} - b^{13} - 3$, with all hypotheses and scope preserved. This is a genuine conclusion re-encoding requiring conversion between modular congruence and divisibility, not mere notation or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent: congruence modulo 7 is restated as divisibility of the difference by 7, with all hypotheses and the conclusion's scope preserved. This is a substantive surface change requiring recognition of the equivalence, not mere renaming or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The divisibility conclusion is exactly equivalent to the parent congruence: $7 \mid (a^{13} - b^{13} - 3)$ iff $a^{13} - b^{13} \equiv 3 \pmod{7}$. The hypotheses and mathematical scope are unchanged, while the surface uses a genuinely different conclusion form.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2418 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem variable_base_residue_divisibility.extracted_1_1 (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :
  7 ∣ a ^ 13 - b ^ 13 - 3
```

Read back by a model that saw only the Lean. For integers a and b , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then 7 divides a raised to the 13th power minus b raised to the 13th power minus 3.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem variable_base_residue_divisibility (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ 13 - b ^ 13 - 3) := by
  have ha13 : a ^ 13 ≡ (29 : ℤ) ^ 13 [ZMOD 7] := ha.pow 13
  have hb13 : b ^ 13 ≡ (5 : ℤ) ^ 13 [ZMOD 7] := hb.pow 13
  have hdiff : a ^ 13 - b ^ 13 ≡ (29 : ℤ) ^ 13 - (5 : ℤ) ^ 13 [ZMOD 7] := ha13.sub hb13
  have hparent : (29 : ℤ) ^ 13 - (5 : ℤ) ^ 13 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hres : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := hdiff.trans hparent
  rw [Int.modEq_iff_dvd] at hres
  rcases hres with ⟨k, hk⟩
  refine ⟨-k, ?_⟩
  omega
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.ms__57316c5a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 57316c5ad6bdfae7

4.533 533. ternary_terminal_digit_exclusion

The problem

A natural number has base-three digits $122d_3$. Show that either $d \neq 2$ or the number equals 53.

Lean statement

```
theorem ternary_terminal_digit_exclusion
  (d x : ℕ)
  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  d ≠ 2 ∨ x = 53
```

Parent. Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Parent in Lean

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent under the digit hypothesis: the parent's conditional is exactly expressed as $d \neq 2 \vee x = 53$, while $x = d + 51$ is already forced by the shared hypothesis. The disjunctive exclusion form is a genuine surface re-encoding, not decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the shared digit hypothesis, $x = d + 51$ is forced, so the parent's substantive conditional is equivalent to $d \neq 2 \vee x = 53$. The child materially re-encodes the implication as an exclusion disjunction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the shared digit hypothesis, $x = d + 51$ is derivable, so the parent's conjunction is equivalent to $d \neq 2 \vee x = 53$; the child presents this as a genuinely different exclusion-form surface.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2577 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_terminal_digit_exclusion.extracted_1_1 (d x : ℕ) (h : Nat.digits 3 x = [d, 2, 2, 1]) :  
  d ≠ 2 ∨ x = 53
```

Read back by a model that saw only the Lean. For natural numbers d and x , if the base-3 digits of x , listed in `Nat.digits` order, are exactly $[d, 2, 2, 1]$, then d is not equal to 2 or x equals 53.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem ternary_terminal_digit_exclusion  
  (d x : ℕ)  
  (h : Nat.digits 3 x = [d, 2, 2, 1]) :  
  d ≠ 2 ∨ x = 53 := by  
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=  
    (Nat.ofDigits_digits 3 x).symm  
  have hvalue : x = d + 51 := by  
    calc  
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode  
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]  
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]  
  by_cases hd : d = 2  
  · right  
    rw [hvalue, hd]  
  · exact Or.inl hd
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me.ms__3d7174de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 3d7174de1c8ee629

4.534 534. complementary_pascal

The problem

For positive integers n and k with $k \leq n$, show that $\binom{n}{n-k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Lean statement

```
theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

Parent. Show that for positive integers n and k with $k \leq n$, we have $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem numbertheory_nckeqnm1ckpnm1ckm1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

Under $k \leq n$, binomial symmetry gives $\binom{n}{n-k} = \binom{n}{k}$, so the child is mathematically equivalent to the parent with the same hypotheses. The complementary-index formulation is a genuine surface change, and the iff restatement reinforces the equivalent bounded-index form.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the complementary-index identity $\binom{n}{n-k} = \binom{n}{k}$ under the preserved hypothesis $k \leq n$, so it has exactly the parent's scope and conclusion after a genuine re-encoding of the left-hand side.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves all hypotheses and uses choose symmetry under $k \leq n$ to replace $\text{Nat.choose } n \ k$ with $\text{Nat.choose } n \ (n - k)$, leaving the Pascal identity otherwise unchanged. This is a genuine complementary-index re-encoding that requires recognizing the equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2623 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_pascal.extracted_1_1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  n.choose (n - k) = (n - 1).choose k + (n - 1).choose (k - 1)
```

Read back by a model that saw only the Lean. For natural numbers n and k , if $0 < n$ and $0 < k$, and $k \leq n$, then the binomial coefficient " n choose $n - k$ " equals the sum of " $n - 1$ choose k " and " $n - 1$ choose $k - 1$ ".

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
    Nat.choose_symm h₁
  have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
    simpa [Nat.add_comm] using (Nat.choose_eq_choose_pred_add h₀.1 h₀.2)
  exact hsymm.trans hpascal

theorem complementary_pascal_iff (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  (Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)) ↔
  (Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)) := by
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
    Nat.choose_symm h₁
  constructor
  · intro h
    exact hsymm.trans h
  · intro h
    exact hsymm.symm.trans h
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms__b312e50e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b312e50ee99a4d20

4.535 535. square_dvd_three_iff_dvd

The problem

For an integer a , show that 3 divides a^2 exactly when 3 divides a .

Lean statement

```
theorem square_dvd_three_iff_dvd (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a
```

Parent. For an integer a , its square is congruent to 0 modulo 3 exactly when a is divisible by 3.

Parent in Lean

```
theorem square_mod_three_zero_iff_dvd (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent to the parent: for integers, $a^2 \equiv 0 \pmod 3$ exactly means $3 \mid a^2$, so both directions have the same scope. Replacing modular congruence with divisibility is a genuine but shallow surface re-encoding, not mere decoration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent for every integer: divisibility by 3 is exactly congruence to 0 modulo 3, applied to a^2 . Replacing the modular-equivalence relation with divisibility materially changes the surface while preserving the theorem.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent over integers: $a^2 \equiv 0 \pmod 3$ exactly when $3 \mid a^2$, so it has the same hypotheses, scope, and conclusion. Replacing modular congruence with divisibility is a genuine surface re-encoding requiring the ModEq-divisibility bridge.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2713 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_dvd_three_iff_dvd.extracted_1_1 (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a
```

Read back by a model that saw only the Lean. For every integer a , 3 divides a squared if and only if 3 divides a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_dvd_three_iff_dvd (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a := by
  constructor
  · intro hsq
    have hsqmod : a ^ 2 ≡ 0 [ZMOD 3] := Dvd.dvd.modEq_zero_int hsq
    have hres : a % 3 = 0 ∨ a % 3 = 1 ∨ a % 3 = 2 := by
      omega
    rcases hres with hres | hres | hres
    · refine ⟨a / 3, ?_⟩
      omega
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsqmod
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsqmod
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsqmod
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsqmod
  · intro hdiv
    rcases hdiv with ⟨k, hk⟩
    refine ⟨3 * k ^ 2, ?_⟩
    rw [hk]
    ring
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__mh.ms__ca1f8653

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ca1f86533f25425c